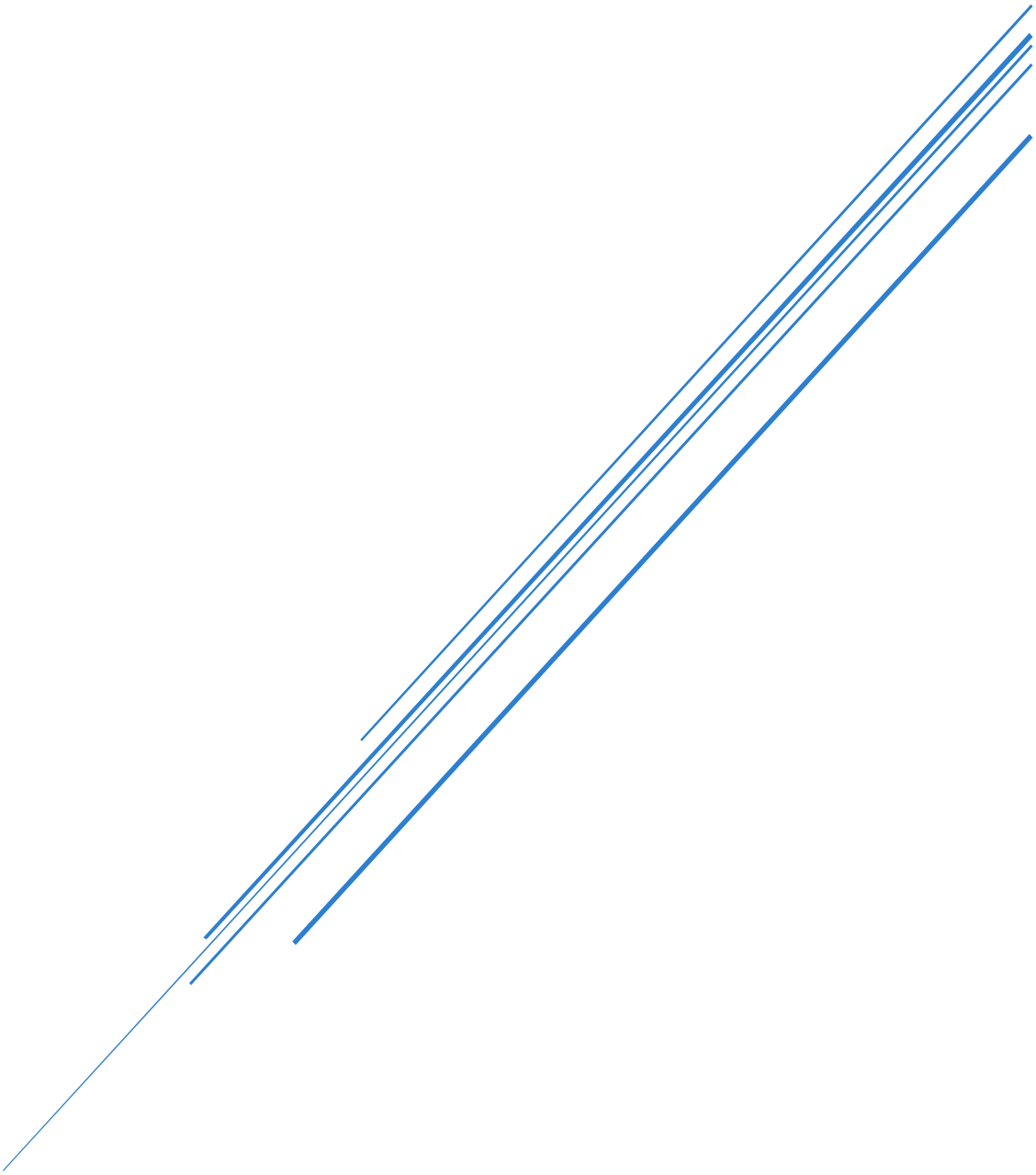
REVISION ASSISTANT

Aditya Sainath (9187)



Dormers Wells High School (12414)
Computer Science Project

Contents

Section 1 Analysis 11
1.1 Description of the Problem 11
1.1.1 Problem Identification 11
1.1.2 Computational methods 12
1.2 Stakeholders 14
1.3 Existing solutions 15
1.3.1 Quizlet 15
1.3.1.1 Quizlet's Home Page 16
1.3.1.2 Quizlet's Library 17
1.3.1.3 Quizlet's Create Folder 18
1.3.1.4 Quizlet's Flashcards 18
1.3.1.5 Quizlet's Practice Tests 19
1.3.1.6 Quizlet's Experts Solutions 19
1.3.1.7 Quizlet's other features 20
1.3.2 Seneca 20
1.3.2.1 Seneca's Home Page 20
1.3.2.1.1 Seneca's attempt at gamification 21
1.3.2.2 Seneca's Course Page 22
1.3.2.3 Seneca's actual revision 23
1.3.2.3.1 Pre-Study Knowledge Overview 23
1.3.2.3.2 Study Options 23
1.3.2.4 Seneca's Navigation Bar 25
1.3.2.5 Seneca's Other Features 26
1.3.3 Summary 26
1.3.4 Possible Approaches to the Solution 29
1.3.4.1 Justification of Language Choice 30
1.3.5 Essential Features of the Proposed Solution 30
1.3.6 Limitations of the Proposed Solution 32
1.4 Requirement Specification 34
1.4.1 Developer's Hardware/Software Specification 34
1.4.2 User's Hardware/Software Specification 35

1.4.3 Success Criteria 36
Section 2 Design 40
2.1 Decomposition 40
2.1.1 Stage 1 32
2.1.2 Stage 2 32
2.1.3 Stage 3 33
2.1.4 Stage 4 34
2.1.5 Stage 5 35
2.1.6 Stage 6 36
2.1.7 Stage 7 37
2.2 Interface 38
2.2.1 Interface Flow Diagram 38
2.2.2 Interface Mock-Up 40
2.2.2.1 Splash Screen 40
2.2.2.2 Registration / Login Screen 41
2.2.2.3 Main Menu 42
2.2.2.4 Revision System 43
2.2.2.5 System Settings 44
2.2.2.6 Flashcard Management 45
2.2.2.7 Study Modes 46
2.2.2.8 End of Revision 47
2.2.2.9 Summary of Usability Features consistent throughout Screens 48
2.3 System Design 50
2.3.1 Class Diagram 50
2.3.1.2 Stage 1 53
2.3.1.2 Stage 2 54
2.3.1.3 Stage 3 55
2.3.1.4 Stage 4 56
2.3.2 Algorithms 57
2.3.2.1 User Authentication 57
2.3.2.2 Exporting Flashcards as PDF/CSV 60
2.3.2.3 Importing Flashcards as CSV 63

2.3.2.4 Shuffling *66*
2.3.2.5 Flashcard Searching *68*
2.3.3 Data Dictionary 69
2.4 Approach to Testing 73
2.4.1 Testing Strategy 73
2.4.2 Test Plan 75
2.4.3 Post-Development Testing 80
Section 3 Development 86
3.1 Prototype 1: Initial User Interface 86
3.1.1 Purpose of Prototype 1 86
3.1.2 Success Criteria for Prototype 1 86
3.1.2.1 Splash Screen Criteria *86*
3.1.2.2 Login Criteria *87*
3.1.2.3 Registration Criteria *87*
3.1.2.4 Data Storage Criteria *87*
3.1.3 Developing Prototype 1 87
3.1.3.1 Separation of Project Components *87*
3.1.3.2 Splash Screen *89*
3.1.3.3 User Registration *91*
3.1.3.3.1 Remedial Action (File path) 95
3.1.3.3.2 Testing User Registration 97
3.1.3.3.3 Remedial action (Email validation) 101
3.1.3.4 User Login *102*
3.1.3.4.1 Remedial Action (Setter Method) 105
3.1.3.4.2 Testing User Login 106
3.1.3.4.3 Remedial Action (Adding validator to the RecoveryWord Input) 110
3.1.3.5 Data Storage Requirements *111*
3.1.3.5.1 Testing Stored Data (Destruction Testing) 111
3.1.3.5.2 Remedial Action (Handling Corrupted Files) 114
3.1.4 Evaluation of Prototype 1 116
3.2 Prototype 2: Main Menu and Settings Screen 116
3.2.1 Improvements from Prototype 1 116

3.2.2 Purpose of Prototype 2 117
3.2.3 Success Criteria for Prototype 2 117
3.2.3.1 Theme Criteria *117*
3.2.3.2 Audio Control Criteria *117*
3.2.4 Developing Prototype 2 118
3.2.4.1 Post Login and Main Menu Screen *118*
3.2.4.2 Theme Switching *121*
3.2.4.2.1 Initial Remedial Action 123
3.2.4.2.2 Testing theme switching 127
3.2.4.3 Brightness *129*
3.2.4.3.1 Remedial Action for Slider 131
3.2.4.3.2 Testing brightness adjustments 132
3.2.4.4 Audio Control *134*
3.2.4.4.1 Testing Audio 135
3.2.4.4.2 Remedial Action 139
3.2.4.5 Other Options *140*
3.2.4.5.1 Testing Other Options 144
3.2.5 Evaluation of Prototype 2 149
3.3 Prototype 3: Flashcard Control 150
3.3.1 Improvements from Prototype 2 150
3.3.2 Purpose of Prototype 3 151
3.3.3 Success Criteria for Prototype 3 151
3.3.3.1 Flashcard Deck Creation and Modification Criteria *152*
3.3.3.2 Flashcard Folder Organisation Criteria *152*
3.3.3.3 Import / Export Criteria *152*
3.3.4 Developing Prototype 3 152
3.3.4.1 Revision System and Flashcard Management Screen *152*
3.3.4.2 Flashcard Creation *155*
3.3.4.2.1 Adding flashcards to a deck 155
3.3.4.2.2 Saving Flashcards 160
3.3.4.2.3 Testing Flashcard Creation 164
3.3.4.2.4 Remedial Action 168

3.3.4.3 Post-Creation Flashcard (Deck) Modification *170*
3.3.4.3.1 Initial Modification Page 170
3.3.4.3.2 Populating the decksTable table 171
3.3.4.3.3 Populating the cardPreview Table 173
3.3.4.3.4 Searching Tables 175
3.3.4.3.5 Actions within the decksTable 177
3.3.4.5.6 Testing Flashcard Modification 181
3.3.4.5.7 Remedial Action 186
3.3.4.4 Organising Flashcard Decks *188*
3.3.4.4.1 Building the tree 188
3.3.4.4.2 Creating and Destroying Directories 192
3.3.4.4.3 cardPreview Table 194
3.3.4.4.4 Testing Organisation 195
3.3.4.5 Import / Export *198*
3.3.4.5.1 Import as CSV 198
3.3.4.5.2 Export as CSV 199
3.3.4.5.3 Export as PDF 199
3.3.4.5.4 Import / Export Screen 201
3.3.4.5.5 Testing Import / Export 203
3.3.5 Evaluation of Prototype 3 205
3.4 Prototype 4: Study Process and Analytics 206
3.4.1 Improvements from Prototype 3 206
3.4.2 Purpose of Prototype 4 210
3.4.3 Success Criteria for Prototype 4 211
3.4.3.1 XP and Achievement System Criteria *211*
3.4.3.2 Session Time-Tracking Criteria *211*
3.4.3.3 Text-to-Speech Criteria *211*
3.4.3.4 Analytics Criteria *211*
3.4.4 Developing Prototype 4 212
3.4.4.1 Study Modes *212*
3.4.4.1.1 Sound Effects 218
3.4.4.1.2 Flashcard Buttons 219

3.4.4.1.3 End of Revision Screen 221
3.4.4.1.4 Text-To-Speech 224
3.4.4.1.5 Testing Study Modes 228
3.4.4.1.6 Remedial Actions 234
3.4.4.2 Statistics, Achievements and XP 235
3.4.4.2.1 Statistics 235
3.4.4.2.2 XP and Achievements 238
3.4.4.2.3 Displaying Stats and Achievements to Users 245
3.4.4.2.3.3 Testing Statistics, Achievement and XP 251
3.4.4.3 Analytics 256
3.4.4.3.1 End of Revision Screen Graphs 256
3.4.4.3.2 Analytics Page 264
3.4.4.3.3 Testing Analytics 272
3.4.5 Evaluation of Prototype 4 275
3.5 Prototype 5: Security, Stability and System Recovery 276
3.5.1 Purpose of Prototype 5 276
3.5.2 Success Criteria for Prototype 5 276
3.5.3 Developing Prototype 5 277
3.5.3.1 File Backup 277
3.5.3.1.1 Testing File Backup 279
3.5.3.1.2 Remedial Actions 281
3.5.3.2 Encrypting Users.csv 282
3.5.3.2.1 Testing Encryption 285
3.5.3.2.2 Remedial Actions 290
3.5.4 Evaluation of Prototype 5 293
Section 4 Evaluation 293
4.1 Destruction Testing 293
4.1.1 Importing CSVs 293
4.1.1.1 Corrupting Flashcard CSVs 294
4.1.1.1.1 Numerically Corrupting CSVs 294
4.1.1.1.2 Structurally Corrupting CSVs 296
4.1.1.1.3 Stress Testing Flashcard Import 297

4.1.1.2 Corrupting System CSVs *300*
4.1.1.2.1 Structurally Corrupting CSVs 300
4.1.1.2.2 Empty CSV Import 301
4.1.2 Spam Clicking 302
4.1.2.1 Navigating with an Auto Clicker *302*
4.1.2.2 Auto Clicking after Login *303*
4.1.2.3 Auto Clicking after Logout *303*
4.1.2.4 Inputting Invalid Data and Spam Clicking *304*
4.1.2.5 Spam Clicking when Saving Flashcards *305*
4.2 Beta Testing 306
4.2.1 Stakeholders / Beta Testers 306
4.2.1.1 Stakeholder 1 Kavusihan Mohanakumar *306*
4.2.1.2 Stakeholder 2 Wasay Amir *306*
4.2.1.3 Stakeholder 3 Mohammed El-Khafaji *307*
4.2.1.4 Stakeholder 4 Shamsudiin Mohamed *307*
4.2.1.5 Stakeholder 5 Daanyar Agha *308*
4.2.2 Summary of Beta Tests 308
4.2.3 Implementing Suggested Changes 309
4.3.3.1 Text-to-Speech Rate *310*
4.3.3.2 Exporting as HTML Design *312*
4.3 Usability Testing 313
4.4 User Acceptance Testing 319
4.5 Success of the Solution 324
4.5.1 Review against Success Criteria 324
4.5.2 Criteria Consideration in Future Development 326
4.5.3 Review against Usability Features 328
4.5.4 Usability Features for Future Development 331
4.5.5 Limitations of the Solution 332
4.5.6 Maintainability Issues 333
4.5.7 Maintenance of the Solution 334
Section 5 Final Code 337
5.1 Main.java 337

5.2 Screens 337
5.2.1 SplashScreen.java 337
5.2.2 AuthScreen.java 339
5.2.3 SystemScreens.java 346
5.2.4 MainMenuScreen.java 349
5.2.5 RevisionSystemScreen.java 353
5.2.6 SettingsScreen.java 358
5.2.7 FlashcardManagementScreen.java 368
5.2.8 StudyModesScreen.java 377
5.2.9 EndOfRevisionScreen.java 383
5.3 Controllers 387
5.3.1 Controller.java 387
5.3.2 AuthenticationController.java 389
5.3.3 SettingsController.java 393
5.3.4 FlashcardController.java 395
5.3.5 SessionController.java 401
5.3.6 TTSController.java 403
5.3.7 StatsController.java 404
5.3.8 XPController.java 407
5.3.9 AchievementController.java 407
5.4 Models 408
5.4.1 User.java 408
5.4.2 Settings.java 409
5.4.3 Stack.java 410
5.4.4 Flashcard.java 411
5.4.5 Session.java 412
5.4.6 UsersStats.java 413
5.5 Components 415
5.5.1 UIComponents.java 415
5.5.2 CustomAlert.java 419
5.5.3 ContentNavigator.java 420
5.5.4 ScreenManager.java 420

5.6 RecordClasses 422
5.6.1 TextBundle.java 422
5.6.2 PasswordBundle.java 423
5.6.3 RectangleButtonBundle.java 423
5.6.4 TextAreaBundle.java 423
5.6.5 SessionRecord.java 423
5.7 Stylesheets 424
5.7.1 styles.css 424
5.7.2 dark.css 428
5.7.3 highContrast.css 429
5.7.4 light.css 429
5.7.5 alert.css 430

Section 1 Analysis

1.1 Description of the Problem

This report documents the development of a program for revision flashcards and quizzes. which allows users to create flashcards, allocate them into folders, be able to shuffle them and be quizzed on flashcards made by the user. The flashcard program aims to enhance revision and learning by adding an element of competitivity through timed challenges, scoring systems, progress charts, and objectives. This program is designed for students cannot pay for premium versions of alternative flashcard software, aiming to help rise grades.

1.1.1 Problem Identification

A significant percentage of students aged 11-18 studying in secondary school working towards either their GCSE or A Levels, both male and female, struggle with categorising study material effectively, retaining information due to: the lack of passive recalling, the increase in procrastination, and insufficient motivation. The Revision Assistant program aims to confront these problems by increasing attentiveness of students, whilst simultaneously elevating the students' motivation to revise. The Revision Assistant is specifically designed as a desktop application exclusively to be used by students on laptops and/or PCs and is customised to assist students in achieving top grades in their approaching public examinations including GCSEs or A-Levels. Whilst flashcards have been scientifically proven to aid students in learning information such as key definitions, dates, cause-and-effect relationships; many students utilise flashcards ineffectively because of the deficiency of formal teaching of revision methods in secondary school education. The Revision Assistant is built to bridge the gap by providing a platform for creating, organising, and revising flashcards by using evidence-based techniques such as the Leitner System.

The Revision Assistant will allow the user to do the following:

- Create flashcards and organise them into numerous folders (i.e. flashcards about Physics can be stored in the Physics folder which will be stored in the Science folder)– this keeps revision materials in order thus reducing the overwhelming feeling of revision being too chaotic to start and making it easier to focus on actual revision rather than wasting time searching for content.
- Importing flashcards from (e.g. CSV files) – saving time of students as it allows them to upload pre-made flashcard sets from teachers or friends, reducing the amount of manual creation of flashcards, enabling collaborative learning.
- Save and export flashcards as a CSV/PDF – this increases the portability as well as accessibility of revision, students may want to study on their cellular devices or create printouts to study offline, exportation allows students to view them on

any device without the need of the application, also supporting students' different learning preferences (for example paper-based revision).

- Flashcards will be available to shuffle (so they can be presented in reverse-order or completely randomised order) – this blocks student from memorising the sequence instead of the actual content encouraging active recall rather than promoting passive reviewing (when you learn information by rereading continuously and not engaging with it.
- Track revision using progress bars as well as charts – this gives students a vivid depiction of the amount they have revised and their improvement which followed, students can use this to reminisce where they started from and how far they have come. Visualising progression would provide motivation, and utilising tracking options would allow students to identify areas for improvement.
- Complete challenges or quests (via daily goals, streaks, topic-specific tasks) – the revision process is now gamified, helping students with low attention spans (as well as any other student) stay motivated and engaged with studying by rewarding their consistency and effort with in-game achievements. In addition, quests can create regular revision habits, supplying students with a sense of advancement in their learning, which just learning flashcards may not do.

By using this application students will be able to improve their overall grade, also reducing exam-related stress, boosting confidence, and increasing chances of advancement to their desired sixth forms, universities, or apprenticeships, in concert.

1.1.2 Computational methods

There are methods of computational methods implemented when creating a flashcard and quiz program. This includes:

- **Decomposition –** The breaking down of a large complex problem into smaller simpler sub-problems. These sub-problems consist of more manageable tasks since they are easier to solve/understand, making them easier to code. In the future, these components of the program are put together to create the whole solution. I can apply decomposition my project by breaking down the flashcard app into smaller, easier to solve tasks. I can do this because different parts of my project are self-contained. For example:
    1. Saving flashcards.
    2. Importing CSV files and converted into flashcards.
    3. Exporting flashcards as PDF.
- **Abstraction** – The removal process of unnecessary detail during the development stage, allowing the developer to focus on the main components of the program, saving coding time and computer resources. For instance, in real life flash cards have lines to help the user write neatly, these will be removed from my app in order

to reduce the memory footprint and processing time whilst also saving development time.

- **Heuristics –** Making use of experience to solve problems quickly; using concepts like “rules of thumbs” and educated guessing to find solutions to problems, prioritising speed rather than accuracy. I can apply heuristics to my application with:
    1. Flashcard Review Scheduling – Flashcards that the student got wrong recently will be prioritised.
    2. Progress recommendations – recommending topics with lower progression

  Heuristics are easy to implement since they require less information and computation compared to complex algorithms, they also save time and allow you to find the solution close to the best available.

- **Divide and Conquer –** Strategy to make a complex task easier by breaking it into smaller, more manageable tasks, but with the same method every single time. I can apply divide and conquer to my project when the user wants to sort or search flashcards. I can use this because sorting and searching will continuously make use of the same method. For example, merge sort to sort the flashcards in ascending or descending order.
- **Backtracking –** technique used where program attempts to solve a problem with different methods, if this method does not work it goes back and tries another option. Since backtracking comprehensively explores all possible paths to a desired solution, I can apply backtracking in my program to correct the progression graphs which may be incorrect.
- **Performance Modelling –** eliminates the need for true performance testing by providing mathematical methods to test a variety of loads on different operating systems. This method supplies a less expensive, less time-consuming and safer method of testing applications. Since the students can upload CSV files for flashcards, I can apply Performance Modelling to my application by uploading a large CSV file and see how well it copes with converting to flashcards. Performance modelling will help identify bottlenecks and improve user experience.
- **Pipelining –** arranging a process in a linear fashion, where the output of one stage is the input of the next stage. Since some sub-problems within my program must be solved sequentially, I can apply pipelining to the Revision Assistant by:
    1. **Stage 1 –** Flashcard data is fetched from the imported CSV file.
    2. **Stage 2 –** CSV file's data is parsed into flashcard's objects (making use of OOP as stated above).
    3. **Stage 3 –** Flashcards are ordered based on user settings.
    4. **Stage 4 –** These ordered flashcards are now displayed to the user.

Using pipelining can reduce waiting time and lag which improves user experience but can also enables parallel development where different stages can be tested individually.

1.2 Stakeholders

In the context of this project, a stakeholder is any individual that will interact with the system and be influenced by its capabilities (e.g. improving long term information retention). The end user of this solution will be targeted towards students who want a method of studying with increased efficiency and engagement, also consisting of an element of competitiveness to help staying motivated. The goal, overall, of this solution is aid students develop confidence in their education by providing an application which promotes active recall, structured revision, and progress tracking. Thus, the primary client for this solution will be the students from Dormers Wells High School, who are currently preparing for their public GCSE, or A-Level examinations. As stated within the problem identification, pupils often experience challenges during their revision which include disorganised revision materials, the lack of motivation and difficulty retaining information; the system is designed to address all these issues and improve the study process for its users.

Dormers Wells High School is a diverse, comprehensive academic institution, consisting of around 1490 students; 100% of these students are obliged to study Mathematics, English, and Sciences until they complete their GCSEs, whilst many also pick them as subjects to complete as A-Levels for their post-16 education. Furthermore, all of these students have also studied content heavy subjects such as: History, Computer Science, Geography, and multi-foreign languages until a point of their secondary education. At the end of Year 8 or Year 9, students select their GCSE options, and once again in Year 12, they choose their A-Level or T-Level routes.

Since I have attended Dormers Wells High School since the age of eleven, I have experienced the intensity of the curriculum, and without organised study materials, it is near end impossible to achieve top grades. On average, a student at Dormers Wells High School completes 9 GCSEs; to achieve top grades, you need to retain information across various subjects, some which may have no links to each other at all, thus the Revision Assistant would be highly beneficial for students as it is a tool which specialises in making study clearly ordered, efficient, and motivating.

The folder-based system ensures students will be organizing their revision topics according to their subjects effectively, reducing revision anxiety and mental fatigue – an absence of structured revision resources can lead to a cognitive overload where students will find it difficult to begin revising. The shuffle feature prevents students from memorising the sequence of the flashcards, and enables active recall, enabling better information retention. In addition, challenges and progress tracking gamify the study

process and aid in tackling the issues of procrastination and encourage consistency in studying. Furthermore, to help students in revising more adeptly, the application allows the importation via CSV files and exportation via PDF or CSV; this means students can upload pre-made flashcards from teachers/peers, decreasing the time spent on producing revision materials and increasing the time spent on revising. Additionally, the exportation of flashcards allows more portable revision, offline and is suitable for different learning preferences.

1.3 Existing solutions

There are many applications currently available to help students with revision, these applications usually have features which include: flashcards, spaced revision, progress tracking and gamification. Although, each has their own limitations in terms of user engagement, organisation, adaptability and accessibility. These are all factors which the Revision Assistant aims to address.

1.3.1 Quizlet

Quizlet is a web or mobile application learning platform which provides interactive tools to help users study and master various subjects, offering features like digital flashcards, tests and study games. It allows users to create study sets or use pre-published ones created by others. Quizlet stores its data from users on the cloud.

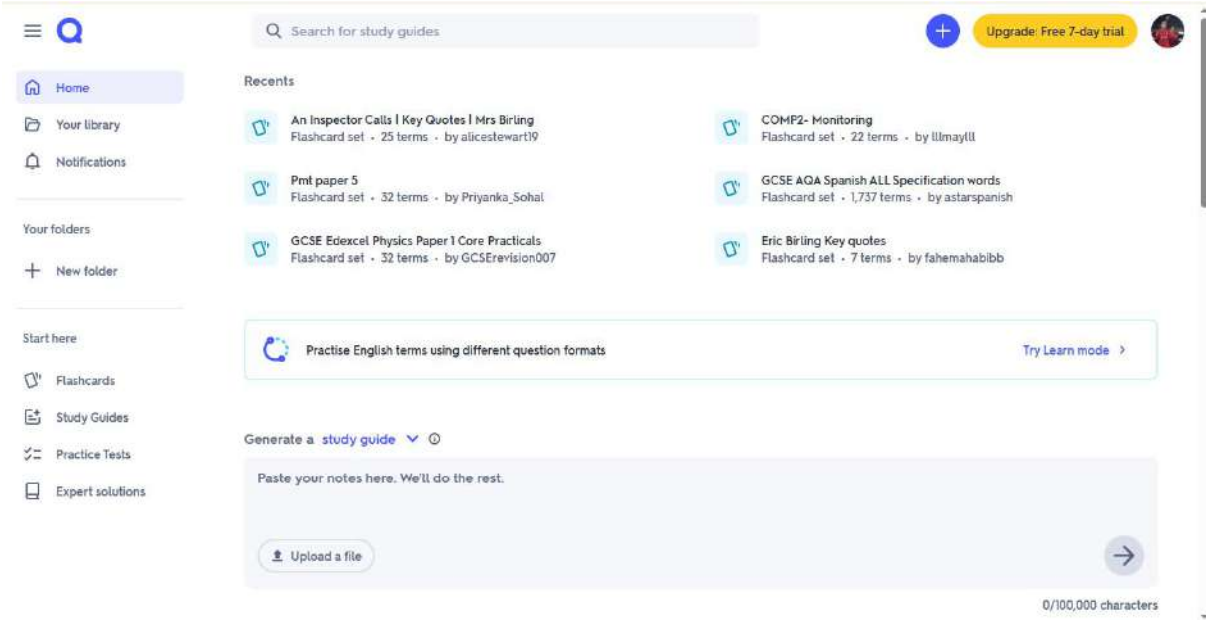
Advantages of Cloud Storage	Disadvantages of Cloud Storage
Studying is accessible from any device, anywhere with an internet connection.	Dependent on the internet connection, there is limited offline use as most features require a connection to the internet.
Automatic backup, hence, disaster recovery – so revision data isn't lost.	Privacy concerns as data is stored on third-party servers and can be different for every company depending on their privacy policies.
Less physical storage, reducing physical footprints, whilst also reducing costs.	Data retention: since Quizlet decides how long data is kept, deleted materials may remain on servers for a while
Enhanced Security where data is stored on secure servers with encryption – making it more difficult for unauthorised users to access your account information.	Annual Subscriptions for data storage. Additionally other features, for example analytical evaluations with stored data, can require extra payments.
	Third-Party Sharing Quizlet can share anonymized data for marketing or analytics.
	Compliance issues across countries affecting how user data is stored, processed and transferred across

	borders, so some features might be restricted or work differently in different regions.

However, since the Revision Assistant is designed for the students of Dormers Wells High School, data will not be stored on an international cloud but will be stored locally which can improve privacy and ensures data protection compliance.

Quizlet uses a menu driven interface, and its homepage is designed for users to have quick access to any revision needed. The interface is structured and prioritises usability as well as navigation.

1.3.1.1 Quizlet's Home Page

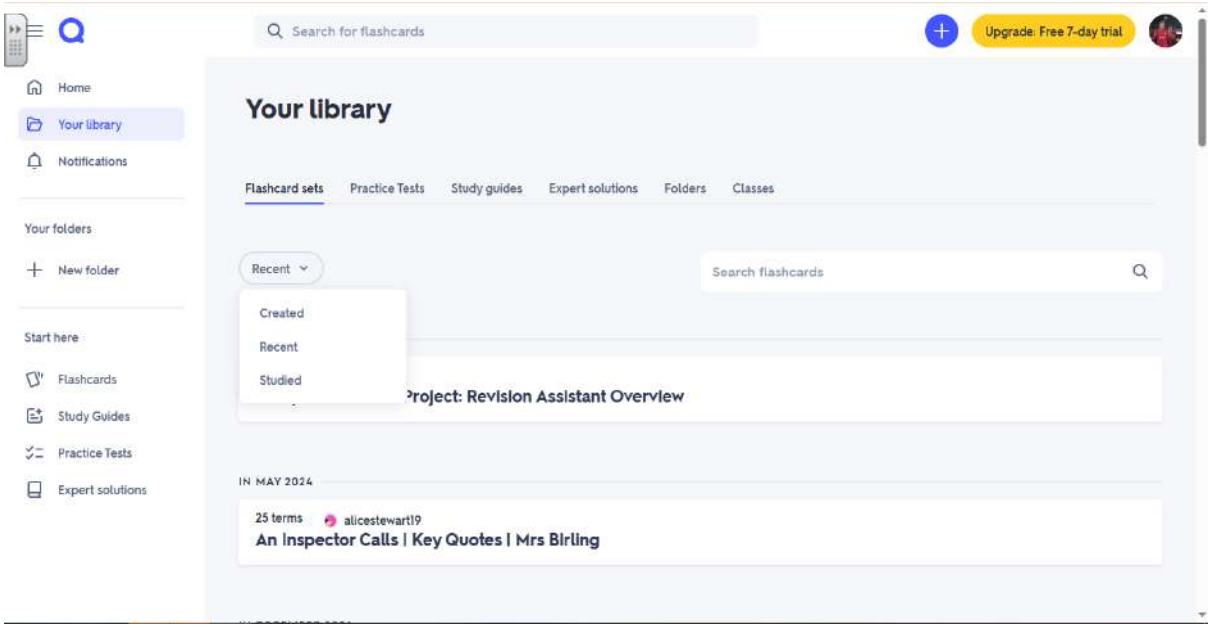


- Section for “Recents” – displays recently used flashcard sets which allows the user to carry on with their previous study session, encouraging ease of access.
- Side Navigation Bar – The bar contains clearly labelled options in a clear structure in persistent placement (users do not need to scroll to access key areas). There are also icons adjacent to the text increasing the readability and visual recognition
    - The bar is on the left, as most users in the western world read from left to right, so the navigation bar will be one of the first thing they see when on the page allowing easier access to main features.
    - Choice of Icons – icons in the navigation bar are all familiar and universal symbols so users can understand what the option is without reading the actual option's text.

- Affordance – when the user hovers over an icon or text, it changes to a darker colour to suggest interactivity, helping user understand these are actions and not plain text.

- Consistent Design – the setup of the application is not cluttered by using spacing and spreading the information out across different panels. This prevents the user from being overwhelmed by many different options. Quizlet have used consistent colours and fonts which support clarity as well as accessibility.
- Study Guide Generator – users can input text or file containing information about topic and the application will turn this into flashcard sets to study from.
- Encouragement to upgrade – constant encouragement to pay to gain more features, with a yellow button on the top right of the screen. This suggests that the free version would be very limited.

1.3.1.2 Quizlet's Library



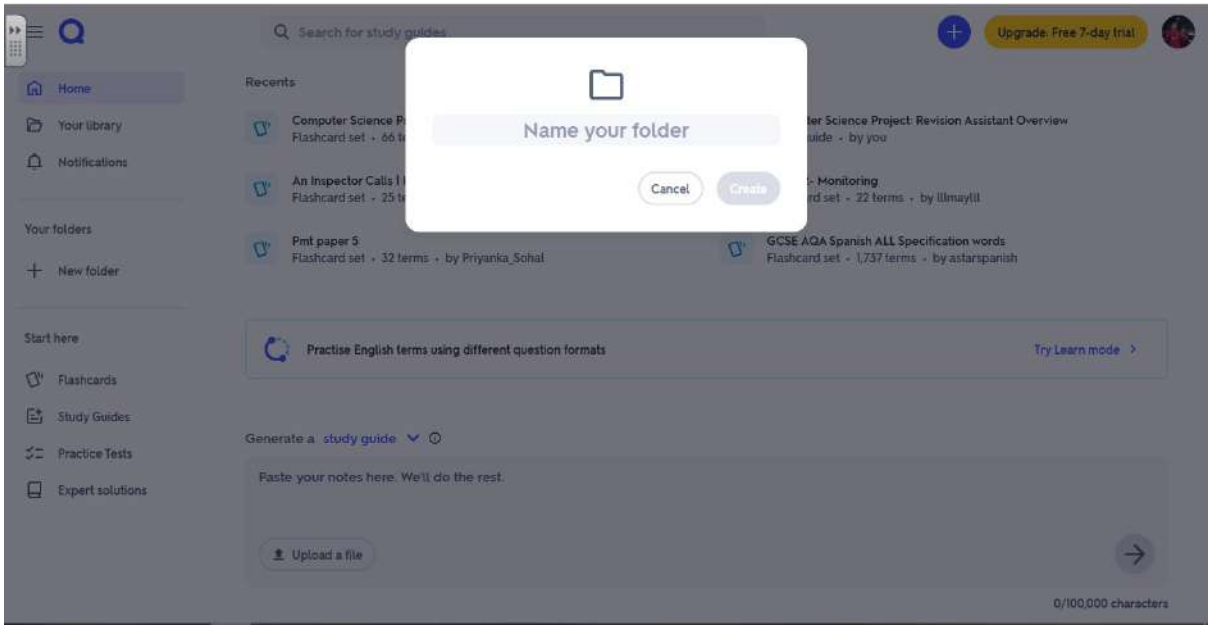
The library can be used to navigate the following:

- Flashcard Sets
- Practice Tests
- Study Guides
- Expert Solutions
- Folders
- Classes

On every sub-section of the library, there are options to filter the search or even search manually, as shown in the image. These enhance relevance as well as user experience, making it easier for users to find what they are looking for, reducing frustration of going through every possible option serially. In addition, by using filters and search options,

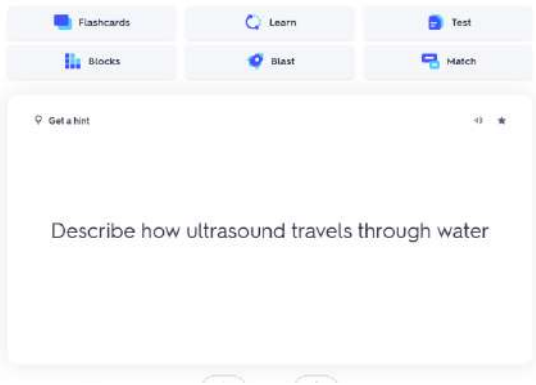
there is an improved efficiency of the application's task allowing users to quickly locate their criteria.

1.3.1.3 Quizlet's Create Folder



This feature allows users to group their revision together, thus similar topics can be categorised in folders, enhances organisation and reducing the possible chaos of revision materials. There are also clearly labelled buttons “Cancel” and “Create”, which are significant for users who rely on keyboard navigation. In addition, with the minimalistic design, there are fewer distractions in the interface which increases concentration, letting users focus on their important tasks, for example naming the folder in this instance.

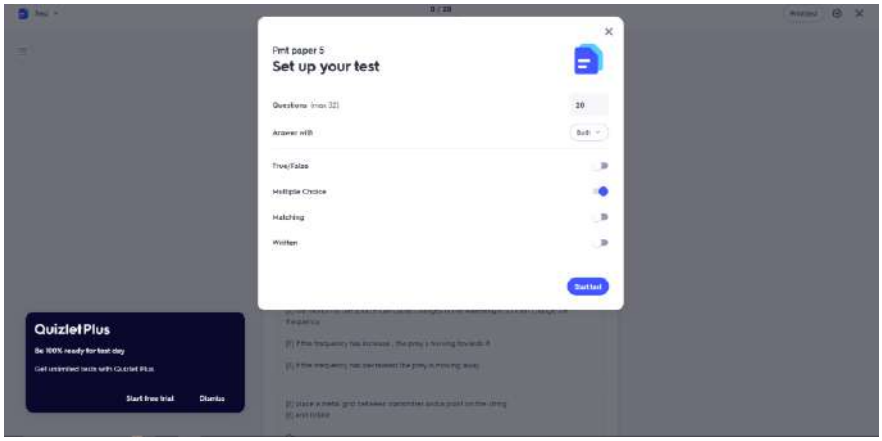
1.3.1.4 Quizlet's Flashcards



Points about the Flashcard Options:

- Has a variety of modes: flashcards, learn, test, blocks, blast, match. Gamification of learning, so students can maintain motivation.
- Hint function – building students' confidence as it gives students, who struggle with recall, clues.
- Text to Speech – reading aloud may help some students in revision.
- Star Button – allows students to favourite flashcards which they might struggle to remember.
- Design – high contrast and large font makes it easier to read.

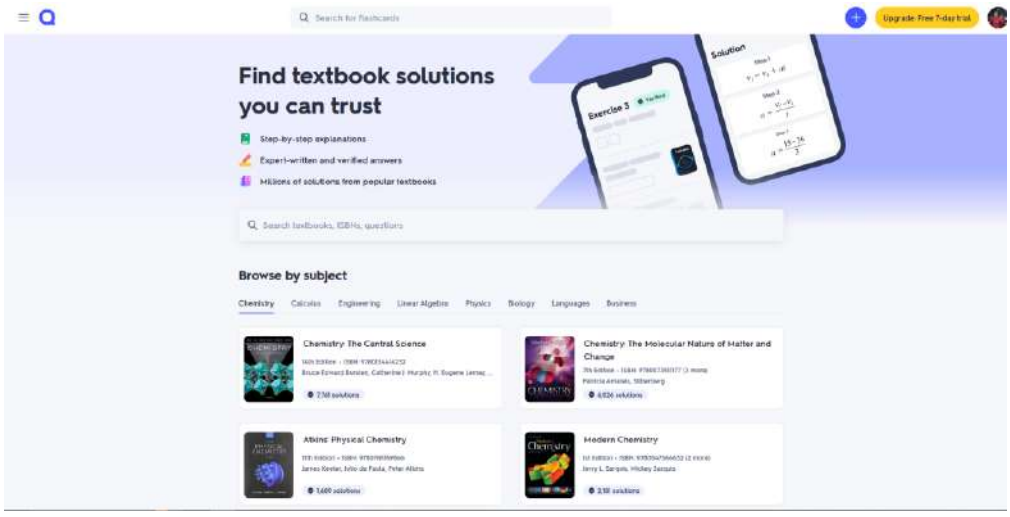
1.3.1.5 Quizlet's Practice Tests



The create test feature is helpful as:

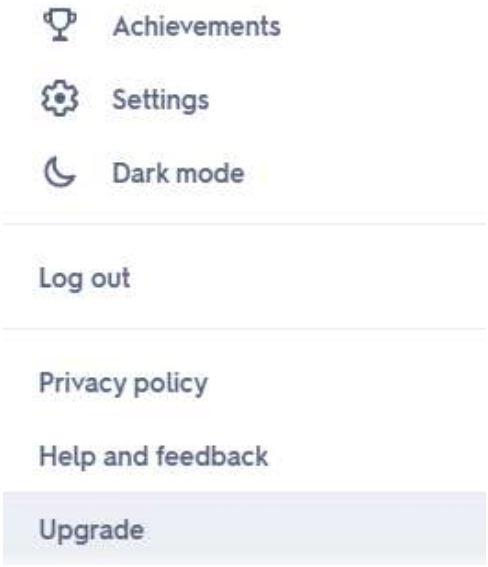
- Customisable – the test can be adapted to fit your needs, with fewer questions if you need to revise quickly, or if you need a bigger challenge.
- Variety of question types – supports different learning styles and students, for example people with dyslexia might prefer multiple choice questions over written.
- Focus areas – can turn off certain types of questions if you would like to focus on a specific skill.
- Progress tracking – shows what the user doesn't understands, simultaneously motivating them as seeing scores improve over time is encouraging.

1.3.1.6 Quizlet's Experts Solutions



Since all of Quizlet's flashcards are user-published, (new) users may have concerns over the validity of the statements; with the expert solution it breaks down the solution into clear steps, so you understand the whole process rather than memorising the answer whilst the answer also being verified by experts, building trust between the user and the application.

1.3.1.7 Quizlet's other features



Points about these options:

- Achievements option – allows user to see how far they have come from the start of their revision, can increase their motivation to continue revising. Furthermore, the users are more likely to come back and revise more using Quizlet as it feels as if they are working towards something with achievements in place, improving user retention.
- Dark mode – allows user to change the style sheet to their preference or depending on the time of day, as in low-light environments dark mode may be more comfortable for the user. In addition, this feature increases the accessibility as some people with certain visual impairments might find it easier to read dark backgrounds.

1.3.2 Seneca

Seneca is a free online learning platform designed to help students revise more effectively and is often used by A-Level, GCSE or KS3 students as they are recommended it by their teachers. It combines active recall and other scientifically proven techniques together and is designed to make revision more engaging and effective for students.

1.3.2.1 Seneca's Home Page



- Account level – gamification of revision, adding an element of competitiveness as students may want to the highest level possible, which would make them revise more.
- Creative, randomly generated nicknames – adds an element of fun and surprise, making the learning experience more playful.
- Assignments – allows teachers to set revision to do, which is helpful as sometimes students would not know where to begin.

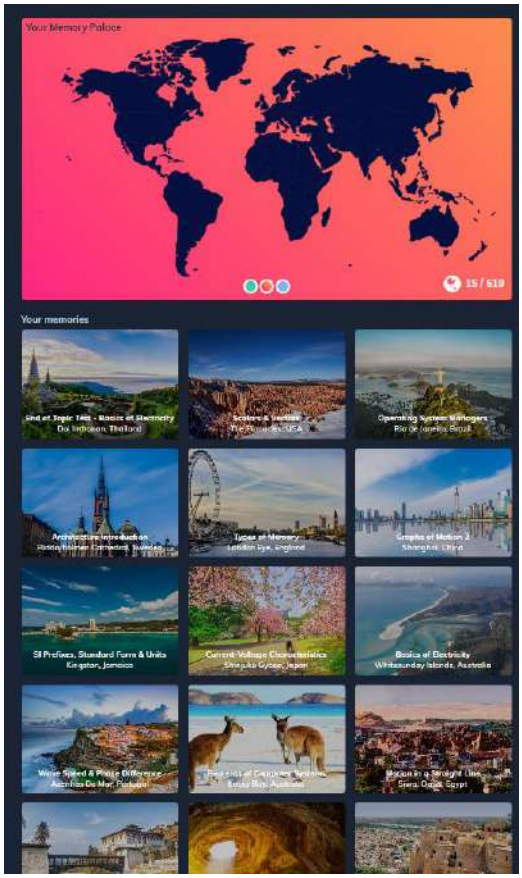
1.3.2.1.1 Seneca's attempt at gamification

1.3.2.1.1.1 Seneca's XP and Leaderboards



- Students can set XP goals, which provides a sense of achievement when reached and promotes self-motivation.
- Question Counter – helps student measure their engagement and progress over time.
- Total Learning Time – encourages time management and helps track student's dedication toward studying.
- Leaderboards – builds pride as well as friendly competition as students would want to be the top of the leaderboard, making them need to study more.

1.3.2.1.1.2 Seneca's Memory Palace

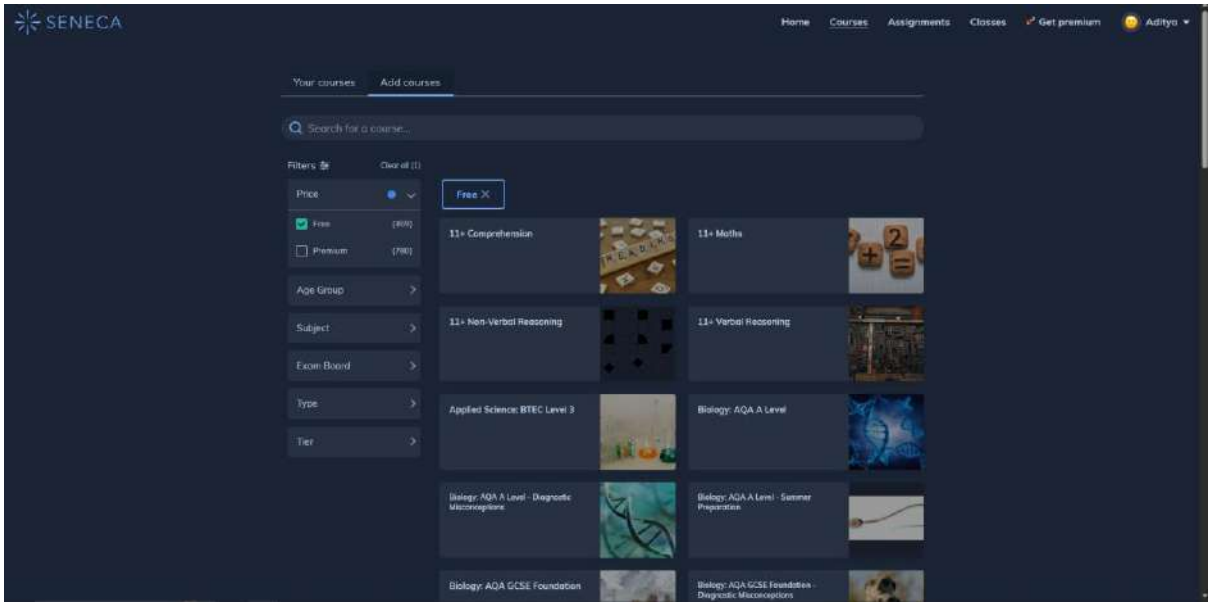


After completing a subsection of revision, you can assign this revision a city/landmark, this can help with information retention as thinking of the place can assist in you remembering what you assigned to it.

- Grid Layout – topics are in a uniform, organised grid for easy scanning and selection to study.
- Each card has the topic name, so it is clear to the user what they are picking.
- Progress indicator – heart icon and completion fraction to the bottom right of the map show progress, motivating the user to continue.
- Large, Distinct Click targets – the cards are big enough to avoid mis clicks, also implementing affordance where the card you are hovering over expands and your cursor changes to a pointer.

Seneca's attempt at gamification of revision through XP, leaderboards, the “Memory Palace” and progress tracking can increase motivation by adding a competitive aspect to the game. This competitive aspect also makes revision feel more rewarding when completed, making the user want to use the application more often. In addition, adding progress tracking will help with self-assessment and identifying studying habits. However, leaderboards also may demotivate users who are consistently ranked low, and the XP feature can change the point of the program from studying to just constant point farming to top the leaderboards. Furthermore, with the excess of visual features such as images would increase the program size for the Revision Assistant but for Seneca since it is a web application, it may load slower for users with slower internet connections.

1.3.2.2 Seneca's Course Page



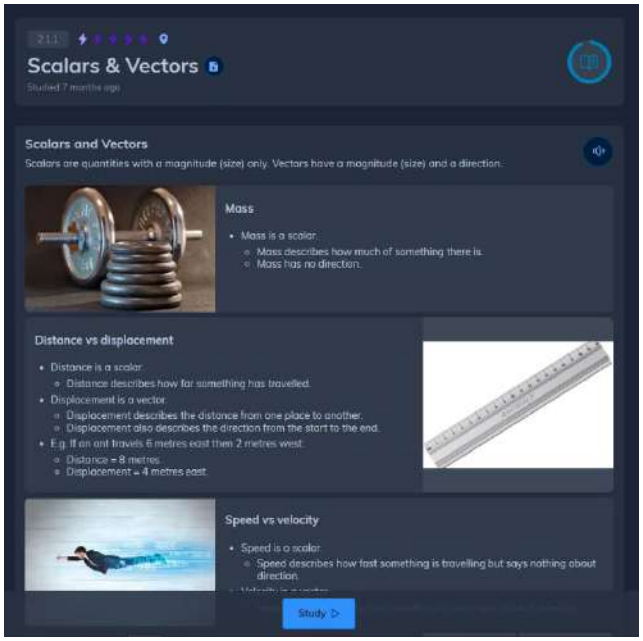
Features of Seneca's Course Page:

- Search filters – allows the user to narrow down the search making use of divide and conquer algorithms, this improves the user experience, and they can find revision suited to their needs quicker.
- Different versions – premium and free, where premium offers many more learning options than free, which students may not be able to access due to their financial circumstances.
- Courses – made by professionals so details are most likely correct and specific to the exam board it is made for, providing online revision with students can trust.
- “Your courses” and “Add Courses” – shows user's previous courses to allow instant resumption of revision or allows user to begin further revision on a different topic.

- Colour scheme – dark blue reducing eye strain and supporting late night studying.

1.3.2.3 Seneca's actual revision

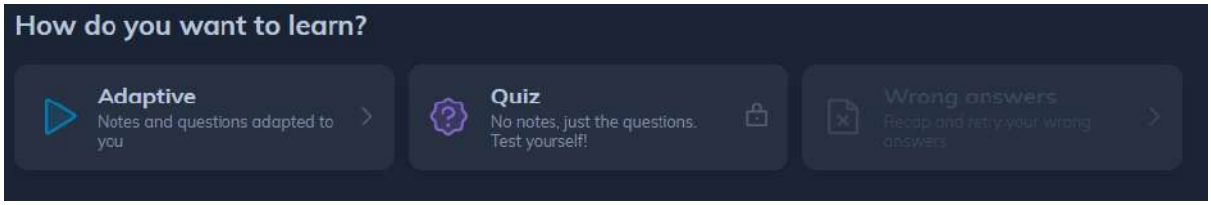
1.3.2.3.1 Pre-Study Knowledge Overview



When you first open a revision topic, the application provides you a knowledge overview of the topic and has constant a study option at the bottom, for when you think you are ready to begin studying.

Below the title of the topic, it tells you the last time you have studied this topic (if you have done so before), this allows students to do their own spaced revision, for example going over this topic every two weeks.

1.3.2.3.2 Study Options



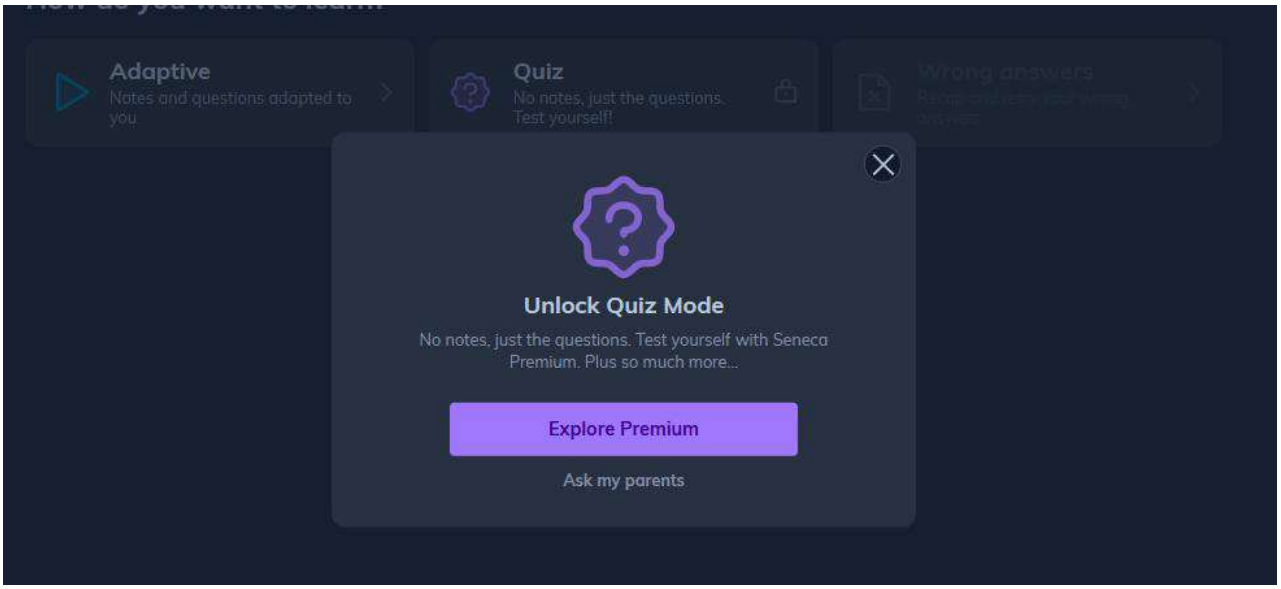
There are three different study options for the user: in adaptive:



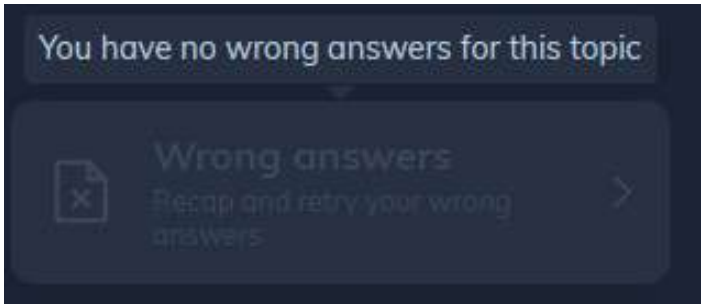
The user is given informative flashcards about the question topic which they are expected to read through and after they click the next option, a question appears which can either be for a few sentences, a fill in the blank or a multiple choice. Different question types can help the user study as they will be tested on all fronts when using Seneca. There are also images to help users who prefer visual learning. The questions can be adapted to the user with the use of heuristics, where if the user gets three harder

questions wrong, they might start giving out multiple choice questions to test on knowledge.

Furthermore, when looking at content or questions within Seneca, there is always an option to convert the text to speech, ensuring users who prefer auditory learning to have an option. In addition, text to speech increases the accessibility of a program, as it supports people with visual impairments, dyslexia or other reading difficulties.

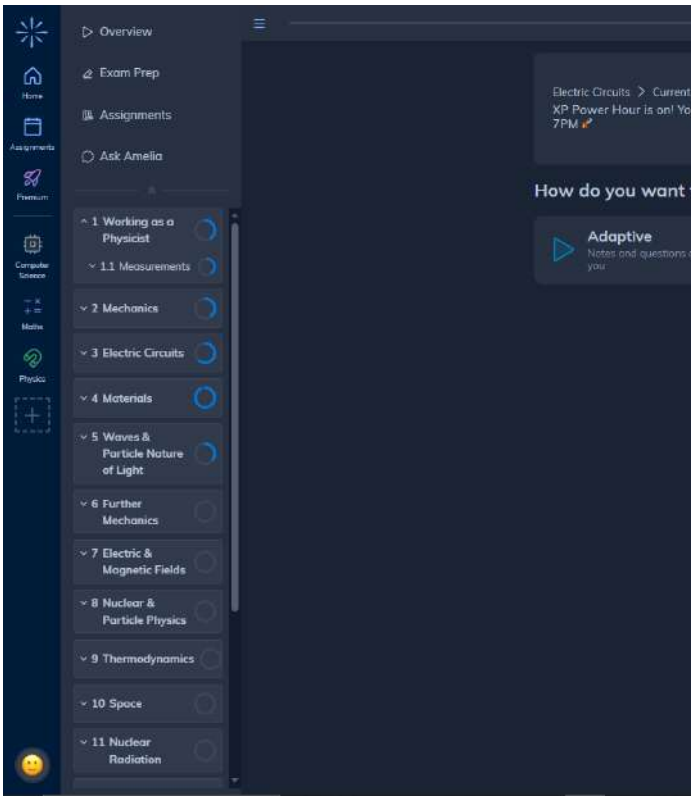


To access the quiz option, where there are only questions and no notes, to help consolidate knowledge; you must have the premium version of Seneca, many students may not be able to afford this or want to spend their money on this. However, the quiz option is an incredibly useful feature to enable active recall.



For the wrong answers option: the user is tested only on questions they have got incorrect, it aids the user in error correction as revision is focused and adapted for the user in topics/questions they find harder. Seneca, with the use of this study option, increases learning efficiency by allowing users to be targeting weaknesses and can give a motivation boost as the user can see progress as the wrong answers option becomes unavailable when all their questions are answered correctly.

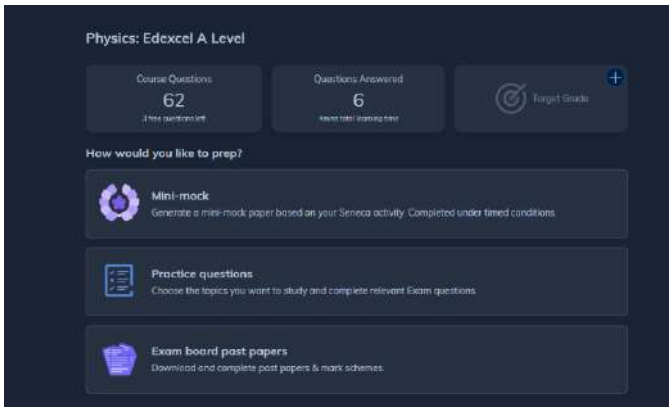
1.3.2.4 Seneca's Navigation Bar



Features:

- Option to go to the homepage, possibly to check progress.
- Option to change current subject (which the user is revision) or add other subjects.
- Collapsible subject menus, keeping the interface uncluttered, so revision looks less chaotic.
- By pressing the four-line icon at the top left, the navigation bar can be closed so the user can be focused on revision.
- Colour Contrast between icons and labels, ensuring accessibility for users with visual impairments.

The navigation bar also has options to change within the subject you are currently studying, breaking down the topic into subtopics and having progress indicators, to let the user know how much has been completed. These topics are also collapsible, so revision is uncluttered and is not overwhelming for the user due to how chaotic it may be. In addition, there is also options to overview the course, for example in the A Level Physics Course, it displays all the topics and what the user must know to sit the exam.

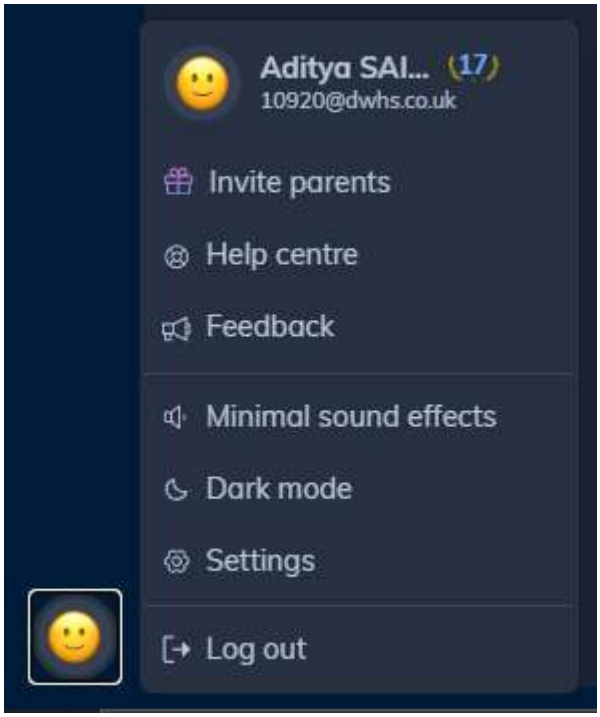


Moreover, there is also section dedicated to exam preparation, this helps with time-management with its mini-mocks and introduces the user to exam questions which is more difficult than content, helping the user becoming more familiar with the exam style. Seneca also supplies an option for past papers which provide authentic and exam board-specific questions for students to practice using.

Seneca also has its own AI Chat Bot (Ask Amelia) which supplies help with questions, homework or topics the user is struggling with. The chat bot will give responses in real time to help the user overcome their misunderstandings.

Furthermore, the navigation bar is designed with recognisable icons, so users don't have to memorise where things are as the visual symbols act as indicators. These icons are universally recognisable, so it's easy for new users to adapt to the application. The navigation bar is also located on the left side of the page, this is because in the Western world, users read from left to right thus by having the navigation bar on the left, it will be the first thing the user sees.

1.3.2.5 Seneca's Other Features



Seneca has the option to enable/disable dark mode which would be beneficial for user with visual impairments, or for users who prefer to have dark mode for night studies to reduce eye strain. In addition, dark mode can also save battery since it is more energy efficient, making it more useful for mobile and laptop users.

There is also an option to have minimal sound effects, this ensures enhanced focus on studying as some users may prefer studying in silent, with sound effects possibly distracting them. This feature can also be used to adapt to the study environment, for example in a library you'd have minimal sound effects.

1.3.3 Summary

Based on the analysis of solutions above, I have summarised the key takeaways for my project in the table below.

Features to Avoid	Features I may implement
• Animations and distracting visuals ○ Both solutions above avoid the use of superfluous animations. I will also avoid doing this as it will prevent the user from being overwhelmed and distracted therefore allowing them to focus on revision.	• Text to Speech ○ Both solutions above use text-to-speech, allowing users to listen to revision content, supporting users who prefer auditory learning. I will add TTS with adjustable volumes and speeds to ensure learning is personalised therefore allowing the user to have the maximum positive impact on their revision from this app.
• Locking features behind paywalls ○ Quizlet and Seneca lock some essential features behind a paywall, so if you do not have the premium version, you do not have full access to the revision website. I will avoid this as it reduces the effectiveness of the program for those not able to pay, ensuring all users have equal access to revision.	• Achievements ○ Both Quizlet and Seneca use achievements to motivate users to continue studying, I aim to implement a similar system which awards users for milestones, ensuring that these are strictly learning related to avoid over-gamification. This combats procrastination and helps users who struggle to stay focussed.
• Inflexible Study Order ○ Restricting users to only follow one strict learning path limits personalisation and discourages active recall, forcing all users to follow the same pattern of learning, even if they do not prefer to learn this way. I will allow flexible topic navigation options, allowing users to choose their own revision, leading to improved outcomes.	• Different Style Sheet options ○ Both solutions offer different stylesheet options for users, for example dark mode. I will let users switch between multiple stylesheets, so this program is accessible to the visually impaired, or others who prefer different colours. In addition, dark mode in particular could be used to reduce eye strain, supporting students who revise at night.
• Inconsistent Fonts ○ Using multiple fonts can make content harder to read, causing visual discomfort. I will avoid the use of inconsistent fonts to ensure a clean, professional, and accessible interface. This will make my application suitable for students with a range of needs including neurodivergent users.	• Clean interface ○ Both the analysed solutions avoid unnecessary clutter within their interfaces, reducing the chaos, allowing easier access for users. I will integrate a clean, consistent layout to reduce the chances of overwhelming the user, so their working memory is focussed on revision.
• Random Colour Schemes ○ Random, or intense colour combinations can also harm readability, hence I will avoid this by using a consistent, high contract	• Multiple study modes ○ Seneca offers users adaptive practice, mini mocks, etc; Quizlet offers flashcards quizzes and test modes.

colour scheme. This will meet the needs of autistic users above all.	○ Shuffle mode strengthens long term retention of information through active recall, enhancing exam performance. ○ Wrong answers only mode allows students to focus on topics of weakness, increasing effectiveness of revision, reducing wasted time on already mastered content. ○ With inspiration, I will add multiple modes to the revision assistant to aid different versions of studying which users will used for different purposes.
• Over-Gamification of Studying ○ Excessive gamification would shift the focus away from studying to topping the leaderboards, or farming XP. I will ensure all gamified elements avoid overshadowing the purpose of the Revision Assistant.	• Simple navigation ○ Both solutions have the navigation bar on the left-hand side (following the western culture) to make it the first thing users see. Furthermore, this reduces cognitive load, supporting visual learners, as the system is easily navigable due to the lack of dense text. I will adopt a similar design, making navigation simple, easy and logical for the user – this will ensure there is more focus on studying.
• Sounds ○ Overusing sound effects can distract users from studying and would not be convenient in quiet areas. Sound effects can also overwhelm neurodivergent users. To have minimal distractions, ensuring a calm, focused environment , I will allow sounds to be disabled entirely or be reduced to the user's preference.	• Meaningful Progress Tracking Options. ○ Both solutions track progress visually; with visual progress tracking, users can see how far they have come, motivating them. Many students struggle to judge their revision is effective, which can cause anxiety. I will provide clear performance indicators within the program to help students track their progress.
• Cloud Data Storage ○ Relying on cloud storage would not be efficient, since it requires a stable internet connection, hence offline study won't be available. I will avoid the use of cloud storage and will use local storage for data.	• XP ○ Both solutions have gamified studying through being able to gain experience points (XP), I will use a similar system as gaining XP, levels, etc will give the user a sense of progression which is similar to games. XP provides positive feedback in real time, so revision feels more rewarding to the user.
	• Collapsible Menus ○ Seneca uses collapsible topic menus, as well collapsible navigation bars. I will let users collapse menus for when they are not needed. This ensures students can shift

	their focus away from the clutter and focus on their revision.
	• Offline Study ○ Since these are both web applications, they do not provide offline access. I will ensure the Revision Assistant can be accessed offline so users can learn without the need of an internet connection. This reduces distractions from notifications for the user.

1.3.4 Possible Approaches to the Solution

The solution that I propose must involve efficient memory management, object-oriented programming support, and a GUI library in order to make the Revision Assistant possible. In addition, since the program is designed to be a desktop application for students, it must also be cross-platform as well as accessible. Not many programming languages support all the features the Revision Assistant requires, but the three main candidates are:

Python	Java	C++
		
Python is a high-level, interpreted programming language that runs on the system. Python supports the object-oriented programming paradigm and offers many libraries such as PyQT or TKinter for GUI development. For the most part, python is slower than compiled languages, whilst deploying and distributing applications may be more complex as Python is required to be installed on the user's system. My source code will also be visible opening my	Java is a high-level compiled programming language; it is a fully object-oriented that runs on the Java Virtual Machine, making it cross-platform, increasing portability. Java provides a range of GUI libraries including JavaFX and Swing which allow intuitive user interface development. Although, it is less efficient than C++, there is a balance between performance, readability and portability for Java.	C++ is a compiled low-level language which allows direct access to memory as well as system resources. The efficiency and power of C++, combined with its libraries such as Qt, for GUI development make it one of the best candidates for the Revision Assistant. However, C++ is machine dependent, making it less portable since it is converted into machine code when executed. In addition, it is also more complex to learn and implement, which is less suitable for this project.

application to being copied (violating my IP) or security breaches.		

1.3.4.1 Justification of Language Choice

Amongst these three choices, my solution will be developed using Java. The main reasons being I am familiar with the syntax since I have learnt Java throughout my years in education, Java's balance between performance, portability and readability. Java has built in support for OOP making it optimal for modular applications, for example the Revision Assistant, where various features are expected to be developed as different components.

Furthermore, Java offers JavaFX, which is designed to be a versatile GUI library, allowing the creation of clean, professional-looking interfaces. This is ideal to make sure the Revision Assistant is accessible and friendly for students, whilst simultaneously supporting key features such as data visualisation, etc.

Java is also beneficial for its increased portability (as it is run within a virtual machine, hence the file to run is not machine specific), since the Revision Assistant targets a wide range of users, being able to run smoothly across Windows, Linux and Mac ensures wider accessibility.

However, there are some limitations to Java. Firstly, since the code is run within a Java Virtual Machine (JVM), there is a performance overhead from it which means it can be slower than other compiled languages such as C++, although this is not critical for the purpose of the Revision Assistant. Memory usage is generally higher within Java than other lightweight languages such as Python, which can affect users on older hardware. Moreover, users also require the Java Runtime Environment (JRE) to be able to run the application, which adds an extra step to installation.

Despite these limitations, Java remains the optimal choice for the Revision Assistant due to its balance of cross-platform support, libraries, security and maintainability, providing a structured environment for a reliable and scalable solution.

1.3.5 Essential Features of the Proposed Solution

1) Clean Interface with Collapsible Menus
    - I will ensure the Revision Assistant consists of minimalist navigation, as it will avoid clutter, helping neurodivergent users, hence allowing them to focus on studying rather than navigating. This also minimises the distractions whilst studying, reducing the chance of revision being disrupted.
2) Stylesheet Options

- I will add an option for users to be able to switch between multiple stylesheets depending on their preference/need. This will be important to increase accessibility for users, for example: high contract for users with visual impairments such as colour blindness; dark mode to reduce eye strain, etc. In addition, the option to change stylesheets can increase user comfort for longer study sessions.

3) Minimal Sound Effects
   - The solution will avoid having excessive sound effects, catering for users with ADHD, autism or sensory sensitivities, which can distract users and the program will aim to stick to small auditory cues for correct/incorrect answers. This can be turned off, if too distracting; however, can also be used to reinforce feedback since these cues are non-visual confirmations of successful actions when right, or to indicate errors when incorrect.
4) Offline Accessibility
   - The solution is entirely offline, so no internet will be required, I am going to do this to ensure reliable access to revision, even without internet connectivity. Many other similar applications do not provide this, reducing the portability of revision.
5) Local User Profiles
   - All user data will be stored locally, and access rights will be allocated to authorised personnel only. I am going to do this to avoid the use of cloud storage, increasing costs of the solution. Furthermore, the use of third-party storage may also increase the risk of exposure of sensitive information – avoiding cloud storage will increase data security within the Revision Assistant.
6) Accounts
   - Whilst, this solution is primarily designed for single-user devices, there will be registration and login functionality to tackle scenarios where multiple users will use the same device for the assistant. This also ensures all users can keep a separate and secure profile – allowing more personalisation for users with different studying needs, e.g. colour schemes, sounds, etc.
7) Progress Tracking
   - The solution will have different ways of gamification to help with progress tracking, for example experience points (XP), achievements, etc. I am going to do this since many users struggle to measure how effective their revision is, causing anxiety. With progress tracking, users can stay motivated throughout their studying and can see their growth over time.
8) Advanced Analytics
   - I will have charts, graphs and other ways of data visualisation to display users their progress. These will help the user also identify their strengths

and weaknesses, which can allow learners to focus on topics they find difficult.

9) Text-to-Speech (TTS)
    - I will TTS libraries to allow users to play flashcards/questions out loud to users, improving the accessibility of the solution for visually impaired users, or users with dyslexia.
10) Multiple Study Modes
    - The Revision Assistant will offer a variety of study modes for users with different preferences for learning; this will increase engagement and effectiveness of studying for leaners. For example, there could be a study mode for questioning incorrect answers only – this will help users improve on their mistakes.
11) Customisable Flashcard Sets
    - Users will be able to create their own flashcards, adding personalisation towards the learning experience according to their preferences. This enables users to focus on areas which they need the most practice. By users having full control over their learning, the program encourages better information retention, active engagement and a more efficient revision process, overall.
12) Session Time-Tracking
    - Students may want to know how long they have spent on the Revision Assistant; this will help them allocate time more effectively (in the future), simultaneously supporting self-evaluations and promoting sturdy discipline.
13) Importable/Exportable Study Sets
    - Teachers may want to set their students flashcard sets to revise from, or users may want to share their flashcard sets with their friends. I will make it so the Revision Assistant's flashcard sets which are made from the users can be exported and imported. This will reduce the duplication of efforts as users can build collaborative resource, increasing the portability of study material.

1.3.6 Limitations of the Proposed Solution

1. Single-User Only
    - Since data is stored locally, the program is single user, which means there will be no libraries to share flashcard sets, group progress tracking and teacher dashboards. I am avoiding this, to reduce ongoing costs as well as decrease unnecessary complexity within the program.
2. Local Data Storage Only

   - All data will be stored locally, instead of the cloud. I am going to do this because it will help me tackle the issues of setting up online servers and making sure data storage complies with data protection laws, keeping the project achievable. However, it means progress will not be synced across devices.
3. Requires Java Runtime Environment (JRE)
   - Since the program will be coded using Java, it can only be run on devices with the JRE installed. This is because Java is run within a virtual machine to promote cross platform capabilities. However, users without the JRE will have install it separately.
4. Lack of Data-Back Up
   - If users delete the folders storing their data, the data can be lost. If the Revision Assistant backs the data up locally, it could take too much storage for the user's preference. Furthermore, implementing automated backup systems would increase development time and complexity of the project.
5. Limited Customisation of the Interface
   - Users can switch the stylesheets, and turn off audio effects, however the Revision Assistant does not allow deeper customisations, font packs for instance. I am going to do this as these features would increase development time which could be spent on increasing the efficiency of revision on the program.
6. No mobile, or web version.
   - The solution will only be available as a desktop application; I am going to do this because creating a web version or developing for Android/IOS users would additional frameworks and testing. This would be outside of the project scope.
7. Lack of real-time collaboration
   - Users will not be able to share progress, flashcards or leaderboards with each other in real time. Flashcard sharing is tackled through the exporting and importing of files. However, the rest will not be implemented as adding networking features would shift the focus away from personal revision, which is the main goal of the Revision Assistant.
8. No AI-generated explanations or sets
   - The program will not use generate flashcards, explain questions/answers with the use of artificial intelligence. I am going to do this because students should be refraining from the use of AI when studying since it can be unreliable, due to consistent hallucinations. This will ensure users create and use accurate material which is personalised for their own use.
9. Limited Media Integration

- Unlike Seneca, the program will primarily support text-based flashcards, not with interactive videos or images. Adding more media would significantly increase the memory footprint of the program as well as the file size.

10. Basic Sound Control Only
    - The program will include minimal sound effect and not advanced audio customisation. This is because adding different sound packs, or dynamic audio feedback would increase the complexity of the project. Sound control will be limited to volume sliders as well as mute options.
11. Monolingual Program
    - The Revision Assistant will only support English, this is because multi-language support would require more testing and other resource bundles.
12. No Built-In Content Libraries
    - Unlike Quizlet, the program will not provide users will pre-made flashcard sets in libraries. This is because user content creation would increase the project's scope, requiring additional research, data validation and storage management. In addition, this will reduce the unreliability of studying when using the tool.

1.4 Requirement Specification

1.4.1 Developer's Hardware/Software Specification

Based off IntelliJ IDEA's recommended requirements, I have generated the following hardware and software requirements:

Item	Minimum	Recommended	Justification
CPU (Clock speed / Cores / Cache)	Intel Core I3-10100 (4 cores, 3.6GHz, 6mb)	AMD Ryzen 5 5600 (6 Cores, 4.4GHz, 16mb)	A faster multi-core CPU allows smoother multitasking, which lets the IDE and Revision Assistant run concurrently without lag, when coding, debugging or testing.
RAM	2GB DDR4	8GB DDR4	Having more RAM will improve performance when running IntelliJ IDEA, the JDK and multiple testing and debugging sessions without relying on virtual memory.
GPU	Integrated Intel UHD Graphics	NVIDIA GTX 1650	Although the program will not be graphically demanding, a graphics co-processor will improve rendering of the IDEA and support dual monitor setups for increased productivity.
Secondary Storage	256GB HDD/SSD	512GB NVMe SSD	Having an SSD will improve indexing, project compilation speed, system boot

			times; all of these collectively make the development process quicker and smoother.
Keyboard	Any keyboard, TKL, 60% or 108 Key	Mechanical 108-Key Keyboard	Having a 108-key keyboard is beneficial as it has dedicated keys such as a number pad, arrow keys and function keys. This enables faster, more comfortable coding sessions.
Mouse	Standard Optical Mouse	Mouse with Scroll Wheel and Additional Buttons	A scroll wheel will improve navigation between codes and the IDE, whilst additional buttons will speed up common tasks like undoing, etc.
Monitor	Any monitor with a suitable display	Dual Monitor, 1080p, 24'	Dual monitors allow running the IDE, browser, debugging tools and others side by side to help programming. In addition, a bigger monitor improves readability of code.
Speakers / Headphones	Basic Onboard Audio	High-Quality USB Headset	Allows for testing of features such as text-to-speech. Better quality audio will improve accessibility related features.
Operating System	Windows 10 (64-bit)	Windows 11 (64-bit)	OS provides a user interface, allowing the user to interact with the computer and provides a platform to run software, allowing the user to run the IDE, JDK and other project dependencies.
IDE	IntelliJ IDEA Community Edition	IntelliJ IDEA Ultimate Edition	IntelliJ is an IDE which provides features such as syntax highlighting, code completion, code correction, auto indentation, breakpoints, step through/step back, and compilation. These will all help develop the Revision Assistant.
JDK	Java 17	Java 21	The Java Development Kit is required for compiling and running the program efficiently. The latest version is recommended to ensure compatibility and security.

1.4.2 User's Hardware/Software Specification

Based off the Java Runtime Environment's (JRE) recommended requirements, I have generated the following hardware and software requirements:

Item	Minimum	Recommended	Justification
CPU (Clock speed / Cores / Cache)	Intel Core i3-7100 (2 cores, 3.9Ghz, 3mb)	AMD Ryzen 5 4500 (6 cores, 4.0Ghz, 12mb)	The Revision Assistant is lightweight; however, a multicore processor ensures smooth performance when multitasking with other applications.

RAM	4GB DDR4	8GB DDR4	The program itself does not have a large memory footprint, however more RAM will allow users to run the JRE alongside other programs including browsers, documents or music applications in the background, without the need of virtual memory.
GPU	Integrated Intel UHD Graphics	Integrated or Dedicated GPU	No graphical processing is required but a GPU is necessary to handle the interface of the Revision Assistant smoothly.
Secondary Storage	128GB HDD/SSD	256GB SSD	The Revision Assistant does not have a large storage footprint, but with an SSD, load times will be reduced which increases the overall responsiveness of the application.
Keyboard	Any keyboard, TKL, 60% or 108 Key	Any Keyboard	The keyboard is of the user's choice, whatever they feel comfortable to revise with will be used to create flashcards and use the Assistant.
Mouse	Standard Optical Mouse	Mouse with Scroll Wheel	A scroll wheel on the mouse will allow easier navigation of the study material.
Monitor	Any monitor with a suitable display	Any monitor with a suitable display	A larger monitor will help display more content at once, improving readability of revision content.
Speakers / Headphones	Basic Onboard Audio	USB or Bluetooth Headset	Essential for using features like text-to-speech and will be used to hear sound effects.
Operating System	Windows 10 (64-bit)	Windows 11 (64-bit)	Required to run the JRE and provide compatibility with the Revision Assistant software.
JRE	Java 17	Java 21	Required to execute the compiled Java program. Newer Versions offer better performance, security and compatibility.

1.4.3 Success Criteria

To be able to evaluate the effectiveness and reliability of the Revision Assistant, it is essential to define a measurable success criterion. The following criteria will set the benchmarks which will be used to determine whether the final product is sufficient for its intended users' needs. By ensuring the Success Criteria is measurable, I will be able to test the solution's functionality, usability, performance and overall sustainability.

The following indicates the benchmarks of my solution:

1) The program should launch within 10 seconds on a recommended-spec (or higher) device.

- Fast startup will prevent the user from frustration in a time where they may be overwhelmed with stress. This will also ensure a smooth user experience.
    i. Test by timing how long the program takes to open after execution.

2) The Revision Assistant should display a splash screen when the application is launched, prior to registration/login screen.
    - A splash screen provides a platform for a low effort command to confirm the application is loaded and responsive, the user will control the pace of the interface launch without frustration, supporting efficiency. It will also allow for any initial processes to load before important user interaction (such as logging in), reducing the likelihood of errors such as unresponsive behaviour.
        i. Test by launching the program and checking the splash screen is displayed first and can shift to the login screen immediately.
3) The system should allow multiple users to register and log in on the same device.
    - The Revision Assistant may be used on a shared device, such as a family or school computer. With multiple accounts, the program ensures each user's login credentials and revision data remain separate and secure from each other, preventing potential data overwriting.
        i. Test by registering multiple accounts with unique email addresses on the same device and attempting to log in with each account separately.
4) The program should not store duplicate email addresses and must not store passwords in plain text (store hashed password instead).
    - The email address acts as a unique identifier for the user within the system, if there would be the same email for multiple accounts, wrong data could be loaded. Storing passwords in plain text would be insecure, allowing anyone with access to the storage file can read them, hence credentials will be hashed with a salt to make the system more secure.
        i. Test by registering multiple users with the same email address, checking whether it is possible.
        ii. Opening the csv file with credentials are stored and observing whether passwords are stored in plain text.
5) The solution will store data locally and must not crash if the data file corrupted.
    - Since all user data will be stored locally, the program must make sure data is not lost on restarting the app, ensuring the Revision Assistant remains usable even if a data file is corrupted
        i. Test by storing progress locally, restarting the app, and verifying whether data is still there.

    ii. Purposefully corrupt a data file to check file handlers do not crash the program.

6) The program will use a high contrast black and white style sheet for users with colour blindness, and will have other options available (dark mode, light mode, etc); changes must apply instantly.
    - This increases the accessibility of the program for users and is critical for preventing eyestrain or users with visual impairments or learning preferences.
        i. Test by timing how long it takes to change style sheets.
7) The solution should accurately measure XP and Achievements. This includes:
    a. Revision of 1 hour will be 500XP
    b. Completing an entire deck successfully with 90% accuracy will award 100XP
    c. Each correct answer awards 15XP and incorrect awards 5XP.
    d. Achievements must unlock automatically when conditions are met, (milestones such as number of study sessions, hours completed, etc).
    - Gamification elements can only motivate users if they reflect real effort, if the XP gain is not accurate, the students may not gain the motivation they need to keep studying.
        i. Complete a study session and see whether XP gained is accurate, and if achievements are given immediately.
8) The program must be available for offline use.
    - Many students study in places without stable internet connections, offline access will ensure they can use the Assistant to study anywhere, at any time.
        i. Test by disconnecting your internet connection and use the program.
9) The program must allow users to import (csv) / export (csv/pdf) flashcards.
    - This will save students' time when studying and increases collaboration with other students or teachers. Furthermore, this also makes revision more portable as students can print flashcard sets for paper-based learning.
        i. Test by importing and exporting flashcard sets.
10) The solution must have session time-tracking in hours and minutes.
    - Students can measure how long they revise, which will help improve their time management and study discipline.
        i. Test by completing revision sessions and see whether recorded times are correct.
11) The program must have volume sliders to turn on and off sound effects.
    - Some students may see sound effects as distractions, whilst others may see it as useful indicators of progress. To please everyone, the program

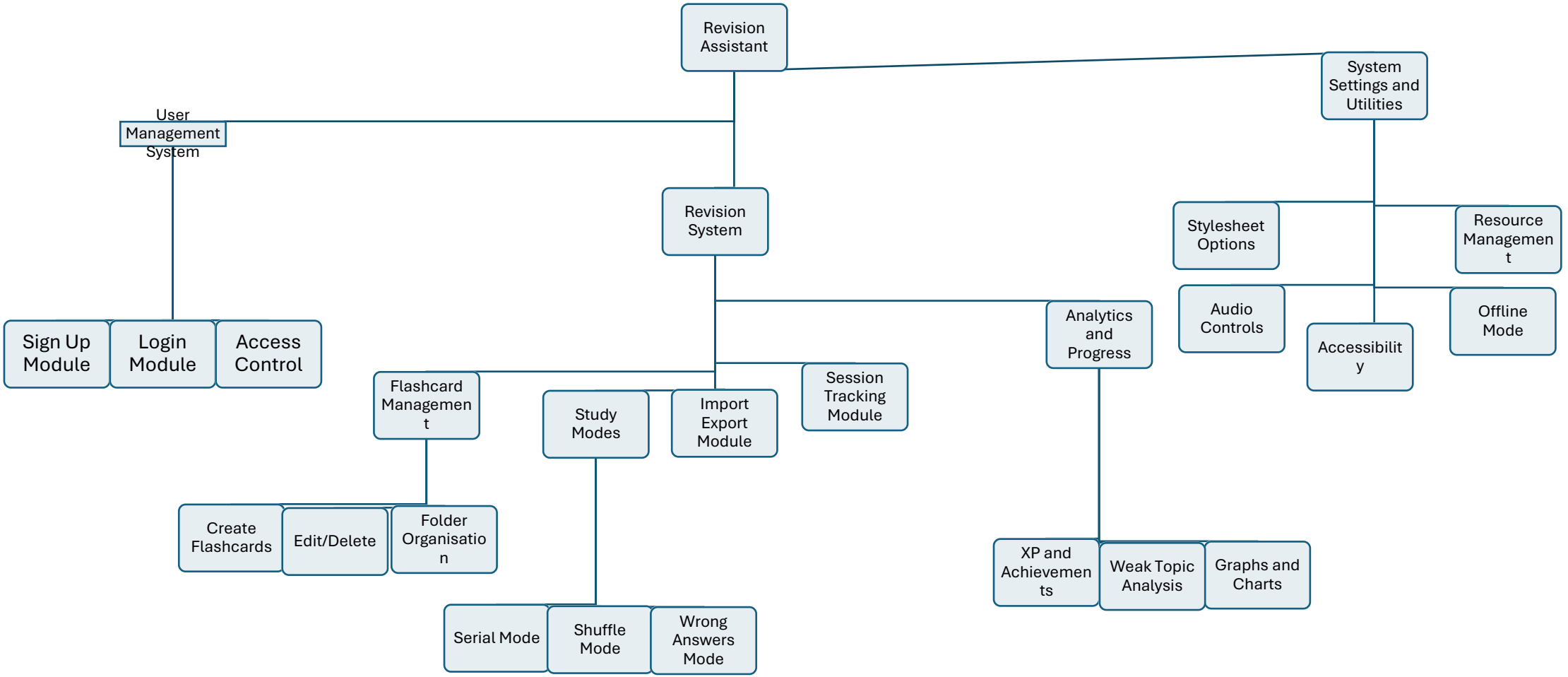
must have volume sliders so sound effects can be disabled completed or have their volume change.

i. Test by completing tasks at different volume levels to see whether sound effects can be minimalised.

12) The program must have folder organisation systems, allowing the user to pick which folders flashcard decks go in.
    - Students struggle with organising their study materials, if there is no way of sorting their study within the app, it might be overwhelming and reduce motivation to keep revising.
        i. Test by creating, modifying and deleting flashcards in folders must behave correctly.
13) The solution must contain a valid text-to-speech feature.
    - Some users may have visual impairments, dyslexia, or some may struggle with reading continuous amounts of information. With the text-to-speech feature, revision would be inclusive and would allow students with concentration difficulties to retain and process information in another way.
        i. Test by enabling TTS to be used in flashcards and it must read out correctly in 95% of the time.
14) The solution must provide advanced analytics.
    - With advanced analytics such as graphs, or charts; students can identify their weaknesses and have an insight on where to improve. This will help improve their knowledge for the long term. The data for analytics will need to be saved to an external file
        i. Test by completing test sessions and compare analytics with raw results, the data must match 95% of the time.
15) The system must allow the user to create flashcard decks, full of flashcards, and allow the modification specific flashcards or the whole deck post-creation
    - Without allowing the user to create flashcards, the solution is almost pointless since this is a core function of the system. Revision is also an iterative process; hence material might change over time, this will allow users to edit decks and add to them.
        i. Test by allowing creating a deck, and then modifying content within it, renaming it and deleting it.

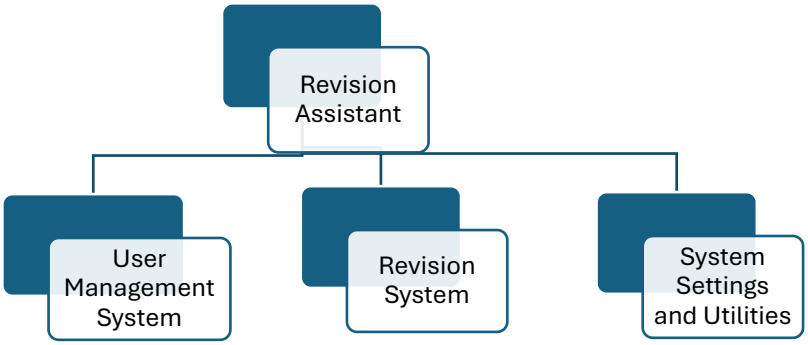
Section 2 Design

2.1 Decomposition



2.1.1 Stage 1

The **Revision Assistant** (the problem) has been broken down into three core sub-problems. Stage 1 focuses on recognising the main functional areas of the application which will help achieve the program's ultimate goal: providing a structured, efficient, and motivating revision platform for students.

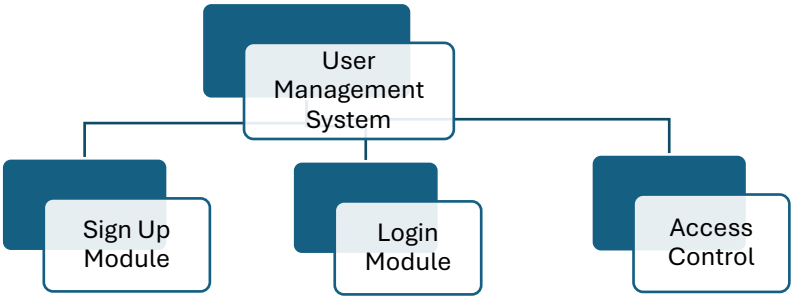


1) User Management System
    - This will handle everything related to user profile such as creation, logging in and access rights of accounts. Ensuring each user's data is stored separately and securely, enabling multiple users to use the application on the same computer without interference
2) Revision System
    - Core of the application managing flashcards, study modes, progress tracking and analytics. Allows students to revise with imported/created flashcards and monitor progress efficiently as well as effectively, combining scientifically proven study methods with gamification to improve knowledge retention and motivation to revise.
3) System Setting and Utilities
    - Subsystem designed to control accessibility and usability features, focusing on enhancing user experience and inclusivity by introducing personalisation; used to aid students who have learning needs or preferences.

I have identified these as the three main areas because they represent the fundamental components of the problem which are required for efficient operation. The problem being broken down this way allows for modular design as each subproblem is made to perform an independent but related role in the application.

2.1.2 Stage 2

User management has been split further into smaller modules which are intended to handle all user access, data storage and security issues. This ensures that the login process is solvable algorithmically, easy to test and modular.

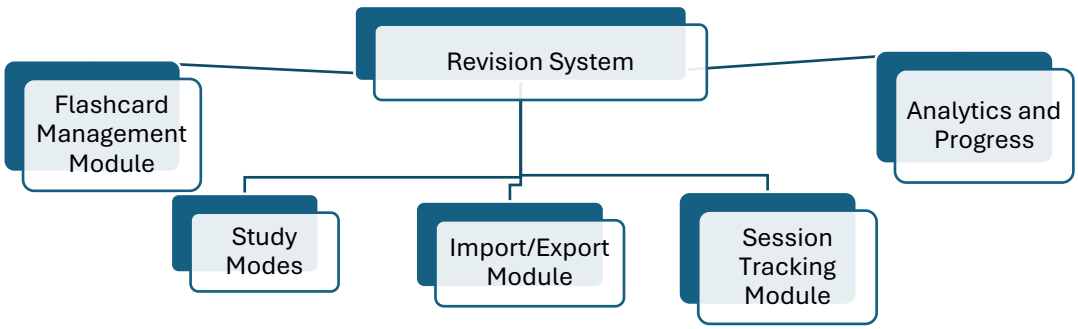


1) Sign Up Module
    - Allows for new users to create local accounts by setting required data (i.e. username, password), which will be securely stored in files on the secondary storage. This will enable multiple students to use the same device but have their information separate from other users.
2) Login Module
    - Allows existing users to sign into their local accounts by entering data which will be checked against stored data. Without this feature, user specific data would be mixed between multiple users, which would not be personalised for a user; hence it is mandatory for keeping data secure between users/accounts.
3) Access Control
    - Will be used to manage data privacy and protection between users by restricting access to each account's individual files. This will isolate user data from unauthorised users, preventing students from viewing or modifying another student's flashcards.

The modules are split up in this way since they all involve different functionality, as stated above. Through the splitting up of the functionality, the program can be easily expanded in the future to have additional features such as multi-user profiles, without affecting the login system's stability.

2.1.3 Stage 3

The **Revision System** has been decomposed further into function modules, each which are responsible for a singular revision related process to cater to easier development and testing.

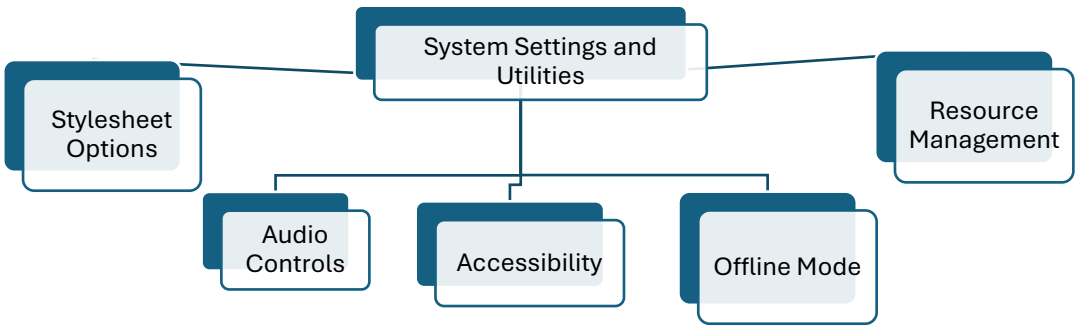


1) Flashcard Management Module
    - Will allow users to create, edit or delete flashcards and then organise them into further folders by subject (or by their choice). This is the main aspect of the system, users need to be able to create and organise their own study content, ensuring revision will not be too overwhelming.
2) Study Modes
    - This will add different ways of revising into app such as shuffle, or wrong answers only. Multiple study modes allow the support of different learning styles, encouraging active recall and making revision more personalised.
3) Import/Export Module
    - Will allow for the importation of flashcards in CSV form and exportation to CSV or PDF to use when not able to access their computer. This feature will allow for teachers to set students what to revise and will also save time.
4) Session Tracking Module
    - This aspect will record how long the revision sessions will last and will display the time spent revision. This will be essential to give students information of their study habits, helping them integrate revision into their schedules perfectly.
5) Progress Tracking and Analytics Module
    - Designed to log user performances int terms of XP (for instance) and will present data to student in chart/summary form. This visual feedback will assist students in locating areas of weaknesses and motivating students by letting them see the progress they have made.

This module is associated with directly associated with learning – the core of the solution, by separating the revision features from settings and the user management modules, the program groups together different revision related tasks, supporting maintenance and scalability.

2.1.4 Stage 4

The **System Settings and Utilities** module has been decomposed into smaller sub-problems that will manage user preferences, accessibility and performance, which is essential to ensure the program will be accessible as well customisable for all users

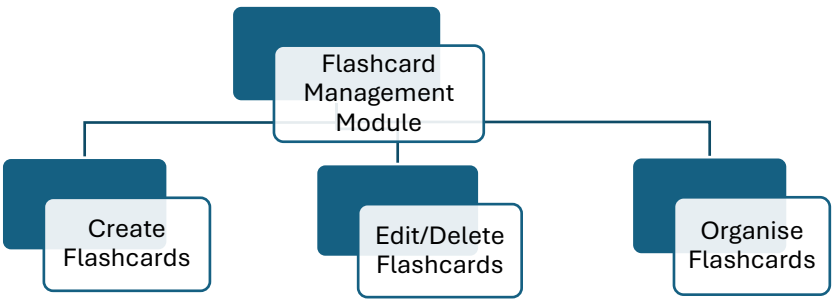


1) Stylesheet Options
    - Will allow users to switch between different themes (such as light/dark) to suit their needs. This will accommodate for people with eye conditions and will also help in decreasing eye strain during long revision sessions.
2) Audio Control Module
    - Provides volume sliders and options circling sound effects. While allowing students to disable distracting sounds, it also aims to enhance engagement by letting students personalise their revision.
3) Accessibility Module
    - This will allow students to modify text-to-speech settings, catering for users with dyslexia, visual impairments or people who struggle to absorb large amounts of knowledge easily. The accessibility module will ensure inclusivity by providing a different learning option.
4) Offline Mode Module
    - This module will let students utilise the system fully without an internet connection, since all data is stored locally. Many students may want to focus completely on revision so may turn off their internet connection to prevent distractions.
5) Resource Management Module
    - This will ensure the Revision Assistant will be making efficient use of computer resources for smooth performance, no matter the device. Allows for school or budget laptops to use the application without lag or other issues.

Since the module is responsible for managing user preferences, it has been separated from the revision and user management as its role is to ensure a customisable user experience. Isolating utilities and settings from other processes will allow easier optimisation and debugging, while enhancing system performance and resource efficiency.

2.1.5 Stage 5

The **Flashcard Management Module**, the foundation of the Revision Assistant, has been broken down into submodules which will handle the creation, modification and organisation of flashcards.

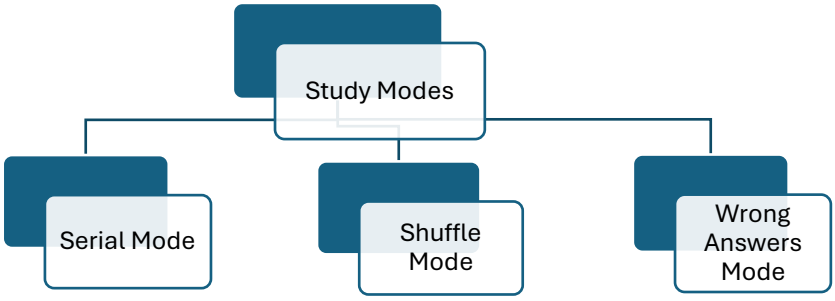


1) Create and Save Flashcards
    - Allows users to input question/key word and answer to generate flashcard which will be stored into local storage immediately. Ensuring flashcard data is saved as soon as it is created will prevent the loss of any data and preserves user progress immediately.
2) Edit/Delete Flashcards
    - Enables users to modify flashcards (existing) or remove unnecessary ones. This feature will ensure efficient management of flashcards and prevents outdated flashcards from cluttering.
3) Organising Flashcards
    - Allows the user to select what flashcards go into what groups (hierarchical model) which can also be exported. This improves navigation as well as will make sure revision will not become too overwhelming since it aims to reduce cognitive load by allowing students to focus on specific content areas.

I have grouped these functions together since they are associated with how users will control their revision material. The separation between flashcard management and other revision supports data integrity as all changes to content are managed through a single process, reducing the chances of errors.

2.1.6 Stage 6

The **Study Modes** module has been broken down further into modules of separate functionality to ensure that each study mode can work independently and is open to expansion in the future, as well as easy debugging and reusability.



1) Serial Mode
    - This will present flashcards in the order which they were created or stored, supporting learners who prefer structured revision sessions; ideally for reviewing content in a logical or chronological sequence.
2) Shuffle Mode

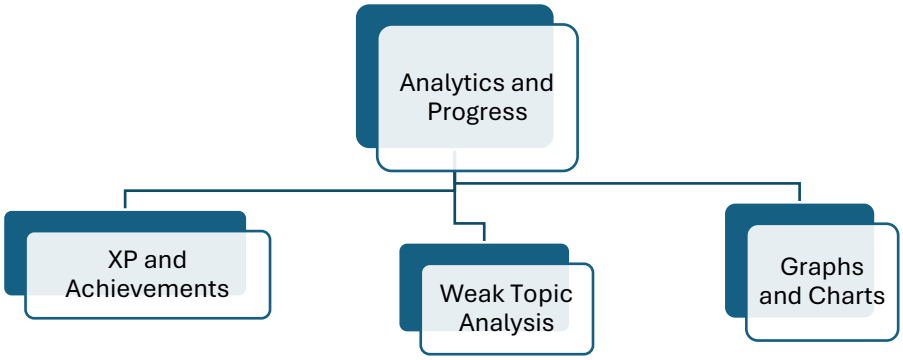
- The flashcards can be put into a randomised order to be used to learn from. This mode will prevent students from memorising the sequence and instead encouraging active recall by testing the user unpredictably.

3) Wrong Answers Mode
   - This feature will only display previously incorrect flashcards until they are answered correctly, ensuring efficiency as time will not be wasted on mastered content- areas of uncertainty can be improved.

This subsystem is responsible for ways users can interact with flashcards (when memorising). I have separated this module from the flashcard management system as its purpose differs from data creation/storage, aiming to increase personalisation and accessibility. Furthermore, by placing these algorithms in one module will allow easier maintenance as well as better performance efficiency.

2.1.7 Stage 7

The **Analytics and Progress** module has been decomposed into three smaller submodules that will aim to ensure that students' progress is monitored accurately and represented visually, which should be used to increase motivation levels.



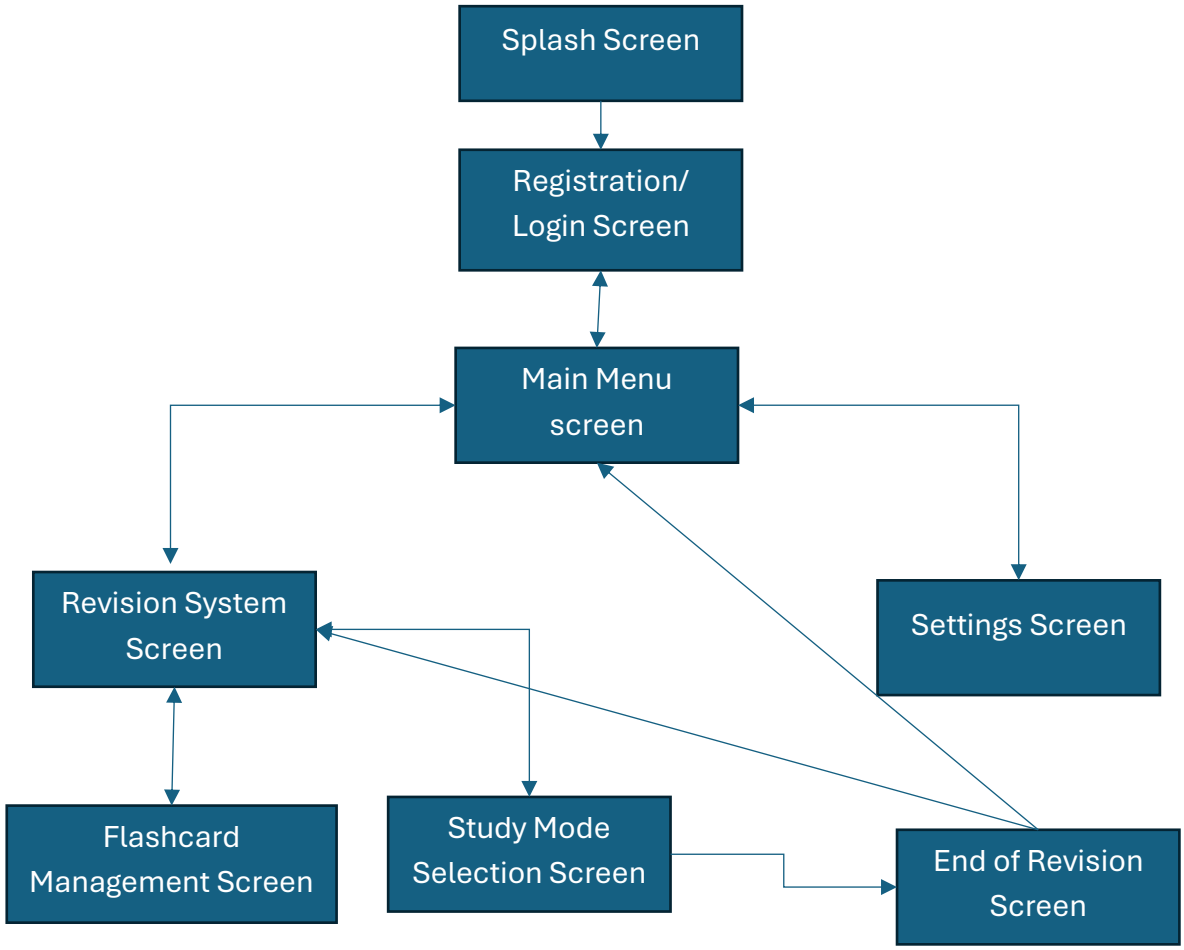
1) XP and Achievements
   - This focusses on the gamification of revision, creating objectives for users to reach to maintain engagement. Assigning experience points for study sessions, correct answers, or streaks, and award achievements for milestones. This will give the user a sense of accomplishment, whilst encouraging efficiency, to keep the user motivated to revise.
2) Weak Topic Analysis
   - This will analyse user performance when revising through flashcards, or generated quizzes to help identify areas with lower accuracy rates. Through highlighting these areas, the user will be provided with more targeted learning so they can master where they struggle, increasing efficiency of studying.
3) Graphs and Charts

- This module will visually represent the user data using progress bars, line graphs and pie charts. This makes progress easy to understand and helps maintain motivation through visible feedback.

The Analytics and Progress module has been decomposed into these three subproblems to ensure data is processed, interpreted and presented in a way to benefit the user's learning experience. This decomposition improves maintainability as each submodule can be enhanced independently. It also follows modular design, ensuring that motivation analysis, and visualisation can be tested and extended for future versions easily.

2.2 Interface

2.2.1 Interface Flow Diagram

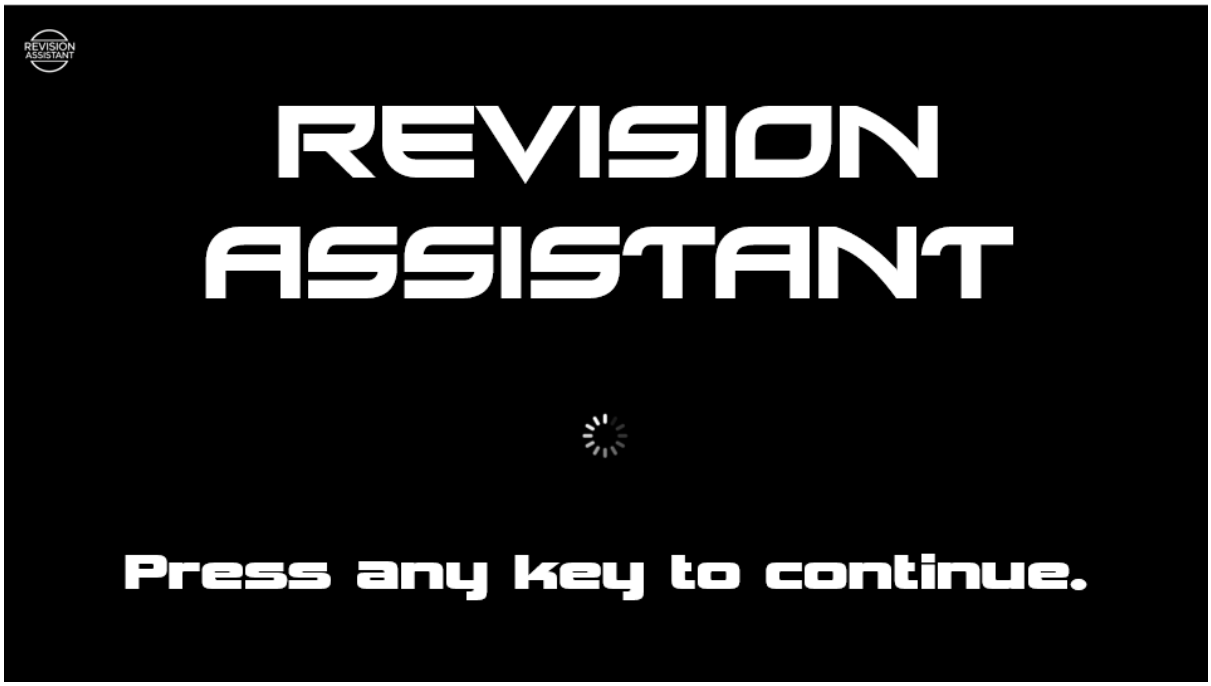


The flow diagram displays the logical navigation of the solution. The Splash Screen creates a polished first impression of the system whilst loading background processes. After the Splash Screen, the user would see the Registration/Login Screen which separates user profiles to allow individual tracking of data. After registering or logging in, the user is taken to the main menu which is used to centralise navigation and had

options to go the Revision System Screen (focussing on core program functionality: studying and progress tracking); the Settings Screen (allowing users to personalise their experiences) and the About Screen (providing software and developer information). The Revision System Screen connects to the Flashcard Management Screen (where users can organise, modify or create their flashcards), as well as the Study Mode Selection Screen allowing users to select their preferred study mode such as wrong answers only. After completing revision from the study mode, the user is directed to the End of Revision Screen (where the performance metrics are displayed such as XP gained, total time, etc) and has the option to go back to the Revision System or the Main Menu.

2.2.2 Interface Mock-Up

2.2.2.1 Splash Screen



The Splash Screen is the introduction to the application for the user and acts as the entry gate. It displays the system's name and prompts the user to begin interaction to proceed to the login/registration screen.

Usability Features specific to Screen:

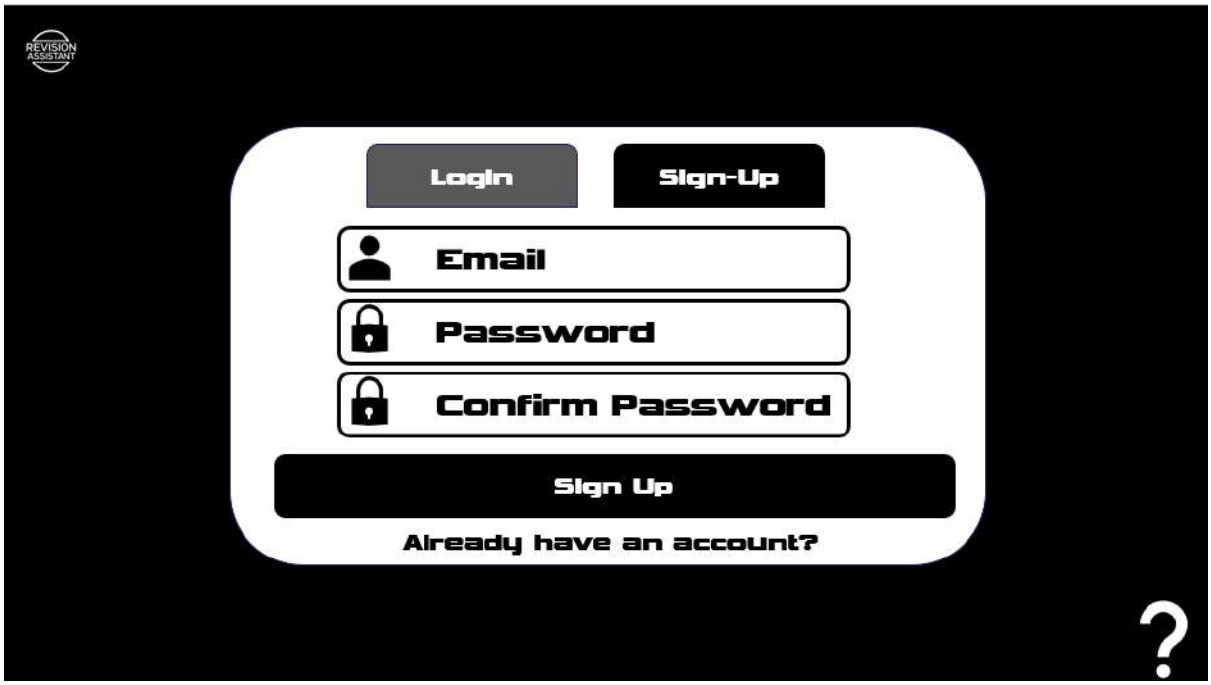
- Interaction Prompt
  - Ensures user is engaging with the application prior to loading, preventing accidental launching or background usage.
- Minimal text and clean background
  - Reduces visual clutter and cognitive load thus reducing frustration and improving ease of use for users with additional needs such as autism.

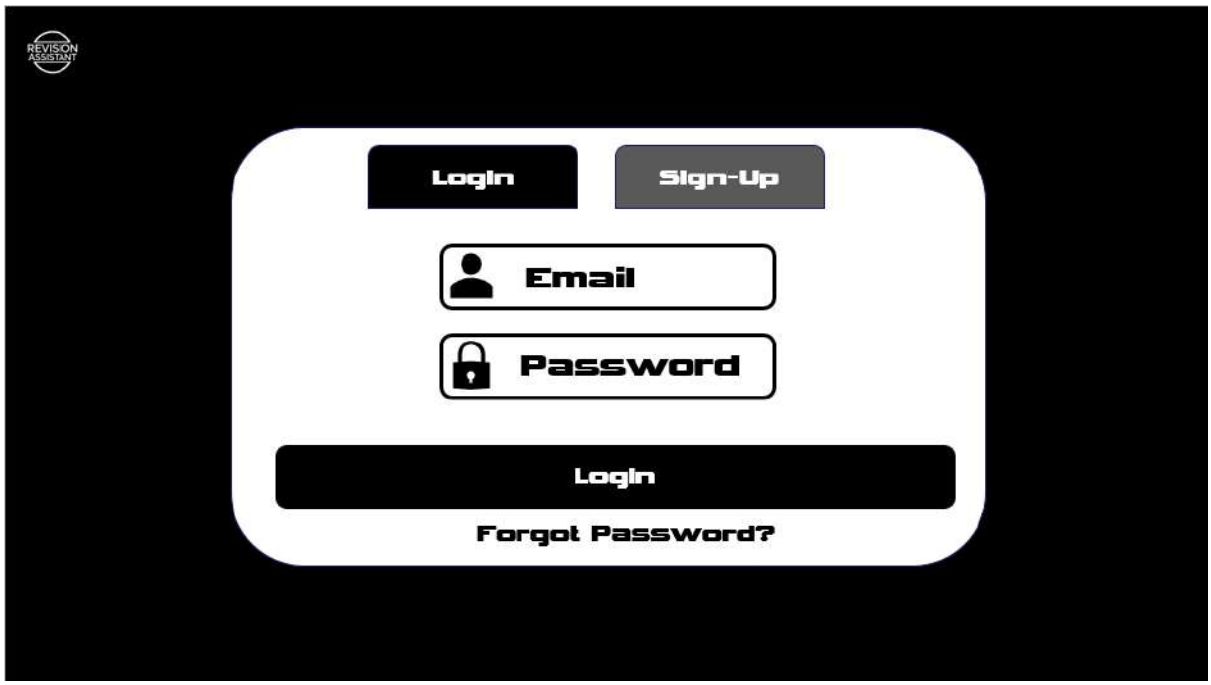
Alternative:

- The application could automatically transition to the registration/login screen after a set time.
  - Even though automated loading is faster, requiring a manual press reassures the user is in control of what they do from the beginning.

This design will meet the user's needs as by providing a low effort command that confirms the application is loaded and responsive, the user will control the pace of the interface launch without frustration, supporting efficiency.

2.2.2.2 Registration / Login Screen





This dual-purpose screen breaks down authentication into logging in and signing up, allowing new users to register, as well as existing users to log in to access their locally stored data and progress.

Usability Features specific to Screen:

- The tabs visually separate log in and sign up, as well as difference in colour defining what tab you are on.
    - This will prevent confusion between returning and new users, the difference in colour ensures users can differentiate between sections,

reducing input errors. This is beneficial for younger users or those with autism or ADHD, who may struggle with ambiguous layouts.

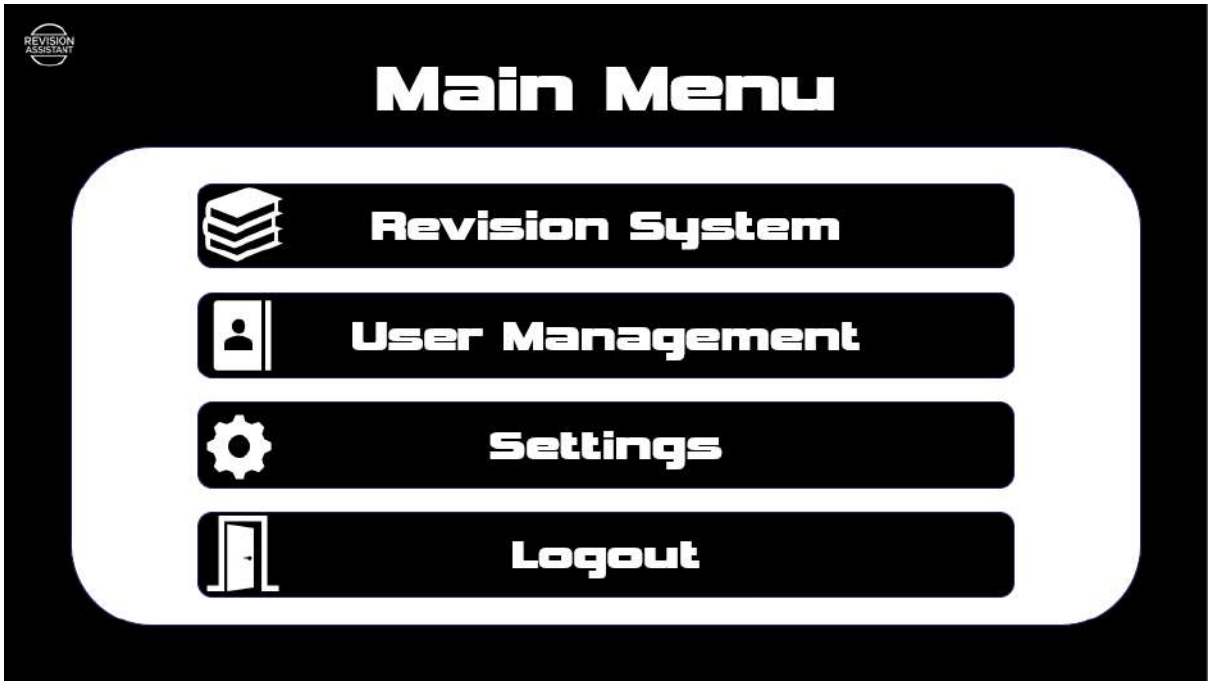
- “Forgot Password”.
    - Provides an easy and accessible way to recover your account, reducing frustration. It is predictably placed underneath the password field, which is common in many interfaces, this makes it easy to be recognised for users with memory-related difficulties such as dyslexia.
- “Confirm Password”.
    - Allows user enables real-time validation, minimising user input errors. This will benefit users with attention difficulties, since they are more likely to forget their original input, or mistype a password.

Alternative:

- The application could use an adaptive login form, for example, when the email is written in to the text field but not recognised in the database, an additional field of "Confirm Password" is added allowing the user to create a new account.
    - This would require extra validation, introducing unnecessary complexity. This method could also cause confusion leading to multiple accounts created by the same user.

This design meets the user's needs by providing distinct pathways, and visual indicators for entry into the system, without unnecessary complexity which caters for less tech-savvy students.

2.2.2.3 Main Menu



The Main Menu screen acts as the central navigation hub of the application, providing entry to all major components: the Revision System, User Management and Settings.

Usability Features specific to Screen:

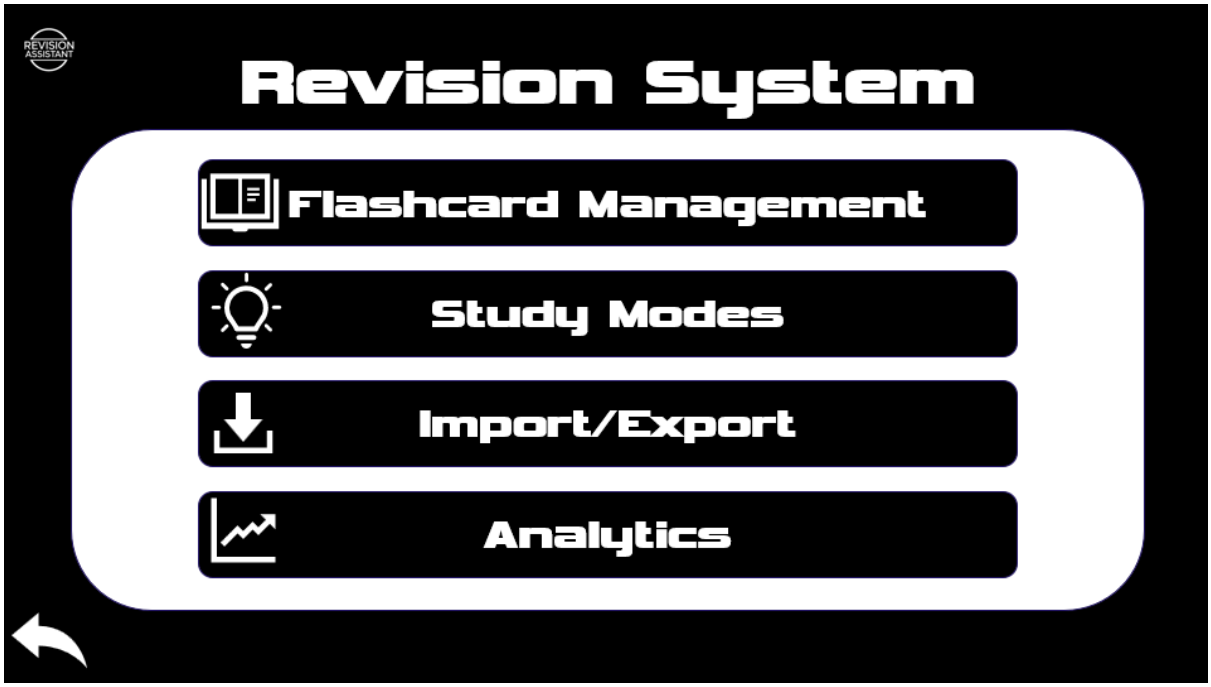
- Logout Option.
    - Separated from the other management options, providing a secure exit from the system. Logout is the last option to prevent accidental logout, reducing frustration. Furthermore, the larger separation from other buttons helps users with motor difficulties who can click wrong options if buttons are close together.

Alternative:

- The main screens of the system could have used a collapsible or side-bar menu.
    - I will make the use of collapsible menus in other cases, however in this case, presenting all possible options clearly to the user reduces number of clicks needed to access the core of the system. This encourages quicker decision making and ensures better focus.

This design meets the user's needs by creating a predictable, clearly defined structure which allows users to navigate through the core functionality of the system quickly, minimising wasted time and cognitive load.

2.2.2.4 Revision System



The Revision System screen is dedicated to managing study materials, initiating study sessions and analysing progress.

Usability Features specific to Screen:

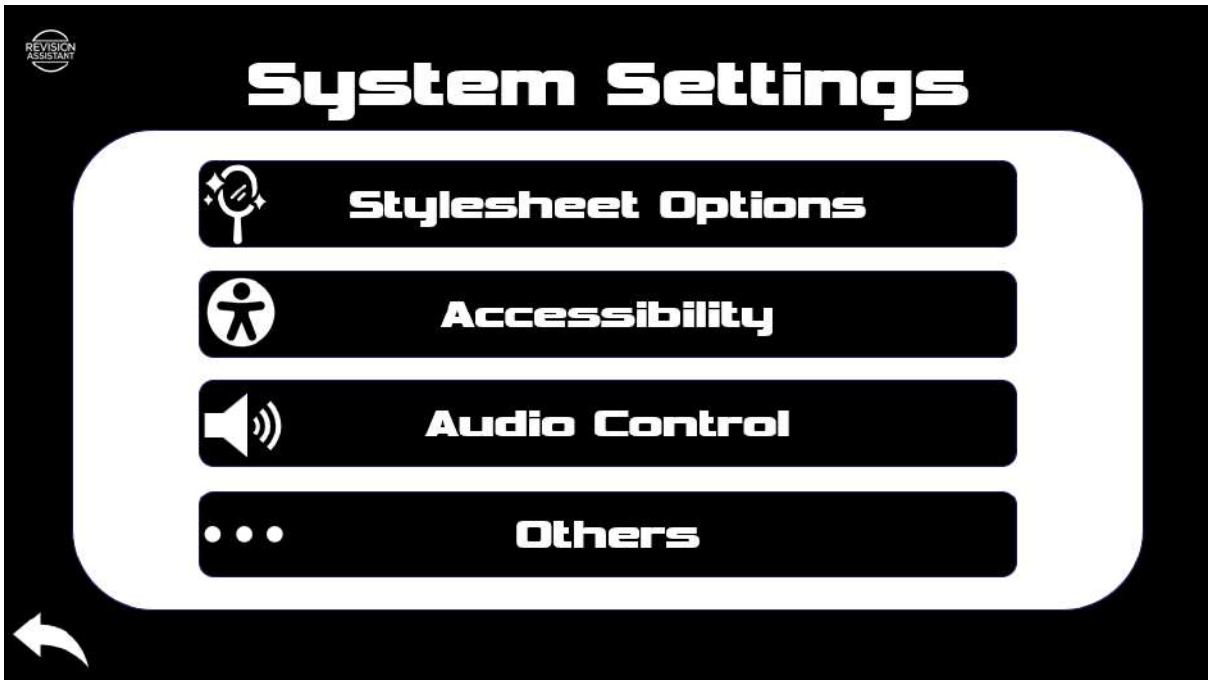
- Breakdown of functional groups.
    - This reduces cognitive load, preventing confusion between unrelated tasks, for example selecting a study mode versus creating revision material, since it allows for the user to engage with one sub-problem at a time. Supporting users with autism, ADHD, or anxiety who can be overwhelmed with visual clutter.

Alternative:

- A collapsible drop-down menu.
    - This would hide functionality, increasing navigation time. Therefore, by displaying all options directly, visibility and user engagement is consistently maintained.

This meets the user's needs by logically grouping together related tasks, reducing complexity of the revision system by ensuring all options are easily accessible.

2.2.2.5 System Settings



The System Settings screen centralises all configuration controls for user experience; the user can customise the theme of the application, volume, accessibility and other settings to match their preferences.

Usability Features specific to Screen:

- Volume Options
    - Allows users (particularly with sensory processing sensitivities, autism, or ADHD) to turn off sound effects or other background audios which can be seen as overstimulating and/or distracting. This can also be benefitting

towards user using the application in a public space such as a library, ensuring control over their experience and comfort for all users.

- Stylesheet changes possible.
    - Accommodates for people with preferences or learning needs in terms of the design of the web page. For example, students with dyslexia may prefer high-contrast colour schemes, while other users may benefit from dark mode. This enhances personalisation and comfort, supporting users for longer, more productive revision sessions.

Alternative:

- The settings could have every option laid out in a list.
    - This would be more difficult to navigate, as with no categorisation there will be less clarity for the user, making them search linearly for their wanted option.

This meets the user's needs by centralising and categorising all personalisation options, allowing the user to easily adapt the environment to their needs, for example a different stylesheet.

2.2.2.6 Flashcard Management



The Flashcard Management screen focusses directly on the creation, modification and the organisation of study material.

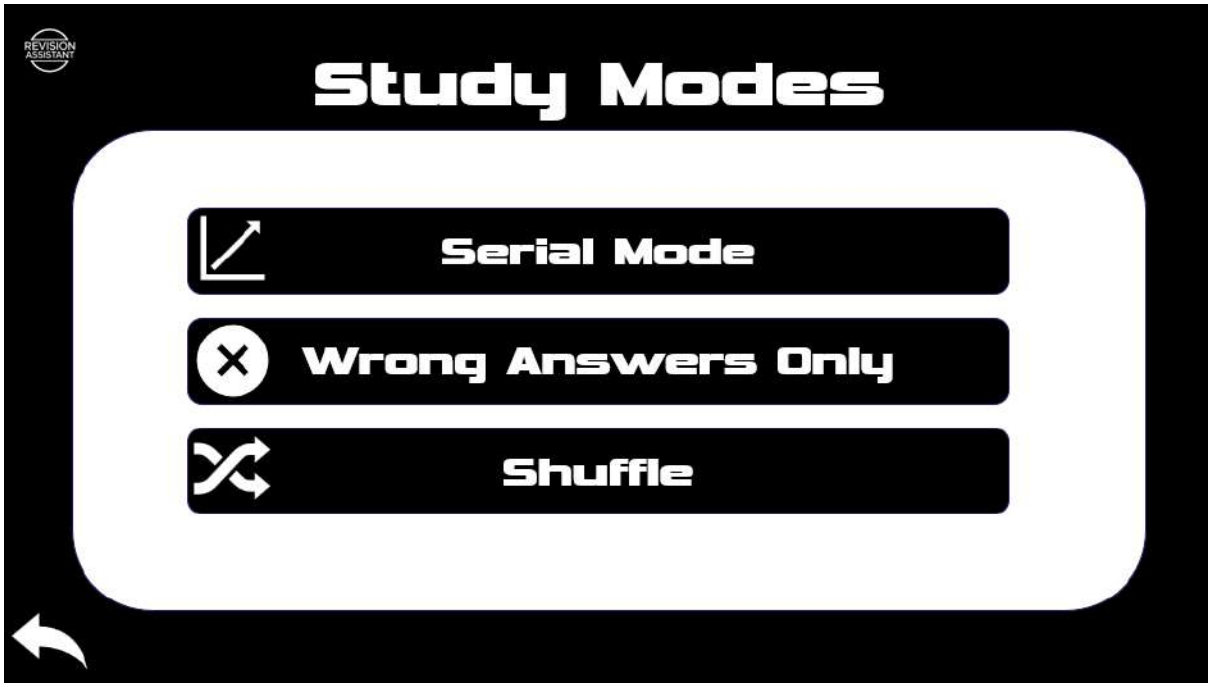
Alternative:

- Embedding all actions within the creation of flashcards.

- This would reduce number of clicks needed but can clutter the screen; through grouping related tasks, the workspace remains clean and not overwhelming.

This meets the user's needs as the functionality is separated, so when users are creating flashcards, they will not be distracted by different functions such as search. This maximises the efficiency of content management.

2.2.2.7 Study Modes



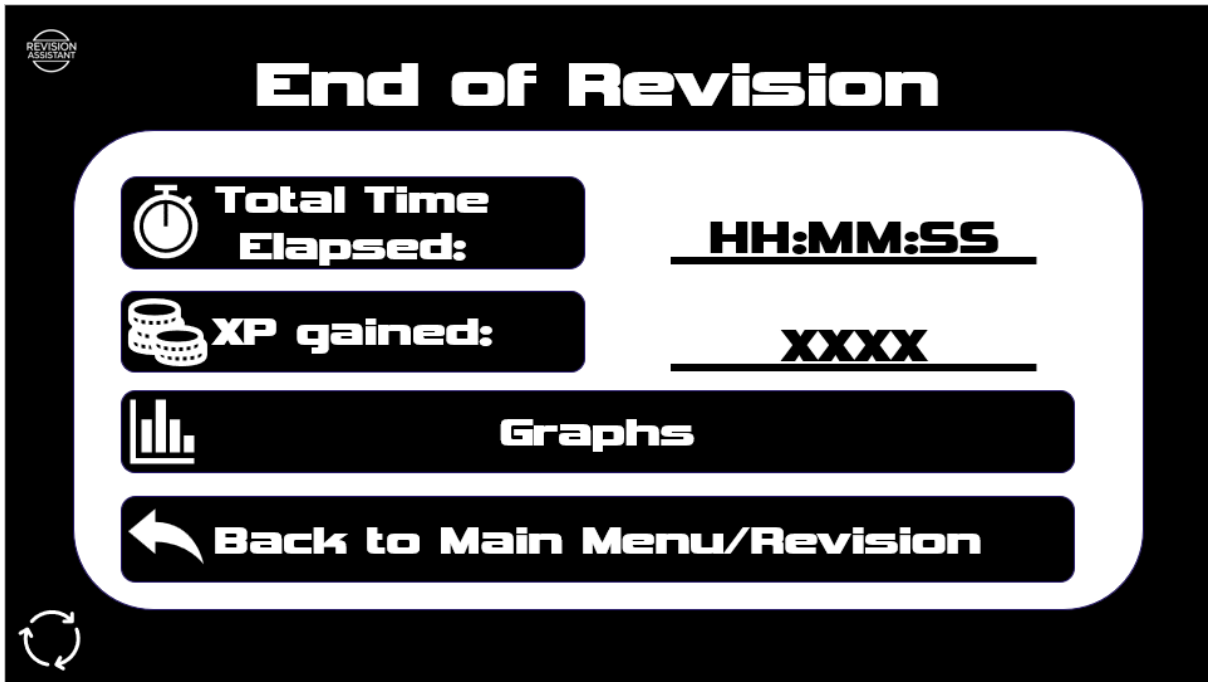
The Study Modes screen presents various algorithms available for a study session, allowing the user to pick their preferred method.

Alternative:

- Radio buttons.
  - These buttons only allow one selection; however, it requires more text and can clutter the screen. Larger buttons are more visible at first glance.

This meets the user's needs by offering multiple revision methods that maximise learning, ensuring inclusivity for all learning types.

2.2.2.8 End of Revision



The End of Revision screen displays post-revision performance metrics such as XP gained, total revision time, and analytical graphs.

Usability Features specific to Screen:

- Colours connoting performance.
    - Colours will be used define progress without the need of additional test, saving time. Making the use of familiar Western cultural colour associations (green = good, red = bad). The system promotes progress awareness and engagement, which is beneficial for students with short attention spans, or lower confidence levels.
- Retry Option
    - If the user is not happy with their performance, they have a chance to retry the same questions to improve. Supporting users aiming for mastery and helps build confidence by giving users an immediate opportunity to improve. This can reduce frustration for users with perfectionist tendencies, giving them a sense of control after poor control.

Alternative:

- Automatic redirect to home/revision system screen.
    - Removes reflection time, with manual navigation, the user remains in control.

This meets the user's needs by delivering comprehensive and motivational metrics, allowing the user to assess their performance.

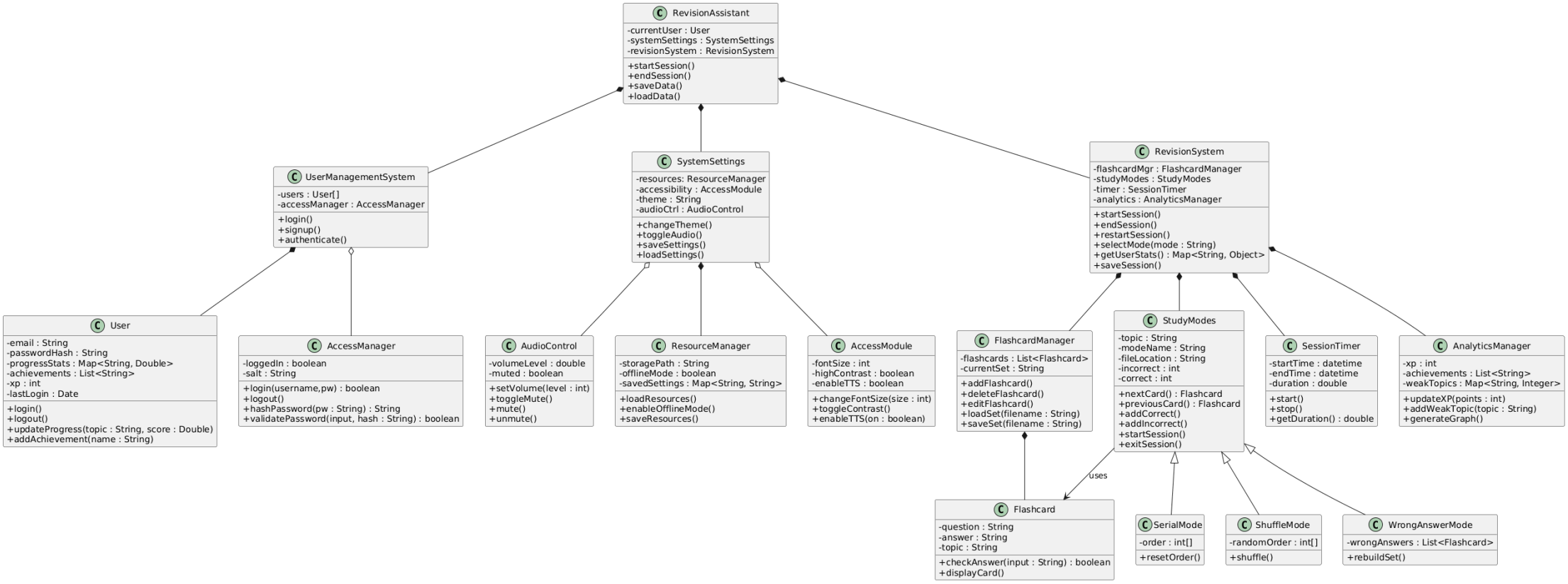
2.2.2.9 Summary of Usability Features consistent throughout Screens

Usability Feature	Caters For	Justification
Consistent Layout and Structure	Users with autism, and/or anxiety.	With consistency, the screen's layouts become predictable as well as navigation, reducing confusion and cognitive load. This allows students to focus on revision, instead of locating features.
High Contrast Colour Scheme	Users with visual impairments, and/or dyslexia.	This will improve readability and prevents visual strain when using the application for long revision sessions. This also allows for elements to be emphasises such as important options.
Simple, descriptive buttons.	Users with cognitive disabilities.	Ensures the users can understand each action clearly, for example "Study Modes" instead of vague labels. This will reduce mis clicks, reducing frustration.
Larger clickable area	Users with motor disabilities.	With larger buttons / areas to interact with, the need for precision is reduced, making it easier for users with motor or other physical disabilities to select options.
Consistent Icons	All users.	Accompanying text with icons, aligns with knowledge in the head and improves readability. This can improve speed and accuracy when navigating through the system, removing the need of relying on text
Visual Feedback	All users, or users with anxiety.	Colours aligning with Western cultural colour associations will give the user immediate visual feedback on performance and stays consistent throughout screens.
Lack of superfluous animations.	Users with epilepsy, and/or autism.	The lack of animations within the system will allow the main focus to stay on revision, preventing sensory overload and distractions.
Tooltips	First time users, or dyslexia.	There will be options to offer addition explanations without cluttering the interface. This will help users understand the interface's functions, for example, tooltips on password criteria.
Confirmation Popups	Users with anxiety, or all learners.	This will reduce errors by reassuring the users before performing irreversible actions such as deleting flashcards.
Spacing between clickable elements	Users with motor disabilities.	Prevents accidental mis clicks and improves usability.
Affordance	All users.	Buttons will visually indicate interactivity (e.g. highlighted when hovered over), showing they can be clicked. This will reduce uncertainty for inexperienced users.

Left-Right icon alignment	All users.	Icons are position according to reading flow in the Western world, this allows for comprehension from users when selecting options rather than having to read the description of each button. This increases the speed of navigation.
Position of “Back” button at bottom left of page	All users.	This mirrors the common interface conventions (bottom left for return actions). This will improve predictability as well as muscle memory for quicker navigation.
Centralisation of text/options	Users with cognitive and/or visual impairments.	Buttons and text are placed centrally so they are immediately visible to users, reducing the need for eye movement and cognitive effort. This makes the interface feel organised and less intimidating.
Grouping of related functions	Users with cognitive difficulties and/or ADHD.	This will reduce mental effort by structuring related actions such as create, edit and deleting flashcards together, supporting efficiency.
Hierarchical Navigation Flow	Users with dyslexia, cognitive impairments, and or screen readers.	By placing the exit/logout action as the last option, accidental exits will be prevented, and it will provide a predictable navigation order.
Minimal use of heavy text	Users with dyslexia, and/or ADHD.	Prevents the user from being overloaded with information which then reduces visual fatigue.
Keyboard Accessibility	Users with motor disabilities.	This will allow users to navigate and interact with the system with a keyboard – without the need of a mouse. This will improve inclusivity and accessibility within the application.

2.3 System Design

2.3.1 Class Diagram



The design of the system is object oriented in order to allow modularity, reusability and maintainability. The decomposition of the system (2.1 Decomposition), has been used to split the solution into distinct classes, for example: UserManagementSystem, SystemSettings, and RevisionSystem. These classes represent real-world entities and address separate subproblems, improving the scalability of the program as well as the ability to debug efficiently.

Each class encapsulates its own attributes and methods, making sure modules are independent, hence changes in one module will have little impact on other modules. Encapsulation also increases cohesion within the system, meaning elements are closely related and focussed on a single purpose. Coupling is the measure of the degree of interdependence between the modules. Low coupling combined with high cohesion make the system easier to maintain and improve.

The class diagram for the Revision Assistant follows a Model-View-Controller (MVC) structure, although the system is not a full MVC implementation

The MVC framework is an architectural pattern that separates an application into three main logical components Model, View and Controller. Each component is built to handle specific development aspects of an application.

The Controller component enables the interconnection between the views and the model, so it acts as an intermediary. The Controller does not worry about handling data logic; it just tells the Model what to do. In the Revision Assistant's case the main class RevisionAssistant (and possibly components like AccessManager) acts as the Controller; their responsibilities are receiving user input and interpreting it, updating the Model based on user action and selecting and displaying the appropriate View. The Controller manages program flow and does not store data.

The View component is used for all the interface logic of an application. Views are created by data, which is collected by the Model component, this data is not taken directly but through the Controller. The View only interacts with the controller. In this case, the View is all the user facing screens (stated in 2.2 Interface). They display information to the user and collect inputs (such as button presses, or text inputs). This does not contain any revision logic and just presents data provided by the Model.

The Model component corresponds to all the data-related logic that the user work with, hence classes like Flashcard, FlashcardManager, AnalyticsManager, User and SystemSettings handle all data storage and logic. They store all revision content, track progress and represent all user information. The responsibility of the Model is to manage the creation, reading, updating and deleting of data as well as notifying the View and Controller of state changes.

The MVC structure improves the maintainability of the solution since changes to the interface do not affect the logic classes. This structure increases scalability, allowing future features (such as extension to mobile apps) to reuse the same Model without having to change the logic. Furthermore, as each component has a single responsibility, the solution becomes more robust, reducing the possibility of redundancy of code.

Relationships in the system were chosen carefully:

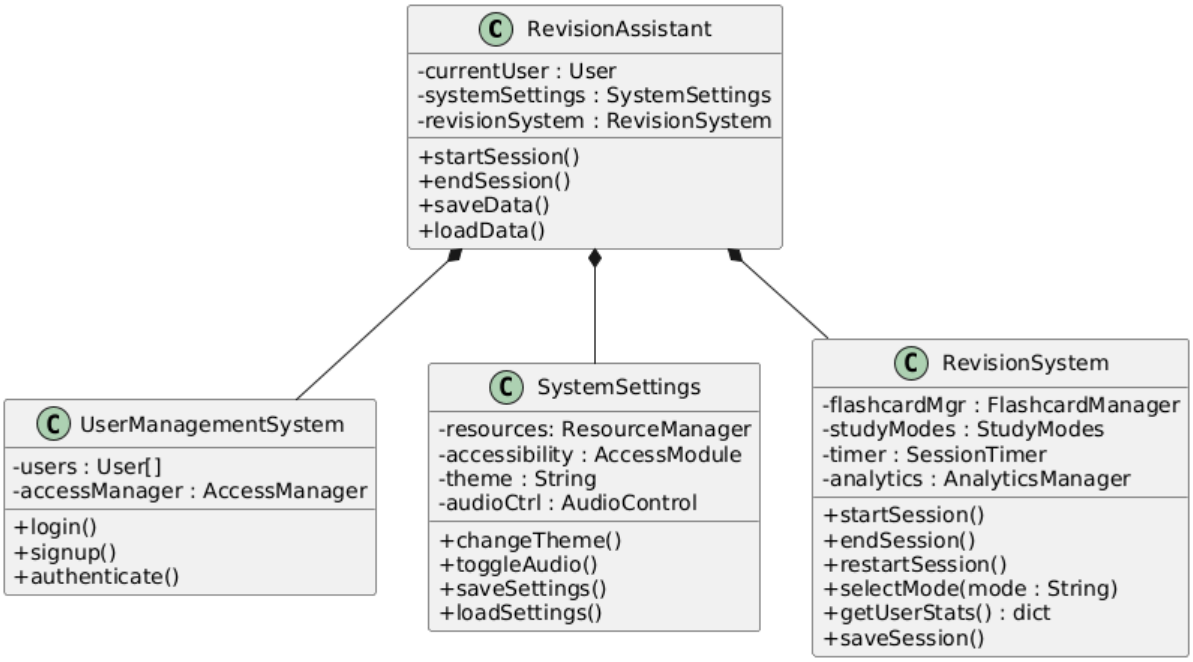
- Association (represented by a solid line)
    - Bi-directional relationship between two classes, indicating the instances of one class is connected to the instances of another class.
        - This system uses directed association for temporary dependencies, such as StudyModes accessing Flashcard.
- Aggregation (represented by a line with an empty diamond)
    - This denotes a stronger relationship where one class contains or is composed of another class. The child class can exist independently of its parent class in this relationship; hence this is used where subsystems collaborate but are reusable.
        - SystemSettings uses an AudioControl instance but does not own its lifestyle.
- Composition (represented by line with a filled diamond)
    - This indicated a more significant ownership or dependency relationship. In composition, if the container object is destroyed, then the contained object is also destroyed. This ensures strict ownership.
        - RevisionAssistant owns UserManagementSystem, so their life cycles are linked.
- Inheritance (represented by an empty arrowhead pointing from subclass to superclass)
    - The subclass (child) takes on all attributes and methods from its superclass (parent), supporting polymorphism.
        - The different study modes inherit from the StudyMode class.

Data Structures were selected for efficiency and simplicity for the system to work in a single user, local environment:

- Lists and arrays are used for storing sequential, indexable data (i.e. users and flashcards)
- Maps are used to store associative data such as tracking progress, or weak topics.
- Primitive data types (int, boolean, double) are used for counters, flags and numerical tracking.

- Strings are users for identifiers and content containing text such as questions and answers.

2.3.1.2 Stage 1



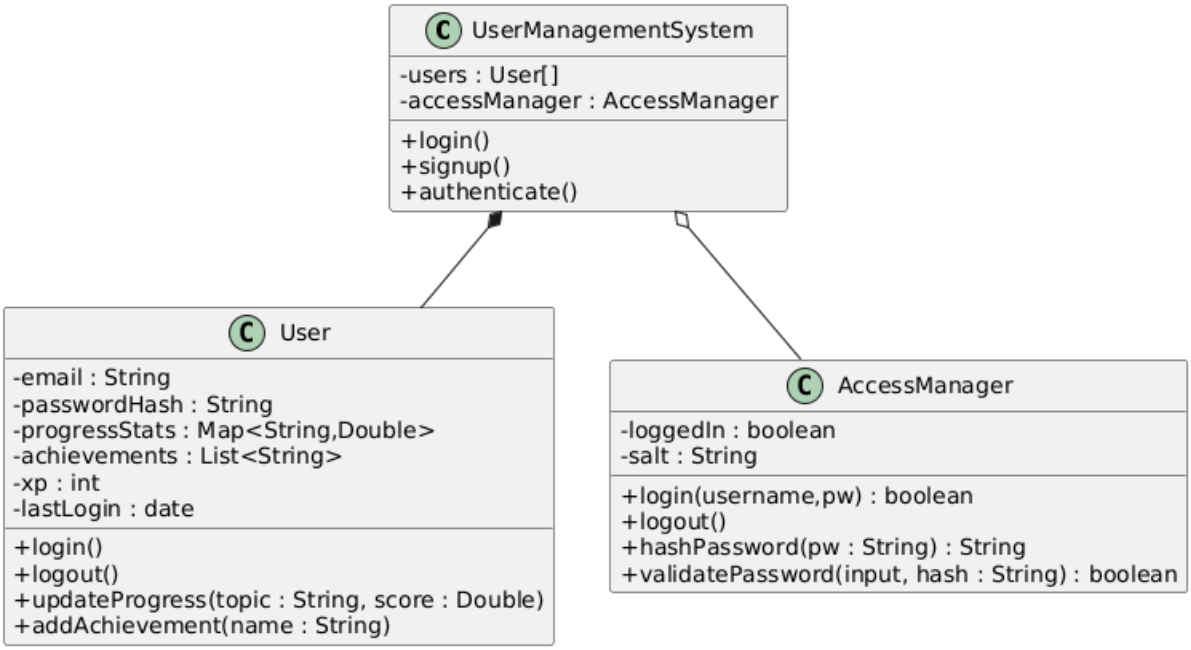
The RevisionAssistant class is the coordinator for all modules, keeping the system organised and preventing any unnecessary dependencies between modules. It tracks what user is currently logged in and shares it with other modules (managing application state). The RevisionAssistant class composes UserManagementSystem, SystemSettings and RevisionSystem; these subsystems cannot function independently of the RevisionAssistant, hence when the main class shuts down, all the subsystems are destroyed.

Data Structures:

- User currentUser
  - Stored as a single instance since only one user can be logged in at a time.
- Subsystems stored as objects
  - Each subsystem is stored as a single object reference because only one instance of each can be running during a program's life cycle.
- Boolean flags
  - These can help manage the runtime behaviour, such as knowing when a session is active. These controls ensure efficient state management and prevent unnecessary object recreation.

This stage is used to ensure the overall program stability and data is saved correctly and accurately.

2.3.1.2 Stage 2



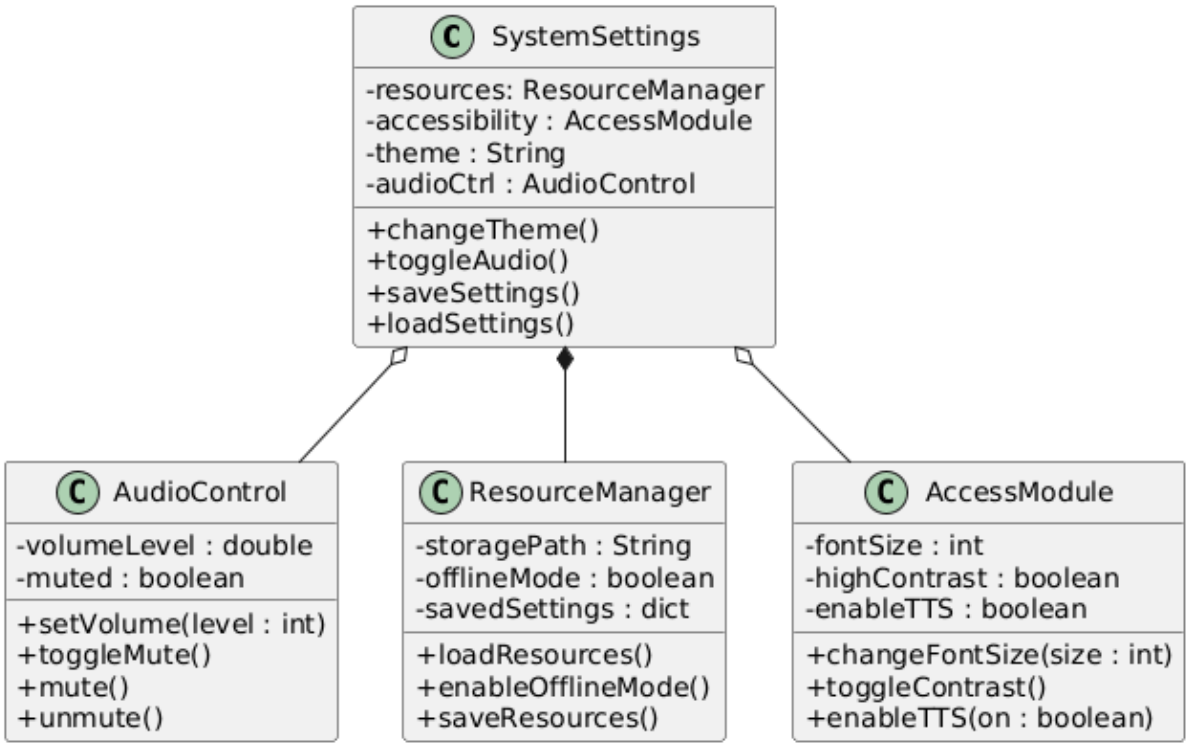
The UserManagementSystem is responsible for responsible for creating, storing and deleting users and their data. Composition between the UserManagementSystem and User, shows that all User objects depend on UserManagementSystem and cannot exist separately. The UserManagementSystem has an aggregation relationship with the AccessManager class, which means it uses AccessManager for password hashing and login logic. However, these features can be reused independently if needed.

Data Structures:

- User[]
  - An array is used to store all user objects for lookup and authentication. Since arrays allow direct access via index, this allows for easy iteration when searching for users.
- progressStats - Map<String, Double>
  - This data structure stores topic names as keys and performance as values, allowing quick look up and updating of progress for specific topics. This is much faster than using a list.
- achievements – List
  - The list is used to store unlocked achievements in the order they were earned. Lists maintain insertion order and allow direct addressing, while queues preserve order. A queue was also not used since achievements are not removed in FIFO order, they are stored permanently.
- int xp and Date lastLogin

- These simple data types are used for lightweight performance tracking and date-based analysis, allowing user to track progress and session activity.

2.3.1.3 Stage 3



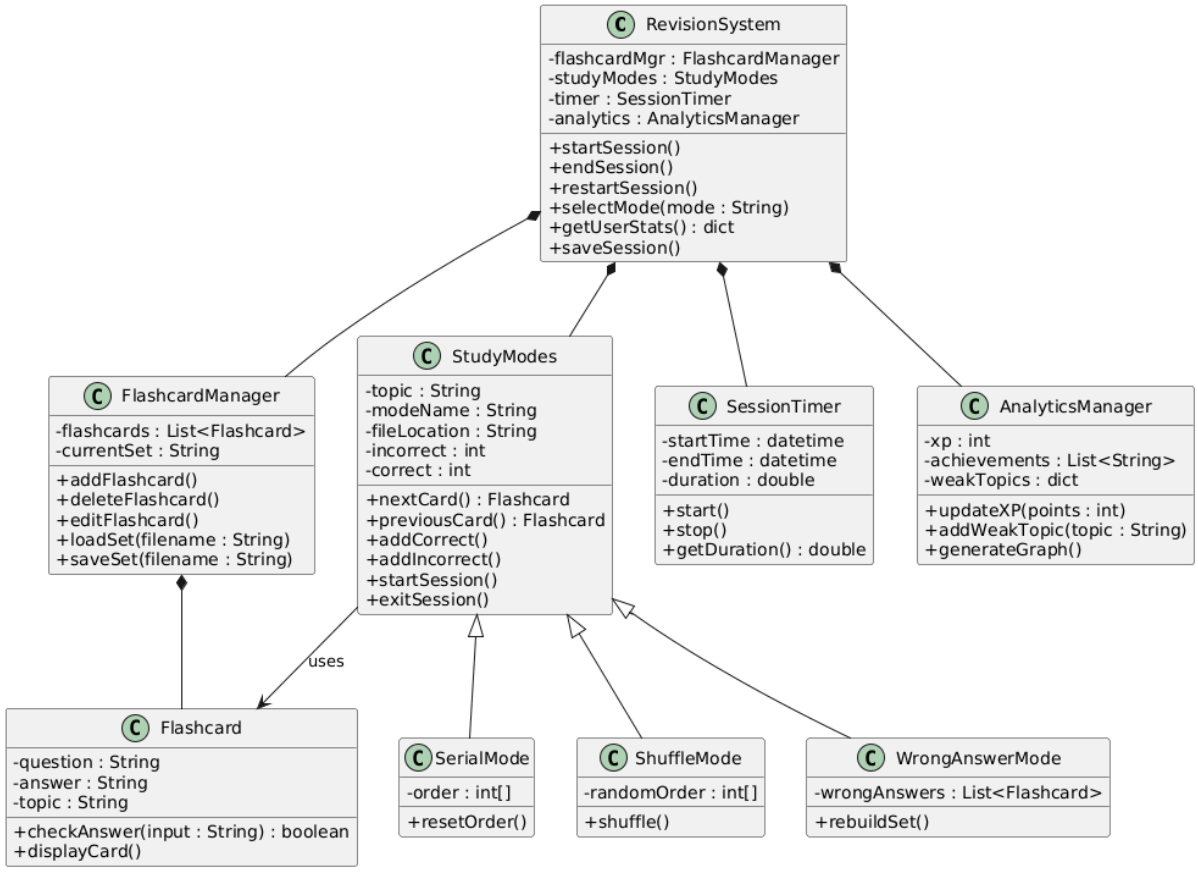
System Settings are divided into separate modules, which each are responsible for a specific configuration. This makes the system more organised, easier to maintain, reuse and update. The SystemSettings composition with ResourceManager ensures settings and stored data cannot exist independently, preventing mismatched or orphaned data, and improving reliability. The aggregation with AudioControl and AccessModule allows these tools to be reused by other parts of the program (e.g. accessibility settings in different scenes), allowing code reuse and consistency across scenes.

Data Structures:

- Strings and Booleans
  - These are used for simple toggle states such as muted. Booleans are ideal for efficient memory use and are easy to check.
- savedSettings – dict
  - Dict allow the storage of flexible, diverse settings in key-value pairs; this allows dynamic storage of different types without creating separate fields for each one. Retrieval is efficient when searching by key.
- Primitive numeric fields

- User adjustable options which require numeric operations (increase/decrease). They are lightweight and allow fast adjustments, and comparisons.

2.3.1.4 Stage 4



This subsystem is the core of the learning functionality within the program. The composition between RevisionSystem and its subsystems means the lifecycle of the subsystems are tied to the RevisionSystem, preventing orphaned data. Inheritance of methods and attributes from StudyModes to the different study modes allows for polymorphism, making code more reusable. Furthermore, more study methods can be added without rewriting the core of the system. The association between StudyModes and Flashcard means that the StudyModes accesses Flashcard objects when needed but doesn't own them. Flashcards can be reused in different modes, with the system avoiding unnecessary duplication of data.

Data Structures:

- List<Flashcard>
  - This stores all the flashcards in the deck. A list is preferred a queue since lists allow direct access via indexes, allowing iteration in any order, shuffling and jumping to a specific card. Queues are first in first out, so are restricted to only processing cards in order.

- Int[] array
    - Arrays provide direct access via indexes as well, with a very low memory overhead. So, this array is used to store the order of questions.
- Map<String, Double>
    - Stores the performance metrics per topic, or flashcard set. Maps allow fast lookup and updating of performance without searching a list.

2.3.2 Algorithms

2.3.2.1 User Authentication

The user authentication algorithm oversees allowing users to log in to an existing account or register a new one; ensuring that all user progress, and data (flashcard sets, settings, etc) is successfully stored separately, maintaining privacy across accounts as well as data integrity. This algorithm is crucial from preventing other users from accessing other students' data.

The algorithm will compare input credential against a local user data file (storing emails and hashed passwords, read in by the authentication module) against file contents to verify login attempts. If the inputted email exists and the password matches, the user is granted access to the system with their data being loaded. If the email exists, but the password is incorrect; the user is prompted with an error message stating that they can either retry or reset the password. If the email does not exist, the user is prompted to create a new account. There will be validation in place to prevent empty inputs, duplicate usernames, etc.

```
1  function authenticate()
2      email = input("Enter Email: ")
3      password = input("Enter Password: ")
4      if email == "" OR password == "" then
5          print("Enter login credentials, they cannot be empty")
6          return "Invalid"
7      endif
8      userDataFile = openRead("users.csv")
9      userLocated = false
10     while NOT userDataFile.endOfFile()
11         line = userDataFile.readLine()
12         userData = line.split(",")
```

```
13                  userEmail = data[0]
14                  userPassword = data[1]
15                  if email == UserEmail then
16                          userLocated = true
17                          if hash(password) == userPassword then
18                                  print("Successful Login")
19                                  userDataFile.close()
20                                  loadData(email)
21                                  return "Success"
22                          else
23                                  print("Incorrect Password")
24                                  userDataFile.close()
25                                  return "Unsuccessful"
26                          endif
27                  endif
28          endwhile
29          userDataFile.close()
30          if userLocated == false then
31                  print("You can either retry, or create an account")
32                  return "Invalid Username"
33          endif
34  endfunction
```

Line	email	password	userLocated	userEmail	userPassword	Output
1	Null	Null	Null	Null	Null	-
2	aditya@gmail.com	Null	Null	Null	Null	Enter Email
3	aditya@gmail.com	1234	Null	Null	Null	Enter Password
4	aditya@gmail.com	1234	Null	Null	Null	-
8	aditya@gmail.com	1234	Null	Null	Null	-
9	aditya@gmail.com	1234	false	Null	Null	-
10	aditya@gmail.com	1234	false	Null	Null	-
11	aditya@gmail.com	1234	false	Null	Null	-
12	aditya@gmail.com	1234	false	Null	Null	-
13	aditya@gmail.com	1234	false	aditya@gmail.com	Null	-
14	aditya@gmail.com	1234	false	aditya@gmail.com	81dc9bdb	-
15	aditya@gmail.com	1234	false	aditya@gmail.com	81dc9bdb	-
16	aditya@gmail.com	1234	true	aditya@gmail.com	81dc9bdb	-
17	aditya@gmail.com	1234	true	aditya@gmail.com	81dc9bdb	-
18	aditya@gmail.com	1234	true	aditya@gmail.com	81dc9bdb	Successful Login
19	aditya@gmail.com	1234	true	aditya@gmail.com	81dc9bdb	-
20	aditya@gmail.com	1234	true	aditya@gmail.com	81dc9bdb	-
21	aditya@gmail.com	1234	true	aditya@gmail.com	81dc9bdb	Return “Success”

Line	email	password	user Located	userEmail	userPassword	Output
1	Null	Null	Null	Null	Null	-
2	test@gmail.com	Null	Null	Null	Null	Enter Email
3	test@gmail.com	pw	Null	Null	Null	Enter Password
4	test@gmail.com	pw	Null	Null	Null	-
8	test@gmail.com	pw	Null	Null	Null	-
9	test@gmail.com	pw	false	Null	Null	-
10	test@gmail.com	pw	false	Null	Null	-
11	test@gmail.com	pw	false	Null	Null	-
12	test@gmail.com	pw	false	Null	Null	-
13	test@gmail.com	pw	false	aditya@gmail.com	Null	-
14	test@gmail.com	pw	false	aditya@gmail.com	81dc9bdb	-
15	test@gmail.com	pw	false	aditya@gmail.com	81dc9bdb	-
10	test@gmail.com	pw	false	Null	Null	-
11	test@gmail.com	pw	false	Null	Null	-
12	test@gmail.com	pw	false	Null	Null	-
13	test@gmail.com	pw	false	sainath@gmail.com	Null	-
14	test@gmail.com	pw	false	sainath@gmail.com	a3f1c2d9	-
15	test@gmail.com	pw	false	sainath@gmail.com	a3f1c2d9	-
10	test@gmail.com	pw	false	Null	Null	-
11	test@gmail.com	pw	false	Null	Null	-
12	test@gmail.com	pw	false	Null	Null	-
13	test@gmail.com	pw	false	as@gmail.com	Null	-
14	test@gmail.com	pw	false	as@gmail.com	c8d4a2f1	-
15	test@gmail.com	pw	false	as@gmail.com	c8d4a2f1	-

29	test@gmail.com	pw	false	as@gmail.com	c8d4a2f1	-
30	test@gmail.com	pw	false	as@gmail.com	c8d4a2f1	-
31	test@gmail.com	pw	false	as@gmail.com	c8d4a2f1	Retry message
32	test@gmail.com	pw	false	as@gmail.com	c8d4a2f1	Return

For dry run 1:

Lines 4-7 are not executed since the inputs are sanitised; hence the inputs are valid. Lines 22-34 are not executed since the algorithm terminates immediately after returning "Success". This was the test of correct credentials being inputted, hence an account with password: "1234" and email: "aditya@gmail.com", exists within users.csv. The userEmail stores the email of the first user in users.csv, whilst userPassword stores the hashed password. In this case, the first user in users.csv is the one attempting to log in. This verifies the algorithm performs as expected under standard inputs.

For dry run 2:

Lines 4-7 are not executed since the inputs are sanitised; hence the inputs are valid. Lines 10-15 are iteratively executed until the end of the users.csv file is reached. The user's email is still not located when the csv has been fully searched, so the algorithm jumps to line 29, closing the data file and then checks if the Boolean is false (which it was) and outputs a retry message and then returns an error.

2.3.2.2 Exporting Flashcards as PDF/CSV

The exporting flashcard algorithm manages the conversion of flashcard data (question and answer) into either a comma-separated values (csv) file, or portable document format (pdf) file. This is important to the system because it will allow the user to share flashcard sets for importing, for backing up, or to transfer across devices. The CSV export is expected to be used for data portability and re-importing into the program, whilst the PDF export is expected to be used for ease of access (since these can be printed) for review, when not able to use a computer to run the Revision Assistant

The algorithm will ask the user what export they want (PDF or CSV) and the file name (validation in place to prevent runtime errors). The algorithm will try to open or create a new file for writing. If the file cannot be created or accessed due to permission issues, an error message is displayed to the user, followed by the termination of the process. Each flashcard object contains three attributes: question, answer and topic; so, the algorithm will iterate through each flashcard object, fetching the question and answer and doing checks to ensure they are not empty. If CSV is selected commas in questions and answers are removed and then joined together but separated by a comma to preserve the structure. If PDF is selected, the text will be formatted with a focus on readability to allow easy revision when not on the application. After all flashcards within

the deck are processed and converted, the file will be closed, and a confirmation message is displayed.

```
1   function exportFlashcards(flashcardSet){
2       do
3           exportFormat = input("Enter export format (1. CSV or 2. PDF): ")
4       until exportFormat == 1 or exportFormat == 2
5       do
6           fileName = input("Enter the file name: ")
7           if fileName == "" then
8               print("File name cannot be empty")
9           endif
10      until fileName != ""
11      if exportFormat == 1 then
12          flashcardFile = openWrite(fileName + ".csv")
13      else
14          flashcardFile = openWrite(fileName + ".pdf")
15      endif
16      if flashcardFile == null then
17          print("Cannot create or access file")
18          return "Error"
19      endif
20      for i = 0  to flashcardSet.length – 1
21          q = flashcardSet[i].getQuestion()
22          a = flashcardSet[i].getAnswer()
23          if q == ""  or a == "" then
24              print("Empty Flashcard has been skipped")
25          else
```

```
26                          if exportFormat == 1 then
27                                  qWoutC = q.replace(“,”, “;”)
28                                  aWoutC= a.replace(“,”, “;”)
29                                  flashcardFile.writeLine(qWoutC + “,” + aWoutC)
30                          else
31                                  text = “Question: “ + q + “\nAnswer: “ + a + “\n”
32                                  flashcardFile.writeLine(text)
33                          end if
34                  endif
35          next i
36          flashcardFile.close()
37          print(“Flashcards exported as ” + exportFormat)
38          return “Success”
39  endfunction
```

Line	exportFormat	fileName	i	q	a	flashcardFile	Output
1-2	-	-	-	-	-	-	-
3	9	-	-	-	-	-	“Exp Format?”
4	9	-	-	-	-	-	-
3	1	-	-	-	-	-	“Exp Format?”
4-5	1	-	-	-	-	-	-
6	1	“”	-	-	-	-	“File name?”
7	1	“”	-	-	-	-	-
8	1	“”	-	-	-	-	“Can’t be empty”
9	1	“”	-	-	-	-	-
10	1	“”	-	-	-	-	-
6	1	“deck”	-	-	-	-	“File name?”
11	1	“deck”	-	-	-	-	-
12	1	“deck”	-	-	-	deck.csv	-
16	1	“deck”	-	-	-	deck.csv	-
20	1	“deck”	0	-	-	deck.csv	-
21	1	“deck”	0	“1+1”	-	deck.csv	-
22	1	“deck”	0	“1+1”	2	deck.csv	-
26	1	“deck”	0	“1+1”	2	deck.csv	-
27	1	“deck”	0	“1+1”	2	deck.csv	-

28	1	“deck”	0	“1+1”	2	deck.csv	-
29	1	“deck”	0	“1+1”	2	deck.csv	-
35	1	“deck”	0	“1+1”	2	deck.csv	-
20	1	“deck”	1	-	-	deck.csv	-
21	1	“deck”	1	“”	-	deck.csv	-
22	1	“deck”	1	“”	“”	deck.csv	-
23	1	“deck”	1	“”	“”	deck.csv	-
24	1	“deck”	1	“”	“”	deck.csv	“Skipped”
36	1	“deck”	-	-	-	deck.csv	-
37	1	“deck”	-	-	-	deck.csv	“Exp as csv”
38	1	“deck”	-	-	-	deck.csv	“Success”

This dry run displays the algorithm being executed under normal as well as erroneous conditions. In line 3, an erroneous value of 9 is inputted when asked for the export format to test the condition controlled loop, then a correct value of 1 is inputted allowing the program to continue to the next condition controlled loop where on line 5-10, where the file name entry is tested to see how it responds to an empty string, hence the re-execution of line 6. After this the deck.csv file is created, and the passed in flashcard set of length 2 is traversed through replacing comma in questions and answers with semi-colons and then they are placed into deck.csv with a comma separating them. Line 37-38 give confirmation of the file successfully being exported.

2.3.2.3 Importing Flashcards as CSV

The importing flashcard algorithm will allow the user to load flashcard sets from a .csv file into the system. This feature is integral to the solution since teachers may want to create a fully accurate deck for their students to use for revision, or users may want to share their revision with their friends. The import feature prevents students from creating flashcard sets (when already available), reducing wasted time on preparing for revision. Importing also supports collaboration and reduces workload, assisting the achievement of the solution’s main goal: efficient revision.

The program will prompt the user to choose a CSV file to import, the algorithm will attempt to open the file; if it cannot be accessed (due to permission issues or being invalid), an error message will be returned, and the process terminates. Within a loop, the algorithm will read each line until the end of the file; every line is split at the first comma to extract the question and answer of each flashcard. Since at exportation commas within the questions and answers were replaced with semicolons, the import algorithm must reverse this, replacing all semicolons back into commas. If question or answer fields are empty, the card will be skipped alongside a message notifying the user about this. Else, a flashcard object is created and added to the flashcard deck.

After all the lines within the csv are processed, the file will be closed, and a confirmatory message will be returned to the user.

```
function importFlashcards()
    do
        fileName = input("Enter .csv file name to import: ")
        if fileName == "" then
            print("File name cannot be empty.")
        endif
    until fileName != ""
    fcFile = openRead(fileName + ".csv")
    if fcFile == null then
        print("Cannot access file")
        return "Error"
    end if
    importedDeck = []
    while NOT fcFile.endOfFile()
        line = fcFile.readLine()
        fcData = line.split(",")
        if fcData.length < 2 then
            print("Invalid, skipped")
        else
            ques = fcData[0]
            ans = fcData[1]
            if ques == "" or ans == "" then
                print("Empty flashcard, skipped")
            else
                ques = ques.replace(";", ",")
                ans = ans.replace(";" , ",")
```

```
27                                  card = NEW Flashcard(ques, ans)
28                                  importedDeck.append(card)
29                              endif
30                          endif
31              endwhile
32              fcFile.close()
33              saveDeck(importedDeck)
34              print("Flashcards Imported")
35              return "Success"
36      endfunction
```

Line	fileName	line	data	ques	ans	Output
1-2	-	-	-	-	-	-
3	""	-	-	-	-	"File name?"
4	""	-	-	-	-	-
5	""	-	-	-	-	"Name != empty"
7	""	-	-	-	-	-
3	Maths	-	-	-	-	-
4	Maths	-	-	-	-	-
7	Maths	-	-	-	-	-
8	Maths	-	-	-	-	-
13	Maths	-	-	-	-	-
14	Maths	-	-	-	-	-
15	Maths	"2+2,4"	-	-	-	-
16	Maths	"2+2,4"	["2+2", "4"]	-	-	-
17	Maths	"2+2,4"	["2+2", "4"]	-	-	-
20	Maths	"2+2,4"	["2+2", "4"]	"2+2"	-	-
21	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
22	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
25	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
26	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
27	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
28	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
29	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
30	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
14	Maths	"2+2,4"	["2+2", "4"]	"2+2"	"4"	-
15	Maths	"3x3; easy,9"	["2+2", "4"]	"2+2"	"4"	-
16	Maths	"3x3; easy,9"	["3x3; easy", "9"]	"2+2"	"4"	-
17	Maths	"3x3; easy,9"	["3x3; easy", "9"]	"2+2"	"4"	-
20	Maths	"3x3; easy,9"	["3x3; easy", "9"]	"3x3; easy"	"4"	-

21	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3; easy”	“9”	-
22	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3; easy”	“9”	-
25	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
26	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
27	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
28	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
29	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
30	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
14	Maths	“3x3; easy,9”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
15	Maths	“,32”	[“3x3; easy”, “9”]	“3x3, easy”	“9”	-
16	Maths	“,32”	[“”, “32”]	“3x3, easy”	“9”	-
17	Maths	“,32”	[“”, “32”]	“3x3, easy”	“9”	-
20	Maths	“,32”	[“”, “32”]	“”	“9”	-
21	Maths	“,32”	[“”, “32”]	“”	“32”	-
22	Maths	“,32”	[“”, “32”]	“”	“32”	-
23	Maths	“,32”	[“”, “32”]	“”	“32”	“Empty, skipped”
31	Maths	“,32”	[“”, “32”]	“”	“32”	-
32	Maths	“,32”	[“”, “32”]	“”	“32”	-
33	Maths	“,32”	[“”, “32”]	“”	“32”	-
34	Maths	“,32”	[“”, “32”]	“”	“32”	“Imported”
35	Maths	“,32”	[“”, “32”]	“”	“32”	“Success”

The dry run displays the algorithm being run under normal conditions alongside some erroneous tests. On line 3 when an empty string is inputted for the file name, hence the post-condition loop’s condition was met; this is why line 3 was run again, and this time with a correct input of “Maths”, lines 8-9 was then run to check whether the file could be opened to read. After this, line 13 initialised the array for the deck of new imported flashcard objects. Lines 14-31 was iteratively run (until the end of the file was reached), converting each line into question-and-answer attributes to create instances of flashcard objects. At the end of this, the deck was saved, and “Success” was returned.

2.3.2.4 Shuffling

The flashcard shuffling algorithm will randomise the order of the flashcards in the deck (which was selected for revision) when shuffle mode has been chosen as the study option. Shuffling prevents the user from relying on fixed sequences of cars, supporting active recall, this will make each session more cognitively demanding by making each session’s flashcard sequence unpredictable.

The flashcard deck will be passed in as an argument, and the algorithm will iterate through it, iteratively swapping each flashcard with another random index. This makes sure that all the flashcards have an equal chance of coming up in any position. This algorithm prevents students from memorising the order of flashcards, hence making revision more effective.

```
1   function shuffleFlashcards(deck)
2       for i = deck.length-1 to 1 step -1
3           randIndex = random(0,i)
4           temp = deck[i]
5           deck[i] = deck[randIndex]
6           deck[randIndex] = temp
7       next i
8       return deck
9   endfunction
```

Line	i	randIndex	temp	deck[i]	deck[randIndex]	deck State
1	-	-	-	-	-	[“A”, “B”, “C”, “D”]
2	3	-	-	“D”	-	[“A”, “B”, “C”, “D”]
3	3	1	-	“D”	“B”	[“A”, “B”, “C”, “D”]
4	3	1	“D”	“D”	“B”	[“A”, “B”, “C”, “D”]
5	3	1	“D”	“B”	“B”	[“A”, “B”, “C”, “B”]
6	3	1	“D”	“B”	“D”	[“A”, “D”, “C”, “B”]
2	2	-	“D”	“C”	-	[“A”, “D”, “C”, “B”]
3	2	0	“D”	“C”	“A”	[“A”, “D”, “C”, “B”]
4	2	0	“C”	“C”	“A”	[“A”, “D”, “C”, “B”]
5	2	0	“C”	“A”	“A”	[“A”, “D”, “A”, “B”]
6	2	0	“C”	“A”	“C”	[“C”, “D”, “A”, “B”]
2	1	-	“C”	“D”	-	[“C”, “D”, “A”, “B”]
3	1	1	“C”	“D”	“D”	[“C”, “D”, “A”, “B”]
4	1	1	“D”	“D”	“D”	[“C”, “D”, “A”, “B”]
5	1	1	“D”	“D”	“D”	[“C”, “D”, “A”, “B”]
6	1	1	“D”	“D”	“D”	[“C”, “D”, “A”, “B”]
7	1	1	“D”	“D”	“D”	[“C”, “D”, “A”, “B”]
8	1	1	“D”	“D”	“D”	[“C”, “D”, “A”, “B”]
9	1	1	“D”	“D”	“D”	[“C”, “D”, “A”, “B”]

The dry run shows the flashcard deck’s state prior every swap and after every swap. In each iteration, a random index is generated between 0 and i and, the two cards’ positions are swapped. Every flashcard has an equal chance of moving of moving to any position in the deck. In this case the dry run of a flashcard deck containing four flashcards repeated 3 times to shuffle the deck in a random order, making it very efficient.

2.3.2.5 Flashcard Searching

The flashcard searching algorithm will allow the user to search for flashcards via keywords. Flashcards containing the keyword, the user entered, in either the question or answer will be returned to the user. Searching allows the user to easily access flashcards they may need to change in case they are wrong or not detailed enough. This algorithm provides users with an efficient way of locating specific flashcards within large flashcard decks.

The algorithm will take a flashcard deck in as an argument; the user will then be prompted to enter the keyword they are looking for. This keyword is converted into lower case, and a flag will be declared to confirm whether the keyword has been found. In a count-controlled loop, the question and answer of a flashcard will be converted into lowercase and checked to see if they contain the keyword (by sub stringing the question in the same length as the keyword). The loop runs until every flashcard has not been checked. The index of the flashcard containing the keyword will be outputted, if none are found; an appropriate message will be outputted.

```
1  procedure searchForKeywords(deck)
2      kw = input("Enter the search criteria/keyword: ")
3      kw = kw.toLowerCase()
4      kwFound = false
5      for i = 0 to deck.length-1
6          ques = deck[i].getQuestion().toLowerCase()
7          ans = deck[i].getAnswer().toLowerCase()
8          if ques.substring(0, ques.length).contains(kw) or ans.substring(0,
9  ans.length).contains(kw) then
10             print("Found at flashcard with index: " + i)
11             kwFound = true
12         endif
13     next i
14     if kwFound == false then
15         print("None found")
16     endif
17 endprocedure
```

Line	kw	i	ques	ans	kwFound	Output
1	-	-	-	-	-	-
2	“Pho”	-	-	-	-	“Enter kw”
3	“pho”	-	-	-	-	-
4	“pho”	0	-	-	false	-
5	“pho”	0	-	-	false	-
6	“pho”	0	“2+2”	-	false	-
7	“pho”	0	“2+2”	“4”	false	-
8	“pho”	0	“2+2”	“4”	false	-
5	“pho”	1	“2+2”	“4”	false	-
6	“pho”	1	“photosynthesis”	“4”	false	-
7	“pho”	1	“photosynthesis”	“used by plants”	false	-
8	“pho”	1	“photosynthesis”	“used by plants”	false	-
10	“pho”	1	“photosynthesis”	“used by plants”	false	“Found:”+ i
11	“pho”	1	“photosynthesis”	“used by plants”	true	-
5	“pho”	2	“photosynthesis”	“used by plants”	true	-
6	“pho”	2	“photon”	“used by plants”	true	-
7	“pho”	2	“photon”	“quanta of E”	true	-
8	“pho”	2	“photon”	“quanta of E”	true	-
10	“pho”	2	“photon”	“quanta of E”	true	“Found:”+ i
11	“pho”	2	“photon”	“quanta of E”	true	-
14	“pho”	2	“photon”	“quanta of E”	true	-
17	“pho”	2	“photon”	“quanta of E”	true	-

The dry run shows the execution of the algorithm when a flashcard deck of length 3 is passed into the procedure. The user is prompted to enter the keyword on line 2, which is then converted to lower case on line 3. A count-controlled loop is then used to extract all the questions and answers of the flashcards, serially. The questions and answers were checked to see whether it contained the keyword. On the first flashcard, it was not found, however the next two both contained the keyword; hence the index of the flashcard was outputted.

2.3.3 Data Dictionary

Name	Data type	Storage Size (bytes)	Validation	Justification
email	String	Number of character x 2	Must follow the classic email format (test@example.com), cannot match an email in the csv already.	Email address will identify every user uniquely and prevents duplicate accounts on the same device. Email addresses are also easy to remember, helping ensure users won’t lose their accounts if they forget their credentials.
password	String	Number of character x 2	Minimum of 6 characters	Enforcing a minimum length alongside enforcing having a special character will increase password strength, making it

			including a special character	less likely for people to password guess and potentially destroy user revision data.
confirmPassword	String	Number of character x 2	Minimum of 6 characters including a special character and must be the same as the password.	Confirming the password will reduce the chances of user error during registration, so the use can successfully log in later within needing to reset.
hashedPassword	String	Number of character x 2	N/A	Storing hashed password instead of plain text will protect user passwords.
salt	String	Number of character x 2	N/A	Ensures identical password do not result in identical hashed passwords, used for verifying login.
recoveryWord	String	Number of character x 2	>= 3 characters	A word more than 2 characters will be easy to remember for the user to verify identity to reset their password and recover their account.
question	String	Number of Characters x 2	Checked to see whether empty or not, as well as character limit (10 – 300)	A flashcard without a question/prompt would be useless and could end up breaking display logic. Furthermore, with many characters the questions would become too long and would become an inefficient way to revise.
answer	String	Number of Characters x 2	Checked to see whether empty or not, as well as character limit (1-600)	A flashcard without an answer wouldn't help with revision and would maybe break the revision loop; the question is needed to check whether the user is correct when using the application. The answer also has a character limit, so the flashcards are effective, hence easy to memorise.
deckName	String	Number of Characters x 2	Must not be null, can't start with white space, must be a suitable file name.	The user must select their own name for the deck to ensure their revision is personalised. This must be a suitable file name as well, otherwise it would not be able to be saved.
lastReviewed	String	Number of Characters x 2	Must be in the form dd/mm/yyyy	This is stored as text to simplify the CSV data saving whilst also avoiding LocalDate objects which have a large footprint. Incorrect format may confuse the user and would be hard for the system to calculate the difference in time (days) since last studied.
correctAns	int	4	>= 0	This tracks how many times the users get flashcard answers correct; if negative, it would sway the statistics to an invalid

				point, disrupting possible visualisations of user data (in charts).
incorrectAns	int	4	>= 0	This tracks how many times the user gets questions wrong; it will be used once again for analytics, hence cannot be negative.
flashcards	ArrayList<>	8 per element	>= 0 <=60	An ArrayList would allow the dynamic addition as well as deletion of cards and provides O(1) direct access to indexes. Students cannot study a flashcard list of length 0, hence there must be present flashcards to maintain functionality. Too many flashcards in single list would be impossible for the user to revise completely; hence each list will have a limit of 50 for the best possible revision.
fileLocation	String	Number of Characters x 2	Must be a valid path, the solution must be able to access it.	The file path must be valid so the program can store and save flashcard sets, without access user data might not be saved hence lost, which is against the aims of the Revision Assistant.
topic	String	Number of Characters x 2	Cannot be empty	This will be assigned to each set (or individual flashcard, depending on user choice), supporting filtering and accurate analytics.
currentSet	String	Number of Characters x 2	Must match the file location	This is loaded up when the user selects which flashcard set to modify or to learn from, it must match the file location of the flashcard set so the correct set is loaded, avoiding any frustration caused to the user.
modeName	String	Number of Characters x 2	Must match a valid mode	This will prevent the invalid selection of study modes, limiting the user to what the solution offers: Wrong Answers Only; Shuffle; Serial.
serialOrder	int[]	4 x n	n = Set Size Each element must be unique	This makes sure all the flashcards within the set can be reviewed by the user, ensuring no flashcards are missed, or repeated.
randomOrder	int[]	4 x n	n = Set Size Each element must be unique	When flashcards are shuffled, this makes sure all flashcards within a set are used and no flashcards are repeated.
wrongAnswers	ArrayList<>	8 per element	Only added if user was incorrect on current flashcard.	This will build the wrong answers only study mode, if flashcards which the user has got right are moved to this list, the study mode would be ineffective since

				users are continuously covering already mastered topics.
startTime	Date	12	Must be before end time	If the startTime is after the endTime, this would be a negative, invalid duration.
endTime	Date	12	Must be after start time	The endTime being before the startTime would also give a negative, invalid duration. So, these are prevented through validating the times.
duration	double	8	>= 0	The duration of a study session cannot be negative, this would lead to inaccurate representation of user progress/data, when analytics are displayed.
xp	int	4	>=0, assigning experience points must align with standards set in success criteria.	A user's experience points cannot be negative, the minimum is 0, and this is when they are beginning a new level. The assignment of XP must align with the standards set in the success criteria, ensuring all experience point allocation is fair and equal.
achievements	List<>	8 per reference	Must be a valid achievement, with the user meeting the criteria, specified within the program's achievement specification.	The program should not assign random/invalid achievements to users who have not met the criteria, for example the achievement "Complete a 1-hour Study Session" cannot be given to users who have only completed a 30-minute session.
sfxVolume	Double	8	Must be between 0.0 – 100.0	The volume level is restricted to values between 0.0 and 100.0. The volume being 0.0 represents silence, whilst 100.0 represents the maximum volume intake of the system. Levels outside this range do not have a physical or logical meaning to the program, hence these must be validated to prevent run time errors.
muted	Boolean	1	Inputs can only be true or false, also set to true if sfxVolume is 0	The muted variable will only take explicit Boolean values (true or false), ensuring the audio is consistently defined within the system. Other forms of inputs, such as numbers, may be too ambiguous and reliant on inferring, hence preventing misinterpretation / unintended audio behaviour.
darkness	Double	8	0.0 – 80.0	Darkness is restricted between 0.0 and 80.0, 80.0 being the darkest and 0.0 being the brightest. This is because over 80.0 is

				too dark for use and below 0.0 would be complex to code, as well as unnecessary.
highContrast	Boolean	1	true/false	Can only accept Boolean values to ensure the user interface is either in the normal mode, or high contrast mode. Ambiguous entries such as numbers need interpretations, hence the input is limited to explicit Boolean values, guaranteeing predictable behaviour when rendering colours.
isTTSEnabled	Boolean	1	true/false, volumeLevel > 0 (if true) or muted==false	Once again, the system will only take explicit Boolean values, not ambiguous ones, preventing misinterpretation from the program. Furthermore, this Boolean can only be set to true if the volumeLevel is greater than 0 or if muted if false, so this can be heard by the user. If muted or volume is zero while TTS is enabled, the user may get confused.
ttsVolume	Double	8	Must be between 0.0 – 100.0	Same validation as sfxVol
Theme	String	Number of Characters x 2	Must match a theme in the theme array.	If the user inputted theme does not match a theme in the theme array, the program will not be able to display this for the user. Validatory checks against the theme array ensures only supported themes can be selected, preventing rendering errors whilst allowing predictable system behaviour.

2.4 Approach to Testing

2.4.1 Testing Strategy

Within the development of the Revision Assistant, I will make use of the following testing strategies to identify defects, ensuring the solution meets the specified criteria. Tests will be a combination of white-box and black-box testing being used across unit and integration testing. These tests will be followed by alpha, beta and user acceptance testing.

White-box testing is a testing technique which involves the testing of the internal structure of an application. Testers have access to the source code and use this knowledge to design tests that can verify the system works. I will use this to test the logic of methods and algorithms through checking flow, conditions, loops, etc. For instance, this can be used to test XP calculations (10XP for a correct answer, 2XP for an incorrect answer). White-box testing is effective since it leads to an early detection of

logic errors which are not always visible through user interaction alone and allows code to be optimised, by removing extra lines of code when removing errors.

Unit testing will be used to ensure individual classes and subroutines work correctly in isolation to the entire program before being recombined with other modules of the program. I will create effective tests using boundary checks, seeing if the system can handle errors, etc. Unit testing prevents defects from being carried forward to later stages of development, reducing the debugging time overall.

Integration testing will be used once unit testing is completed with individual modules to confirm that modules interact with each other correctly with full functionality. The Revision Assistant contains many connected modules, for example importing flashcards will use FlashcardManager, and then will be run in StudyModes. Integration testing will help detect faults within the interface such as incorrect data types being passed into different modules.

Black-box testing is a testing technique where the tester is not concerned with the internal workings of the software and focuses on validating the functionality based on provided specifications. I will make use of this to view the system from a user's perspective to confirm the solution behaves correctly based on inputs and outputs, such as trying to log in with invalid credentials. Black-box testing will help verify the usability of the program.

2.4.2 Test Plan

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	email	aditya@gmail.com	Normal	Account registered / Logged In / Accepted		
2.	email	aditya@gmail	Erroneous	Message displayed to user this input is not possible, encouraging correction.		
3.	email	email which already exists	Erroneous			
4.	(confirm)pw	Do not match	Erroneous	Alert saying the passwords do not match		
5.	password	Dormers*	Normal	Accepted, will call registration/login function.		
6.	password	Ad*tya	Boundary			
7.	password	Adity	Erroneous	Rejected, ask user to input according to requirements.		
8.	password	*”£^	Erroneous			
9.	confirmPassword	Dormers*	Normal	Accepted, will call registration/login function.		
10.	confirmPassword	Ad*tya	Boundary			
11.	confirmPassword	Adity	Erroneous	Rejected, ask user to input according to requirements.		
12.	confirmPassword	*”£^	Erroneous			
13.	recoveryWord	School	Normal	Accepted, attempt to register, or reset password.		
14.	recoveryWord	Dog	Boundary			
15.	recoveryWord	Hi	Erroneous	Not accepted, can’t register.		
16.	question	“What are the 4 stages of mitosis?”	Normal	Visual confirmation the input of a flashcard question. Flashcard with question added to table		
17.	question	“1234567890” 10-character question	Boundary			
18.	question	300-character question	Boundary			
19.	question	301-character question	Erroneous	Message to tell the user the input is not valid.		
20.	question	“ “ (empty string)	Erroneous			

				User given option to re-enter or stop making flashcards.		
21.	answer	“Prophase, metaphase, anaphase, telophase”	Normal	Visual confirmation the input of a flashcard question. Flashcard with question added to table		
22.	answer	“A” 1-character	Boundary			
23.	answer	600-character answer	Boundary			
24.	answer	601-character answer	Erroneous	Message to tell the user the input is not valid. User given option to re-enter or stop making flashcards.		
25.	answer	“ ” (empty string)	Erroneous			
26.	correctAns	10	Normal	Acceptable counter value, data shown on screen.		
27.	correctAns	0	Boundary			
28.	Submit input	Submit input without flipping	Erroneous	Error should be displayed to user		
29.	incorrectAns	4	Normal	Acceptable counter value, data shown on screen.		
30.	incorrectAns	0	Boundary			
31.	Flip Card	Flashcard	Normal	Flashcard should flip		
32.	flashcards (list)	Deck with 5	Normal	The deck is saved, confirmation message shown.		
33.	flashcards (list)	Deck with 60	Boundary			
34.	flashcards (list)	Deck with 0	Erroneous	Error message shown, you cannot revise an empty deck.		
35.	deckName	“Physics”	Normal	Accepted; the file will load/save successfully.		
36.	deckName	“???”	Erroneous	Error message, user can try again.		
37.	currentSet	“bio.csv”	Normal	Correctly loads, matches file selected.		
38.	currentSet	“asdads.csv”	Erroneous	Not a valid option		
39.	modeName	“Serial”	Normal	Serial mode begins; cards appear in original order		

40.	modeName	“User Adaptive Learning”	Erroneous	Not a valid option for Study Modes		
41.	Any study mode order	[0, 1, 2, 3, 4]	Normal	Accepted, all cards are shown until correct.		
42.	Any study mode order	[0, 1, 2, 4, 4]	Erroneous	A session cannot have two of the same cards in the active queue, unless the flashcard deck has two of the same cards in a deck.		
43.	wrongAnswers	User gets flashcard incorrect	Normal	The flashcard should be added into the wrong answers list, and .csv.		
44.	wrongAnswers	User gets flashcard correct	Erroneous	Flashcard should not be added to the wrong answers list.		
45.	duration	300 seconds	Normal	Accepted, duration displayed to user in hh:mm:ss.		
46.	duration	-30 seconds	Erroneous	Duration cannot reach this value; time cannot be negative		
47.	xp	0	Normal	Accepted, new users start at 0.		
48.	xp	Study for a minute, end session with 1 incorrect	Normal	+13 xp, shown to user		
49.	xp	Get 1 answer right, 1 wrong	Normal	+20 after getting 1 correct and 1 incorrect, shown to user		
50.	isTTSEnabled	True	Normal	Accepted, ttsVol changes from 0.0 to default		
51.	isTTSEnabled	False	Normal	Accepted, ttsVol set to 0.0		
52.	sfxVol	59.3	Normal			

53.	sfxVol	0.0	Boundary	Sound plays at the input volume successfully.		
54.	sfxVol	100.0	Boundary			
55.	sfxVol	101.1	Erroneous	Rejected, set to 100.		
56.	ttsVol	59.3	Normal	Sound plays at the input volume successfully.		
57.	ttsVol	0.0	Boundary			
58.	ttsVol	100.0	Boundary			
59.	ttsVol	101.1	Erroneous	Rejected, set to 100.		
60.	muted	False	Normal	sfxVol set to default value from 0.0		
61.	muted	True,	Normal	sfxVol set to 0.0		
62.	darkness	36.2	Normal	Brightness of screen successfully adjusted, instantly applied, consistent across screens.		
63.	darkness	0.0	Boundary			
64.	darkness	80.0	Boundary			
65.	darkness	90.2	Erroneous	Set to 80.0, rejected		
66.	theme	“dark”	Normal	Theme applied instantly.		
67.	theme	“light”	Normal	Theme applied instantly.		
68.	theme	“highContrast”	Normal	Theme applied instantly.		
69.	theme	“BlueTheme”	Erroneous	Not a possible option to be entered by the user		
70.	Shuffle mode	Flashcard deck order	Normal	Must be in different order to the original deck.		
71.	wrongAnswers.csv	Contents	Normal	Must have the same card repeated twice in the .csv.		
72.	TTS	Clicked on question side	Normal	TTS speaks out question		
73.	TTS	Clicked on answer side	Normal	TTS speaks out answer		
74.	Study Session	End the session with 1 correct	Normal	+115xp due to 100% accuracy and Flawless achievement		
75.	Study Session	Retry a session	Normal	Retry a session achievement unlocked		

76.	Study Session	Review at least 10 flashcards	Normal	Getting Started unlocked		
77.	Study Session	Last Study Date	Normal	Last Study Date set to the date of today		
78.	Study Session	Get 10 flashcards right in a row	Normal	Perfect Run achievement unlocked		

2.4.3 Post-Development Testing

Following the completion of the core implementation and initial testing of the program, post-development will be carried out to confirm the system's stability as a release candidate, and whether it meets the success criteria.

Alpha testing is conducted by the developer prior to the release of the program to users to beta test. I will use the program as a normal user would in a sequential process. Firstly, starting off with logging into the system, followed by personalising my settings (brightness, theme, audio) checking if the changes apply instantly and save after relaunch. I will then create flashcards with valid questions and answers, adding them into a deck and testing different study modes. After each session, I will check if the session summary displays all the correct information and whether the data is persistent after a restart. Following this, I will test other features of the program, such as importing a CSV and then exporting straight away, checking to see whether all cards have been exported successfully. This will be used to verify the Revision Assistant is a stable final draft, detecting and fixing issues prior to releasing the system to the users.

Destruction testing will be used to push the Revision Assistant into scenarios where failure is expected to confirm that it remains stable and handles crashes safely. This will be important since user data is saved locally, hence, to prevent data loss, the system must be made to handle crashes effectively. I will test destructively by:

1. Corrupted CSV File
    - Open a csv file with flashcard data and remove a comma in a row, import it on the system. Expecting the system return an error and not accept the csv.
2. Corrupted numeric fields
    - In the user progress file, update the xp value (integer) with a string, load the app. The system should reset the value to the last valid one, alerting the user about potential data loss.
3. Spam Clicking
    - Rapidly click "Next Card" option. Rapidly toggle mute. The program should remain responsive without having duplicate states.
4. Massive dataset
    - Import a flashcard set of the max size, with maximum characters for answers and questions, run a shuffle, search or export. The system should run within an appropriate time, without freezing.

Following the alpha tests, beta testing will be completed. Beta testing is the process of allowing a group of users (representing the stake holders) to trial the system before its release. Feedback from the trial users will help identify issues which are not clear during development such as confusing interface flow, lacklustre usability features,

study modes not matching user expectations. This helps measure the usability of the software in a real-world environment, with feedback being used to refine the system.

Usability testing will also be completed by the trial users, where they will have been given a list of usability and will be providing feedback based on their effective in the system.
User acceptance testing will be carried out as the last stage of testing to conclude the program meets the defined, measurable success criteria and (hence) is fit for its purpose.

Success criteria	Test data	Justification	Expected outcome	Actual outcome
1) The program should launch within 10 seconds on a recommended-spec (or higher) device.	Run the program 10 times, with each time outputting a load time and calculate an average.	Multiple runs reduce the effect of random error; hence a one-off case will have less effect. It also proves performance target is met.	Average <= 10 seconds	
2) The Revision Assistant should display a splash screen when the application is launched, prior to registration/login screen.	Launch the program, check the splash screen is displayed first and can shift to the login screen immediately.	Confirms the application is loaded and responsive, the user controls the pace of the Revision Assistant from the very start, so they are in control of their revision.	Splash screen occurs on launch and will change immediately when a key is pressed.	
3) The system should allow multiple users to register and log in on the same device.	Test by registering multiple users and attempting to log in to each account separately.	Confirms the solution can handle multiple accounts without the overwriting of data, hence confirming it can be used successfully on a shared device.	Each user is expected to log in successfully, with data saved persistently.	
4) The program should not store duplicate email addresses and must not store passwords in plain text (store hashed password instead).	Check if registering multiple users with same email is possible. Open CSV and observe if passwords are stored in plain text	Accounts are less ambiguous and emails can act as a unique identifier, making sure the wrong user's data is never loaded to a user's account. Hashing will increase the system's	One email can be linked to one account; an error will be displayed. No password is stored in plain text, and salt is stored so same password	

		security, protecting user credentials	does not allow produce same hash value.	
5) The solution will store data locally and must not crash if the data file corrupted.	Create an account, import a deck (corrupted and uncorrupted), study this deck gain XP, close the app and reopen checking if data still present.	This ensures the program is reliable for the user, and corrupted data will not destabilise the Revision Assistant.	Persistent data No crash, error message.	
6) The program will use a high contrast black and white style sheet for users with colour blindness, and will have other options available (dark mode, light mode, etc); changes must apply instantly.	Switch from high contract to light, mid sessions multiple times. Verify the text remains readable and changes are consistent across screens in each mode. Make the program output the time taken for this to happen.	Confirms the solution remains accessible, and theme switching will not affect one screen only. Ensures users with visual impairments are not suddenly exposed to situations which can hinder their ability to read.	Theme changes apply immediately and are consistent across different screens.	
7) The solution should accurately measure XP and Achievements.	Run sessions with known outcomes. E.g. 10 correct and 0 wrong, 100 XP; 1 hour session, 500XP. Verify achievements are instantaneously unlocked when criteria are met (e.g. first session completed).	Using predictable data makes any issues in awarding XP obvious, hence easier to identify. Achievements must be trustworthy in the solution otherwise gamification fails.	10 correct with 0 wrong should give 150XP, XP matches the specification exactly. Achievements unlocked immediately when conditions are met.	

8) The program must be available for offline use.	Disconnect from the internet, check functionality of the solution, launch, load a deck, run a session, export a deck.	Proves the solution can be accessed anywhere since there is no internet dependency, supporting stakeholders with unreliable connectivity.	All features of the system work as usual. No crash, or network errors.	
9) The program must allow users to import (csv) / export (csv/pdf) flashcards.	Using a valid CSV with 25 flashcards, import it into the system to check whether they all appear. Export the same deck as CSV and check if it is the same as the import. Export as PDF to check formatting.	Proves the core functionality feature of importing and exporting works successfully without any issues, hence users can have on-the-go revision or pre-made revision sets.	Valid CSV imports successfully and is exported correctly. Invalid CSV does not crash system.	
10) The solution must have session time-tracking in hours and minutes.	Run a timed session, timing it externally (stopwatch). End the session and compare if the displayed time is matching the stopwatch time in the correct format (hh:mm:ss).	Proves the solution meets the criteria, students can measure their revision to help improve their study discipline and time-management.	Recorded time matches the stopwatch time and is displayed in the correct format (hh:mm:ss).	
11) The program must have volume sliders to turn on and off sound effects.	Set volume to different levels (0, 30, 60, 100) and try sound effects to see if volume slider has an effect. Toggle mute on and off.	Proves students who see sound as distractions can limit the effect it has on their studying, allowing them to focus on revision alone.	Volume level changes as requested by user with little to no delay, saving if user requests.	

12) The program must have folder organisation systems, allowing the user to pick which folders flashcard decks go in.	Create folders, saving decks into each of them. Move deck between folders, delete decks, restart system to see if data is persistent.	Proves students can organise their study material easily to prevent revision from becoming overwhelming.	Decks appear in correct folders and moving decks is correctly updated within files. Deleting is safe and gives warning to user that change is permanent. Changes remain post system restart.	
13) The solution must contain a valid text-to-speech feature.	Enable TTS and run 10 flashcards of varying lengths. Check if the deck is successfully read out loud by the TTS.	Proves the revision is accessible to a variety of needs, allows students to process information in an alternative way.	TTS reads information out correctly and will not show features if TTS is disabled	
14) The solution must provide advanced analytics.	Run sessions with known raw results, compare session analytics with known data. Repeat with different data sets to check for consistency.	Proves students are allowed to easily identify their weaknesses and strengths in revision, so they don't waste time on learning already mastered content.	Analytics match raw results and persist post restart.	
15) The system must allow the user to create flashcard decks, full of flashcards, and allow the modification specific flashcards or the whole deck post-creation	Create a deck, save it. Modify the deck: 1. Add cards 2. Edit existing cards 3. Delete cards 4. Rename the deck Delete the deck	Proves the Revision Assistant expects revision to be an iterative process which the user will adjust over time, to make revision more effective.	Deck created and saved, cards added, cards edited, cards deleted, deck renamed and deck deleted successfully.	

Section 3 Development

3.1 Prototype 1: Initial User Interface

3.1.1 Purpose of Prototype 1

Upon initially launching the program, the user will be displayed a splash screen with an animation to symbolise the system is loading, and a prompt to press any key to continue to the authentication screen. This ensures the student is in full control of the program (and hence their revision) from the very beginning, allowing to set the pace of their revision. In the authentication screen, the user will choose between logging in and authentication and will be prompted to input data which will be validated by the system prior to the writing of this data into a file.

Prototype 1 establishes a secure, reliable and user-friendly foundation for the entire system. Authentication has been embedded early on, which will reduce the possibility of potential issues in password handling, file access, validation features (for example). This prototype is a structural backbone for the system and will allow me to focus on extending functionality of the system without worrying about possibility of compromising security. In addition, this prototype addressed a major uncertainty in the project: how external files could be reliably linked and read/written to. Implementation of file handling at this stage has helped me to develop a clear understanding on how to manage file paths, and making sure data is persistently stored.

3.1.2 Success Criteria for Prototype 1

Referencing the Success Criteria (1.4.3 Success Criteria), in this prototype I will be aiming to achieve criterions 2, 3, 4 and 5. These criteria will focus on the ensuring the system is stable, secure and data is persistent, laying the foundation for the future development.

2) The Revision Assistant should display a splash screen when the application is launched, prior to registration/login screen.
3) The system should allow multiple users to register and log in on the same device.
4) The program should not store duplicate email addresses and must not store passwords in plain text (store hashed password instead).
5) The solution will store data locally and must not crash if the data file corrupted.

I will aim to achieve the criteria above by meeting the following sub-criteria:

3.1.2.1 Splash Screen Criteria

- Revision Assistant should display a splash screen when the program is executed.

- The splash screen will be shown until the user interacts with the program.
- Scene switching should be smooth.

3.1.2.2 Login Criteria

- Users log in via email and password.
- Users can recover their account (reset password) using a recovery word set at registration.
- Authentication checks details against locally stored data.
- Multiple distinct user accounts can be supported, but not concurrently.

3.1.2.3 Registration Criteria

- Users must have a unique email address to register, will be checked against stored data.
- Users must set a recovery word prior to success registration.
- Users will be informed of requirements of username, password and recovery word.

3.1.2.4 Data Storage Criteria

- User data is stored locally, if the file is not found then it must be created.
- File read and write operations are defensively programmed.
- Passwords are never stored in plain text, must be hashed.

3.1.3 Developing Prototype 1

This section will document the development process of the prototype, explaining how the foundation of the system was implemented. Development followed an iterative approach where individual components were implemented and then tested (and refined) prior to integration into the system as a whole.

3.1.3.1 Separation of Project Components

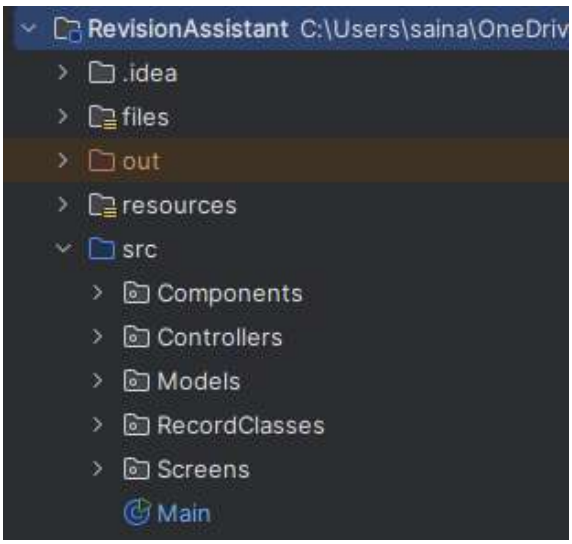
Firstly, the Revision Assistant was structured through the separation of clear project components. Initially I created 2 packages within the source folder of the project: Models (containing the user class) and Controller (containing the AuthenticationController class), having all my code for the screens within the main file of the source folder.

However, as I was developing the authentication screen, I noticed many components were being repeated but used for different features such as the Rectangles which I used for buttons, or the text/password fields used for user input into the system. Hence, these classes were extracted into the RecordClasses (and the initialisation of these classes to create rectangular buttons for example was placed into the UIComponents class within the Components package), increasing the modularity and reusability of code. The Components package contains classes which have reusable utility manage

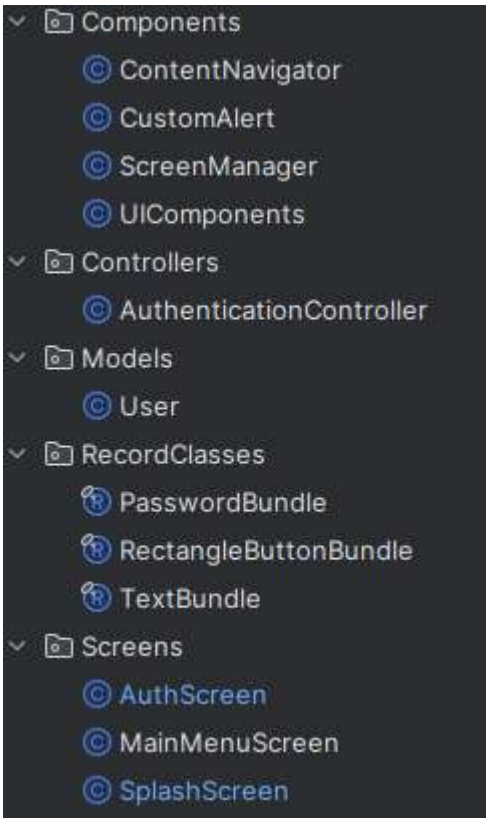
interface components such as the ContentNavigator, ScreenManager, UIComponents, and CustomAlert classes.

Following the completion of all the logic and screens (for this prototype), I realised having too much code within Main.java was unsustainable and significantly decreases the modularity of program, making code harder to maintain and update in case of any future bugs. To resolve this, the screens were refactored into a dedicated package called Screens. Main.java now acts as an entry point to the application, supporting future prototype development.

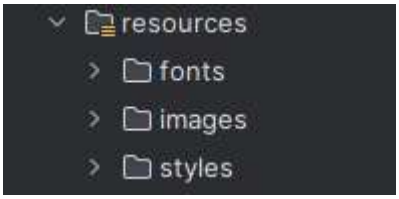
Furthermore, there are two directories marked as resource root folders, “files” and “resources” both located outside of the source directory. User-specific data files, currently only users.csv (storing user authentication data), is stored within the “files” directory. This data is intended to be accessed and modified at runtime; it also must persist across different application sessions. It has been separated from the “resources” folder which stores static assets (e.g., icons, fonts and style sheets) which are only read at runtime. This separation will prevent accidental overwriting during development, simplifying read/write operations. Mutable data being distinctly separated also ensures assets remain stable and predictable, hence users will not see varying fonts in the application, reducing usability, as an example.



Project Structure Image 1 - Project Structure



Project Structure Image 2 - Project Structure Source Expanded



Project Structure Image 3 - Resource Root



Project Structure Image 4 - User File Root

3.1.3.2 Splash Screen

Addressing Success Criterion 2, a Splash Screen was implemented as the first user-facing stage of the application. It will create a professional start up experience for the user, whilst also ensuring the program moves at a pace the user controls.

```
public void start(Stage primaryStage) throws Exception {  new *
    // Set stage reference for scene management
    ScreenManager.setStage(primaryStage);
    // Creates and configures the splash screen, linking the stylesheet to it.
    SplashScreen splashScreen = new SplashScreen();
    AuthScreen auth = new AuthScreen();
    Scene splashScene = splashScreen.create(() -> ScreenManager.switchScene(auth.create()));
    primaryStage.setTitle("Revision Assistant");
    primaryStage.setScene(splashScene);
    primaryStage.show();
}
```

In

Main.java Code 1 – start() subroutine

Main.java, when the program is run, a splash screen and authentication screen object is created and the splashScreen.create function is called, which returns a scene which is set onto the primary stage. A callback is passed into the splash screen, hence when the user interacts with the screen, that callback will be run (switching the screen to the authentication screen).

```
public static void switchScene(Scene root){  3 usages  new *
    // Check if there's currently a scene being displayed
    if(stage.getScene() != null){
        // Save current scene to stack before switching
        history.push(stage.getScene());
    }
    // Switch scene to new scene
    stage.setScene(root);
}
```

ScreenManager Code 1 - switchScene() subroutine

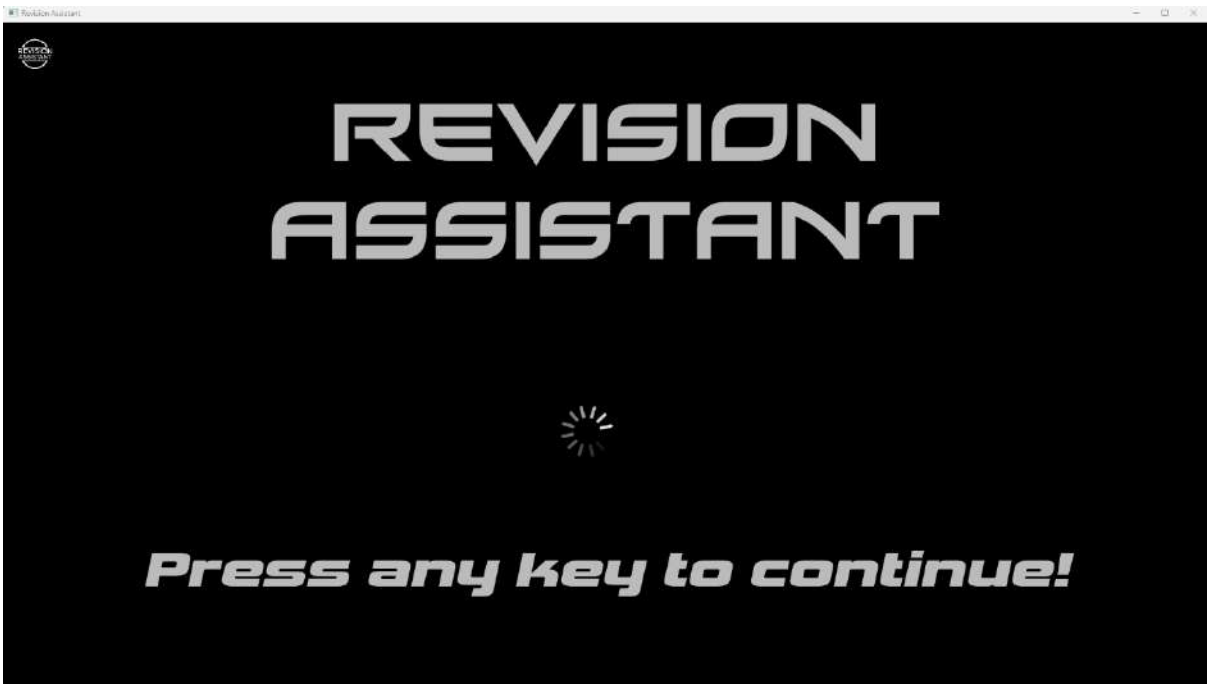
The switchScene() subroutine takes a scene as a parameter, for the code above, this scene is provided by auth.create() and will be set to the stage. The current scene will be pushed onto a stack for easy backtracking if the user decided against a scenario (such as registering) in the future.

The create function within the SplashScreen class takes in a Runnable parameter called onContinue, and code will be executed when onContinue.run() is called. The

splash screen uses two concurrent animations to provide visual feedback to the users, implying that the system is loading (however in reality, it is just an effect). Both animations run indefinitely, hence will run until the user interacts with the system to change the screen. With the use of an event handler, once the user interacts with the application with a mouse click or a keypress, the onContinue.run() is ran and it will change the screen to the authentication screen.

```
// Animate the loading indicator with continuous rotation
RotateTransition loadingTransition = new RotateTransition(Duration.seconds( s: 8), loadingView);
loadingTransition.setFromAngle(0);
loadingTransition.setToAngle(360);
loadingTransition.setInterpolator(Interpolator.EASE_IN);
loadingTransition.setCycleCount(Animation.INDEFINITE);
loadingTransition.play();
// Group text elements to create a pulsing fade effect for text
Group texts = new Group(topText, bottomText);
FadeTransition varyingOpacity = new FadeTransition(Duration.seconds( s: 2.5), texts);
varyingOpacity.setFromValue(1.0);
varyingOpacity.setToValue(0.25);
varyingOpacity.setCycleCount(FadeTransition.INDEFINITE);
varyingOpacity.setAutoReverse(true);
varyingOpacity.play();
// Event Handlers, will execute the code that was passed in, when user interacts with splash screen
scene.setOnKeyPressed( KeyEvent _ -> onContinue.run());
scene.setOnMouseClicked( MouseEvent _ -> onContinue.run());
```

SplashScreen Code 1 - More Complex Aspects of SplashScreen Code



SplashScreen Image 1

I could have used a timed delay to automatically switch to the authentication screen without needing user interaction, however I chose to use a user-initiated transition to avoid forcing the user to wait unnecessarily (which can increase frustration, when in a rush, e.g., cramming before an exam) and ensuring accessibility for users who may require more time to view the screen. Furthermore, screen switching could have been integrated within the SplashScreen class, but by separating the logic, I have improved the modularity of the program, and it will allow the splash screen to be reused in different scenarios without affecting the functionality of authentication.

Through testing the splash screen, it is the first thing that users see on launch, animations are played correctly and the screen transition is seamless and only following user interaction.

3.1.3.3 User Registration

Addressing Success Criterion 3, 4 and 5, user registration was implemented. Registration will allow new users to create account an account which will have data specific to them, persisting across multiple sessions.

Registration is triggered when all inputs are validated and use the AuthenticationController class to hash the password and then write it into users.csv.

```
// Create input field
String iconURL = "/images/user.png";
TextBundle emailInput = ui.buildTextFieldInput( rectX: 531, rectY: 345, rectW: 815, fieldX: 630, fieldY: 355, promptText: "Enter Email", iconURL, iconX: 531, iconY: 345, iconW: 95, iconH: 90);
// Create password input fields
PasswordBundle registrationPassword = ui.buildPasswordInput( rectX: 531, rectY: 458, rectW: 815, fieldX: 630, fieldY: 468, promptText: "Enter Password", iconX: 531, iconY: 463 , rconX: 1280, rconY: 483);
PasswordBundle confirmRegistrationPassword = ui.buildPasswordInput( rectX: 531, rectY: 572, rectW: 815, fieldX: 630, fieldY: 582, promptText: "Confirm password", iconX: 531, iconY: 576, rconX: 1288, rconY: 598);
// attach real-time validators to input fields
attachValidator(registrationPassword.password(), this::isValidPassword, errorClass: "error", registrationPassword.copyPw());
attachValidator(confirmRegistrationPassword.password(), this::isValidPassword, errorClass: "error", confirmRegistrationPassword.copyPw());
attachValidator(emailInput.tf(), this::isValidEmail, errorClass: "error");
// Create a question mark icon for displaying input requirements
ImageView question = ui.createBRImageView( name: "question");
```

AuthScreen Registration Code 2 - Creating the Input Fields

The input fields are created with reusable text/password field builders, allowing an icon specific to the field as well as extra icons (eyes for hiding/showing password in the password field builders). Real-time validation has been attached to each input field, providing immediate feedback to user in terms of colour connotation (red = not valid). This improves reusability as well as reducing invalid submissions prior to the user pressing the register button.

The attachValidator method attaches a listener to the input field's text property. After every update of the input, the validation rule (passed as a predicate to allow same validation logic to be reused across multiple fields) is applied. The CSS class of the field is updated depending on the result of the rule against the input to provide immediate

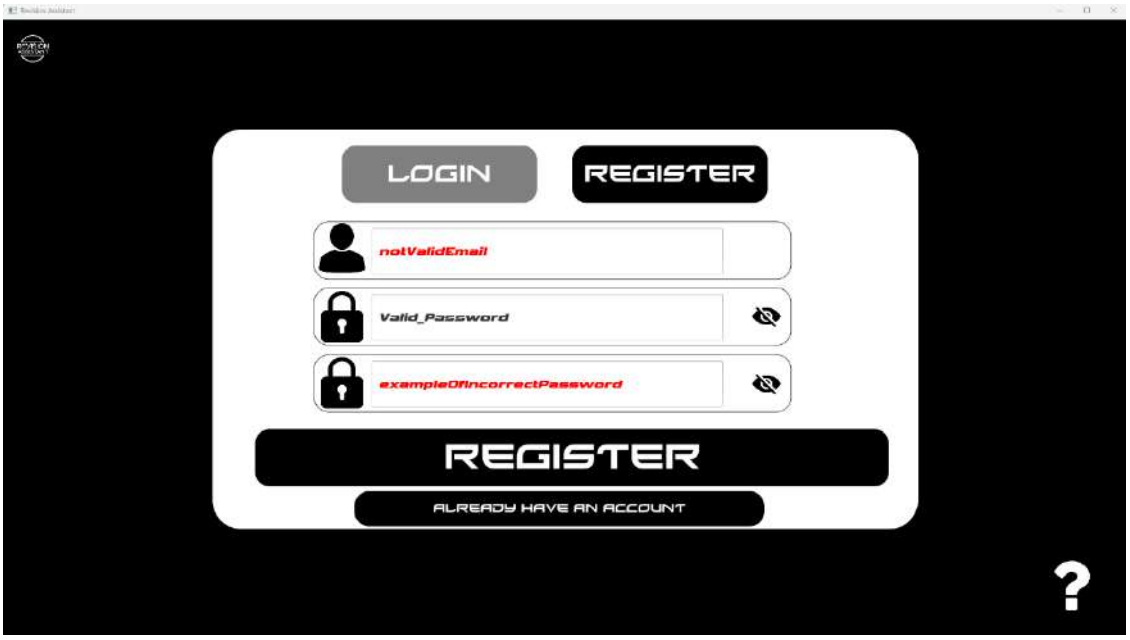
visual feedback. The validator takes extra inputs of linked fields, so the CSS style sheet updates are synchronised across all UI components representing the same input.

```
// Adds a real-time validator to input fields and error styling validation fields.
private void attachValidator(TextInputControl field, Predicate<String> rule, String errorClass, TextInputControl... linkedField) {
    field.textProperty().addListener(( ObservableValue<extends String> obs, String oldText, String newText) -> {
        boolean invalid = !rule.test(newText);
        updateStyle(field, errorClass, invalid);
        // Apply same styling to linked fields (such as visible copy of password field)
        for(TextInputControl f : linkedField){
            updateStyle(f, errorClass, invalid);
        }
    });
}
// Updates CSS styling of field depending on validation state
private void updateStyle(TextInputControl field, String errorClass, boolean invalid) {  2 usages  Aditya Sainath
    field.getStyleClass().remove(errorClass);
    if (invalid && !field.getText().isEmpty()) {
        field.getStyleClass().add(errorClass);
    }
}

// Validates email format via regex
private boolean isValidEmail(String email){  4 usages  Aditya Sainath
    if (email == null) return false;
    return email.matches( regex: "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]");
}
// Validates strength of password
private boolean isValidPassword(String pw){  8 usages  Aditya Sainath
    if(pw == null) return false;
    return pw.length()>=6 && pw.matches( regex: ".*[^a-zA-Z0-9].*");
}
// Makes sure recovery is something rememberable
private boolean isValidRecoveryW(String recoveryWord){  2 usages  Aditya Sainath
    return recoveryWord != null && recoveryWord.length() >= 3;
}
```

AuthScreen Code 1 - Real-time Validation

This will help reduce invalid data being written into the csv and will prevent unnecessary processing in the AuthenticationController class.



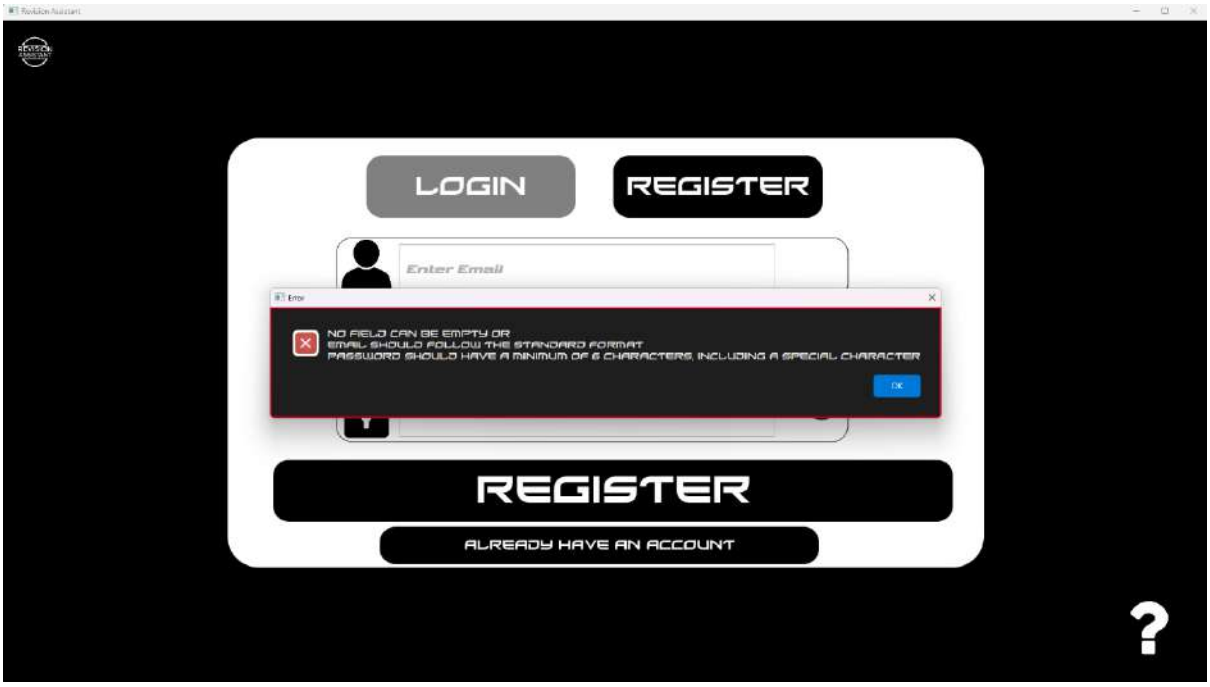
AuthScreen Image 1 - Registration Content

Using an event handler, when registration is clicked there are multiple cases:

```
// Handle registration button click
register.button().setOnMouseClicked(MouseEvent event ->{
    Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
    // Check if inputs are valid
    boolean validInputs = isValidEmail(emailInput.tf().getText()) && isValidPassword(registrationPassword.password().getText()) && isValidPassword(confirmRegistrationPassword.password().getText());
    if(!validInputs) {
        CustomAlert.error(alertWindow, message: "No field can be empty or \n" + emailReq + "\n" + passwordReq);
    }else if(!registrationPassword.password().getText().equals(confirmRegistrationPassword.password().getText())) {
        CustomAlert.error(alertWindow, message: "Passwords do not match");
    }else{
        try {
            // Check if email already exists
            if (ac.doesEmailExist(emailInput.tf().getText())) {
                CustomAlert.error(alertWindow, message: "Email already exists.");
            } else {
                // Proceed to Recovery word setup
                buildRecoveryWordEntrance(contentLayer, emailInput.tf().getText(), registrationPassword.password().getText());
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
});
```

AuthScreen Registration Code 2 - On Registration Button Click

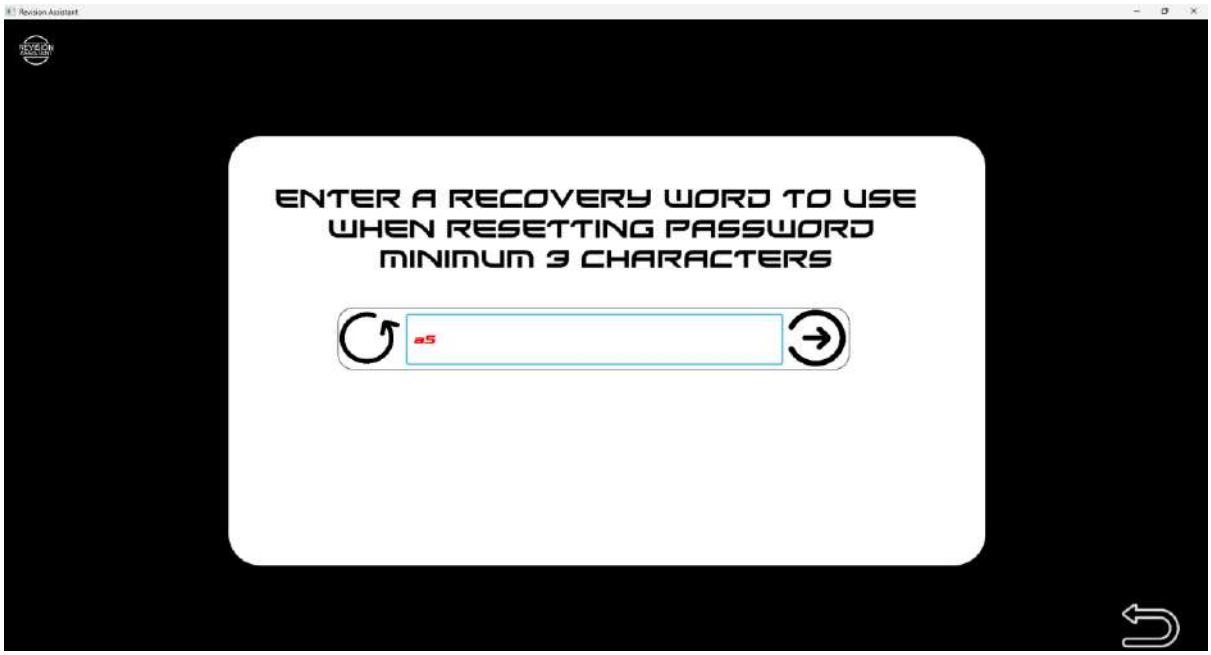
On invalid inputs the user is provided with a window which displays an error, preventing interaction with the system until the window is closed.



CustomAlert 1 - Error Example

I could have used a simple, default Alert from the JavaFX library, however with the CustomAlert class the consistency, styling control and maintainability of the alerts have been improved significantly. The Alert library offers limited customisation and would have many lines of repeated code. Through making alert creation in the utility class, all user feedback will follow the same behaviour and style. In addition, a custom style sheet was applied to ensure different alert types are distinguishable and they match the rest of the interface. With the CustomAlert class, code becomes more readable; it is must easier to read .error() or .info() than creating a new JavaFX alert object every time. Furthermore, through extracting alerts into a utility class, the modularity of code is increased and makes it more reusable.

Following the valid inputs for email, password and confirm password; the user is prompted to submit an input for a recovery word.



AuthScreen Image 2 - Recovery Word Entrance

On clicking of the back button (if user decides against logging in), with the use of the ContentNavigator class, the goBack() method is called, which sets the contentLayer to the result of popping the stack.

```
// Switches content of a layer to new nodes, saving current state to history for backtracking
public void switchContent(Group contentLayer, Node... newContent){  4 usages  Aditya Sainath
    // Save current state to history stack
    history.push(new ArrayList<>(contentLayer.getChildren()));
    // Replace all children with new content
    contentLayer.getChildren().setAll(newContent);
}
// Navigates back to previous content state
public void goBack(Group contentLayer){  3 usages  Aditya Sainath
    if(!history.isEmpty()){
        contentLayer.getChildren().setAll(history.pop());
    }
}
```

ContentNavigator Code 1 - goBack() Procedure and switchContent()

If the user does not decide against registering and enters a valid recovery word, the userRegistration(..) method will be called, which uses the AuthenticationController's Boolean function of registerUser() to write the user data into the csv, if it is valid.

```java
// Registers a user
public boolean registerUser(String email, String inpPassword, String recoveryWord) throws NoSuchAlgorithmException, IOException {
    // Check if email does not already exist within the CSV
    if (!doesEmailExist(email)) {
        // Create a new random salt, preventing identical hashes for same password
        generateSalt();
        // Hash password using SHA-512 and generated salt
        String hashedPassword = hashPassword(inpPassword, getSalt());
        // Store salt so it can be safely written into the CSV
        String saltBase64 = Base64.getEncoder().encodeToString(getSalt());
        // Create new user object
        User user = new User(email, hashedPassword, saltBase64, getDate(), recoveryWord);
        // Opens in append mode, won't overwrite the file
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toString(), append: true))) {
            bw.write( str: email + "," + hashedPassword + "," + saltBase64 + "," + user.getLastLogin() + "," + recoveryWord);
            bw.newLine();
        }
        return true;
    }else{
        // Email already in CSV
        return false;
    }
}
```

AuthenticationController Code 1 - registerUser(..) function

```java
// Hash password algorithm via SHA-512 (even though, others are better, I'll describe in write up); throws exception in case MessageDigest instance fails.
public String hashPassword(String password, byte[] salt) throws NoSuchAlgorithmException {  2 usages  Aditya Sainath
    // Create a MessageDigest instance using the SHA-512 hashing algorithm
    MessageDigest md = MessageDigest.getInstance( algorithm: "SHA-512");
    // Add the salt to the has function so same passwords can produce different hashes per user
    md.update(salt);
    // Hash the password bytes, giving a one-way hash
    byte[] hashedPassword = md.digest(password.getBytes());
    // Convert the raw hash bytes to a base 64 string so it can be stored safely in the .csv file
    return Base64.getEncoder().encodeToString(hashedPassword);
}
```

AuthenticationController Code 2 - hashPassword(..) Function

The user's input password is hashed with SHA-512 hash function. When researching hashing algorithms, I came across PBKDF2, BCrypt, SCrypt and MD5. PBKDF2, BCrypt and SCrypt are recommended since they are purposefully slow and have configurable strength to prevent brute-force attacks. For this prototype, I chose SHA-512 with a unique salt per user (so users with same password can have different hashes); this hash was done using the MessageDigest standard java library, hence it requires no external dependencies. This makes it easier to implement reliably, whilst ensuring passwords are never stored in plain text. It is also very unlikely that the RevisionAssistant's user's data file will be subject of a brute force attack, hence the choice was appropriate. In addition to this, I have also implemented in previous coding projects, meaning I could confidently implement it, reducing the likelihood of implementations errors.

3.1.3.3.1 Remedial Action (File path)

During the initial implementation of the user registration method, specifically writing data into the csv file, I attempted to access the users.csv using getResourceAsStream(). This caused an issue where the registration was a success, but when I checked the csv file during run time (and after execution), it was not updated. As I tried different file readers, the actual issue was embedded into the access of the csv file. This was because getResourceAsStream() is for reading packaged resources (images, for example) at runtime, bundled with the application.

Resources accessed in this way are read-only at runtime, so the changes made with the buffered reader did not persist.

```
//getResourceAsStream() is for reading packaged resources, not for storing/editing data
private final Path csvPath = Paths.get(System.getProperty("user.dir"), ...more: "files", "users.csv");
```

AuthenticationController Code 2 - File Access

Using a path object allows the file to be accessed reliably during runtime, ensuring that it can be read/written to by Buffered Reader/Writer. This ensures that newly registered users are stored correctly and are persistent across multiple sessions.

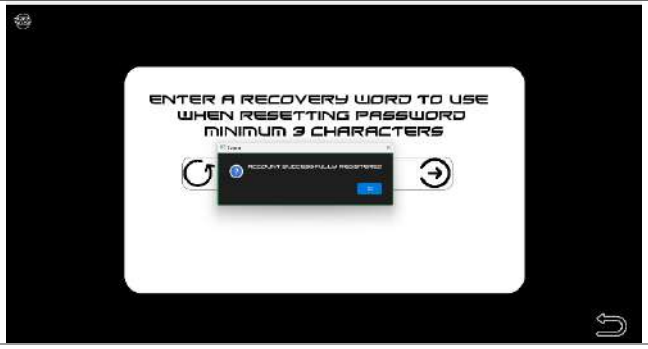
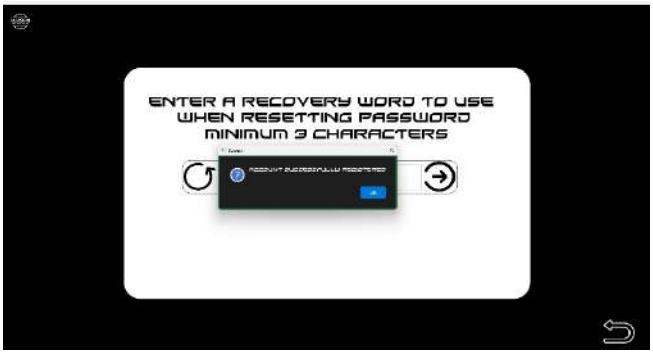
Furthermore, to prevent crashes due to corrupted files, the Revision Assistant will proactively recreate the missing file or directory, when data reading/writing is happening. This is only called in the convertCSVtoArrayList() function, since this is a reused function prior to reading and writing.

```
// Ensures parent folder and CSV exists
private void ensureCSVExists() throws IOException {  1 usage
    // Create a parent directory if it doesn't exist
    if(!Files.exists(csvPath.getParent())){
        Files.createDirectories(csvPath.getParent());
    }
    // Create CSV file if it doesn't exist
    if(!Files.exists(csvPath)){
        Files.createFile(csvPath);
    }
}
```

AuthenticationController Code 3 - Crash Prevention

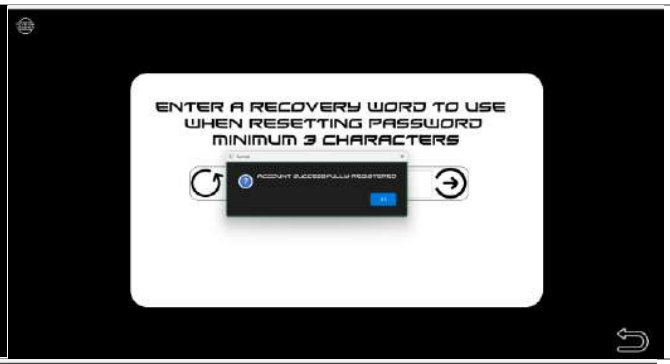
3.1.3.3.2 Testing User Registration

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	email	aditya@gmail.com	Normal	Account registered / Logged In	(With everything else correct) Account Registered	
2.	email	aditya@gmail	Erroneous	Message displayed to user this input is not possible, encouraging correction.	Account Registered	
3.	email	email which already exists	Erroneous		Corrupt error message displayed	
4.	passwords	Passwords which don't match	Erroneous	Alert saying the passwords do not match	Alert window saying passwords don't match	
5.	password	Dormers*	Normal	Accepted, will call registration/login function.	Accepted, function called, account subsequently registered	
6.	password	Ad*tya	Boundary			
7.	password	Adity	Erroneous	Rejected, ask user to input according to requirements.	Correct error message displayed	
8.	password	*"£^	Erroneous			
9.	confirmPassword	Dormers*	Normal	Accepted, will call registration/login function.	Accepted, function called	
10.	confirmPassword	Ad*tya	Boundary			
11.	confirmPassword	Adity	Erroneous	Rejected, ask user to input according to requirements.	Correct error message displayed	
12.	confirmPassword	*"£^	Erroneous			
13.	recoveryWord	School	Normal	Accepted, attempt to register.	Registration Successful	
14.	recoveryWord	Dog	Boundary			
15.	recoveryWord	Hi	Erroneous	Not accepted, can't register.	Prompted to enter longer word	

Test no.	Proof
1.	 
2.	 
3.	 
4.	 

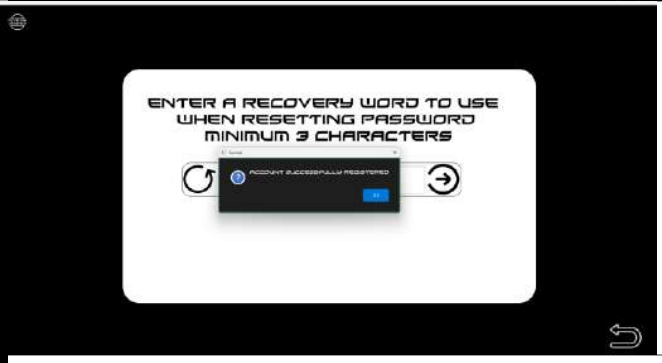
5.


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT


ENTER A RECOVERY WORD TO USE WHEN RESETTING PASSWORD MINIMUM 3 CHARACTERS

6.


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT


ENTER A RECOVERY WORD TO USE WHEN RESETTING PASSWORD MINIMUM 3 CHARACTERS

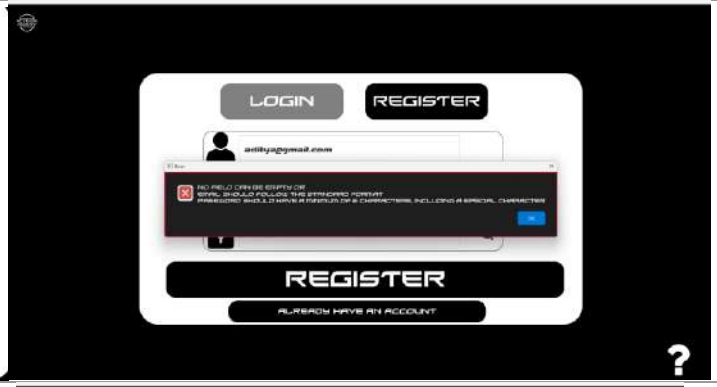
7.


LOGIN
REGISTER
aditya@gmail.com
Adity
Adity
REGISTER
ALREADY HAVE AN ACCOUNT

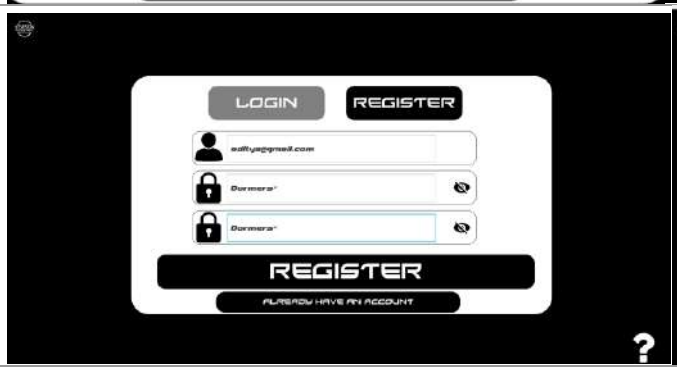

LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT

8.


LOGIN
REGISTER
aditya@gmail.com
REGISTER
ALREADY HAVE AN ACCOUNT


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT

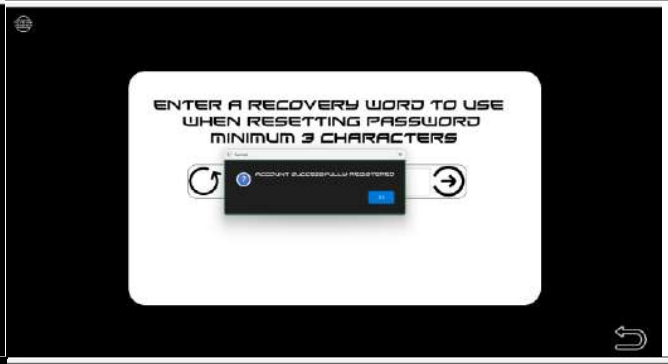
9.


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT


ENTER A RECOVERY WORD TO USE WHEN RESETTING PASSWORD MINIMUM 3 CHARACTERS

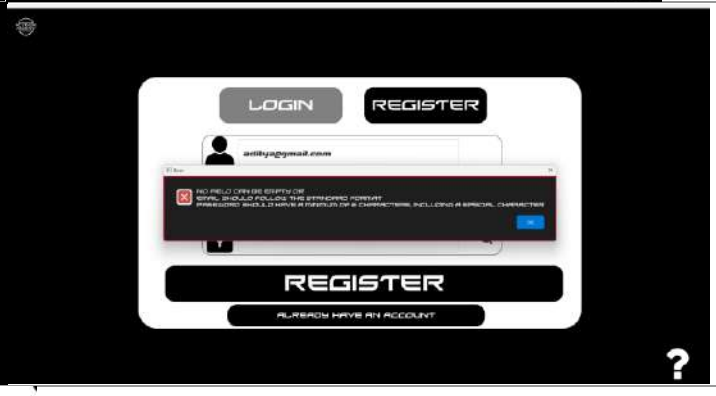
10.


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT


ENTER A RECOVERY WORD TO USE WHEN RESETTING PASSWORD MINIMUM 3 CHARACTERS

11.


LOGIN
REGISTER
adityа@gmail.com
Adity
Adity
REGISTER
ALREADY HAVE AN ACCOUNT

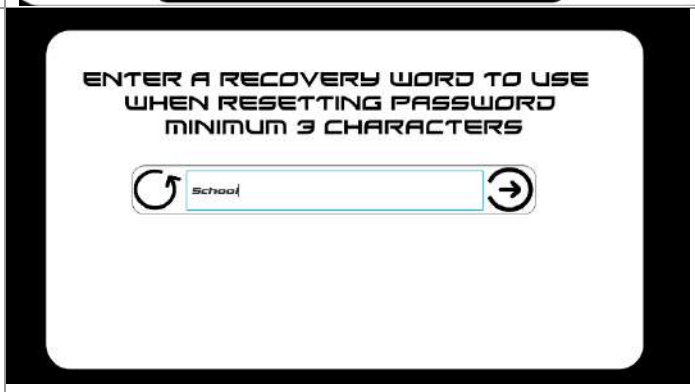

LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT

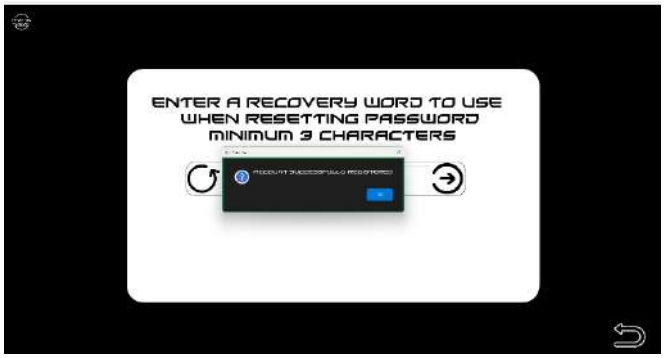
12.

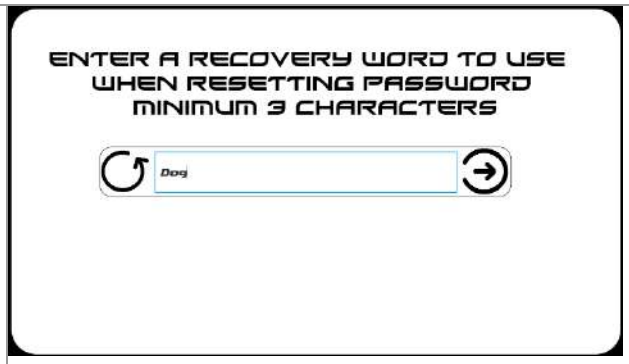
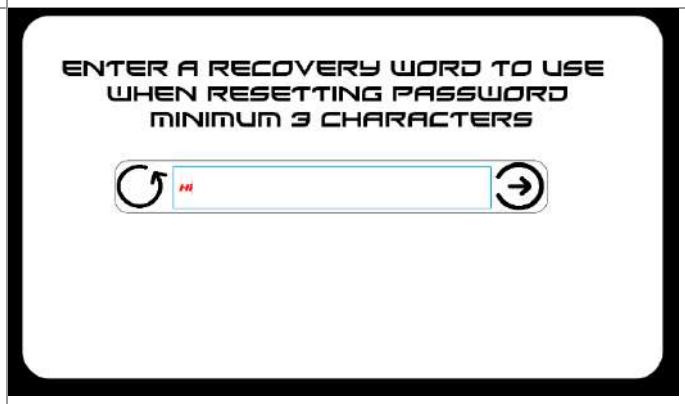

LOGIN
REGISTER
adityа@gmail.com
REGISTER
ALREADY HAVE AN ACCOUNT


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT

13.


ENTER A RECOVERY WORD TO USE WHEN RESETTING PASSWORD MINIMUM 3 CHARACTERS
School


ENTER A RECOVERY WORD TO USE WHEN RESETTING PASSWORD MINIMUM 3 CHARACTERS

14.		
15.		

3.1.3.3.3 Remedial action (Email validation)

During the tests, I identified that email validation logic allowed an invalid email format, (did not have .com, allowed emails without top-level domains). To resolve this, I have updated the validation logic from the Boolean isValidEmail(..) function. The regex was refined so that email must include a domain suffix, which contains at least 2 alphabetic characters, preventing invalid formats.

```
// Validates email format via regex
private boolean isValidEmail(String email){  4 usages  Aditya Sainath
    if (email == null) return false;
    return email.matches( regex: "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
}
// Validates strength of password  8 usages  Aditya Sainath
private boolean isValidPassword(String pw){
    if(pw == null) return false;
    return pw.length()>=6 && pw.matches( regex: ".*[^a-zA-Z0-9].*");
}
```

AuthScreen Code 2 - Remedial Action for Email Input

Updating the regex makes sure the user adds a domain to their email address, this will help in the future if I decide to improve the Revision Assistant in the future, replacing the recovery word for the forgot password feature (for example), with a generated email token.

Retesting has confirmed that “aditya@gmail” is now an invalid input. However, with the refined email validation, it does not always guarantee that all possible invalid emails are rejected. Validating email addresses to the extent of making sure they are real would require more complex methods, for example, an emailed code to confirm your account. For this project, simpler validation is chosen to prevent common user errors while keeping validation logic easy to implement. There is a trade-off between validation and maintainability of the system. The implemented validation meets the solution’s functional requirements without including unnecessary complexity.

With further tests, I noticed that you could store the same email but with different letters capitalised, hence, to fix this solution I updated the doesEmailExist(..) function to:

```
// Checks if a given email exists in the CSV
public boolean doesEmailExist(String email) throws IOException {
    ArrayList<User> users = convertCSVtoArrayList();
    for (User user : users) {
        // Will not be case-sensitive
        if (user.getEmail().equalsIgnoreCase(email)) {
            return true;
        }
    }
    return false;
}
```

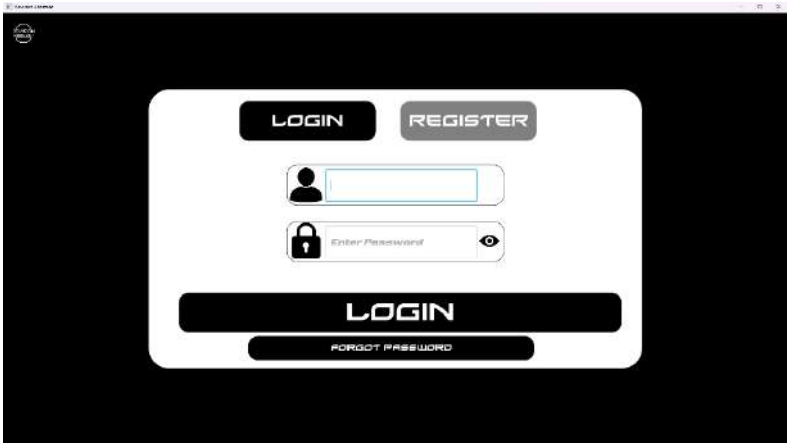
AuthenticationController Code 4 - doesEmailExist function updated

Further tests confirmed that now the same emails with different capitalised letters are not accepted. This will be important in the future for when the system is expanded to use email tokens as stated before.

3.1.3.4 User Login

To address Success Criterion 3, a login functionality has been implemented that compares user input against locally stored data.

Login content was created by the buildLoginContent() subroutine, which changes the contentLayer of the program. Login content is also the initial content that is displayed in the authentication screen



AuthScreen Login Image 1 - Login Inputs

Following valid inputs for the email and password and the login button press, the userLogin(..) function within the AuthScreen class is called, which uses an object of the AuthenticationController class to try the Boolean loginUser(..) function.

```
// Attempts to log in the user
private int userLogin(String email, String password) {  1 usage
    boolean valid;
    try{
        valid = ac.loginUser(email, password);
    } catch (NoSuchAlgorithmException | IOException e) {
        throw new RuntimeException(e);
    }
    if(valid) {
        return 1; // Login Successful
    }else{
        return 2; // Login failed
    }
}
```

AuthScreen Login Code 1 - userLogin function

```
// Logs in a user
public boolean loginUser(String email, String inpPassword) throws NoSuchAlgorithmException, IOException {  1 usage   Aditya Sainath
    // Only attempt if email is found
    if (doesEmailExist(email)) {
        // temp will hold the user we find by email
        User temp = findUserByEmail(email);
        // Check if password matches,
        // Objects.requireNonNull(temp) prevents a NullPointerException without explanation, (if null, it throws immediately)
        return verifyPassword(inpPassword, Objects.requireNonNull(temp).getHashedPassword(), temp.getSaltBase64());
    }
    // Email not found
    return false;
}
```

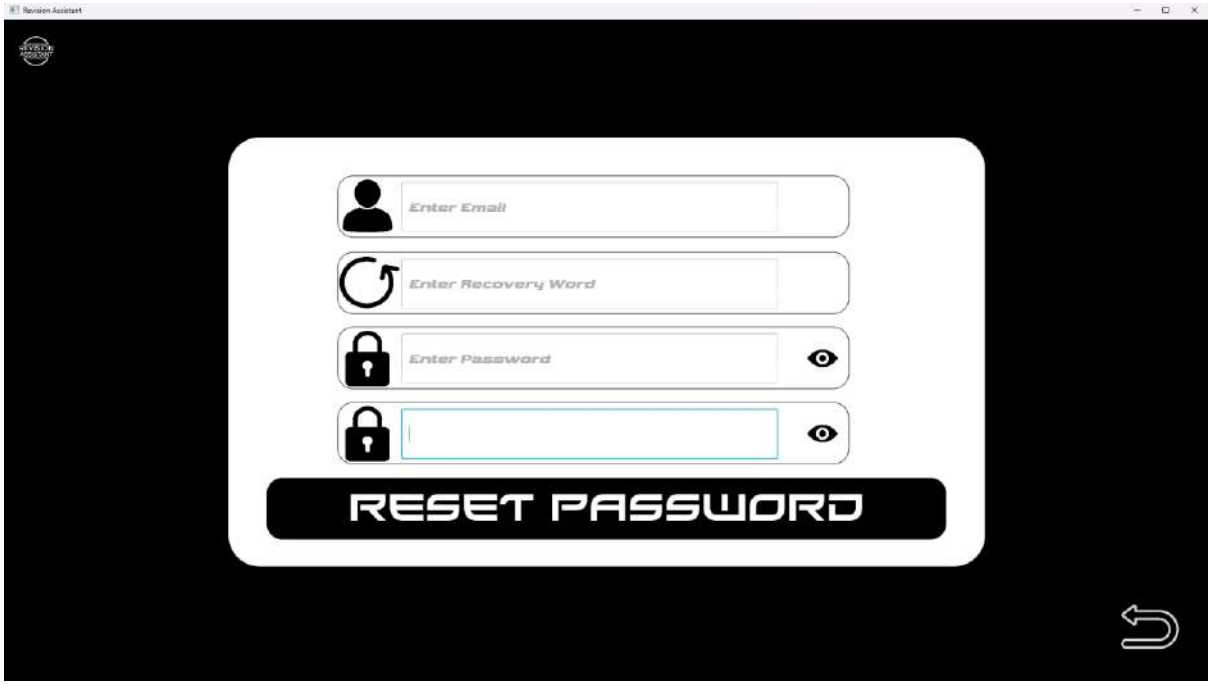
AuthenticationController Code 5 - User Login Function

```
public boolean verifyPassword(String inpPassword, String hashedPassword, String saltBase64) throws NoSuchAlgorithmException {
    // Convert the string salt to a byte array
    byte[] salt = Base64.getDecoder().decode(saltBase64);
    // Create a MessageDirect instance using the same hashing algorithm as before
    MessageDigest md = MessageDigest.getInstance( algorithm: "SHA-512");
    // Add the same salt which the hash password algorithm used
    md.update(salt);
    // hash the input password into a byte array
    byte[] hashedInputPassword = md.digest(inpPassword.getBytes());
    // convert the byte array into a string
    String newHash = Base64.getEncoder().encodeToString(hashedInputPassword);
    // true if the hashed input matches the hashed password
    return newHash.equals(hashedPassword);
}
```

AuthenticationController Code 6 - Password Verification

The login system will rehash the user's input password with the stored salt, comparing the two results to verify the password. I chose this approach since password hashes are one way and cannot be decrypted, so the original password must never be recovered. I could have stored the passwords in plain text or encrypted and decrypted them but both options increase security risks. Rehashing the input password ensures credentials are easily verified without revealing sensitive data. For this verification method, the salt must be stored alongside other user data in the csv file.

If the user wanted to reset their password, the login content is replaced with forgot password content (on the click of the forgot password button), displaying:



AuthScreen Login Image 2 - Reset Password

If the inputs are all valid, on the click of the Reset Password, (with the use of the AuthenticationController object) the resetPassword function is called.

When the user attempts to reset their password, the function will generation a new salt and hash the new password before storing it. This makes it so if the user has chosen a similar password to before, it would be a completely different hash, increasing the security of the storage.

To make these changes persistent, the function calls convertCSVtoArrayList, which returns an ArrayList with all users. Using a count-controlled loop, each user is iterated through until the one with the matching email is found, using the setter methods to update the user data. All these user objects are then rewritten into the csv, overwriting the old data; this prevents the risk of data corruption, guaranteeing that the updated row is stored correctly.

```
// Reset user's password
public boolean resetPassword(User temp, String newPassword) throws NoSuchAlgorithmException, IOException {  1 usage   Aditya Sainath
    // new salt ensures old hash cannot be used and increases security
    generateSalt();
    String hashedPassword = hashPassword(newPassword, getSalt());
    String saltBase64 = Base64.getEncoder().encodeToString(getSalt());
    // Load all users and update the matching one
    ArrayList<User> users = convertCSVtoArrayList();
    for(User u : users){
        if(u.getEmail().equals(temp.getEmail())){
            u.setPasswordHash(hashedPassword);
            u.setSaltBase64(saltBase64);
            break;
        }
    }
    // Rewrite the entire file, so updates persist
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toString()))) {
        for (User u : users) {
            //DEBUGED VIA
              String line = u.getEmail() + "," + u.getHashedPassword() + "," + u.getSaltBase64() + "," + u.getLastLogin() + "," + u.getRecoveryWord();
              bw.write(line);
              System.out.println(line);
            // writes row back into file
            bw.write( str: u.getEmail() + "," + u.getHashedPassword() + "," + u.getSaltBase64() + "," + u.getLastLogin() + "," + u.getRecoveryWord());
            bw.newLine();
        }
    }
    return true;
}
```

AuthenticationController Code 7 - Reset Password Function

Initially the hashed password was not updated within the csv during runtime. In order to debug this, I tried to output each update of the row into the console, seeing data was written into the file. This isolated the issues, making it obvious the updating of data was the source of errors. I checked the User class setter methods and noticed:

3.1.3.4.1 Remedial Action (Setter Method)

The String hashedPassword was always set to the existing hashedPassword, instead of the parameter passwordHash, hence, to remediate this error I updated the setter method accordingly.

```
    // ERROR
//     public void setPasswordHash(String passwordHash){
//         this.hashedPassword = hashedPassword;
//     }
```

User Code 1 - Setter method error

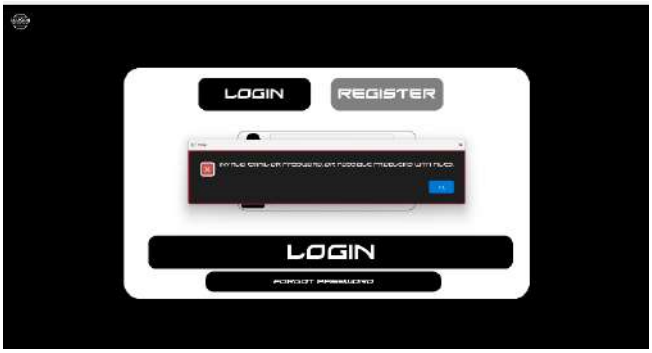
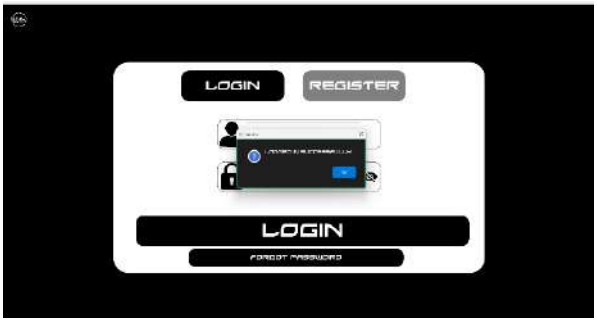
The CSV was then updated with the new hashed password, and I could log in with the new password.

```
public void setPasswordHash(String passwordHash){
    this.hashedPassword = passwordHash;
}
```

User Code 2 - Remediated Setter Method

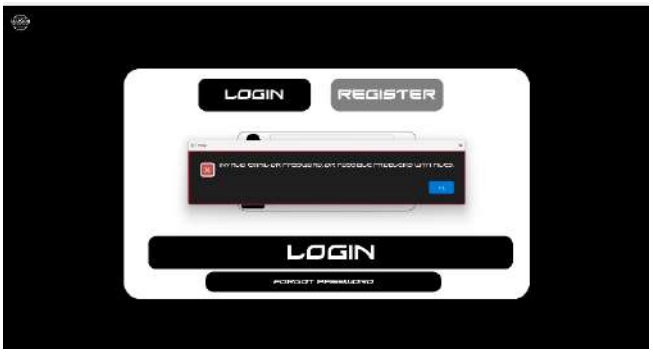
3.1.3.4.2 Testing User Login

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	email (with correct password)	aditya@gmail.com	Normal	Account registered / Logged In / Accepted	Logged in taken to main menu screen.	
2.	email	aditya@gmail	Erroneous	Message displayed to user this input is not possible, encouraging correction.	Not a valid email alert.	
3.	password (with correct email)	Dormers*	Normal	Accepted, will call registration/login function.	Logged in taken to main menu screen.	
4.	password	Dormers**	Erroneous	Password does not match, alert shown to user	Error shown to user.	
5.	password	*”£^	Erroneous	Rejected, ask user to input according to requirements.	Input not accepted	
6.	confirmPassword	Dormers*	Normal	Password reset correctly.	Password reset	
7.	confirmPassword	Ad*tya	Boundary			
8.	confirmPassword	Adity	Erroneous	Rejected, ask user to input according to requirements.	Input not accepted	
9.	confirmPassword	*”£^	Erroneous			
10.	passwords	Do not match	Erroneous	Rejected, alert shown.	Alert shown, rejected	
11.	recoveryWord	School	Normal	Accepted, attempt to register, or reset password.	Password reset successfully	
12.	recoveryWord	Dog	Erroneous	Recovery word does not match, not accepted.	Recovery word does not match error.	
13.	recoveryWord	Hi	Erroneous	Not accepted, can’t reset.	Input accepted, there was no validator	

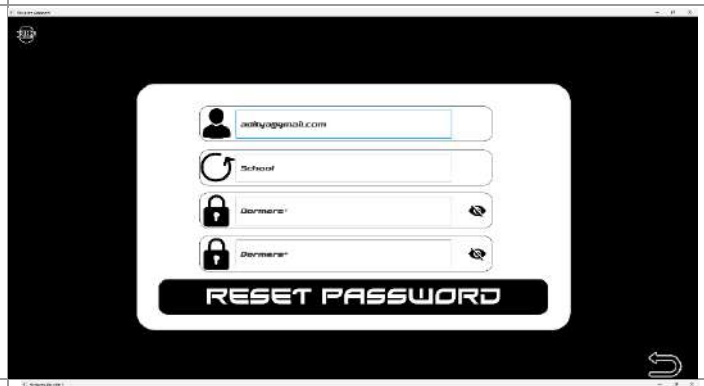
Test No.	Proof
1.	 
2.	 
3.	 
4.	 

5.


LOGIN
REGISTER
LOGIN
FORGOT PASSWORD

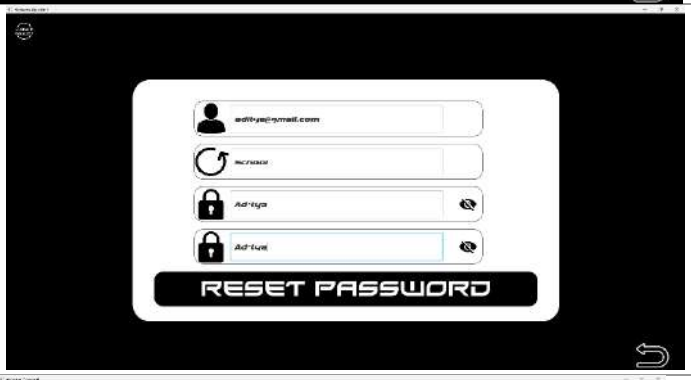

LOGIN
REGISTER
LOGIN

6.


RESET PASSWORD


RESET PASSWORD

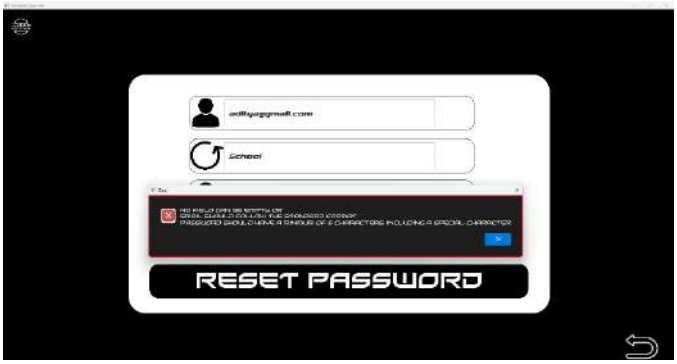
7.


RESET PASSWORD

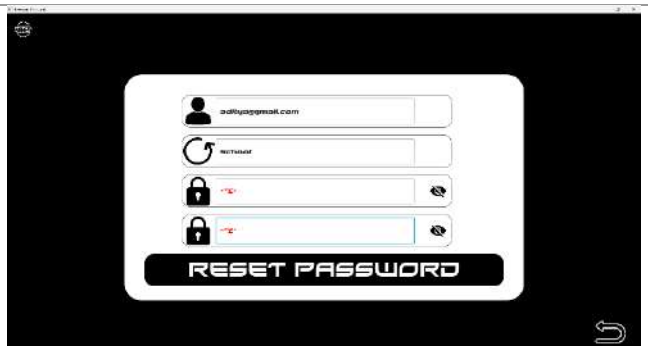

RESET PASSWORD

8.


RESET PASSWORD

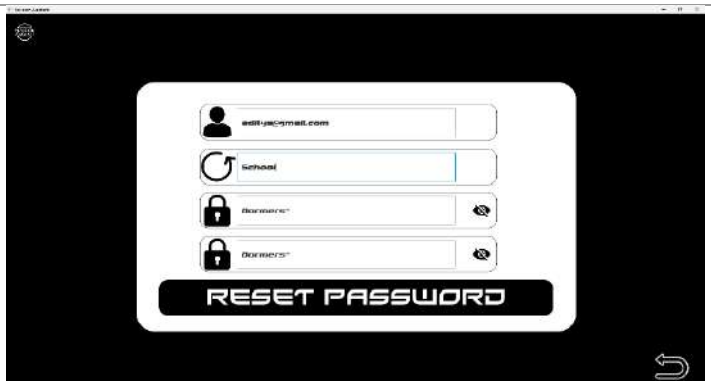

RESET PASSWORD

9.


RESET PASSWORD


RESET PASSWORD

10.


RESET PASSWORD

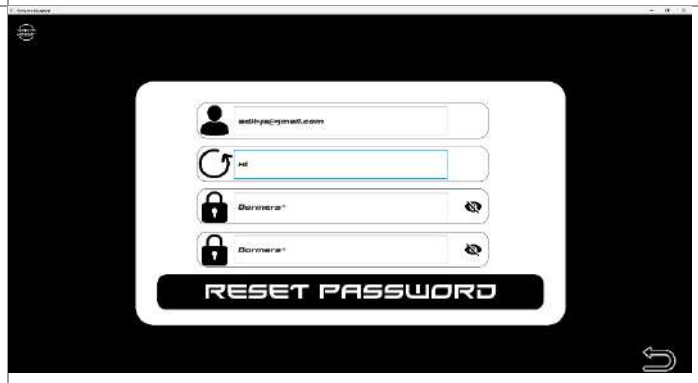

RESET PASSWORD

11.


RESET PASSWORD


RESET PASSWORD

12.


RESET PASSWORD

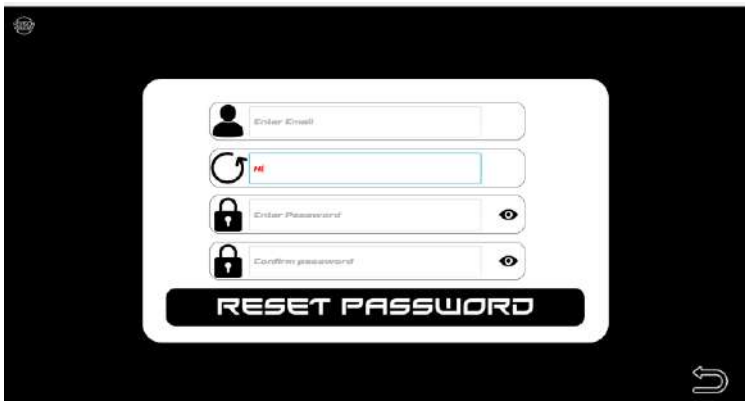

RESET PASSWORD

3.1.3.4.3 Remedial Action (Adding validator to the RecoveryWord Input)

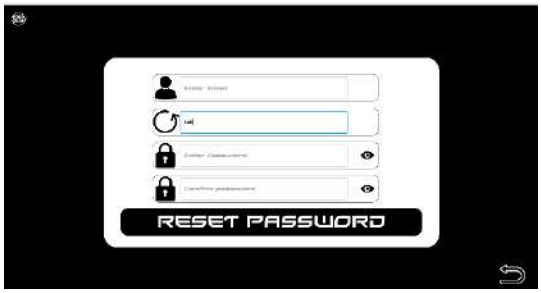
```
attachValidator(recoveryWord.tf(), this::isValidRecoveryW, "error");
boolean validInputs = isValidEmail(emailInput.tf().getText()) &&
isValidPassword(resetPassword.password().getText()) &&
isValidPassword(confirmResetPassword.password().getText()) &&
isValidRecoveryW(recoveryWord.tf().getText());
```

AuthScreen Login Code 2 - Attaching Validator to RecoveryWord in Reset Password Input

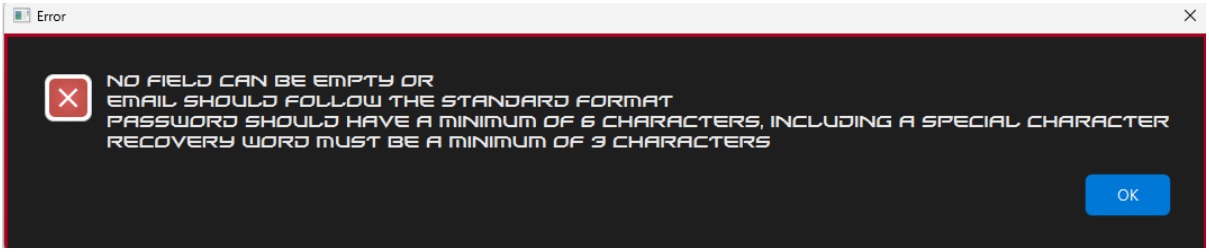
This validator will make sure the entered recovery word is checked in real time, changing colour depending on the user's input (if it is less than 3 characters, it will appear red, otherwise black). This also ensures only valid inputs are accepted, which will reduce unnecessary processing done with the AuthenticationController, improving the overall efficiency of the system.



AuthScreen Reset Password Image 1 - RecoveryWord Visual Feedback



AuthScreen Reset Password Image 2 - Visual Feedback



AuthScreen Reset Password Image 3 Error Message

Further tests have shown that the validator now works with the recovery word and will not accept invalid inputs.

3.1.3.5 Data Storage Requirements

To address Success Criterions 4 and 5, I will observe the contents of the csv; I am expecting:

- Each email to be unique.
- No passwords stored in plain text.

Following these observations (if they are correct), I will:

1. Delete the csv file and see how the system responds
2. Corrupt the csv file to see how the system responds

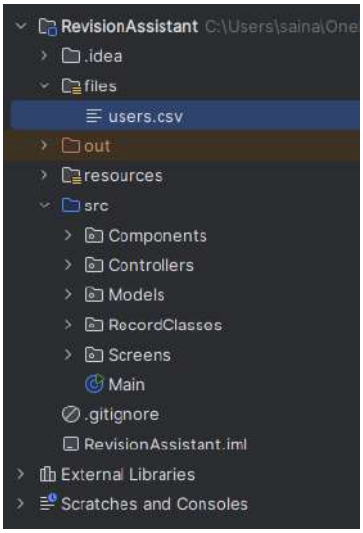


Users Data File 1 - users.csv Content

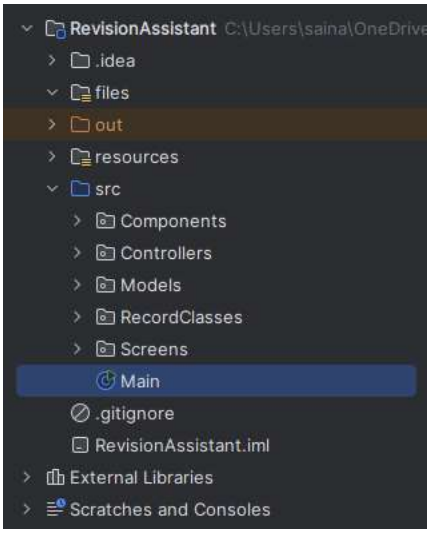
Looking into the CSV file, each email, hashed password and salt are unique. All emails are of the correct format and recovery words are longer than 3 characters.

3.1.3.5.1 Testing Stored Data (Destruction Testing)

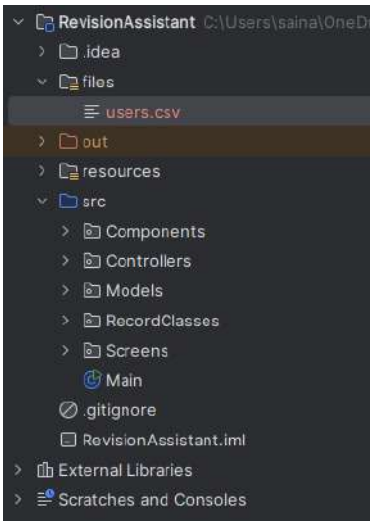
I will now delete (in my case refactor, since I do not want to lose the users.csv data file, so it is not within access of the system) the csv file and attempt to register + login to see how the system reacts.



Project Structure Image 5 - Project Structure Prior to Refactoring



Project Structure Image 6 - users.csv Refactored



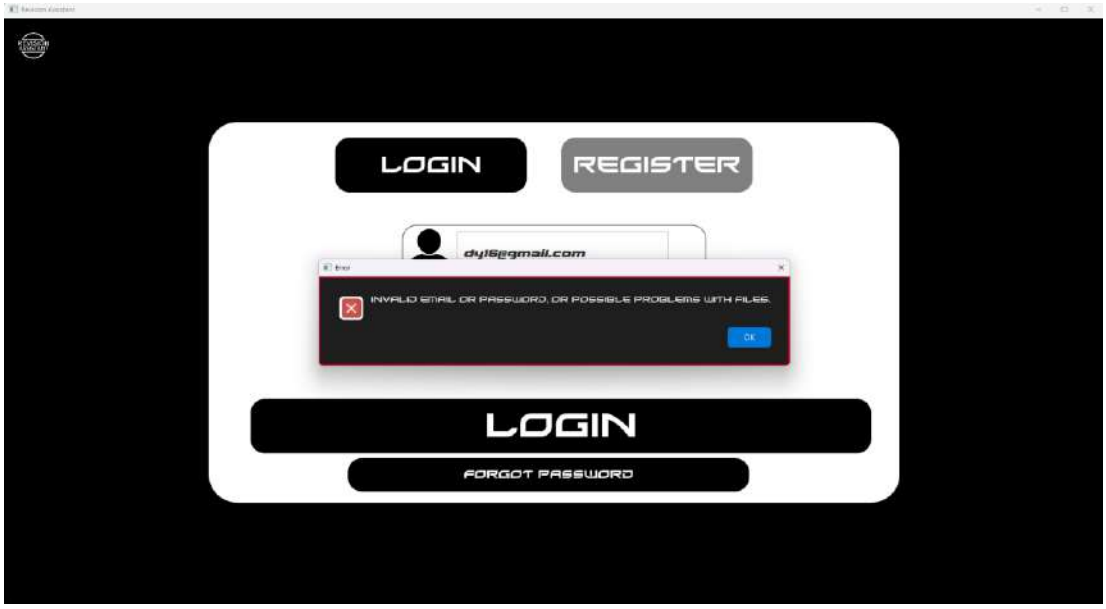
Project Structure Image 7 - Project Structure Post User Registration

The system automatically creates a new users.csv file and will use this as the new storage file for users. The contents of this file currently are only the new registered user.

I attempted to login with these details and details in the prior users.csv, the system responded with:



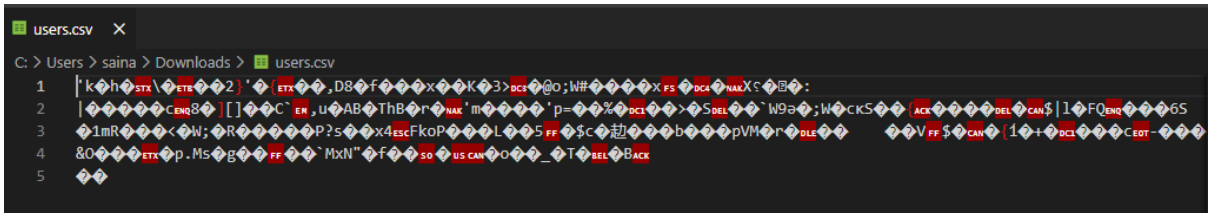
CustomAlert 2 - Successful Login



CustomAlert 3 - Unsuccessful Login

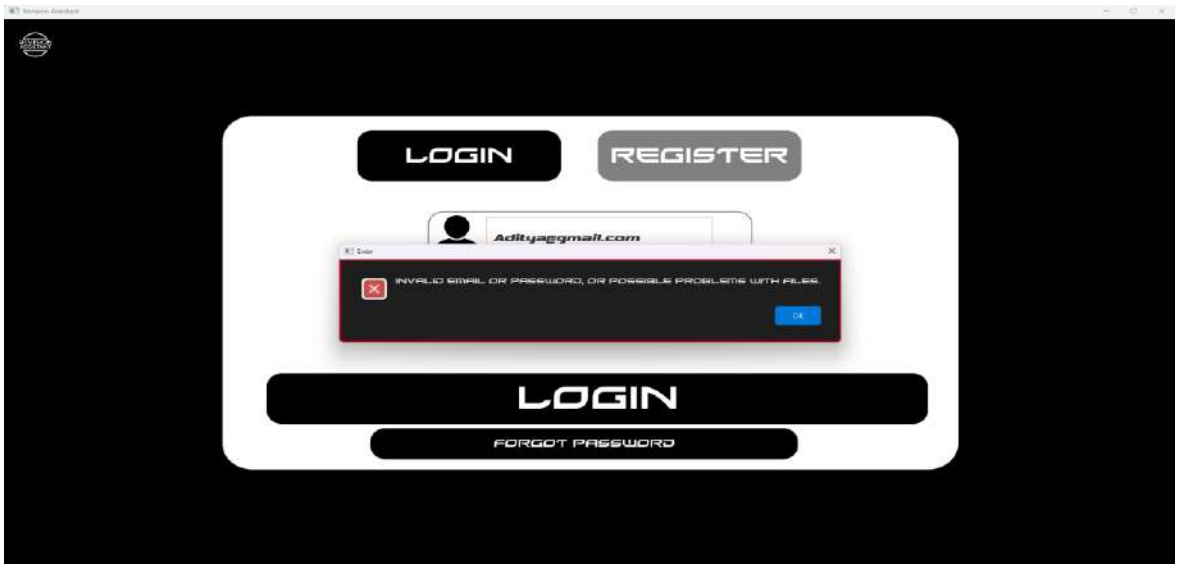
The system now uses the newly created users.csv as the file storing all user authentication information, losing previous logins. This could be a frustration to users; however, it is unlikely that the file will corrupt / be deleted. In the case of this event, user data will be separated from the authentication files, so users can link their new accounts to their former data, if needed.

I will now corrupt the file using (https://corrupt-a-file.net/) to see how the system responds.



Users Data File 2 - Corrupted users.csv

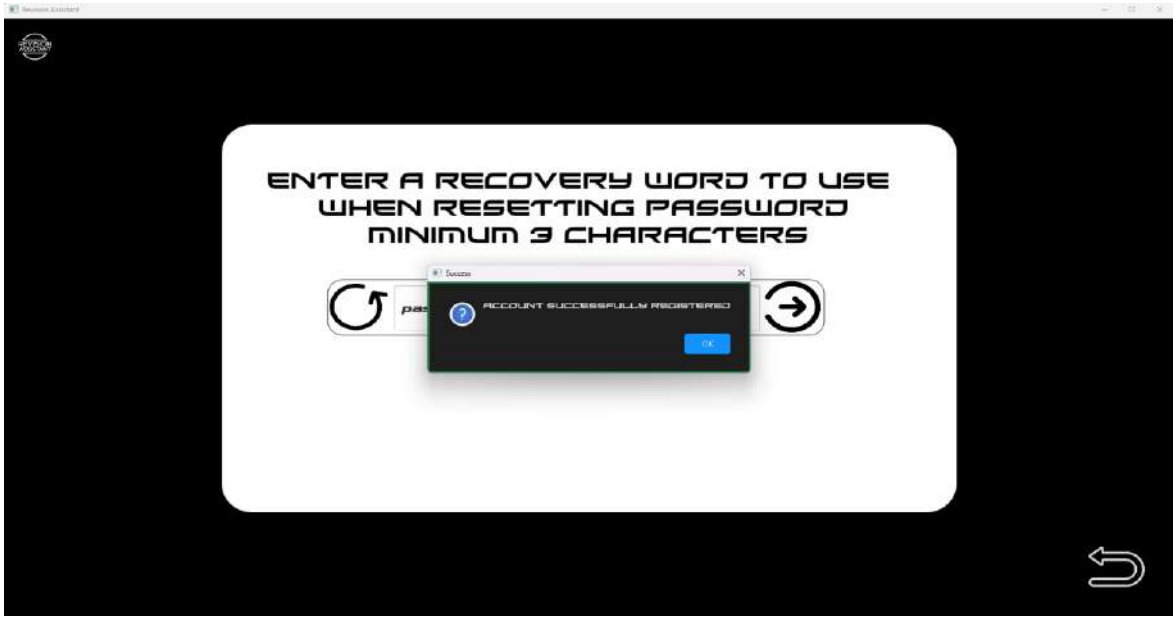
Attempting login with previous data:



AuthScreen Image 2 - Login Attempt on Corrupted CSV

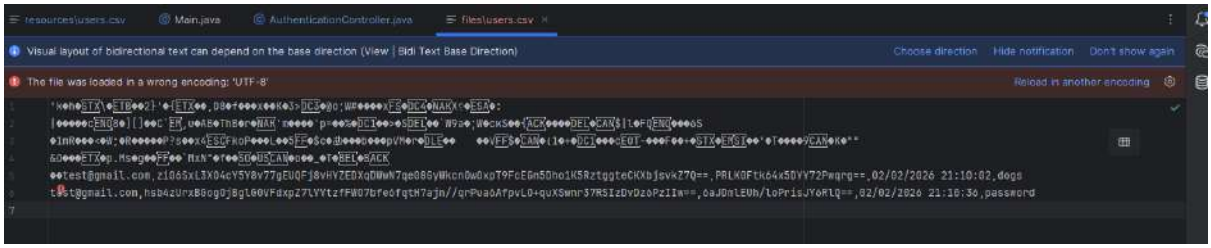
Login was unsuccessful since the data within the file cannot be interpreted.

Attempting registration:



AuthScreen Image 3 - Registration Attempt on Corrupted CSV

Registration was a success; however, I attempted registration with the same email again and it was successful, which means validation is not working correctly, or a problem with a CSV



Users Data File 3 - Corrupted CSV with new Registration Content

3.1.3.5.2 Remedial Action (Handling Corrupted Files)

When the data file is corrupted, existing record within the system cannot be reliably parsed, thus duplicate email validation may not function all the time. The system prioritises recovery and availability of user files over strict validation, allowing invalid data to be inputted and written in the csv. To prevent the system from doing this in the future, I will add a method which will quarantine the corrupted file (if detected) and create a new file for user data.

```
// Reads users.csv and converts each row into a user object
public ArrayList<User> convertCSVtoArrayList() throws IOException {  3 usages  Aditya Sainath
    // Ensure folders and CSV exist before reading
    ensureCSVExists();
    ArrayList<User> users = new ArrayList<>();
    // Read CSV line by line
    try(BufferedReader br = new BufferedReader(new FileReader(csvPath.toString()))) {
        String line;
        while((line = br.readLine())!=null){
            String[] temp = line.split( regex: ",");
            // Validate row lengths to avoid ArrayIndexOutOfBoundsException, throw an exception
            if (temp.length < 5) {
                throw new IllegalArgumentException("Corrupted Row: " + line);
            }
            // Create a Models.User from CSV columns
            User newUser = new User(temp[0], temp[1], temp[2], temp[3], temp[4]);
            users.add(newUser);
        }
    } catch (Exception e) {
        quarantineCorruptedCSV(e);
        // Act as if there are no users if corrupted.
        return new ArrayList<>();
    }
    return users;
}
```

AuthenticationController Code 3 - updated convertCSVtoArrayList()

```java
// Quarantining Corrupted files, creating a replacement
private void quarantineCorruptedCSV(Exception cause) throws IOException {  1 usage  Aditya Sainath
    // Ensure parent folder exists (do NOT recreate the file before moving it)
    if (!Files.exists(csvPath.getParent())) {
        Files.createDirectories(csvPath.getParent());
    }
    // Only quarantine if the corrupted file actually exists
    if(Files.exists(csvPath)){
        // Generate a timestamp so each corrupted back-up has a unique filename
        String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("ddMMyyyy_HHmmss"));
        // Create a new path in same directory
        // Keep the corrupted file for recovery or debugging
        Path backup = csvPath.resolveSibling( other: "user_corrupted" + timestamp + ".csv");
        // Move corrupted file to back up location
        // REPLACE_EXISTING ensures there is no conflict
        Files.move(csvPath, backup, StandardCopyOption.REPLACE_EXISTING);
    }
    // Create an empty users.csv so the system can continue
    Files.createFile(csvPath);
}
```

AuthenticationController Code 4 - quarantineCorruptedCSV() Procedure

At first attempt to see how the quarantine procedure works, the console output was:

```
WARNING: Use --enable-native-access=javafx.graphics to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

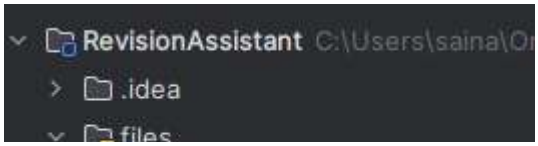
BAD ROW: 'k◆h◆Ø\◆Ø◆◆2}'◆{Ø◆◆,D8◆f◆◆◆x◆◆K◆3>Ø◆@◆;W#◆◆◆◆xØ◆Ø◆ØX◆◆Ø◆:
BAD ROW: |◆◆◆◆◆cØ8◆][]◆◆C`Ø,u◆AB◆ThB◆r◆Ø'm◆◆◆◆'p=◆◆%◆Ø◆◆>◆SØ◆◆`W9a◆;W◆cKS◆◆{Ø◆◆◆◆Ø◆Ø$|l◆FQØ◆◆◆6S
BAD ROW: ◆1mR◆◆◆<◆W;◆R◆◆◆◆◆P?s◆◆x4ØFkoP◆◆◆L◆◆5Ø◆$c◆劫◆◆◆b◆◆◆pVM◆r◆Ø◆◆    ◆◆VØ$◆Ø◆{1◆+◆Ø◆◆◆cØ-◆◆◆F◆◆+◆Ø◆ØØ◆◆'◆T◆◆◆◆9Ø◆K◆''
BAD ROW: &Ø◆◆◆Ø◆p.Ms◆g◆◆Ø◆◆`MxN"◆f◆◆Ø◆ØØ◆o◆◆_◆T◆Ø◆BØ
BAD ROW: 'K◆h◆Ø\◆Ø◆◆2}'◆{Ø◆◆,D8◆f◆◆◆x◆◆K◆5>Ø◆@◆;W#◆◆◆◆xØ◆Ø◆ØX◆◆Ø◆:
BAD ROW: |◆◆◆◆◆cØ8◆][]◆◆C`Ø,u◆AB◆ThB◆r◆Ø'm◆◆◆◆'p=◆◆%◆Ø◆◆>◆SØ◆◆`W9a◆;W◆cKS◆◆{Ø◆◆◆◆Ø◆Ø$|l◆FQØ◆◆◆6S
BAD ROW: ◆1mR◆◆◆<◆W;◆R◆◆◆◆◆P?s◆◆x4ØFkoP◆◆◆L◆◆5Ø◆$c◆劫◆◆◆b◆◆◆pVM◆r◆Ø◆◆    ◆◆VØ$◆Ø◆{1◆+◆Ø◆◆◆cØ-◆◆◆F◆◆+◆Ø◆ØØ◆◆'◆T◆◆◆◆9Ø◆K◆''
BAD ROW: &Ø◆◆◆Ø◆p.Ms◆g◆◆Ø◆◆`MxN"◆f◆◆Ø◆ØØ◆o◆◆_◆T◆Ø◆BØ
```

Console Output 1 - Testing corrupted CSV

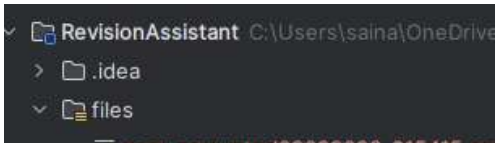
This means the bad rows are being recognised, however because I have coded it to continue after a bad row, there is no exception thrown and the buffered reader continues to read until the end of file, to fix this issue I will update the if statement within the convertCSVtoArrayList() function to:

```java
// Validate row lengths to avoid ArrayIndexOutOfBoundsException, throw an exception
if (temp.length < 5) {
    throw new IllegalArgumentException("Corrupted Row: " + line);
}
```

AuthenticationController Code 5 - convertCSVtoArrayList() Row Validation Update



Project Structure Image 4 - Prior to Execution with Corrupted File



Project Structure Image 3 - Post Execution with Corrupted File

With the use of the quarantine method, when an invalid row is detected within the user authentication data file, it will be quarantined with its name changed for later inspection. The system will create a new users.csv to start to use. This now ensures the application does not crash when data is corrupted, allowing normal operations to resume immediately. With further tests I can confirm, corrupted files are isolated and the registration and login functionality works correctly alongside its necessary validation.

3.1.4 Evaluation of Prototype 1

Prototype 1 aimed to successfully implement the initial user interface alongside the user authentication system, confronting Success Criterions 2, 3, 4 and 5. I successfully implemented a splash screen and authentication screen, meeting user interface requirements. Testing has confirmed the system allows multiple users to register and log in to the system on the same device, with their credential stored securely and passwords hashed via SHA-512 (not stored in plain text). In addition, testing has also reinforced the robustness of local data storage, the Revision Assistant can handle missing and corrupted files seamlessly, without crashing.

Overall, prototype 1 has met the criteria set. The next module of my application I am going to address is the Main Menu screen and following onto Settings.

3.2 Prototype 2: Main Menu and Settings Screen

3.2.1 Improvements from Prototype 1

While developing prototype 2, there were improvements to logic and design which included parts of prototype. These improvements focussed on ensuring the system is more maintainable and scalable for the future.

1. Improved Resource Organisation
    a. Resource folder was restructured to improve clarity and modularity. For example, assets such as images were refactored into sub-directories corresponding to their individual screens, and re-used icons were refactored into a dedicated sub-directory. This reduced the clutter of files, ensuring debugging surrounding resources is simpler.
2. Controller Superclass
    a. In prototype 1, since the only controller which existed was the AuthenticationController but since I created the SettingsController class and some code was repeated, I added a Controller superclass which both controllers (sub-classes) inherit from, reducing the chances of code being repeated.
3. Validation Checks moved to UIComponents Class

a. Since the validatory checks (including real-time validation) have been re-used within the settings screen to change the password, email and/or recovery word of an account, to reduce code from being repeated, I moved it to the UIComponents class so it can be re-used.

3.2.2 Purpose of Prototype 2

The purpose of this prototype was to extend the foundational user interface which was developed in prototype 1. The aim was to implement a Main Menu Screen to act as the central navigation hub to the system (as designed in 2.2.2.3 Main Menu), and subsequently a Settings Screen with full functionality displaying multiple options to the user to personalise their experience in the Revision Assistant and cater to their needs or disabilities, increasing accessibility. The functionality of the Main Menu Screen will increase with further prototypes since not all options are possible as of prototype 2.

3.2.3 Success Criteria for Prototype 2

Referencing (1.4.3 Success Criteria), with this prototype I will be aiming to achieve criterions 5, 6 and 11 which revolve around the accessibility and customisation features of the system. This will ensure the Revision Assistant can be personalised and catered to the user's needs to improve usability.

6) The program will use a high contrast black and white style sheet for users with colour blindness, and will have other options available (dark mode, light mode, etc); changes must apply instantly.
11) The program must have volume sliders to turn on and off sound effects.

I will aim to achieve the following criteria by meeting the following sub-criteria:

3.2.3.1 Theme Criteria

- Multiple CSS stylesheets are implanted and stored in the organised resource folder.
- Changes to theme are possible through the settings screen and when a theme is selected, the current stylesheet will be replaced, and the new style sheet will be applied immediately to all the user interfaces.
- No restart required for changes.

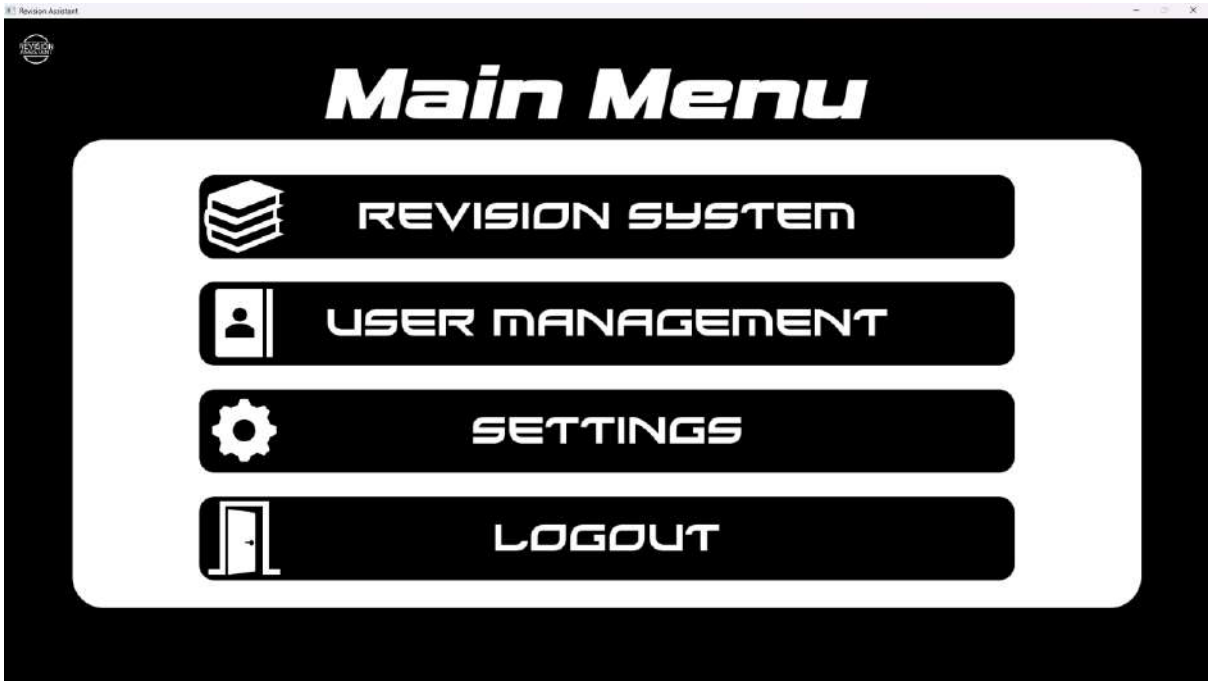
3.2.3.2 Audio Control Criteria

- Sliders will be implemented and will be functional, giving a preview of the audio volume when it is released.
- There will be options to input the value instead of using the slider.
- Setting the slider to zero will toggle the mute sound effects.
- Volume changes will be applied immediately.
- Selected volume will persist throughout runtime.

3.2.4 Developing Prototype 2

3.2.4.1 Post Login and Main Menu Screen

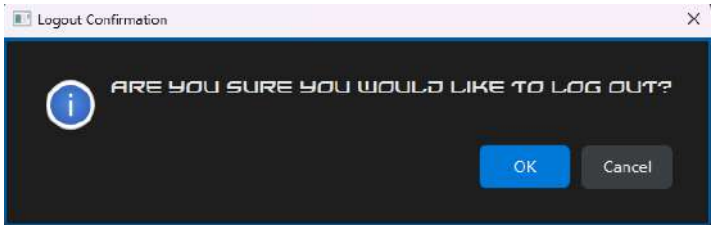
Initially post-authentication the scene is switched to the Main Menu screen where a new Main Menu object is initialised and switched to.



Main Menu Screen Image 1- Actual Screen

Replicating the interface design created in 2.2.2.3 Main Menu, I then coded the functionality for the Logout and Settings.

The Main Menu screen, and other screens (post authentication) will inherit from the SystemScreens superclass which contains the background rectangle which each screen has, as well as the ContentNavigator and UIComponents object to be used throughout screen classes post authentication. This allows the same code to be reused rather than having multiple instances of the same code. This superclass improves scalability as future screens can be easily added, inheriting all the public attributes and methods.



When logout is selected the user is given an alert and asked to confirm the logout.

Main Menu Screen Image 2 - Logout Confirmation Alert

If cancel is selected and clicked, the alert will close and the user will be returned with no effect to anything in the system, resuming program use as usual.

If “OK” is selected, the authentication controller’s logout procedure is called.

```
private void logout(Event e) {  1 usage   Aditya Sainath
    Window alertWindow = ((Node) e.getSource()).getScene().getWindow();
    if(CustomAlert.confirm(alertWindow,  title: "Logout Confirmation",  message: "Are you sure you would like to log out?")){
        ac.logout();
    }
}
```

Main Menu Screen Code 1 - Function called on logout button press

```
// System Logout
public void logout(){  1 usage   Aditya Sainath
    if (getCurrentUser() != null){
        getCurrentUser().setLastLogin(getDate());
    }
    // add a save data method, will be added when more data as been added to system
    // System has no current user, so data cannot be accessed
    setCurrentUser(null);
    // User cannot find a way to go back into Main Menu after logging out
    ScreenManager.clearHistory();
    // Switch to Log in/Registration Screen
    SettingsController sc = new SettingsController();
    setSettingsControllertoSM(sc);
    ScreenManager.switchScene(new AuthScreen(new AuthenticationController(), sc).create());
}
```

AuthenticationController Code 5 - Logout Procedure

Following confirmation of logout, the last login date is set (for future statistics), the current user is set to null, and the scene stack is cleared (so users cannot find a new way to go back into the system post-logout). The user is then returned to the login side of the Authentication Screen.

Within this prototype, a new Settings class (in the models’ package) was created, which is used for the settings throughout the program.

```
// On register the system's current user will be set to the temp user
// Settings will be loaded and the SettingsController set into the ScreenManager class
setCurrentUser(user);
sc.createDefaultSettings(email);
setCurrentSettings(sc.loadSettings(email));
setSettingsControllertoSM(sc);
```

AuthenticationController Code 6 - Code added to registration function

Following the registration of a new account, the default settings of the user is written into Settings.csv alongside their email to use as an identifier. This can be viewed as databases, where the email (primary key) in the user table is used a foreign key in the settings table.

I could have used the same file to store settings alongside user data, however, to improve modularity, reliability and scalability, I chose to separate them this is because:

- In the event of corruption one file, the other file will remain unaffected, reducing the possibility of the system having a single point of failure.
- Moreover, in following prototypes (or the future) I may encrypt the users.csv file since user data must be secure, however settings data does not need to be.
- The frequency of modification of settings is way higher than the modification of users.csv during run time, separating them will prevent excessive rewriting of user account data when only settings are changed.

```
// Write default settings into settings.csv on user registration
public void createDefaultSettings(String email) throws IOException {  1 usage  Aditya Sainath
    ensureCSVExists(csvPath);
    if (!doesSettingsExist(email)) {
        Settings defaultSettings = new Settings();
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toString(),  append: true))) {
            bw.write(toCSVRow(email, defaultSettings));
            bw.newLine();
        }
    }
}
```

SettingsController Code 1 - createDefaultSettings Procedure

This subroutine creates a new settings object (with the default values) and write it into settings.csv alongside the user's email.

```
// Default settings written into settings.csv when user signs up
public Settings() {  6 usages  Aditya Sainath
    // Stylesheet default
    this.theme = "dark";
    // Accessibility
    this.darkness = 0.0;
    // Audio
    this.ttsVolume = 0.8;
    this.sfxVolume = 0.6;
    this.muted = false;
    this.ttsEnabled = true;
}
```

Settings Code 1- Default Constructor

Whereas, on login the following code has been added to the login function:

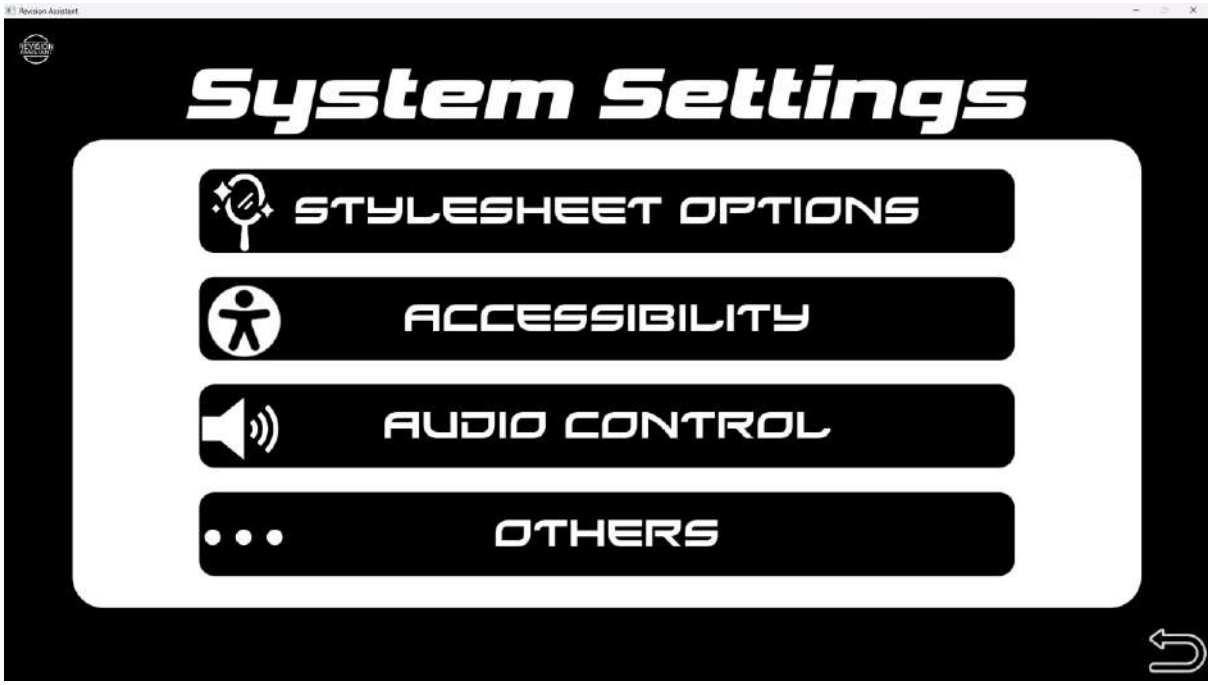
```
// On login the system's current user will be set to the temp user
// Settings will be loaded and the SettingsController set into the ScreenManager class
setCurrentUser(temp);
Settings userSettings = sc.loadSettings(email);
setCurrentSettings(userSettings);
setSettingsControllertoSM(sc);
return true;
```

AuthenticationController Code 7 - Code added to the Login Function

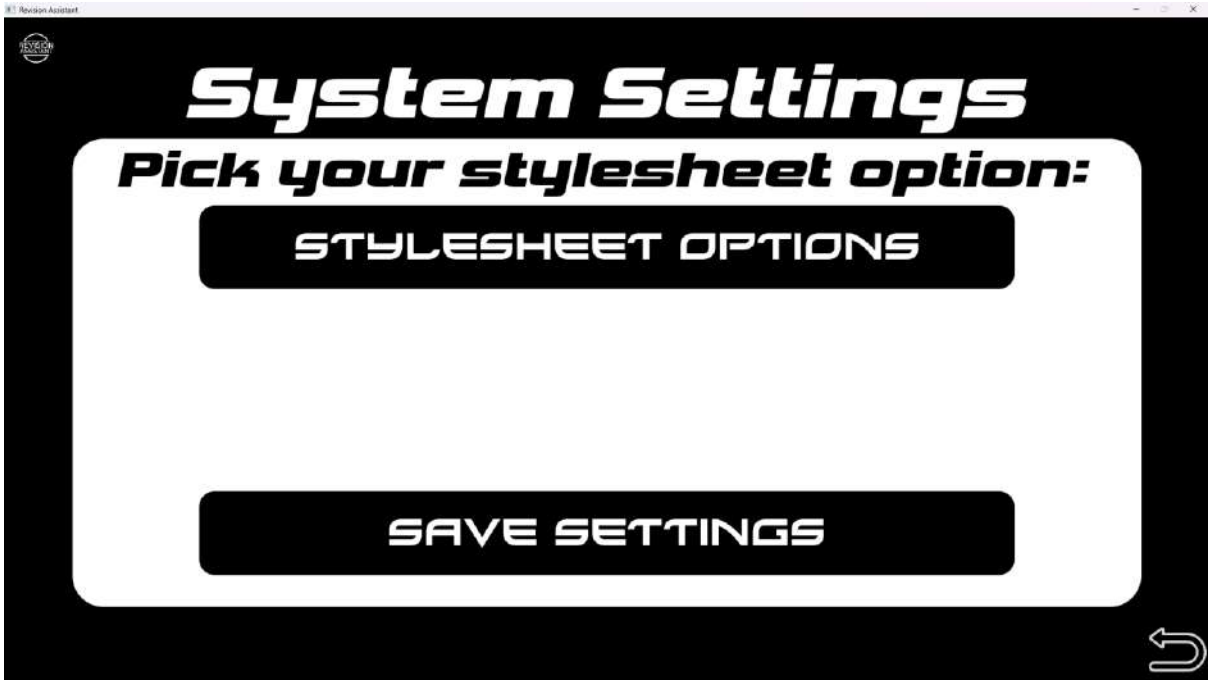
```java
// Load settings for a user
public Settings loadSettings(String email) throws IOException {
    // Method from Controller superclass
    ensureCSVExists(csvPath);
    ArrayList<Settings> settings = convertCSVtoArrayList();
    // Linear search for user
    for(Settings s : settings) {
        if(s.getEmail().equals(email)) {
            return s;
        }
    }
    // If not found
    Settings defaultSettings = new Settings();
    defaultSettings.setEmail(email);
    return defaultSettings;
}
```

This will convert settings.csv into an array list and then search it looking for the correct data and sets the current settings attribute within the AuthenticationController to the returned settings object.

3.2.4.2 Theme Switching



SettingsScreen Image 1 – Settings Screen



SettingsScreen Image 2 - Stylesheet Options Content



SettingsScreen Image 3 - Stylesheet Options

```
MenuItem darkMode = new MenuItem( text: "Dark Mode");
MenuItem lightMode = new MenuItem( text: "Light Mode");
MenuItem highContrast = new MenuItem( text: "High Contrast");

darkMode.getStyleClass().add("options");
lightMode.getStyleClass().add("options");
highContrast.getStyleClass().add("options");

ContextMenu menu = new ContextMenu(darkMode, lightMode, highContrast);
menu.getStyleClass().add("options-menu");
RectangleButtonBundle styles = ui.buildRectangularButtonBundle(rectW, rectH, text: "Stylesheet Options", typeOfText: "buttonText", StackPaneX: 310, StackPaneY: 300);
styles.button().setOnMouseClicked( MouseEvent _-> menu.show(styles.button(), Side.BOTTOM, dx: 0, dy: 0));
```

SettingsScreen Code 1 - Stylesheet Context Menu

Stylesheet Options were displayed to the user using a context menu instead of a split menu button, this was because improves clarity and is more scalable while keeping the UI clean. The context menu essentially acts as a split menu button without its dual behaviour (since it's only shown on the click of the "Stylesheet Options" button). This improved usability since the button has a single purpose, avoiding the two parts of the split menu button where the main click will trigger a default action and the arrow opens the menu. Theme selection is more of a configuration choice instead of an action, so it makes more sense to use menu-only interaction. This also keeps the stylesheet options content less cluttered whilst scaling better as more themes are added and simplifying implementation of them switching since each menu item directly calls the preview method.

To allow theme switching within the Revision Assistant, I created a base stylesheet containing all the shared layouts, fonts and component styling between the themes, with other stylesheets depending on the theme added on, which are focussed on the colours of components.

3.2.4.2.1 Initial Remedial Action

Initially within the SettingsScreen class, I created a preview theme method which would change the stylesheets of the group containing all the content when the menu item was selected, this led to:



Console Output 1 - Error on applying stylesheets to a Group

To prevent this output and the termination of the program following every stylesheet change, I realised the stylesheet changes must be added (or removed) from the Scene itself which the contentLayer group is inside. Thus, I amended the previewTheme subroutine to clear all the stylesheets from the current Scene (settings scene) which will be passed as a parameter and then the existing stylesheets are cleared, subsequently adding the basic style sheet and then the selected stylesheet by the user.

```
// Theme selected with be applied to their screen to give the user a preview
private void previewTheme(Scene root, String theme){  3 usages  Aditya Sainath
    // Removing any style sheet applied currently to avoid theme conflict/stacking
    root.getStylesheets().clear();
    // styles.css contains shared layout, fonts and component styling
    root.getStylesheets().add(Objects.requireNonNull(getClass().getResource( name: "/styles/styles.css")).toExternalForm());
    String themePath;
    switch (theme){
        case "light" -> themePath = "/styles/light.css";
        case "highContrast" -> themePath = "/styles/highContrast.css";
        default -> themePath = "/styles/dark.css";
    }
    root.getStylesheets().add(Objects.requireNonNull(getClass().getResource(themePath)).toExternalForm());
}
```

SettingsScreen Code 2 - previewTheme procedure

This procedure is called when one of the menu items is selected.

```
String infoMessage = "This is only a preview. \n The change will persist until you exist settings. \n Press save to ensure changes persist";
darkMode.setOnAction( ActionEvent _ -> {
    draftSettings.setTheme("dark");
    styles.buttonText().setText("Dark Mode");
    previewTheme(scene, draftSettings.getTheme());
    createInfoAlert(scene, infoMessage);
});
lightMode.setOnAction( ActionEvent _ -> {
    draftSettings.setTheme("light");
    styles.buttonText().setText("Light Mode");
    previewTheme(scene, draftSettings.getTheme());
    createInfoAlert(scene, infoMessage);
});
highContrast.setOnAction( ActionEvent _ -> {
    draftSettings.setTheme("highContrast");
    styles.buttonText().setText("High Contrast Mode");
    previewTheme(scene, draftSettings.getTheme());
    createInfoAlert(scene, infoMessage);
});
```

SettingsScreen Code 3 - calling previewTheme when a menu item is selected

Temporary changes are saved onto the draftSettings object (Settings class), which is copy of the currentSettings object initialised in the constructor method for the SettingsScreen class. However, these objects were always set to null and not updated to the correct user or any user in general. This was because within this screen I initialised new controllers (authentication and settings), hence the behaviour across screen transitions were not consistent. To ensure this behaviour is consistent, controller instances are maintained throughout the program, rather than being recreated when switching scenes. This means the same controller objects remain in memory and are reused across all screens, this allows the application to maintain a single source of truth for the currently authenticated user and their settings. Hence in main.java the controller objects are initialised and will be passed into the new creation of screen classes.

Hence to further ensure currentSettings and draftSettings are not set to null, within the SettingsScreen constructor the following code was added.

```
public SettingsScreen(AuthenticationController ac, SettingsController sC) {  1 usage
    this.ac = ac;
    this.sC = sC;
    // Load settings if not already loaded
    if (sC.getCurrentSettings() == null) {
        try {
            Settings userSettings = sC.loadSettings(ac.getCurrentUser().getEmail());
            sC.setCurrentSettings(userSettings);
        } catch (IOException e) {
            // Default Setting set as a fallback
            Settings userSettings = new Settings();
            sC.setCurrentSettings(userSettings);
        }
    }
    this.currentSettings = sC.getCurrentSettings();
    // Use existing draft if available, otherwise create new one
    if (sC.getDraftSettings() == null) {
        this.draftSettings = currentSettings.copy();
        sC.setDraftSettings(this.draftSettings);
    } else {
        this.draftSettings = sC.getDraftSettings();
    }
}
```

SettingsScreen Code 4 - Constructor Method

This allowed the theme changes to saved and written in the file with the following method:

```
// Save settings.csv for a user if changed
public boolean saveSettings(Settings updatedSettings) throws IOException {
    ArrayList<Settings> settings = convertCSVtoArrayList();
    boolean found = false;
    for(Settings s : settings) {
        if(s.getEmail().equals(updatedSettings.getEmail())) {
            copySettings(updatedSettings, s);
            found = true;
            break;
        }
    }
    // if not found add to array list
    if(!found) {
        Settings s = updatedSettings.copy();
        settings.add(s);
    }
    rewriteCSV(settings);
    return true;
}
```

SettingsController Code 1 - saveSettings Function

Now when the user goes back (existing the Settings Screen, to Main Menu), a new method was created to allow the stylesheets to be applied within the screens post authentication, in the ScreenManager class.

```
public static void switchScene(Scene root, String email) throws IOException {
    // Check if there's currently a scene being displayed
    if(stage.getScene() != null){
        // Save current scene to stack before switching
        history.push(stage.getScene());
    }
    changeStyleSheet(root, email);
    // Switch scene to new scene
    Scene finalScene = sceneWithOverlay(root, email);
    stage.setScene(finalScene);
}
```

ScreenManager Code 2 - new switchScene method for screens post-authentication

```
// Procedure for applying user settings to each screen (stylesheet specific as of) 08/02/2025
private static void changeStyleSheet(Scene root, String email) throws IOException {  2 usages
    Settings userSettings = getUserSettings(email);
    root.getStylesheets().clear();
    root.getStylesheets().add(Objects.requireNonNull(ScreenManager.class.getResource( name: "/styles/styles.css")).toExternalForm());
    root.getStylesheets().add(Objects.requireNonNull(ScreenManager.class.getResource( name: "/styles/" + userSettings.getTheme() + ".css")).toExternalForm());
}
```

ScreenManager Code 3 - changeStyleSheet method

This same method has been called in the goBack method to ensure that stylesheets of scenes (after login/registration) are applied to the written user settings from settings.csv.

The user settings are only applied after successful authentication rather than at application start up, this ensures that settings are more user-specific than applied globally. The preferences vary per user, and if multiple users are using the application on the same device, they might become frustrated with not seeing their settings. Hence on startup, the stylesheet will always be the original dark.css alongside the system's base stylesheet.

3.2.4.2.2 Testing theme switching

Test No.	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	theme	“dark”	Normal	Theme applied instantly.	Theme applied instantly.	
2.	theme	“light”	Normal	Theme applied instantly.	Theme applied instantly.	
3.	theme	“highContrast”	Normal	Theme applied instantly.	Theme applied instantly.	
4.	theme	“BlueTheme”	Erroneous	Not a possible option to be entered by the user	Not a possible option to be entered by the user	

Test No.	Proof
1.	
2.	

3.	 System Settings Pick your stylesheet option: HIGH CONTRAST MODE SAVE SETTINGS	
4.	 System Settings Pick your stylesheet option: HIGH CONTRAST MODE SAVE SETTINGS	

3.2.4.3 Brightness

To further address success criterion 6 and increase system accessibility, I added another accessibility feature which was brightness of the applications screens.

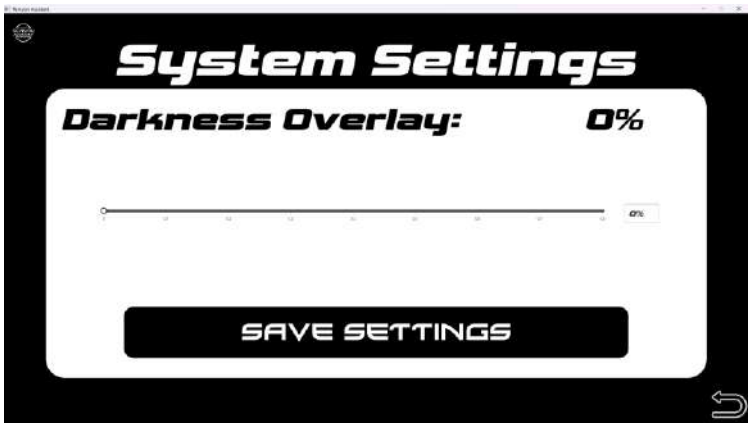
To implement screen brightness adjustments, I used a transparent rectangle overlay with varying opacity (depending on user input) added to a Stack Pane with the scene's content (post-auth). I could have chosen to modify each individual component's colour or generate multiple stylesheets for varying brightnesses, but that would increase the complexity of the program. The rectangle overlay allows a non-destructive way to vary brightness since the underlying theme styles are unchanged, so users can change the brightness without affecting core styling logic. This improves the maintainability of the program, since this reduces the processing overhead and separates the theme management and brightness control.

```
// Darkness/Brightness Control
darknessSlider.setLayoutX(250);
darknessSlider.setLayoutY(500);
darknessSlider.setMajorTickUnit(0.1);
darknessSlider.valueProperty().addListener(( ObservableValue<extends Number> obs,  Number oldVal,  Number newVal) -> {
    double darkness = newVal.doubleValue();
    draftSettings.setDarkness(darkness);
    brightnessOverlay.setOpacity(darkness);
    System.out.println("Set darkness to: " + darkness); // DEBUG
    darknessValue.setText(String.format("%.0f%%", darkness * 100));
    darknessInput.setText(String.format("%.0f%%", darkness * 100));
});

String infoMessage = "Changes to brightness will only persist after save button is pressed";

// Can be changed via user input too
darknessInput.setOnAction( ActionEvent _ -> {
    try{
        double inputValue = Double.parseDouble(darknessInput.getText());
        // Clamp between 0-80
        inputValue = Math.max(0, Math.min(80, inputValue));
        double darkness = inputValue / 100.0;
        draftSettings.setDarkness(darkness);
        brightnessOverlay.setOpacity(darkness);
        darknessSlider.setValue(darkness);
        darknessValue.setText(String.format("%.0f%%", darkness * 100));
        darknessInput.setText(String.format("%.0f", inputValue));
    } catch (NumberFormatException e) {
        // Reset to current value if invalid input
        darknessInput.setText(String.format("%.0f", draftSettings.getDarkness() * 100));
    }
});
```

SettingsScreen Code 5 - Darkness Inputs



SettingsScreen Image 4 - Brightness Adjustment Content

```
// Function to get the scene's original brightnessOverlay
private Rectangle getBrightnessOverlay(Scene scene) throws IOException {  1 usage
    Rectangle brightnessOverlay = null;
    if(scene.getRoot() instanceof StackPane wrapper){
        // Check every node in the StackPane to get the brightness overlay and set it to the new one
        for(var node : wrapper.getChildren()){
            if(node instanceof Rectangle rect && rect.getWidth() == 1920 && rect.getHeight() == 1057){
                brightnessOverlay = rect;
                break;
            }
        }
    }
    if(brightnessOverlay == null){
        brightnessOverlay = ScreenManager.getBrightnessOverlay(ac.getCurrentUser().getEmail());
    }
    return brightnessOverlay;
}
```

SettingsScreen Code 3 - getBrightnessOverlay Function

This function is created to get the root of the scene which will be the stack pane wrapper containing the content and the brightness overlay rectangle. This is always added to screens post-authentication within the ScreenManager class.

```
// Returns a scene with its content put inside a stack pane alongside the brightness overlay
private static Scene sceneWithOverlay(Scene scene, String email) throws IOException {  1 usage
    Parent originalRoot = scene.getRoot();
    StackPane wrapper = new StackPane(originalRoot);
    wrapper.getChildren().add(getBrightnessOverlay(email));
    scene.setRoot(wrapper);
    return scene;
}
// Returns a rectangle to act as a brightness overlay
public static Rectangle getBrightnessOverlay(String email) throws IOException {  2 usages
    Settings userSettings = getUserSettings(email);
    Rectangle brightnessOverlay = new Rectangle( width: 1920,  height: 1057);
    brightnessOverlay.getStyleClass().add("brightnessOverlay");
    brightnessOverlay.setOpacity(userSettings.getDarkness());
    brightnessOverlay.setMouseTransparent(true);
    return brightnessOverlay;
}
```

ScreenManager Code 4 - Brightness Overlay

This is called when switching scenes and going back to ensure the brightness overlay is consistent throughout.

3.2.4.3.1 Remedial Action for Slider

During initial testing of slider and brightness testing, I found out settings changes (on the brightness slider) when navigating through different settings pages were lost. For example, setting the slider to a specific brightness, saving and then coming back to the accessibility page did not behave as expected, having the user's previously set brightness but rather had its value being the value set prior to changes from the user, in my case 0.0.

I discovered this was because of the previous SettingsScreen constructor being reinitiated between scene switches and different setting pages. Within the previous constructor, this line was executed each time:

```
this.draftSettings = currentSettings.copy();
```

SettingsScreen Code 4 - Previous Constructor Contained

This created a new object from the currentSettings object each time the screen was loaded, hence any changes made during the session were discarded (since the currentSettings object was not updated universally).

To fix this issue, I added the following code into the constructor:

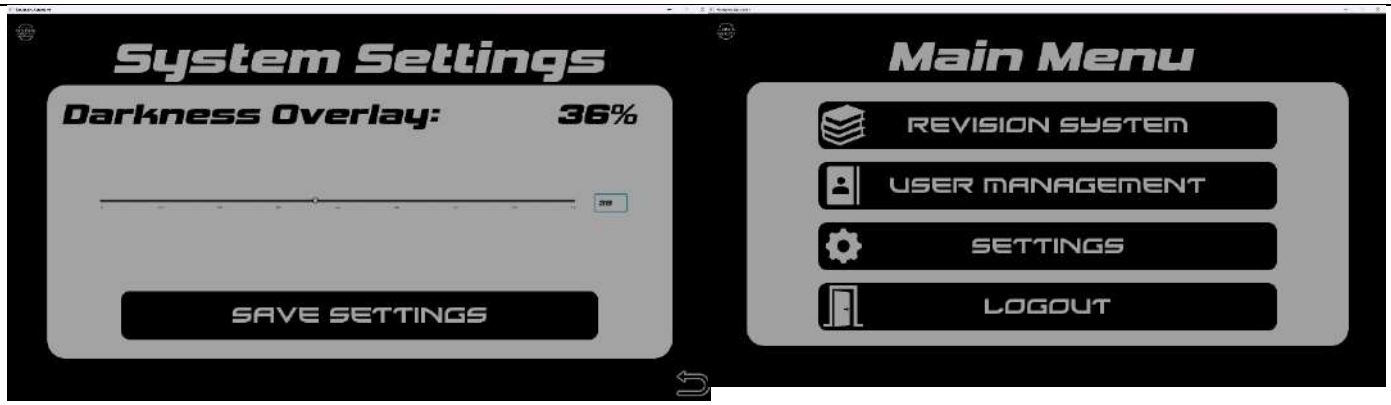
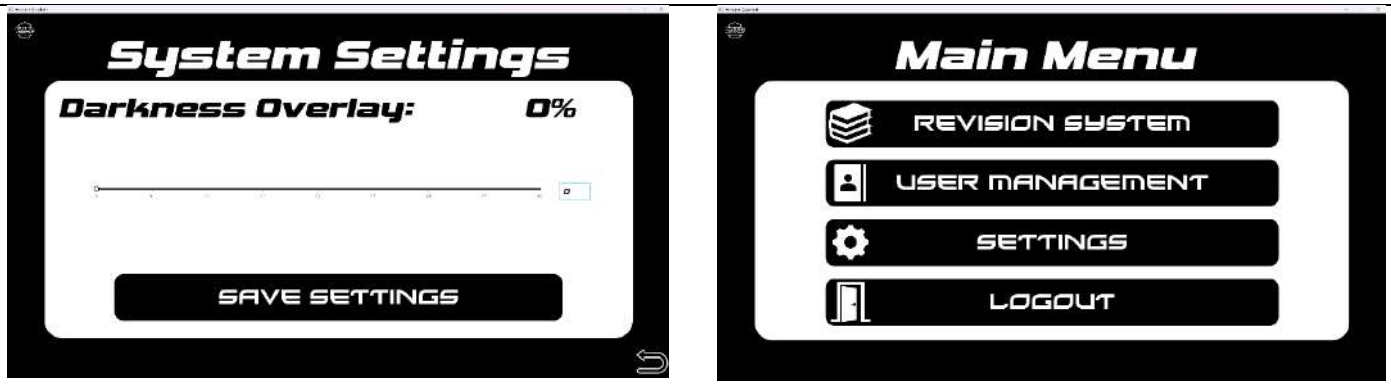
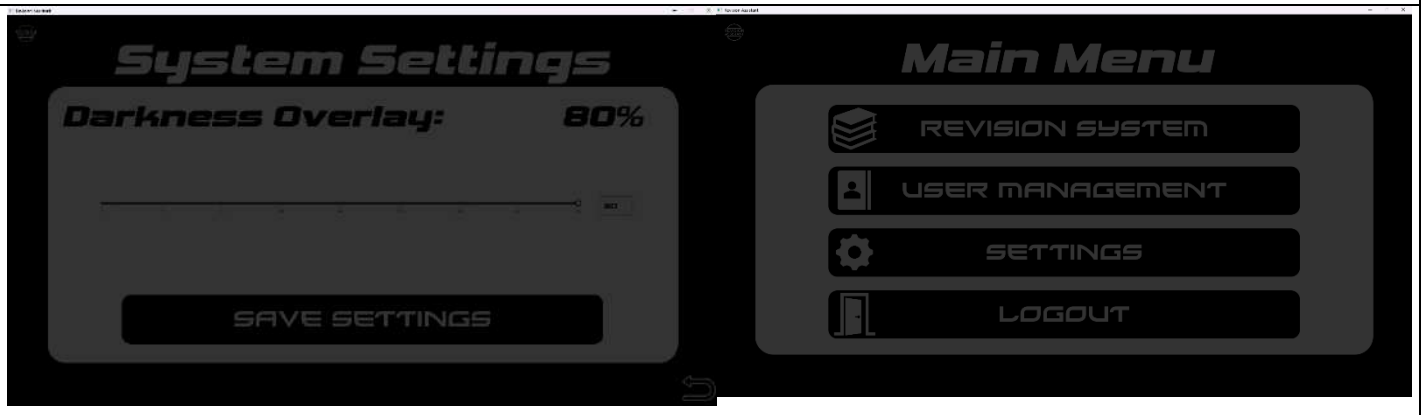
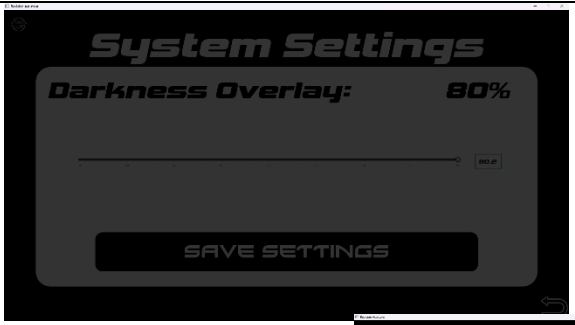
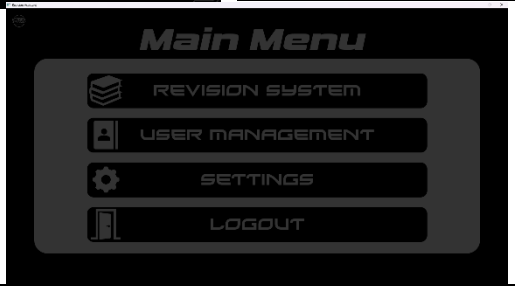
```
public SettingsScreen(AuthenticationController ac, SettingsController sC) {  1 usage
    this.ac = ac;
    this.sC = sC;
    // Load settings if not already loaded
    if (sC.getCurrentSettings() == null) {
        try {
            Settings userSettings = sC.loadSettings(ac.getCurrentUser().getEmail());
            sC.setCurrentSettings(userSettings);
        } catch (IOException e) {
            // Default Setting set as a fallback
            Settings userSettings = new Settings();
            sC.setCurrentSettings(userSettings);
        }
    }
    this.currentSettings = sC.getCurrentSettings();
    // Use existing draft if available, otherwise create new one
    if (sC.getDraftSettings() == null) {
        this.draftSettings = currentSettings.copy();
        sC.setDraftSettings(this.draftSettings);
    } else {
        this.draftSettings = sC.getDraftSettings();
    }
}
```

SettingsScreen Code 5 - Updated Constructor

This meant that the draft is only created once per session (of settings) and the same draft object would be reused across all possible instances of SettingsScreen, this allows the changes to persist throughout navigation.

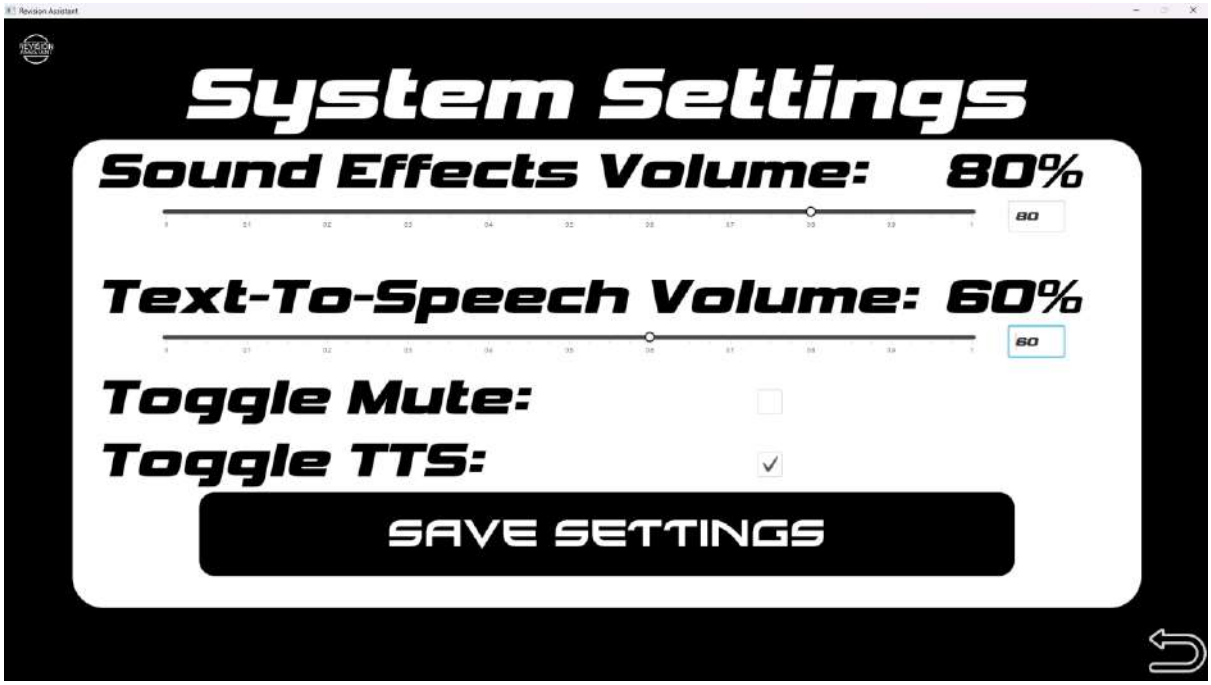
3.2.4.3.2 Testing brightness adjustments

Test No.	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	darkness	36.2	Normal	Brightness of screen successfully adjusted, instantly applied, consistent when switched too.	The brightness of the screen was set the to inputted value, it was applied instantly and was consistent when switched to or from. (Value also rounded to nearest whole number)	
2.	darkness	0.0	Boundary			
3.	darkness	80.0	Boundary			
4.	darkness	90.2	Erroneous	Set to 80.0, rejected	The value was automatically set to 80 and applied instantly, and consistent across screens when saved.	

Test No.	Proof
1.	
2.	
3.	
4.	  

3.2.4.4 Audio Control

To address success criterion 11, the program will have multiple sliders and options to turn on and off sound effects as well as text-to-speech volume. This allows the user to have direct control over audio, which is important for accessibility and usability since many users find audio distracting.



SettingsScreen Image 5 - Audio Control Page

This was coded with 2 sliders (with input boxes as alternatives in terms of text fields) and 2 check boxes for the toggles.

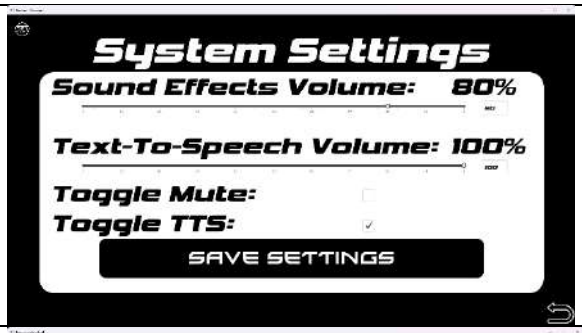
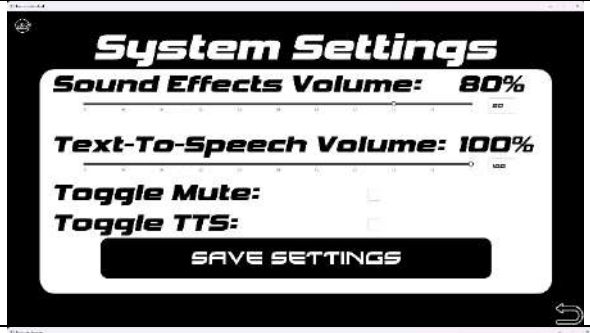
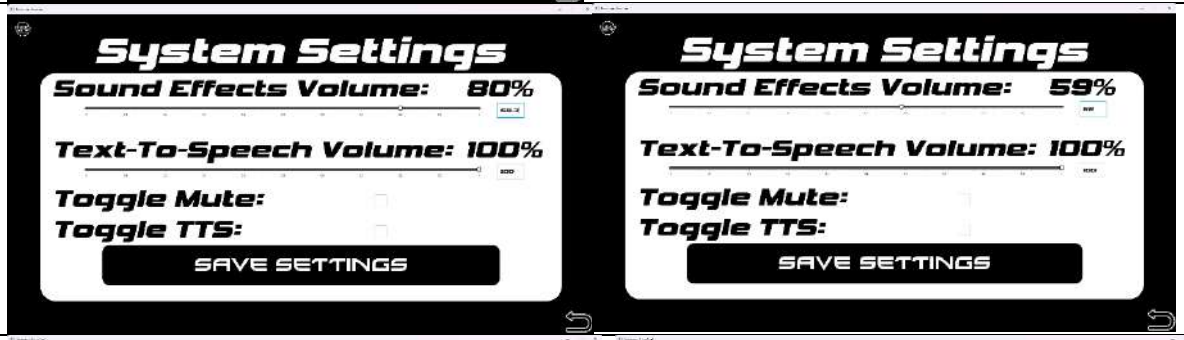
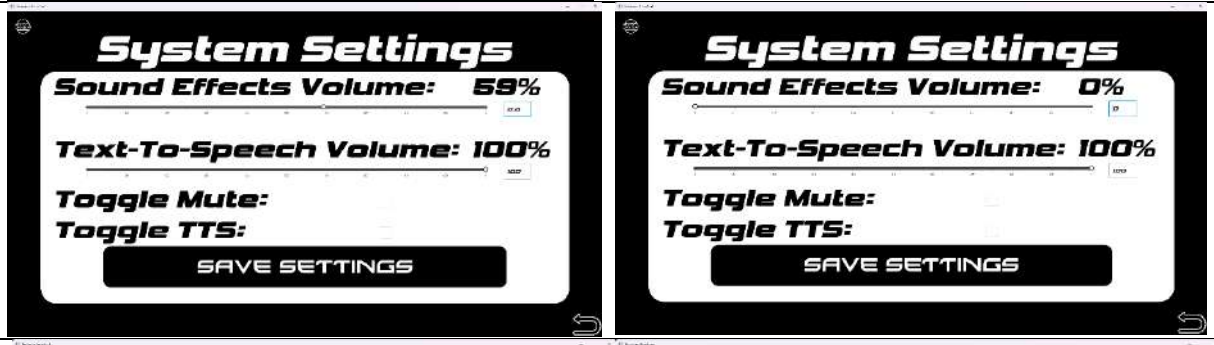
Once the sliders are adjusted and released the function playTestSound is called to give the user a preview of the new volume.

```
// procedure which will play the test sound when volume is altered
private void playTestSound(double volume){  5 usages
    Media testSound = new  Media(Objects.requireNonNull(getClass().getResource( name: "/sounds/testSound.wav")).toExternalForm());
    MediaPlayer player = new MediaPlayer(testSound);
    player.setVolume(volume);
    player.play();
}
```

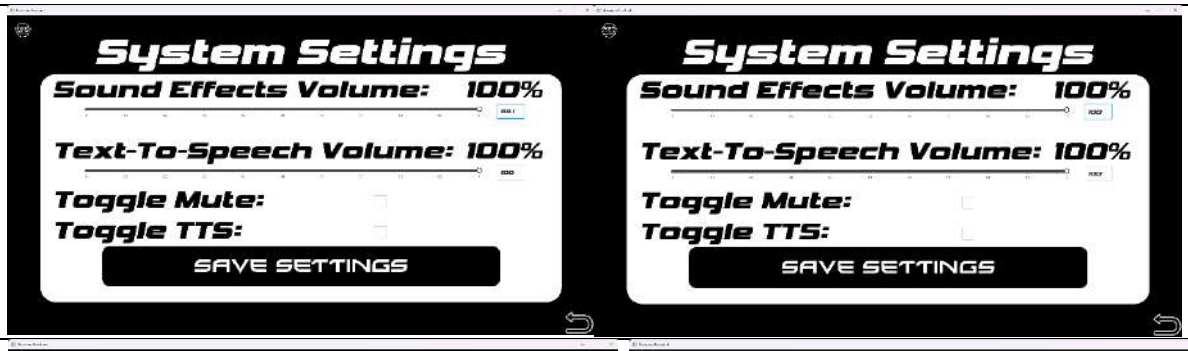
SettingsScreen Code 6 - playTestSound Code

3.2.4.4.1 Testing Audio

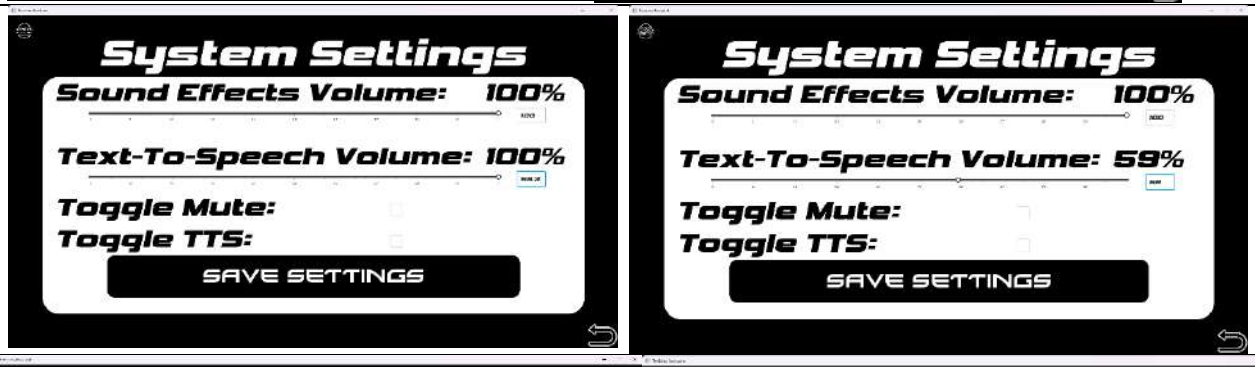
Test No.	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	isTTSEnabled	True	Normal	Accepted, ttsVol changes from 0.0 to default	Accepted, but ttsVol does not change	
2.	isTTSEnabled	False	Normal	Accepted, ttsVol set to 0.0	Accepted, but ttsVol does not change	
3.	sfxVol	59.3	Normal	Sound plays at the input volume successfully.	Sound plays at the input volume successfully.	
4.	sfxVol	0.0	Boundary			
5.	sfxVol	100.0	Boundary			
6.	sfxVol	101.1	Erroneous	Rejected, set to 100.	Rejected, set to 100.	
7.	ttsVol	59.3	Normal	Sound plays at the input volume successfully.	Sound plays at the input volume successfully.	
8.	ttsVol	0.0	Boundary			
9.	ttsVol	100.0	Boundary			
10.	ttsVol	101.1	Erroneous	Rejected, set to 100.	Rejected, set to 100.	
11.	muted	False	Normal	sfxVol set to default value from 0.0	sfxVol did not change	
12.	muted	True	Normal	sfxVol set to 0.0	sfxVol did not change	

Test No.	Proof
1.	
2.	
3.	
4.	
5.	

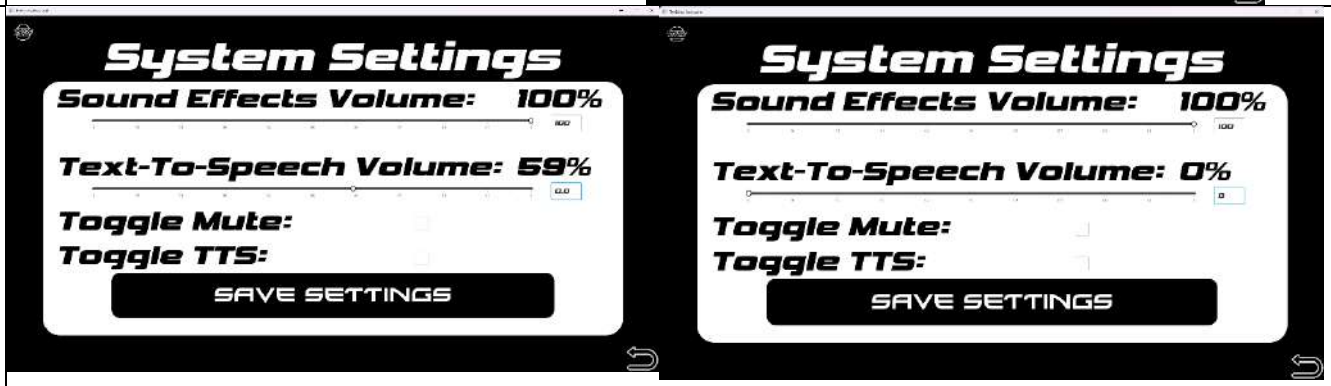
6.


System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 100%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS
System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 100%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS

7.

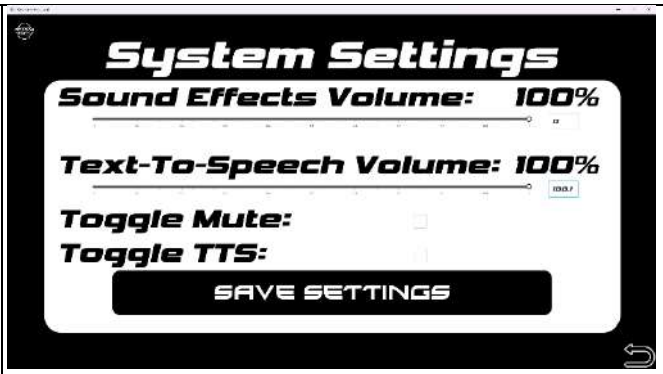
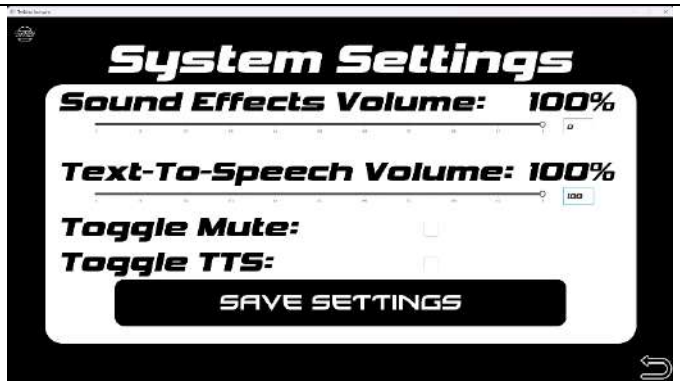
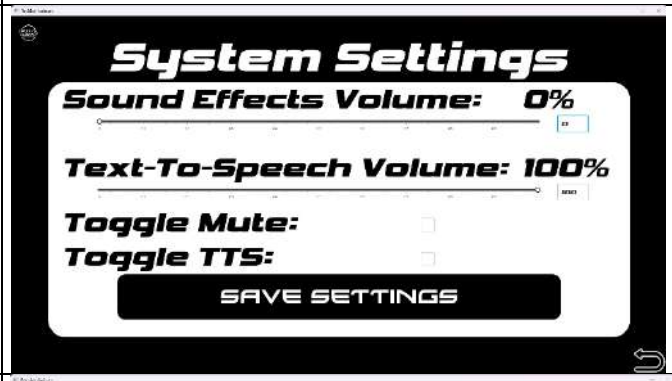
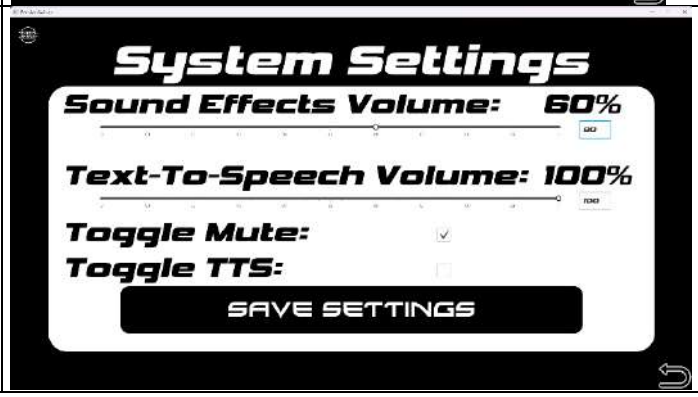

System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 100%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS
System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 59%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS

8.


System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 59%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS
System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 0%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS

9.


System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 0%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS
System Settings
Sound Effects Volume: 100%
Text-To-Speech Volume: 100%
Toggle Mute:
Toggle TTS:
SAVE SETTINGS

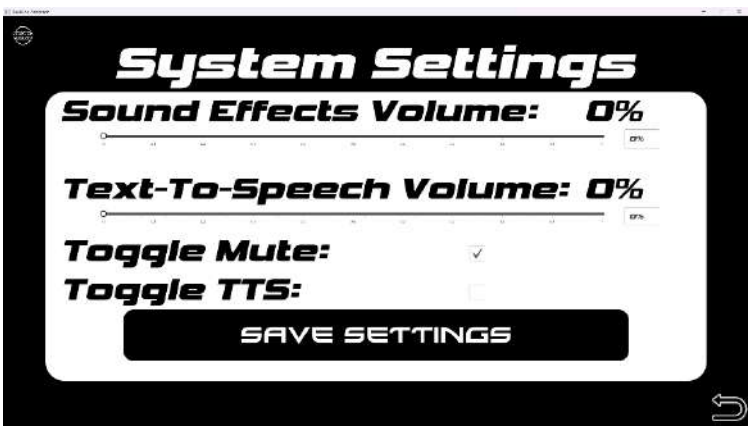
10.	 System Settings Sound Effects Volume: 100% Text-To-Speech Volume: 100% Toggle Mute: Toggle TTS: SAVE SETTINGS  System Settings Sound Effects Volume: 100% Text-To-Speech Volume: 100% Toggle Mute: Toggle TTS: SAVE SETTINGS
11.	 System Settings Sound Effects Volume: 0% Text-To-Speech Volume: 100% Toggle Mute: Toggle TTS: SAVE SETTINGS
12.	 System Settings Sound Effects Volume: 60% Text-To-Speech Volume: 100% Toggle Mute: Toggle TTS: SAVE SETTINGS

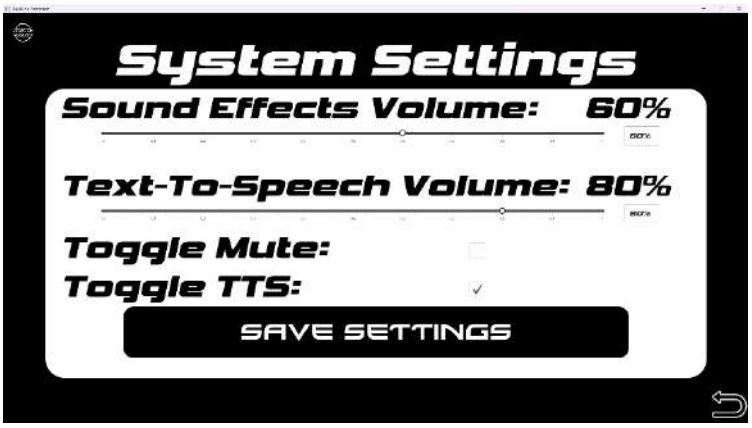
3.2.4.4.2 Remedial Action

While testing I noticed the check boxes do not change the volume levels which goes against the expectations, to remediate this I have added the following procedure to the code and attached them to the text boxes.

```
// Procedure which changes screen info on check boxes
private void onToggle(CheckBox checkBox, TextField inputField, Text value, Slider slider, boolean isTTS) {
    double volume;
    if (checkBox.isSelected()) {
        if (isTTS) {
            // get and set the default value for tts volume (80%)
            volume = new Settings().getTTSVolume();
            draftSettings.setTTSVolume(volume);
        } else {
            // Muted value
            volume = 0.0;
            draftSettings.setSFXVolume(volume);
        }
        // Change info on screen
        inputField.setText(String.format("%.0f%%", volume * 100));
        value.setText(String.format("%.0f%%", volume * 100));
        slider.setValue(volume);
    } else {
        if (isTTS) {
            volume = 0.0;
            draftSettings.setTTSVolume(volume);
        } else {
            // get and set default value for sfx (60%)
            volume = new Settings().getSFXVolume();
            draftSettings.setSFXVolume(volume);
        }
        // Change info on screen
        inputField.setText("0%");
        value.setText("0%");
        slider.setValue(volume);
    }
    // Play test sound
    playTestSound(volume);
}
```

SettingsScreen Code 7 - Procedure added to ensure changes of information on screen

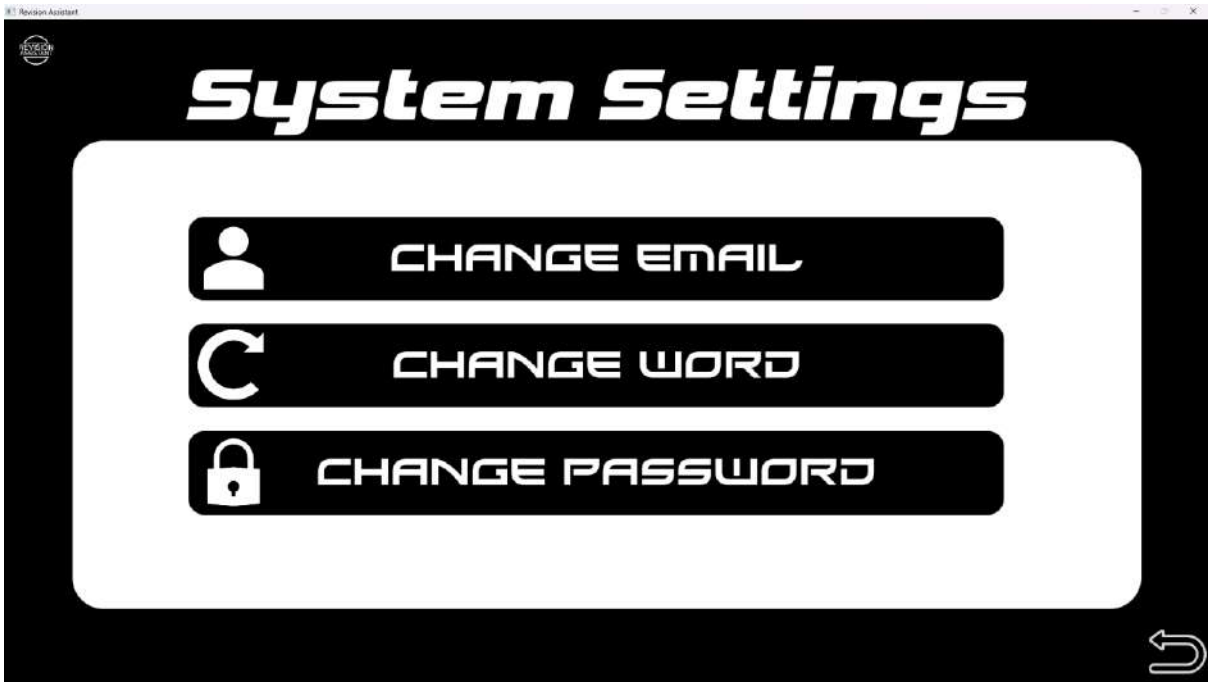




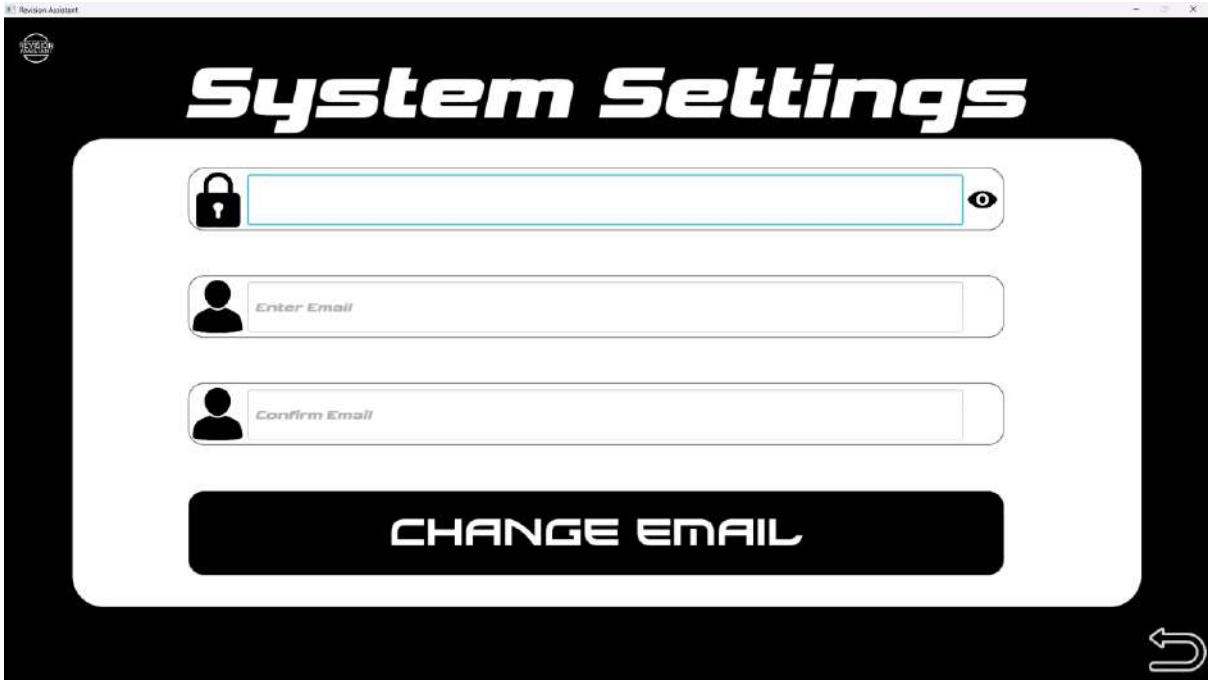
Further tests show this feature now works and is successful. This was procedure implemented to reduce the duplication of code since both check boxes use the same logic, however the feature in particle was implemented to avoid confusing the user by ensuring that audio-related control updates consistently and instantly.

3.2.4.5 Other Options

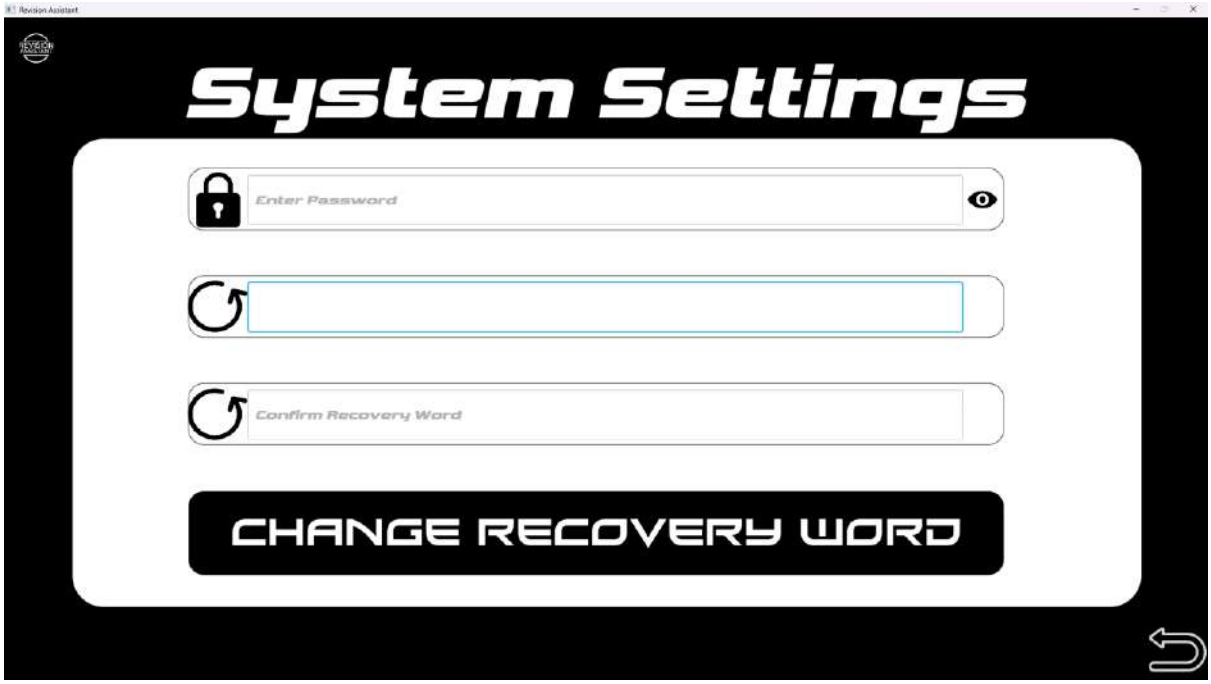
Furthermore, for convenience in the system, I have implemented the other options page within the SettingsScreen to allow the user to change their passwords, emails and recovery words using the same logic which was used to reset password in the authentication screen (but with password this time instead of recovery word). This is important so users can manage their account details securely. Following authentication of a user, it is reasonable to expect them to want to change their details (such as email, password or recovery word) and hence the Revision Assistant will provide this functionality.



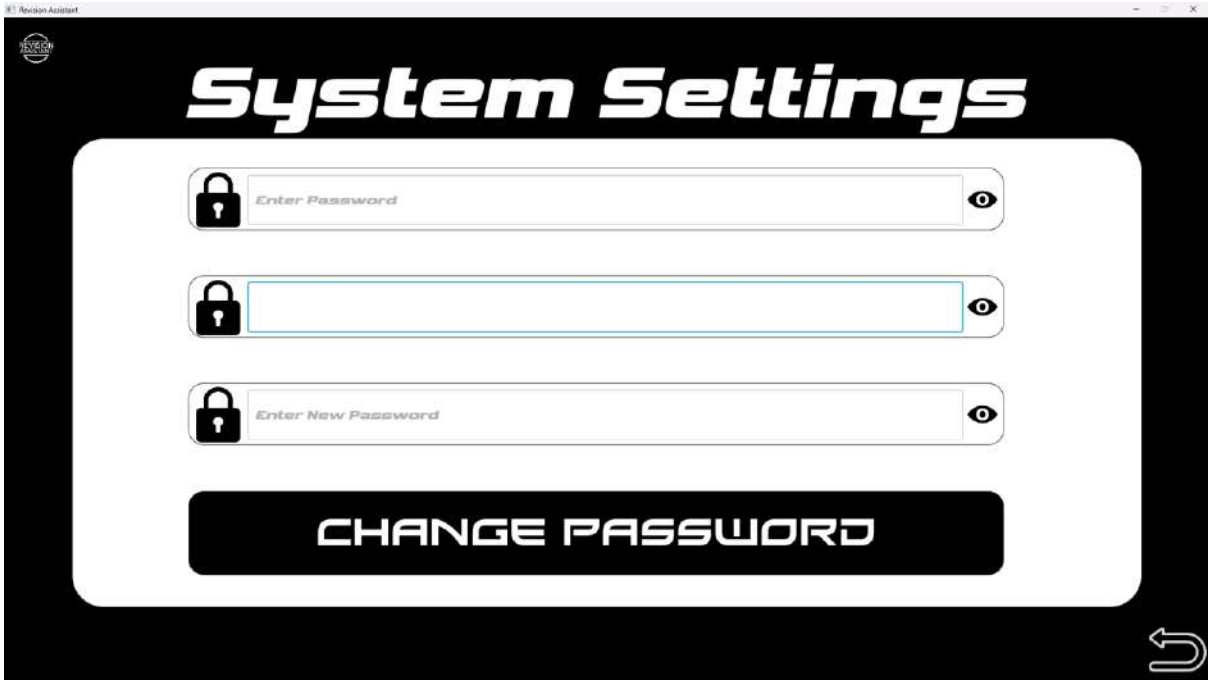
SettingsScreen Image 6 - Other Options



SettingsScreen Image 7 - Change Email



SettingsScreen Image 8 - Change Recovery Word



SettingsScreen Image 9 - Change Password

To re-use code from prototype 1, I extracted the rewriting of the users.csv file into a procedure:

```
// Will be used to update CSV when user makers changes
private void rewriteUsersCSV(ArrayList<User> users) throws IOException {  3 usages
    // Rewrite the entire file, so updates persist
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toString()))) {
        for (User u : users) {
            // writes row back into file
            bw.write( str: u.getEmail() + "," + u.getHashedPassword() + "," + u.getSaltBase64() + "," + u.getLastLogin() + "," + u.getRecoveryWord());
            bw.newLine();
        }
    }
}
```

AuthenticationController Code 8 - rewriteUsersCSV procedure

For all these situations, firstly the function of verifying the password was used from prototype 1 and specifically for:

1. Email Change – calls changeEmail function
2. Recovery Word Change – calls changeRecoveryWord function
3. Password change – re-uses resetPassword from prototype 1

These are all called using the AuthenticationController object.

```java
// Change a user's email
public boolean changeEmail(User temp, String newEmail) throws IOException {
    ArrayList<User> users = convertCSVtoArrayList();
    // Load all users and update the matching one
    for(User u : users){
        if(u.getEmail().equals(temp.getEmail())){
            u.setEmail(newEmail);
            break;
        }
    }
    // Write into file
    rewriteUsersCSV(users);
    return true;
}
```

AuthenticationController Code 9 - changeEmail function

```java
// Change a user's recovery word
public boolean changeRecoveryWord(User temp, String newRecoveryWord) throws IOException {
    ArrayList<User> users = convertCSVtoArrayList();
    for(User u : users){
        if(u.getEmail().equals(temp.getEmail())){
            u.setRecoveryWord(newRecoveryWord);
            break;
        }
    }
    // Write into file
    rewriteUsersCSV(users);
    return true;
}
```

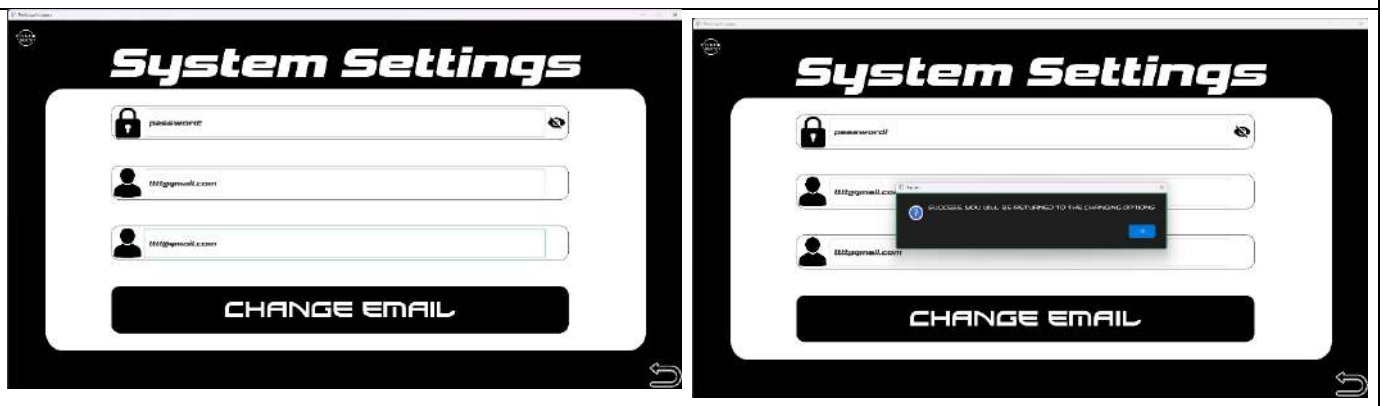
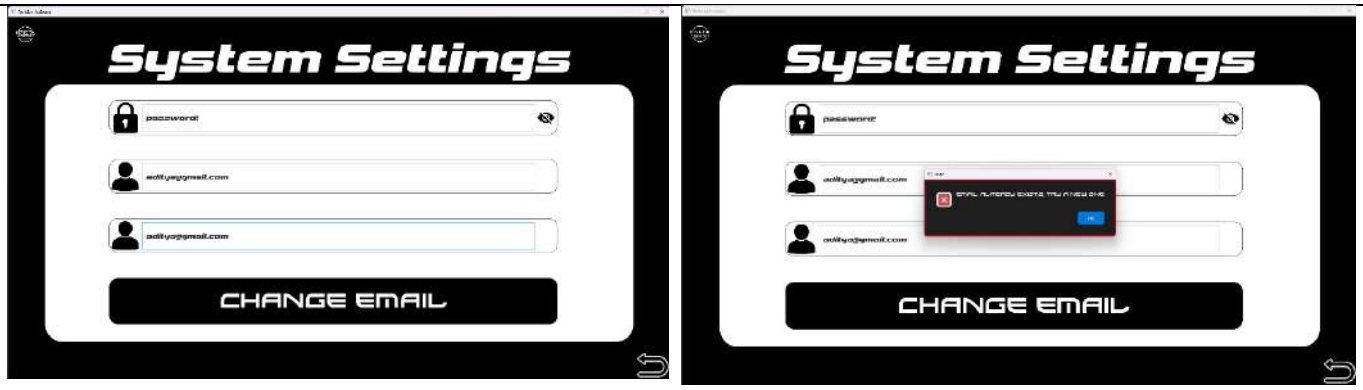
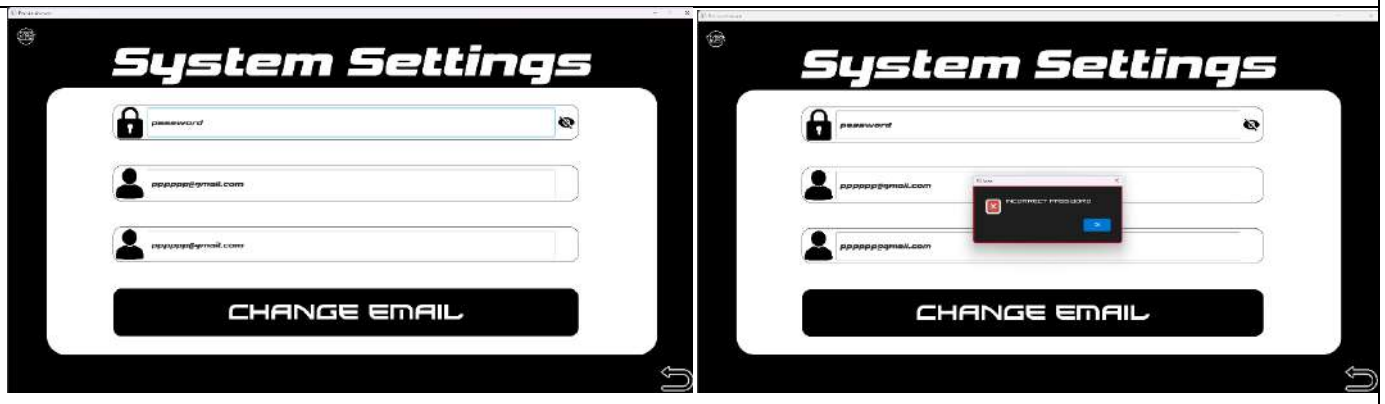
AuthenticationController Code 10 - changeRecoveryWord function

Furthermore, each of these functionalities have specific subroutines within the SettingsScreen class to display an appropriate alert to the user, on success they will be returned the previous content page (where they choose to change options).

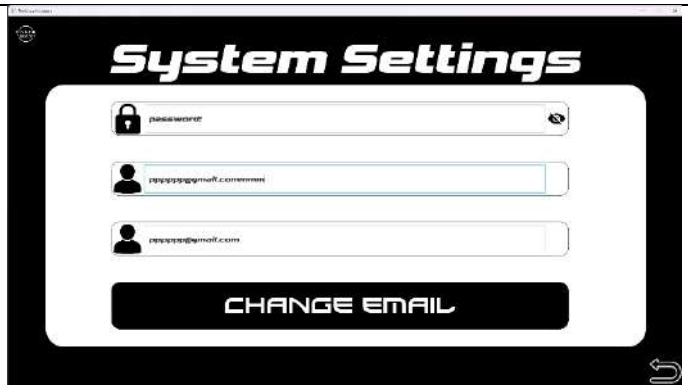
3.2.4.5.1 Testing Other Options

Test No.	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	email	Correct pw with same email as before	Erroneous	User told email is the same as before	User told email is same as before	
2.	email	Correct pw with tttt@gmail.com	Normal	Success, user returned to other options page	Success, user returned to other options page	
3.	email	Correct pw with email which already exists	Erroneous	User told email already exists	User told email already exists	
4.	email	Incorrect pw with valid email	Erroneous	User told password is incorrect	User told password is incorrect	
5.	email	Correct pw with not matching emails	Erroneous	User told emails don't match	User told emails don't match	
6.	recoveryWord	Correct pw with rW > 3 characters	Normal	Success, user returned to other options page	Success, user returned to other options page	
7.	recoveryWord	Correct pw with rW = 3 characters	Boundary	Success, user returned to other options page	Success, user returned to other options page	
8.	recoveryWord	Incorrect pw with rW > 3 characters	Erroneous	*User told password is incorrect	*User told password is incorrect	
9.	recoveryWord	Correct pw with not matching rWs	Erroneous	User told recovery words do not match	User told recovery words do not match	
10.	nPw and nCPw	Correct old pw, with valid new passwords	Normal	Success, user returned to other options page	Success, user returned to other options page	
11.	nPw and nCPw	Correct old pw, with valid new passwords (with 6 characters including a special)	Boundary	Success, user returned to other options page	Success, user returned to other options page	

12.	nPw and nCPw	Incorrect old pw, with valid new passwords	Erroneous	User told passwords is incorrect	User told passwords is incorrect	
13.	nPw and nCPw	Correct old pw, with not matching passwords	Erroneous	User told passwords do not match	User told passwords do not match	
14.	nPw and nCPw	Correct old pw, with passwords not meeting requirement	Erroneous	User told passwords requirements	User told passwords requirements	

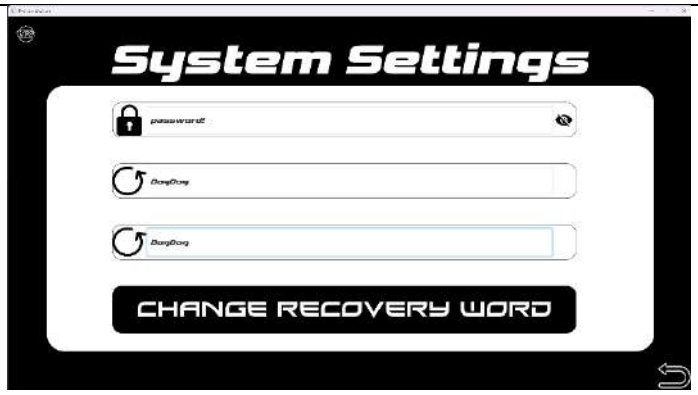
Test No.	Proof
1.	
2.	
3.	
4.	

5.


System Settings
password:
CHANGE EMAIL

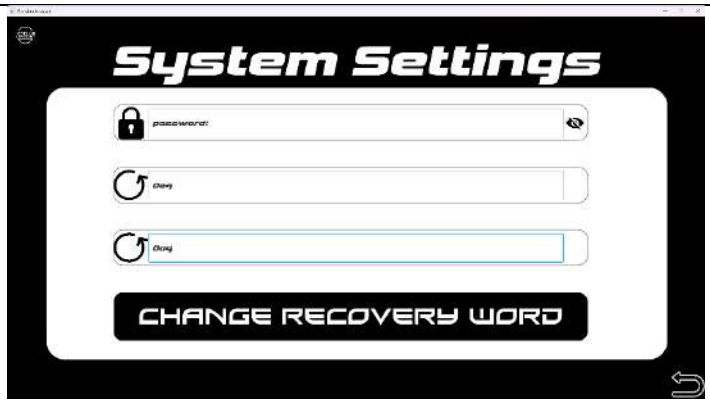

System Settings
password:
CHANGE EMAIL

6.


System Settings
password:
CHANGE RECOVERY WORD

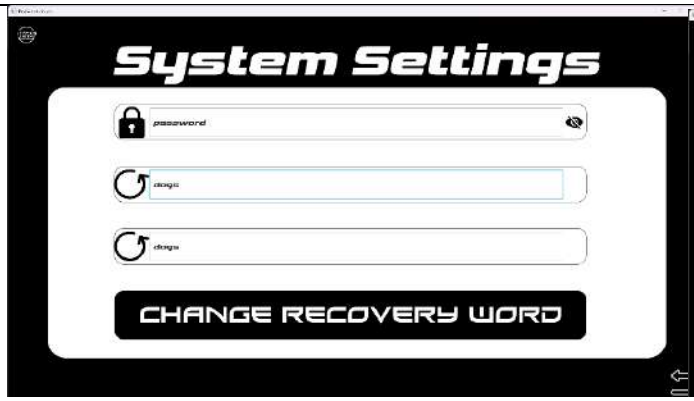

System Settings
password:
CHANGE RECOVERY WORD

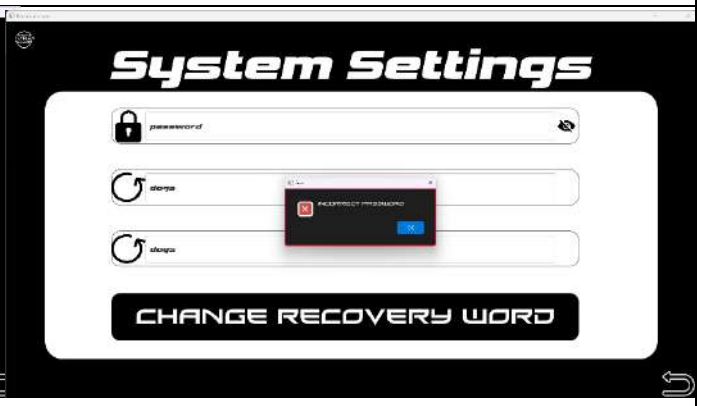
7.


System Settings
password:
CHANGE RECOVERY WORD

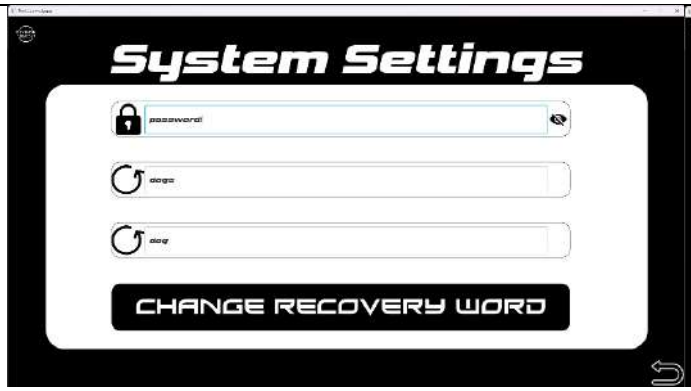

System Settings
password:
CHANGE RECOVERY WORD

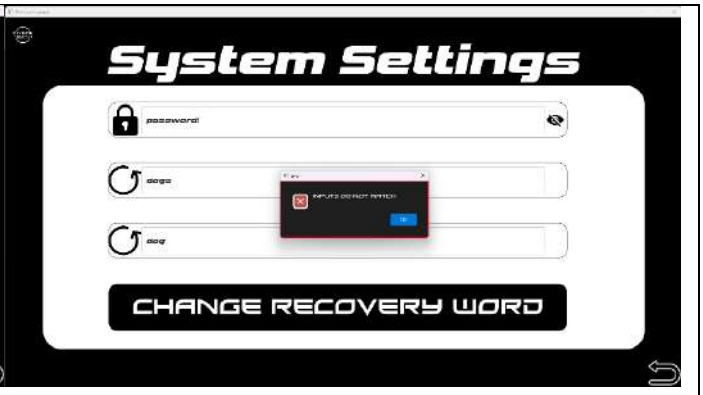
8.


System Settings
password
CHANGE RECOVERY WORD

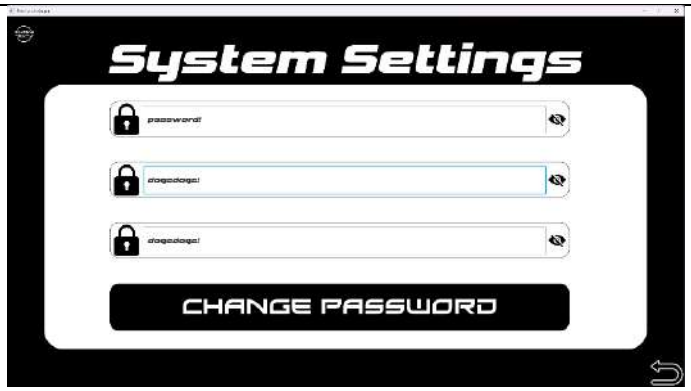

System Settings
password
CHANGE RECOVERY WORD

9.


System Settings
password!
dogs
dog
CHANGE RECOVERY WORD

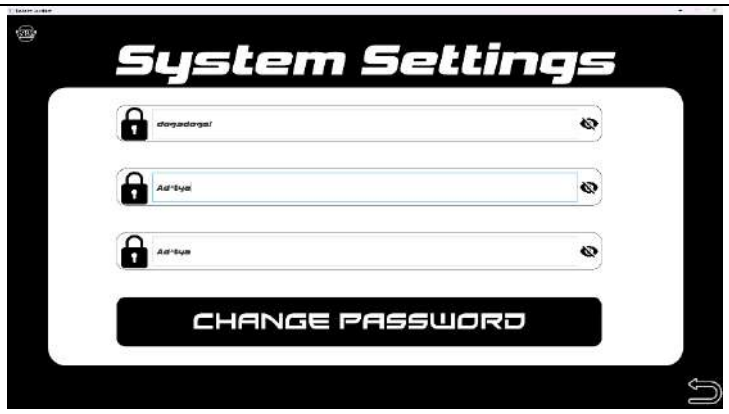

System Settings
password!
dogs
dog
CHANGE RECOVERY WORD

10.


System Settings
password!
dogedoge!
dogedoge!
CHANGE PASSWORD

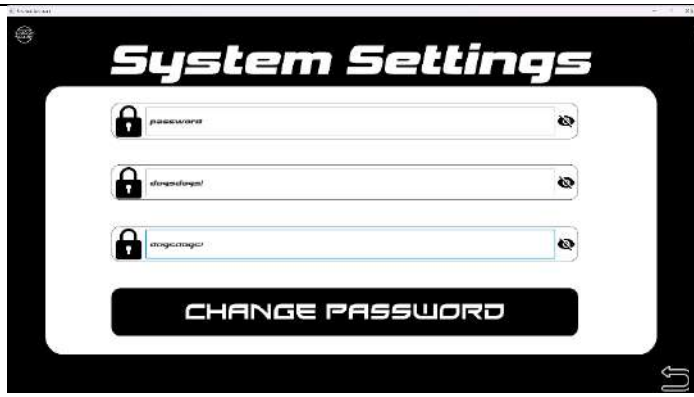

System Settings
password!
dogedoge!
dogedoge!
CHANGE PASSWORD

11.

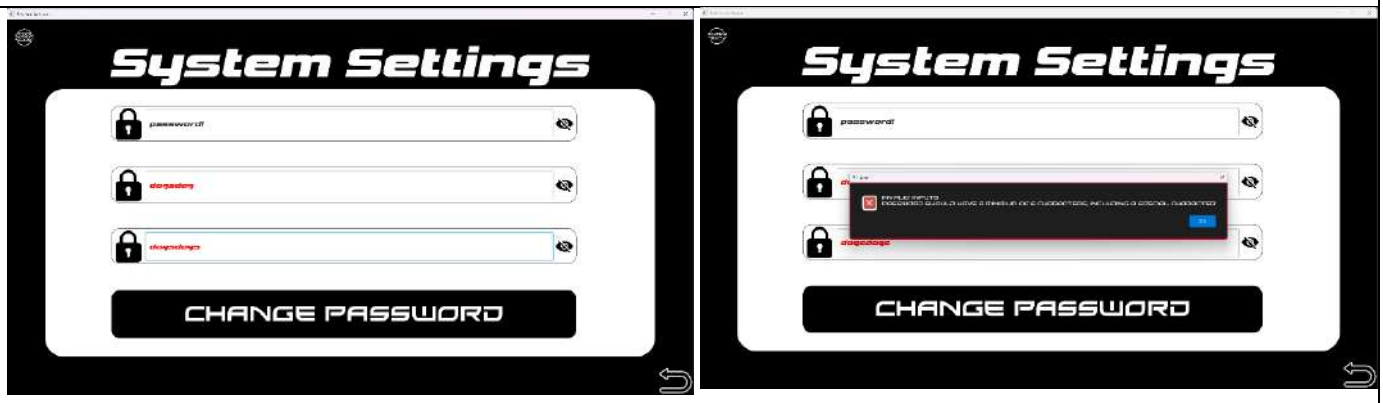

System Settings
dogedoge!
Ad-Sya
Ad-Sya
CHANGE PASSWORD


System Settings
dogedoge!
Ad-Sya
Ad-Sya
CHANGE PASSWORD

12.


System Settings
password
dogedoge!
dogedoge!
CHANGE PASSWORD


System Settings
password
dogedoge!
dogedoge!
CHANGE PASSWORD

13.	
14.	

3.2.5 Evaluation of Prototype 2

Prototype 2 built on the foundations developed in Prototype 1 and extended the UI creating then Main Menu Screen as well as the settings screen, meeting success criterions 6 and 11 successfully. I effectively implemented a main menu screen with a functional log out system, ensuring log out is secure and implemented a fully functional settings screen. This met criterions 6 and 11, and even more, focussing on improving the user experience and making the Revision Assistant convenient and accessible to all. Users can adjust their volumes, switch their stylesheet themes, adjust brightness and change their details easily with this prototype and destruction testing with corrupted files (in prototype 1) has shown the system can handle invalid data files without terminating, expecting this to work with more data added since they are of the same time. Some tests initially failed but with improvements and further tests, I can confirm the system has met these criterions and beyond successfully, hence prototype 2 has met the criteria, allowing me to move on to the next module of my application where I will address implementing the flashcard creation system which is the main functionality component of the application.

3.3 Prototype 3: Flashcard Control

3.3.1 Improvements from Prototype 2

In the previous prototypes, the navigation logic in the ContentNavigator and ScreenManager class made use of java's built in Stack library, however in prototype 3, I have replaced this with my own stack implementation.

```
package Models;
// Stack with a Type parameter, so I can use the same class for separate
data types
public class Stack<T> {
    // Array List due to its mutability;
    private final ArrayList<T> stackData;
    // Pointer pointing to the last element added
    private int top;

    // Constructor for creating the stack
    public Stack(){
        stackData = new ArrayList<>();
        top = -1;
    }

    // Push Method to add an item at the top of the stack
    public void push(T item){
        stackData.add(item);
        top++;
    }

    // Pop method to remove and return the top item
    public T pop(){
        if(!isEmpty()){
            T item = stackData.get(top);
            top--;
            return item;
        }
        return null;
    }

    // Peek method allows user to see what item was last added
    public T peek(){
        if(!isEmpty()){
            return stackData.get(top);
        }
        return null;
    }

    // Function to see if the stack isEmpty
    public boolean isEmpty(){
        return top == -1 || stackData.isEmpty();
    }

    // Function returning the size of the stack
    public int size(){
        return stackData.size();
    }

    // Function to clear the stack
    public void clear(){
        stackData.clear();
    }
}
```

Stack Code 1 - Whole Custom Stack Class

I have implemented my own custom stack class to improve the extensibility of the Revision Assistant. With the custom stack, I have control over its behaviour which will allow me to extend its functionality in future revisions depending on the scenario without being restricted by a standard java library implantation of a stack.

An example of this would be, when using a stack to study a deck of flashcards, it won't need to follow conventional LIFO behaviour, I can implement methods which can allow users to skip cards temporarily (and put missed cards back a few positions down). This could be done by making use of polymorphism and making different push/pop methods.

This custom stack will also be re-used in future protypes where it is needed.

3.3.2 Purpose of Prototype 3

In Prototype 3, I will create and implement the Revision System Screen as well as the Flashcard Management Screen. I will implement the core functionality of flashcard management, which will include creating, modifying, organising and importing/exporting decks. This prototype is essential since creating flashcards is part of the main functionality of the system, without the code which will be implemented within this prototype the system would not be a functional solution, hence would not meet the success criteria.

While previous prototypes have focussed authentication, settings and navigation, within this prototype I will:

- Allow users to create flashcards (decks).
- Allow users to edit decks.
- Allow users to organise their flashcard decks.
- Allow users to import / export flashcard decks.

Hence, the purpose of prototype 3 is to create a functional flashcard tool which will start the initial transformation of the Revision Assistant into a usable revision application.

3.3.3 Success Criteria for Prototype 3

Referencing (1.4.3 Success Criteria), with this prototype I will be aiming to achieve success criterions 9, 12 and 15 which aim to create the content users will use to study with as well as allowing user to organise their decks to ensure revision is never overwhelming, hence preventing the potential reduction of motivation. This prototype aims to have a fully functional flashcard management screen, with some functions of the revision system screen added, however the functionality of the revision system screen will be extended in future prototypes.

9) The program must allow users to import (csv) / export (csv/pdf) flashcards.

12) The program must have folder organisation systems, allowing the user to pick which folders flashcard decks go in.
15) The program must have folder organisation systems, allowing the user to pick which folders flashcard decks go in.

3.3.3.1 Flashcard Deck Creation and Modification Criteria

- User should allow user to create a flashcard deck with flashcards and name it with something of their choice.
- Flashcards contents must not be empty prior to being added to a deck.
- Users can add flashcards to a deck post-creation.
- Users can edit flashcards in a deck post-creation.
- Users can rename flashcard decks.
- Users can delete flashcard decks.
- Users can search for flashcards / decks.

3.3.3.2 Flashcard Folder Organisation Criteria

Since flashcards are saved straight to the user folder post creation, the user can go into the organisation page and:

- Be allowed to create folders.
- Be allowed to delete folders.
- Move decks into folders
- Move folders into sub-folders

with changes made by the user here persisting outside the program, in the actual file structure.

3.3.3.3 Import / Export Criteria

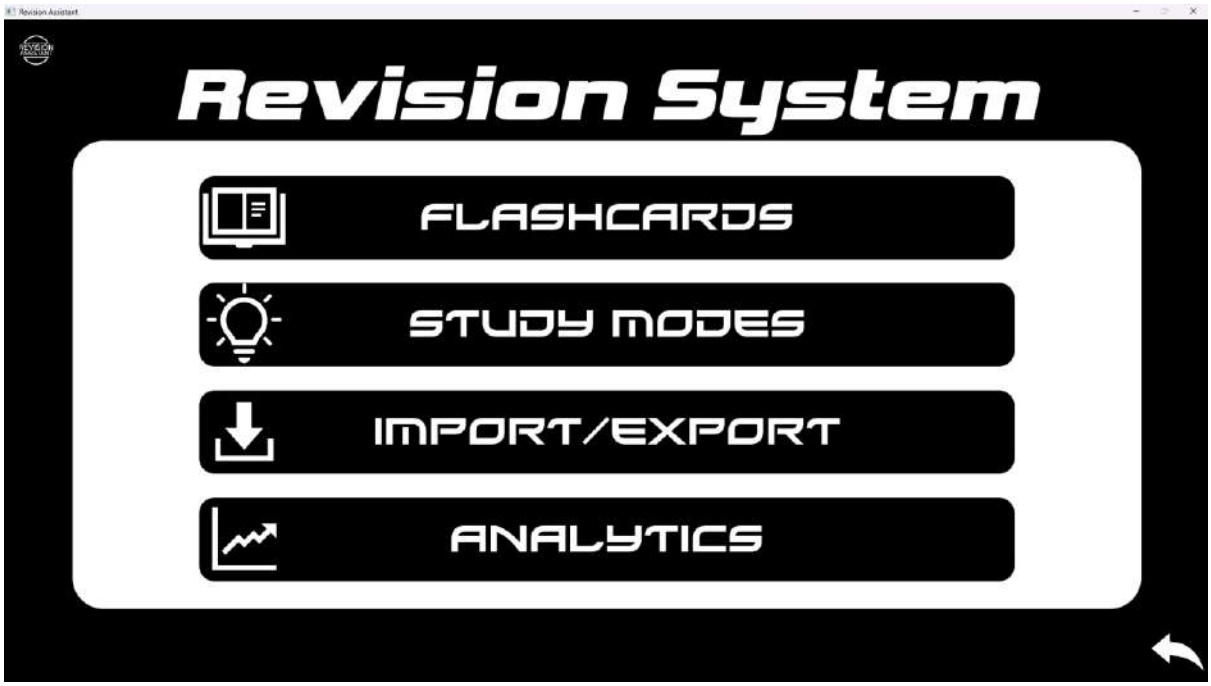
- User can import .csv files of a correct formation:
  - “question”, “answer”
  - New lines replaced with \n, indicating a new line
- Users can export .csv files of this format.
- User can export flashcard decks as .pdf.
- Corrupted files should be handled without crashing.

3.3.4 Developing Prototype 3

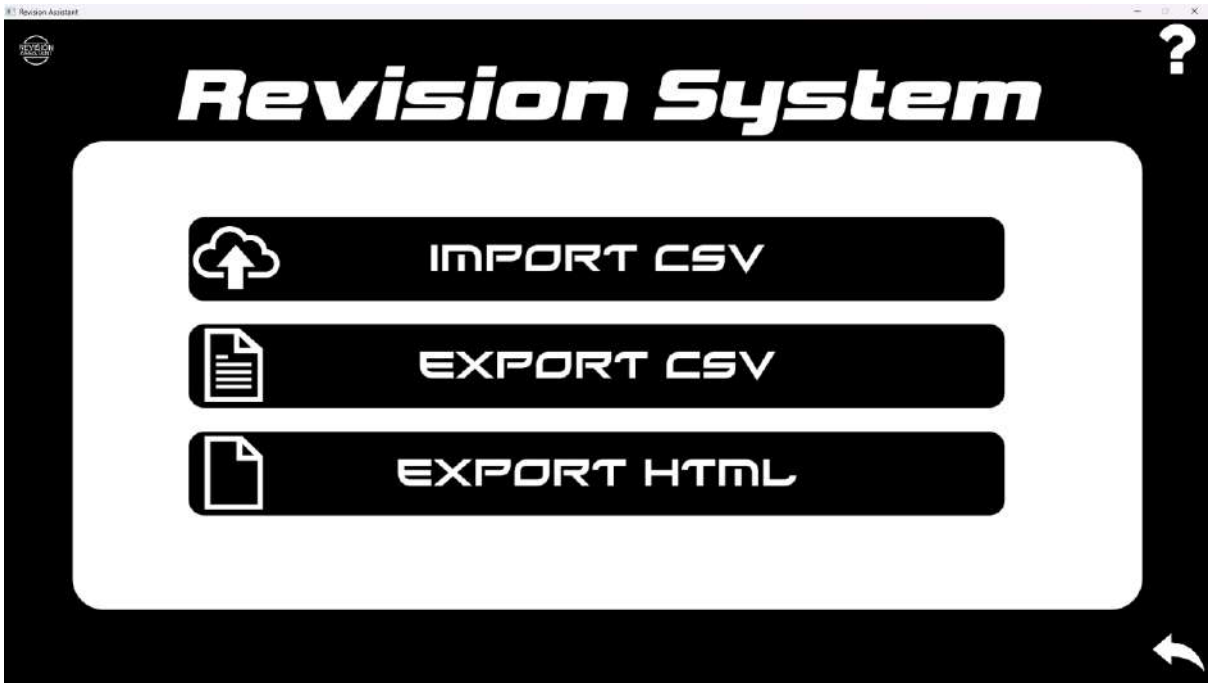
3.3.4.1 Revision System and Flashcard Management Screen

To begin this prototype, I initially created all the buttons and UI of the screens so after this I could only focus on the logic of each button. Creating the buttons using the RectangleButtonBundle, using the creation method from UIComponents, like prior prototypes and inheriting the background from SystemScreens allows this to be easy creation of the interface, whilst being consistent across the whole platform. In addition,

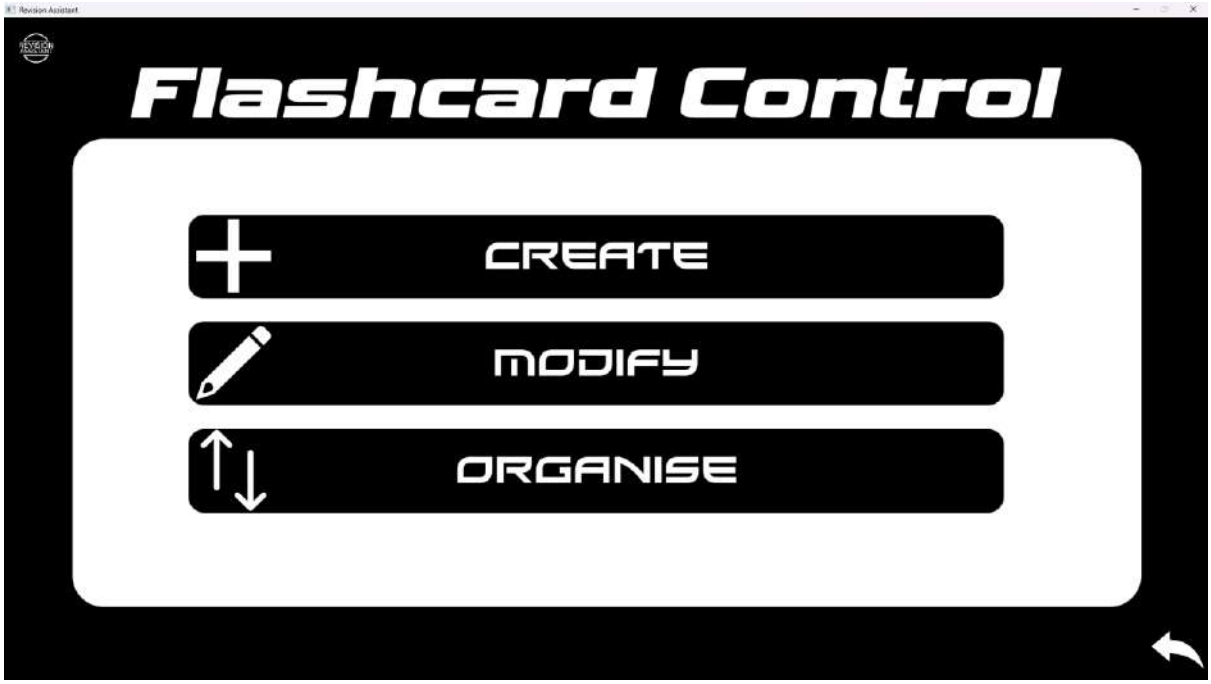
designs from 2.2.2.4 Revision System and 2.2.2.6 Flashcard Management were replicated to produce:



RevisionSystemScreen Image 1 - Main Options



RevisionSystemScreen Image 2 - Import / Export Page



FlashcardManagementScreen Image 1 - Main Options

Within the RevisionSystemScreen's constructor, an instance of the new controller: FlashcardController was initiated which will be used across all following screens because of this screen (flashcard management screen for example) to keep flashcard data consistent per session.

```
public RevisionSystemScreen(AuthenticationController ac, SettingsController sC) {
    this.ac = ac;
    this.sc = sC;
    this.fc = new FlashcardController(ac.getCurrentUser().getEmail());
}
```

RevisionSystemScreen Code 1 - Constructor Method

Hence the FlashcardManagementScreen's constructor would be:

```
/* Constructor class for FMScreen, controllers as parameters to maintain
consistency across screens. */
public FlashcardManagementScreen(AuthenticationController ac, SettingsController sC, FlashcardController fc) {
    this.ac = ac;
    this.sc = sC;
    this.fc = fc;
}
```

FlashcardManagementScreen Code 1 – Constructor Method

The flashcard controller's constructor is:

```java
// Constructor to set the userFolderPath
public FlashcardController(String email) {  1 usage
    this.email = email;
    // Sets the path to the user folder
    this.userFolderPath = Paths.get(System.getProperty("user.dir"), ...more: "userFolders", email).toString();
}
```

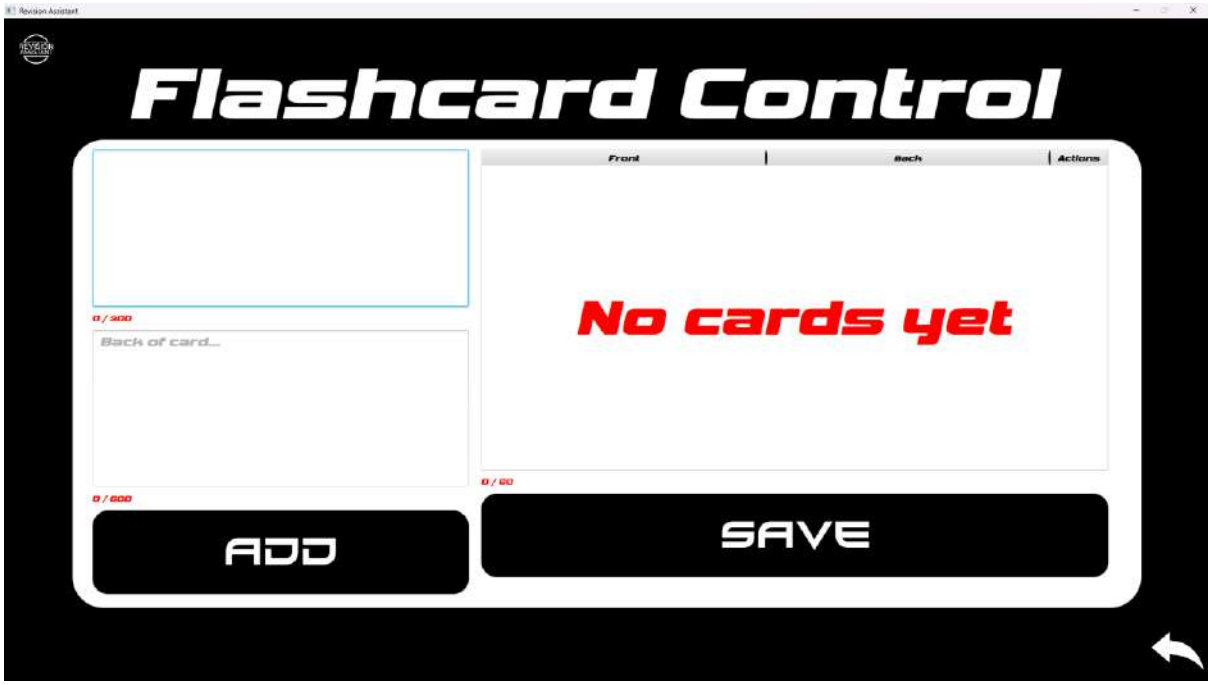
FlashcardController Code 1 - Constructor Method

The FlashcardController class will create a user-specific path to a user's directory, each user has a separate directory where they will store their flashcard data. This userFolders directory is marked as a resource root and is separate from actual resources and files folders. This separates user-generated data from system data, allowing bugs to be found easier and supports future scalability. An alternative approach would have to embedded flashcard data within a database instead of using .csv file storage. This would increase the scalability of the solution for future revisions but would also increase the complexity of the project. Since I have been using .csv file-based storage, to maintain consistency within the program and be able to re-use methods I chose to use .csv data storage for flashcards.

3.3.4.2 Flashcard Creation

3.3.4.2.1 Adding flashcards to a deck

To start off with the flashcard creation, I created the creation page where the user is given two text areas, with prompts and labels indicating character limits, positioned on the left of the page with a table view positioned on the right which shows the user cards which have been added to the deck. These are accompanied by an add and save button respectively.



FlashcardManagementScreen Image 2 - Flashcard Creation

For the text areas, I created a new record class called TextAreaBundle containing the TextArea, a label for the character counter, lower limit and the character limit. With this record class I added a new re-usable method to UIComponents:

```
// Creates a text area with prompt text and a character counter
public TextAreaBundle createTextArea(String promptText, int charLimit, int lowerLimit) {  2 usages
    TextArea textArea = new TextArea();
    textArea.getStyleClass().add("textArea");
    textArea.setPromptText(promptText);
    Label characterCounter = new Label( text: "0 / " + charLimit);
    characterCounter.getStyleClass().add("counter");
    // Initial Colour
    characterCounter.setStyle("-fx-text-fill: red;");
    textArea.textProperty().addListener(( ObservableValue<extends String> _,  String _,  String newValue) -> {
        // Change the number
        characterCounter.setText(newValue.length() + " / " + charLimit);
        // Change label colours depending on limits
        if (newValue.length() < lowerLimit || newValue.length() > charLimit) {
            characterCounter.setStyle("-fx-text-fill: red;");
        }else{
            characterCounter.setStyle("-fx-text-fill: green;");
        }
    });
    return new TextAreaBundle(textArea, characterCounter, charLimit, lowerLimit);
}
```

UIComponents Code 1 - createTextArea function

This method is called twice to create the area for the front of the card (question) and the back of the card (answer). This function improved modularity, centralising all character validation logic in one location instead of duplicating it across multiple locations. At first, I implemented two text areas with the same code repeated twice, hence I realised it was essential to extract this method, increasing the maintainability of the system.

Furthermore, when the user pressed the add button, the addFlashcard function is called, which adds the question and answer as new flashcard objects to the cardList, if they are valid inputs. The cardsList is a global variable, which can be accessed by any subroutine within the class, and is an ObservableList of the Flashcard object type, it is essentially an ArrayList, but it is observable, hence the TableView automatically updates when the list is changed.

```
// Observable lists storing all Flashcard objects being created or being edited
private final ObservableList<Flashcard> cardList = FXCollections.observableArrayList();
```

FlashcardManagementScreen Code 2 - Global variable for observable list

```java
// Procedure to add flashcards to the list
private void addFlashcard(TextArea q, TextArea a){  1 usage
    // Question and Answer constraints
    String qReq = "Questions can be 10-300 characters";
    String aReq = "Answers can be 1-600 characters";
    // Remove white space and get the text
    String question = q.getText().trim();
    String answer = a.getText().trim();
    // Check if valid
    boolean validQ = false;
    boolean validA = false;
    if(q.getLength() >= 10 && q.getLength() <= 300) validQ = true;
    if(a.getLength() >= 1 && a.getLength() <= 600) validA = true;
    // Add to list
    if(validQ && validA){
        cardList.add(new Flashcard(question, answer));
        // Clear the fields for next input
        q.clear();
        a.clear();
    }else{
        // Display error
        CustomAlert.error(scene.getWindow(),  message: qReq + "\n" + aReq);
    }
}
```

FlashcardManagementScreen Code 3 - addFlashcard Procedure

Furthermore, following adding a flashcard to the cardList, since in the buildTable method the cardList is bound to the TableView, it is automatically updated and added flashcards show up instantly.

```java
// Function which builds the table to display Flashcards
private TableView<Flashcard> buildTable(Boolean actionsNeeded){  3 usages
    // Create a new tableview bound to observable list
    TableView<Flashcard> table = new TableView<>(cardList);
    // Create columns and link to the specific property of Flashcard.java
    // to populate the cell
    TableColumn<Flashcard, String> frontColumn = new TableColumn<>( text: "Front");
    frontColumn.setCellValueFactory(new PropertyValueFactory<>("question"));
    TableColumn<Flashcard, String> backColumn = new TableColumn<>( text: "Back");
    backColumn.setCellValueFactory(new PropertyValueFactory<>("answer"));

    // Add this column if in create page, not in Modify
    if (actionsNeeded) {
        TableColumn<Flashcard, Void> actionsColumn = new TableColumn<>( text: "Actions");
        actionsColumn.setMaxWidth(250);
        actionsColumn.setCellFactory( TableColumn<Flashcard, Void> _ -> new TableCell<>() {
            final RectangleButtonBundle delete = ui.buildRectangularButtonBundle( rectW: 80,  rectH: 25,  text: "Delete",  typeOfText: "tableText",  StackPaneX: 0,  StackPaneY: 0);  2 usages
            final RectangleButtonBundle edit = ui.buildRectangularButtonBundle( rectW: 80,  rectH: 25,  text: "Edit",  typeOfText: "tableText",  StackPaneX: 0,  StackPaneY: 0);  2 usages
            final VBox actions = new VBox( spacing: 5, edit.button(), delete.button());  1 usage
            {
                // Delete - removed the card at this row from the list
                delete.button().setOnMouseClicked( MouseEvent _ -> {
                    Flashcard card = getTableView().getItems().get(getIndex());
                    if (CustomAlert.confirm(scene.getWindow(),  title: "Deletion?",  message: "Are you sure you want to delete?")) {
                        cardList.remove(card);
                    }
                });
                // Edit - changes current text field and removes from list so it can be resaved
                edit.button().setOnMouseClicked( MouseEvent _ -> {
                    Flashcard card = getTableView().getItems().get(getIndex());
                    if (CustomAlert.confirm(scene.getWindow(),  title: "Deletion?",  message: "Are you sure you want to edit?")) {
                        questionArea.textArea().setText(card.getQuestion());
                        answerArea.textArea().setText(card.getAnswer());
                        cardList.remove(card);
                    }
                });
            }
```

FlashcardManagementScreen Code 4 - buildTable function (part 1)

```
        @Override
        protected void updateItem(Void item, boolean empty) {
            super.updateItem(item, empty);
            // Only show options on the rows which have data
            setGraphic(empty ? null : actions);
        }
    });
    // Add all columns to the table
    table.getColumns().addAll(List.of(frontColumn, backColumn, actionsColumn));
}else{
    // Add all columns to table
    table.getColumns().addAll(List.of(frontColumn, backColumn));
}
// Columns are resized to fill available width, last column will shrink or expand
table.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
// Set the text when there are no cards
Label placeholderText = new Label( text: "No cards yet");
placeholderText.setStyle("-fx-text-fill: red;");
table.setPlaceholder(placeholderText);
return table;
}
```

FlashcardManagementScreen Code 5 - buildTable function (part 2)

This function creates a TableView with the table columns: front, back and actions (optional, only displays data if there is a row), and placeholder text saying, “No cards yet”.

3.3.4.2.1.1 Remedial Action

On initial attempts to add a flashcard and see how the table updates, I received this console output:

```
Feb 23, 2026 10:42:11 AM javafx.scene.control.cell.PropertyValueFactory getCellDataReflectively
WARNING: Can not retrieve property 'front' in PropertyValueFactory: javafx.scene.control.cell.PropertyValueFactory@1e5333f3 with provided class type: class Models.Flashcard
java.lang.IllegalStateException Create breakpoint : Cannot read from unreadable property front
    at javafx.base/com.sun.javafx.property.PropertyReference.get(PropertyReference.java:107)
    at javafx.controls/javafx.scene.control.cell.PropertyValueFactory.getCellDataReflectively(PropertyValueFactory.java:183)
    at javafx.controls/javafx.scene.control.cell.PropertyValueFactory.call(PropertyValueFactory.java:153)
    at javafx.controls/javafx.scene.control.cell.PropertyValueFactory.call(PropertyValueFactory.java:132)
    at javafx.controls/javafx.scene.control.TableColumn.getCellObservableValue(TableColumn.java:592)
    at javafx.controls/javafx.scene.control.TableColumn.getCellObservableValue(TableColumn.java:577)
    at javafx.controls/javafx.scene.control.TableCell.updateItem(TableCell.java:685)
    at javafx.controls/javafx.scene.control.TableCell.indexChanged(TableCell.java:489)
<44 folded frames>

Feb 23, 2026 10:42:11 AM javafx.scene.control.cell.PropertyValueFactory getCellDataReflectively
WARNING: Can not retrieve property 'back' in PropertyValueFactory: javafx.scene.control.cell.PropertyValueFactory@6391f1ae with provided class type: class Models.Flashcard
java.lang.IllegalStateException Create breakpoint : Cannot read from unreadable property back
    at javafx.base/com.sun.javafx.property.PropertyReference.get(PropertyReference.java:107)
    at javafx.controls/javafx.scene.control.cell.PropertyValueFactory.getCellDataReflectively(PropertyValueFactory.java:183)
    at javafx.controls/javafx.scene.control.cell.PropertyValueFactory.call(PropertyValueFactory.java:153)
    at javafx.controls/javafx.scene.control.cell.PropertyValueFactory.call(PropertyValueFactory.java:132)
    at javafx.controls/javafx.scene.control.TableColumn.getCellObservableValue(TableColumn.java:592)
    at javafx.controls/javafx.scene.control.TableColumn.getCellObservableValue(TableColumn.java:577)
    at javafx.controls/javafx.scene.control.TableCell.updateItem(TableCell.java:685)
    at javafx.controls/javafx.scene.control.TableCell.indexChanged(TableCell.java:489)
<44 folded frames>
```

Console Output 2 - Data not being added to the table

The data had not been added to the table, and this occurred because the cardTable’s PropertyValueFactory relies on reflection to find readable properties within the class, in this case the flashcard class. However, my initial Flashcard constructor used String fields, which do not align with the JavaFX observable property system. Due to this, the cardTable table view could not bind correctly to the flashcard object. Hence to resolve this I adjusted the Flashcard class constructor’s question and answer parameters to being initiated as a SimpleStringProperty. This allows the cardTable to observe the changes and automatically update.

```java
package Models;

import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;
// Class to represent a flashcard
// Using StringProperty so data can be observed and bound to table
public class Flashcard {
    // JavaFX property representing the front of card
    private SimpleStringProperty question;
    // JavaFX property representing the back of card
    private SimpleStringProperty answer;

    // Constructor to initialise a flashcard
    public Flashcard(String question, String answer){
        this.question = new SimpleStringProperty(question);
        this.answer = new SimpleStringProperty(answer);
    }
    // returns JavaFX property for the question and answer for table
binding
    public StringProperty questionProperty() { return question; }
    public StringProperty answerProperty() { return answer; }

    // Setters
    public void setAnswer(String answer) {
        this.answer.set(answer);
    }

    public void setQuestion(String question) {
        this.question.set(question);
    }

    // Getters
    public String getQuestion() {
        return question.get();
    }

    public String getAnswer() {
        return answer.get();
    }
}
```

Flashcard Code 1 - Updated Class



FlashcardManagementScreen Image 3 - Table updated with a flashcard

Moreover, I chose to display flashcards with a table view since it was the best possible option to display the data in a way which supports the editing. It allows each object to be represented as a row alongside each having its own actions. The table view also aligns with the observable list and automatically updates when flashcards are added, reducing code. Furthermore, with a table view, the interface is easier to take in, compare and manage, most commonly when a deck contains a lot of flashcards. I could have manually created a table; however, this would be time-consuming and less scalable for larger decks, requiring more scrolling for the user as well. Table views provide the clearest structure, scaling well as the deck grows and is very readable for users.

3.3.4.2.2 Saving Flashcards

When the user attempts to save flashcards, the procedure saving(..) is called:

```
// Method used to save created flashcards, outputting appropriate messages
private void saving(TableView<Flashcard> cardTable, boolean alreadyExists, boolean[] diffFile, Group contentLayer){ 1 usage
    // Checks if flashcard deck is a valid size
    if(!cardList.isEmpty() && cardList.size() <= 60) {
        // Does this already exist -> write into same file as before
        // Does this not exist already or user has chosen diffFile -> write into a new file
        if (!alreadyExists || diffFile[0]) {
            // Asks user to input the file name
            String deckNameReq = "Enter an appropriate deck name, suitable to be a file name: ";
            Optional<String> deckName = CustomAlert.input(scene.getWindow(), title: "Save Flashcards", deckNameReq, promptText: "Name the deck");
            if (deckName.isPresent()) {
                // Set file to name to flashcard deck name
                fc.setFlashcardDeckName(deckName.get());
                try {
                    // Attempt to save
                    boolean saved = fc.saveFlashcardDeck(cardList, adding: false);
                    if (saved) {
                        // Output Success message
                        CustomAlert.success(scene.getWindow(), message: "Flashcard deck has been saved");
                        // Clear table and list
                        cardList.clear();
                        cardTable.refresh();
                    }
                } catch (IOException e) {
                    // If not possible, display a suitable error message
                    CustomAlert.error(scene.getWindow(), message: "Could not save changes, try again later");
                    throw new RuntimeException(e);
                }
            }
```

FlashcardManagementScreen Code 6 - saving procedure

This will display an alert to the user, asking for the input of a file name:

```
// Displaying an alert which requires an input
public static Optional<String> input(Window alertWindow, String title, String message, String promptText){ 3 usages
    Alert a = build(alertWindow, title, message, Alert.AlertType.CONFIRMATION, ButtonType.CANCEL, ButtonType.OK);
    TextField input = new TextField();
    input.getStyleClass().add("inputAlert");
    input.setPromptText(promptText);
    Button okButton = (Button) a.getDialogPane().lookupButton(ButtonType.OK);
    okButton.setDisable(true);
    // Only enable when there is text
    input.textProperty().addListener(( ObservableValue<extends String> _, String _, String newValue) -> {
        // Remove any illegal characters
        String sanitisedInput = newValue.replaceAll( regex: "[\\\\/:*?\"<>|\\x00-\\x1F]", replacement: "").stripTrailing();
        if(!sanitisedInput.equals(newValue)) input.setText(sanitisedInput);
        okButton.setDisable(sanitisedInput.trim().isEmpty());
    });
    Label messageText = new Label(message);
    VBox content = new VBox( spacing: 15, messageText, input);
    a.getDialogPane().setContent(content);
    // Return text trimmed
    return a.showAndWait().filter( ButtonType b -> b == ButtonType.OK).map( ButtonType _ -> input.getText().trim());
}
```

CustomAlert 4 - Input Alert

The illegal characters are replaced from the input text, so file names cannot take impossible inputs resulting in errors such as:

```
Exception in thread "JavaFX Application Thread" java.nio.file.InvalidPathException Create breakpoint : Illegal char <?> at index 68: C:\Users\saina\OneDrive\Desktop\NEA\RevisionAssistant\userFolders\p/???.csv
    at java.base/sun.nio.fs.WindowsPathParser.normalize(WindowsPathParser.java:191)
    at java.base/sun.nio.fs.WindowsPathParser.parse(WindowsPathParser.java:142)
    at java.base/sun.nio.fs.WindowsPathParser.parse(WindowsPathParser.java:46)
    at java.base/sun.nio.fs.WindowsPath.parse(WindowsPath.java:92)
    at java.base/sun.nio.fs.WindowsFileSystem.getPath(WindowsFileSystem.java:203)
    at java.base/java.nio.file.Path.of(Path.java:148)
    at java.base/java.nio.file.Paths.get(Paths.java:69)
    at Controllers.FlashcardController.ensureFileExists(FlashcardController.java:79)
<29 filtered frames>
```

Console Output 3 - Invalid File Name error

An optional string is an object that may contain a string value or may contain nothing at all. I used this as the user may want to cancel the save operation, a normal string would return a null value which the flashcard controller would attempt to save with, which would be riskier. With an optional string, the robustness of the system is increased whilst the likelihood of an invalid file creation is reduced.

```
// Function to write the flashcard deck into .csv
public boolean saveFlashcardDeck(List<Flashcard> cards, boolean adding) throws IOException { 3 usages
    ensureFileExists(adding);
    String path = getFCDeckPath();
    if (path == null) path = userFolderPath + "/" + flashcardDeckName + ".csv";
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(path))) {
        for (Flashcard c : cards) {
            // Add speech marks, replace speech marks inside
            bw.write( str: "\"" + c.getQuestion().replace( target: "\"", replacement: "\"\"") + "\",\""
                    + c.getAnswer().replace( target: "\"", replacement: "\"\"") + "\"");
            bw.newLine();
        }
    }
    return true;
}
```

FlashcardController Code 2 - saveFlashcardDeck method

This functions first calls the ensureFileExists procedure where:

```
// Check if file and folder exist
private void ensureFileExists(boolean adding) throws IOException {  1 usage
    // Create a Folder if not already there
    Files.createDirectories(Paths.get(userFolderPath));
    // If not adding to a file then
    if (!adding){
        String deckPath = userFolderPath + "/" + flashcardDeckName + ".csv";
        // Create the file, if doesnt exist
        Path path = Paths.get(deckPath);
        if (!Files.exists(path)) {
            Files.createFile(path);
        } else {
            // If exists then add a random number to keep uniqueness
            int random = (int) (Math.random() * 100);
            flashcardDeckName = flashcardDeckName + random;
        }
    }
}
```

FlashcardController Code 3 - ensureFileExists procedure

This procedure will only create the userFolder specific to their email if it does not exist and then checks if there is an existing file with the same name as the user inputted deck name (set to flashcardDeckName in the saving procedure in FlashcardManagementScreen). If the file does not exist, it is created, if it already exists a random number is added to the name to keep the uniqueness.

After this procedure is executed, the saveFlashcardDeck, then calls the getFCDeckPath() function which:

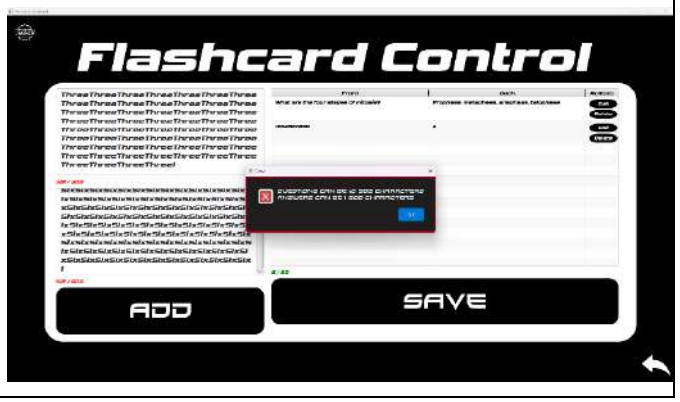
```
private String getFCDeckPath() throws IOException {  6 usages
    // If flashcard deck is stored directly in the user folder, then return it
     File rootFolder = new File(userFolderPath);
     return searchForCSV(rootFolder, flashcardDeckName);
}
//  Find the csv in the user folder
private String searchForCSV(File searchDirectory, String deckName){  2 usages
    File[] contents = searchDirectory.listFiles();
    // If no file, return null
    if (contents==null) return null;
    for(File f : contents){
        // For each file object in the array check, if directory
        // Else check if name matches
        if(f.isDirectory()){
            // Recurse into sub-folder
            String result = searchForCSV(f, deckName);
            // Return path
            if(result!=null) return result;
        } else if (f.getName().equals(deckName + ".csv")) {
            // Return path
            return f.getAbsolutePath();
        }
    }
    return null;
}
```

FlashcardController Code 4 - Searching for Deck File Path

Initially all CSVs were stored in the user folder, hence the path (for the most part) was the same but with a different file name, but as folders was created, the path became more complex, and I had to add recursive searching for the file path.

3.3.4.2.3 Testing Flashcard Creation

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	question	“What are the 4 stages of mitosis?”	Normal	Visual confirmation the input of a flashcard question. Flashcard with question added to table	Visual confirmation, added to table.	
2.	question	“1234567890” 10-character question	Boundary			
3.	question	300-character question	Boundary			
4.	question	301-character question	Erroneous	Message to tell the user the input is not valid. User given option to re-enter or stop making flashcards.	User told not valid	
5.	question	“ “ (empty string)	Erroneous		Visual confirmation, added to table.	
6.	answer	“Prophase, metaphase, anaphase, telophase”	Normal	Visual confirmation the input of a flashcard question. Flashcard with question added to table	Visual confirmation, added to table.	
7.	answer	“A” 1-character	Boundary			
8.	answer	600-character answer	Boundary			
9.	answer	601-character answer	Erroneous	Message to tell the user the input is not valid. User given option to re-enter or stop making flashcards.	User told not valid	
10.	answer	“ ” (empty string)	Erroneous		Visual confirmation, added to table.	
11.	flashcards (list)	Deck with 5	Normal	The deck is saved, confirmation message shown	The deck is saved, confirmation message shown	
12.	flashcards (list)	Deck with 60	Boundary			
13.	flashcards (list)	Deck with 0	Erroneous	Error message shown, you cannot revise an empty deck.	Error message shown	
14.	deckName	“Physics”	Normal	Accepted; the file will load/save successfully.	File successfully saved	
15.	deckName	“???”	Erroneous	Will not accept the input at all	Does not accept input	

Test No.	Proof
1.	 
2.	 
3.	 
4.	 

5.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

6.


Flashcard Control
No cards yet
ADD
SAVE


Flashcard Control
ADD
SAVE

7.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

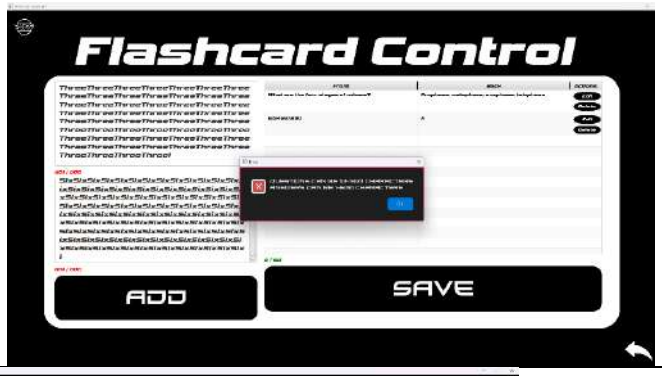
8.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

9.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

10.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

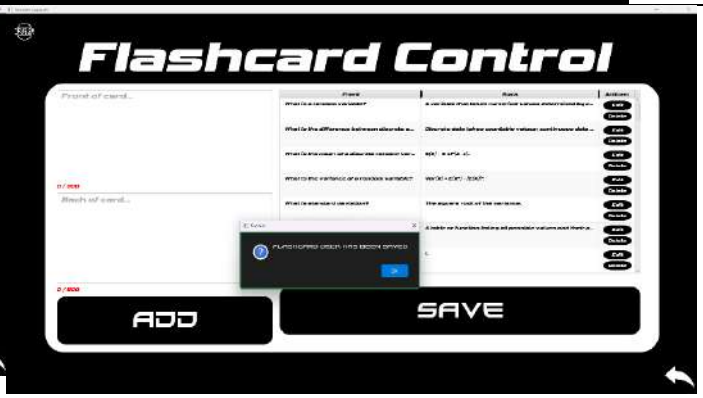
11.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

12.


Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

13.	
14.	 
15.	

3.3.4.2.4 Remedial Action

To remediate the issue of question-and-answer inputs allowing empty strings I have changed the addFlashcard to:

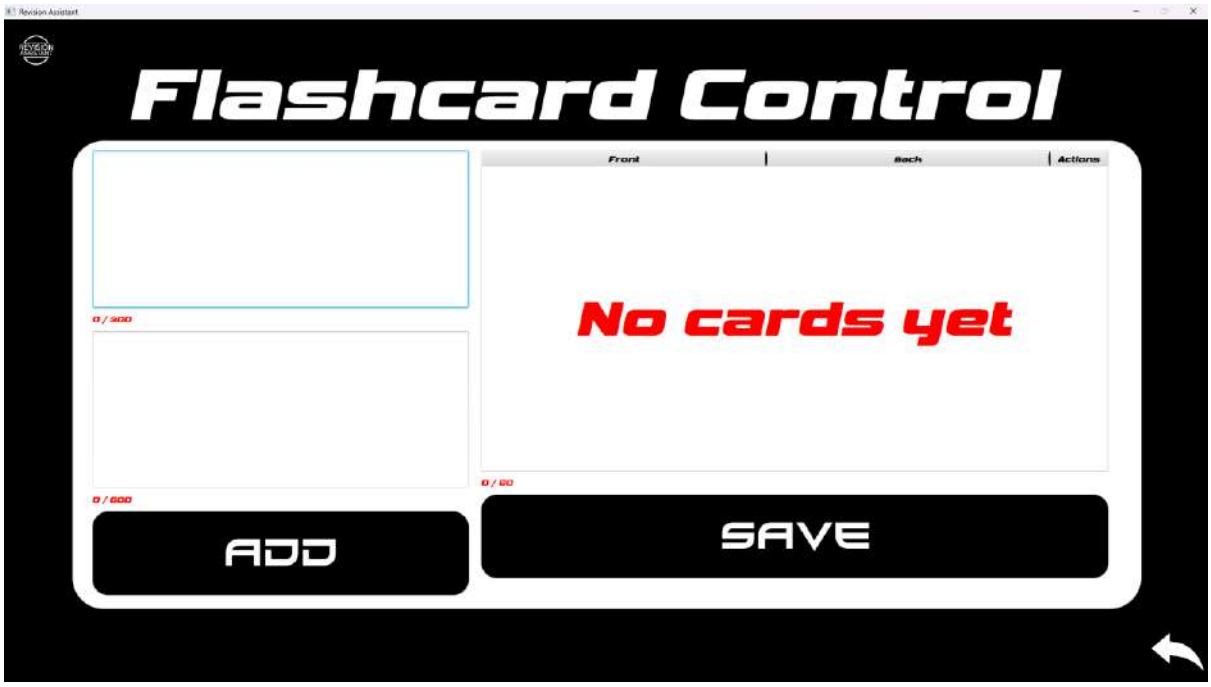
```
// Procedure to add flashcards to the list
private void addFlashcard(TextArea q, TextArea a){  1 usage
    // Question and Answer constraints
    String qReq = "Questions can be 10-300 characters";
    String aReq = "Answers can be 1-600 characters";
    // Remove white space and get the text
    String question = q.getText().trim();
    String answer = a.getText().trim();
    // Check if valid
    boolean validQ = question.length() >= 10 && question.length() <= 300;
    boolean validA = !answer.isEmpty() && answer.length() <= 600;
    // Add to list
    if(validQ && validA){
        cardList.add(new Flashcard(question, answer));
        // Clear the fields for next input
        q.clear();
        a.clear();
    }else{
        // Display error
        CustomAlert.error(scene.getWindow(),  message: qReq + "\n" + aReq);
    }
}
```

FlashcardManagementScreen Code 7 - Updated addFlashcard procedure

The procedure now checks if the trimmed (white space removed) length of question and answer is valid. This was also updated in UIComponents for the text area character counter:

```
// Creates a text area with prompt text and a character counter
public TextAreaBundle createTextArea(String promptText, int charLimit, int lowerLimit) {  2 usages
    TextArea textArea = new TextArea();
    textArea.getStyleClass().add("textArea");
    textArea.setPromptText(promptText);
    Label characterCounter = new Label( text: "0 / " + charLimit);
    characterCounter.getStyleClass().add("counter");
    // Initial Colour
    characterCounter.setStyle("-fx-text-fill: red;");
    textArea.textProperty().addListener(( ObservableValue<extends String> _,  String _,  String newValue) -> {
        // Length when white space is removed
        int trimmedLength = newValue.trim().length();
        // Change the number
        characterCounter.setText(trimmedLength + " / " + charLimit);
        // Change label colours depending on limits
        if (trimmedLength < lowerLimit || trimmedLength > charLimit) {
            characterCounter.setStyle("-fx-text-fill: red;");
        }else{
            characterCounter.setStyle("-fx-text-fill: green;");
        }
    });
    return new TextAreaBundle(textArea, characterCounter, charLimit, lowerLimit);
}
```

UIComponents Code 2 - Updated createTextArea function



FlashcardManagementScreen Image 4 - Empty String Input



FlashcardManagementScreen Image 5 - Error when empty string

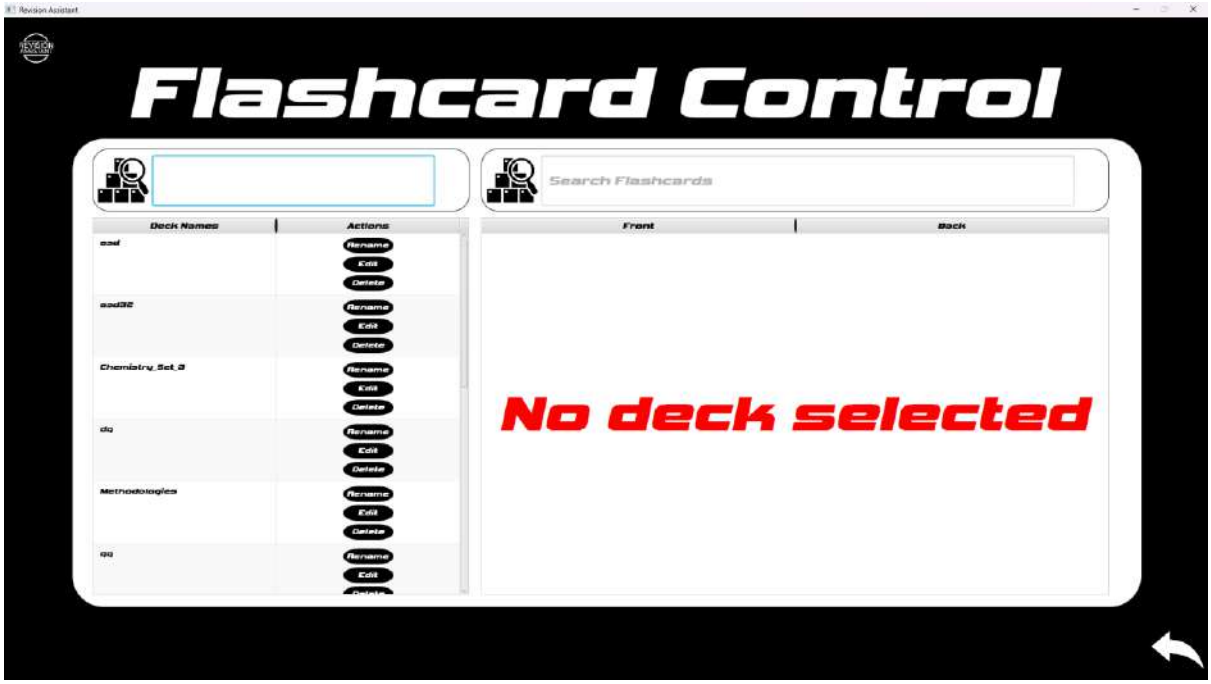
This shows the remediated actions have worked, and the user can not enter empty strings anymore as flashcard inputs, and further tests confirm the system behaves as expected for this stage.

3.3.4.3 Post-Creation Flashcard (Deck) Modification

Following the implementation of flashcard deck creation, it became necessary to have a way for users to modify existing flashcards and decks, whilst being able to find them easily if they have a lot of decks (with search fields). Without having a modification feature, users would be forced to recreate entire decks if they would want to change anything, which is not user-friendly.

3.3.4.3.1 Initial Modification Page

The modification page consists of two search fields, one for searching decks and another for searching flashcards within decks. With two tables below them, a decks table and a flashcard table (built using the re-usable buildTable function, however this time without the actions column, as the user cannot edit individual flashcards on this page).



FlashcardManagementScreen Image 6 - Modification Page

The textfields were created via the re-usable functions in UIComponents which was also used for the email and recovery word inputs in prior prototypes.

```
// Creates a text bundle with an icon
public TextBundle buildTextFieldInput(int rectX, int rectY, int rectW, int rectH, int fieldX, int fieldY,  11 usages
                                      String promptText, String iconURL, int iconX, int iconY, int iconW, int iconH){
    // Create background rectangle for text input
    Rectangle inputRect = new Rectangle(rectX, rectY, rectW, rectH);
    inputRect.getStyleClass().add("inputRect");
    // Create text field
    TextField tf = new TextField();
    tf.setPromptText(promptText);
    tf.setLayoutX(fieldX);
    tf.setLayoutY(fieldY);
    tf.setContextMenu(createContextMenu(tf));
    // Create icon
    ImageView icon = new ImageView(iconURL);
    icon.setLayoutX(iconX);
    icon.setLayoutY(iconY);
    icon.setFitHeight(iconH);
    icon.setFitWidth(iconW);
    Group tfInputs = new Group(inputRect, tf);
    return new TextBundle(tfInputs, tf, icon);
}
```

UIComponents Code 3 - buildTextFieldInput function

3.3.4.3.2 Populating the decksTable table

The decks table which is a table view of type String was populated by calling the getFileContents() method via the flashcard controller object.

```
decksTable.getItems().addAll(fc.getFileContents());
```

FlashcardManagementScreen Code 8 - Populating the decksTable in buildModificationContent()

The decksTable consists of every flashcard deck within the user's custom directory, this means that the getFileContents function will have to recursively search sub-folders in the userFolder directory to add all file names to a string. Without this function, decks within sub-folders in the user directory would not be able to be updated since they would not be looked for.

```
// Return array list of all decks linked to an account
public ArrayList<String> getFileContents() {  1 usage
    ArrayList<String> deckNames = new ArrayList<>();
    File userFolder = new  File(userFolderPath);
    if (!userFolder.exists()){
        return  deckNames;
    }
    getAllCSV(userFolder, deckNames);
    return deckNames;
}
// Recursively search through folder and subfolders
// to add all csv files to the arraylist
private void getAllCSV(File folder, ArrayList<String> deckNames){  2 usages
    File[] contents = folder.listFiles();
    if(contents == null) return;
    // Loop for each file
    for(File file : contents){
        // If folder, then recurse into the directory
        if(file.isDirectory()){
            getAllCSV(file, deckNames);
        } else if (file.getName().endsWith(".csv")) {
            // Add to the array list
            deckNames.add(file.getName().replace( target: ".csv",  replacement: ""));
        }
    }

}
```

FlashcardController Code 5 - getFileContents()

This returns the file name replacing the .csv with blank space so that the file name which the user set can be used as an identifier. This ArrayList is then set to the decksTable.

Before this, I had used the following function:

```
// Return array list of all decks linked to an account
public ArrayList<String> getFlashcardDecks() throws IOException {
    ArrayList<String> deckNames = new ArrayList<>();
    File userFolder = new  File(userFolderPath);
    if (!userFolder.exists()) {
        return  deckNames;
    }
    for(File f : Objects.requireNonNull(userFolder.listFiles())){
        if(f.getName().endsWith(".csv")){
            deckNames.add(f.getName().replace(".csv",""));
        }
    }
    return deckNames;
}
```

FlashcardController Code 6 - getFlashcardDecks()

This function was not ideal for the future addition of methods of the system, since later I was going to add organising flashcard decks into subfolders. This function only checked for the files within the user folder, not recursively into others.

3.3.4.3.3 Populating the cardPreview Table

Now that the decksTable had been successfully implemented, I added a listener to the selection property of the table, so that on selection of a table row, the cardPreview table could be populated with the actual flashcards within its deck.

```
// Listener to change the card preview based on selection
decksTable.getSelectionModel().selectedItemProperty().addListener(( ObservableValue<extends String> _, String _, String newValue) -> {
    if(newValue == null) return;
    try{
        cardsPreview.getItems().setAll(fc.loadFlashcardDeck(newValue));
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
});
decks.getChildren().addAll(searchDecks.group(), decksTable);
```

FlashcardManagementScreen Code 9 - Listener to populate cardPreview based on user selection

The loadFlashcardDeck method was used, which is:

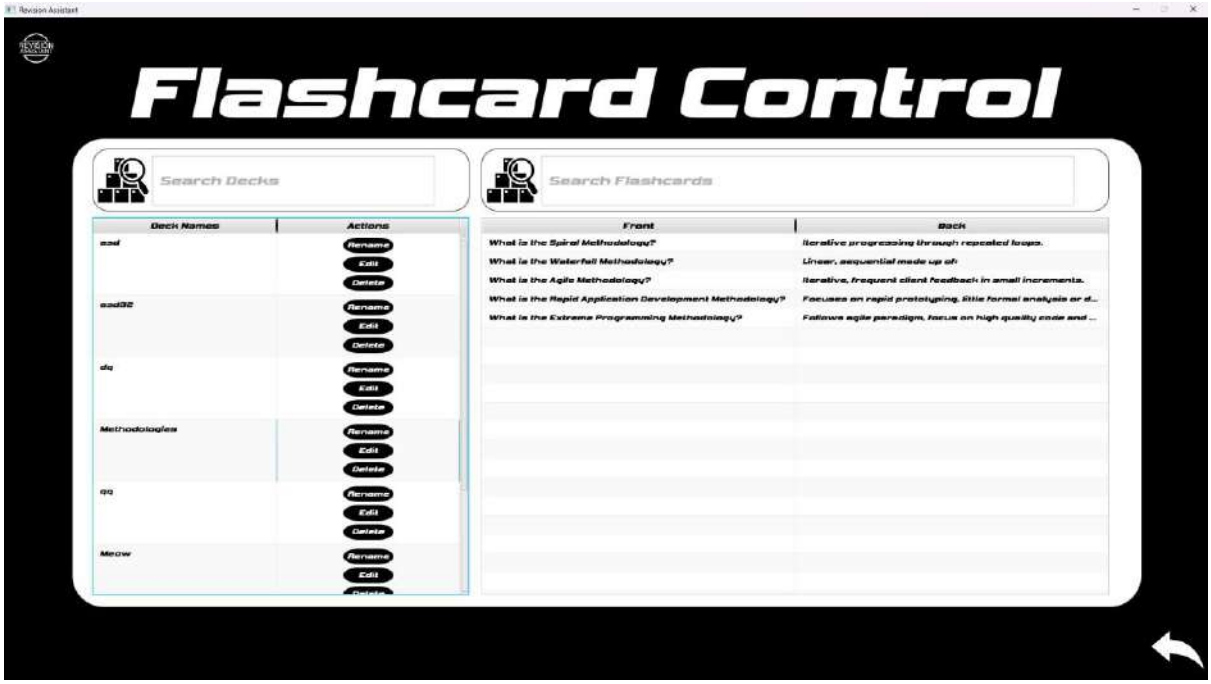
```
// Function to read in flashcards from a .csv into an array list
public ArrayList<Flashcard> loadFlashcardDeck(String deckName) throws IOException {
    ArrayList<Flashcard> cards = new ArrayList<>();
    // Set deck to get from parameter
    this.flashcardDeckName = deckName;
    String path = getFCDeckPath();
    if (path == null) return cards;
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        readData(br, cards);
    }catch (IOException e){
        // if not available, corrupt
        quarantineCorruptedCSV(e, Path.of(getFCDeckPath()));
    }
    return cards;
}
```

FlashcardController Code 7 - loadFlashcardDeck function

This function uses the readData procedure which reverses what saving the flashcard did.

3.3.4.3.3.1 Remediating Flashcard Saving

Whilst testing the card preview function, on clicking a deck I had created to test:



FlashcardManagementScreen Image 7 - cardPreview Working but...

The flashcard, question and answer fields are not saved properly. The flashcard answer should show up with multiple lines, however the system acts as if it is only a one-line answer. The flashcard question and answer would not be loaded properly if on creation you start a new line. This is a CSV parsing error, since the buffered reader reads line by line a new line within the flashcard caused the answer to be split into multiple rows and the parser loses track of the question/answer boundaries.

Hence to remediate this issue, I changed the way the data was written into the file at first. New lines were, within the actual csv were replaced with \n, which was converted back when reading. The updated methods were:

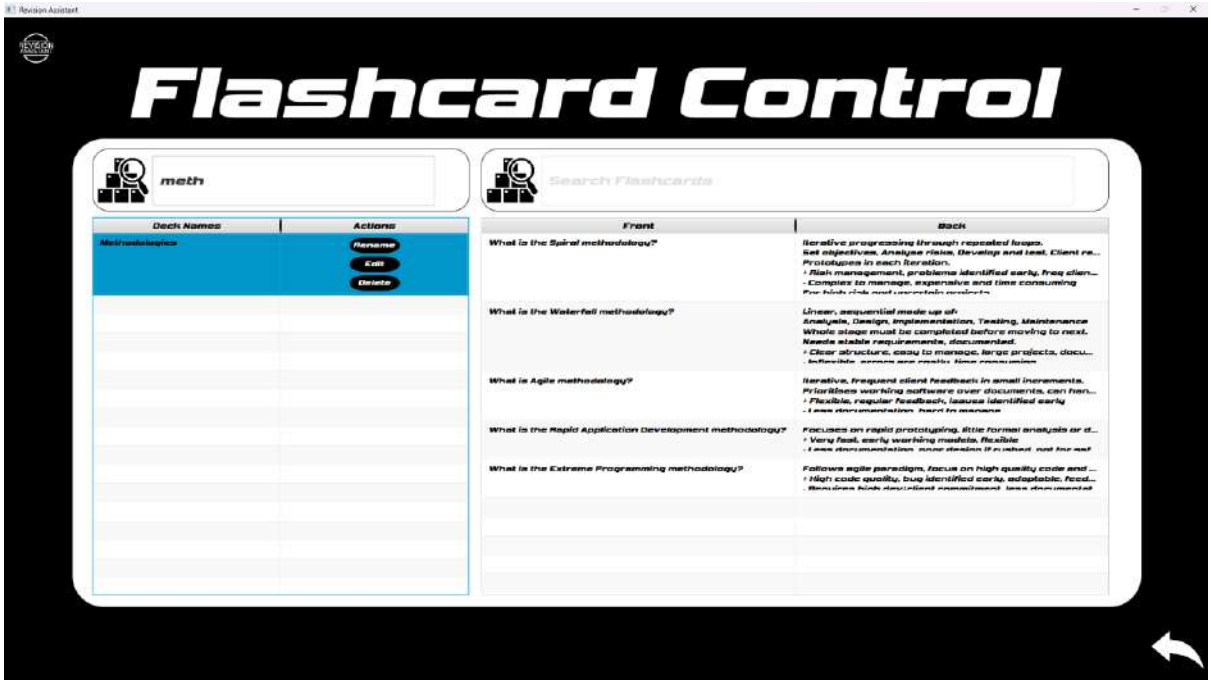
```
// Function to write the flashcard deck into .csv
public boolean saveFlashcardDeck(List<Flashcard> cards, boolean adding) throws IOException {  3 usages
    ensureFileExists(adding);
    String path = getFCDeckPath();
    if (path == null) path = userFolderPath + "/" + flashcardDeckName + ".csv";
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(path))) {
        for (Flashcard c : cards) {
            // Add speech marks, replace speech marks inside
            bw.write( str: "\"" // Opening speech marks
                    + c.getQuestion().replace( target: "\"", replacement: "\"\"") // Get rid of any speech marks within by adding another
                    .replace( target: "\n", replacement: "\\n") // Replace new lines with \n so readline does not break mid-card
                    + "\",\"" // Closing speech marks with comma
                    + c.getAnswer().replace( target: "\"", replacement: "\"\"").replace( target: "\n", replacement: "\\n") + "\n");
            bw.newLine();
        }
    }
    return true;
}
```

FlashcardController Code 8 - updated saveFlashcardDeck method

```
// Re-usable method convert csv to flashcard object list
private void readData(BufferedReader br, ArrayList<Flashcard> cards) throws IOException {  3 usages
    String line;
    while ((line = br.readLine()) != null) {
        // Split at ,
        String[] data = line.split( regex: "\",\"");
        if(data.length == 2){
            // Replace speech marks with empty space
            String question = data[0].replace( target: "\"",  replacement: "").replace( target: "\\n",  replacement: "\n");
            String answer = data[1].replace( target: "\"",  replacement: "").replace( target: "\\n",  replacement: "\n");
            // Add a new flashcard object
            cards.add(new Flashcard(question, answer));
        }
    }
    // Error indication to user, if file can't be read
    if(cards.isEmpty()) CustomAlert.error(javafx.stage.Window.getWindows().getFirst(),
            message: "Problems reading file, might not be of correct format");
}
```

FlashcardController Code 9 - readData method updated

With these updates the flashcard data was correctly written into files, and read, with the preview table also working as expected



FlashcardManagementScreen Image 8 - cardPreview Fixed

3.3.4.3.4 Searching Tables

Two separate search fields were added to the modification page for distinct functionalities. The field on the left (search decks) will allow the user to search for flashcard deck names, whilst the field on the right (search flashcards) will search the individual flashcards within decks to see if results match and show what deck they belong to. Although both searches update the same table (decksTable), they were split to reduce the ambiguity of the interface. I could have implemented one search field, combining both searching mechanisms but that would cause the behaviour of the

search to be ambiguous, user would not be able to determine if the result of their search is an actual deck name or a flashcard within a deck. Hence, separating these fields into two would improve usability and reduce cognitive load.

```
// Listeners for searches
// All initial decks
ArrayList<String> allDecks = new ArrayList<>(decksTable.getItems());
searchDecks.tf().textProperty().addListener(( ObservableValue<extends String> _, String _, String newValue) -> {
    TextField searchFlashcardTF = searchFlashcards.tf();
    cardsPreview.getItems().clear();
    if(newValue.isEmpty()){
        searchFlashcardTF.setDisable(false);
        searchFlashcards.group().setOnMouseClicked(null);
        // If no search just keep results to all decks
        decksTable.getItems().setAll(allDecks);
        return;
    }else {
        // Can't have two searches at once
        searchFlashcards.group().setOnMouseClicked( MouseEvent _-> CustomAlert.error(scene.getWindow(),
                message: "Clear the deck search to search flashcards"));
        searchFlashcardTF.setDisable(true);
        // Set tableView contents to the following
        decksTable.getItems().setAll(
                allDecks.stream() // Convert the array list into a stream for processing
                        // Filter so decks which match the search condition are shown
                        .filter( String deck -> deck.toLowerCase().contains(newValue.trim().toLowerCase()))
                        // Add decks matching the criteria to a list
                        .collect(Collectors.toList()));
    }
});
```

FlashcardManagementScreen Code 10 - Search Decks Listener

```
searchFlashcards.tf().textProperty().addListener(( ObservableValue<extends String> _, String _, String newValue) -> {
    TextField searchDeckTF = searchDecks.tf();
    cardsPreview.getItems().clear();
    if(newValue.isEmpty()){
        searchDeckTF.setDisable(false);
        searchDecks.group().setOnMouseClicked(null);
        // If no search just keep results to all decks
        decksTable.getItems().setAll(allDecks);
        return;
    }else{
        searchDecks.group().setOnMouseClicked( MouseEvent _-> CustomAlert.error(scene.getWindow(),
                message: "Clear the flashcard search to search decks"));
        searchDeckTF.setDisable(true);
        // Create an Array List of the decks which match criteria
        ArrayList<String> decksMatchingCriteria = new ArrayList<>();
        for (String deck : allDecks){
            try{
                ArrayList<Flashcard> cards = fc.loadFlashcardDeck(deck);
                boolean matchFound = cards.stream().anyMatch(
                        Flashcard card ->
                                card.getQuestion().toLowerCase().contains(newValue.trim().toLowerCase()) ||
                                card.getAnswer().toLowerCase().contains(newValue.trim().toLowerCase())
                );
                if(matchFound){
                    decksMatchingCriteria.add(deck);
                }
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }decksTable.getItems().setAll(decksMatchingCriteria);
    }
});
```

FlashcardManagementScreen Code 11 - Search Flashcards Listener

Both searches cannot be conducted at the same time so, the user must clear one field before using the other, this is because the results would be inaccurate as the system would not know what to prioritise (the deck search or the flashcard search).

For the deck search, all the rows within the table are added to an array list and using the .stream method, a sequential stream is returned which is then filtered to only have deck names which match the search criteria in it, then converted to a list to be set as the decksTable's items in real time.

For the flashcard search, for each deck in the allDecks ArrayList, their cards are loaded into an ArrayList and converted into a stream to check if they match the search criteria, if so add them to the decksMatchingCriteria ArrayList which will be set to the decksTable in real time.

3.3.4.3.5 Actions within the decksTable

3.3.5.3.5.1 Renaming

There are three action buttons a user can perform within the decksTable: rename, edit and delete.

```
// Renaming the file logic
rename.button().setOnMouseClicked( MouseEvent _->{
    String deckName = getTableView().getItems().get(getIndex());
    if(CustomAlert.confirm(scene.getWindow(), title: "Confirmation", message: "Are you sure you want to rename " + deckName + "?")){
        Optional<String> newName = CustomAlert.input(scene.getWindow(), title: "Rename the deck",
                message: "Enter a suitable new deck name", promptText: "Enter new deck name");
        if (newName.isPresent()) {
            fc.renameDeck(deckName, newName.get());
            decksTable.getItems().set(getIndex(), newName.get());
            CustomAlert.success(scene.getWindow(), message: "Successfully renamed deck " + deckName + " to " + newName.get());
        }
    }
});
```

FlashcardManagementScreen Code 12 - on Rename button click

This asks the user for a file name as an input and then calls the renameDeck() procedure and update the table in real time, outputting a success message.

```
// Function to change the name of a flashcard deck
public boolean renameDeck(String oldName, String newName) {  1 usage
   this.flashcardDeckName = oldName;
    try{
        String oldPath = getFCDeckPath();
        if(oldPath == null) return false;
        // Write into same folder
        Path newPath = Paths.get( first: new File(oldPath).getPath() + "/" + newName + ".csv");
        Files.move(Paths.get(oldPath), newPath, StandardCopyOption.REPLACE_EXISTING);
        return true;
    }catch (IOException e){
        return false;
    }
}
```

FlashcardController Code 10 - renameDeck Function

However, on initial tests, the name was updated in the table and had a confirmation message, but within the actual file structure it was not updated and when selected, the flashcards within the deck did not show.





To remediate this, I updated renameDeck to:

```
// Function to change the name of a flashcard deck
public boolean renameDeck(String oldName, String newName) {  1 usage
    this.flashcardDeckName = oldName;
    try{
        String oldPath = getFCDeckPath();
        if(oldPath == null) return false;
        // Write into same folder
        Path oldNamePath = Paths.get(oldPath);
        // Joins directory with new filename to build destination
        Path newNamePath = oldNamePath.getParent().resolve( other: newName + ".csv");
        Files.move(oldNamePath, newNamePath, StandardCopyOption.REPLACE_EXISTING);
        // update after successfull rename
        this.flashcardDeckName = newName;
        return true;
    }catch (IOException e){
        return false;
    }
}
```

FlashcardController Code 11 - renameDeck updated

The problem was the original function changed the file to be placed in for examples “/userFolders/user/oldName.csv” to “/userFolders/user/oldName.csv/newname.csv”. This means the file was not actually renamed, as the system attempted to move it into a directory which did not exist leaving an invalid path. To fix it, I used the resolve() function to append the file name to the parent directory instead.

3.3.5.3.5.2 Deleting Decks

```
// Deleting the flashcard file logic
delete.button().setOnMouseClicked( MouseEvent _->{
    String deckName = getTableView().getItems().get(getIndex());
    if(CustomAlert.confirm(scene.getWindow(), title: "Confirmation", message: "Are you sure you want to delete " + deckName + "?")){
        try {
            fc.deleteDeck(deckName);
            String selectedRow = getTableView().getSelectionModel().getSelectedItem();
            getTableView().getItems().remove(selectedRow);
            cardsPreview.refresh();
            CustomAlert.success(scene.getWindow(), message: "Successfully deleted deck " + deckName);
        } catch (IOException e) {
            CustomAlert.error(scene.getWindow(), message: "Unsuccessful, try again later");
            throw new RuntimeException(e);
        }
    }
});
```

FlashcardManagementScreen Code 13 - on Delete Mouse Click

The user will be asked to confirm the deletion, if confirmed, the deleteDeck method will be called and the row will be removed from the table and the file structure, with a confirmation message.

```
// Procedure to delete the deck user selects
public void deleteDeck(String deckName) throws IOException {
    // Set deck name so getFCDeckPath can search
    this.flashcardDeckName = deckName;
    // get path
    String path = getFCDeckPath();
    if (path == null) return;
    // delete from path
    Files.delete(Paths.get(path));
}
```

FlashcardController Code 12 - deleteDeck method

3.3.5.3.5.3 Editing existing decks

```
// Editing the flashcard deck
edit.button().setOnMouseClicked( MouseEvent _->{
    String deckName = getTableView().getItems().get(getIndex());
    if(CustomAlert.confirm(scene.getWindow(), title: "Confirmation", message: "Are you sure you want to add to " + deckName + "?")){
        fc.setFlashcardDeckName(deckName);
        try {
            cardList.setAll(fc.loadFlashcardDeck(deckName));
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        // Takes you to creation page
        buildCreationContent(contentLayer, alreadyExists: true);
    }
});
```

FlashcardManagementScreen Code 14 - on edit button mouse click

The observable card list is set to the ArrayList of flashcards and the content is changed to the creation of flashcards content.

```
    }else{
        // Tell user they are writing to same file
        if(CustomAlert.confirm(scene.getWindow(), title: "Confirmation", message: "You are writing to the previous file, if this not ideal, cancel and try again")){
            try {
                boolean saved = fc.saveFlashcardDeck(cardList, adding: true);
                if(saved) {
                    cardList.clear();
                    cardTable.refresh();
                    CustomAlert.success(scene.getWindow(), message: "Changes have been saved, you will be returned to modification");
                    nav.goBack(contentLayer);
                }
            } catch (IOException e) {
                CustomAlert.error(scene.getWindow(), message: "Could not save changes, try again later");
                throw new RuntimeException(e);
            }
        }else{
            // Ask for confirmation they want to write into a new file
            if(CustomAlert.confirm(scene.getWindow(), title: "Confirmation", message: "Would you like to write into a different file?")){
                diffFile[0] = true;
            }
        }
    }
}else{
    // Remind user of constraints
    CustomAlert.error(scene.getWindow(), message: "Minimum of 0 and maximum of 50 cards in a deck");
}
```

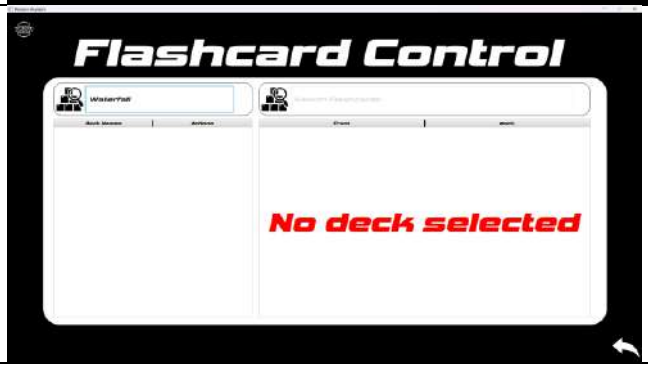
FlashcardManagementScreen Code 15 - second part of saving function

After the user makes the changes (they want to make), and press save, the saving function will be called again and alerts the user they will be saving to the same file, if they don't want to do this, they can cancel this and save again to save to a new file which they name.

3.3.4.5.6 Testing Flashcard Modification

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	Search deck	“Methodologies”	Normal	Deck appears on the table	Deck appears on the table	
2.	Search deck	“METHODOLOGY”	Normal	Deck appears on the table	Deck appears on the table	
3.	Search deck	“ ”	Erroneous	All decks on decksTable	All decks on decksTable	
4.	Search deck	“Waterfall” (to see if deck comes up since it’s in a in a flashcard)	Erroneous	No decks on decksTable	No decks on decksTable	
5.	Search fc	“Waterfall”	Normal	Deck with flashcard appears on table	Deck with flashcard appears on table	
6.	Search fc	“WATERFALL”	Normal	Deck with flashcard appears on table	Deck with flashcard appears on table	
7.	Search fc	“ ”	Erroneous	All decks on decksTable	All decks on decksTable	
8.	Search fc	Search a deck name	Erroneous	No decks on decksTable	No decks on decksTable	
9.	Renaming file	“renamed”	Normal	File renamed successfully	File renamed successfully	
10.	Renaming file	“????”	Erroneous	Does not accept input	Does not accept input	
11.	Deleting file	Delete a deck	Normal	File deleted	File deleted	
12.	Editing deck	Add a flashcard then save to same file	Normal	Deck saved with new flashcard successfully and deck in table when returned to modification page	Deck saved successfully, but not in table when returned to page.	

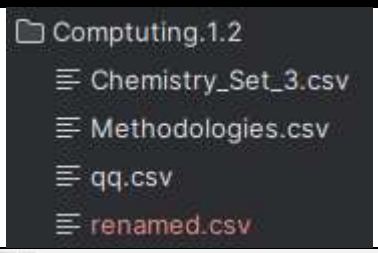
13.	Editing deck	Add a flashcard and save to different file	Normal	New deck saved with new flashcard successfully and deck in table when returned to modification page	Deck saved successfully, but not in table when returned to page.	

Test No.	Proof
1.	
2.	
3.	
4.	

5.	 Flashcard Control No deck selected
6.	 Flashcard Control No deck selected
7.	 Flashcard Control No deck selected
8.	 Flashcard Control No deck selected
9.	


Flashcard Control


Flashcard Control

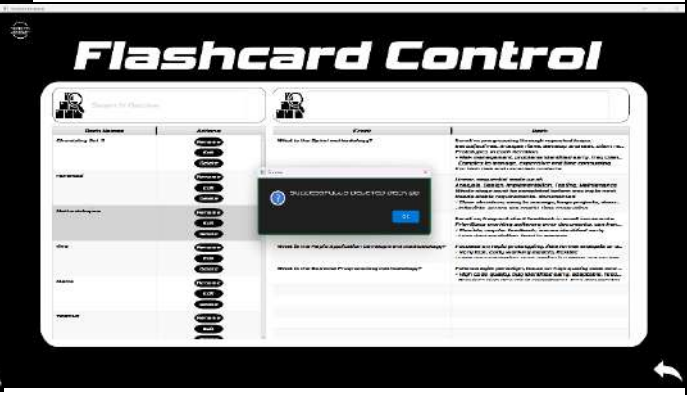

Comptuting.1.2
Chemistry_Set_3.csv
Methodologies.csv
qq.csv
renamed.csv

10.


Flashcard Control

11.


Flashcard Control


Flashcard Control

12.

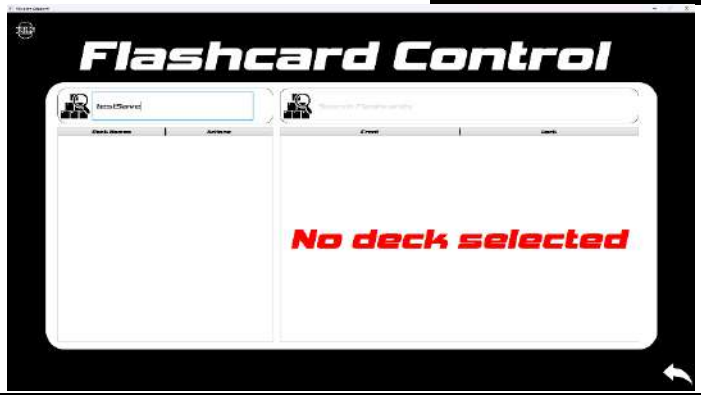

Flashcard Control
ADD
SAVE


Flashcard Control
ADD
SAVE

13.







3.3.4.5.7 Remedial Action

Once the file was renamed, nav.goBack(contentLayer) was called, this restores the old modification pages' nodes from the history stack, and since the getFileContents() is not called in this, the file contents are the same as prior to saving. To fix this I have decksTable and allDecks global variables and they will be updated accordingly when saving occurs.

```
// Decks table to be updated in save
private final TableView<String> decksTable = new TableView<>();
// ArrayList to be updated when saved.
private ArrayList<String> allDecks;  9 usages
```

FlashcardManagementScreen Code 16 - new global variables

When saving with a new file name:

```
if(alreadyExists) {
    allDecks.clear();
    allDecks.addAll(fc.getFileContents());
    // Update deck table
    decksTable.getItems().addAll(fc.getFileContents());
    nav.goBack(contentLayer);
}
```

FlashcardManagementScreen Code 17 - code added to saving procedure

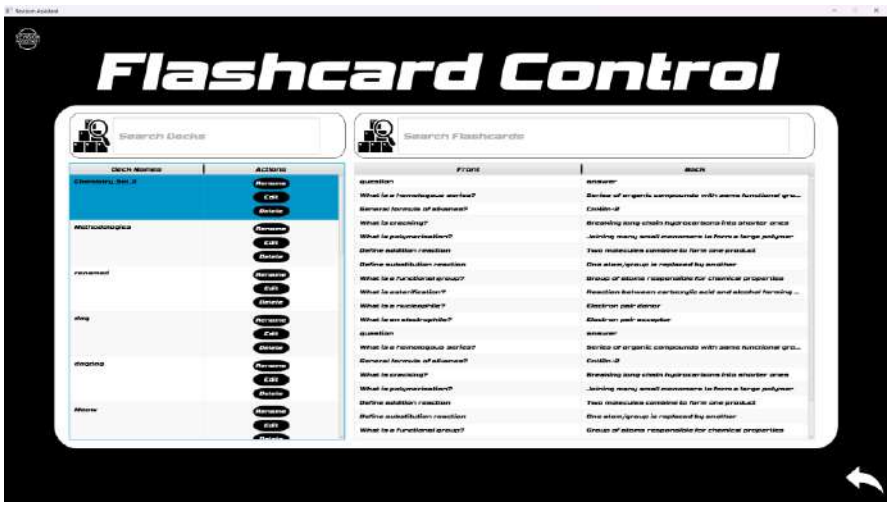
When saving to same file:

```
// Update deck table
allDecks.clear();
allDecks.addAll(fc.getFileContents());
decksTable.getItems().addAll(fc.getFileContents());
nav.goBack(contentLayer);
```

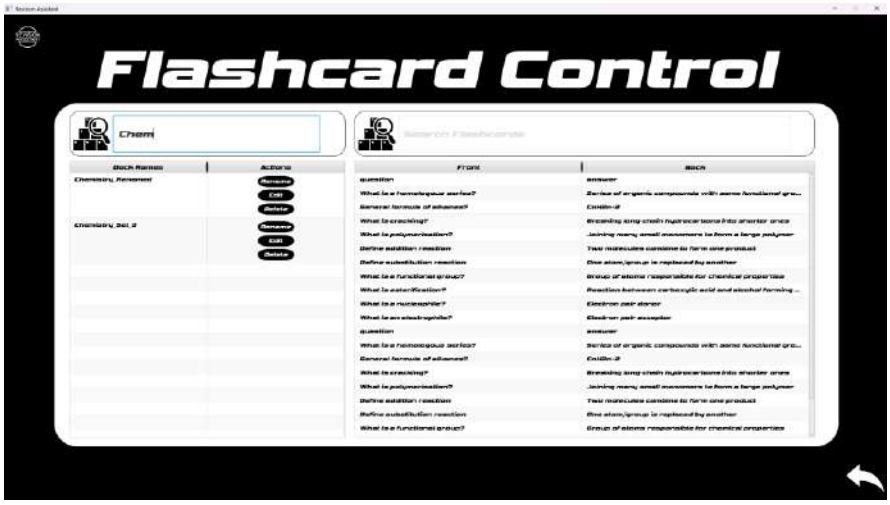
FlashcardManagementScreen Code 18 - code added to saving procedure

This will update the table and ArrayList according to the new file contents, so new files are saved and then the content is switched backed.

I will test this by attempting to edit the Chemistry_Set_3 and save it as “Chemistry_Renamed”.



FlashcardManagementScreen Image 9 - Chemistry deck selected



FlashcardManagementScreen Image 10 - Chemistry_Renamed found

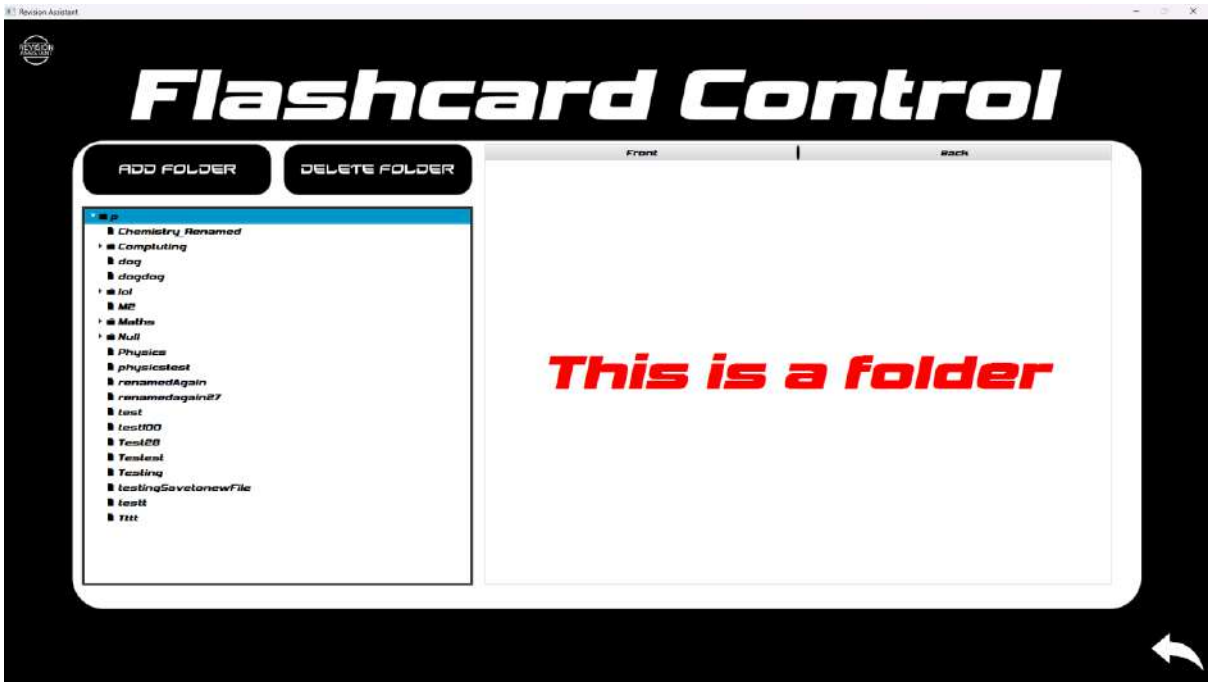
This test proves the table is updated after saving and returned here, and the remedial actions have been successfully implemented.

3.3.4.4 Organising Flashcard Decks

As the number of flashcard decks increases, revision can become too overwhelming if they are all placed inside ONE folder, hence it's necessary to implement an organisation solution, preventing the decks from being cluttered. Without this, the user would have to resort to manually search through an expanding list of deck, decreasing readability.

To implement this, in the organisation page of the flashcard management screen, I have added two buttons: add and delete folder, a tree view to replicate the user directory structure as well as a table view without the action column to give the user a preview of the cards.

A tree view maintains a clear file hierarchy, reflecting the actual file storage structure on the user's disk, making it easier for them to understand how structuring their revision on the RevisionAssistant works. Compared to other views (list, table), the tree view would provide improved scalability and clearer navigation, whilst reducing clutter with having collapsible branches.



FlashcardManagementScreen Image 11 - Organisation Page

3.3.4.4.1 Building the tree

The tree was built with the help of the buildTree function in FlashcardManagementScreen which uses the getUserDirectoryStructure function via the FlashcardController object.

```
// Function to build the tree which displays the folder for organisation
private TreeView<String> buildTree(){  1 usage
    // Create the initial TreeView
    TreeView<String> folderTreeView = new TreeView<>();
    folderTreeView.getStyleClass().add("folderTreeView");
    // Enables possibility of edit
    folderTreeView.setEditable(true);
    folderTreeView.setShowRoot(true);

    // Build existing tree copying directory structure
    TreeItem<String> folderStructure = fc.getUserDirectoryStructure(ac.getCurrentUser().getEmail());
    folderTreeView.setRoot(folderStructure);
    folderTreeView.getRoot().setExpanded(true);
    return folderTreeView;
}
```

FlashcardManagementScreen Code 19 - buildTree function

```
// Function to return the full directory structure for the current user
public TreeItem<String> getUserDirectoryStructure(String email) {  1 usage
    File userDirectory = new File(userFolderPath);
    // create root node of tree with first user folder
    TreeItem<String> root = new TreeItem<>( value: "\uD83D\uDCC1 " + email);
    // Populate tree with remaining contents
    buildStructureTree(root, userDirectory);
    return root;
}
// Recursively search a directory and add its content to the tree
private void buildStructureTree(TreeItem<String> node, File parentDirectory) {  2 usages
    // Place all files and folders inside the parent directory into an array
    File[] allUserFiles = parentDirectory.listFiles();
    // If there are no files stop here
    if(allUserFiles == null) return;
    // Loop through all the contents
    for(File f : allUserFiles) {
        System.out.println("Found: " + f.getName() + " isDirectory: " + f.isDirectory());
        if(f.isDirectory()) {
            // If it's a folder, create a folder node with the prefix and file name
            TreeItem<String> folderNode = new TreeItem<>( value: "\uD83D\uDCC1 " + f.getName());
            // Recursively go into the subfolder to add its contents
            buildStructureTree(folderNode, f);
            // add current node as a child to the current node
            node.getChildren().add(folderNode);
        }else if (f.getName().endsWith(".csv")) {
            // csv file storing deck info
            // Add as a leaf node to a tree, replace .csv with prefix and the file name
            node.getChildren().add(new TreeItem<>( value: "\uD83D\uDCC4 " + f.getName().replace( target: ".csv", replacement: "")));
        }
    }
    // All other files which are not a folder or ending in .csv are ignored
}
```

FlashcardController Code 13 - getting User Directory Structure

This function is used to build a tree structure replicating the folder structure of the user directory for their flashcards. It returns the children all attached to the root node (user directory) and this is set to the root of the tree view; it also adds the Unicode characters for the emojis of folders and files.

To enable the user to edit the file and folder structure easily, the addDragAndDrop procedure was run on the tree view, which was split into 5 parts: drag detected, drag over, drag entered, drag exited and drag dropped.

```
// Procedure which will allow the dragging and dropping of files within the tree
private void addDragAndDrop(TreeView<String> tree){  1 usage
    // Creating custom cells with drag and drop feature
    tree.setCellFactory( TreeView<String> _-> {
        // Create a new cell with item's string value
        TreeCell<String> cell = updateItem(item, empty) -> {
                super.updateItem(item, empty);
                // Show text if cell has content, otherwise leave it blank
                setText(empty || item == null ? null : item);
        };
        // User begins dragging
        cell.setOnDragDetected( MouseEvent e->{
            // Don't allow the moving of empty cells
            if(cell.isEmpty()) return;
            // Start a move drag operation on the cell
            Dragboard db = cell.startDragAndDrop(TransferMode.MOVE);
            // Put cell's text into a clipboard to know what's being dragged
            ClipboardContent cc = new ClipboardContent();
            cc.putString(cell.getItem());
            db.setContent(cc);
            e.consume();
        });
        // User drags over cell
        cell.setOnDragOver( DragEvent e ->{
            // Only allow the drag if it has a string and is a folder and not over itself
            if(e.getDragboard().hasString() && e.getGestureSource() != cell){
                e.acceptTransferModes(TransferMode.MOVE);
            }
            e.consume();
        });
```

FlashcardManagementScreen Code 20 - addDragAndDrop procedure (Drag Detected and Drag Over)

```
// User has entered the drag area of a cell
cell.setOnDragEntered( DragEvent e->{
    // Change the styling of the cell to show it's a valid move
    if(e.getDragboard().hasString() && e.getGestureSource() != cell){
        cell.setStyle("-fx-background-color: #6891de; -fx-background-radius: 6px;");
    }
});
// When the dragged item has left the cells area, remove the styling
cell.setOnDragExited( DragEvent _-> cell.setStyle(""));
```

FlashcardManagementScreen Code 21 - addDragAndDrop (Drag Entered and Drag Exited)

```
// User releases dragged item on cell
cell.setOnDragDropped( DragEvent e->{
    Dragboard db = e.getDragboard();
    // If there's no string, return;
    if(!db.hasString()) return;
    // Get item being dragged and item being dropped into
    TreeItem<String> draggedF = tree.getSelectionModel().getSelectedItem();
    TreeItem<String> droppingF = cell.getTreeItem();
    // Conditions to stop the file movement
    if(draggedF == null || droppingF == null) return;
    // If drop over same cell
    if(draggedF == droppingF) return;
    // If destination is already an ancestor of the dragged item
    if(isAncestor(draggedF, droppingF)) return;
    // if it is the root
    if(draggedF.getParent() == null) return;
    if(draggedF == tree.getRoot()) return;
    // If folder going into a file
    if(isFolder(draggedF) && isFile(droppingF)) return;
    // if item already in folder
    if (droppingF == draggedF.getParent()) return;
    String draggedName = draggedF.getValue().replace( target: "\uD83D\uDCC1 ", replacement: "").replace( target: "\uD83D\uDCC4 ", replacement: "");
    String droppingName = droppingF.getValue().replace( target: "\uD83D\uDCC1 ", replacement: "").replace( target: "\uD83D\uDCC4 ", replacement: "");
    try {
        fc.move(draggedName, isFolder(draggedF), droppingName);
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
```

FlashcardManagementScreen Code 22 – addDragAndDrop (Drag Dropped Part 1)

```
        // Remove from previous folder
        draggedF.getParent().getChildren().remove(draggedF);
        // Add to selected folder
        if(droppingF.getValue().startsWith("\uD83D\uDCC1 ")) {
            droppingF.getChildren().add(draggedF);
            // User can see change
            droppingF.setExpanded(true);
        }else{
            // Get parent of deck being dropped into
            TreeItem<String> parent = droppingF.getParent();
            // Find position of target deck
            int idx = parent.getChildren().indexOf(droppingF);
            // Insert one after target deck
            parent.getChildren().add( index: idx+1, draggedF);
        }
        // Drop was successfully handled
        e.setDropCompleted(true);
        e.consume();
    });
    return cell;
});
```

FlashcardManagementScreen Code 23 - addDragaAndDrop (Drag Dropped Part 2)

```
// Checks if tree item's ancestor is somewhere is above the item in the tree
private boolean isAncestor(TreeItem<String> potentialAncestor, TreeItem<String> item){
    // Start at item's parent
    TreeItem<String> current = item.getParent();
    while (current != null){
        // if match found, block the drop
        if(current == potentialAncestor) return true;
        // move a level up if not
        current = current.getParent();
    }
    return false;
}
// Check if folder
private boolean isFolder(TreeItem<String> potentialFolder){  2 usages
    return potentialFolder.getValue().startsWith("\uD83D\uDCC1");
}
// Check if file
private boolean isFile(TreeItem<String> potentialFile){  3 usages
    return potentialFile.getValue().startsWith("\uD83D\uDCC4");
}
```

FlashcardManagementScreen Code 24 - addDragAndDrop helper methods

The addDragAndDrop procedure allows drag and drop reordering and reorganisation of cells within the tree view. The method replaces the default cell factory with a custom one where each cell is given 5 drag event handlers. When a drag begins, the cell's text is stored in a clipboard, so the system knows what is being moved. As the item is moved over other cells, they are highlighted in blue to indicate where the user is dropping the item. When the item is released, there is a series of validity checks to prevent illegal moves such as dropping a folder into a file. If the move is valid, the file is moved with the move method in FlashcardController, the tree is then updated as well as the local disk structure.

```
// Moving file/folder to a new destination
public boolean move(String fName, boolean isFolder, String destinationFolder) throws IOException {
    // Find path of file or folder being moved
    String fPath = getFilePath(fName, isFolder);
    if(fPath == null) return false;
    // Find destination's path
    String dPath;
    if(destinationFolder.equals(email)){
        dPath = userFolderPath;
    }else{
        dPath = getFilePath(destinationFolder, isFolder: true);
    }
    if(dPath == null) return false;
    // Create File objects
    File initial = new  File(fPath);
    File end = new  File( pathname: dPath + "/" + initial.getName());
    // switch file locations
    Files.move(initial.toPath(), end.toPath(), StandardCopyOption.REPLACE_EXISTING);
    return true;
}
```

FlashcardController Code 14 - move function

3.3.4.4.2 Creating and Destroying Directories

Directly above the tree, there are two buttons which allows the user to add folders or delete folders. When the delete folder button is clicked, the deleteFolder procedure is called which:

```
// Subroutine to delete a folder
private void deleteFolder(TreeView<String> tree) throws IOException {  1 usage
    TreeItem<String> selected = tree.getSelectionModel().getSelectedItem();
    if(selected == null){
        CustomAlert.error(scene.getWindow(),  message: "Pick something to delete");
        return;
    } else if (selected == tree.getRoot()) {
        CustomAlert.error(scene.getWindow(),  message: "Cannot delete this folder");
        return;
    }else if (isFile(selected)) {
        CustomAlert.error(scene.getWindow(),  message: "To delete files, go to the modification page");
        return;
    }
    boolean confirmed = CustomAlert.confirm(scene.getWindow(),  title: "Confirmation of deletion",
            message: "Are you sure you would like to delete this folder? \n It will delete all contents");
    if(confirmed){
        boolean deleted = fc.deleteFolder(selected.getValue().replace( target: "\uD83D\uDCC1 ",  replacement: ""));
        CustomAlert.success(scene.getWindow(),  message: "Folder successfully deleted");
        selected.getParent().getChildren().remove(selected);
    }
}
```

FlashcardManagementScreen Code 25 - deleteFolder procedure

If the user has selected a folder and confirms deletion, the deleteFolder function via the FlashcardController is called.

```
// Function deleting a folder
public boolean deleteFolder(String folderName) throws IOException {  1 usage
    // get the path
    String path = getFilePath(folderName,  isFolder: true);
    // if null return not deleted
    if(path == null) return false;
    // Walk through all files and subfolders from given path
    Files.walk(Path.of(path))  Stream<Path>
            // Sort them in reverse order so they are deleted before parent directories
            .sorted(Comparator.reverseOrder())
            // Convert each path to an object
            .map(Path::toFile)  Stream<File>
            // Delete each file
            .forEach(File::delete);
    return true;
}
```

FlashcardController Code 15 - deleteFolder function

Although the tree is specifically used for organising folders (deleting and creating), leaving the modification page for deletion of decks. If a user chooses to delete a folder, all the contents are deleted too, but this cannot happen to the root of the tree. The contents being deleted with a folder mirrors standard operating system directory logic, hence implemented for consistency. This deletion also prevented orphaned files and maintains structural integrity.

```
// Subroutine to add a new folder within the tree
private void addFolder(TreeView<String> tree){  1 usage
    TreeItem<String> selected = tree.getSelectionModel().getSelectedItem();
    // If nothing is selected, use the root, and if a deck is selected use its parent
    TreeItem<String> folder;
    if(selected == null){
        folder = tree.getRoot();
    } else if (isFile(selected)) {
        folder = selected.getParent();
    }else {
        folder = selected;
    }
    // Ask for folder name input
    Optional<String> folderName = CustomAlert.input(scene.getWindow(),  title: "New Folder",
            message: "What do you want to name the folder?",  promptText: "Enter Folder Name");
    String parentPath;
    if(folder == tree.getRoot()){
        // If target is root, use the user's folder path
        parentPath = fc.getUserFolderPath();
    }else{
        // Search for file path on disk
        String dirName = folder.getValue().replace( target: "\uD83D\uDCC1 ",  replacement: "");
        parentPath = fc.getFilePath(dirName,  isFolder: true);
    }
    int outcome;
    if(folderName.isPresent()){
        outcome = fc.createFolder(parentPath, folderName.get());
        switch(outcome){
            case 2 -> folder.getChildren().add(folder(folderName.get()));
            case 1 -> CustomAlert.error(scene.getWindow(),  message: "Folder already exists");
            default -> CustomAlert.error(scene.getWindow(),  message: "Did not work try again later");
        }
    }
}
```

FlashcardManagementScreen Code 26 - addFolder procedure

On the click of the addFolder button, the user is asked to input a folder name and once inputted, the createFolder function is called via the FlashcardController object.

```
// Function to create a new directory
public int createFolder(String parentPath, String folderName){ 1 usage
    // Create file object with specified path
    File newFolder = new File( pathname: parentPath + "/" + folderName);
    if(newFolder.exists()){
        return 1; // if exists return 1
    }else{
        if(newFolder.mkdir()){
            return 2; // if added return 2
        }else{
            return 3; // anything else add 3 return 3
        }
    }
}
```

FlashcardController Code 16 - createFolder function

3.3.4.4.3 cardPreview Table

The cardPreview table is updated with the decks associated with a file when a cell in the tree is selected via the following listener:

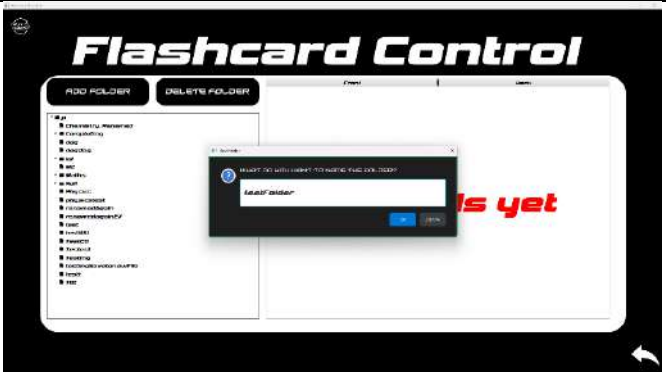
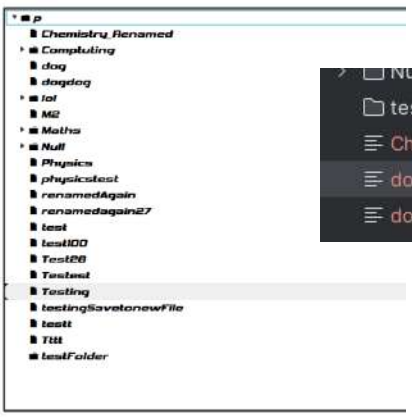
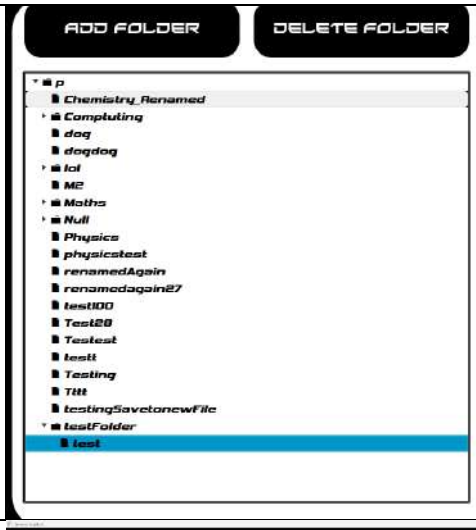
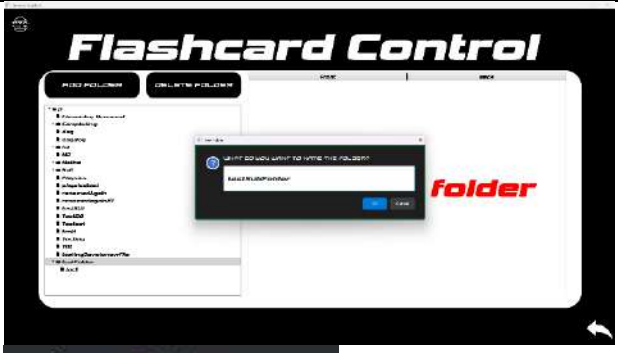
```
// Listener to change the card preview based on selection
tree.getSelectionModel().selectedItemProperty().addListener(( ObservableValue<extends TreeItem<...>> _, TreeItem<String> _, TreeItem<String> newValue) -> {
    if(newValue == null) return;
    try{
        String item = newValue.getValue();
        if(item.startsWith("\uD83D\uDCC1 ")) {
            cardsPreview.getItems().clear();
            Label phText = new Label( text: "This is a folder");
            phText.setStyle("-fx-text-fill: red;");
            cardsPreview.setPlaceholder(phText);
        };
        if(item.startsWith("\uD83D\uDCC4 ")){
            String fileNode = item.replace( target: "\uD83D\uDCC4 ", replacement: "");
            cardsPreview.getItems().setAll(fc.loadFlashcardDeck(fileNode));
        }

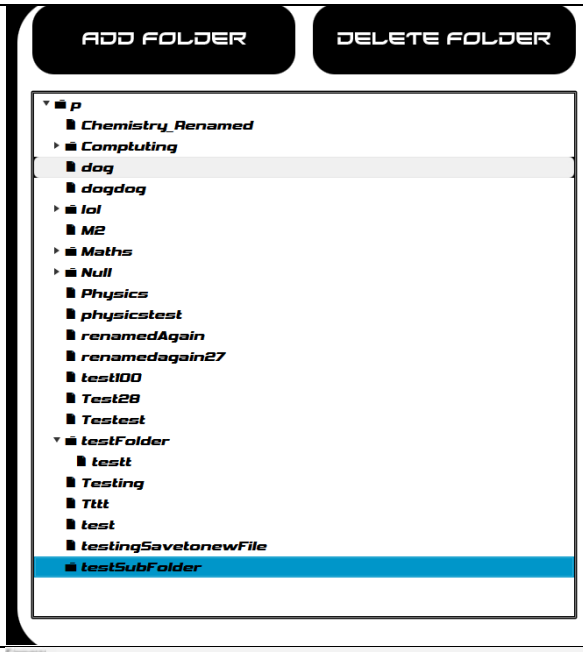
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
});
```

FlashcardManagementScreen Code 27 - cardPreview based on tree selection

3.3.4.4.4 Testing Organisation

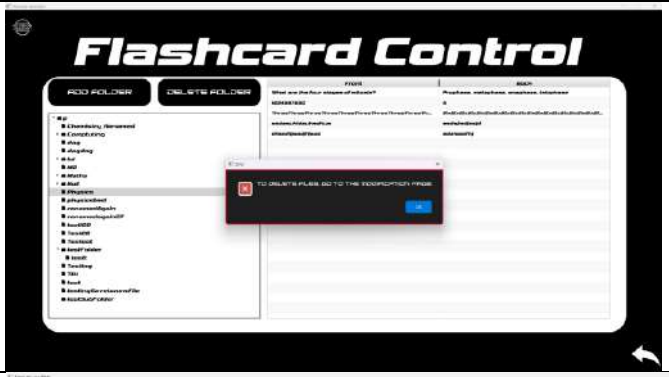
Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	addFolder	Create a folder	Normal	Folder created and shown on tree	Folder created and shown on tree	
2.	Moving deck	Move deck into created folder	Normal	Deck is moved into folder and updated on tree	Deck is moved into folder and updated on tree	
3.	addFolder	Create a subfolder inside the created folder	Normal	Folder created and shown on tree	Folder created and shown on tree	
4.	Moving folder	Move folder out of prior parent folder to root folder	Normal	Folder moved to root and shown on tree	Folder moved to root and shown on tree	
5.	Deletion	Select root and click delete	Erroneous	Not deleted, error shown	Not deleted, error shown	
6.	Deletion	Select a deck and click delete	Erroneous	Not deleted, error shown	Not deleted, error shown	
7.	Deletion	Delete the firstly created folder	Normal	Deleted, shown on tree	Deleted, shown on tree	

Test No.	Proof
	  
	 
	  

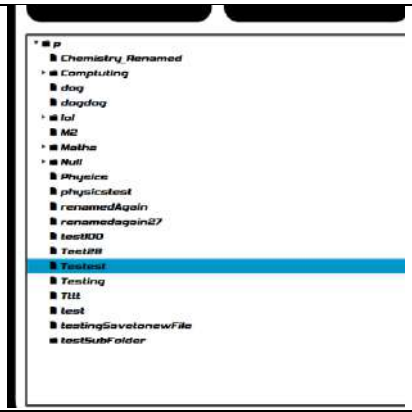

ADD FOLDER
DELETE FOLDER
p
Chemistry_Renamed
Computing
dog
dogdog
lol
M2
Maths
Null
Physics
physicstest
renamedAgain
renamedagain27
test100
Test28
Testest
testFolder
testt
Testing
Tttt
test
testingSavetonewFile
testSubFolder


testFolder
testSubFolder


Flashcard Control
ADD FOLDER
DELETE FOLDER
a folder


Flashcard Control
ADD FOLDER
DELETE FOLDER


Flashcard Control
ADD FOLDER
DELETE FOLDER
a folder


p
Chemistry_Renamed
Computing
dog
dogdog
lol
M2
Maths
Null
Physics
physicstest
renamedAgain
renamedagain27
test100
Test28
Testest
Testing
Tttt
test
testingSavetonewFile
testSubFolder

3.3.4.5 Import / Export

Import and export functionality has been implemented to allow flashcard decks to be transferred between devices to increase portability of created flashcard decks.

3.3.4.5.1 Import as CSV

```
// Procedure which allows the user to import a csv file into the system
public boolean importCSV(Window owner) throws IOException {  1 usage
    // Open the file chooser, filtered to csv files only
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Import CSV File");
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "CSV Files",  ...extensions: "*.csv"));
    // Store user's chosen file
    File file = fileChooser.showOpenDialog(owner);
    // If user doesn't choose a file, don't do anything
    if(file == null) return false;
    // Check size limits of flashcard deck
    ArrayList<Flashcard> cardsList = loadFCDeckFromPath(file.getAbsolutePath());
    if(cardsList.isEmpty() || cardsList.size() > 60){
        CustomAlert.error(owner,  message: "A deck needs a minimum of 1 card and can have a maximum of 60 cards");
        return false;
    }
    // Check if user is importing flashcard deck from another user, or one which already exists within their folder
    String allUserFolders = Paths.get(System.getProperty("user.dir"),  ...more: "userFolders").toString();
    if(file.getCanonicalPath().startsWith(new File(allUserFolders).getCanonicalPath())){
        CustomAlert.error(owner,  message: "Not allowed to import from this directory");
        return false;
    }
    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
        String importDeck = file.getName().replace( target: ".csv",  replacement: "");
        // Set the file name
        setFlashcardDeckName(importDeck);
        // Use previously initiated array list
        readData(br, cardsList);
        // Use default save method
        saveFlashcardDeck(cardsList,  adding: true);
    }
    return true;
}
```

FlashcardController Code 17 - importCSV function

This function is called when the import csv button is clicked, it asks the user to choose the file from their disk and then loads the flashcards into an ArrayList via loadFCDeckFromPath, which is the same code as loadFlashcardDeck, however this one is for files outside of the user-specific directory. It then checks if the deck is a valid size, if so, it then checks if it is being imported from another user's file or an existing file within the user's folder. If it is, then an error alert is displayed since this would just go against the reason the system has accounts. User should not be allowed to import from other users' files without their consent.

3.3.4.5.2 Export as CSV

```
// Procedure to export a user-chosen flashcard deck to CSV or PDF, at a location of their choosing
public boolean export(Window owner, boolean isPDF) throws IOException {  2 usages
    CustomAlert.info(owner,  message: "Choose the deck to export, can only be from this directory (and sub-directories of this one)");
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Export CSV File");
    fileChooser.setInitialDirectory(new File(userFolderPath));
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "CSV Files",  ...extensions: "*.csv"));
    // Ask user where to save the file
    File chosenDeck = fileChooser.showOpenDialog(owner);
    System.out.println("Chosen deck: " + chosenDeck);
    CustomAlert.info(owner,  message: "Choose the location to save the deck");
    File userFolder = new File(userFolderPath);
    // If cancelled return false
    if(chosenDeck == null){
        return false;
    }
    // If outside user directory, return false
    // Canonical path -> fully resolved path
    if(!chosenDeck.getCanonicalPath().startsWith(userFolder.getCanonicalPath())) return false;
    // Get file name
    String exportDeck = chosenDeck.getName().replace( target: ".csv",  replacement: "");
    // Ask user to choose where to save the file
    FileChooser locationChooser = new FileChooser();
    locationChooser.setTitle("Choose Save Location");
    // Set initial save file name to current file name
    if(!isPDF){
        locationChooser.setInitialFileName(chosenDeck.getName());
        // Filter for only .csv
        locationChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "CSV Files",  ...extensions: "*.csv"));
    }
```

FlashcardController Code 18 - exportCSV

```
if(chosenLocation == null) return false;
if(!isPDF) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(chosenLocation))) {
        for (Flashcard card : loadFlashcardDeck(exportDeck)) {
            bw.write( str: "\"" + card.getQuestion().replace( target: "\"",  replacement: "\"\"").replace( target: "\n",  replacement: "\\n")
                    + "\",\"" + card.getAnswer().replace( target: "\"",  replacement: "\"\"").replace( target: "\n",  replacement: "\\n") + "\"");
            bw.newLine();
        }
    }
```

FlashcardController Code 19 - writingCSV

The user is asked to select the deck from their own directory, if it's not from there; it would not work. If they have picked, they would then select a save location, the buffered writer then writes the csv in the correct format, and it is saved in the correct location.

3.3.4.5.3 Export as PDF

The same export function is used to export as PDF to reduce the repetition of code; hence the user has picked the deck and the save location.

```
if(isPDF){
    locationChooser.setInitialFileName(chosenDeck.getName().replace( target: "csv",  replacement: "pdf"));
    // Filter for only .PDF
    locationChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "PDF Files",  ...extensions: "*.pdf"));
}
```

FlashcardController Code 20 - isPDF in export function

This filters it so in the save dialog the user can only see other pdfs initially.

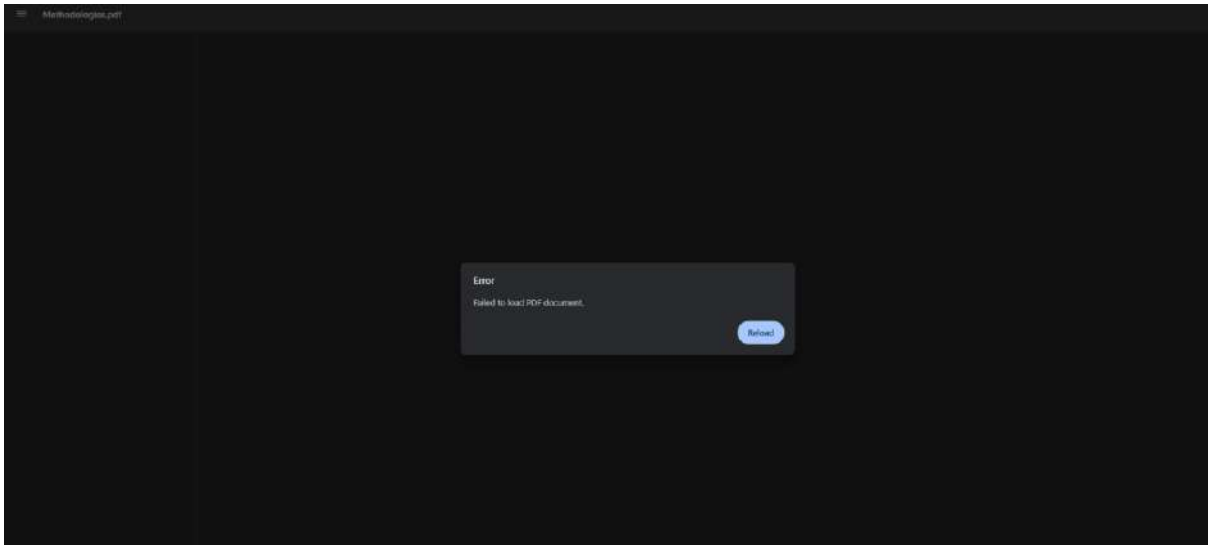
The deck is then written into a pdf via a PrintWriter:

```
try (PrintWriter pw = new PrintWriter(new FileWriter(chosenLocation)))
{
    pw.println("%PDF-1.4"); // PDF Header identifier
    pw.println(exportDeck);
    pw.println("");
    for(Flashcard card : loadFlashcardDeck(exportDeck)){
        pw.println("Question: " + card.getQuestion());
        pw.println("Answer: " + card.getAnswer());
        pw.println(""); // Blank line for extra readability
    }
}
```

FlashcardController Code 21 - PrintWriter to write PDF

3.3.4.5.3.1 Remedial Action

With initial tests on exporting as a PDF when I opened the PDF file, I received this error:



Export as PDF 1 – Error

This occurred since PDF files are not plain text documents unlike .csv or .txt files, hence writing text directly into a file ending in .pdf does not generate a valid PDF structure, resulting in not being able to open.

Generating PDFs successfully would need an external generation library (like iText), these libraries are necessary to build the internal object structure and formatting automatically. This would increase the project complexity, hence I looked at different options which were more accessible but as useful as PDFs.

Hence, the implementation was revised to export decks as HTML instead. HTML will provide a clear structure, full control over styling, compatibility across all browsers and a built-in print to pdf function. This means that decks can still be exported as PDF, with a better structure, but with an extra step.

```
if(chosenLocation == null) return false;
if(!isHTML) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(chosenLocation))) {
        for (Flashcard card : loadFlashcardDeck(exportDeck)) {
            bw.write( str: "\"" + card.getQuestion().replace( target: "\"", replacement: "\"\"").replace( target: "\n", replacement: "\\n")
                    + "\",\"" + card.getAnswer().replace( target: "\"", replacement: "\"\"").replace( target: "\n", replacement: "\\n") + "\"");
            bw.newLine();
        }
    }
}else{
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(chosenLocation))) {
        writeHTML(bw, exportDeck);
    }
}
return true;
```

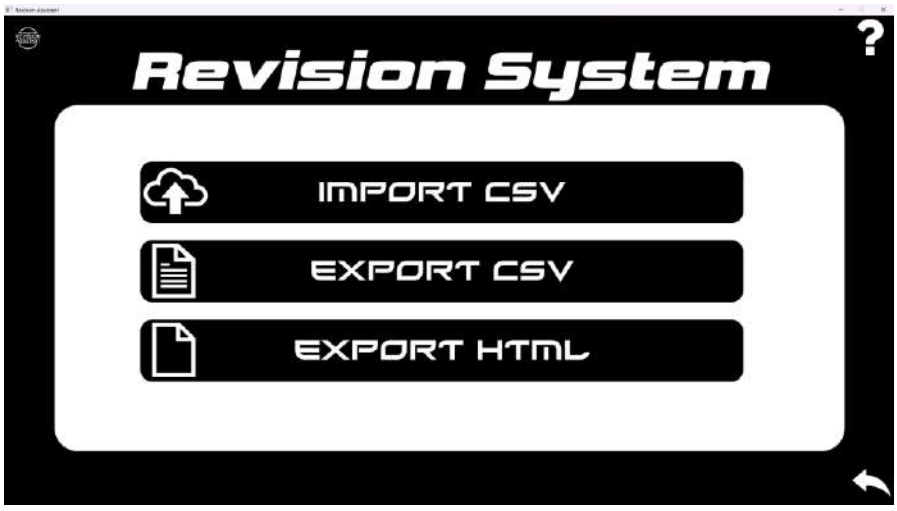
FlashcardController Code 22 - writing to changed inside export function

This function calls the writeHTML procedure which creates a table and populated it with deck data.

```
// Procedure to write HTML File
private void writeHTML(BufferedWriter bw, String exportDeck) throws IOException {  1 usage
    bw.write( str: "<!DOCTYPE html>");
    bw.newLine();
    bw.write( str: "<html><head><title>" + exportDeck + "</title><style>");
    bw.newLine();
    bw.write( str: "body{font-family: Arial, sans-serif; margin: 40px;}");
    bw.newLine();
    bw.write( str: "h1{color: black; text-align: center;}");
    bw.newLine();
    bw.write( str: "table{width: 100%; border-collapse: collapse; margin-top: 20px;}");
    bw.newLine();
    bw.write( str: "th{background-color: black; color: white; padding: 40px; text-align: center; border-right: 1px solid black;}");
    bw.newLine();
    bw.write( str: "td{padding: 50px; border-bottom: 1px solid black; text-align: center; border-right: 1px solid black; word-wrap: break-word; max-width: 250px;}");
    bw.newLine();
    bw.write( str: "tr:nth-child(even){background-color: #f0f0f0;}");
    bw.newLine();
    bw.write( str: "</style></head><body>");
    bw.newLine();
    bw.write( str: "<h1>" + exportDeck + "</h1>");
    bw.newLine();
    bw.write( str: "<table><tr><th>Question</th><th>Answer</th></tr>");
    for(Flashcard card : loadFlashcardDeck(exportDeck)){
        bw.write( str: "<tr><td>" + card.getQuestion() + "</td><td>" + card.getAnswer() + "</td></tr>");
    }
    bw.write( str: "</table></body></html>");
}
```

FlashcardController Code 23 - writing HTML table

3.3.4.5.4 Import / Export Screen



RevisionSystemScreen Image 3 - Import/Export

On the top right of the screen, there is a question mark which when clicked on will give the user an idea on what this page is about.

```
ImageView hintView = ui.getHintView();
String importInfo = "You can import a CSV which follows the " +
        "question, answer format with speech marks \n surrounding both and new lines indicated by backslash and n";
String exportCSVInfo = "You will get an exported csv following the same lines as import versions";
String exportHTMLInfo = "You can export your decks to a HTML page, and then print to PDF for portability";
hintView.setOnMouseClicked( MouseEvent _-> CustomAlert.info(owner,  message: importInfo + "\n" + "\n" + exportCSVInfo + "\n" + "\n" + exportHTMLInfo));
```

RevisionSystemScreen Code 2 - in buildImportExport procedure

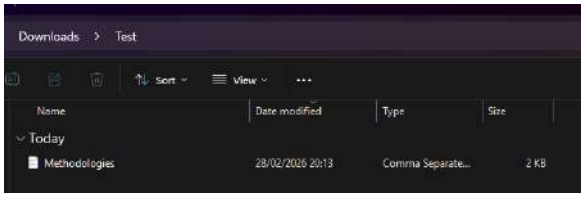
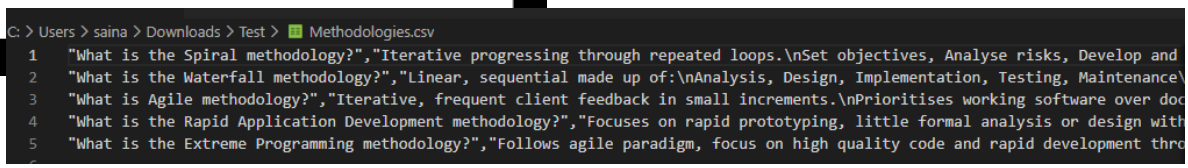
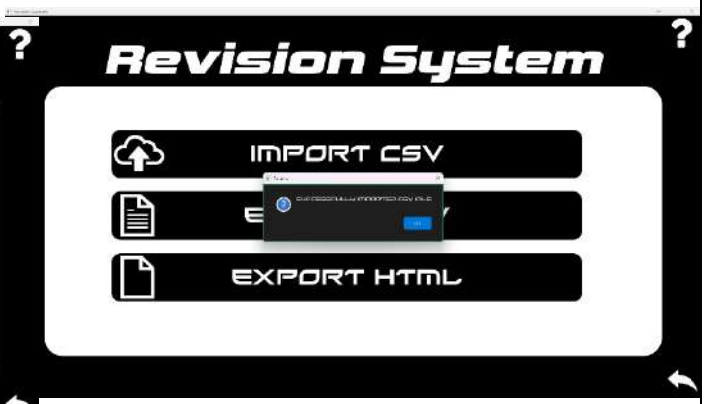
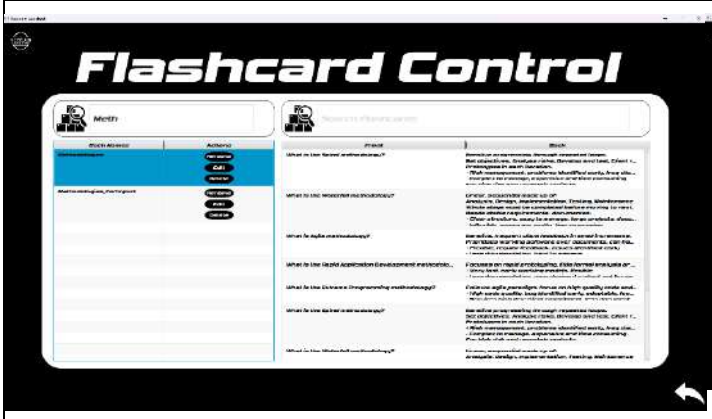
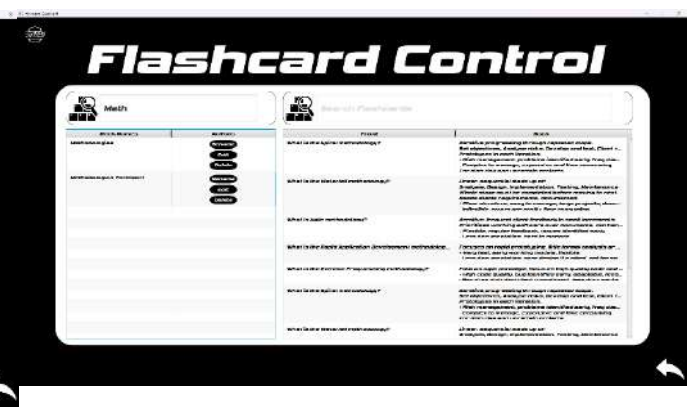


RevisionSystemScreen Image 4 – Hint

This increases the overall usability of the system. Positioning the help icon in the top right corner follows common UI conventions and users usually expect help to be located there. The on-demand help will clarify the functionality for users without leaving the page, reducing the likelihood of errors as instructions are available before actions.

3.3.4.5.5 Testing Import / Export

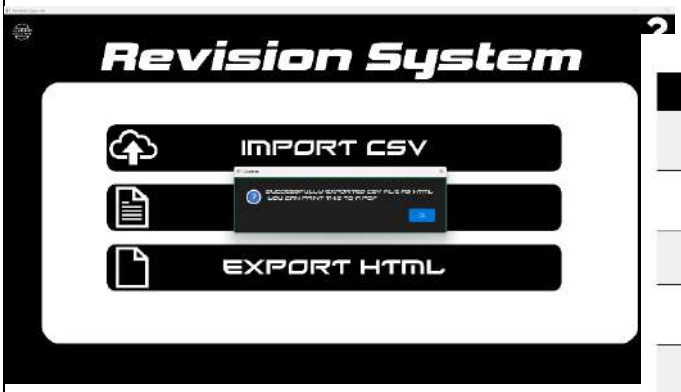
Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	Export	Export an existing deck out of the system in csv	Normal	Correct deck exported in correct location	Correct deck exported in correct location	
2.	Import	Import the exported csv into the system (rename first)	Normal	Deck is imported, and the same as the exported one when you compare in Modification	Deck is imported, and the same as the exported one when you compare in Modification	
3.	Export	Export the imported csv as HTML	Normal	HTML page displaying a table with 2 columns, question and answer	HTML page displaying a table with 2 columns, question and answer	

Test No.	Proof
1.	    
2.	   

3.









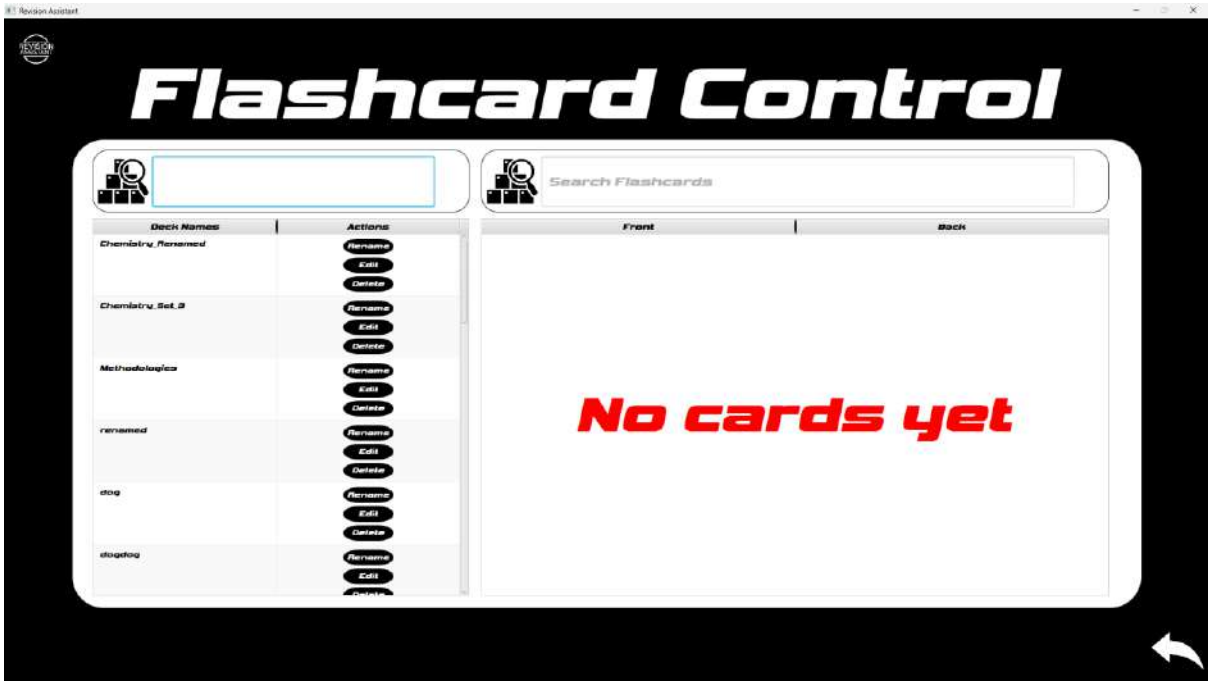
3.3.5 Evaluation of Prototype 3

Prototype 3 has extended the functionality of the RevisionAssistant, now making it a functional revision tool since it allows the creation of flashcards, whilst being able to modify, organise, import and export them. Prototype 3 aimed to satisfy success criterions 9, 12 and 15 by implementing a fully functional flashcard management system. I can confirm the user can import decks as CSV files, and can export decks as CSV files, however the solution does not allow direct exportation to PDF, instead you can export to HTML, receiving a structured table for a deck and print to pdf from there. Hence, success criterion 9 has been met. The user can create flashcard decks, save them to their own directory with custom names as well as modify them post-creation with ease. The user can also organise these decks into folders, and subfolders, of their choice with features such as drag and drop on a tree view. Thus, it is fair to say that success criterions 12 and 15 have both successfully been met. Therefore, prototype 3 has met the criteria, allowing me to move on to the next module of my application where I will tackle the actual revision process. The next prototype will introduce study mechanisms and performance tracking, transforming the system from a storage tool (as it is currently) to a fully functional revision tool.

3.4 Prototype 4: Study Process and Analytics

3.4.1 Improvements from Prototype 3

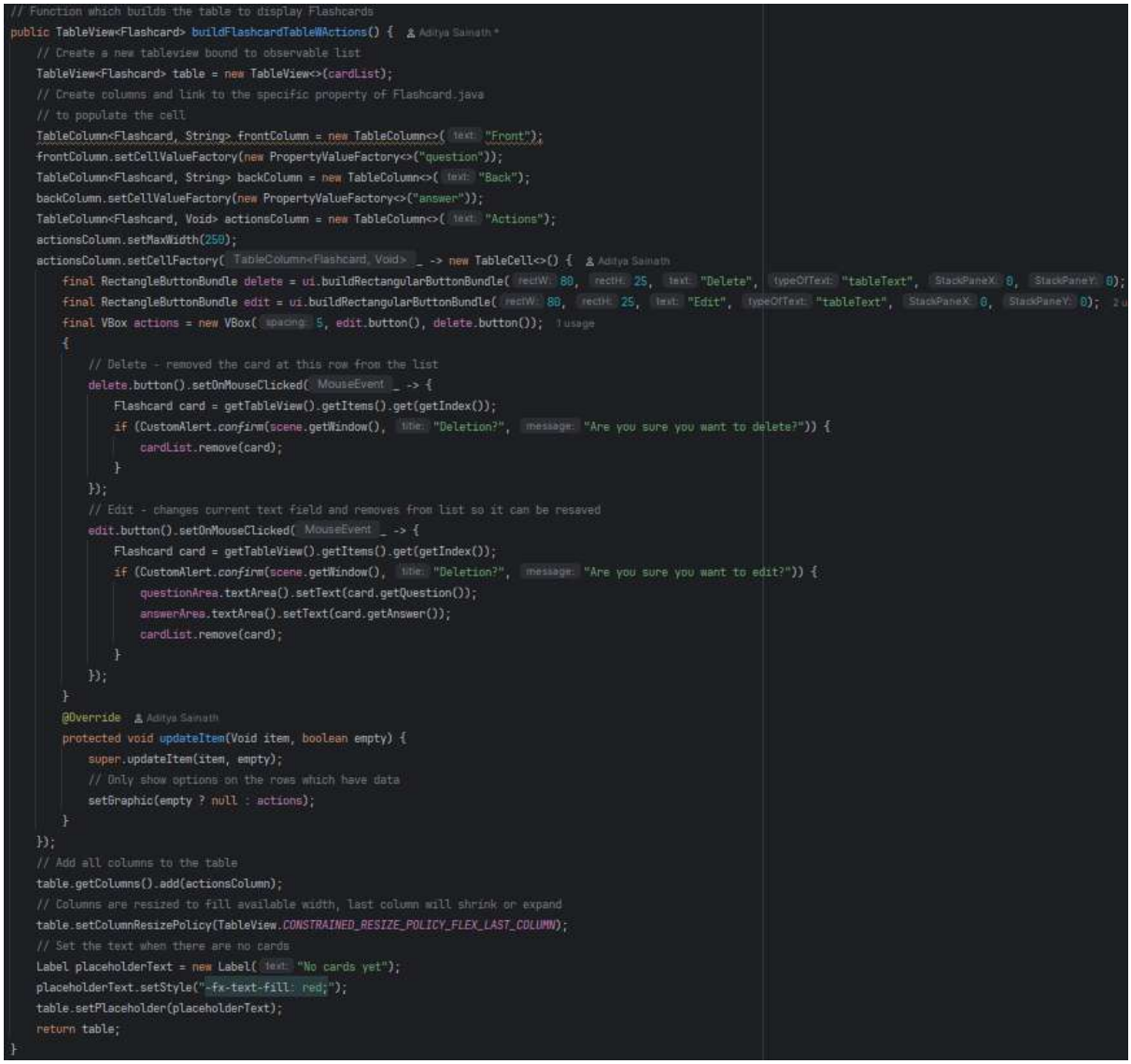
The code to design the modification page displaying 2 table views (one for the decks, and one for cards preview) with two search fields was extracted into a method. This is because I will be using this design multiple times across the system, for example, when users begin a study session, they need to select a deck to begin it with and they will do so with the table view.



FlashcardManagementScreen Image 12 - Modification Page

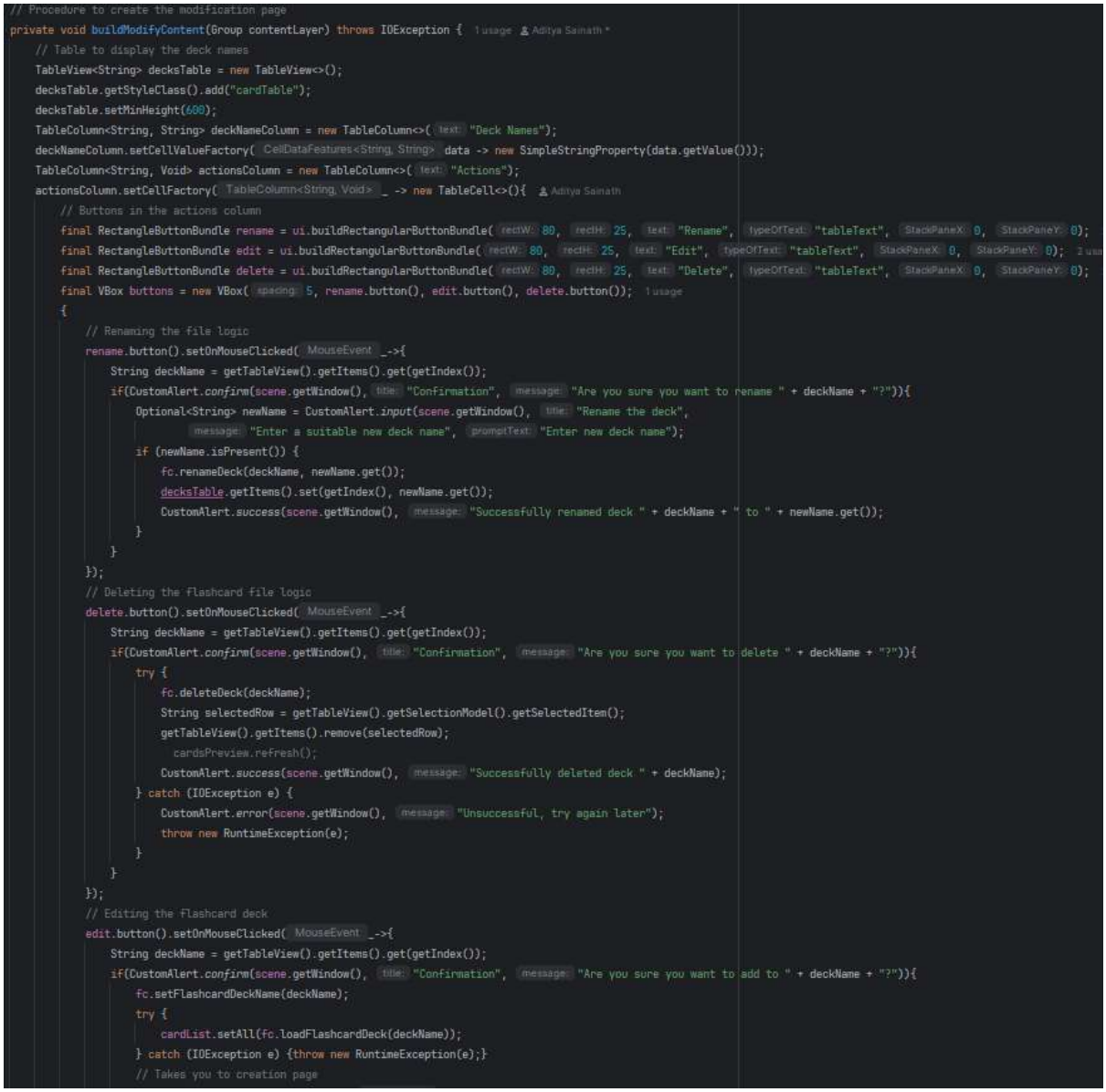
This HBox will be re-used to allow the user to select a deck to start a session.

In FlashcardManagementScreen.java, with this change, the buildCreationContent, buildModifyContent and buildOrganisationContent have been updated. Firstly, the buildTable() function to build the card preview table (used by all three subroutines) was changed into buildFlashcardTableWActions() which is now only used by the creation page since this the only page which needs the preview table to have an actions column (to edit and delete flashcards when adding to or making a deck). The modification and organisation page now use an extracted method placed in SystemScreens which builds a card preview table without an “Actions” column just for viewing, this is called buildCardPreviewTable() and will be used in the organisational page, modification page, and to allow the user to select a deck in serial or shuffle mode pages.



```
// Function which builds the table to display Flashcards
public TableView<Flashcard> buildFlashcardTableWActions() {  Aditya Sainath *
    // Create a new tableview bound to observable list
    TableView<Flashcard> table = new TableView<>(cardList);
    // Create columns and link to the specific property of Flashcard.java
    // to populate the cell
    TableColumn<Flashcard, String> frontColumn = new TableColumn<>( text: "Front");
    frontColumn.setCellValueFactory(new PropertyValueFactory<>("question"));
    TableColumn<Flashcard, String> backColumn = new TableColumn<>( text: "Back");
    backColumn.setCellValueFactory(new PropertyValueFactory<>("answer"));
    TableColumn<Flashcard, Void> actionsColumn = new TableColumn<>( text: "Actions");
    actionsColumn.setMaxWidth(250);
    actionsColumn.setCellFactory( TableColumn<Flashcard, Void> _ -> new TableCell<>() {  Aditya Sainath
        final RectangleButtonBundle delete = ui.buildRectangularButtonBundle( rectW: 80, rectH: 25, text: "Delete", typeOfText: "tableText", StackPaneX: 0, StackPaneY: 0);
        final RectangleButtonBundle edit = ui.buildRectangularButtonBundle( rectW: 80, rectH: 25, text: "Edit", typeOfText: "tableText", StackPaneX: 0, StackPaneY: 0);
        final VBox actions = new VBox( spacing: 5, edit.button(), delete.button());  1 usage
        {
            // Delete - removed the card at this row from the list
            delete.button().setOnMouseClicked( MouseEvent _ -> {
                Flashcard card = getTableView().getItems().get(getIndex());
                if (CustomAlert.confirm(scene.getWindow(), title: "Deletion?", message: "Are you sure you want to delete?")) {
                    cardList.remove(card);
                }
            });
            // Edit - changes current text field and removes from list so it can be resaved
            edit.button().setOnMouseClicked( MouseEvent _ -> {
                Flashcard card = getTableView().getItems().get(getIndex());
                if (CustomAlert.confirm(scene.getWindow(), title: "Deletion?", message: "Are you sure you want to edit?")) {
                    questionArea.textArea().setText(card.getQuestion());
                    answerArea.textArea().setText(card.getAnswer());
                    cardList.remove(card);
                }
            });
        }
        @Override  Aditya Sainath
        protected void updateItem(Void item, boolean empty) {
            super.updateItem(item, empty);
            // Only show options on the rows which have data
            setGraphic(empty ? null : actions);
        }
    });
    // Add all columns to the table
    table.getColumns().add(actionsColumn);
    // Columns are resized to fill available width, last column will shrink or expand
    table.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
    // Set the text when there are no cards
    Label placeholderText = new Label( text: "No cards yet");
    placeholderText.setStyle("-fx-text-fill: red;");
    table.setPlaceholder(placeholderText);
    return table;
}
```

FlashcardManagementScreen Code 28 - new buildFlashcardTableWActions function



```
// Procedure to create the modification page
private void buildModifyContent(Group contentLayer) throws IOException {  1 usage  Aditya Sainath *
    // Table to display the deck names
    TableView<String> decksTable = new TableView<>();
    decksTable.getStyleClass().add("cardTable");
    decksTable.setMinHeight(600);
    TableColumn<String, String> deckNameColumn = new TableColumn<>( text: "Deck Names");
    deckNameColumn.setCellValueFactory( CellDataFeatures<String, String> data -> new SimpleStringProperty(data.getValue()));
    TableColumn<String, Void> actionsColumn = new TableColumn<>( text: "Actions");
    actionsColumn.setCellFactory( TableColumn<String, Void> _ -> new TableCell<>(){  Aditya Sainath
        // Buttons in the actions column
        final RectangleButtonBundle rename = ui.buildRectangularButtonBundle( rectW: 80,  rectH: 25,  text: "Rename",  typeOfText: "tableText",  StackPaneX: 0,  StackPaneY: 0);
        final RectangleButtonBundle edit = ui.buildRectangularButtonBundle( rectW: 80,  rectH: 25,  text: "Edit",  typeOfText: "tableText",  StackPaneX: 0,  StackPaneY: 0);  1 usa
        final RectangleButtonBundle delete = ui.buildRectangularButtonBundle( rectW: 80,  rectH: 25,  text: "Delete",  typeOfText: "tableText",  StackPaneX: 0,  StackPaneY: 0);
        final VBox buttons = new VBox( spacing: 5, rename.button(), edit.button(), delete.button());  1 usage
        {
            // Renaming the file logic
            rename.button().setOnMouseClicked( MouseEvent _->{
                String deckName = getTableView().getItems().get(getIndex());
                if(CustomAlert.confirm(scene.getWindow(),  title: "Confirmation",  message: "Are you sure you want to rename " + deckName + "?")){
                    Optional<String> newName = CustomAlert.input(scene.getWindow(),  title: "Rename the deck",
                            message: "Enter a suitable new deck name",  promptText: "Enter new deck name");
                    if (newName.isPresent()) {
                        fc.renameDeck(deckName, newName.get());
                        decksTable.getItems().set(getIndex(), newName.get());
                        CustomAlert.success(scene.getWindow(),  message: "Successfully renamed deck " + deckName + " to " + newName.get());
                    }
                }
            });
            // Deleting the Flashcard file logic
            delete.button().setOnMouseClicked( MouseEvent _->{
                String deckName = getTableView().getItems().get(getIndex());
                if(CustomAlert.confirm(scene.getWindow(),  title: "Confirmation",  message: "Are you sure you want to delete " + deckName + "?")){
                    try {
                        fc.deleteDeck(deckName);
                        String selectedRow = getTableView().getSelectionModel().getSelectedItem();
                        getTableView().getItems().remove(selectedRow);
                          cardsPreview.refresh();
                        CustomAlert.success(scene.getWindow(),  message: "Successfully deleted deck " + deckName);
                    } catch (IOException e) {
                        CustomAlert.error(scene.getWindow(),  message: "Unsuccessful, try again later");
                        throw new RuntimeException(e);
                    }
                }
            });
            // Editing the Flashcard deck
            edit.button().setOnMouseClicked( MouseEvent _->{
                String deckName = getTableView().getItems().get(getIndex());
                if(CustomAlert.confirm(scene.getWindow(),  title: "Confirmation",  message: "Are you sure you want to add to " + deckName + "?")){
                    fc.setFlashcardDeckName(deckName);
                    try {
                        cardList.setAll(fc.loadFlashcardDeck(deckName));
                    } catch (IOException e) {throw new RuntimeException(e);}
                    // Takes you to creation page
```

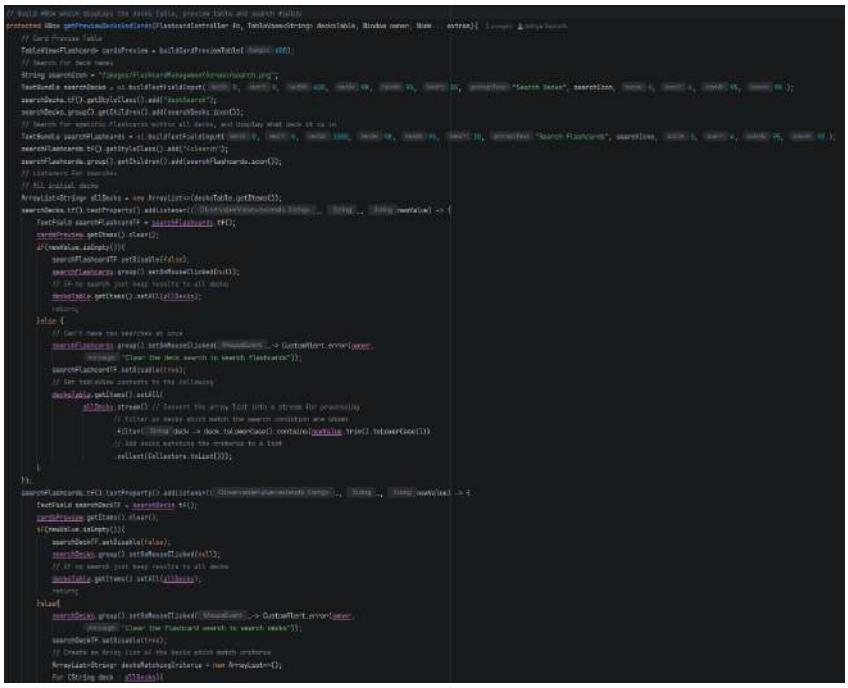
FlashcardManagementScreen Code 29 - updated buildModifyContent() part 1



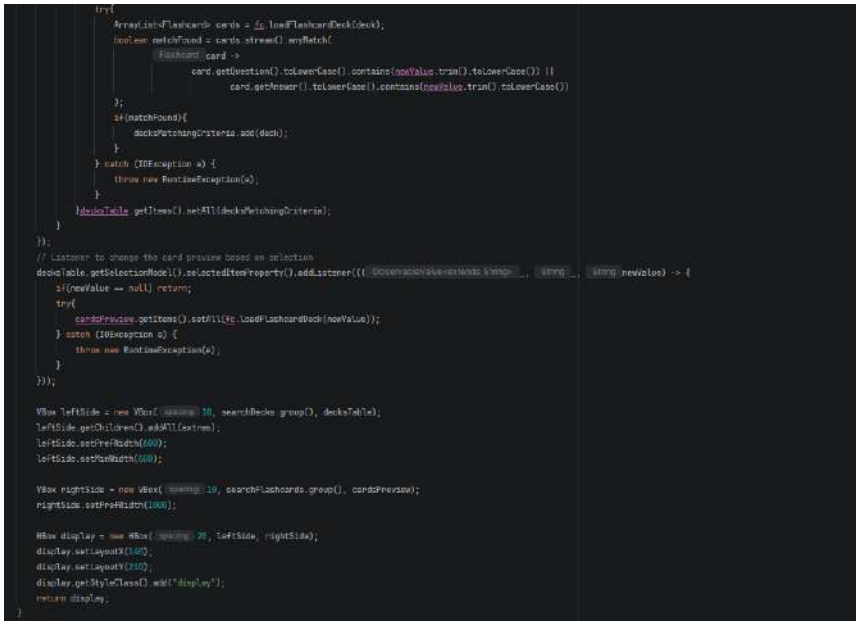
```
                    // Takes you to creation page
                    buildCreationContent(contentLayer,  alreadyExists: true);
                }
            });
        }
        @Override  Aditya Sainath
        protected void updateItem(Void item, boolean empty){
            super.updateItem(item, empty);
            // Only show buttons if there is data
            setGraphic(empty ? null : buttons);
        }
    });
    decksTable.getColumns().addAll(List.of(deckNameColumn, actionsColumn));
    decksTable.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
    decksTable.getItems().addAll(fc.getFileContents());
    allDecks = new ArrayList<>(decksTable.getItems());
    HBox display = getPreviewDecksAndCards(fc, decksTable, scene.getWindow());
    nav.switchContent(contentLayer, display, getBackView(contentLayer));
}
```

FlashcardManagementScreen Code 30 - buildModifyContent() part 2

This procedure builds a deck table with an actions column to rename, delete and edit flashcard decks. Each time a card preview has been used the deck table differs, for example when selecting study modes, a decks table would not have these options since the user is not supposed to edit, rename or delete flashcard decks from this page. This procedure calls a function within SystemScreens to receive the HBox which holds all the content for this specific page.



SystemScreens Code 1 - getPreviewDecksAndCards function part 1



SystemScreens Code 2 - getPreviewDecksAndCards function part 2

This function will add the search fields and the two tables views to an HBox to be displayed to the user, there is also an extra option where you can pass in extra nodes

which will be added to the left VBox, for example a start session button. This function also calls the buildCardPreviewTable function which returns a table view with the card preview table, this is used by the buildOrganisationContent() procedure in the FlashcardManagementScreen as well.

```
// Method to build the card preview table to be used in multiple screens
protected TableView<Flashcard> buildCardPreviewTable(int height){  2 usages  Aditya Sainath
    // Create a new tableview bound to observable list
    TableView<Flashcard> table = new TableView<>();
    table.getStyleClass().add("cardTable");
    table.setMinHeight(height);
    table.setPrefHeight(height);
    // Create columns and link to the specific property of Flashcard.java
    // to populate the cell
    TableColumn<Flashcard, String> frontColumn = new TableColumn<>( text: "Front");
    frontColumn.setCellValueFactory(new PropertyValueFactory<>("question"));
    TableColumn<Flashcard, String> backColumn = new TableColumn<>( text: "Back");
    backColumn.setCellValueFactory(new PropertyValueFactory<>("answer"));
    // Add all columns to table
    table.getColumns().addAll(List.of(frontColumn, backColumn));
    // Columns are resized to fill available width, last column will shrink or expand
    table.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
    // Set the text when there are no cards
    Label placeholderText = new Label( text: "No cards yet");
    placeholderText.setStyle("-fx-text-fill: red;");
    table.setPlaceholder(placeholderText);
    return table;
}
```

SystemScreens Code 3 - buildCardPreviewTable() function

These methods are placed into the SystemScreens super class to prevent code from being repeated across multiple classes. Furthermore, the access modifiers on the classes are marked as protected, this means an instance of the SystemScreen class cannot call these methods, only subclasses, although it is not possible to create an object of an abstract class.

3.4.2 Purpose of Prototype 4

The purpose of Prototype 4 is to transform the system from a storage tool for flashcards into an interactive platform for revision. Prototype 3 successfully implemented flashcard (deck) creation, modification, import/export and organisation but it did not provide another core functionality of the Revision Assistant: the actual study process with performance analytics. This prototype aims to introduce study modes such as Serial Mode, Wrong Answers Only and Shuffle Mode, encouraging active recall instead of passive review. User analytics also should be implemented within this prototype allowing users to track their performance and see where to improve in their revision.

Prototype 4 builds on the foundation established in previous prototypes, now that decks can be reliably stored; created; organised and edited, the only next logical step would be to implement functionality which allows meaningful revision for users.

3.4.3 Success Criteria for Prototype 4

Referencing (1.4.3 Success Criteria), with this prototype I will be aiming to achieve success criterions 7, 8, 10, 13 and 14 which aim to allow the user to use the content they have imported / created to study with, whilst being able to view statistics about their performances when studying.

7) The solution should accurately measure XP and Achievements.
8) The program must be available for offline use.
10) The solution must have session time-tracking in hours and minutes.
13) The solution must contain a valid text-to-speech feature.
14) The solution must provide advanced analytics.

3.4.3.1 XP and Achievement System Criteria

- Must accurately calculate and award XP based on user activity (such as correct answers, session time)
- XP must persist throughout different sessions and be on view for the user
- Achievements are unlocked when pre-defined conditions are met (such as completed first study session).
- Achievements must not trigger twice.

3.4.3.2 Session Time-Tracking Criteria

- Time tracking must be displayed to the user during the study session.
- Session time should be stored persistently.
- Time tracking must be accurate.

3.4.3.3 Text-to-Speech Criteria

- System must functional TTS available to the user when reviewing flashcards.
- TTS must be accurate and correctly reflect the selected flashcard's text.
- Users should be able to trigger TTS with the pressing of a button.
- System must handle irregular inputs correctly with TTS.
- Must be available to be used offline.

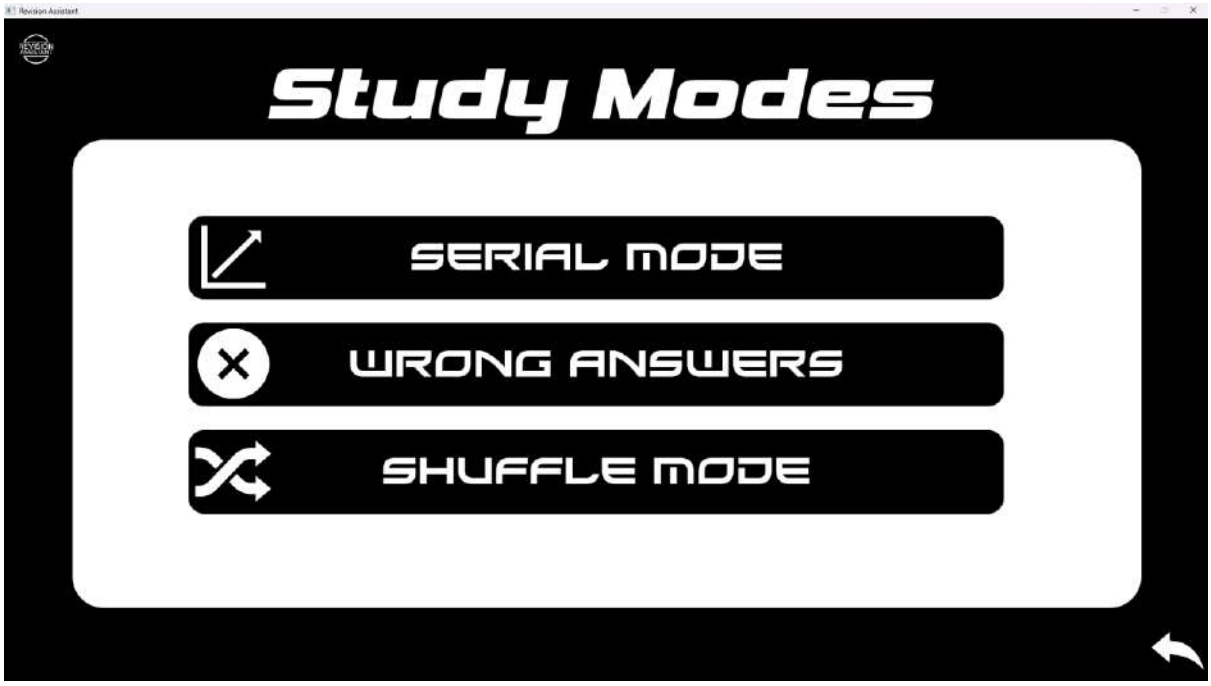
3.4.3.4 Analytics Criteria

- System must record performance stats allowing them to persist across unlimited sessions.
- Analytics must be meaningful metrics (average accuracy instead total flashcards viewed, for example).
- Users should be able to view analytics easily and data must be accurate.

3.4.4 Developing Prototype 4

3.4.4.1 Study Modes

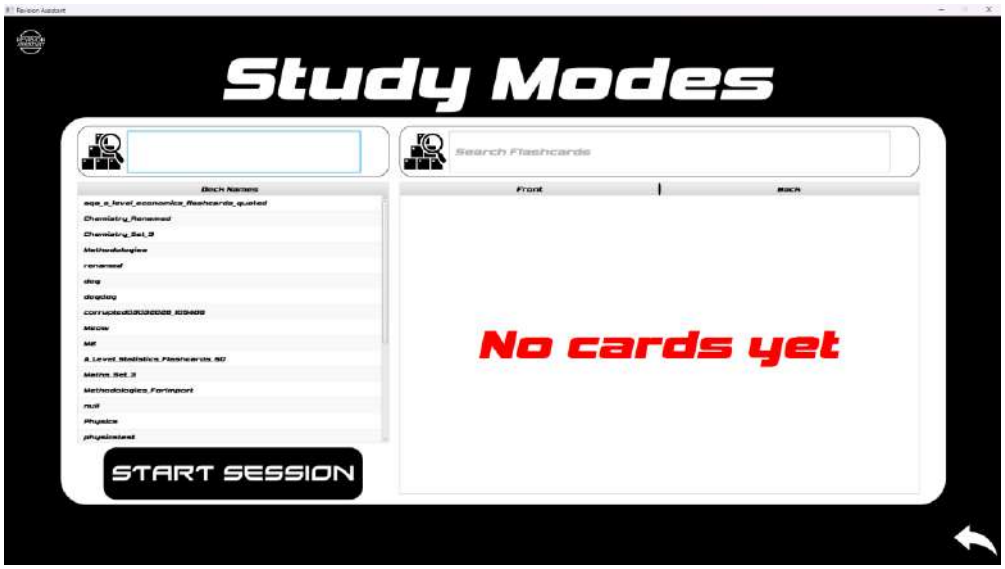
Initially I created the user interface where the user can select the study mode via buttons:



StudyModesScreen Image 1 - Initial UI

To implement Serial and Shuffle mode, I realised the user would need to select a deck, hence the content from the buildModifyContent in FlashcardManagementScreen was extracted into a method placed in the SystemScreens super class.

The deck selector was built via the buildDeckSelector method which looks like:



StudyModesScreen Image 2 - deck selector Page

```
// Content where user can select a deck
private void buildDeckSelector(Group contentLayer, String mode){ 2 usages
    Window owner = contentLayer.getScene().getWindow();
    // Create table with decks
    TableView<String> decksTable = new TableView<>();
    decksTable.getStyleClass().add('cardTable');
    decksTable.setMinHeight(500);
    TableColumn<String, String> deckNameColumn = new TableColumn<>( text: "Deck Names");
    deckNameColumn.setCellValueFactory( CellDataFeatures<String,String> data -> new SimpleStringProperty(data.getValue()));
    decksTable.getColumns().add(deckNameColumn);
    decksTable.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
    decksTable.getItems().addAll(fc.getFileContents());
    Label deckTablesPh = new Label( text: "No decks found");
    deckTablesPh.setStyle("-fx-text-fill: red; -fx-wrap-text: true; -fx-text-alignment: center;');
    decksTable.setPlaceholder(deckTablesPh);
    // Start session button which creates the session
    RectangleButtonBundle startSession = vi.buildRectangularButtonBundle( rectW: 500, rectH: 100, text: "Start Session",
            typeOfText "smallButtonText", StackPaneX: 760, StackPaneY: 0);
    startSession.button().setOnMouseClicked( MouseEvent _->{
        String deckName = decksTable.getSelectionModel().getSelectedItem();
        if(deckName == null) CustomAlert.error(owner, message: "Select a deck before starting a session");
        try{
            ArrayList<Flashcard> cards = fc.loadFlashcardDeck(deckName);
            // If shuffle mode, shuffle the cards
            if(mode.equals('Shuffle')) sessionC.shuffle(cards);
            sessionC.startSession(cards, deckName, mode);
            sessionC.setLastCards(new ArrayList<>(cards));
            buildSessionContent(contentLayer);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });
    // Get preview and search icon
    HBox display = getPreviewDecksAndCards(fc, decksTable, owner, startSession.button());
    nav.switchContent(contentLayer, display, getBackView(contentLayer));
}
```

StudyModesScreen Code 1 - buildDeckSelector() procedure

This creates a custom decks table without an action column and adding a start button to the bottom of the page. If the user has selected and deck and clicks start, the flashcards will be loaded into an ArrayList, if the mode is shuffle then the cards will be shuffled via the Fisher-Yates method. I used the Fisher-Yates method since it gives a truly uniform shuffle, the time complexity is O(n), hence it will be efficient for a flashcard deck of maximum 60 items. Furthermore, this also sorts in place, so it has a low memory usage. An alternative would have been using the Java built-in Collections.shuffle() method, however implementing my own method would give me the flexibility to extend functionality in future revisions of the system.

```
// Shuffle ArrayList of cards for Shuffle mode
public void shuffle(ArrayList<Flashcard> cards) { 2 usages
    // Create a random object to generate random indices
    Random rand = new Random();
    // Starting at the last element and move backwards
    for(int i = cards.size()-1; i > 0; i--) {
        // Generate a random index between 0 and i inclusive
        int j = rand.nextInt( bound: i+1);
        // Store element at index i temporarily
        Flashcard temp = cards.get(i);
        // Replace element at index i with element at index j
        cards.set(i, cards.get(j));
        // Place stored element into index j
        cards.set(j, temp);

    }
}
```

SessionController Code 1 - procedure to shuffle

After this, the SessionController instance starts a new session by:

```
// Start session
public void startSession(ArrayList<Flashcard> cards, String deck, String mode){
    this.lastCards = new ArrayList<>(cards);
    // Create a new session object
    this.session = new Session(cards, deck, mode);
}
```

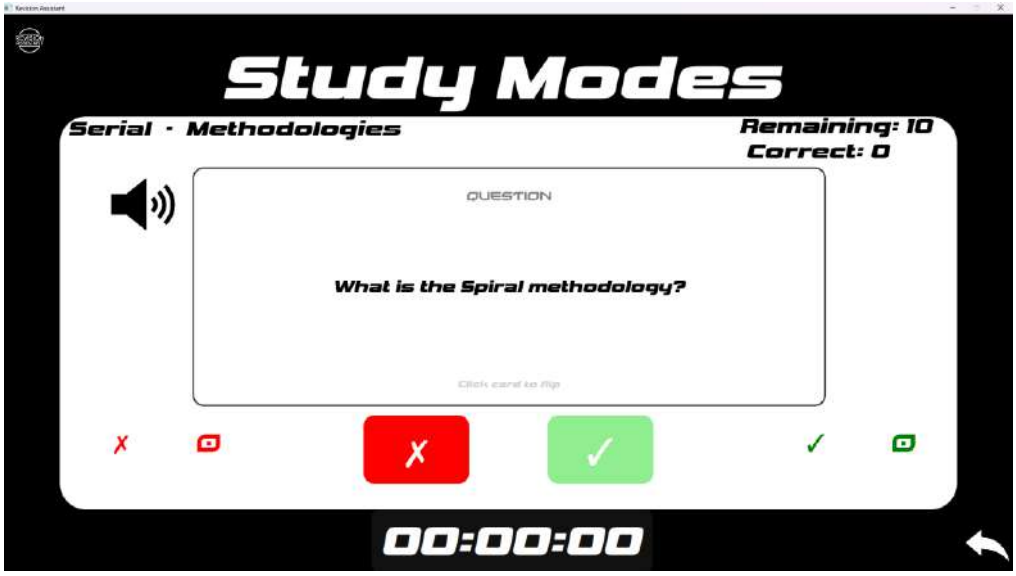
SessionController Code 2 - startSession()

This creates a new settings object by:

```
// New session
public Session(ArrayList<Flashcard> cards, String selectedDeck, String studyMode){
    this.selectedDeck = selectedDeck;
    this.studyMode = studyMode;
    this.currentQueue = new ArrayList<>(cards);
    this.correctList = new ArrayList<>();
    this.incorrectList = new ArrayList<>();
    this.elapsedTime = 0;
    this.showingAnswer = false;
    this.date = LocalDateTime.now();
}
```

Session Code 1- Session Constructor

After starting a new session, the session content is displayed, where it shows the flashcard, the flashcard buttons (tick or cross), elapsed time and other information text.



StudyModesScreen Image 3 - Session Content

This was done by a collection of methods, firstly the buildSessionContent():

```
private void buildSessionContent(Group contentLayer){  3 usages
    // Timer text
    timerLabel = new Label(sessionC.getFormattedTime());
    timerLabel.getStyleClass().add("timerLabel");
    timerLabel.setLayoutX(780); timerLabel.setLayoutY(940);
    progressText = new Text(sessionC.getProgressText());
    progressText.getStyleClass().add("infoText");
    progressText.setLayoutX(1400); progressText.setLayoutY(225);
    // Actual flashcard
    StackPane flashcard = buildCard();
    flashcard.setLayoutX(360); flashcard.setLayoutY(290);
    // TTS Icon
    ImageView ttsIcon = createIcon( iconX: 100,  iconY: 100,  iconURL: "/images/StudyModesScreen/tts.png");
    ttsIcon.setLayoutX(200); ttsIcon.setLayoutY(290);
    ttsIcon.setCursor(Cursor.HAND);
    ttsIcon.setOnMouseClicked( MouseEvent _->{
        // If showing answer, read out answer
        if(sessionC.isShowingAnswer()) tts.speak(sessionC.getCurrentCard().getAnswer());
        // If showing question, read out question
        else tts.speak(sessionC.getCurrentCard().getQuestion());
    });
    StackPane tick = buildFlashcardButton( symbol: "✓", colour: "LightGreen");
    tick.setOnMouseClicked( MouseEvent _-> {
        try {
            handleAnswer( correct: true);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });
    tick.setLayoutX(1835); tick.setLayoutY(760);
    StackPane cross = buildFlashcardButton( symbol: "x",  colour: "red");
    cross.setOnMouseClicked( MouseEvent _-> {
        try {
            handleAnswer( correct: false);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });
    cross.setLayoutX(685); cross.setLayoutY(760);
    // Text for correct count
    correctText = new Text("✓      " + sessionC.getCorrectCount());
    correctText.getStyleClass().add("correctC");
    correctText.setLayoutX(1520); correctText.setLayoutY(830);
    // Text for incorrect count
    incorrectText = new Text("×      " + sessionC.getIncorrectCount());
    incorrectText.getStyleClass().add("incorrectC");
    incorrectText.setLayoutX(200); incorrectText.setLayoutY(830);
    // Extra info text
    Text mode = new Text(sessionC.getStudyMode() + "  -  "
            + sessionC.getSelectedDeck());
    mode.getStyleClass().add("infoText");
    mode.setLayoutX(125); mode.setLayoutY(230);
    Group newContent = new Group(timerLabel, progressText, flashcard, tick, cross, correctText, incorrectText, mode);
    // If TTS is enabled, add the ttsIcon
    if(sc.getCurrentSettings().isTTSEnabled()) newContent.getChildren().add(ttsIcon);
    nav.switchContent(contentLayer, newContent, backButton(contentLayer));
    // Start the timer as soon as this content is built
    startTimer();
```

StudyModesScreen Code 2 - buildSessionContent() method

The flashcard is built by the buildCard() function which returns a StackPane with all the content:

```
// Function returning the flashcard display
private StackPane buildCard() {  1 usage
    cardRect = new Rectangle( width: 1200,  height: 450);
    cardRect.getStyleClass().add("fcRect");

    cardLabel = new Text("Question");
    cardLabel.getStyleClass().add("cardLabel");
    cardLabel.setTranslateY(-170);

    cardBody = new Text(sessionC.getCurrentCard().getQuestion());
    cardBody.getStyleClass().add("cardBody");
    cardBody.setWrappingWidth(1100);

    Text flipHint = new Text("Click card to flip");
    flipHint.getStyleClass().add("flipH");
    flipHint.setTranslateY(185);

    StackPane card = new  StackPane(cardRect, cardLabel, cardBody, flipHint);
    card.getStyleClass().add("flashcard");
    // When clicked, it will flip
    card.setOnMouseClicked( MouseEvent _-> flipFlashcard(card));
    return card;
}
```

StudyModesScreen Code 3 - buildCard()

The flipping is controlled by the flipFlashcard() subroutine which uses two Timelines to perform the flipping animation in the y-axis. Completing the first timeline, switching content and then completing the second timeline.

```
// Subroutine to handle the flipping of the card
private void flipFlashcard(StackPane flashcard){  1 usage
    // Set flag to true, so tick and cross inputs can be handled
    hasFlipped = true;
    // Set the axis of rotation to be y-axis
    flashcard.setRotationAxis(Rotate.Y_AXIS);
    // Add a perspective camera to add 3d effect
    flashcard.getScene().setCamera(new PerspectiveCamera());
    // Create first half of flip, rotating card half way
    Timeline half1 = new Timeline(new KeyFrame(Duration.millis( ms: 150),
            // 150ms, 90 degrees, start slowly
            new KeyValue(flashcard.rotateProperty(),  endValue: 90, Interpolator.EASE_IN)));
    half1.setOnFinished( ActionEvent _->{
        // toggle whether answer side is showing
        sessionC.setShowingAnswer(!sessionC.isShowingAnswer());
        // if answer should now be shown
        if(sessionC.isShowingAnswer()){
            // Change flashcard content to answer
            cardBody.setText(sessionC.getCurrentCard().getAnswer());
            cardLabel.setText("Answer");
            cardRect.getStyleClass().remove( o: "fcRect");
            cardRect.getStyleClass().add("fcBRect");
        }else{
            // Flip back question content
            cardBody.setText(sessionC.getCurrentCard().getQuestion());
            cardLabel.setText("Question");
            cardRect.getStyleClass().remove( o: "fcBRect");
            cardRect.getStyleClass().add("fcRect");
        }
        // rotate card back to -90, so text change is not visible mid-flip
        flashcard.setRotate(-90);
        // Create a second half animation -90 to 0
        Timeline half2 = new Timeline(new KeyFrame(Duration.millis( ms: 150),
                new KeyValue(flashcard.rotateProperty(),  endValue: 0, Interpolator.EASE_OUT)));
        half2.play();
    });
    half1.play();
}
```

StudyModesScreen Code 4 - flipFlashcard animation

In the buildSessionContent(), the buttons are created by the buildFlashcardButton() function returning a StackPane with the symbol.

```
// Function to get tick and cross buttons
private StackPane buildFlashcardButton(String symbol, String colour){
    Rectangle buttonRect = new Rectangle( width: 200, height: 130);
    buttonRect.getStyleClass().add("fcButtonRect");
    buttonRect.setStyle("-fx-fill: " + colour + ";");
    Text label = new Text(symbol);
    label.getStyleClass().add("fcButtonText");
    StackPane flashcardButton = new StackPane(buttonRect, label);
    flashcardButton.getStyleClass().add("fcButton");
    return flashcardButton;
}
```

StudyModesScreen Code 5 – buildFlashcardButton()

If the user clicks these buttons, the handleAnswer() procedure will take care of the following results.

```
// Subroutine to handle the answer when a button is clicked
private void handleAnswer(boolean correct) throws IOException {  2 usages
    // if not flipped, then display error
    if(!hasFlipped){
        CustomAlert.error(correctText.getScene().getWindow(),  message: "Check the answer first");
        // Audial feedback
        playSFX( sound: "incorrect");
        return;
    }
    // If correct, play correct.mp3, else incorrect.mp3
    if(correct) playSFX( sound: "correct");
    if(!correct) playSFX( sound: "incorrect");
    hasSessionStarted = true;
    // Stop TTS when above to switch
    tts.stop();
    boolean sessionComplete = sessionC.recordAnswer(correct);
    // Update counters
    correctText.setText("✓      " + sessionC.getCorrectCount());
    incorrectText.setText("×      " + sessionC.getIncorrectCount());
    // If no remaining cards, stop timer switch to EoRScreen, play sessionComplete.mp3
    if(sessionComplete){
        stopTimer();
        ScreenManager.switchScene(new EndOfRevisionScreen(ac, sc, sessionC, fc).create(), ac.getCurrentUser().getEmail());
        playSFX( sound: "sessionComplete");
        return;
    }
    // Change flashcard content to next car
    cardBody.setText(sessionC.getCurrentCard().getQuestion());
    cardLabel.setText("Question");
    // Change style class and update text
    cardRect.getStyleClass().remove( o: "fcBRect");
    if(!cardRect.getStyleClass().contains("fcRect")) cardRect.getStyleClass().add("fcRect");
    progressText.setText(sessionC.getProgressText());
    hasFlipped = false;
}
```

StudyModesScreen Code 6 - handleAnswer() procedure

If the card has not been flipped yet, the user is displayed an error alert, as you cannot find out whether you are correct or incorrect without checking the answer, playing the incorrect audio effect to indicate, it's not possible.

3.4.4.1.1 Sound Effects

Audio effects were played by the procedure playSFX() which:

```
// procedure which will play the sound fx when requested
private void playSFX(String sound) {  5 usages
    try{
        // Create a temp file on the system to store the sound
        // JavaFX media requires a URI
        InputStream is = getClass().getResourceAsStream( name: "/sounds/" + sound + ".mp3");
        File tempSoundFile = File.createTempFile( prefix: "sound",  suffix: ".mp3");
        // Ensure it is deleted on exit
        tempSoundFile.deleteOnExit();
        // Copy sound data from sound file to file
        Files.copy(Objects.requireNonNull(is), tempSoundFile.toPath(), StandardCopyOption.REPLACE_EXISTING);
        // Play sound via MediaPlayer
        Media testSound = new Media(tempSoundFile.toURI().toString());
        MediaPlayer player = new MediaPlayer(testSound);
        System.out.println(sc.getSFXVolume());
        player.setVolume(sc.getSFXVolume());
        player.play();
    } catch (IOException e){
        System.out.println("play fx error: " + e.getMessage());
    }
}
```

StudyModesScreen Code 7 - playSFX()

This replicates the same method used in SettingsScreen when playing a test sound on release of the slider, a temporary file is created, the data from the sound file is copied into the temp file (which deletes when exited) and MediaPlayer plays it at the correct level according to the user settings.

3.4.4.1.1.1 Sound Effects Remedial Action

Initially when setting this up, the code did not run providing an error message in the console as:

```
Exception in thread "JavaFX Application Thread" java.lang.NullPointerException: Cannot invoke "Models.User.getEmail()" because the return value of "Controllers.SettingsController.getCurrentUser()" is null
    at Controllers.SettingsController.getSFXVolume(SettingsController.java:133)
    at Screens.StudyModesScreen.playSFX(StudyModesScreen.java:362)
    at Screens.StudyModesScreen.lambda$buildSessionContent$0(StudyModesScreen.java:158)
    at javafx.base/com.sun.javafx.event.CompositeEventHandler.dispatchBubblingEvent(CompositeEventHandler.java:86)
    at javafx.base/com.sun.javafx.event.EventHandlerManager.dispatchBubblingEvent(EventHandlerManager.java:272)
    at javafx.base/com.sun.javafx.event.EventHandlerManager.dispatchBubblingEvent(EventHandlerManager.java:189)
    at javafx.base/com.sun.javafx.event.CompositeEventDispatcher.dispatchBubblingEvent(CompositeEventDispatcher.java:59)
    at javafx.base/com.sun.javafx.event.BasicEventDispatcher.dispatchEvent(BasicEventDispatcher.java:58)
21 related frames
```

Console Output 4 - Error Message

The cause of this bug was the existence of two separate instances of SettingsController existing within the system. There was an instance created within the AuthenticationController as well as a second instance created in main (universalSettings) to be passed to the screen classes. Furthermore, user authentication, the setCurrentSettings() setter was only called on the instance of the

SettingsController initialised in the AuthenticationController, hence the other instance never had its currentSettings populated. This means the internal settings of the object remained null.

This was fixed by creating only one instance of the SettingsController in the AuthenticationController class and updating authentication methods.

```
// Create a SettingsController object
private final SettingsController sc = new SettingsController();
```

AuthenticationController Code 11 - Creating a new SettingsController

```
setCurrentUser(user);
sc.createDefaultSettings(email);
sc.setCurrentSettings(sc.loadSettings(email));
```

AuthenticationController Code 12 - update in registerUser()

```
Settings userSettings = sc.loadSettings(email);
sc.setCurrentSettings(userSettings);
setSettingsControllertoSM(sc);
```

AuthenticationController Code 13 - update in loginUser()

```
public SettingsController getSettingsController(){
    return sc;
}
```

AuthenticationController Code 14 - new getter method

```
private final AuthenticationController universalAuth = new AuthenticationController();  2 usag
private final SettingsController universalSettings = universalAuth.getSettingsController();
```

Main Code 1 - updated universal controller objects

3.4.4.1.2 Flashcard Buttons

Now testing the input buttons, the sound effects now played successfully, remediating the possibility of the controllers returning null.

Following this, the function recordAnswer is called, the instance of a SessionController demonstrates controlled delegation, instead of directly allowing the session object's recordAnswer to be called, the controller will intercept the call and execute addition logic (adding the wrong answer into the wrongAnswers.csv).

```
// Function to record answer
public boolean recordAnswer(boolean correct) throws IOException {
    // add to wrongAnswers.csv to
    if(!correct) fc.saveWrongAnswers(session.getCurrentCard());
    // Update it in the session object
    return session.recordAnswer(correct);
}
```

SessionController Code 3 - recordAnswer function

If the flashcard has been marked as incorrect, it will be saved to wrongAnswers.csv via the saveWrongAnswers method.

```
// Procedure to save user's wrong answers to WrongAnswers.csv for the WrongAnswers mode
public void saveWrongAnswers(Flashcard card) throws IOException {  1 usage   Aditya Sainath *
    ArrayList<Flashcard> cards = loadWrongAnswers();
    // Check if already exists
    boolean exists = cards.stream().anyMatch(
            Flashcard c -> c.getQuestion().equals(card.getQuestion()) && c.getAnswer().equals(card.getAnswer())
    );
    if(exists) return;
    try(BufferedWriter bw = new BufferedWriter(new FileWriter(wrongAnswersPath.toFile(),  append: true))){
        // Add speech marks, replace speech marks inside
        bw.write( str: "\"" // Opening speech marks
                + card.getQuestion().replace( target: "\"",  replacement: "\"\"") // Get rid of any speech marks within by adding another
                .replace( target: "\n",  replacement: "\\n") // Replace new lines with \n so readline does not break mid-card
                + "\",\"" // Closing speech marks with comma
                + card.getAnswer().replace( target: "\"",  replacement: "\"\"").replace( target: "\n",  replacement: "\\n"));
        bw.newLine();
    }
}
```

FlashcardController Code 24 - saveWrongAnswers()

The cards are loaded into an ArrayList via loadWrongAnswers() which uses the loadFCDeckFromPath method (with the wrongAnswers.csv path), the array list is checked to see if the card already exists there, if not add it to the .csv.

This csv is placed within the user folder; hence the user can add or remove flashcards from the file if they deem them to be learnt with the flashcard management screen. This allows the user to always be in full control of their revision.

After this, regardless of the input, the recordAnswer function within the Session class is called where:

```
// Returns true if session is complete
public boolean recordAnswer(boolean correct){  1 usage   Aditya Sainath *
    // Remove flashcard from active queue
    Flashcard current = currentQueue.removeFirst();
    // add it to correct list if correct
    if(correct) correctList.add(current);
    else{
        // Add to incorrect list, and place at end of queue again
        incorrectList.add(current);
        currentQueue.add(current);
    }
    // Set to question side for next flashcard
    showingAnswer = false;
    // Return true if the session is complete
    return currentQueue.isEmpty();
}
```

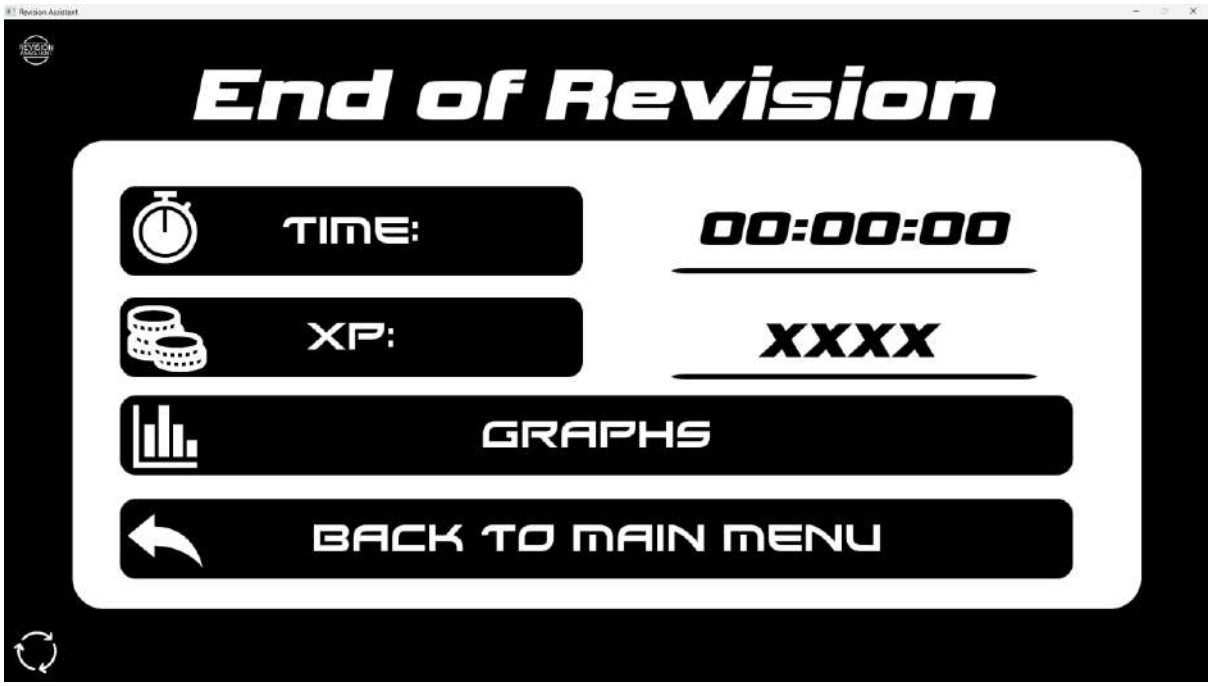
Session Code 2 - recordAnswer()

The algorithm for revision was implemented via a queue-based data structure which allows dynamic re adding of incorrect flashcards, ensuring weaker content is revisited later which therefore creates a spaced revision effect. Alternatively, I could have implemented immediate iteration, however this fails to reinforce weak areas effectively

and encourage short term memorisation instead of building up the user's long-term memory. Instead of incrementing a counter for the incorrect answers and correct answers, they were added to an ArrayList, this was in case of future revisions of the Revision Assistant showing users their strong topics/flashcards within a deck, or weaker topics.

Following this, if the session has not ended the text on screen is updated in line with the user's stats currently; if it has ended, the timer is stopped and the end of revision screen is displayed.

3.4.4.1.3 End of Revision Screen



EndOfRevisionScreen Image 1 – EndOfRevisionScreen

The user is shown their elapsed time of the session, the xp gained (which will be added later), an option to see their stats in terms of graphs (added later too), an option to go back to main menu and an option to retry the session with the same cards.

```
backToMainMenu.button().setOnMouseClicked( MouseEvent _ -> {
    try {
        ScreenManager.switchScene(new MainMenuScreen(ac, sc).create(), ac.getCurrentUser().getEmail());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    ScreenManager.clearHistory();
});
```

EndOfRevisionScreen Code 1 - backToMainMenu, on MouseClick

The system creates a new main menu screen and switches it using the ScreenManager's switchScene method, the stack's history is then cleared so the user cannot navigate back to the end of revision screen by accident.

3.4.4.1.3.1 Remedial Action for Stack Error

Testing this, I went from the EndOfRevisionScreen → Main Menu → Settings → Back, and received this error:

```
> Exception in thread "JavaFX Application Thread" java.lang.IndexOutOfBoundsException Create breakpoint : Index 7 out of bounds for length 2 <3 internal lines>
    at java.base/java.util.Objects.checkIndex(Objects.java:365)
    at java.base/java.util.ArrayList.get(ArrayList.java:428)
    at Models.Stack.pop(Stack.java:20)
    at Components.ScreenManager.goBack(ScreenManager.java:67)
    at Screens.SystemScreens.lambda$getSwitchScreenBackView$0(SystemScreens.java:71)
> <20 folded frames>
```

Console Output 5 - Index out of bounds error

This is because when I switched from the Stack library to my own implementation of a stack, when clearing the stack, I forgot to reset the top counter to -1, hence, to remediate this error within the stack's clear procedure I added the resetting of the top pointer.

```
// Function to clear the stack
public void clear(){  1 usage
    top = -1;
    stackData.clear();
}
```

Stack Code 2 - Remedial Action

Following further tests, the navigation of the Revision Assistant now works fully functionally.

If the user would like to retry the session, clicking the image view on the bottom left, the retrySession() is called, then new StudyModesScreen is created and the continueSession() function returns a scene with the last cards

```
retry.setOnMouseClicked( MouseEvent _ -> {
    session.retrySession();
    StudyModesScreen retryModes = null;
    try {
        retryModes = new StudyModesScreen(ac, sc, fc, session);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    ScreenManager.switchScene(retryModes.continueSession());
});
```

EndOfRevisionScreen Code 2 - on retry MouseClick

```
// Retry
public void retrySession(){  1 usage  Aditya Sainath
    // Shuffle cards
    if(getStudyMode().equals("Shuffle")) shuffle(lastCards);
    // Continue session
    session.reset(lastCards);
}
```

SessionController Code 4 - retrySession()

```
// Method to restart the session with same card
public void reset(ArrayList<Flashcard> lastCards) {
    // Clear the current cards
    currentQueue.clear();
    // add the last cards
    currentQueue.addAll(lastCards);
    // set to question side
    showingAnswer = false;
}
```

Session Code 3 - reset() procedure

```
// if user wants to retry, session is continued
public Scene continueSession(){  1 usage  Aditya Sainath
    Scene scene = create();
    // Find content layer and start a session
    Group contentLayer = (Group) ((Pane) scene.getRoot()).getChildren().get(2);
    buildSessionContent(contentLayer);
    return scene;
}
```

StudyModesScreen Code 8 - continueSession function

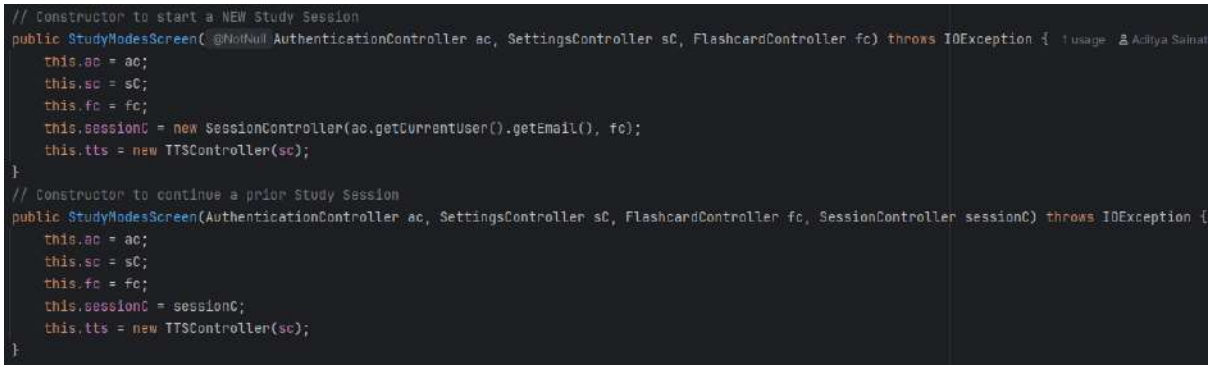
3.4.4.1.3.2 Remedial Action for Continuing a Session

Initially when testing this, the console outputted the following error:

```
Exception in thread "JavaFX Application Thread" java.lang.NullPointerException Create breakpoint : Cannot invoke "Models.Session.getProgressText()" because "this.session" is null
    at Controllers.SessionController.getProgressText(SessionController.java:96)
    at Screens.StudyModesScreen.buildSessionContent(StudyModesScreen.java:135)
    at Screens.StudyModesScreen.continueSession(StudyModesScreen.java:295)
    at Screens.EndOfRevisionScreen.lambda$create$1(EndOfRevisionScreen.java:89)
    at javafx.base/com.sun.javafx.event.CompositeEventHandler.dispatchBubblingEvent(CompositeEventHandler.java:86)
    at javafx.base/com.sun.javafx.event.EventHandlerManager.dispatchBubblingEvent(EventHandlerManager.java:232)
    at javafx.base/com.sun.javafx.event.EventHandlerManager.dispatchBubblingEvent(EventHandlerManager.java:189)
    at javafx.base/com.sun.javafx.event.CompositeEventDispatcher.dispatchBubblingEvent(CompositeEventDispatcher.java:59)
<22 Folded frames>
```

Console Output 6 - error when retrying a session

This was because when creating a new study modes screen, a new session controller object was initialised which could not access data from the previous session to remediate this, I created a new constructor for the class.



```
// Constructor to start a NEW Study Session
public StudyModesScreen( @NotNull AuthenticationController ac, SettingsController sC, FlashcardController fc) throws IOException {  1 usage  Aditya Sainat
    this.ac = ac;
    this.sc = sC;
    this.fc = fc;
    this.sessionC = new SessionController(ac.getCurrentUser().getEmail(), fc);
    this.tts = new TTSController(sc);
}
// Constructor to continue a prior Study Session
public StudyModesScreen(AuthenticationController ac, SettingsController sC, FlashcardController fc, SessionController sessionC) throws IOException {
    this.ac = ac;
    this.sc = sC;
    this.fc = fc;
    this.sessionC = sessionC;
    this.tts = new TTSController(sc);
}
```

StudyModesScreen Code 9 - constructors in StudyModesScreen

This means when the same SessionController is passed back from the EndOfRevisionScreen to the StudyModesScreen.

3.4.4.1.4 Text-To-Speech

Furthermore, in the buildSessionContent() method if the user has TTS enabled (which is the case on default), there will be a volume icon on the screen adjacent to the flashcard, allowing the user to click if they would like to hear the text-to-speech of the flashcard content. Otherwise, this will not be added to the screen.

To implement text-to-speech, I researched two main approaches:

1. Using a JavaFX WebView and calling the browser's built-in speechSynthesis.speak() method via JavaScript through the WebView's engine.
2. Using an external Java Text-to-Speech library.

Initially, the WebView approached was preferred over the external libraries, since it did not require external libraries (making use of JavaFX's built-in libraries) which then avoids increasing the overall program size. However, I was sure that a WebView would have a higher memory overhead than external libraries at runtime, since it loads a browser engine inside the application, although it is hidden.

To begin implementing TTS, I created a TTSController class which would have methods to start the TTS and stop it.

3.4.4.1.4.1 WebView Attempt

```
package Controllers;

import javafx.scene.web.WebEngine;
import javafx.scene.web.WebView;
import java.io.IOException;

public class TTSController {
    // Webview to access browser's built-in speechSynthesis API
    // Hidden from user as it only used for audio
    private final WebEngine engine;
    private final WebView webView;
    private SettingsController sc;
    // Default values
    private double ttsVolume;
    private double rate = 1.0;
    private double pitch = 1.0;

    public TTSController(SettingsController sc) throws IOException {
        this.sc = sc;
        // Create a hidden WebView
        webView = new WebView();
        webView.setVisible(false);
        webView.setPrefSize(0, 0);
        this.ttsVolume = sc.getTTSVolume();
        // Get engine from webview to execute JavaScript
        this.engine = webView.getEngine();
        // Load a blank page so engine is ready to execute scripts
        engine.loadContent("<html><body></body></html>");
    }
    // Speak the given tet using browser's built in speechSynthesis API
    public void speak(String text){
        // Cancel any currently playing speech prior to starting a new
one
        // Create a new utterance with the text
        // Apply current volume, rate and pitch settings
        engine.executeScript(
                "speechSynthesis.cancel();" +
                "var u = new SpeechSynthesisUtterance('" + text + "');"
+
                "u.volume = " + ttsVolume + ";" +
                "u.pitch = " + pitch + ";" +
                "u.rate = " + rate + ";" +
                "speechSynthesis.speak(u);"
        );
    }
    public WebEngine getEngine() {
        return engine;
    }
    public WebView getWebView() {
        return webView;
    }
}
```

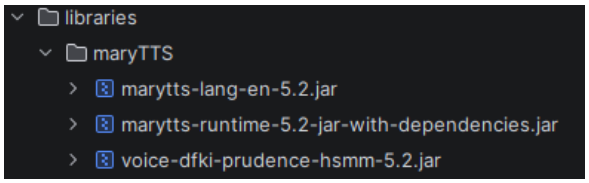
TTSController Code 1 - Initial Attempt

Following this implementation, I attempted to test the TTS where I found out the JavaFX WebEngine uses an older version of WebKit which does not support the Web Speech API. This meant that the speechSynthesis object was undefined, hence the API would not be available, making the WebView approach technically impossible.

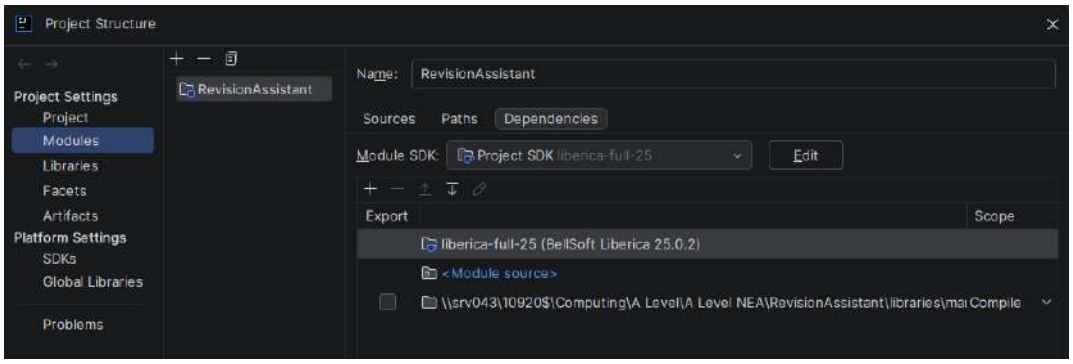
3.4.4.1.4.2 MaryTTS

Since the WebView method did not work, I moved on to using an external library. There were three external libraries which I could have used: FreeTTS, MaryTTS, or Voices. FreeTTS is a really outdated TTS library (around 2009) and the voices sound very robotic, hence I chose against not picking this for the Revision Assistant. The next option was MaryTTS; MaryTTS is fully java based (does not depend on browsers), can run offline (meeting success criteria), provides control over voice configuration. However, the additional libraries increase program size, and the voices may sound a bit robotic compared to other cloud-based AI voices.

Hence to start using the external library I created a library directory within the RevisionAssistant module in IntelliJ, storing the .jars needed for the library.



Following this, in the Project Structure → Modules → Dependencies, I added a new dependency with a link to the MaryTTS directory.



Within TTSController, I imported (the below) to help with playing the audio.

```
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
import marytts.LocalMaryInterface;
import javax.sound.sampled.AudioFileFormat;
import javax.sound.sampled.AudioInputStream;
import javax.sound.sampled.AudioSystem;
import java.io.File;
import java.io.IOException;
```

TTSController Code 2 - imports

```java
// Synthesises and plays the given text
public void speak(String text){  2 usages  Aditya Sainath
    // Stop any audio playing first
    stop();
    try{
        // generate audio from text
        AudioInputStream audio = maryTTS.generateAudio(text);
        File tempfile = File.createTempFile( prefix: "tts",  suffix: ".wav");
        AudioSystem.write(audio, AudioFileFormat.Type.WAVE, tempfile);
        tempfile.deleteOnExit();

        Media media = new Media(tempfile.toURI().toString());
        audioPlayer = new MediaPlayer(media);
        audioPlayer.setVolume(ttsVolume);

        audioPlayer.play();
    }catch(Exception e){
        System.out.println(e.getMessage());
    }
}
// Stop the audio
public void stop(){  4 usages  Aditya Sainath
    if(audioPlayer != null){
        audioPlayer.stop();
        audioPlayer = null;
    }
}
```

TTSController Code 3 - speak and stop methods

The MaryTTS object (initialised in the constructor) will generate an audio for the text parameter, which is put into an AudioInputStream object, audio. Replicating how sound is played in other parts of the solution, I created a temporarily file, writing the audio into the file and playing it at the ttsVolume (set by user, else initially 0.8).

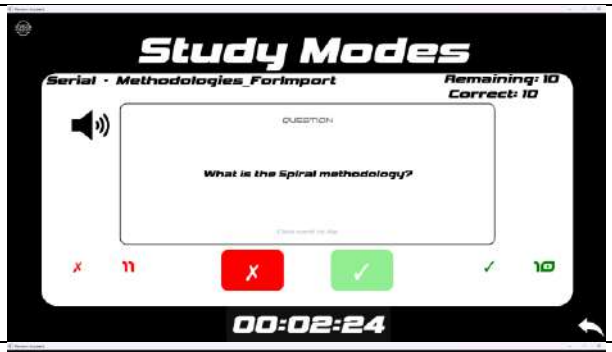
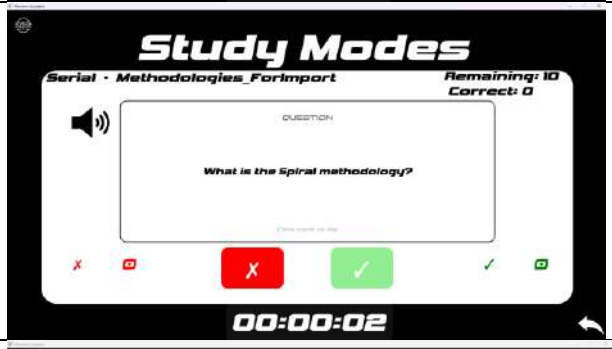
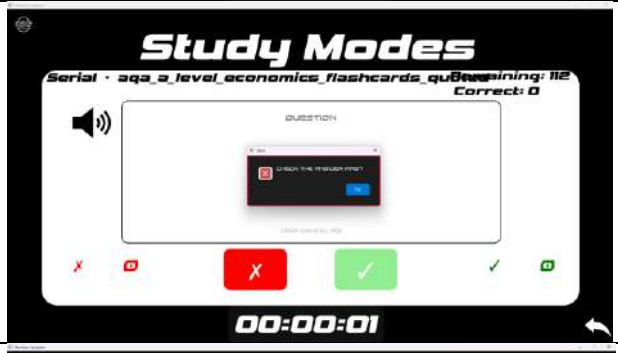
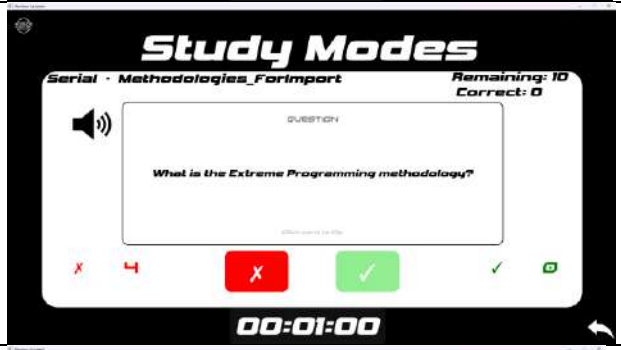
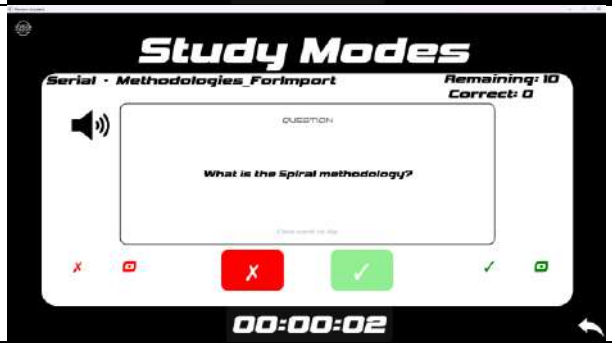
In the StudyModesScreen a TTSController object was initialised and was used when the user clicked on the volume icon adjacent to the flashcard, it then called the speak procedure with the data from the flashcard depending on its side.

Moreover, MaryTTS provides a fully reliable and offline solution for text-to-speech however the voices available sounded a bit robotic, not replicating how technology is right now. At this stage of development, my priority was to implement a fully functional TTS, however in future revisions I expect to gather structured user feedback regarding the voice clarity, pronunciation accuracy, and overall usefulness of the text-to-speech implementation, if user feedback indicates that MaryTTS is having a negative impact of the usability of the system, I will implement other possible alternatives solutions.

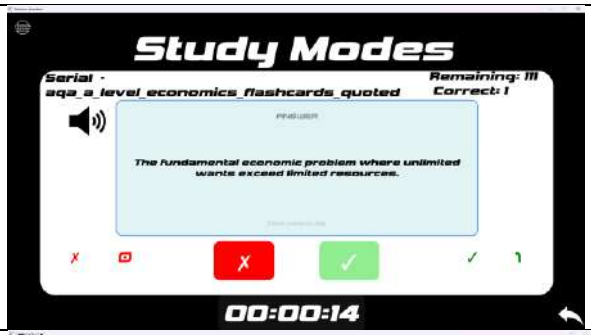
3.4.4.1.5 Testing Study Modes

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	correctAns	10	Normal	Acceptable counter value, data shown on screen.	Acceptable counter value, data shown on screen.	
2.	correctAns	0	Boundary			
3.	Submit input	Submit input without flipping	Erroneous	Error should be displayed to user	Error displayed to user	
4.	incorrectAns	4	Normal	Acceptable counter value, data shown on screen.	Acceptable counter value, data shown on screen.	
5.	incorrectAns	0	Boundary			
6.	Flip card	Click on flashcard to flip	Normal	Flashcard flips	Flash card flipped	
7.	currentSet	“bio.csv”	Normal	Correctly loads, matches file selected.	Correctly loads the file	
8.	currentSet	“asdads.csv”	Erroneous	Not a valid option	Could not find	
9.	modeName	“Serial”	Normal	Serial mode begins; cards appear in original order	Serial mode began	
10.	modeName	“User Adaptive Learning”	Erroneous	Not a valid option for Study Modes	Not an option to select	
11.	Any study mode order	[0, 1, 2, 3, 4]	Normal	Accepted, all cards are shown until correct.	All cards are shown until correct	
12.	Any study mode order	[0, 1, 2, 4, 4]	Erroneous	A session cannot have two of the same cards in the active queue, unless the flashcard deck has two of the same cards in a deck.	Cards are not repeated in current queue unless multiple instances in the deck.	
13.	wrongAnswers	User gets flashcard incorrect	Normal	The flashcard should be added into the wrong answers list, and .csv.	Added to wrongAnswers.csv and incorrect list	

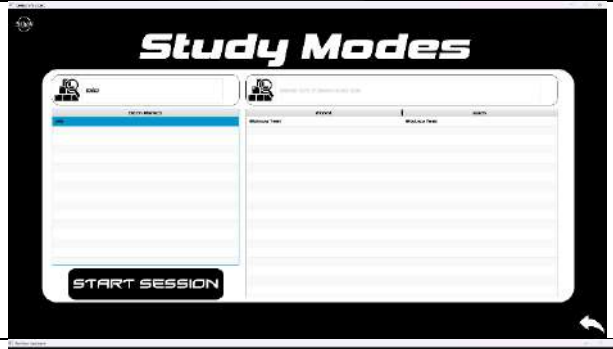
14.	wrongAnswers	User gets flashcard correct	Erroneous	Flashcard should not be added to the wrong answers list.	Flashcard not added to wrong answers list.	
15.	duration	300 seconds	Normal	Time shown as 5 mins	Time shown as 5 mins	
16.	duration	-30 seconds	Erroneous	Duration cannot reach this value; time cannot be negative	Duration cannot reach this value; time cannot be negative	
17.	Shuffle mode	Flashcard deck order	Normal	Must be in different order to the original deck.	Different order to original deck	
18.	wrongAnswers.csv	Contents	Normal	Must not have the same card repeated twice in the .csv.	No repeated cards	
19.	TTS	Clicked on question side	Normal	TTS speaks out question	Question spoke out	
20.	TTS	Clicked on answer side	Normal	TTS speaks out answer	Answer spoke out	

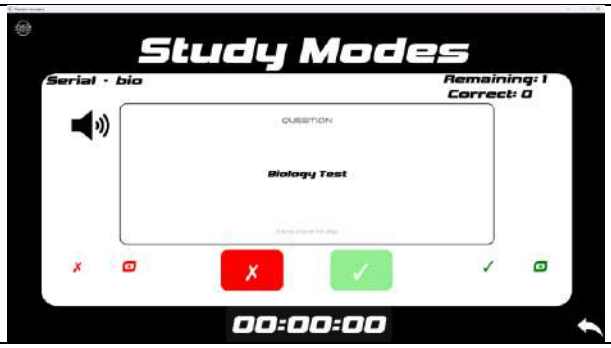
Test No.	Proof
1.	
2.	
3.	
4.	
5.	

6.

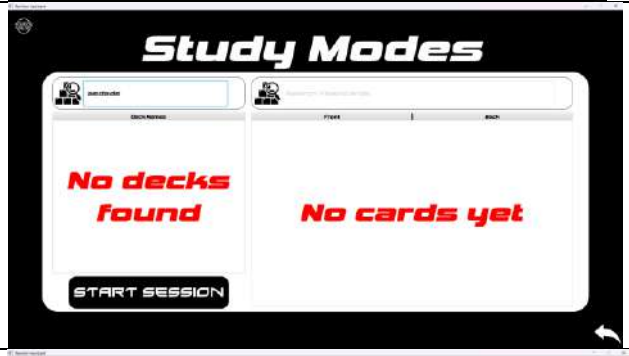

Study Modes
Serial - aqa_a_level_economics_flashcards_quoted
Remaining: 111
Correct: 1
The fundamental economic problem where unlimited wants exceed limited resources.
00:00:14

7.

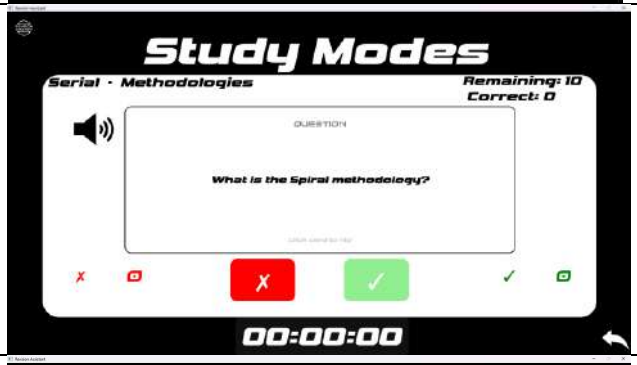

Study Modes
START SESSION


Study Modes
Serial - bio
Remaining: 1
Correct: 0
QUESTION
Biology Test
00:00:00

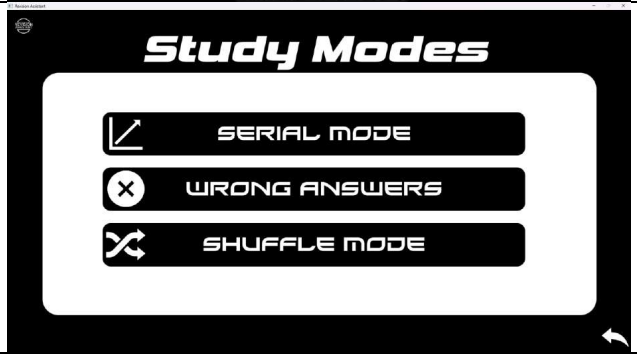
8.


Study Modes
No decks found
No cards yet
START SESSION

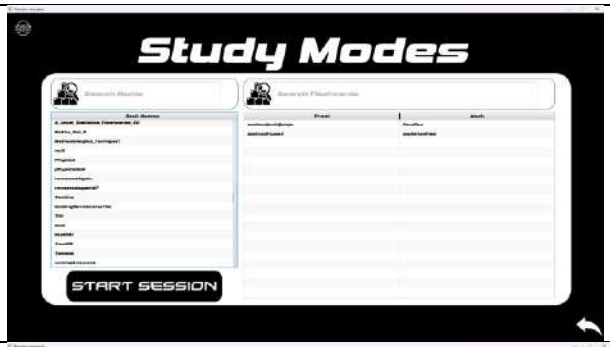
9.

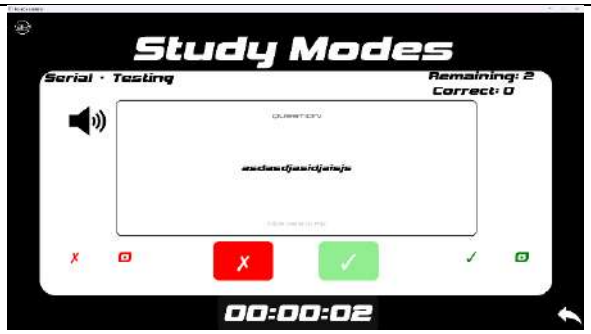

Study Modes
Serial - Methodologies
Remaining: 10
Correct: 0
QUESTION
What is the Spiral methodology?
00:00:00

10.

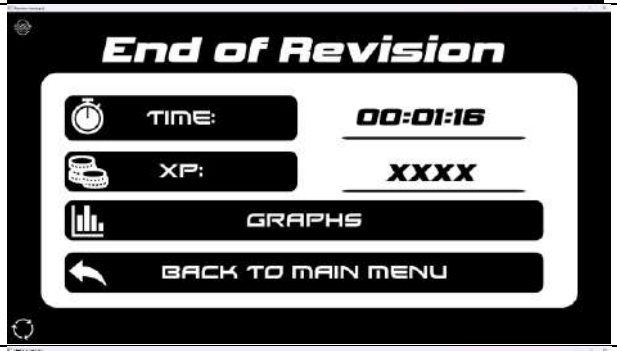

Study Modes
SERIAL MODE
WRONG ANSWERS
SHUFFLE MODE

11.

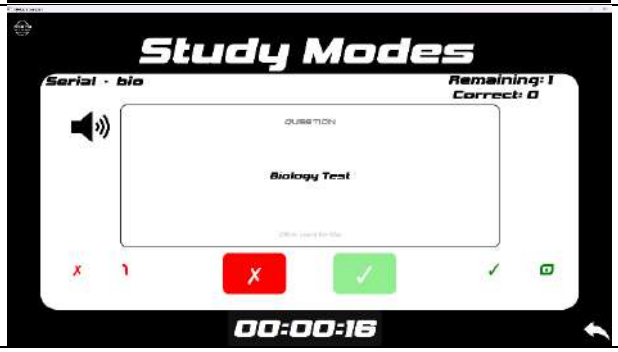

Study Modes
START SESSION

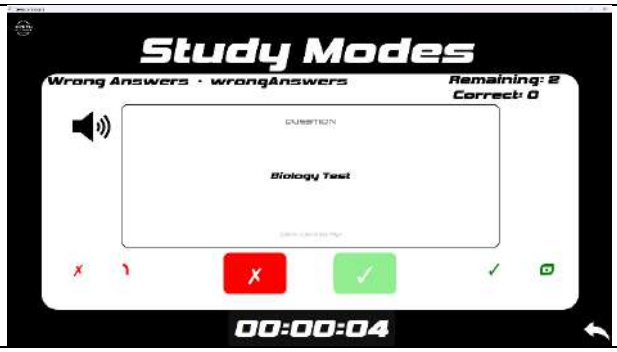

Study Modes
Serial · Testing
Remaining: 2
Correct: 0
00:00:02

12.

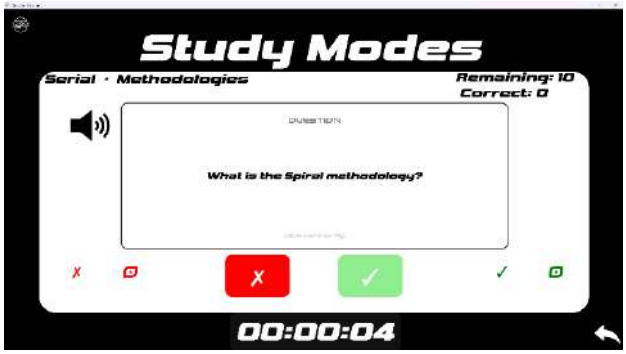

End of Revision
TIME:
00:01:16
XP:
XXXX
GRAPHS
BACK TO MAIN MENU

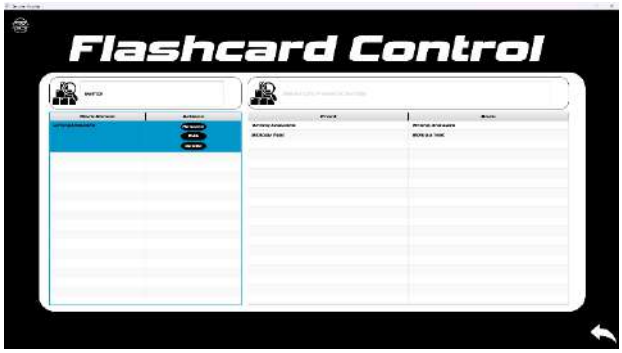
13.


Study Modes
Serial · bio
Remaining: 1
Correct: 0
Biology Test
00:00:16


Study Modes
Wrong Answers · wrongAnswers
Remaining: 2
Correct: 0
Biology Test
00:00:04

14.

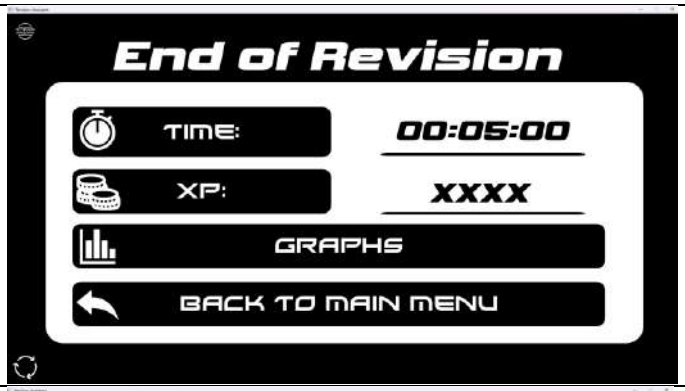

Study Modes
Serial · Methodologies
Remaining: 10
Correct: 0
What is the Spiral methodology?
00:00:04


Flashcard Control

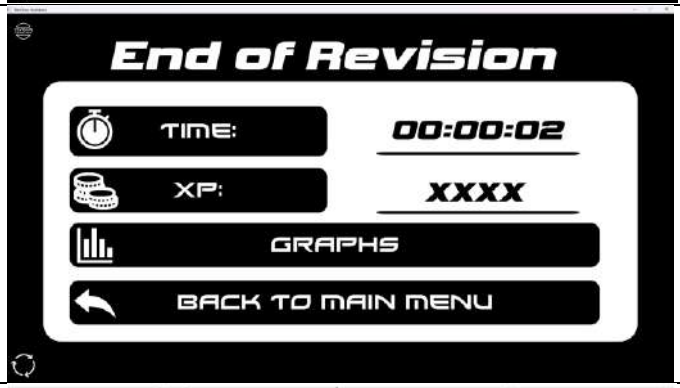




15.

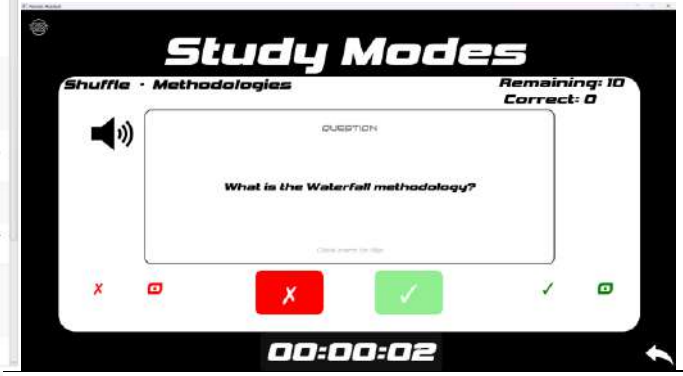

End of Revision
TIME:
00:05:00
XP:
XXXX
GRAPHS
BACK TO MAIN MENU

16.

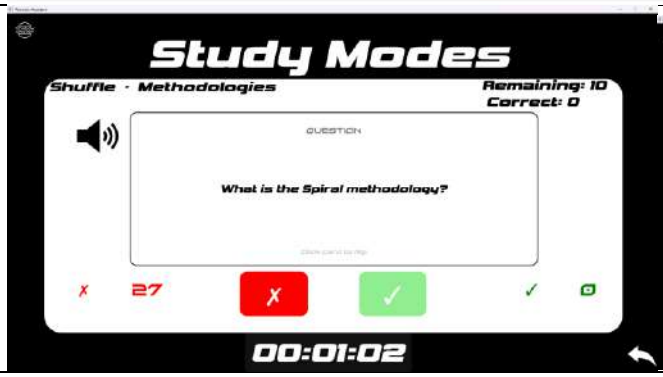

End of Revision
TIME:
00:00:02
XP:
XXXX
GRAPHS
BACK TO MAIN MENU

17.

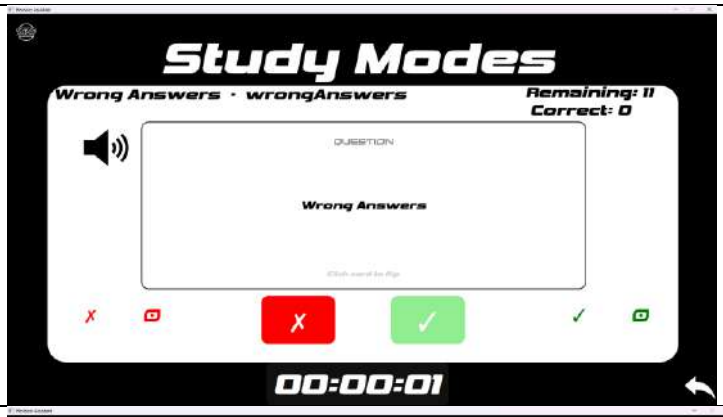
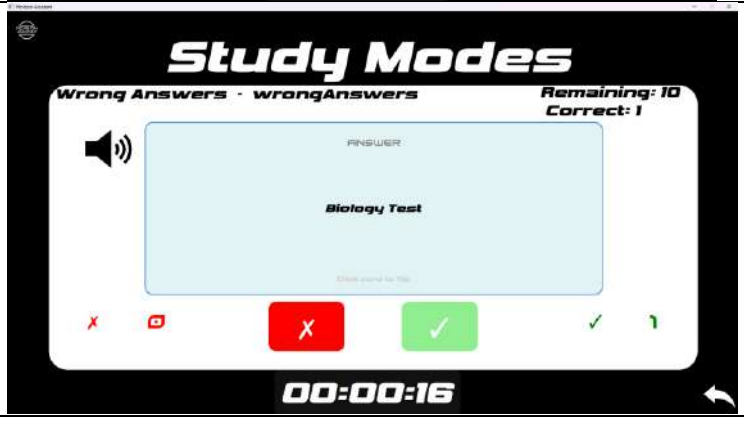

Front
Back
What is the Spiral methodology?
What is the Waterfall methodology?
What is Agile methodology?
What is the Rapid Application Development methodolog...
What is the Extreme Programming methodology?
What is the Spiral methodology?
What is the Waterfall methodology?


Study Modes
Shuffle · Methodologies
Remaining: 10
Correct: 0
QUESTION
What is the Waterfall methodology?
00:00:02

18.


Study Modes
Shuffle · Methodologies
Remaining: 10
Correct: 0
QUESTION
What is the Spiral methodology?
27
00:01:02


Flashcard Control

19.	
20.	

3.4.4.1.6 Remedial Actions

During testing, I noticed that when the “Study Modes” button is clicked in the Revision System Screen, there is a delay before the screen loads. This is most likely because of creating an instance of the TTSController (because of MaryTTS). MaryTTS requires the loading of voice data and configuration files when first instantiated, increasing the set-up time. This delay is only there on the first-time when clicking “Study Modes”. Currently, the initialisation of the TTSController happens when entering the StudyModes screen, hence the user sees a delay.

To fix this delay or reduce it; I have moved the MaryTTS intialisation into a background Thread via Java’s Thread class, so the heavy work concurrently runs without blocking the user UI.

```
// Constructor
public TTSController(double ttsVolume){  2 usages
    this.ttsVolume = ttsVolume;
    // Initialise the MaryTTS on a separate thread -> prevent slow start
    new Thread(() -> {
        try{
            // Initialise MaryTTS
            maryTTS = new LocalMaryInterface();
            // Set default voice
            maryTTS.setVoice(voice);
        }catch(Exception e){
            System.out.println(e.getMessage());
        }
```

TTSController Code 4 - updated Constructor

3.4.4.2 Statistics, Achievements and XP

3.4.4.2.1 Statistics

The system is expected to track multiple user statistics to give the user an insight on their activity, progress and to motivate continued studied. To begin implementing user statistics, I created a new UserStats class in the Models package to represent the data being stored.

```
public class UserStats {  31 usages
    // Constructors for new users
    public UserStats(String email) {  2 usages
        this.email = email;
        this.totalXP = 0;
        this.totalSessions = 0;
        this.totalCorrect = 0;
        this.totalIncorrect = 0;
        this.totalStudyTime = 0;
        this.currentStreak = 0;
        this.longestStreak = 0;
        this.lastStudyDate = "";
        this.consecutiveCorrect = 0;
        this.totalRetries = 0;
        this.unlockedAchievements = new ArrayList<>();
    }
    // Constructor for existing users
    public UserStats(String email, int totalXP, int totalSessions, int totalCorrect,
                     int totalIncorrect, long totalStudyTime, int currentStreak,
                     int longestStreak, String lastStudyDate, int consecutiveCorrect,
                     int totalRetries, ArrayList<String> unlockedAchievements) {
        this.email = email;
        this.totalXP = totalXP;
        this.totalSessions = totalSessions;
        this.totalCorrect = totalCorrect;
        this.totalIncorrect = totalIncorrect;
        this.totalStudyTime = totalStudyTime;
        this.currentStreak = currentStreak;
        this.longestStreak = longestStreak;
        this.lastStudyDate = lastStudyDate;
        this.consecutiveCorrect = consecutiveCorrect;
        this.totalRetries = totalRetries;
        this.unlockedAchievements = unlockedAchievements;
    }
```

UserStats Code 1 - Constructors for UserStats Class

Each UserStats object is instantiated alongside an email, this is so it can be written in to stats.csv just like the settings were, with the email being the identifier for the object. There are two constructors within the system, one for new users and another for loading existing users (from stats.csv) into the file.

The UserStats model is controlled by the StatsController class which performs actions of reading, writing and updating data when statistics are changed. The stats.csv is stored alongside settings.csv, and users.csv files in the files folder (marked as a resource root).

On user registration, a temporary StatsController is used to call the createDefaultUserStats() method.

```
// Write stats into stats.csv for new user
new StatsController().createDefaultUserStats(email);
```

AuthenticationController Code 15 - added to registerUser()

```
// Create default stats for user on registration
public void createDefaultUserStats(String email) throws IOException {
    saveUserStats(new UserStats(email));
}
```

StatsController Code 1 - createDefaultUserStats()

This uses the first constructor to create the default stats and write it into stats.csv by the saveUserStats method.

```
// Save updated stats for a user
public void saveUserStats(UserStats updated) throws IOException {
    ArrayList<UserStats> statsList = convertCSVtoArrayList();
    boolean found = false;
    for(UserStats s : statsList) {
        if(s.getEmail().equals(updated.getEmail())){
            // Replace with updated stats
            statsList.set(statsList.indexOf(s), updated);
            found = true;
            break;
        }
    }
    // If not found, add to list
    if(!found) statsList.add(updated);
    rewriteCSV(statsList);
}
```

StatsController Code 2 - saveUserStats()

Following the same method in SettingsController and Authentication, the stats.csv is converted into an ArrayList of UserStats object, which are then iterated through to find the user, if found replace the UserStats object in the ArrayList with the updated object. If it's not found, then it is added to the stats.csv list and then csv is rewritten.

```
// Rewrite stats.csv with updated data
private void rewriteCSV(ArrayList<UserStats> statsList) throws IOException {  1 usage
    try(BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toFile()))) {
        for(UserStats s : statsList) {
            bw.write(toCSVRow(s));
            bw.newLine();
        }
    }
}
```

StatsController Code 3 - rewriteCSV()

This uses the helper function toCSVRow which converts the UserStats instance into a string to be written into the csv file.

```
// Convert user stats to a csv row for writing
private String toCSVRow(UserStats s){ 1 usage
    StringBuilder line = new StringBuilder(s.getEmail() + "," + s.getTotalXP() + "," + s.getTotalSessions() + ","
            + s.getTotalCorrect() + "," + s.getTotalIncorrect() + ","
            + s.getTotalStudyTime() + "," + s.getCurrentStreak() + ","
            + s.getLongestStreak() + "," + s.getLastStudyDate() + "," + s.getConsecutiveCorrect()
            + "," + s.getTotalRetries());
    for(String achievement : s.getUnlockedAchievements()){
        line.append(",").append(achievement);
    }
    return line.toString();
}
```

StatsController Code 4 - toCSVRow function

Since the number of achievements unlocked by the user are unknown, there is a for each loop iterating through each achievement in the unlocked achievement ArrayList and appending the StringBuilder with the achievement's name.

```
// Read stats.csv into an ArrayList of UserStats objects
private ArrayList<UserStats> convertCSVtoArrayList() throws IOException { 3 usages
    // Ensure folders and files exist before reading
    ensureCSVExists(csvPath);
    ArrayList<UserStats> statsList = new ArrayList<>();
    // Read CSV line by line
    try(BufferedReader br = new  BufferedReader(new FileReader(csvPath.toFile()))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] stats = line.split( regex: ",");
            if(stats.length < 11) throw new IllegalArgumentException("Corrupted Row: " + line);
            String email = stats[0];
            int totalXP = Integer.parseInt(stats[1]);
            int totalSessions = Integer.parseInt(stats[2]);
            int totalCorrect = Integer.parseInt(stats[3]);
            int totalIncorrect = Integer.parseInt(stats[4]);
            long totalStudyTime = Long.parseLong((stats[5]));
            int currentStreak = Integer.parseInt(stats[6]);
            int longestStreak = Integer.parseInt(stats[7]);
            String lastStudyDate = stats[8];
            int consecutiveCorrect = Integer.parseInt(stats[9]);
            int totalRetries = Integer.parseInt(stats[10]);
            ArrayList<String> achievements = new ArrayList<>();
            // After this achievements are listed
            if(stats.length > 11) {
                achievements = new ArrayList<>(
                        Arrays.asList(Arrays.copyOfRange(stats,  from: 11, stats.length))
                );
            }
            UserStats s = new UserStats(email, totalXP, totalSessions, totalCorrect,
                    totalIncorrect, totalStudyTime, currentStreak, longestStreak,
                    lastStudyDate, consecutiveCorrect, totalRetries, achievements);
            statsList.add(s);
        }
    }catch(Exception e) {
        quarantineCorruptedCSV(e, csvPath);
        return new ArrayList<>();
    }
    return statsList;
}
```

StatsController Code 5 - convertCSVtoArrayList()

Since StatsController.java is a subclass of the Controller superclass, it can use the methods ensureCSVExists, as well as quarantineCorruptedCSV, preventing any errors with file reading/writing. When converting the CSV into an ArrayList, each line's variables are extracted and used to make a new UserStats object, after the first 11 parts of the line (which the BufferedReader reads), the unlocked achievements are stored,

the achievement ArrayList is created using the Arrays.copyOfRange method to add index 11 to the end to the ArrayList without manually iterating through. These objects are added to the ArrayList which is then returned.

In AuthenticationController in loginUser, the following code was added to once again ensure that users have their stats data saved in stats.csv.

```
// Write default stats if not saved already, via a temporary object
if(!new StatsController().isUserStatsSaved(email)) new StatsController().createDefaultUserStats(email);
```

AuthenticationController Code 16 - added to loginUser()

```
// is user stats saved?
public boolean isUserStatsSaved(String email) throws IOException {
    ArrayList<UserStats> statsList = convertCSVtoArrayList();
    for(UserStats s : statsList) {
        if(s.getEmail().equals(email)) return true;
    }
    return false;
}
```

StatsController Code 6 - isUserStatsSaved()

If the user is not found, then then the default stats are saved for them.

3.4.4.2.2 XP and Achievements

3.4.4.2.2.1 XP

The user's xp and achievements are controlled by the XPController and AchievementController respectively.

As of right now, there were only 4 xp rules to follow: 1 hour is 500xp, 90%+ accuracy gives a bonus of 100xp, a correct answer is 15xp and an incorrect answer rewards 5xp. Hence it is quite excessive to create an XPController for such little code, however planning for future revisions of the RevisionAssistant, there may be changes to XP rewarding, adding new rules in place such as bonuses for streaks, achievements, or completing a deck. Hence creating an XPController class the maintainability and scalability of the system will improve since all XP related calculations are centred in one component.

```java
// Function which returns the xp from a session
public int calculateXP(Session session){  2 usages
    int xp = 0;
    // Per answer
    xp += session.getCorrectCount() * 15;
    xp += session.getIncorrectCount() * 5;
    // Time XP, 500 per hour
    xp += (int) ((session.getElapsedTime() / 3600) * 500);
    // Accuracy bonus
    if(session.getAccuracy() >= 90.0) xp += 100;
    return xp;
}
```

XPController Code 1 - calculateXP() function

3.4.4.2.2.1.1 XP Remedial Action

On initial tests of this, I noticed that the XP was not being rewarded for the time of the session, this was because the session.getElapsedTime() / 3600 performs integer division, so if the session is less than an hour, it will divide to zero. To remediate this issue, I had to force floating point division by replaces 3600 with 3600.0.

```java
// Function which returns the xp from a session
public int calculateXP(Session session){  2 usages
    int xp = 0;
    // Per answer
    xp += session.getCorrectCount() * 15;
    xp += session.getIncorrectCount() * 5;
    // Time XP, 500 per hour
    xp += (int) Math.round(((session.getElapsedTime() / 3600.0) * 500));
    // Accuracy bonus
    if(session.getAccuracy() >= 90.0) xp += 100;
    return xp;
}
```

XPController Code 2 - Remediated calculateXP() function

Furthermore, I also realised placing (int) after the = sign does not automatically round the number, instead it just drops the decimal and keeps the integer, hence I am using Math.round() to round the xp rewarded due to time upwards. With following tests, the remedial action was successful, and I no longer receive these issues.

3.4.4.2.2.2 Achievements

In the current implementation of achievements, I have added 14 possible achievements for users to achieve:

1. Sharp Mind: get 5 flashcards correct in a row.
2. Perfect Run: get 10 flashcards correct in a row.
3. Flawless: 100% accuracy on a session.
4. Getting Started: 10 flashcards reviewed.
5. Knowledge Expert: 100 flashcards correct.
6. 100 Club: 100 flashcards reviewed.

7. Revision Rookie: 5 sessions completed.
8. Revision Master: 50 sessions completed.
9. Study Streak: Study 3 days in a row.
10. Dedicated Learner: Study 7 days in a row.
11. Revision Veteran: Study 30 days in total.
12. Keep Trying: Retry a session.
13. Never Give Up: Retry 10 sessions.
14. Ultimate Scholar: Achieve all achievements.

These conditions to meet these achievements are coded into AchievementController's checkForAchievements function which uses the tryUnlock procedure.

```
// Unlocks an achievement if the conditions have been met and not already unlocked
private void tryUnlock(String achievement, UserStats stats,  14 usages
                       boolean condition, ArrayList<String> unlockedAchievements) {
    if(condition && !stats.isAchievementUnlocked(achievement)) {
        stats.addAchievement(achievement);
        unlockedAchievements.add(achievement);
    }
}
```

AchievementController Code 1- tryUnlock()

If the condition is met and the user does not already have this achievement, then it is added to the ArrayList in stats as well as the unlockedAchievements (parameter) ArrayList.

```
public boolean isAchievementUnlocked(String achievement){  3
    return this.unlockedAchievements.contains(achievement);
}
```

UserStats Code 2 - isAchievementUnlocked() function

```
public void addAchievement(String achievement){  1 usage
    if(!isAchievementUnlocked(achievement)) this.unlockedAchievements.add(achievement);
}
```

UserStats Code 3 - addAchievement() procedure

The addAchievement procedure also does a check to see if the achievement is unlocked, this is just to ensure that the achievement is not repeated within the unlockedAchievements ArrayList.

The function checkForAchievements() is called after a session and will call tryUnlock procedure 14 (since there are 14 achievements) times to check if any new achievements have been added.

```java
// Return list of newly unlocked achievements after a session
public ArrayList<String> checkForAchievements(UserStats stats, Session session, boolean isRetrySession){
    ArrayList<String> newAchievements = new ArrayList<>();
    // Correct streak achievements
    // 5 in a row
    tryUnlock( achievement: "Sharp Mind", stats,
            condition: stats.getConsecutiveCorrect() >= 5, newAchievements);
    // 10 in a row
    tryUnlock( achievement: "Perfect Run", stats,
            condition: stats.getConsecutiveCorrect() >= 10, newAchievements);
    // Accuracy
    tryUnlock( achievement: "Flawless", stats,
            condition: session.getAccuracy() == 100.0, newAchievements);
    // All time answer achievements
    // 10 flashcards reviewed
    tryUnlock( achievement: "Getting Started", stats,
            condition: stats.getTotalCorrect() + stats.getTotalIncorrect() >= 10, newAchievements);
    // 100 correct
    tryUnlock( achievement: "Knowledge Expert", stats,
            condition: stats.getTotalCorrect() >= 100, newAchievements);
    // 100 flashcards reviewed
    tryUnlock( achievement: "100 Club", stats,
            condition: stats.getTotalCorrect() + stats.getTotalIncorrect() >= 100, newAchievements);
    // Session count achievements
    // 5 sessions
    tryUnlock( achievement: "Revision Rookie", stats,
            condition: stats.getTotalSessions() >= 5, newAchievements);
    // 50 sessions
    tryUnlock( achievement: "Revision Master", stats,
            condition: stats.getTotalSessions() >= 50, newAchievements);
    // Streaks
    // 3 days in a row
    tryUnlock( achievement: "Study Streak", stats,
            condition: stats.getCurrentStreak() >= 3, newAchievements);
    // 7 days in a row
    tryUnlock( achievement: "Dedicated Learner", stats,
            condition: stats.getCurrentStreak() >= 7, newAchievements);
    // 30 days in revision total
    tryUnlock( achievement: "Revision Veteran", stats,
            condition: stats.getTotalStudyTime() >= 30*24*3600, newAchievements);
    // Retrying achievements
    // Retry a session
    tryUnlock( achievement: "Keep Trying", stats,
           isRetrySession, newAchievements);
    // Retry 10 sessions
    tryUnlock( achievement: "Never Give Up", stats,
            condition: stats.getTotalRetries() >= 10, newAchievements);
    // If user has unlocked all achievements
    tryUnlock( achievement: "Ultimate Scholar", stats,
            condition: stats.getUnlockedAchievements().size() >= 13 , newAchievements);
    return newAchievements;
}
```

AchievementController Code 2 - checkForAchievements function

This function makes use of getters within the UserStats class.

3.4.4.2.2.3 Updating Statistics Post-Study-Session

When the session has been completed, or the user decides to end the session, the changeToSummary() procedure is called. This will call the calculateXP function,

updateStats function, switch the scene to the EndOfRevisionScreen and play the sound effect of the session finishing.

```
// Procedure which will get updated UserStats object and switch the scene to EORScreen
private void changeToSummary() throws IOException {  2 usages
    int xpGained = new XPController().calculateXP(sessionC.getSession());
    UserStats updated = stats.updateStats(ac.getEmail(), sessionC.getSession(),
            xpGained, isRetrySession);
    ScreenManager.switchScene(new EndOfRevisionScreen(ac, sc, sessionC, fc, stats, updated).create(),
            ac.getEmail());
    playSFX( sound: "sessionComplete");
}
```

StudyModesScreen Code 10 - changeToSummary() procedure

With there being achievements above retrying sessions, there was a new global Boolean flag created for isRetrySession, this is set to true in the continueSession function.

The xp gained for the session is calculated and the updateStats function in StatsController is called.

```
// Update all stats after a session and returns the updated stats
public UserStats updateStats(String email, Session session,  1 usage
                             int xpGained, boolean isRetry) throws IOException {
    UserStats stats = loadUserStats(email);
    // Add XP earned
    stats.addXP(xpGained);
    // Increment session count
    stats.addSession();
    // Add correct and incorrect counts from the session
    stats.addCorrect(session.getCorrectCount());
    stats.addIncorrect(session.getIncorrectCount());
    // Add elapsed time
    stats.addStudyTime(session.getElapsedTime());
    // If retried, increment the counter
    if(isRetry) stats.addRetry();
    // Update the consecutive streak
    if(stats.getConsecutiveCorrect() < session.getConsecutiveCorrect())
        stats.setConsecutiveCorrect(session.getConsecutiveCorrect());
    // Update streak
    updateDayStreak(stats);
    // Update achievements
    AchievementController achC = new AchievementController();
    ArrayList<String> updatedAchievements = achC.checkForAchievements(stats, session, isRetry);
    saveUserStats(stats);
    return stats;
}
```

StatsController Code 7 - updateStats function

The user stats are loaded via the loadUserStats function:

```
// Finds the user stats for a given email, if not find return default
public UserStats loadUserStats(String email) throws IOException {  3 usages
    ArrayList<UserStats> statsList = convertCSVtoArrayList();
    for(UserStats s : statsList) {
        if(s.getEmail().equals(email)) return s;
    }
    // If not found
    return new UserStats(email);
}
```

StatsController Code 8 – loadUserStats

Helper methods within the UserStats class are then used to update the statistics:

```
// Helper Methods
public void addXP(int xp) { this.totalXP += xp; }
public void addSession() { this.totalSessions++; }
public void addCorrect(int correct) { this.totalCorrect+=correct; }
public void addIncorrect(int incorrect) { this.totalIncorrect+=incorrect; }
public void addStudyTime(long st) { this.totalStudyTime += st; }
public void addRetry() { this.totalRetries++; }
public boolean isAchievementUnlocked(String achievement) { return this.unlockedAchievements.contains(achievement); }
public void addAchievement(String achievement){...}
```

UserStats Code 4 - Helper Methods

A new attribute was added to the Session class: consecutiveCorrect, this was updated in SessionController's recordAnswer function, if the user gets an answer correct it is incremented, otherwise it is reset to 0.

```
// Function to record answer
public boolean recordAnswer(boolean correct) throws IOException {
    // add to wrongAnswers.csv
    if(!correct) {
        fc.saveWrongAnswers(session.getCurrentCard());
        session.resetConsecutiveCorrect();
    }else session.incrementConsecutiveCorrect();
    // Update it in the session object
    return session.recordAnswer(correct);
}
```

SessionController Code 5 - updated recordAnswer()

In the updateStats function, if the user has got a higher consecutively correct streak than previously, it is updated otherwise it is not.

Following that the day streak is updated by the following method:

```
// Update the day streak
private void updateDayStreak(UserStats stats){ 1 usage
    String today = LocalDate.now().format(dtf);
    String last = stats.getLastStudyDate();
    if(last == null || last.isBlank()){
        stats.setCurrentStreak(1);
    }else if (!last.equals(today)) {
        // Consecutive days
        LocalDate lastDate = LocalDate.parse(last, dtf);
        if(lastDate.plusDays( daysToAdd: 1).equals(LocalDate.now())){
            stats.setCurrentStreak(stats.getCurrentStreak() + 1);
        }else{
            // New streak
            stats.setCurrentStreak(1);
        }
    } else {
        // no change
    }
    if(stats.getCurrentStreak() > stats.getLongestStreak()){
        stats.setLongestStreak(stats.getCurrentStreak());
    }
    // Update the day
    stats.setLastStudyDate(today);
}
```

StatsController Code 9 - updateDayStreak()

The dtf is a global variable setting the date format to dd/MM/yyyy to be written into stats.csv. If this the first session, or the study days are not on consecutive days, the streak is set to 1. If it is consecutive days, then the streak is incremented by 1.

The achievements are then checked for, and the updated stats are written into stats.csv.

After this, the screen is changed to the EndOfRevisionScreen.

```
// Constructor
public EndOfRevisionScreen(AuthenticationController ac, SettingsController sc, SessionController session,
                           FlashcardController fc, StatsController stats,  UserStats userStats){
    this.ac = ac;
    this.sc = sc;
    this.session = session;
    this.fc = fc;
    this.stats = stats;
    this.userStats = userStats;
}
```

EndOfRevisionScreen Code 4 - updated Constructor

```
Text xpText = new Text(String.valueOf(new
XPController().calculateXP(session.getSession())));
```

EndOfRevisionScreen Code 3 - used to display XP text

This was added to the create function to display the XP gained.

3.4.4.2.3 Displaying Stats and Achievements to Users

When the user goes the Main Menu screen and clicks the “User Management” button, they will be displayed the following options:



Main Menu Screen Image 3 - User Management Page

As of prototype 4, the user can only click the “Statistics” or “Achievement” buttons with File Management to be added in future prototypes.

```
// Procedure which builds the content for the User Management Page
private void buildUMContent(Group contentLayer) {
    RectangleButtonBundle statistics = ui.buildRectangularButtonBundle(rectW, rectH, "Statistics", "buttonText", 293, 315);
    ImageView sIcon = createIcon(293, 310, "/images/MainMenuScreen/stats.png");
    statistics.button().setOnMouseClicked(_ -> {
        try {
            buildStatisticsContent(contentLayer);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });

    RectangleButtonBundle achievements = ui.buildRectangularButtonBundle(rectW, rectH, "Achievements", "buttonText", 293, 485);
    ImageView aIcon = createIcon(293, 480, "/images/MainMenuScreen/achievement.png");
    achievements.button().setOnMouseClicked(_ -> {
        try {
            buildAchievementsContent(contentLayer);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });

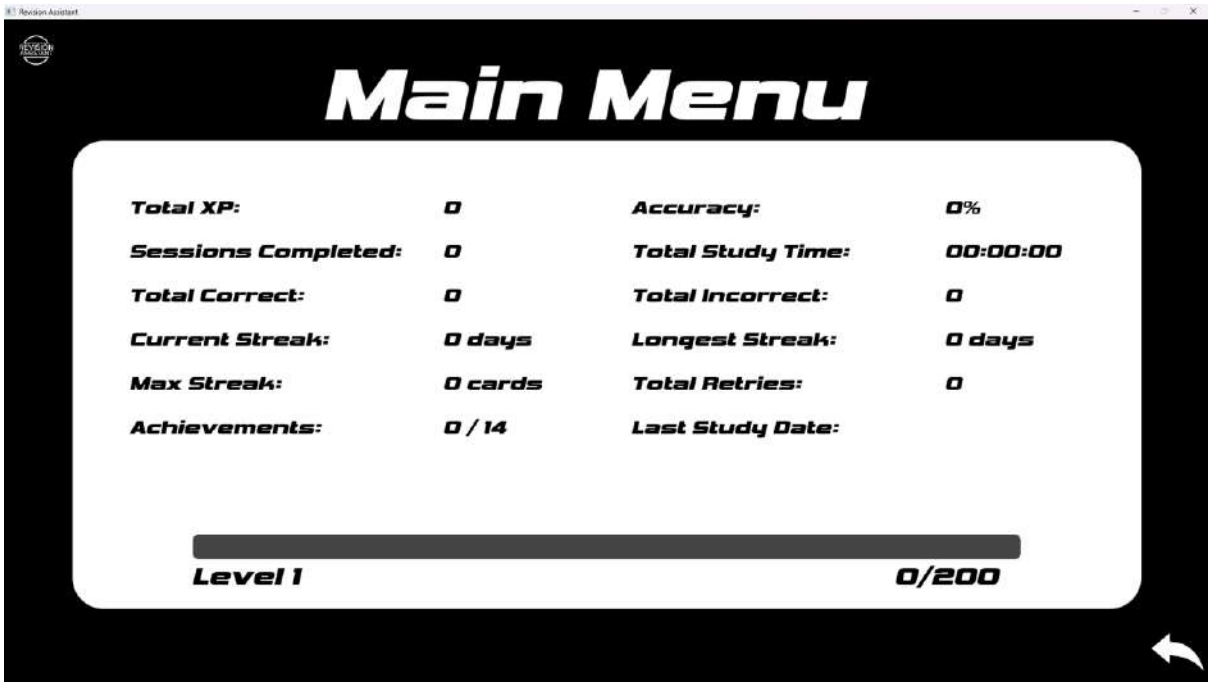
    RectangleButtonBundle fileManagement = ui.buildRectangularButtonBundle(rectW, rectH, "File Management", "buttonText", 293, 656);
    ImageView fmIcon = createIcon(293, 651, "/images/MainMenuScreen/folder.png");
      fileManagement.button().setOnMouseClicked(_ -> );

    Group options = new Group(statistics.button(), fileManagement.button(), achievements.button(), sIcon, aIcon, fmIcon);

    nav.switchContent(contentLayer, options, getBackView(contentLayer));
}
```

Main Menu Screen Code 2 - buildUMContent() procedure

3.4.4.2.3.1 All Time Statistics



Main Menu Screen Image 4 - Statistics Page

This is displayed to the user when they click on the Statistics button.

```
// Build content where user can see his stats
private void buildStatisticsContent(Group contentLayer) throws IOException {  1 usage
    UserStats uStats = stats.loadUserStats(ac.getEmail());
    // Level header
    Text levelText = new Text("Level " + uStats.getLevel());
    levelText.getStyleClass().add("levelText");
    ui.setPosition(levelText,  x: 300,  y: 900);
    // XP Bar
    Rectangle bar = new Rectangle( x: 300,  y: 820,  width: 1320,  height: 40);
    bar.getStyleClass().add("XPBar");
    // XP Progress
    double progress = (double) uStats.getXPIntoCurrentLevel() / uStats.getXPForNext();
    progress = Math.min(progress, 1.0); // clamp to 1.0
    Rectangle progressFill = new Rectangle( x: 300,  y: 820,  width: 1320*progress,  height: 40);
    progressFill.getStyleClass().add("progressFill");
    // Text under bar
    Text xpProgressText = new Text( uStats.getXPIntoCurrentLevel() + "/" + uStats.getXPForNext());
    xpProgressText.getStyleClass().add("infoText");
    ui.setPosition(xpProgressText,  x: 1420,  y: 900);

    nav.switchContent(contentLayer, createStatsGrid(uStats), levelText, xpProgressText, bar, progressFill, getBackView(contentLayer));
}
```

Main Menu Screen Code 3 - buildStatisticsContent()

The level bar as well as text is created and switched via nav.switchContent, the actual statistics data is built by the createStatsGrid function which returns a group.

The following helper methods are used for this stage:

```java
// Get the user's current level
public int getLevel(){  3 usages
    // Level N needs N*100 XP
    int level = 1;
    while(true){
        // Sum to n formula
        int xpForNext = (level * (level+1) / 2) * 100;
        if(totalXP < xpForNext) break;
        level++;
    }
    return level;
}
public int getXPForNext(){  2 usages
    int level = getLevel();
    return (level + 1) * 100;
}
// How much xp user has earned in their current level
public int getXPIntoCurrentLevel(){  2 usages
    int level = getLevel();
    int xp = 0;
    // Calculate total xp required to reach current level
    for(int i = 0; i < level; i++){
        xp += i*100;
    }
    // How much xp user has got in their current level
    return totalXP-xp;
}
```

UserStats Code 5 - extra helper methods

3.4.4.2.3.1.1 Remedial Actions

Initially when entering the statistics page as a new user, the console outputted the following error message:

```
Exception in thread "JavaFX Application Thread" java.lang.ArithmeticException Create breakpoint : / by zero
    at Screens.MainMenuScreen.createStatsGrid(MainMenuScreen.java:149)
    at Screens.MainMenuScreen.buildStatisticsContent(MainMenuScreen.java:141)
    at Screens.MainMenuScreen.lambda$buildUMContent$0(MainMenuScreen.java:97)
    at javafx.base/com.sun.javafx.event.CompositeEventHandler.dispatchBubblingEvent(CompositeEventHandler.java:86)
    at javafx.base/com.sun.javafx.event.EventHandlerManager.dispatchBubblingEvent(EventHandlerManager.java:232)
    at javafx.base/com.sun.javafx.event.EventHandlerManager.dispatchBubblingEvent(EventHandlerManager.java:189)
    at javafx.base/com.sun.javafx.event.CompositeEventDispatcher.dispatchBubblingEvent(CompositeEventDispatcher.java:59)
    at javafx.base/com.sun.javafx.event.BasicEventDispatcher.dispatchEvent(BasicEventDispatcher.java:58)
> <21 folded frames>
```

Console Output 7 - divide by zero error

This was because when calculating the accuracy within the 2D array for the statistics, there was a divide by 0 error for users who have never study a flashcard deck.

```java
String[][] grid = {
        {"Total XP", String.valueOf(uStats.getTotalXP())},
        {"Accuracy", (100 * uStats.getTotalCorrect() /  (uStats.getTotalIncorrect() + uStats.getTotalCorrect()) + "%")},
        {"Sessions Completed", String.valueOf(uStats.getTotalSessions())},
        {"Total Study Time", formatTime(uStats.getTotalStudyTime())},
        {"Total Correct", String.valueOf(uStats.getTotalCorrect())},
        {"Total Incorrect", String.valueOf(uStats.getTotalIncorrect())},
        {"Current Streak", uStats.getCurrentStreak() + " days"},
        {"Longest Streak", uStats.getLongestStreak() + " days"},
        {"Max Streak", uStats.getConsecutiveCorrect() + " cards"},
        {"Total Retries", String.valueOf(uStats.getTotalRetries())},
        {"Achievements", uStats.getUnlockedAchievements().size() + " / 14"},
        {"Last Study Date", uStats.getLastStudyDate()},
};
```

Main Menu Screen Code 4 - Statistics 2D Array Grid

This was replaced with:

```
// Preventing divide by 0 error
int correct = uStats.getTotalCorrect();
int incorrect = uStats.getTotalIncorrect();
int accuracy;
if(correct + incorrect == 0){accuracy = 0;}
else{accuracy = correct / (correct + incorrect);}
// 2D array used to store each stat as a pair, avoiding repeating code
String[][] grid = {
        {"Total XP", String.valueOf(uStats.getTotalXP())},
        {"Accuracy", (100 * accuracy + "%")},
        {"Sessions Completed", String.valueOf(uStats.getTotalSessions())},
        {"Total Study Time", formatTime(uStats.getTotalStudyTime())},
        {"Total Correct", String.valueOf(correct)},
        {"Total Incorrect", String.valueOf(incorrect)},
        {"Current Streak", uStats.getCurrentStreak() + " days"},
        {"Longest Streak", uStats.getLongestStreak() + " days"},
        {"Max Streak", uStats.getConsecutiveCorrect() + " cards"},
        {"Total Retries", String.valueOf(uStats.getTotalRetries())},
        {"Achievements", uStats.getUnlockedAchievements().size() + " / 14"},
        {"Last Study Date", uStats.getLastStudyDate()},
};
```

Main Menu Screen Code 5 - updated Stats grid

If the user has not studied flashcards, the accuracy is set to 0, else the calculation is done.

The full createStatsGrid function is:

```
// Creates a grid of stats
private Group createStatsGrid(UserStats uStats) {  1 usage
    Group statsGrid = new Group();
    // Preventing divide by 0 error
    int correct = uStats.getTotalCorrect();
    int incorrect = uStats.getTotalIncorrect();
    int accuracy;
    if(correct + incorrect == 0){accuracy = 0;}
    else{accuracy = correct / (correct + incorrect);}
    // 2D array used to store each stat as a pair, avoiding repeating code
    String[][] grid = {
            {"Total XP", String.valueOf(uStats.getTotalXP())},
            {"Accuracy", (100 * accuracy + "%")},
            {"Sessions Completed", String.valueOf(uStats.getTotalSessions())},
            {"Total Study Time", formatTime(uStats.getTotalStudyTime())},
            {"Total Correct", String.valueOf(correct)},
            {"Total Incorrect", String.valueOf(incorrect)},
            {"Current Streak", uStats.getCurrentStreak() + " days"},
            {"Longest Streak", uStats.getLongestStreak() + " days"},
            {"Max Streak", uStats.getConsecutiveCorrect() + " cards"},
            {"Total Retries", String.valueOf(uStats.getTotalRetries())},
            {"Achievements", uStats.getUnlockedAchievements().size() + " / 14"},
            {"Last Study Date", uStats.getLastStudyDate()},
    };
    int leftX = 200;
    int rightX = 1000;
    int startY = 310;
    int rowH = 70;
    for(int i = 0; i < grid.length; i++){
        // Even rows go on the left, odd goes to the right
        int x = (i % 2 == 0) ? leftX : rightX;
        int y = startY + (i/2) * rowH;
        // Create the stat text
        Text statNameText = new Text(grid[i][0] + ":");
        statNameText.getStyleClass().add("statText");
        ui.setPosition(statNameText, x, y);
        // Create the actual statistic to accompany this text
        Text statistic = new Text(grid[i][1]);
        statistic.getStyleClass().add("statText");
        ui.setPosition(statistic,  x + 500, y);
        statsGrid.getChildren().addAll(statNameText, statistic);
    }
    return statsGrid;
```

Main Menu Screen Code 6 – createStatsGrid

This function creates a visual grid of user stats in the UI. The stats to be displayed are created in a 2D string array storing the statistic and then actual value. There is then a for loop for each row in the grid (even rows go on the left, with odd ones on the right). This performs integer division so there will be no double values, hence when i = 0 or 1, the y will be 310. Text objects with the statistic name are created and positioned, then text objects with the statistic value are created and positioned adjacent to the statistic name. This is added to the group and repeated for all the rows.

I used a 2D array since each row needs two pieces of data (the stat name and the statistic value), the array will keep these paired together for easy access. This also avoid repeating code when generating the interface, instead of manually creating numerous text objects for each statistic, the for loop will do it. This makes it easier to manage, update and for the future, easier to upgrade; if I want to add more stats to the statistics page, I will not need to create new text objects, I will only need to add it to the array. Furthermore, the array also keeps everything in one place, hence it's a more organised and efficient way to handle the data making it easier to notice mistakes.

3.4.4.2.3.2 Achievements Page



Main Menu Screen Image 5 - Achievement Page

This content is populated via the buildAchievementsContent() procedure:

```java
// Builds content where the user can see what achievements they have
public void buildAchievementsContent(Group contentLayer) throws IOException {  1 usage
    UserStats uStats = stats.loadUserStats(ac.getEmail());
    // 2D Array with Achievements and descriptions
    String[][] achievements = {
            {"Sharp Mind", "Get 5 answers correct in a row"},
            {"Perfect Run", "Get 10 answers correct in a row"},
            {"Flawless", "Complete a session with 100% accuracy"},
            {"Getting Started", "Answer 10 flashcards"},
            {"Knowledge Expert", "Answer 100 flashcards correctly"},
            {"100 Club", "Answer 100 questions total"},
            {"Revision Rookie", "Complete 5 sessions"},
            {"Revision Master", "Complete 50 sessions"},
            {"Study Streak", "Study for 3 days in a row"},
            {"Dedicated Learner", "Study for 7 days in a row"},
            {"Revision Veteran", "Study for a total of 30 days"},
            {"Keep Trying", "Get 5 answers correct in a row"},
            {"Never Give Up", "Retry a session"},
            {"Ultimate Scholar", "Unlock all achievements"},
    };
    Group grid = new Group();
    int leftX = 450;
    int rightX = 1000;
    int startY = 250;
    int rowH = 105;
    // build the Text
    for(int i = 0; i < achievements.length; i++){
        String achievement = achievements[i][0];
        String description = achievements[i][1];
        boolean unlocked = uStats.isAchievementUnlocked(achievement);
        int x = (i % 2 == 0) ? leftX : rightX;
        int y = startY + (i/2) * rowH;

        // Add correct emoji depending on
        Text achievementText = new Text((unlocked ? "🏆  " : "🔒  ") + achievement);
        achievementText.getStyleClass().add("statText");
        ui.setPosition(achievementText, x, y);

        Text descriptionText = new Text(description);
        descriptionText.getStyleClass().add("descriptionText");
        ui.setPosition(descriptionText,  x: x + 40,  y: y + 30);
        // Set the colour to locked achievements differently
        if(!unlocked){
            achievementText.setStyle("-fx-fill: #888888;");
            descriptionText.setStyle("-fx-fill: #888888;");
        }
        grid.getChildren().addAll(achievementText, descriptionText);
    }
    nav.switchContent(contentLayer, grid, getBackView(contentLayer));
}
```

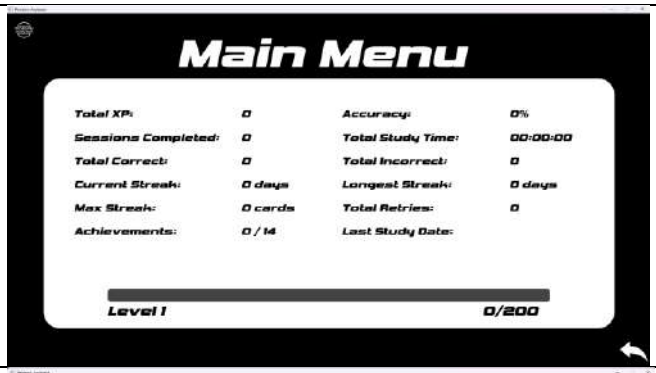
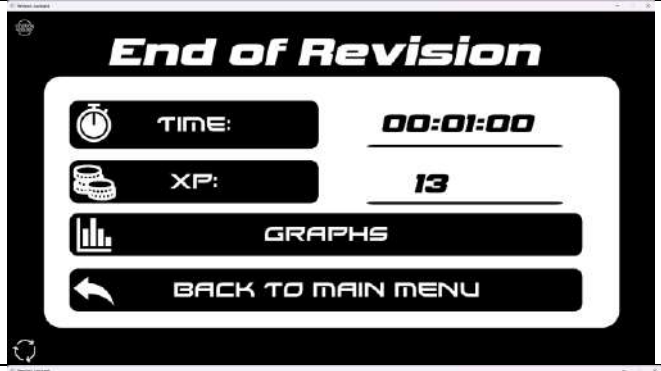
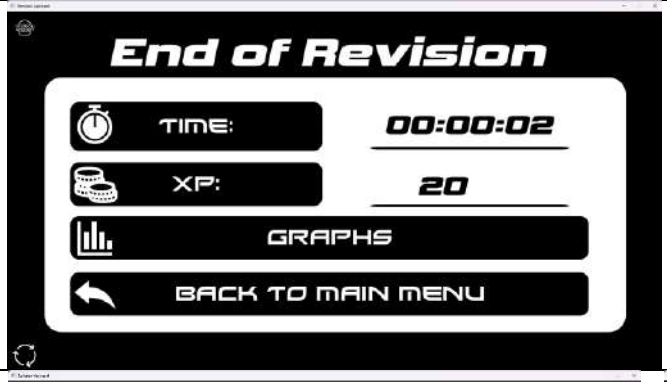
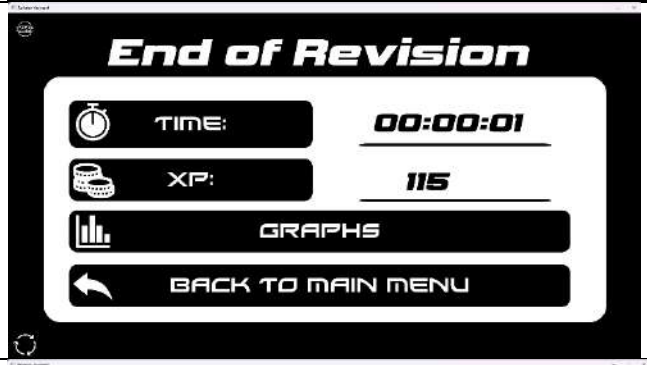
Main Menu Screen Code 7 – buildAchievementsContent

The same iterative method used in the statistics page is used here to populate the group with text objects. If the achievement is locked, there will be a locked emoji and the text will be grey whereas, if it is unlocked there will be a trophy emoji and the text will be darker. This will show the user that they are yet to achieve these achievements.

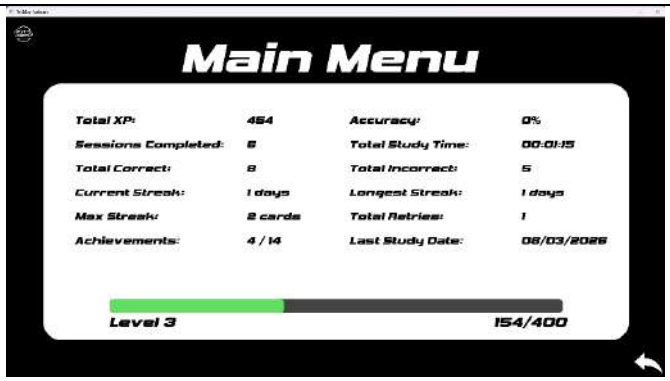
Furthermore, using the 2D array makes the achievements page more expandable for the future since I will not have to continue manually create text objects for each achievement. Hence when I add more achievements in future revisions, I will only need to update the 2D array and the initial positionings.

3.4.4.2.3.3 Testing Statistics, Achievement and XP

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	xp	0	Normal	Accepted, new users start at 0.	New users begin with 0.	
2.	xp	Study for a minute, end session with 1 incorrect	Normal	+13 xp, shown to user	13 xp shown to user	
3.	xp	Get 1 answer right, 1 wrong	Normal	+20 after getting 1 correct and 1 incorrect, shown to user	20 xp shown to user	
4.	Study Session	End the session with 1 correct	Normal	+115xp due to 100% accuracy and Flawless achievement	Flawless received as well as XP	
5.	Study Session	Retry a session	Normal	Keep Trying achievement unlocked	Achievement unlocked, but wrong text	
6.	Study Session	Review at least 10 flashcards	Normal	Getting Started unlocked	Achievement unlocked.	
7.	Study Session	Last Study Date	Normal	Last Study Date set to the date of today	Last Study Date updated to today's date	
8.	Study Session	Get 10 flashcards right in a row	Normal	Perfect Run achievement unlocked	Achievement unlocked.	

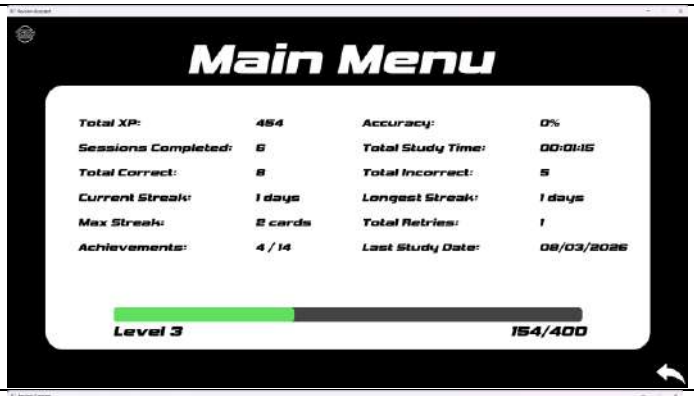
Test No.	Proof
1.	
2.	
3.	
4.	 
5.	

6.





7.



8.



3.4.4.2.3.3.1 Remedial Actions

When testing I noticed the text descriptions of some of the Achievements were wrong so to remediate this I will edit the 2D array where their data is stored.

```
String[][] achievements = {
        {"Sharp Mind", "Get 5 answers correct in a row"},
        {"Perfect Run", "Get 10 answers correct in a row"},
        {"Flawless", "Complete a session with 100% accuracy"},
        {"Getting Started", "Answer 10 flashcards"},
        {"Knowledge Expert", "Answer 100 flashcards correctly"},
        {"100 Club", "Answer 100 questions total"},
        {"Revision Rookie", "Complete 5 sessions"},
        {"Revision Master", "Complete 50 sessions"},
        {"Study Streak", "Study for 3 days in a row"},
        {"Dedicated Learner", "Study for 7 days in a row"},
        {"Revision Veteran", "Study for a total of 30 days"},
        {"Keep Trying", "Retry a session"},
        {"Never Give Up", "Retry 10 sessions"},
        {"Ultimate Scholar", "Unlock all achievements"},
};
```

Main Menu Screen Code 8 - remediated 2D array storing the achievements



Main Menu Screen Image 6 - Achievement Page

The “Keep Trying” and “Never Give Up” achievements now have accurate descriptions.

Another issue I noticed during the tests with the XP system was when the user retries a session, the XP earned from the previous attempt is not cleared, so the XP from both the original session and the retried session are added together, giving the user more XP than intended.

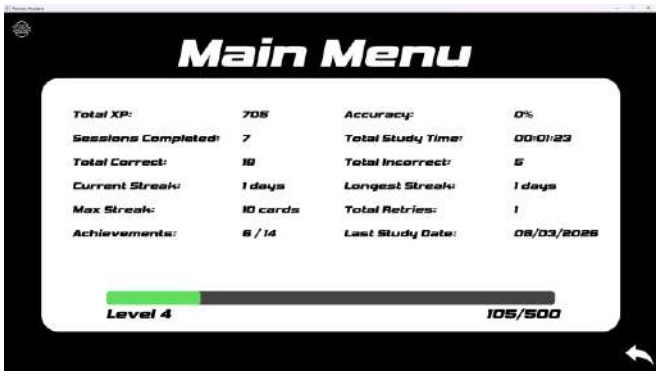
To remediate this issue, inside the EndOfRevisionScreen, when retrying, the xp gained from the session is subtracted from the total xp and then saved to csv. When the user completes the retry session, the xp will then be saved and added to the csv.

```
retry.setOnMouseClicked( MouseEvent _ -> {
    session.retrySession();
    // If retrying, the xp gained is subtracted and saved to csv
    // It will be resaved on the end of the session completely
    userStats.setTotalXP(userStats.getTotalXP() - xpGained);
    try {
        stats.saveUserStats(userStats);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    StudyModesScreen retryModes = null;
    try {
        retryModes = new StudyModesScreen(ac, sc, fc, session, stats);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    ScreenManager.switchScene(retryModes.continueSession());
});
```

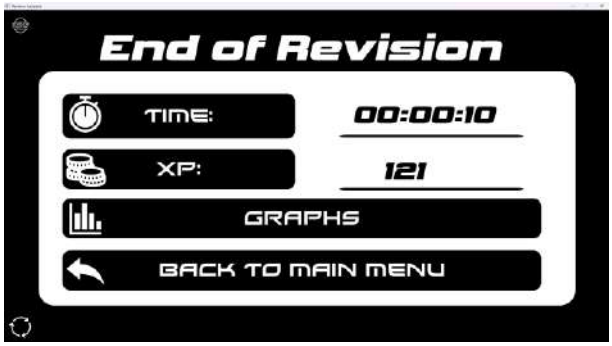
EndOfRevisionScreen Code 5 - Remedial Action

To test this, I will record my initial XP, start a session and complete a session, record the xp gained from this session, then retry the session and record the total xp gained.

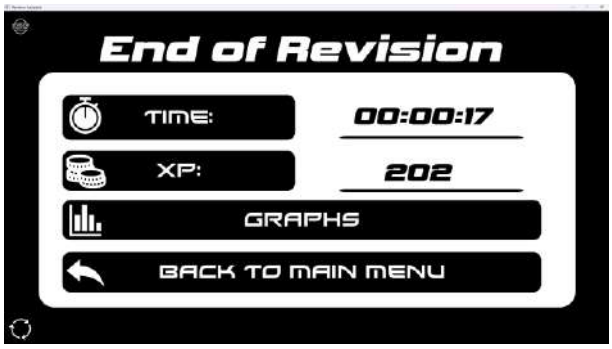
The total xp shown in the statistics page should be equal to the initial xp + the total xp gained.



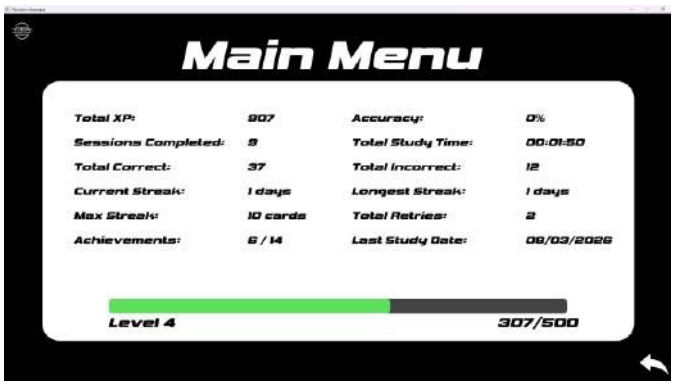
Main Menu Screen Image 7 - Initial XP



EndOfRevisionScreen Image 2 - XP gained after first session



EndOfRevisionScreen Image 3 - total XP gained after retrying



Main Menu Screen Image 8 - total XP after

The test shows I started off with 705, completed a session where I gained 121, then retried the session to gain a total of 202 throughout both sessions, so my total xp after should be 905, which it was. This shows the remedial action was successful.

3.4.4.3 Analytics

Analytics have been within the Revision Assistant to analyse the performance of users during sessions and/or across multiple sessions. The system will use session stored data to display visuals of the user's progress within the application over time. This will users to identify their areas of improvement and see whether they have been improving over time.

3.4.4.3.1 End of Revision Screen Graphs

At the end of a revision session, the user has an option to view visual analytical graphs to provide immediate feedback on the previous session. The graphs have been designed so they are simple to interpret and allow users to analyse how their session went.

In this part of the program, I chose 3 graphs to represent the user's performance:

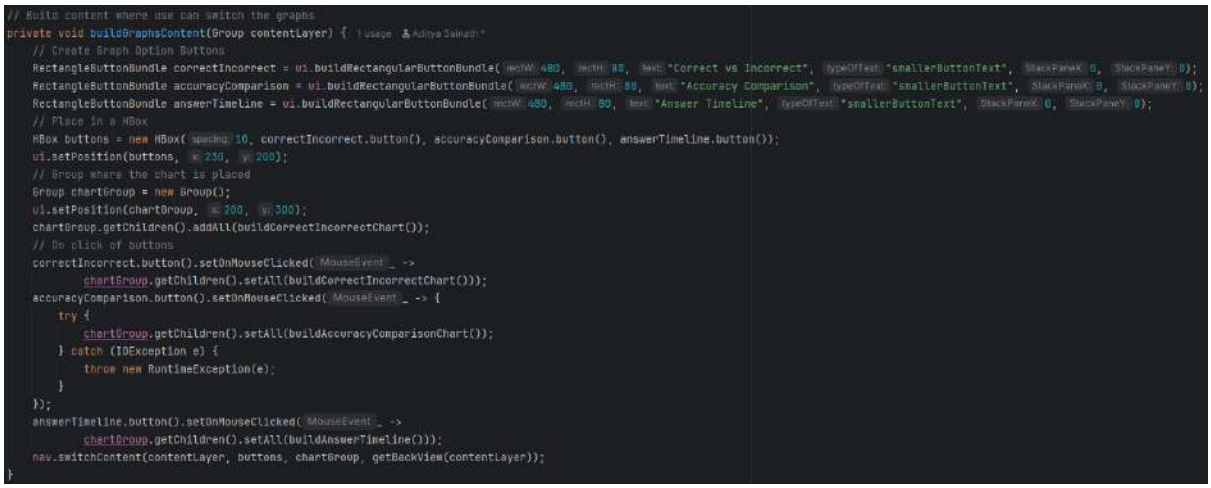
1. Correct vs Incorrect Bar Chart.
2. Accuracy Comparison Against Last Sessions with this Deck.
3. Answer Timeline Visualisation.

The correct vs incorrect bar chart allows the user to instantly compare the quantity of their correct answers against mistakes; this gives a clear performance of their overall performance within the previous session. I chose to use a bar chart since it is effective when comparing two discrete variables, the difference between the height of the bars will make the performance of the user in the previous session clear. An alternative could have been a pie chart, which shows the proportion of correct answers vs incorrect answers, but this would just have been a repeat of accuracy, and it is also harder for users to compare exact quantities visually. Furthermore, if the session did not have many flashcards the pie chart's data could also be misleading since the sample size for the proportion is too small. Another alternative was to display the values as text, but this was rejected since visualisations are easier to interpret for the user.

The accuracy comparison was also visualised with a bar chart. This visualisation allows the user to observe short-term performance trends (with accuracy against their last 3 sessions), whilst allowing users to determine whether their accuracy is improving or declining. Initially, I implemented this visualisation to be a comparison of accuracy against any of the last 3 sessions, which I then realised was not a good indicator of user progress, since some decks may be harder than others. Therefore, I have filtered the bar chart to display accuracy from studying this deck only which would be a better comparative point for the user. Although, if the user has not studied the deck before it

would not be a useful graph since the chart would only be populated with the current session's accuracy, this is still a better graphic to display to users to show their progress. Alternatively, I could have displayed this data within a line graph which is used to display trends over time, but I decided against implementing this since the data was only across 4 sessions all together, with such a small sample size a line graph would not provide any more additional clarity or an insight into data than a bar chart. Another alternative would have been to use numerically list accuracy values across the last sessions, but this would force the user to mathematically compare the values, whereas a chart provides an immediate visual comparison.

The answer timeline displays each flashcard answered during the session as a coloured block (following the western colour connotations of green to represent correct and red representing incorrect). The boxes are arranged in the order the flashcards were answered in, this allows users to track how their performance changed throughout the session. For example, a series of red boxes would indicate the user struggles with a certain part of the deck, this could be used to allow them to focus on the those flashcards alone, however this would only be possible in wrong answers mode, or serial mode since shuffle mode would shuffle the deck not allowing the user to find the series of flashcards they struggled on. An alternative would be to display a table showing whether each flashcard is correct or incorrect, which would have been helpful to allow the user to locate their area of improvement within a deck (but that is what the wrong answers mode is for), this would also require the user to read through rows of text, increasing the interpretation time. I could have also displayed this data with a line chart, showing how the accuracy changes with each card answered but this approach would require calculating an accuracy value after each card, increasing the complexity as well as the complexity of the visualisation.



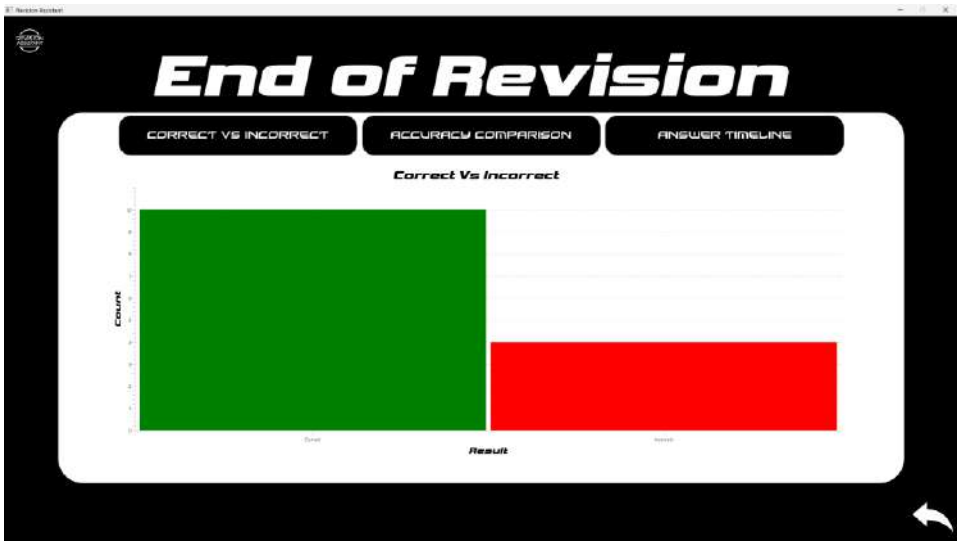
```
// Build content where use can switch the graphs
private void buildGraphsContent(Group contentLayer) {  1 usage   Aditya Sainath *
    // Create Graph Option Buttons
    RectangleButtonBundle correctIncorrect = ui.buildRectangularButtonBundle( rectW: 480,  rectH: 80,  text: "Correct vs Incorrect",  typeOfText: "smallerButtonText",  StackPaneX: 0,  StackPaneY: 0);
    RectangleButtonBundle accuracyComparison = ui.buildRectangularButtonBundle( rectW: 480,  rectH: 80,  text: "Accuracy Comparison",  typeOfText: "smallerButtonText",  StackPaneX: 0,  StackPaneY: 0);
    RectangleButtonBundle answerTimeline = ui.buildRectangularButtonBundle( rectW: 480,  rectH: 80,  text: "Answer Timeline",  typeOfText: "smallerButtonText",  StackPaneX: 0,  StackPaneY: 0);
    // Place in a HBox
    HBox buttons = new HBox( spacing: 10, correctIncorrect.button(), accuracyComparison.button(), answerTimeline.button());
    ui.setPosition(buttons,  x: 230,  y: 200);
    // Group where the chart is placed
    Group chartGroup = new Group();
    ui.setPosition(chartGroup,  x: 200,  y: 300);
    chartGroup.getChildren().addAll(buildCorrectIncorrectChart());
    // On click of buttons
    correctIncorrect.button().setOnMouseClicked( MouseEvent _ ->
            chartGroup.getChildren().setAll(buildCorrectIncorrectChart()));
    accuracyComparison.button().setOnMouseClicked( MouseEvent _ -> {
        try {
            chartGroup.getChildren().setAll(buildAccuracyComparisonChart());
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });
    answerTimeline.button().setOnMouseClicked( MouseEvent _ ->
            chartGroup.getChildren().setAll(buildAnswerTimeline()));
    nav.switchContent(contentLayer, buttons, chartGroup, getBackView(contentLayer));
}
```

EndOfRevisionScreen Code 6 - buildGraphsContent

On the EndOfRevisionScreen when the user clicks the graphs button, the buildGraphsContent procedure is called which changes the content of the page to

display the graph with options to switch, the default graph is the incorrect vs correct bar chart.

3.4.4.3.1.1 Correct vs Incorrect Chart



EndOfRevisionScreen Image 4 - Initial graph on Graphs button click

The bar chart is built with the buildCorrectIncorrectChart function which returns a bar chart which is set to the chartGroup to be displayed.

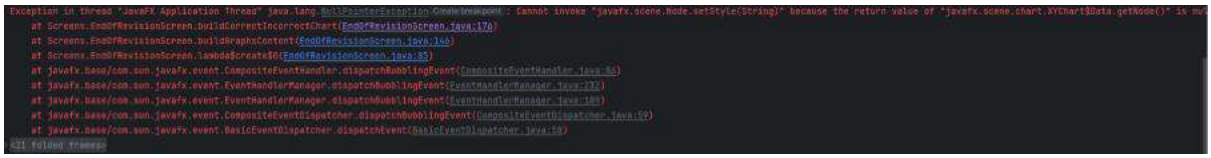
```
// Default chart -> correct vs incorrect bar chart
private @NotNull BarChart<String, Number> buildCorrectIncorrectChart(){ 2 usages  Aditya Sainath
    // Create the axis and label them
    CategoryAxis xAxis = new CategoryAxis();
    int max = Math.max(session.getCorrectCount(), session.getIncorrectCount());
    NumberAxis yAxis = new NumberAxis( lowerBound: 0,  upperBound: max + 1,  tickUnit: 1);
    yAxis.setTickUnit(1.0);
    yAxis.setTickLabelGap(1.0);
    xAxis.setLabel("Result");
    yAxis.setLabel("Count");

    // Create the actual chart
    BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
    chart.getStyleClass().add("charts");
    chart.setTitle("Correct Vs Incorrect");
    chart.setPrefSize( prefWidth: 1500,  prefHeight: 600);
    chart.setLegendVisible(false);

    // Create data series to store the results
    XYChart.Series<String, Number> series = new XYChart.Series<>();
    XYChart.Data<String, Number> correct = new XYChart.Data<>( xValue: "Correct", session.getCorrectCount());
    XYChart.Data<String, Number> incorrect = new XYChart.Data<>( xValue: "Incorrect", session.getIncorrectCount());
    series.getData().addAll(List.of(correct, incorrect));
    // Check if in scene then assign colours, otherwise node is null
    chart.sceneProperty().addListener(( ObservableValue<extends Scene> _,  Scene _,  Scene newValue) -> {
        if (newValue != null) {
            correct.getNode().setStyle("-fx-background-color: green;");
            incorrect.getNode().setStyle("-fx-background-color: red;");
        }
    });
    // Add series to chart
    chart.getData().add(series);
    return chart;
}
```

EndOfRevisionScreen Code 7 - buildCorrectIncorrectChart()

This creates the string category x axis labelled “Result” and the number y axis labelled the “Count”, the chart is then created with these axes, and the legend is set to false. A series is created and the data is populated with the session object’s correct and incorrect count.



Console Output 8 - styling error

Initially to style the bars within the chart, I had just got the node from the XYChart.Data object and then set the style to green and red respectively, however it gave me this error. This was because if node is not within a scene, it is returned to be null, so I am styling a null object. To fix this I added a listener to the chart’s scene property to only do this when the scene is not null.

3.4.4.3.1.2 Answer Timeline

To implement the answer timeline, a new attribute was added to the Session class. ArrayList<Boolean> answerHistory, which stores the answer history of a user throughout a study mode.

This was updated within the Session class’s recordAnswer function:

```
// Returns true if session is complete
public boolean recordAnswer(boolean correct){  1 usage   Aditya Sainath
    answerHistory.add(correct);
    // Remove flashcard from active queue
    Flashcard current = currentQueue.removeFirst();
    // add it to correct list if correct
    if(correct) correctList.add(current);
    else{
        // Add to incorrect list, and place at end of queue again
        incorrectList.add(current);
        currentQueue.add(current);
    }
    // Set to question side for next flashcard
    showingAnswer = false;
    // Return true if the session is complete
    return currentQueue.isEmpty();
}
```

Session Code 4 - updated recordAnswer

If the answer is correct, it adds a true otherwise, false.

The getter for this ArrayList was called within the buildAnswerTimeline function which returns a group of coloured rectangle objects to represent the correct and incorrect answers. The total width the blocks can take, the size of the block , the starting x value and the gap between blocks is stored as variables and with these variables the number of blocks per row is calculated via total width divided by (block size added with gap).

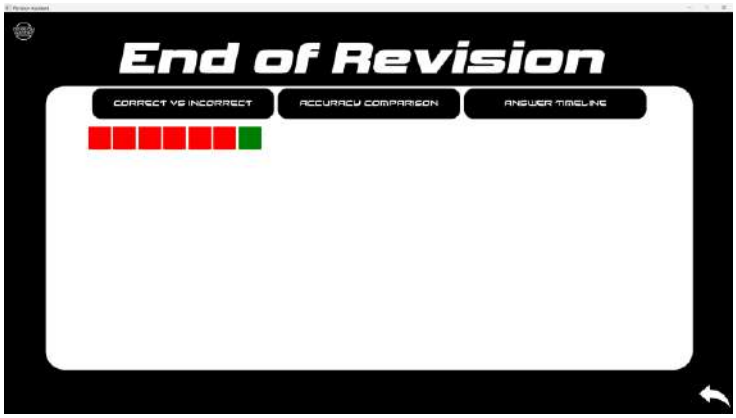
The answerHistory ArrayList is then iterated through with a for loop, its column, row, x position and y position is determined, and a new rectangle object is created. If the element at the index is true, then the colour is set to green, if not the colour is set to red.

```
// Chart 3 -> answer timeline -> visual representation of using coloured blocks
private @NotNull Group buildAnswerTimeline(){ 1 usage  Aditya Sainath
    Group group = new Group();
    ArrayList<Boolean> history = session.getSession().getAnswerHistory();
    if(history.isEmpty()) return group;

    // Layout settings for grids
    int totalW = 1500; // total width
    int blockS = 60; // Size of each block
    int startX = 20; // Starting x position
    int startY = 0; // starting y position
    int gap = 6; // Space between blocks
    int blocksPerRow = totalW / (blockS + gap); // How many blocks can fit in a row
    // loop through each answer
    for(int i = 0; i < history.size(); i++){
        // Determine column and row of current block
        int column = i%blocksPerRow;
        int row = i/blocksPerRow;
        // Calculate the screen position
        int x = startX + column*(blockS + gap);
        int y = startY + row*(blockS + gap);
        Rectangle block = new Rectangle(x, y, blockS, blockS);
        block.getStyleClass().add("block");
        // Colour changes based on true or false
        block.setStyle("-fx-fill: " + (history.get(i) ? "green" : "red") + ";");
        group.getChildren().add(block);
    }
    return group;
}
```

EndOfRevisionScreen Code 8 - buildAnswerTimeline()

On the click of the “Answer Timeline” button, the group which this function returns is set to the chartGroup, removing the previous visual and replacing it with the new one.



EndOfRevisionScreen Image 5 - Answer Timeline

3.4.4.3.1.3 Accuracy Comparison Chart

To be able to do an accuracy comparison with previous sessions with a deck, or with previous sessions at all, I realised session data for a user must be stored after every session. To ensure session data retrieval is efficient, I had to make sure each user has a .csv file for themselves, so I would not have to conduct multiple linear searches within the program looking for session records for a certain user. To do this, I created a directory within the "files" directory (which has been marked as a resource root, storing stats.csv, settings.csv and users.csv) named sessions. This will store the user's session data, adding it to the file after each session. Hence firstly, I created a SessionRecord record class, which stores all relevant session data to be saved to a file.

```
/**
 * @param date Date of Session to be used for streaks
 * @param accuracy The accuracy, used for graph data
 * @param studySeconds Elapsed time of a session
 * @param correct  Number of correct answers
 * @param incorrect Number of incorrect answers
 * @param deckName The deck name of the session
 * @param answerHistory correct = t, incorrect = f
 */
public record SessionRecord(String date, double accuracy, long studySeconds, int correct, int incorrect,
                            String deckName, ArrayList<Boolean> answerHistory) {  4 usages
}
```

SessionRecord Code 1 - Whole record class

Within StatsController the following code was added:

```
// Function to return the path where individual Session data is stored for user analytics
private Path sessionPath(String email){  2 usages   Aditya Sainath
    return Paths.get(System.getProperty("user.dir"),  ...more: "files", "sessions", email + ".csv");
}
```

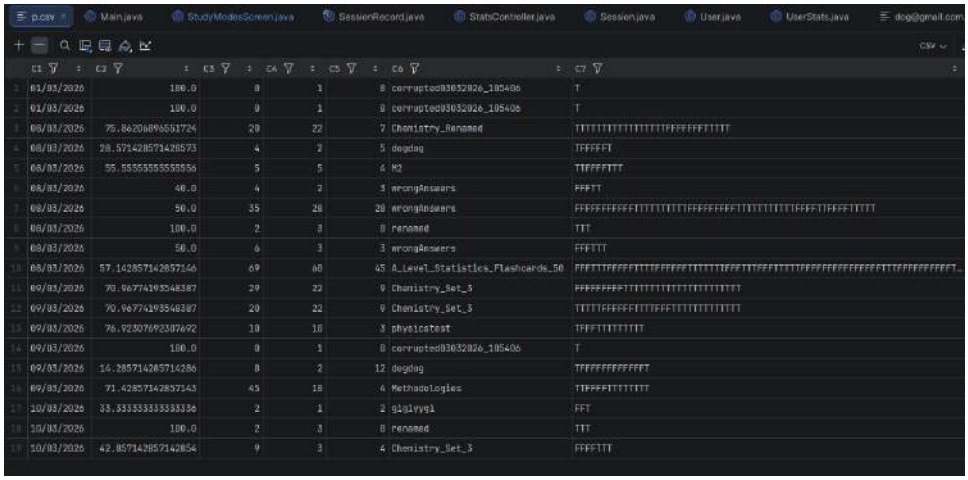
StatsController Code 10 - sessionPath Function

This returns the path of the of file where session data will be written to.

```
// Saving a session to the user's session history CSV
public void saveSessionRecord(String email, SessionRecord sr) throws IOException {  1 usage   Aditya Sainat
    Path sessionPath = sessionPath(email);
    // Create directory and folder if not there already
    Files.createDirectories(sessionPath.getParent());
    if(!Files.exists(sessionPath)) Files.createFile(sessionPath);
    // Join the answer history as Ts and Fs to represent true and false
    StringBuilder answers = new StringBuilder();
    for(boolean b : sr.answerHistory()){answers.append(b ? "T" : "F");}
    // Write it into the csv
    try(BufferedWriter bw = new BufferedWriter(new FileWriter(sessionPath.toFile(),  append: true))) {
        bw.write( str: sr.date() + "," +
                sr.accuracy() + "," +
                sr.studySeconds() + "," +
                sr.correct() + "," +
                sr.incorrect() + "," +
                sr.deckName() + "," +
                answers);
        bw.newLine();
    }
}
```

StatsController Code 11 - savingSessionRecord()

This will convert the Boolean ArrayList into Ts and Fs, with a for each loop adding to a string builder, then the buffered writer will data in the user specific csv as the date, accuracy, studySeconds, correct, incorrect, deckName and answers.



	c1	c2	c3	c4	c5	c6	c7
1	01/03/2026	100.0	8	1	8	corrupted03032026_185406	T
2	01/03/2026	100.0	8	1	8	corrupted03032026_185406	T
3	08/03/2026	75.86206896551724	20	22	7	Chemistry_Renamed	TTTTTTTTTTTTTTTTTFFFFFFFFTTTT
4	08/03/2026	28.571428571428573	4	2	5	dogdog	TFFFFFT
5	08/03/2026	55.55555555555556	5	5	4	M2	TTFFFFTTT
6	08/03/2026	40.0	4	2	3	wrongAnswers	FFFTT
7	08/03/2026	50.0	35	28	20	wrongAnswers	FFFFFFFFFFFTTTTTTTTTTFFFFFFFFFTTTTTTTTTTTFFFFTTFFFFTTTT
8	08/03/2026	100.0	2	3	0	renamed	TTT
9	08/03/2026	50.0	6	3	3	wrongAnswers	FFFTTT
10	08/03/2026	57.142857142857146	69	60	45	A_Level_Statistics_Flashcards_50	FFFTTTFFFFFTTTTTFFFFFFTTTTTTTTFFFTTTFFFTTTTTFFFFFFFFFFFFFTTTFFFFFFFFFT...
11	09/03/2026	70.96774193548387	29	22	9	Chemistry_Set_3	FFFFFFFFFTTTTTTTTTTTTTTTTTTTT
12	09/03/2026	70.96774193548387	29	22	9	Chemistry_Set_3	TTTTTFFFFFFTTTTFFFTTTTTTTTTTT
13	09/03/2026	76.92307692307692	19	16	3	physicstest	TFFFTTTTTTTT
14	09/03/2026	100.0	8	1	8	corrupted03032026_185406	T
15	09/03/2026	14.285714285714286	8	2	12	dogdog	TFFFFFFFFFFFT
16	09/03/2026	71.42857142857143	45	18	4	Methodologies	TTFFFFTTTTTTT
17	10/03/2026	33.333333333333336	2	1	2	giglyyg1	FFT
18	10/03/2026	100.0	2	3	0	renamed	TTT
19	10/03/2026	42.857142857142854	9	3	4	Chemistry_Set_3	FFFFTTT

User Specific Session Record CSV 1

The session record is written to the user's csv when the user clicks the back button on the end of revision screen, so it is only saved once when the user cannot retry anymore.

```
backToMainMenu.button().setOnMouseClicked( MouseEvent _ -> {
    try {
        // Save session record when user is done with the session
        stats.saveSessionRecord(ac.getEmail(), new SessionRecord(
                LocalDate.now().format(dtf), session.getAccuracy(),
                session.getElapsedTime(), session.getCorrectCount(),
                session.getIncorrectCount(), session.getSelectedDeck(),
                session.getSession().getAnswerHistory()
        ));
        ScreenManager.switchScene(new MainMenuScreen(ac, sc).create(), ac.getEmail());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    ScreenManager.clearHistory();
});
```

EndOfRevisionScreen Code 9 - writing into user specific csv

After saving, the screen is switched to the main menu screen, and the stack history is cleared so the user can not go back to the end of revision screen.

When the data is being read, it will be converted into an ArrayList of SessionRecord class objects, this is done using the same methods as before, reading each line, splitting it at the "," and then adding to the ArrayList as a new object. The answer history array list is also converted back to Booleans when being read as a string of Ts and Fs.

```java
// Load all session records for a user from their session csv
public ArrayList<SessionRecord> loadSessionRecords(String email) throws IOException {  2 usages  Aditya Sainath
    Path sessionPath = sessionPath(email);
    ArrayList<SessionRecord> historicalSessions = new ArrayList<>();
    if(!Files.exists(sessionPath)) return historicalSessions; // no sessions yet
    // Read data line by line
    try(BufferedReader br = new BufferedReader(new FileReader(sessionPath.toFile()))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] data = line.split( regex: ",");
            // Not enough data in a row
            if(data.length < 7) throw new IllegalArgumentException("Corrupted Row: " + line);
            ArrayList<Boolean> answerHistory = new ArrayList<>();
            // Reconvert Ts and Fs into trues and falses
            for(char c : data[6].toCharArray()){
                answerHistory.add(c == 'T');
            }
            // Add new SessionRecord object to ArrayList
            historicalSessions.add(new SessionRecord(data[0], Double.parseDouble(data[1]),
                    Long.parseLong(data[2]), Integer.parseInt(data[3]), Integer.parseInt(data[4]),
                    data[5], answerHistory));
        }
    } catch (Exception e) {
        quarantineCorruptedCSV(e, sessionPath);
        return new ArrayList<>();
    }
    return historicalSessions;
}
```

StatsController Code 12 – loadSessionRecords()

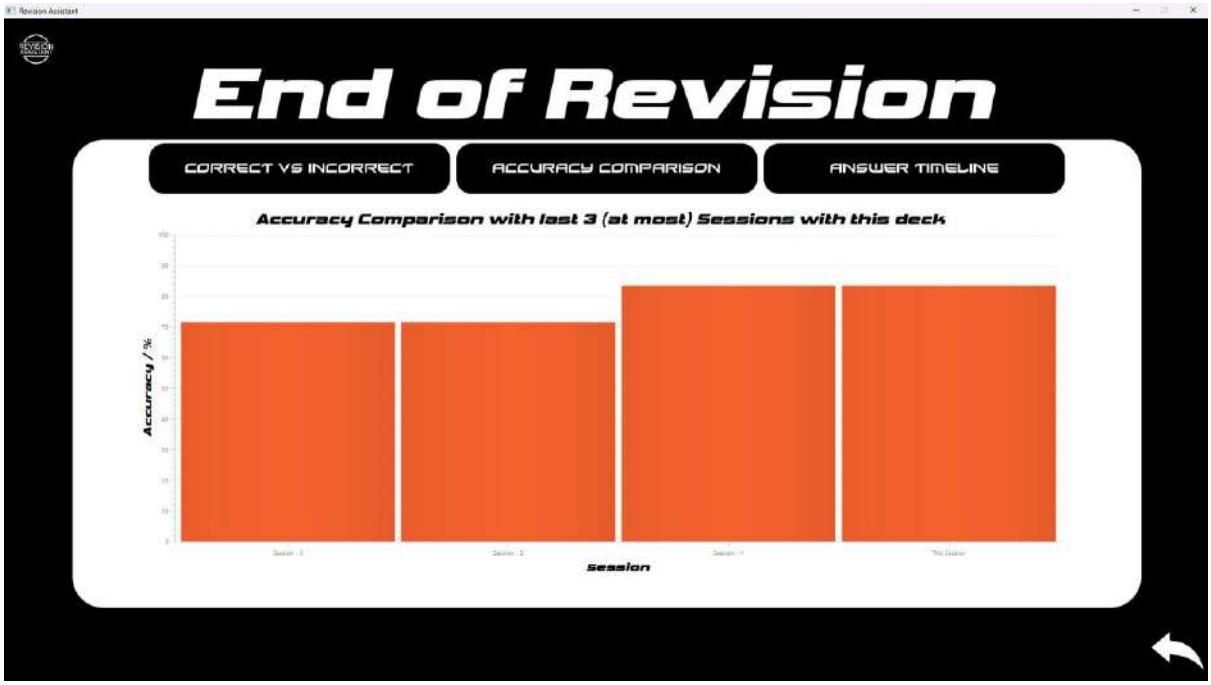
Now to create the accuracy comparison bar chart, the buildAccuracyComparisonChart() function is called.

```java
// Chart 2 -> answer comparison for last three session
private BarChart<String, Number> buildAccuracyComparisonChart() throws IOException {  1 usage
    // Create the axis
    CategoryAxis xAxis = new CategoryAxis();
    NumberAxis yAxis = new NumberAxis( lowerBound: 0,  upperBound: 100,  tickUnit: 10);
    xAxis.setLabel("Session");
    yAxis.setLabel("Accuracy / %");
    // Create the chart
    BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
    chart.getStyleClass().add("charts");
    chart.setTitle("Accuracy Comparison with last 3 (at most) Sessions with this deck");
    chart.setPrefSize( prefWidth: 1500,  prefHeight: 600);
    chart.setLegendVisible(false);
    // Create the series and populate it with data
    XYChart.Series<String, Number> series = new XYChart.Series<>();
    ArrayList<SessionRecord> sessionHistory = stats.loadSessionRecords(ac.getEmail());
    // Create a list for only this deck
    ArrayList<SessionRecord> deckSessionHistory = new ArrayList<>();
    for(SessionRecord s : sessionHistory){
        // if deck name matches, add to list
        if(s.deckName().equals(session.getSelectedDeck())){
            deckSessionHistory.add(s);
        }
    }
    // Get last three sessions
    int size = deckSessionHistory.size();
    int start = Math.max(0, size - 3);
    for(int i = start; i < size; i++) {
        // Add each accuracy to the chart
        series.getData().add(new XYChart.Data<>( xValue: "Session - " + (size - i),
                deckSessionHistory.get(i).accuracy()));
    }
    // Add this session's accuracy
    series.getData().add(new XYChart.Data<>( xValue: "This Session", session.getAccuracy()));

    chart.getData().add(series);
    return chart;
}
```

EndOfRevisionScreen Code 10 - buildAccuracyComparisonChart()

The axes, chart and series are created, then all the user's sessions history is loaded and a new ArrayList is initiated storing the SessionRecords which have the same deck name as the current deck. The session history is then iterated through and the deckSessionHistory is added too; for the last 3 (at maximum) session records, their accuracy is added as a bar to the bar chart.



EndOfRevisionScreen Image 6 - Accuracy Comparison Chart

3.4.4.3.2 Analytics Page

Following the same methods, I also added four charts to the analytics page in the revision system screen, which unlike of the end of revision screen, display all time statistics to the user. The four graphs I chose to display were:

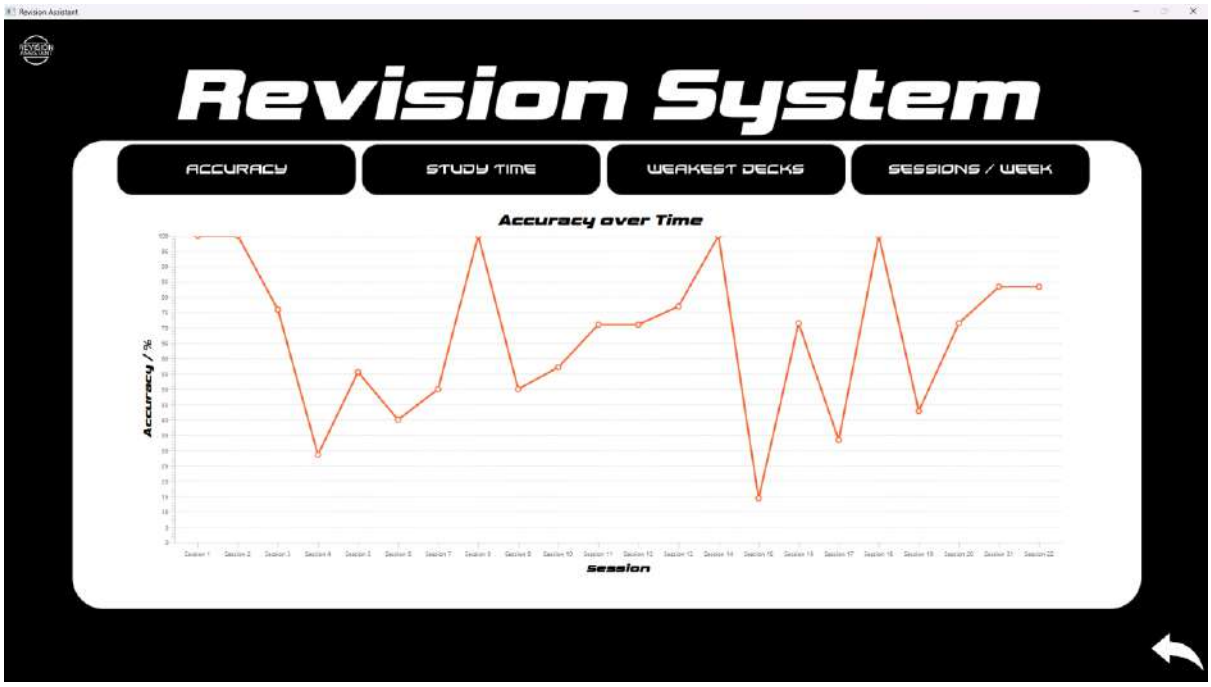
1. All time accuracy - line graph
2. All time elapsed study time - line graph
3. Average deck accuracy – bar chart
4. Sessions per week – bar chart

The all-time accuracy chart displays the user's accuracy across all recorded revision sessions, each point on the graph represents the accuracy achieved in the session. A line chart was chosen since it is effective in displaying trends over time, in future revisions of the system, I plan to add features which allow the user to filter the time frame to see their progress in a period of time, as well as a regression line since this would provide a better insight into the actual progress. An alternative visualisation could have been bar chart to display his data, however since this is all time, the sample size would be bigger hence line graphs would allow the user to easily see if their accuracy is improving or not.

The all-time study time chart displays how the user's elapsed study time changes across sessions, allowing the user to monitor how consistently they are revising and identify times where their elapsed study time increases, or decreases. A line graph allows an easy visualisation of the data allowing the user to interpret easily as well. I could have implemented bar charts where each bar shows the study time in a period (e.g. a month, this could provide a higher level of feedback for the user, it would reduce the detail available to them and potentially hide the variations in detail between individual sessions. In future revisions of the system, I will add filters to this graph too so the user can customise how they would like their data visualised.

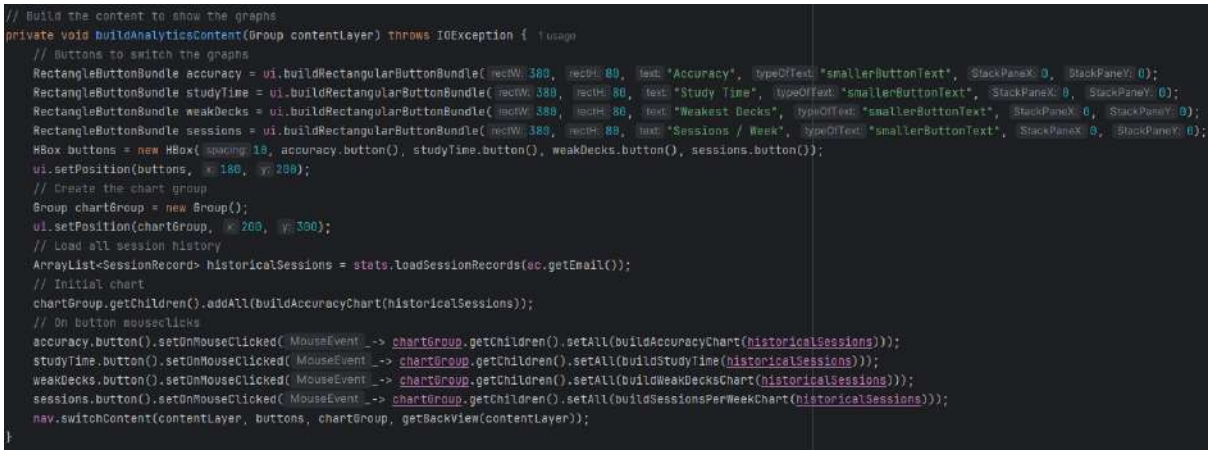
The average deck accuracy displays the average accuracy achieved for each revision deck; the system will group every session by deck name and calculate an average accuracy for the deck before displaying it as a bar on the bar chart. Each deck is distinct from each other, hence, to prevent users from thinking there is correlation between data, a line graph was avoided. Another alternative would have been to display the data within a pie chart, but to make it a useful visual representation there would have to be a pie chart for each deck which is excessive and would not resize as well as a bar chart.

The sessions per week filters weeks by the weeks of the year (via the ISO format, hence weeks begin on Monday), allowing users to track the consistency of their revision and help them to determine if their regular revision schedule is maintained over longer periods. I could have displayed this data in a calendar-style visualisation showing daily activity, this could have been more useful, but it would have increased the complexity of the implementation and deemed unnecessary.



RevisionSystemScreen Image 5 - Analytics Page

Like the end of revision screen, the buttons are placed at the top and are used to switch the graph within the chart group.



```
// Build the content to show the graphs
private void buildAnalyticsContent(Group contentLayer) throws IOException {
    // Buttons to switch the graphs
    RectangleButtonBundle accuracy = ui.buildRectangularButtonBundle( rectW: 380, rectH: 80, text: "Accuracy", typeOfText: "smallerButtonText", StackPaneX: 0, StackPaneY: 0);
    RectangleButtonBundle studyTime = ui.buildRectangularButtonBundle( rectW: 380, rectH: 80, text: "Study Time", typeOfText: "smallerButtonText", StackPaneX: 0, StackPaneY: 0);
    RectangleButtonBundle weakDecks = ui.buildRectangularButtonBundle( rectW: 380, rectH: 80, text: "Weakest Decks", typeOfText: "smallerButtonText", StackPaneX: 0, StackPaneY: 0);
    RectangleButtonBundle sessions = ui.buildRectangularButtonBundle( rectW: 380, rectH: 80, text: "Sessions / Week", typeOfText: "smallerButtonText", StackPaneX: 0, StackPaneY: 0);
    HBox buttons = new HBox( spacing: 10, accuracy.button(), studyTime.button(), weakDecks.button(), sessions.button());
    ui.setPosition(buttons, x: 180, y: 280);
    // Create the chart group
    Group chartGroup = new Group();
    ui.setPosition(chartGroup, x: 200, y: 300);
    // Load all session history
    ArrayList<SessionRecord> historicalSessions = stats.loadSessionRecords(sc.getEmail());
    // Initial chart
    chartGroup.getChildren().addAll(buildAccuracyChart(historicalSessions));
    // On button mouseclicks
    accuracy.button().setOnMouseClicked( MouseEvent _-> chartGroup.getChildren().setAll(buildAccuracyChart(historicalSessions)));
    studyTime.button().setOnMouseClicked( MouseEvent _-> chartGroup.getChildren().setAll(buildStudyTime(historicalSessions)));
    weakDecks.button().setOnMouseClicked( MouseEvent _-> chartGroup.getChildren().setAll(buildWeakDecksChart(historicalSessions)));
    sessions.button().setOnMouseClicked( MouseEvent _-> chartGroup.getChildren().setAll(buildSessionsPerWeekChart(historicalSessions)));
    nav.switchContent(contentLayer, buttons, chartGroup, getBackView(contentLayer));
}
```

RevisionSystemScreen Code 3 - Analytics Content

3.4.4.3.2.1 All-Time Accuracy Line Graph

```
// Accuracy Line Graph
private LineChart<String, Number> buildAccuracyChart(ArrayList<SessionRecord> historicalSessions) {
    // Create the axis
    CategoryAxis xAxis = new CategoryAxis();
    NumberAxis yAxis = new NumberAxis( lowerBound: 0, upperBound: 100.0, tickUnit: 5);
    xAxis.setLabel("Session");
    yAxis.setLabel("Accuracy / %");
    // Create the chart
    LineChart<String, Number> lineGraph = new LineChart<>(xAxis, yAxis);
    lineGraph.getStyleClass().add("charts");
    lineGraph.setTitle("Accuracy over Time");
    lineGraph.setMaxSize( maxWidth: 1500, maxHeight: 600);
    lineGraph.setPrefSize( prefWidth: 1500, prefHeight: 600);
    lineGraph.setLegendVisible(false);
    lineGraph.setCreateSymbols(true); // data points at each plot point
    // Populate the graph
    XYChart.Series<String, Number> series = new XYChart.Series<>();
    int sessionNumber = 1;
    for(SessionRecord s : historicalSessions){
        series.getData().add(
                new XYChart.Data<>( xValue: "Session " + sessionNumber,
                        s.accuracy()));
                sessionNumber++;
    }
    lineGraph.getData().add(series);
    return lineGraph;
}
```

RevisionSystemScreen Code 4 - buildAccuracyChart()

This function returns a line chart with each session and its accuracy added as a point on the graph.



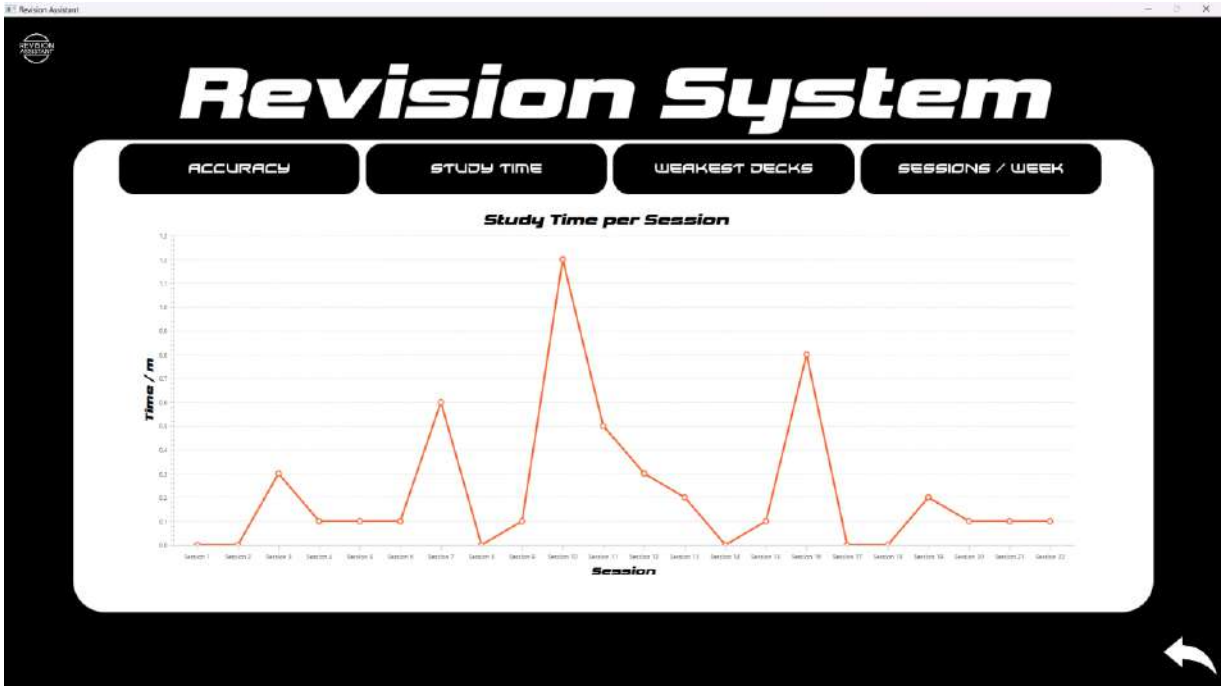
RevisionSystemScreen Image 6 - Accuracy over Time

3.4.4.3.2.2 All-Time Study Time Graph

```
// Study time per session line graph
private LineChart<String, Number> buildStudyTime(ArrayList<SessionRecord> historicalSessions) {
    // Create the axis
    CategoryAxis xAxis = new CategoryAxis();
    NumberAxis yAxis = new NumberAxis();
    xAxis.setLabel("Session");
    yAxis.setLabel("Time / m");
    // Create the chart
    LineChart<String, Number> lineGraph = new LineChart<>(xAxis, yAxis);
    lineGraph.getStyleClass().add("charts");
    lineGraph.setTitle("Study Time per Session");
    lineGraph.setMaxSize( maxWidth: 1500,  maxHeight: 600);
    lineGraph.setPrefSize( prefWidth: 1500,  prefHeight: 600);
    lineGraph.setLegendVisible(false);
    lineGraph.setCreateSymbols(true);
    // Populate the graph
    XYChart.Series<String, Number> series = new XYChart.Series<>();
    int sessionNumber = 1;
    for(SessionRecord s : historicalSessions){
        double minutes = s.studySeconds() / 60.0;
        minutes = Math.round(minutes * 10.0) / 10.0;
        series.getData().add(
                new XYChart.Data<>( xValue: "Session " + sessionNumber,
                        minutes));
        sessionNumber++;
    }
    lineGraph.getData().add(series);
    return lineGraph;
}
```

RevisionSystemScreen Code 5 - buildStudyTime()

This function creates and returns line graph in the same way as previously, with the session time rounded to the nearest .1 minute.



RevisionSystemScreen Image 7 - Study Time per Session

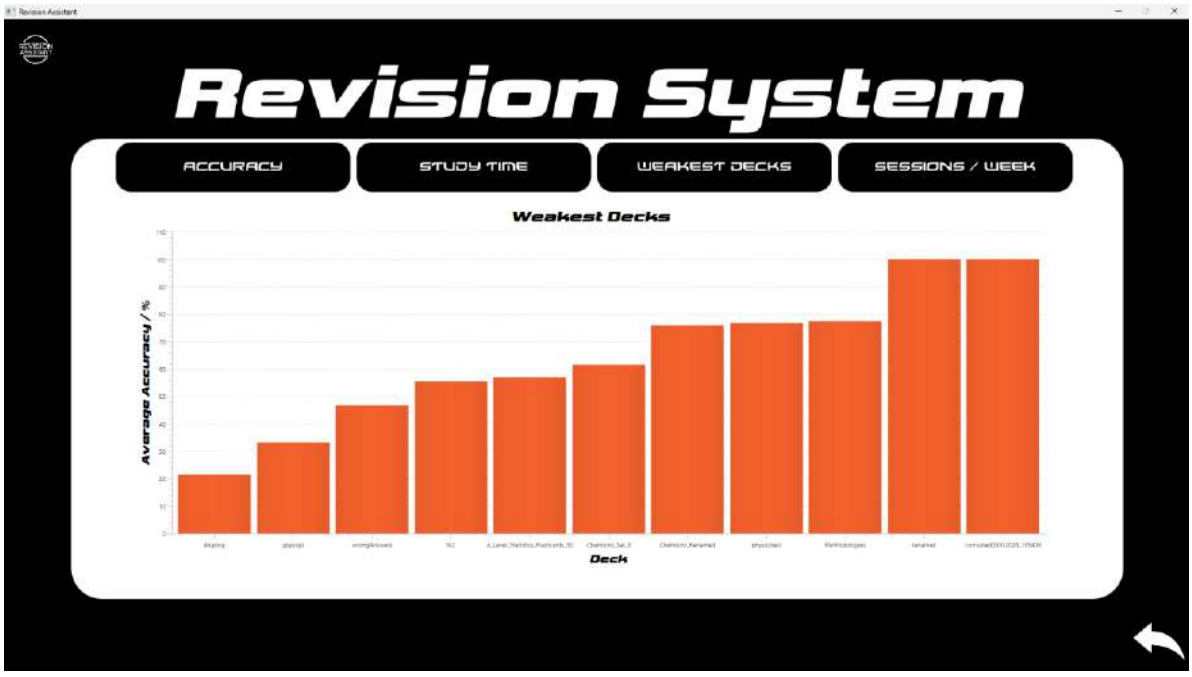
3.4.4.3.2.3 Weak Decks Chart

```
// Create the weak decks bar chart
private BarChart<String, Number> buildWeakDecksChart(ArrayList<SessionRecord> historicalSessions){  1 usage
    // Create the axis
    CategoryAxis xAxis = new CategoryAxis();
    NumberAxis yAxis = new NumberAxis();
    xAxis.setLabel("Deck");
    yAxis.setLabel("Average Accuracy / %");
    // Create the chart
    BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
    chart.getStyleClass().add("charts");
    chart.setTitle("Weakest Decks");
    chart.setMaxSize( maxWidth: 1500,  maxHeight: 600);
    chart.setPrefSize( prefWidth: 1500,  prefHeight: 600);
    chart.setLegendVisible(false);
    // Group sessions by deck and avg accuracy
    HashMap<String, ArrayList<Double>> deckAccuracies = new HashMap<>();
    // Iterate through each session record
    for(SessionRecord s : historicalSessions){
        // If deck name does not exist in the map, create a new ArrayList and key for it
        // Session accuracy added to list
        deckAccuracies.computeIfAbsent(s.deckName(),  String _->
                new ArrayList<>()).add(s.accuracy());
    }
    // Create the series to populate the chart
    XYChart.Series<String, Number> series = new XYChart.Series<>();
    // convert each map entry from deck name and list of accuracies to deck name and average accuracies
    deckAccuracies.entrySet().stream().map(
            // Calculate the avg of the accuracy values in the list
            Entry<String, ArrayList<...>> e ->{
                double average = e.getValue().stream()
                        // convert to primitive doubles instead of double objects
                        // Find average
                        .mapToDouble(Double::doubleValue).average().orElse( other: 0);
                return Map.entry(e.getKey(), average);
                // Sort in ascending order
            }).sorted(Map.Entry.comparingByValue())
            // Add each entry to the series
            .forEach( Entry<String, Double> e -> series.getData().add(new XYChart.Data<>(e.getKey(),  yValue: Math.round(e.getValue() * 10.0) / 10.0)));
    chart.getData().add(series);
    return chart;
}
```

RevisionSystemScreen Code 6 – buildWeakDecksChart()

The bar chart with axes is created and then populated by first creating a HashMap to store the deck name alongside an ArrayList of its accuracies. Hash maps allow data to be stored in key/value pairs and provide very efficient data look-up and insertion operations (time complexity on average being O(1), hence time taken would not increase significantly with more sessions being added). Thus, this improves the scalability of the solution, if the user has studied a lot of decks across a lot of sessions, once the accuracies have been grouped the program can easily iterate through each entry to calculate the average accuracy. An alternative would be to have used an ArrayList of a new record class for storing the deck name and another ArrayList of accuracy values for this deck, while this solution would work in theory, it would require the system to iterate through the array list to locate the deck each time a new session from the user specific's csv is processed. Therefore, when inserting new data, the program would need to conduct a linear search each time and as number of sessions increase the time complexity would increase in direct proportion due to time complexity of O(n).

The historical sessions were iterated through and if the deck does not already exist within the HashMap a new entry is created, if it does exist the accuracy is added to the ArrayList. After this, the series for the chart is created and the HashMap's accuracy ArrayList is converted to a stream and then converted to the primitive double data type so it can be used to calculate the average, the deck with its average accuracy is returned and sorted in ascending order to display weaker decks on the left with stronger decks on the right.



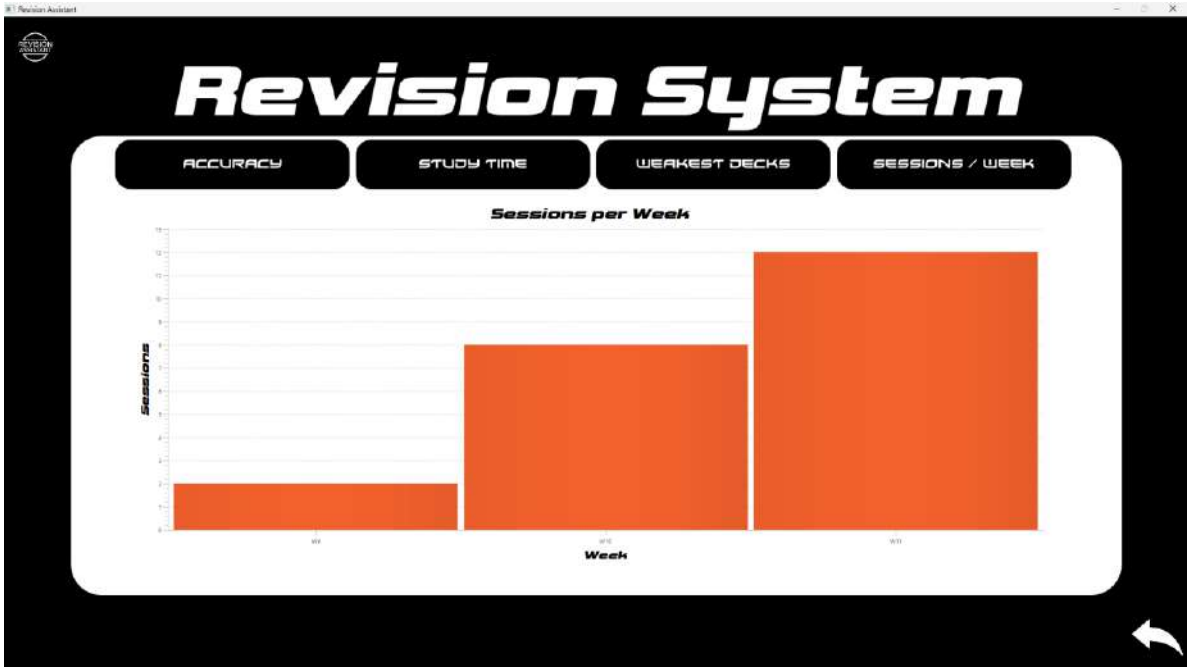
RevisionSystemScreen Image 8 - Weak Decks Chart

3.4.4.3.2.4 Sessions per Week

```
// Sessions per Week -> bar chart
private BarChart<String, Number> buildSessionsPerWeekChart(ArrayList<SessionRecord> historicalSessions){ 1 usage  Aditya Sainath
    // Create the axis
    CategoryAxis xAxis = new CategoryAxis();
    NumberAxis yAxis = new NumberAxis();
    xAxis.setLabel("Week");
    yAxis.setLabel("Sessions");
    // Create the chart
    BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
    chart.getStyleClass().add("charts");
    chart.setLegendVisible(false);
    chart.setTitle("Sessions per Week");
    chart.setMaxSize( maxWidth: 1500,  maxHeight: 600);
    chart.setPrefSize( prefWidth: 1500,  prefHeight: 600);
    // Populate the chart with a hash map
    HashMap<String, Integer> weeklySessions = new HashMap<>();
    for(SessionRecord s : historicalSessions){
        // Split date into day, month and year
        String[] date = s.date().split( regex: "/");
        // Ensure date is valid
        if(date.length == 3){
            // Convert the date into a week number and label it
            String week = "W" + getWeek(date[0], date[1], date[2]);
            // Add session to correct week, if week exists increment, if not create it with value 1
            weeklySessions.merge(week,  value: 1, Integer::sum);
        }
    }
    XYChart.Series<String, Number> series = new XYChart.Series<>();
    // Convert to stream and sort it by week
    weeklySessions.entrySet().stream().sorted(( Entry<String, Integer> a,  Entry<String, Integer> b) -> {
                    int weekA = Integer.parseInt(a.getKey().substring( beginIndex: 1)); // Remove the W and compare
                    int weekB = Integer.parseInt(b.getKey().substring( beginIndex: 1));
                    return Integer.compare(weekA, weekB);
                    })
            // For each week, add the session count to chart
            .forEach( Entry<String, Integer> e -> series.getData().add(new XYChart.Data<>(e.getKey(), e.getValue())));
    chart.getData().add(series);
    return chart;
}
// Function to determine the week number from a date
private int getWeek(String day, String month, String year){ 1 usage  Aditya Sainath
    // Convert into a date object
    LocalDate date = LocalDate.of(Integer.parseInt(year), Integer.parseInt(month), Integer.parseInt(day));
    // Return the week number of the year (ISO, weeks start on Monday)
    return date.get(IsoFields.WEEK_OF_WEEK_BASED_YEAR);
}
```

RevisionSystemScreen Code 7 - buildSessionsPerWeek() and getWeek()

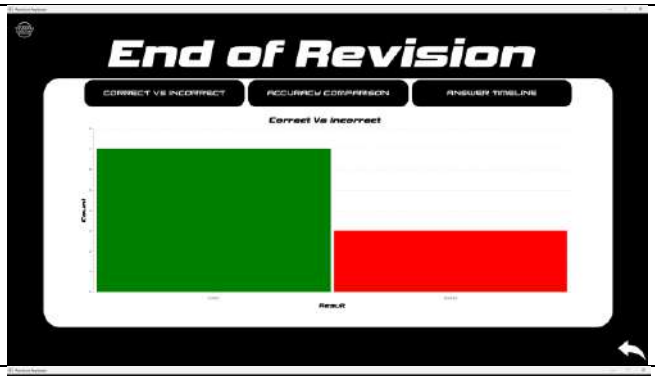
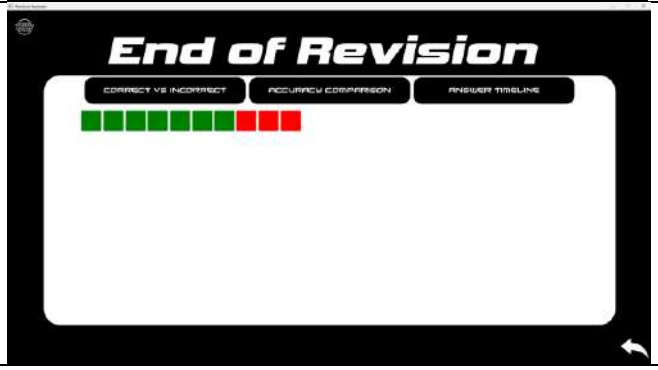
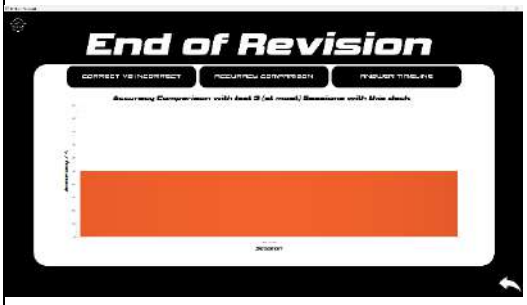
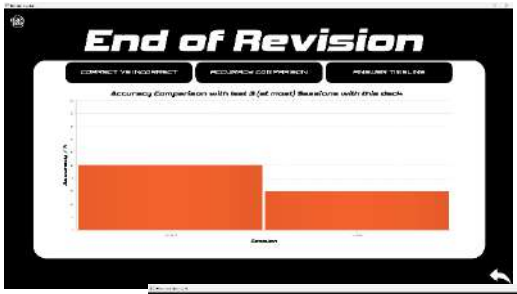
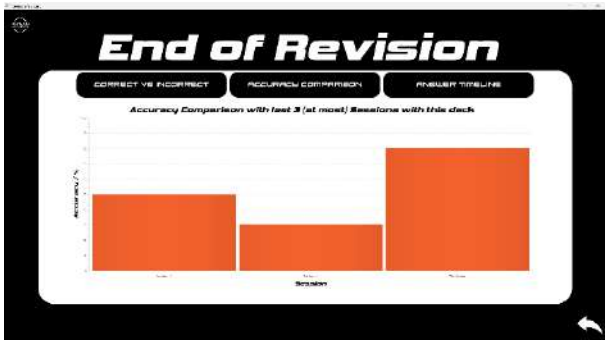
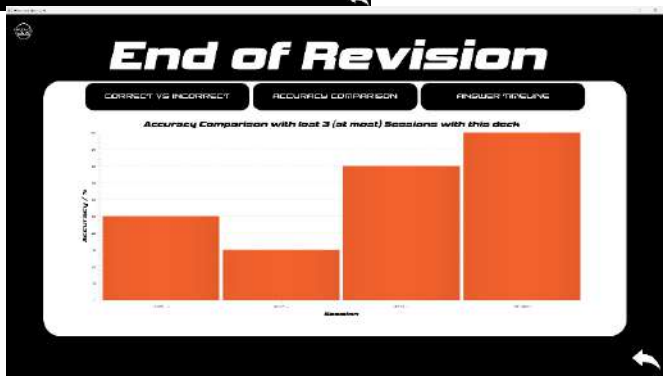
This function also uses a HashMap to group the week number (string), with the number of sessions in this week (int). For each session in the historical sessions, the date is split into the day, month and year and passed into the getWeek function, which calculates the week of the year it is (according to the iso format). The session is them added to the correct week, if the week exists, then increment the counter otherwise create it with the start value of one. The weeks are then ordered in chronological order to be displayed as bars in the bar chart.



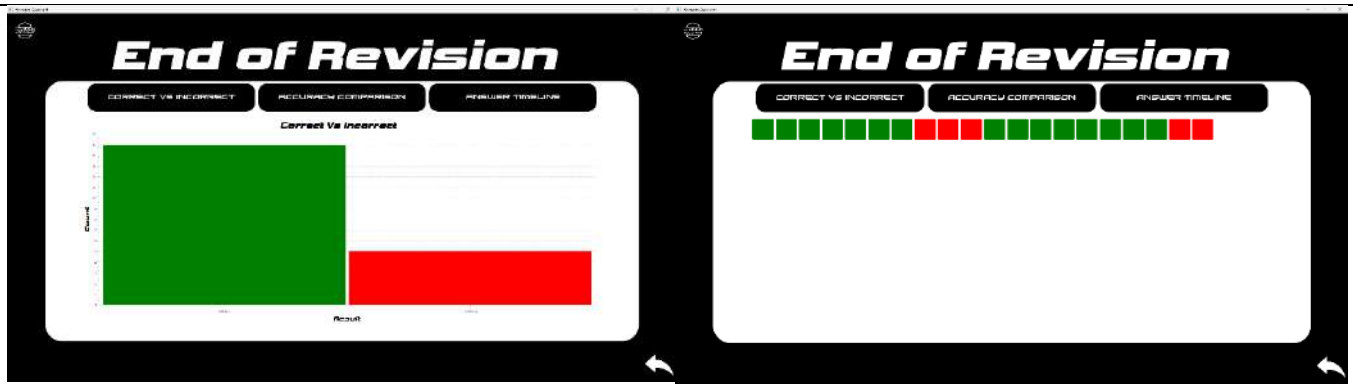
RevisionSystemScreen Image 9 - Sessions per Week Chart

3.4.4.3.3 Testing Analytics

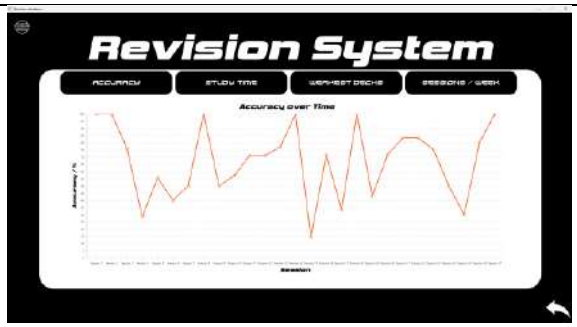
Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	Correct vs Incorrect Chart	10 answered, 7 correct, 3 incorrect	Normal	Correct bar with 7, incorrect with 3	Correct bar with 7, incorrect with 3	
2.	Answer Timeline	10 answered, 7 correct, 3 incorrect in order	Normal	7 green boxes, 3 red, in correct order	7 green boxes, 3 red, in correct order	
3.	Accuracy Comparison Chart	Create a deck, do the deck four times and on fourth attempt check if accuracy comparison is correct.	Normal	Accuracy comparison with last 3 sessions is completely valid.	Accuracy comparison with last 3 sessions is completely valid.	
4.	Retry a session, see charts	Retry a session, now get 8 correct and 2 incorrect	Normal	The correct bar now has 15, incorrect has 5. Timeline also shows this	The correct bar now has 15, incorrect has 5. Timeline also shows this	
5.	All-time accuracy line graph	Check validity of the graph against the stored data	Normal	The accuracy data is valid.	The accuracy data is valid.	
6.	All-time study time graph	Study a 5-minute session.	Normal	There is now a new point on the line graph (on the very right).	There is now a new point on the line graph (on the very right).	
7.	Weak decks chart	Study a high performing chart, get everything wrong, check how the bar chart is then updated	Normal	Deck average accuracy goes down, hence bar on bar chart moves to the left.	Deck average accuracy goes down, hence bar on bar chart moves to the left.	
8.	Sessions per week	Change the date of a session on the csv to 04/01/2026 and check graph	Normal	New bar with x labels as “W1”, with the count of 1.	New bar with x labels as “W1”, with the count of 1.	

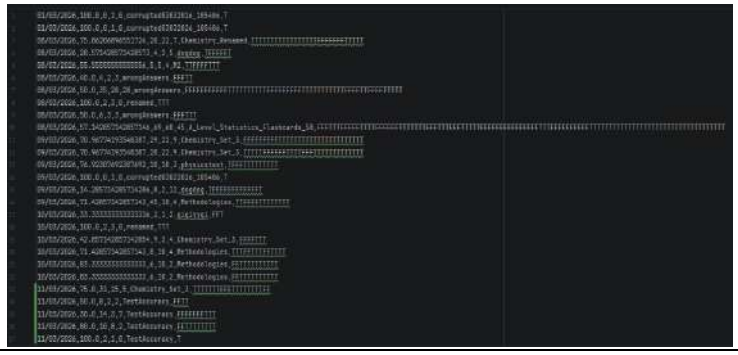
Test No.	Proof
1.	
2.	
3.	Session 1: 50%, Session 2: 30%, Session 3: 80%, Session 4: 100%.    

4.

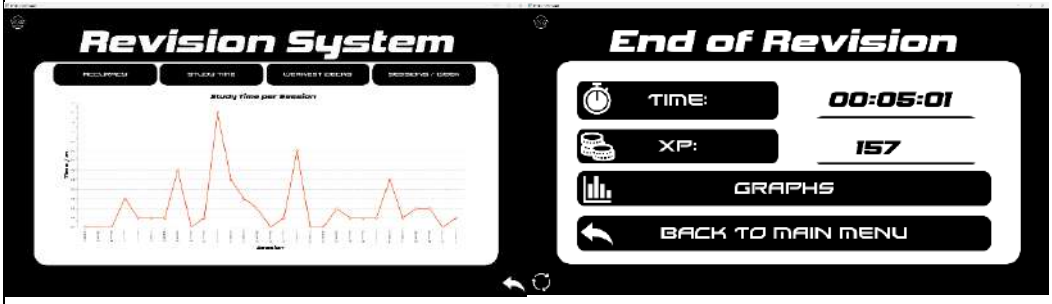

End of Revision
CORRECT VS INCORRECT
ACCURACY COMPARISON
ANSWER TIMELINE
Correct Vs Incorrect
Count
Result
End of Revision
CORRECT VS INCORRECT
ACCURACY COMPARISON
ANSWER TIMELINE

5.

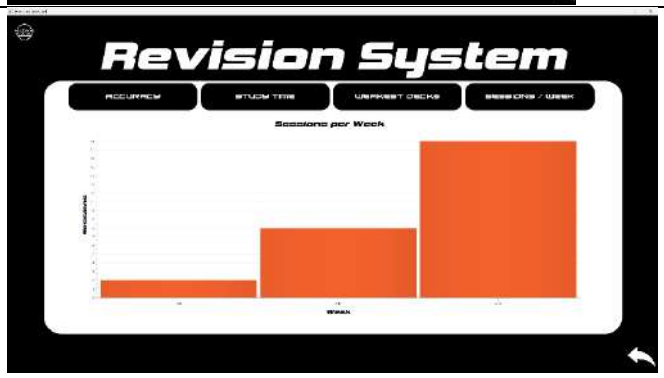
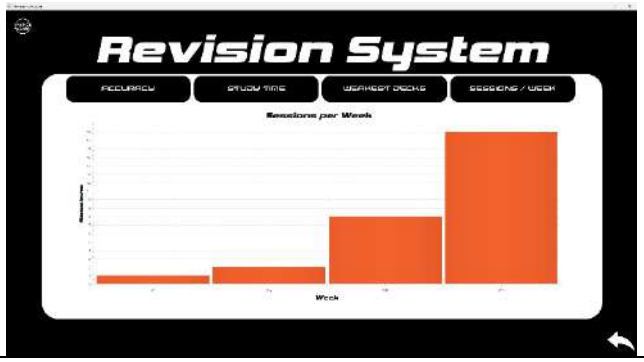

Revision System
ACCURACY
STUDY TIME
WORKOUT DECKS
SESSIONS / WEEK
Accuracy over Time



6.


Revision System
Study Time per Session
End of Revision
TIME:
00:05:01
XP:
157
GRAPHS
BACK TO MAIN MENU


Revision System
ACCURACY
STUDY TIME
WORKOUT DECKS
SESSIONS / WEEK
Study Time per Session

7.	
8.	  

3.4.5 Evaluation of Prototype 4

Prototype 4 implemented features which allow the Revision Assistant to develop from a flashcard storing tool to a tool which actively tracks and motivates user progress when studying. When studying, in serial, shuffle or wrong answers mode, the system records the duration of each revision session, converting this into hour and minutes to display to the user when studying, or after a session, allowing users to see how long they have studied, satisfying success criterion 10. Furthermore, I have implemented a text-to-speech library which works offline; hence the program is able to read flashcard questions and answers out loud to the user, useful for auditory learners. This feature helps meet success criterions 8 and 13, since the text-to-speech works correctly and is available offline. The solution also accurately measures XP and achievements, making it extremely scalable for the future with classes specific to these features. The program automatically calculates the XP values at the end of each session and updates the user's total xp accurately, according to numerous tests. Achievements are also given to the user when certain milestones are reached. Following tests, it is clear that the

system has met the requirements to satisfy success criterion 7. Finally, success criterion 14 was met with users being able to visualise their study data, including various forms of visualisations such as bar charts and line graphs allowing user to view advanced analytics and use this information to optimise their studying. Overall, prototype 4 has successfully implemented the final core features of the program allowing me to move on the final prototype where I will aim to improve the reliability and security of the Revision Assistant with file encryption and the ability to back up files, providing a possible solution for cases of file corruption.

3.5 Prototype 5: Security, Stability and System Recovery

3.5.1 Purpose of Prototype 5

Prototype 5 aims to improve the security, stability and reliability of the Revision Assistant, the purpose of this prototype is to protect sensitive user data (already done in Prototype 1, but insufficient) as well as ensure the application can recovery from potential file corruption or data loss. Since the solution aims to satisfy success criterion 8 (offline access), email tokens were not used to recover lost accounts, but a recovery word was. This recovery word was stored along side other user information, if a user can access this data, they would be able to view other users' recovery words allowing them to reset passwords of accounts that do not belong to them. To prevent this, I will encrypt the users.csv file, this also adds another layer of security to accessing user information such as hashed passwords. It does not make the hashing redundant; it provides an additional layer of security to the password specifically. This is because if the file was decrypting, the original passwords would still not be able to be recovered easily.

Furthermore, a potential solution / proactive plan for important data files has been implemented where the user can export and therefore import the following files: users.csv, stats.csv and settings.csv. This allows the user to help the system recover data if the primary files corrupt, improving the overall stability of the system by reducing the risk of entire data loss.

3.5.2 Success Criteria for Prototype 5

Referencing (1.4.3 Success Criteria), with this prototype I will be aiming to achieve success criterion 4, where the system does not store passwords and (adding to this) recovery words in plain text preventing users from resetting other users' passwords.

4) The program should not store duplicate email addresses and must not store passwords in plain text (store hashed password instead).

This is the only criterion from the previously documented Success Criteria, but to be able to evaluate this prototype, I will be setting the following specific criteria:

1) System should allow users to create important files including users.csv, stats.csv and settings.csv.

3.5.3 Developing Prototype 5

3.5.3.1 File Backup

To begin implementing file backup, within the User Management page's File Management page I created 3 sets of two buttons (export and import) placed alongside each other in VBoxes and then added to the page in a VBox.

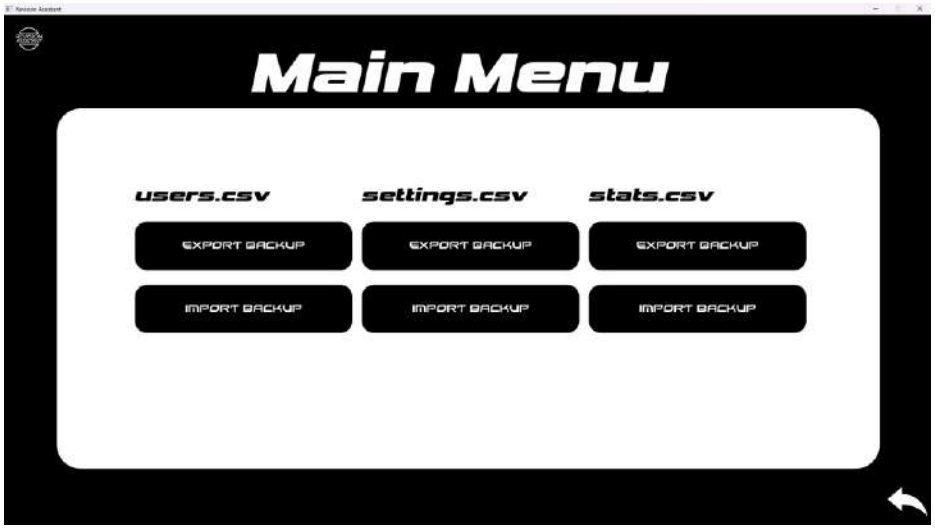
```
// Create file management content
private void buildFileManagementContent(Group contentLayer) {
    Window window = contentLayer.getScene().getWindow();
    HBox content = new HBox( spacing: 20);
    ui.setPosition(content, x: 270, y: 350);
    String[] files = { "users.csv", "settings.csv", "stats.csv" };
    // Create buttons for each
    for(int i = 0; i < files.length; i++){
        String file = files[i];

        Text text = new Text(file);
        text.getStyleClass().add("infoText");

        RectangleButtonBundle exp = ui.buildRectangularButtonBundle( rectW: 450, rectH: 100, text: "Export Backup", typeOfText: "smallerButtonText", StackPaneX: 0, StackPaneY: 0);
        exp.button().setOnMouseClicked( MouseEvent _ -> {
            try {
                exportFile(window, file);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        RectangleButtonBundle imp = ui.buildRectangularButtonBundle( rectW: 450, rectH: 100, text: "Import Backup", typeOfText: "smallerButtonText", StackPaneX: 0, StackPaneY: 0);
        imp.button().setOnMouseClicked( MouseEvent _ -> {
            try {
                importFile(window, file);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        VBox box = new VBox( spacing: 30, text, exp.button(), imp.button());
        content.getChildren().addAll(box);
    }
    nav.switchContent(contentLayer, content, getBackView(contentLayer));
}
```

Main Menu Screen Code 9 – buildFileManagementContent

This makes the use of a for loop to create these buttons to prevent repeating of code, since the code for each file would be the same.



Main Menu Screen Image 9 - File Management Page

If the user clicks the export button, the exportFile procedure is called which is similar to exporting the flashcard decks, but in this case the user only has to pick the save location.

```
// Copies the system CSV to a location chosen by user
private void exportFile(Window window, String fileName) throws IOException {  1 usage
    // Create a file chooser
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Export " + fileName);
    fileChooser.setInitialFileName(fileName);
    // Filter to csv
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "CSV",  ...extensions: "*.csv"));
    File destination = fileChooser.showSaveDialog(window);
    // if save cancelled, return
    if(destination == null) return;

    // Find the source and copy it to save location
    Path source = Paths.get(System.getProperty("user.dir"),  ...more: "files", fileName);
    Files.copy(source, destination.toPath(), StandardCopyOption.REPLACE_EXISTING);

    CustomAlert.success(window,  message: fileName + " has been exported");
}
```

Main Menu Screen Code 10 - exportFile()

If the user selects the import button, the importFile() procedure is called which allows the user to pick a file, confirm this change and then logout the user from the system in case they no longer exist within the files.

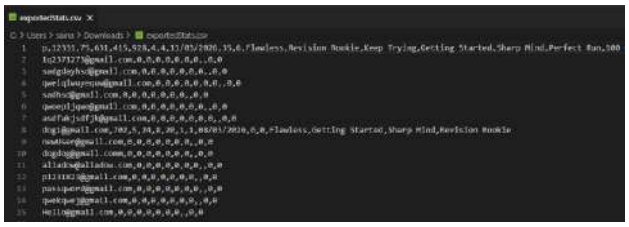
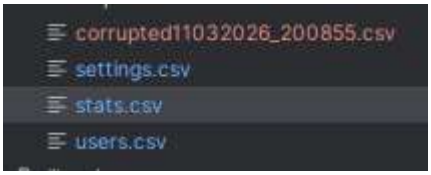
```
// Replaces the system CSV with user chosen file
private void importFile(Window window, String fileName) throws IOException {  1 usage
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Import " + fileName);
    // Filter to csv
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "CSV",  ...extensions: "*.csv"));
    File source = fileChooser.showOpenDialog(window);
    // return if empty or cancelled
    if(source == null) return;
    if(source.length() == 0){
        CustomAlert.error(window,  message: "File has no contents");
        return;
    }
    // Ask for user confirmation
    boolean confirmed = CustomAlert.confirm(window,  title: "Import Backup",  message: "This will replace your current " + fileName
            + ". Are you sure you want to continue? You will be logged out following the import.");
    if(!confirmed) return;
    Path destination = Paths.get(System.getProperty("user.dir"),  ...more: "files", fileName);
    Files.copy(source.toPath(), destination, StandardCopyOption.REPLACE_EXISTING);
    CustomAlert.success(window,  message: fileName + " has been imported");
    // Log out so imported files can be read from logging in
    ac.logout();
}
```

Main Menu Screen Code 11 - importFile()

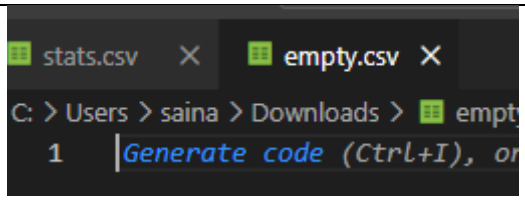
This recovery approach was chosen since it provides a simple and reliable method of protecting user data from corruption, it allows the user to download backup copies of important files which can be used to restore data in case of problems with the primary files. I chose this approach instead of a complex recovery system such as automated backups since they would increase the complexity of the system significantly and may require additional infrastructure such as database systems and/or cloud storage (hence internet connections going against the success criteria). Files corrupting by themselves is unlikely, hence recovery is kept simple, and gives the user control over where and when backups are stored.

3.5.3.1.1 Testing File Backup

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	Exporting	Export stats.csv	Normal	File matches the stats.csv within the file.	File matches the stats.csv within the file.	
2.	Import	Import the exported file back into the system	Normal	System should work as normal, file not labelled as corrupted by proactive methods implemented in prior prototypes. Should be logged out	System works normally, file matches imported file, file not labelled as corrupted, user logged out.	
3.	Import	Change stats.csv, remove the line with current user's email	Erroneous	File imported, user logged out but when logged back in, default stats will be written into the csv	File corrupted, default stats written into a new file.	
4.	Import	Import an empty file to settings.csv	Erroneous	Error shown to user, not imported, not logged out	Error shown to user, not imported, not logged out	
5.	Import	Import a with missing values within the csv file in settings.csv	Erroneous	File is labelled as corrupted, new file created.	File is labelled as corrupted, new file created.	

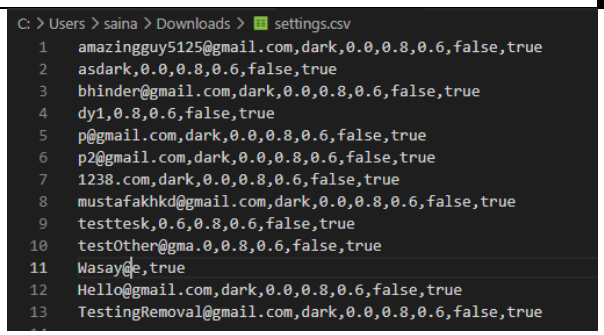
Test No.	Proof
1.	  
2.	  
3.	    

4.

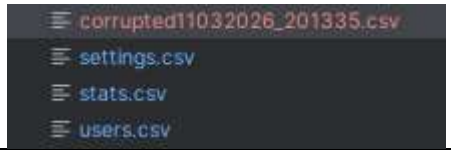




5.

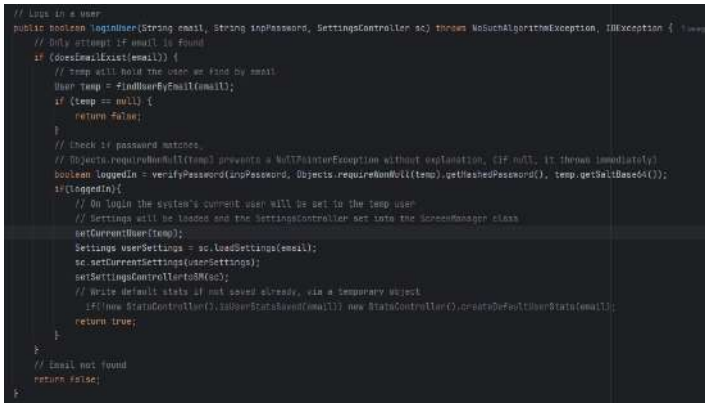






3.5.3.1.2 Remedial Actions

The issue was caused by the loginUser function within AuthenticationController, it checked if the user stats were saved and called createDefaultUserStats if no row was found with the user's email, this overwrote any data that may have existed previously. To fix this, I removed the default stats initialisation check from loginUser, and the createDefaultUserStats is only called during the registerUser function which is the only point where the user is guaranteed not to have an existing stats row. If stats are missing for an existing user, they will be written in to the .csv the next time the user completes a session.



```
// Logs in a user
public boolean loginUser(String email, String inpPassword, SettingsController sc) throws NoSuchAlgorithmException, IOException {
    // Only attempt if email is found
    if (doesEmailExist(email)) {
        // temp will hold the user we find by email
        User temp = findUserByEmail(email);
        if (temp == null) {
            return false;
        }
        // Check if password matches
        // Objects.requireNonNull(temp) prevents a NullPointerException without explanation, if null, it throws immediately)
        boolean loggedIn = verifyPassword(inpPassword, Objects.requireNonNull(temp).getHashedPassword(), temp.getSaltBase64());
        if(loggedIn){
            // On login the system's current user will be set to the temp user
            // Settings will be loaded and the SettingsController set into the ScreenManager class
            setCurrentUser(temp);
            Settings userSettings = sc.loadSettings(email);
            sc.setCurrentSettings(userSettings);
            setSettingsControllerToSM(sc);
            // Write default stats if not saved already, via a temporary object
            if(!new StatsController().isUserStatsSaved(email)) new StatsController().createDefaultUserStats(email);
            return true;
        }
    }
    // Email not found
    return false;
}
```

AuthenticationController Code 17 - loginUser issue commented out

Following further tests, the remedial action has been confirmed to work, and the stats row is only reconstructed after a session. This was not a problem with settings.csv since the row in settings is only constructed when the user edits a setting.

3.5.3.2 Encrypting Users.csv

To protect the user.csv's recovery word and adding an additional layer of security to the passwords, I encrypted the file with Advanced Encryption Standard (AES). AES is a symmetric encryption algorithm which means the same key is used to encrypt and decrypt, hence it is efficient and suitable for the Revision Assistant which needs data to be encrypted and decrypted quickly so the user does not have to wait during login. AES is also supported natively in Java's built in cryptography libraries making it easier to implement with the help of the Cipher class. I could have used RSA which is a form of asymmetric encryption which using two different keys, a public to encrypt and private to decrypt; but I decided against it since it is computationally slower than AEs which would not be efficient for when users are registering/logging in.

The rewriteUsersCSV procedure was changed to call the writeEncrypted function:

```
// Will be used to update CSV when user makers changes
private void rewriteUsersCSV(ArrayList<User> users) throws IOException {  4 usages
    StringBuilder sb = new StringBuilder();
    // Rewrite the entire file, so updates persist
    for (User u : users) {
        sb.append(u.getEmail()).append(",").append(u.getHashedPassword()).append(",").append(u.getSaltBase64()).append(",")
                .append(u.getLastLogin()).append(",").append(u.getRecoveryWord()).append("\n");
    }
    writeEncrypted(sb.toString());
}
```

AuthenticationController Code 18 - updated rewriteUsersCSV

```
// Encrypt the given content and write to users.csv
private void writeEncrypted(String text) throws IOException{
    // get encrypted text
    String encrypted = encrypt(text);
    // Write to the file
    Files.writeString(csvPath, encrypted);
}
```

AuthenticationController Code 19 - writeEncrypted procedure

This procedure calls the encrypt function which will encrypt the text passed into the subroutine with AES.

```java
// Encryption / Decryption
// Hardcoded 16-byte AES key -> 16 characters exactly for AES-128
private static final String encryptionKey = "RevAssistant2026";  2 usages
// Encrypt a string and returns base 66 encrypted string
private String encrypt(String line){  1 usage
    try{
        // Create an AES secret key using the predefined encryption key
        SecretKey key = new SecretKeySpec(encryptionKey.getBytes(StandardCharsets.UTF_8),  algorithm: "AES");
        // Create and configure the cipher for encryption with padding
        Cipher cipher = Cipher.getInstance( transformation: "AES/ECB/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, key);
        // Convert the text into bytes and encrypt it
        byte[] encrypted = cipher.doFinal(line.getBytes(StandardCharsets.UTF_8));
        // Convert encrypted bytes to base 64 so it can be saved safely as text
        return Base64.getEncoder().encodeToString(encrypted);
    }catch(Exception e){
        throw new RuntimeException("Encryption did not work: " + e.getMessage());
    }
}
```

AuthenticationController Code 20 - encrypt function

The encryption key must be 16 characters since 1 character is 1 byte and therefore 16 characters has 128 bits, for AES-128 encryption.

The convertCSVtoArrayList function was updated to:

```java
// Reads users.csv and converts each row into a user object
public ArrayList<User> convertCSVtoArrayList() throws IOException {  6 usages
    // Ensure folders and CSV exist before reading
    ensureCSVExists(csvPath);
    ArrayList<User> users = new ArrayList<>();
    // Read CSV line by line
    try {
        ArrayList<String> lines = getDecryptedLines();
        for (String line : lines) {
            String[] temp = line.split( regex: ",");
            if(temp.length < 5) throw new IllegalArgumentException("Corrupted Row: " + line);
            users.add(new User(temp[0], temp[1], temp[2], temp[3], temp[4]));
        }
    } catch (Exception e) {
        quarantineCorruptedCSV(e, csvPath);
        // Act as if there are no users if corrupted.
        return new ArrayList<>();
    }
    return users;
}
```

AuthenticationController Code 21 - updated convertCSVtoArrayList function

This calls the getDecryptedLines function, which returns an ArrayList of each line of the users.csv file decrypted.

```java
// Read encrypted users.csv, decrypt and returns the line in an ArrayList
private ArrayList<String> getDecryptedLines() throws IOException{  1 usage
    ArrayList<String> lines = new ArrayList<>();
    // Read all content
    byte[] fileBytes = Files.readAllBytes(csvPath);
    // If file is empty, return empty list
    if(fileBytes.length == 0) return lines;
    // Decrypt the entire file
    String decrypted = decrypt(new String(fileBytes, StandardCharsets.UTF_8).trim());
    // Iterate through and add to ArrayList
    for(String line : decrypted.split( regex: "\n")){
        if(!line.isBlank()) lines.add(line);
    }
    return lines;
}
```

AuthenticationController Code 22 - getDecryptedLines function

This decrypts the file contents with the decrypt function:

```
// Decrypt a base 64 encoded string to plain text
private String decrypt(String line){  1 usage
    try{
        // Create an AES secret key using the predefined encryption key
        SecretKey key = new SecretKeySpec(encryptionKey.getBytes(StandardCharsets.UTF_8),  algorithm: "AES");
        // Create and configure the cipher for encryption with padding
        Cipher cipher = Cipher.getInstance( transformation: "AES/ECB/PKCS5Padding");
        cipher.init(Cipher.DECRYPT_MODE, key);
        // Decode the Base64 string back to plain text
        byte[] decrypted = cipher.doFinal(Base64.getDecoder().decode(line));
        return new String(decrypted, StandardCharsets.UTF_8);
    }catch(Exception e){
        throw new RuntimeException("Decryption did not work: " + e.getMessage());
    }
}
```

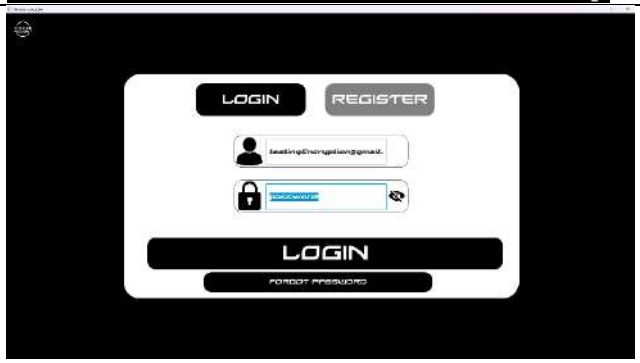
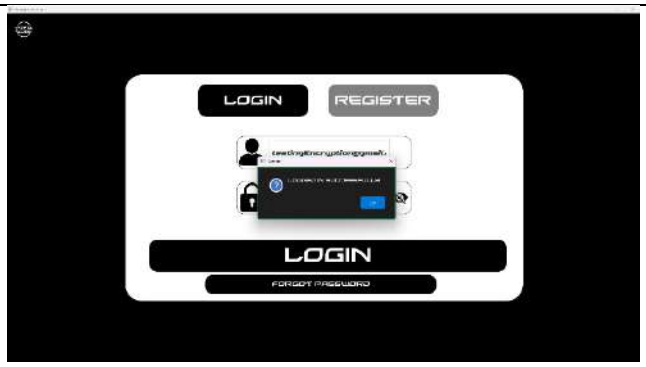
AuthenticationController Code 23 - decrypt function

3.5.3.2.1 Testing Encryption

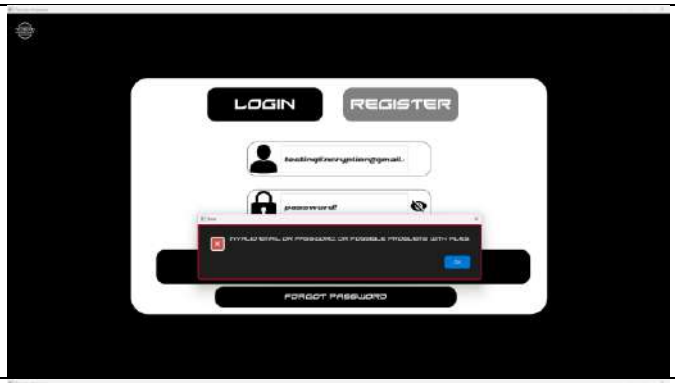
To test the encryption and decryption, I will be testing the user login, registration, forgot password, reset (email/password/recovery word).

Test No	Test item	Test data	Test type	Expected outcome	Actual outcome	Pass / Fail
1.	Registration	email: “testingEncryption@gmail.com” password: “password!” rW: “hello”	Normal	User is registered and loaded into the main menu screen after success alert	User is registered and loaded into the main menu screen after success alert	
2.	Registration	Try to register with the same email	Erroneous	User shown error, email already exists	User shown error, email already exists	
3.	Login	Log in the previous credentials	Normal	User is logged in and loaded into the main menu screen, after success alert	User is logged in and loaded into the main menu screen, after success alert	
4.	Forgot Password	Use recovery word to reset password to “password!!”	Normal	Password is reset	Password is reset	
5.	Login	Attempt to login with the previous password	Erroneous	Passwords do not match, error shown.	Passwords do not match, error shown.	
6.	Login	Attempt to login with the new password	Normal	User is logged in and loaded into the main menu screen, after success alert	User is logged in and loaded into the main menu screen, after success alert	
7.	Change RW	Change recovery word to “dog”	Normal	Recovery word changed	Recovery word changed	

8.	Forgot password	Attempt to use old recovery word in forgot password	Erroneous	Recovery word does not match	Recovery word does not match	
9.	Change password	Change password to “hello123!”	Normal	Password change	Password change	
10.	Login	Attempt to login with this password	Normal	User is logged in and loaded into the main menu screen, after success alert	User is logged in and loaded into the main menu screen, after success alert	
11.	Change email	Change the email to “emailChanged@gmail.com”	Normal	Email changed	Email changed	
12.	Register	Register with the previous email: “testingEncryption@gmail.com”	Normal	User is registered and loaded into the main menu screen, after success alert	User is registered and loaded into the main menu screen, after success alert	

Test No.	Proof
1.	 
2.	
3.	 
4.	 

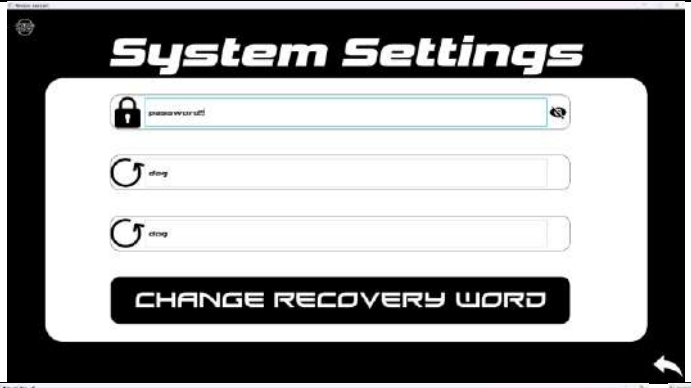
5.


LOGIN
REGISTER
FORGOT PASSWORD

6.


LOGIN
REGISTER
FORGOT PASSWORD

7.


System Settings
CHANGE RECOVERY WORD


System Settings
CHANGE RECOVERY WORD

8.

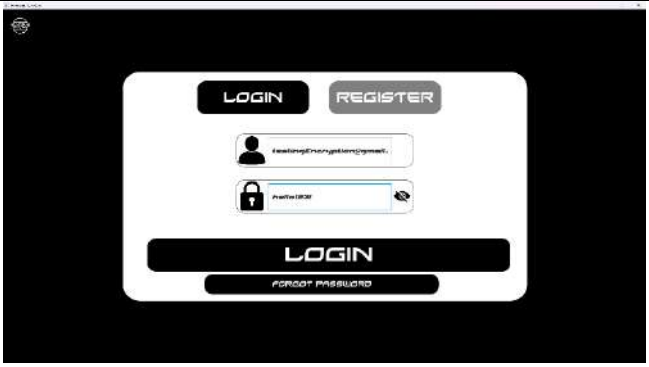

RESET PASSWORD


RESET PASSWORD

9.

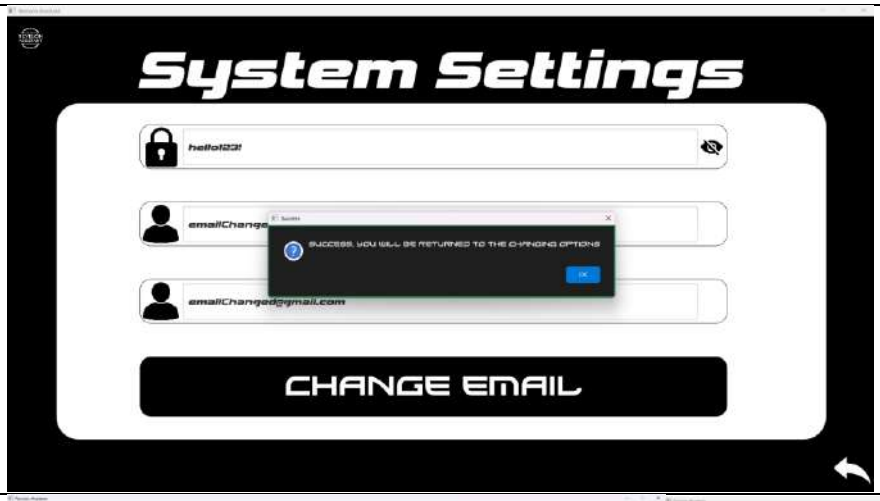

System Settings
CHANGE PASSWORD

10.

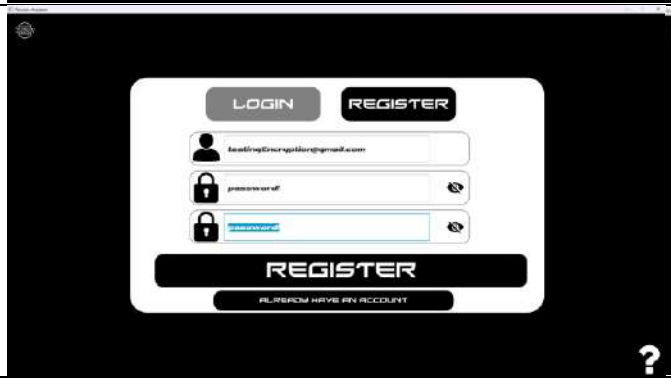

LOGIN
REGISTER
LOGIN
FORGOT PASSWORD

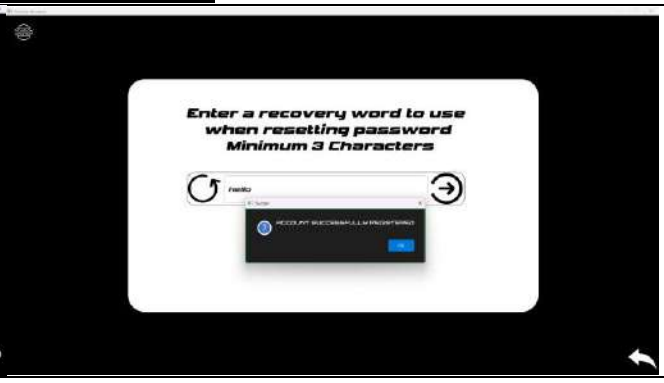

LOGIN
REGISTER
LOGIN
FORGOT PASSWORD

11.


System Settings
CHANGE EMAIL

12.


LOGIN
REGISTER
REGISTER
ALREADY HAVE AN ACCOUNT


Enter a recovery word to use when resetting password
Minimum 3 Characters

3.5.3.2.2 Remedial Actions

While testing the changing email, I realised I had created this functionality in prototype 2 alongside settings, this means the settings.csv and stats.csv is not updated with the new email; /files/sessions/(email).csv is not updated and /userFolders/(email) is not updated, so the user will not have access to their previous email related data. To remediate this, I will update the changeEmail function within AuthenticationController.

```
// Change a user's email
public boolean changeEmail(User temp, String newEmail) throws IOException {  1 usage
    ArrayList<User> users = convertCSVtoArrayList();
    // Load all users and update the matching one
    for(User u : users){
        if(u.getEmail().equals(temp.getEmail())){
            u.setEmail(newEmail);
            break;
        }
    }
    // Write into file
    rewriteUsersCSV(users);
    // Update the stats.csv and settings.csv, replace old email with new
    sc.updateEmail(temp.getEmail(), newEmail);
    new StatsController().updateEmail(temp.getEmail(), newEmail);
    // Rename the Sessions csv
    Path oldSession = Paths.get(System.getProperty("user.dir"),  ...more: "files", "sessions", temp.getEmail() + ".csv");
    Path newSession = Paths.get(System.getProperty("user.dir"),  ...more: "files", "sessions", newEmail + ".csv");
    if(Files.exists(oldSession)) Files.move(oldSession, newSession, StandardCopyOption.REPLACE_EXISTING);
    // Rename the decks folder
    Path oldFolder = Paths.get(System.getProperty("user.dir"),  ...more: "userFolders", temp.getEmail());
    Path newFolder = Paths.get(System.getProperty("user.dir"),  ...more: "userFolders", newEmail);
    if(Files.exists(oldFolder)) Files.move(oldFolder, newFolder, StandardCopyOption.ATOMIC_MOVE);
    // Update current user in memory
    temp.setEmail(newEmail);
    setCurrentUser(temp);
    return true;
}
```

AuthenticationController Code 24 - updated changeEmail() function

The emails are updated within the users.csv, stats.csv and settings.csv with the updateEmail procedures which performs the same task as the first half the changeEmail function but for their specific csv files. The path for the file storing the session data is labelled as older path, and a new path is created, this new path overwrites the old file, essentially renaming it. The same is done for the decks folder and the current user is updated with the new email.

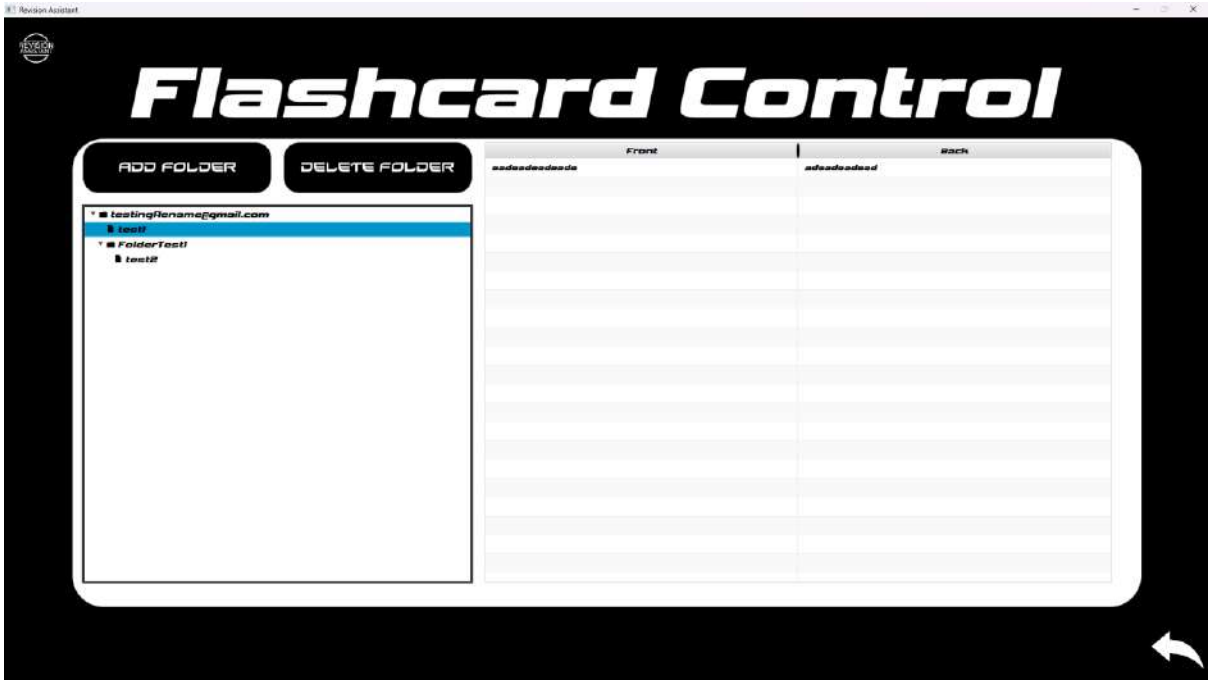
```
// Procedure to update the email within the file when the user changes their email
public void updateEmail(String oldEmail, String newEmail) throws IOException {  1 usage
    // load all users
    ArrayList<UserStats> statsList = convertCSVtoArrayList();
    for(UserStats stats : statsList){
        if(stats.getEmail().equals(oldEmail)){
            // change the email
            stats.setEmail(newEmail);
            break;
        }
    }
    // rewrite the file
    rewriteCSV(statsList);
}
```

StatsController Code 13 - updateEmail()

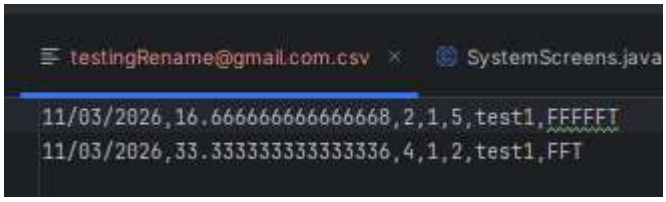
```java
// Procedure to update the email within the file when the user changes their email
public void updateEmail(String oldEmail, String newEmail) throws IOException {  1 usage
    // load all users
    ArrayList<Settings> settingsList = convertCSVtoArrayList();
    for(Settings settings : settingsList){
        if(settings.getEmail().equals(oldEmail)){
            // change the email
            settings.setEmail(newEmail);
            break;
        }
    }
    // rewrite the file
    rewriteCSV(settingsList);
}
```

SessionController Code 6 - updateEmail()

To test this solution, I will create a new account with email “testingRename@gmail.com”, I will create 2 flashcard decks placing one within a newly created folder. I will then do a study session, so there is data stored within user stats and the session’s csv. I will then change my email to “emailchange2@gmail.com” and check if everything has updated.



Account registered, created 2 decks, placed one within a folder.



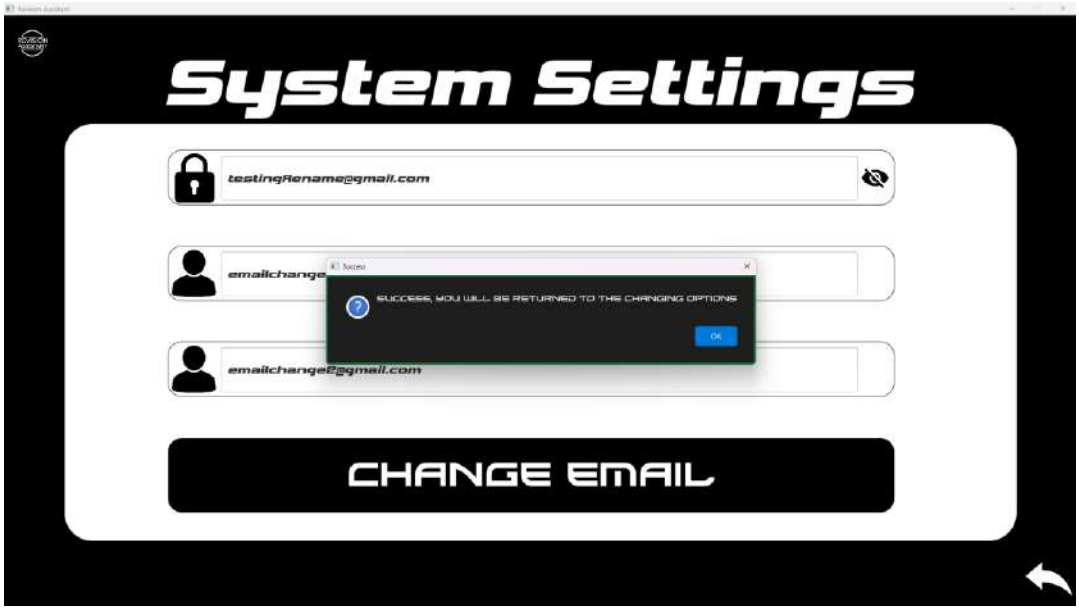


→ within settings.csv

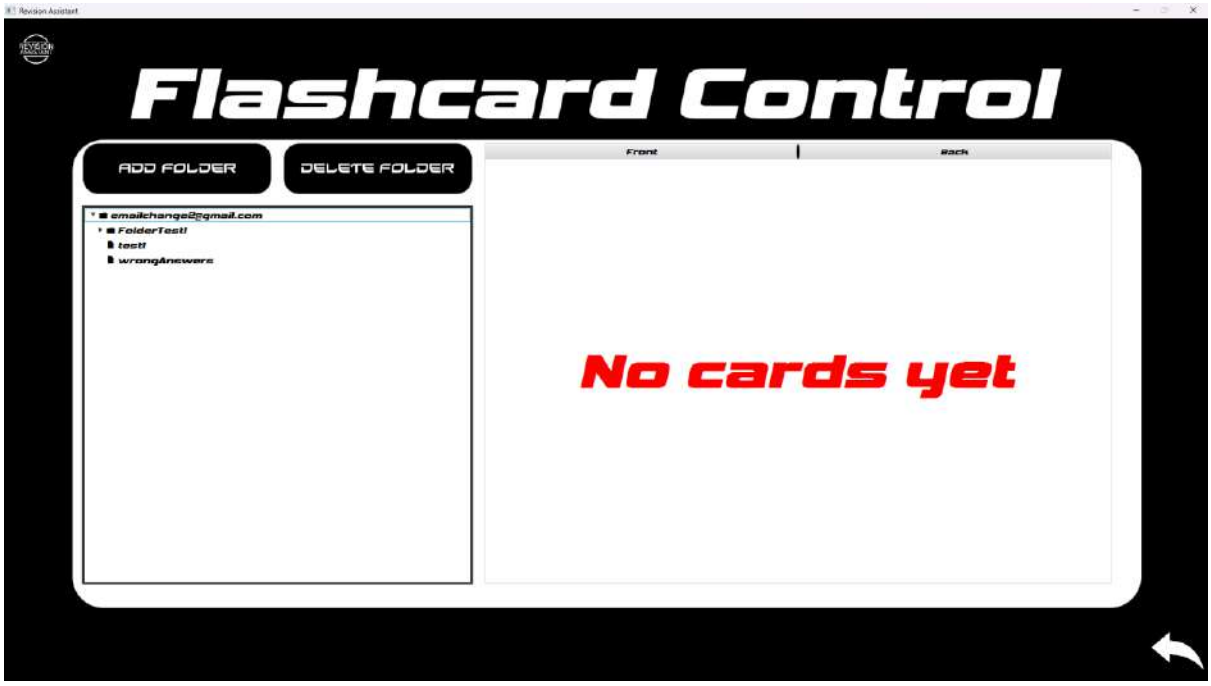
```
testingRename@gmail.com,66,2,2,7,6,1,1,11/03/2026,1,0
```
→ stats.csv



→ folder structure

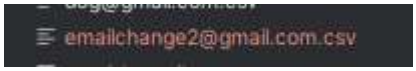


Email changed.



Directory structure the same, but with changed root path according to email.

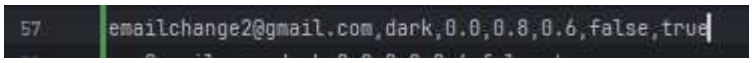
```
emailchange2@gmail.com,66,2,2,7,6,1,1,11/03/2026,1,0
```
→ stats.csv updated



→ sessions csv updated



with contents same



→ settings.csv updated

The email has been updated everywhere, so user data will not be lost when a user's email has been changed, thus this remedial action was successful.

3.5.4 Evaluation of Prototype 5

Prototype 5 has successfully implemented security and stability improvements while remediating hidden bugs within code implemented in prior prototypes. The system now encrypts the users.csv file meaning that the recovery word cannot be read by people going through the files on their computer's file explorer, giving them access to other users' accounts. In addition to the AES encryption, the program now also lets users create backups to important data files, this allows users to restore their progress (for example) if the primary data file is lost/corrupted. Overall, this prototype meets the success criteria and meets the requirements expected at the start of development, this prototype concludes the development of the Revision Assistant.

Section 4 Evaluation

4.1 Destruction Testing

Destruction testing will be conducted to ensure that Revision Assistant can function under unexpected inputs/events, this stage is necessary to test the robustness of the solution, making sure the program is safe and reliable for users.

To begin destruction testing I will begin by following the test plans set in 2.4.3 Post-Development Testing.

4.1.1 Importing CSVs

I will destructively test the import CSV functionality by purposely using corrupted files (by corrupting rows with wrong number of data items, incorrect data types and invalid data format). I expect the system to be able to handle these inputs, rejecting the invalid data whilst displaying appropriate error messages.

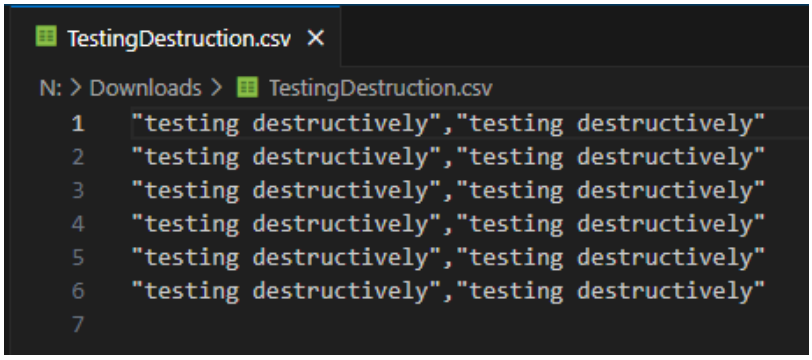
4.1.1.1 Corrupting Flashcard CSVs

Following the creation and the exportation of a valid flashcard deck, I will conduct the following tests.

1. Change a string field to numeric, then import, expecting the program to reject.
2. Remove a comma within a row, then import, expecting the program to reject.
3. Separate to prior, import a flashcard deck at boundaries of the program → 300 char questions, 600 char answers, 60 cards.

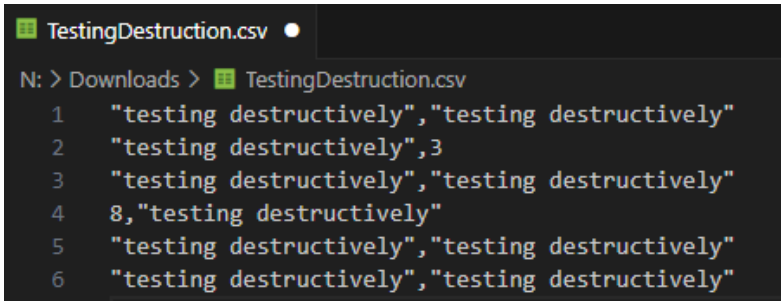
4.1.1.1.1 Numerically Corrupting CSVs

For test number 1 and 2, I have created and exported the following flashcard deck:

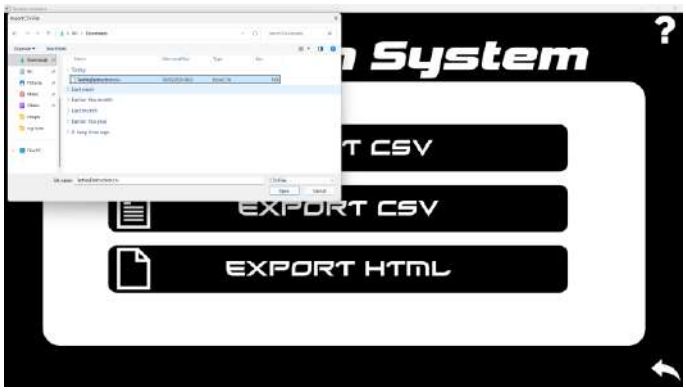


Destruction Testing 1 - TestingDestruction.csv

For test 1, I have updated the csv file to have numeric fields alongside the string fields, this is of the incorrect format, hence should be rejected when user attempts to import.

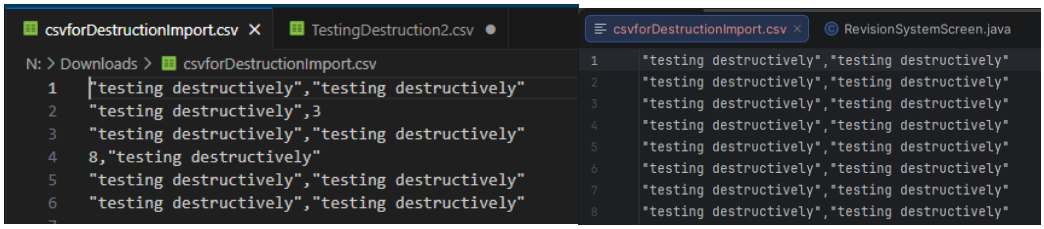


Destruction Testing 2 - corrupting TestingDestruction.csv



RevisionSystemScreen Image 10 - Selecting csv to import

When I import this, although the rows are corrupted, the system is allowing the import and still replacing the incorrect fields with previous data:

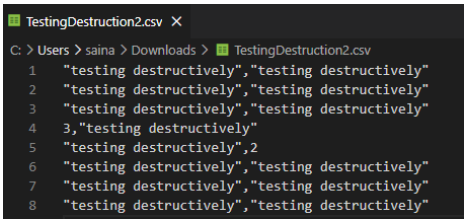


The illegal import was accepted into the system, and the corrupted rows were appended to the existing file, this happened because corrupted rows were treated as new entries and appended without further validation. To fix this, I have implemented a method to validate the csv prior to import; for the specific file type it will verify the correct number of columns per row and whether if it should be numeric or not, this is checked for each row and if any validation fails, the import is rejected and user is displayed an error.

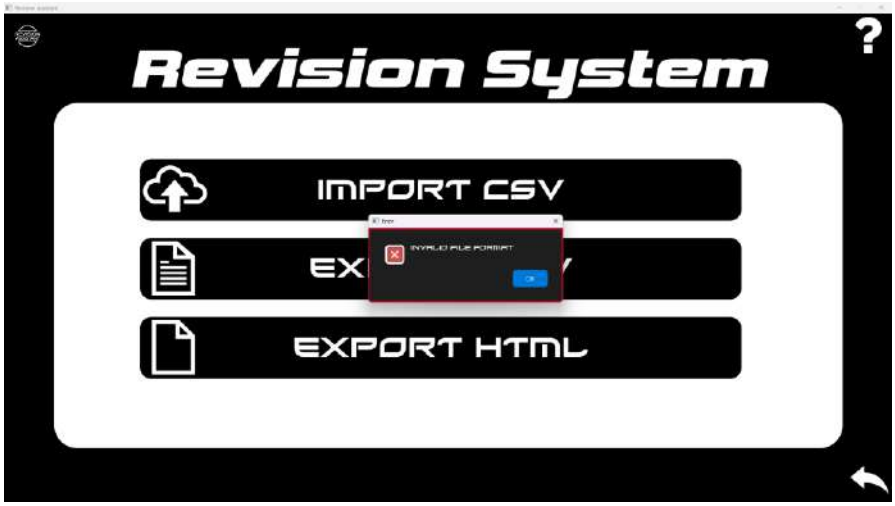
```
// function to validate if the csv is suitable for import
public boolean validateCSV(File source, String fileName) throws  IOException{  2 usages
    try(BufferedReader br = new BufferedReader(new FileReader(source))){
        String line;
        while ((line = br.readLine()) != null) {
            if(fileName.equals("flashcard")) {
                // Split at ,
                String[] data = line.split( regex: "\",\"");
                if(data.length != 2) return false;
                // Replace speech marks with empty space
                String question = data[0].replace( target: "\"",  replacement: "").replace( target: "\\n",  replacement: "\n");
                String answer = data[1].replace( target: "\"",  replacement: "").replace( target: "\\n",  replacement: "\n");
                if(isNumeric(question) || isNumeric(answer)) return false;
            }else{
                String[] data = line.split( regex: ",");
                if(fileName.equals("stats.csv") && data.length >= 11) return false;
                if(fileName.equals("settings.csv") && data.length != 7) return false;
            }
        }
    }
    return true;
}
// Check if the value is numeric
private boolean isNumeric(String s) {  2 usages
    try {Double.parseDouble(s);return true;}
    catch (NumberFormatException e) {
        return false;}
}
```

Controller Code 1 - validateCSV function placed in super class so can be used by multiple subclasses

Attempting to import the following file:



Gives an output of:



With following tests, the illegal CSV imports are now rejected, rectifying the previous issue. This will prevent the user from importing invalid CSVs.

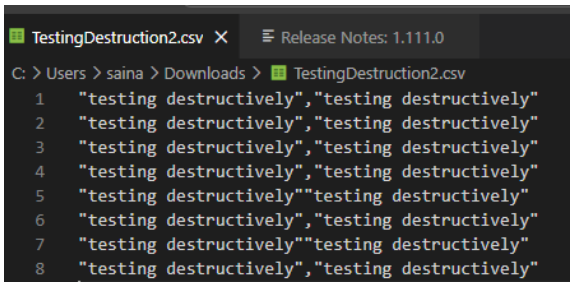
4.1.1.1.2 Structurally Corrupting CSVs

For the following tests I will:

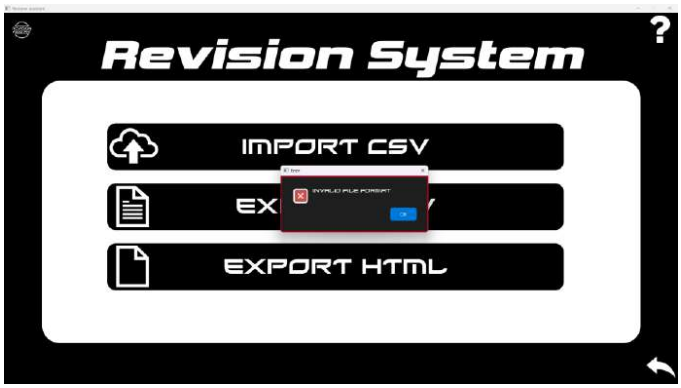
1. Remove delimiters in random rows, then test import.
2. Remove fields from random rows, then test import.

I expect the inputs to be rejected, and an error should be displayed to the user.

Corrupted CSV:



Output:

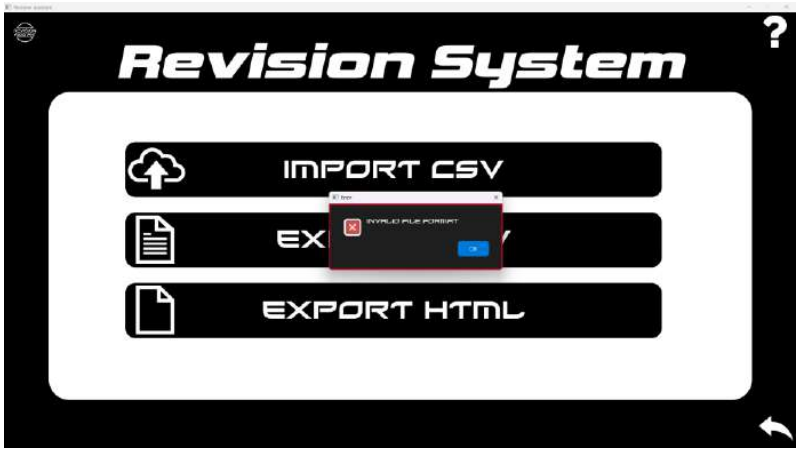


Corrupted File:

C: > Users > saina > Downloads > TestingDestruction2.csv

```
1  "testing destructively","testing destructively"
2  "testing destructively","testing destructively"
3  "testing destructively",
4  "testing destructively","testing destructively"
5  ,"testing destructively"
6  "testing destructively","testing destructively"
```

Output:

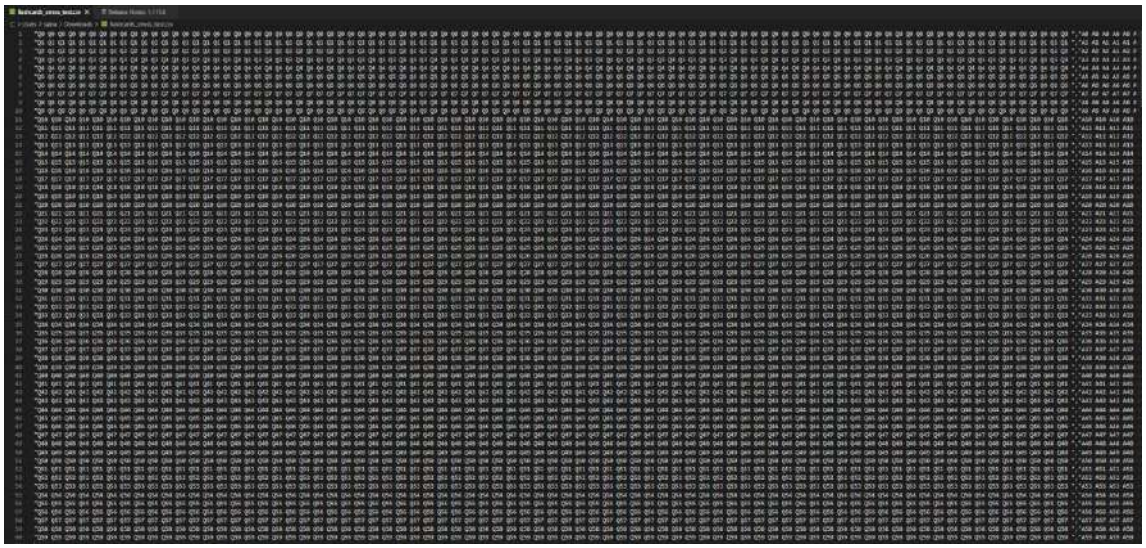


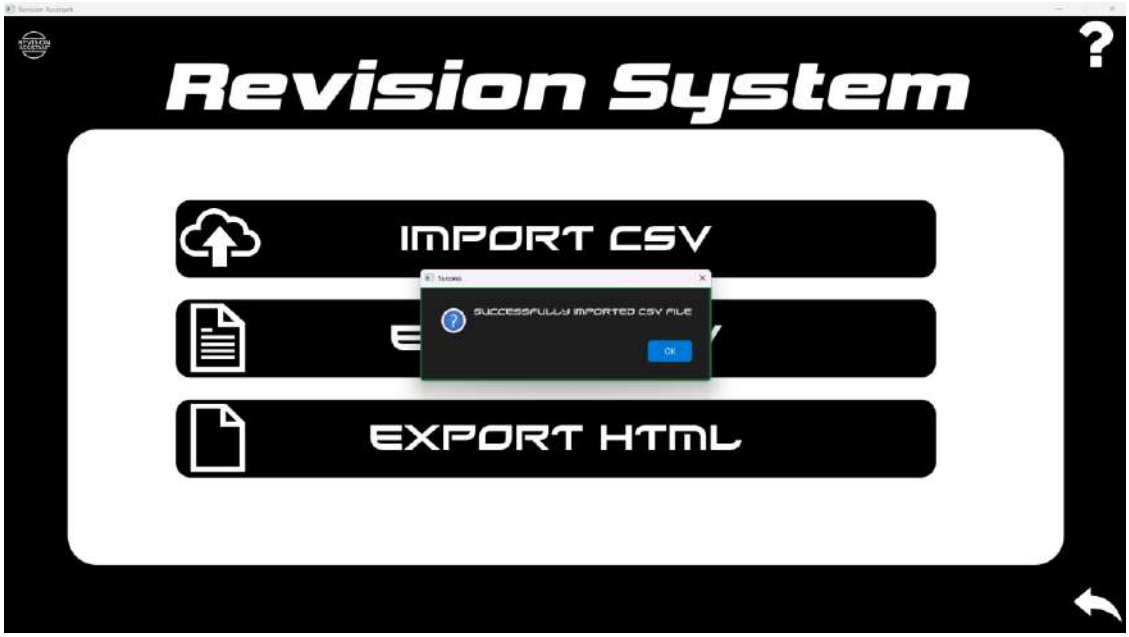
The outcomes match the expectation, and both structurally corrupted files were rejected on attempt to import.

4.1.1.1.3 Stress Testing Flashcard Import

For this test, I will import a flashcard deck of 60 cards (the maximum), with 300-character questions and 600-character answers, I will then test these cards in a session. I am expecting everything to work as normal.

I will import the following file:



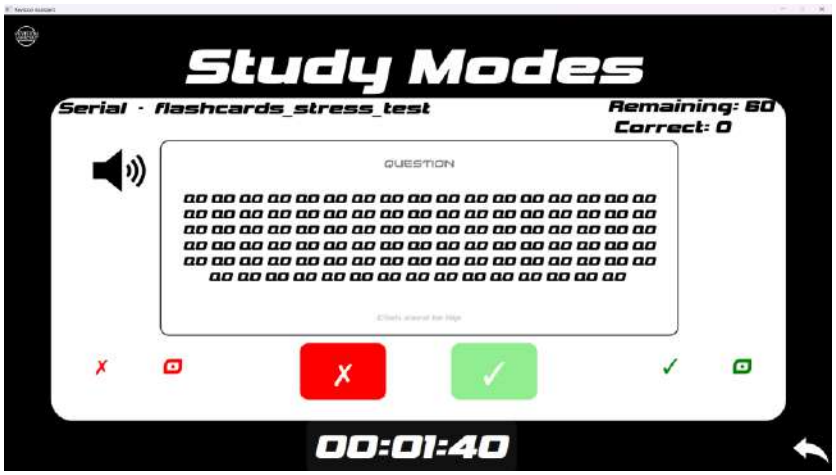


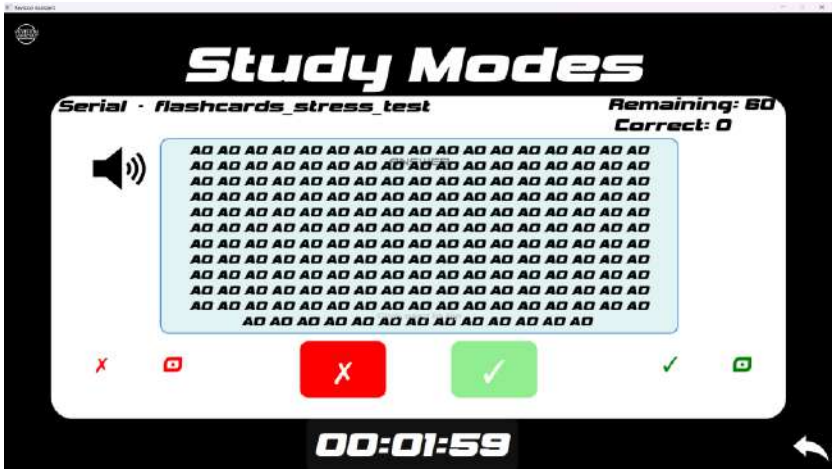
While importing this, I realised the CSV import was 120 rows instead of 60 rows; this was because in the importCSV, the read data was called twice (and I did not experience this issue during development) so the cardsList was added to twice.

```
// Procedure which allows the user to import a csv file into the system
public boolean importCSV(Window owner) throws IOException {  1 usage
    // Open the file chooser, filtered to csv files only
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Import CSV File");
    fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter( description: "CSV Files",  ...extensions: "*.csv"));
    // Store user's chosen file
    File file = fileChooser.showOpenDialog(owner);
    // If user doesn't choose a file, don't do anything
    if(file == null) return false;
    // Check size limits of flashcard deck
    ArrayList<Flashcard> cardsList = loadFCDeckFromPath(file.getAbsolutePath(),  isWrongAnswers: false);
    if(!validateCSV(file , fileName: "flashcard")){
        CustomAlert.error(owner,  message: "Invalid file format");
        return false;
    }
    if(cardsList.isEmpty() || cardsList.size() > 60){
        CustomAlert.error(owner,  message: "A deck needs a minimum of 1 card and can have a maximum of 60 cards");
        return false;
    }
    // Check if user is importing flashcard deck from another user, or one which already exists within their folder
    String allUserFolders = Paths.get(System.getProperty("user.dir"),  ...more: "userFolders").toString();
    if(file.getCanonicalPath().startsWith(new File(allUserFolders).getCanonicalPath())){
        CustomAlert.error(owner,  message: "Not allowed to import from this directory");
        return false;
    }
    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
        String importDeck = file.getName().replace( target: ".csv",  replacement: "");
        // Set the file name
        setFlashcardDeckName(importDeck);
        // Use previously initiated array list
        readData(br, cardsList,  isWrongAnswers: false);
        // Use default save method
        saveFlashcardDeck(cardsList,  append: true);
    }
    return true;
}
```

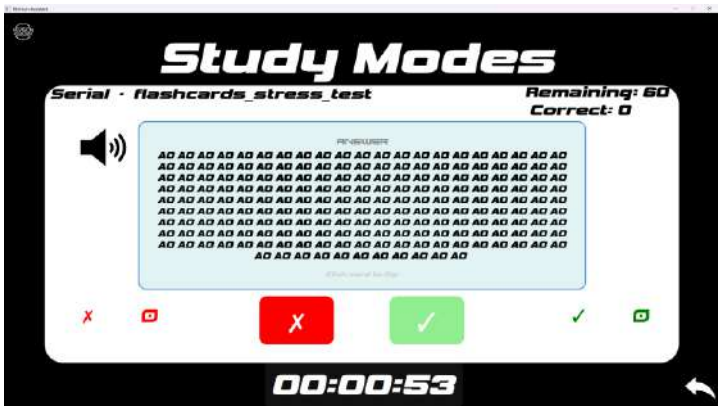
The cards were loaded into the ArrayList via the loadFCDeckFromPath method, and then read into the card list again, then finally saved. To remediate this issue; I will remove the highlighted line of code, so the cardsList is not populated twice.

Once again importing shows that the flashcard deck now only has 60 cards, however while studying the deck, I have noticed that when answers are near the character limit, the flashcard becomes very populated.





To remediate this, I changed the font size to 25px instead of 30px making the flashcard less populated to look like:



I then conducted a study session, and it worked as normal, hence the Revision Assistant has passed the stress test.

4.1.1.2 Corrupting System CSVs

In the following tests, to test the importation of the of system files, I will:

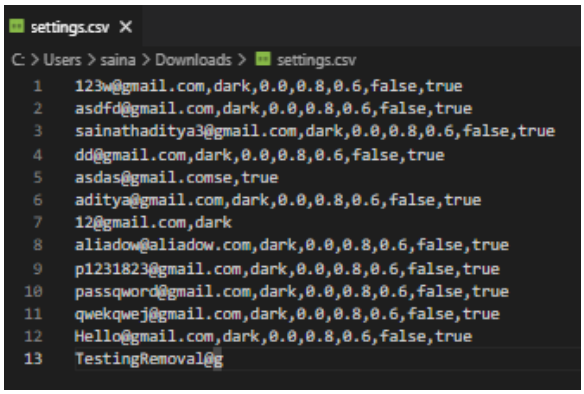
1. Export the files, change the structure and import once again, expecting an error to be displayed.
2. Import an empty CSV file, expecting it to be rejected on input.

4.1.1.2.1 Structurally Corrupting CSVs

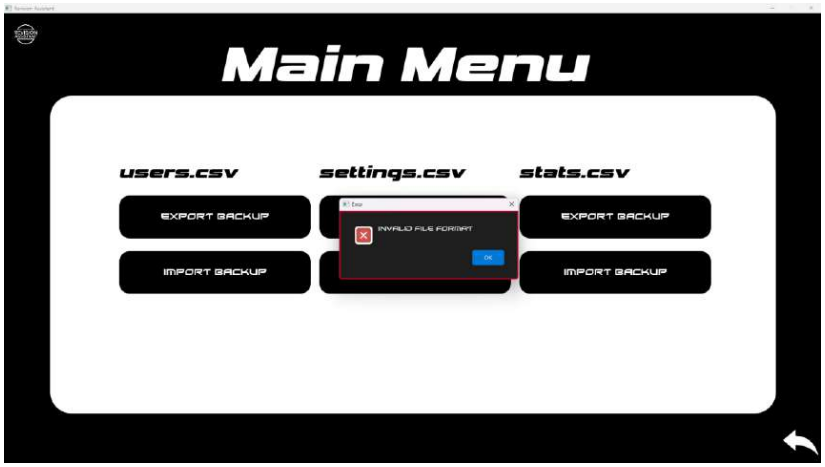
Initially, I exported settings.csv:



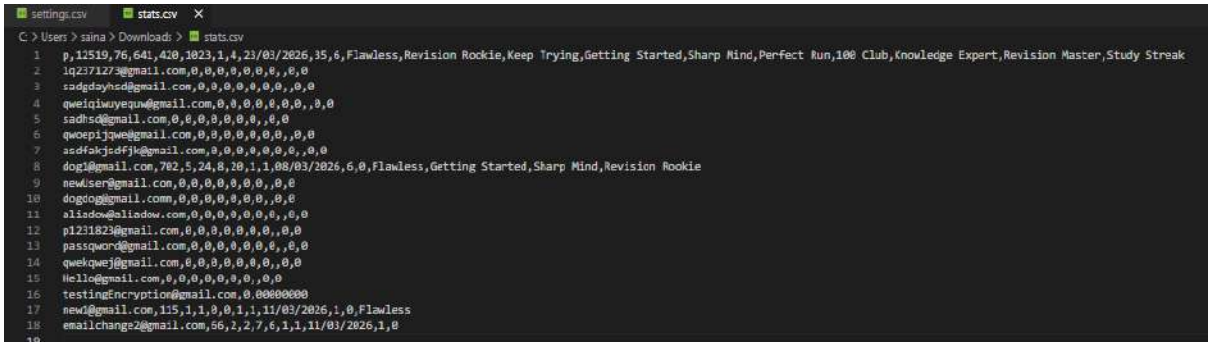
I have then corrupted the file by randomly deleting fields:



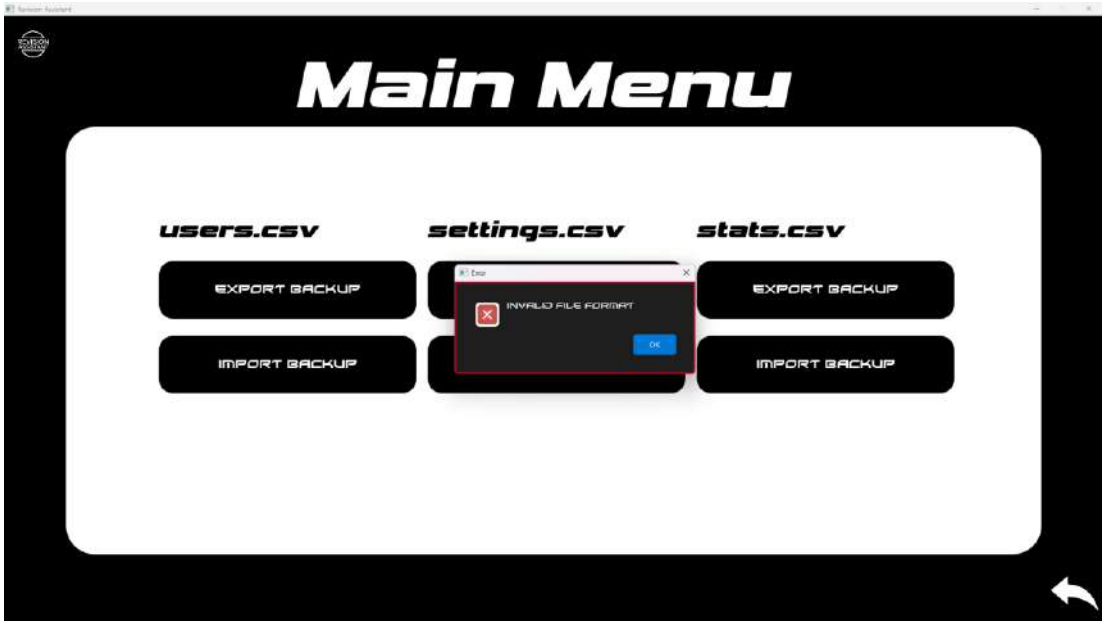
On import, the following error was shown:



I will now try to remove the delimiters from an exported stats.csv:



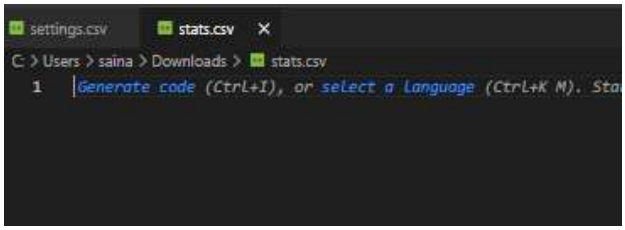
On line 17, I have removed the delimiters, now I will attempt to import this back into the system:

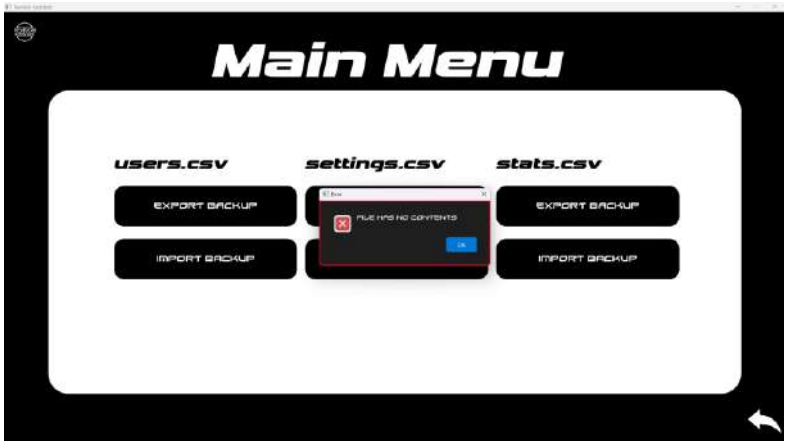


These tests prove the RevisionAssistant won't accept the most corrupted CSV files as inputs, and even if they do the quarantineCorruptedCSV from the development stage will label the file as corrupted and create a new empty one.

4.1.1.2.2 Empty CSV Import

I have cleared the content of stats.csv and will attempt to import:





The corresponding error is correct, and the file would not be accepted into the system.

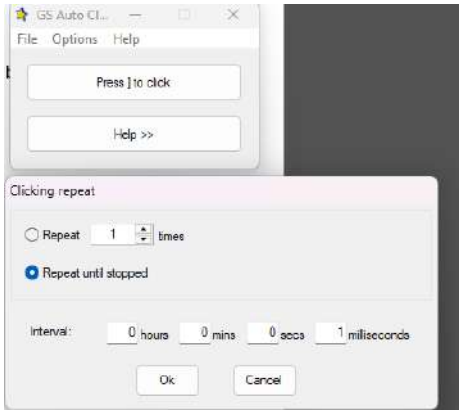
4.1.2 Spam Clicking

I will now destructively test the program by randomly clicking in the following scenarios:

1. Navigate the system with an auto clicker, expecting no duplicated UI elements and the state persists correctly.
2. Login to the system, find the closest back button and use an auto clicker on it, system should not take you back to AuthScreen.
3. Log out of the system, find the closest back button and auto click on it, user should not be returned to MainMenuScreen.
4. Input invalid data and spam click on add flashcard, expecting rejection and error displayed to user.
5. Spam click when saving flashcard data, expecting no invalid data to be stored and validation preventing the submission.

4.1.2.1 Navigating with an Auto Clicker

The auto clicker I will be using is GS Auto Clicker, set on left click to be clicked every 1 millisecond until stopped.

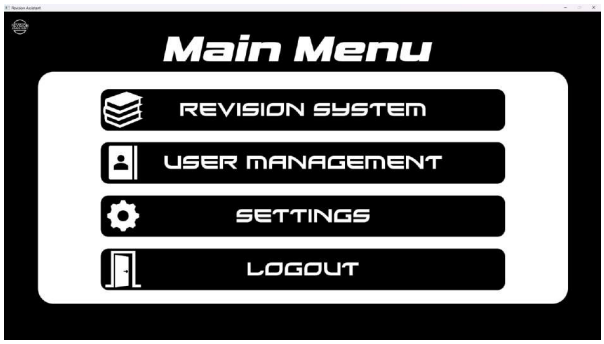


I will move my cursor around the buttons, allowing random navigation to see if there are any errors. Despite the auto clicking to navigate unpredictably, the application

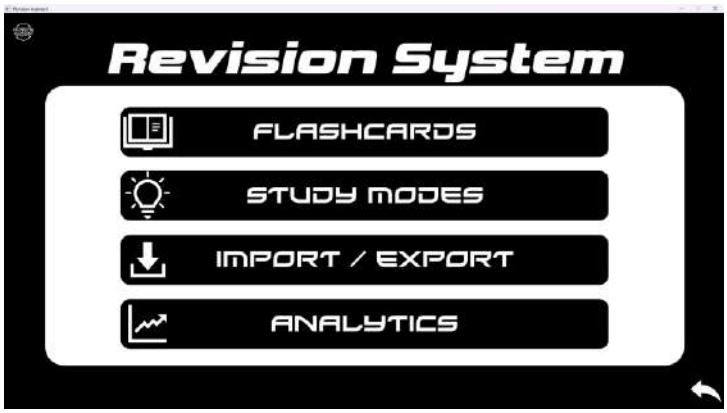
remained robust and there were no duplicated UI elements or state errors observed, hence the program has passed this test.

4.1.2.2 Auto Clicking after Login

First screen after login:



Finding closest back button:



I will now auto click the back button, expecting to be taken to main menu and nothing else more than that. Once again, under extreme clicking conditions, the system remained stable and returned me to the main menu screen and nothing else, hence the system has passed this test too.

4.1.2.3 Auto Clicking after Logout

After logout:



Closest back button:



I will now auto click the button which should take me back to the login page and should not return me to the state of being logged in. The test has succeeded and did not return me to the main menu screen; hence the revision system can handle spam clicks whilst navigating.

4.1.2.4 Inputting Invalid Data and Spam Clicking

I will now create a flashcard with invalid data and spam click add, expecting any input to be rejected.





Add could only be clicked once, since the error was not closed, hence the system has now withstood another spam click destruction test.

4.1.2.5 Spam Clicking when Saving Flashcards

I have added one flashcard to a deck, and will attempt to save this, spamming the save button initially and then then the “OK” button after I have inputted the file name. I expect only one file to be saved and no duplicate input alerts to appear.



After spam clicking save:

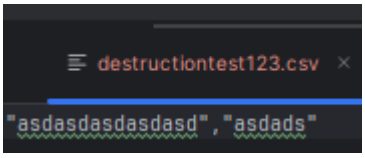


Only 1 input alert, now I will input a name and spam “OK”.



Within the actual folders, following the spam clicks there was only one file created and only contained the one flashcard I created:





Therefore, the program has passed another destruction test which means it can withstand spam clicks when saving data as well.

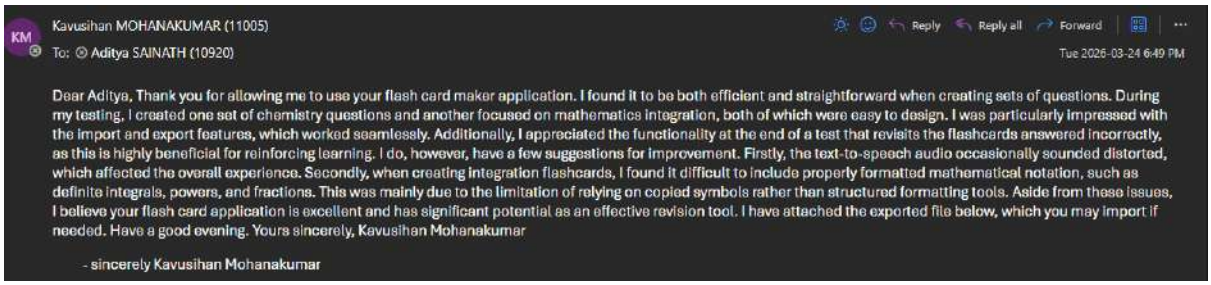
4.2 Beta Testing

Following alpha testing during development, as well as testing the robustness of the solution, the program was distributed to multiple potential users to gather unbiased user feedback on the system's interface design, usability features and any unidentified bugs during alpha tests. Some issues only arise during real user interaction (such as navigation difficulties); hence I selected a range of fellow students (all identified as stakeholders) to test the Revision Assistant. These stakeholders were given free roam of the application and were prompted to try features and provide unbiased feedback within 2 days.

4.2.1 Stakeholders / Beta Testers

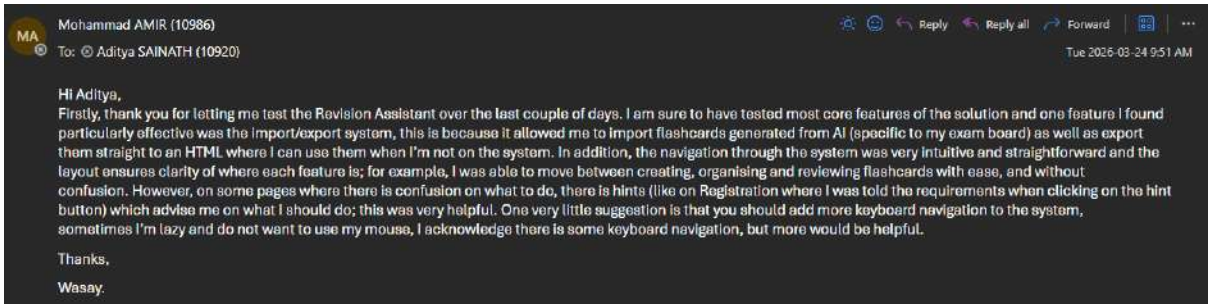
4.2.1.1 Stakeholder 1 Kavusihan Mohanakumar

Kavusihan Mohanakumar is studying Further Mathematics, Mathematics, Physics and Chemistry at A-Level; he was keen on testing the application since he uses flashcards to revise consistently, this makes him an appropriate representative for the target user group, allowing for relevant and reliable feedback on the program's overall effectiveness.



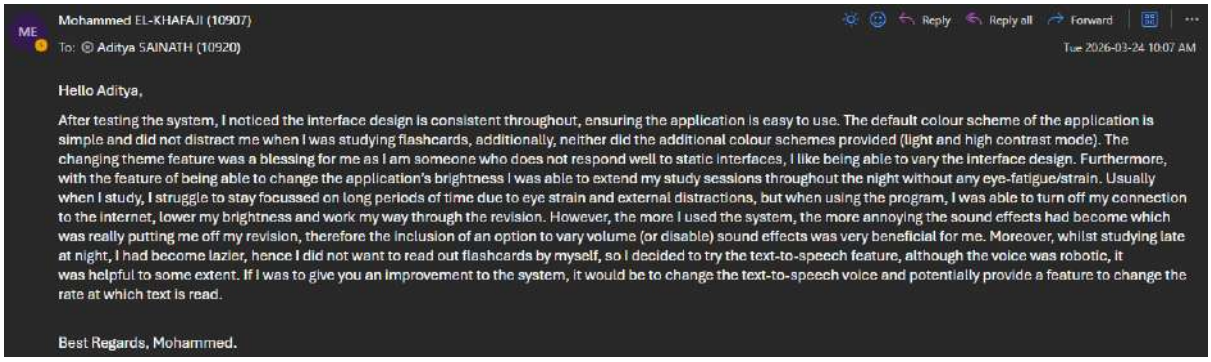
4.2.1.2 Stakeholder 2 Wasay Amir

Wasay Amir currently studies Mathematics, Physics and Chemistry at A-Level, unlike other testers he has made it clear to me he does not regularly make use of flashcards, since he finds them inconvenient to manage when in physical form. Therefore, this makes Wasay a valuable stakeholder, where he represents users who may be reluctant to use flashcard-based revision methods.



4.2.1.3 Stakeholder 3 Mohammed El-Khafaji

Mohammed El-Khafaji is an A-Level Chemistry, Biology and Psychology student. He informed me prior to testing that he tends to “cram” revision sessions, often late at night for longer periods. This makes him a relevant stakeholder to evaluate the Revision Assistant’s usability over long study sessions, focussing on how the features help with user comfort. His feedback will be essential in determining how well the application supports revision for long durations.



4.2.1.4 Stakeholder 4 Shamsudiin Mohamed

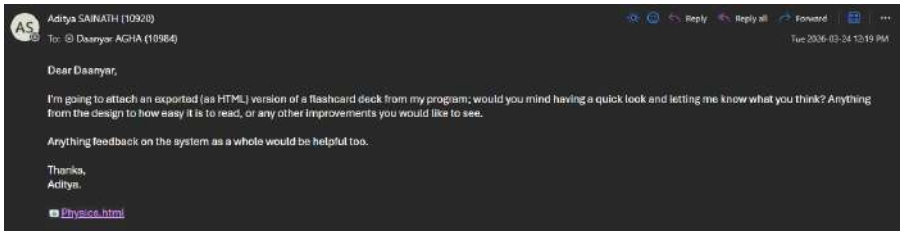
Shamsudiin Mohamed studies Mathematics, Economics and Business at A-Level, his strong interest in video games makes him suited to evaluate the elements of gamification, specifically their effectiveness. His perspective will help to assess if the achievements, progress tracking, and statistics assist in maintaining user motivation when studying. His feedback will also help in deciding whether there is a healthy balance between education and gamification.



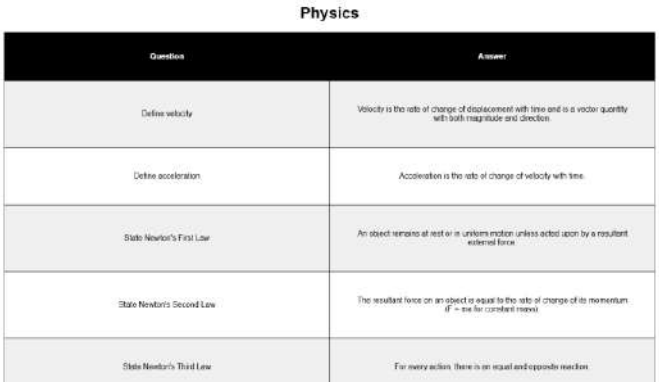
4.2.1.5 Stakeholder 5 Daanyar Agha

Daanyar Agha is an A-Level student studying Economics, Mathematics and Computer Science, due to his knowledge of Computer Science, he is a key stakeholder assist in evaluating the interface design as well as the quality of exported HTML files. Therefore, I assigned him with an extra task alongside beta testing the system which was to review exported HTML flashcard deck.

I sent the following email to him:

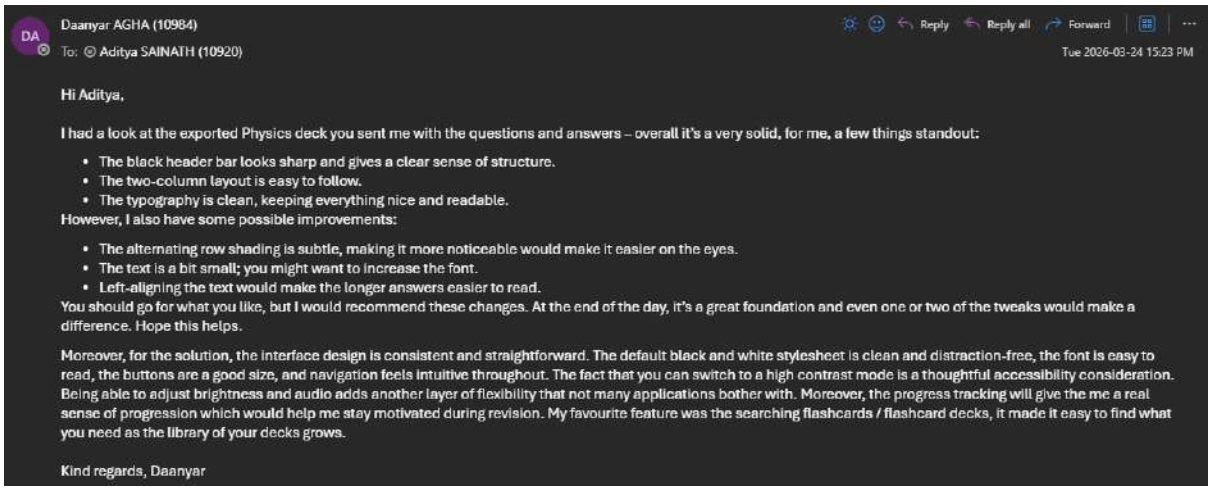


Attached with:



Physics

Question	Answer
Define velocity	Velocity is the rate of change of displacement with time and is a vector quantity with both magnitude and direction.
Define acceleration	Acceleration is the rate of change of velocity with time.
State Newton's First Law	An object remains at rest or in uniform motion unless acted upon by a resultant external force.
State Newton's Second Law	The resultant force on an object is equal to the rate of change of its momentum. $F = ma$ for constant mass.
State Newton's Third Law	For every action, there is an equal and opposite reaction.



4.2.2 Summary of Beta Tests

Strengths	Weaknesses
Intuitive Navigation – users find it easy to navigate through the system	Text-to-Speech Quality – voice is too robotic, distorted, "off-putting"

Good accessibility features – different themes, variable brightness and audio levels.	Text-to-Speech Rate – speed of talking should be variable
Effective gamification – the gamification aspects have helped with motivation successfully.	Keyboard Navigation – keyboard navigation is limited
Good import/export system – seamless import/export	Formatting Constraints – system lacks the ability to support mathematical notations natively
Helpful study tools – incorrect cards mode, progression statistics	Export as HTML's design can be improved
Clean, consistent interface – throughout whole system	
Helpful guidance when needed – hint buttons	

4.2.3 Implementing Suggested Changes

Following the beta tests, I was considering implementing all the suggestions as stated above in the "Weaknesses" column. At first, I saw three of these suggestions as viable but as I tried implementing, I realised for this project's scope only two were possible.

All the suggestions were reasonable and would increase usability of the system for the users, but due to technical limitations not all improvements were implemented. I plan on addressing these issues in future revisions of the system, with the focus right now being to deliver a fully functional revision tool catering to all users.

Keyboard Navigation:

- Although this is super beneficial for accessibility, only one stakeholder commented on the need for this, and this was due to him being "lazy". I acknowledge I have not had the Revision Assistant tested by users with motor disabilities or screen readers hence I cannot say for sure, but I believe the keyboard controls currently within the system is sufficient for all users. Implementing full keyboard navigation would require completely restructuring how input events are handled across all the screens within the application. This scale of changes to be made across the entire code made implementing this impractical within the remaining time.

Mathematical Notations:

- Implementing a feature to allow the user to input mathematical notations such as definite integrals would require the Revision Assistant needing to support LaTeX or MathML rendering, which would require the integration of a third-party library. This increases the complexity and file size of the program whilst also

introducing potential compatibility risks with the existing JavaFX UI code. In the current implementation, if users need mathematical notations, they can be copied from the internet and pasted into the text areas when making flashcards, as a work-around; cases where does not work is with sub-scripts, super-scripts, or definite integrals for example. This was considered an acceptable compromise to prioritise core functionality, usability and stability. Future iterations of the program could address this and add a dedicated mathematical rendering functionality if required.

Text-to-Speech Voice Quality:

- I attempted to replace MaryTTS (current TTS third-party library) with a less robotic, newer library called “Voices”, which makes use of neural engines/networks to create a more natural intonation and smoother output when speaking compared to MaryTTS which uses traditional speech synthesis techniques. Voices would have been a great solution since it is also lightweight and runs fully offline, supporting Success Criterion 8.
  The project was initially developed as a standalone JavaFX application with required dependencies manually added to the project’s library folder as JARs. The approach was chosen at the beginning of development as it gave me direct control over what libraries were included without needing extra knowledge of build configuration files. Build automation tools, e.g. Apache Maven and Gradle, are powerful for large projects with numerous external dependencies, however the solution is small and has limited external dependencies, hence the overhead of developing the solution with a build system would not have many benefits. However, this became a limitation when I attempted to implement a more complex TTS library; most modern TTS libraries are distributed through Maven Central and rely on numerous transitive dependencies which are automatically resolved by Maven at build time. Integrating the “Voices” library, which I tried, would require to individually download every single linked library and upload it to the library folder within the project. This becomes unmanageable given the number of dependencies involved. Hence, I decided against improving on the quality as of right now, if I were to develop the solution further in the future, switching to a build automation tool would be the logical first step, allowing for easier integration of more complex libraries. Switching to Maven right now would introduce unnecessary risks to an already functioning system.

4.3.3.1 Text-to-Speech Rate

Text-to-Speech Rate controlling was seen as a viable suggestion as it was not too complex and seen as necessary as different users process information at different paces. Making the rate adjustable would improve the accessibility of the system as well

as the learning experience being able to be personalised, allowing the Revision Assistant to support a larger range of users' learning needs.

TTS Rate control was implemented via adding increment and decrement options to the StudyModesScreen, where the user can vary the rate between 1.0 and 2.0 in steps of 0.1. Validation for this was within the TTSController where the rate was always clamped between 1.0 and 2.0.

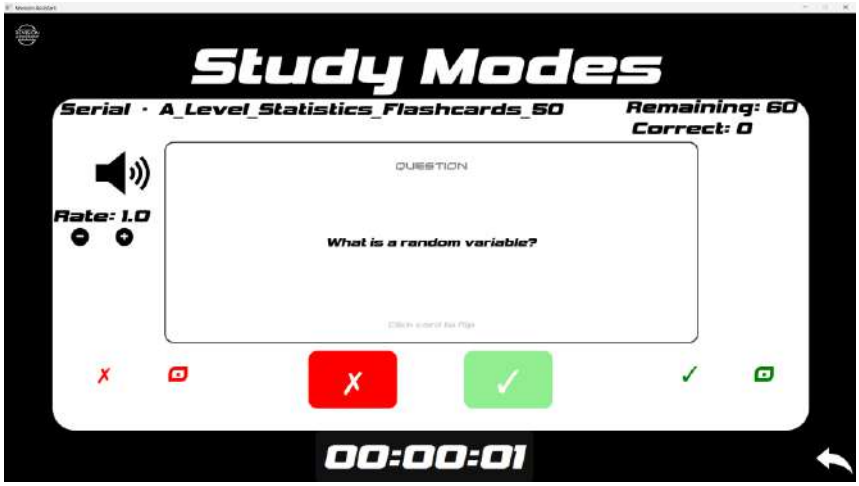
```
// Create rate vary option
Text rateDisplay = new Text("Rate: 1.0");
rateDisplay.getStyleClass().add("infoText");
vi.setPosition(rateDisplay, 110, 470);
// Create buttons
RectangleButtonBundle minus = vi.buildRectangularButtonBundle( rectW: 40, rectH: 40, text: "-", typeOfText: "smallButtonText", StackPaneX: 150, StackPaneY: 480);
RectangleButtonBundle plus = vi.buildRectangularButtonBundle( rectW: 40, rectH: 40, text: "+", typeOfText: "smallButtonText", StackPaneX: 250, StackPaneY: 480);
// Array value
double[] rate = {1.0};
// Reset per session
tts.setRate(rate[0]);
// On plus
plus.button().setOnMouseClicked( MouseEvent _-> {
    tts.setRate(rate[0] + 0.1);
    rate[0] = tts.getRate();
    rateDisplay.setText(String.format("Rate: %.1f", rate[0]));
});
// On minus
minus.button().setOnMouseClicked( MouseEvent _-> {
    tts.setRate(rate[0] - 0.1);
    rate[0] = tts.getRate();
    rateDisplay.setText(String.format("Rate: %.1f", rate[0]));
});
// If TTS is enabled, add the ttsIcon and rate option
Group ttsGroup = new Group(ttsIcon, rateDisplay, minus.button(), plus.button());
if(sc.getCurrentSettings().isTTSEnabled()) newContent.getChildren().add(ttsGroup);
```

StudyModesScreen Code 11 - added within buildSessionContent()

```
public void setRate(double rate){
    this.rate = Math.min(2.0, Math.max(1.0, Math.round(rate * 10.0) / 10.0));
}
public double getRate() { return rate; }
```

TTSController Code 5 - getter and setter for rate

This clamps the rate between 1.0 and 2.0



StudyModesScreen Image 4 - varying rate option

Testing has confirmed that the rate of the speech changes when the user changes the rate.

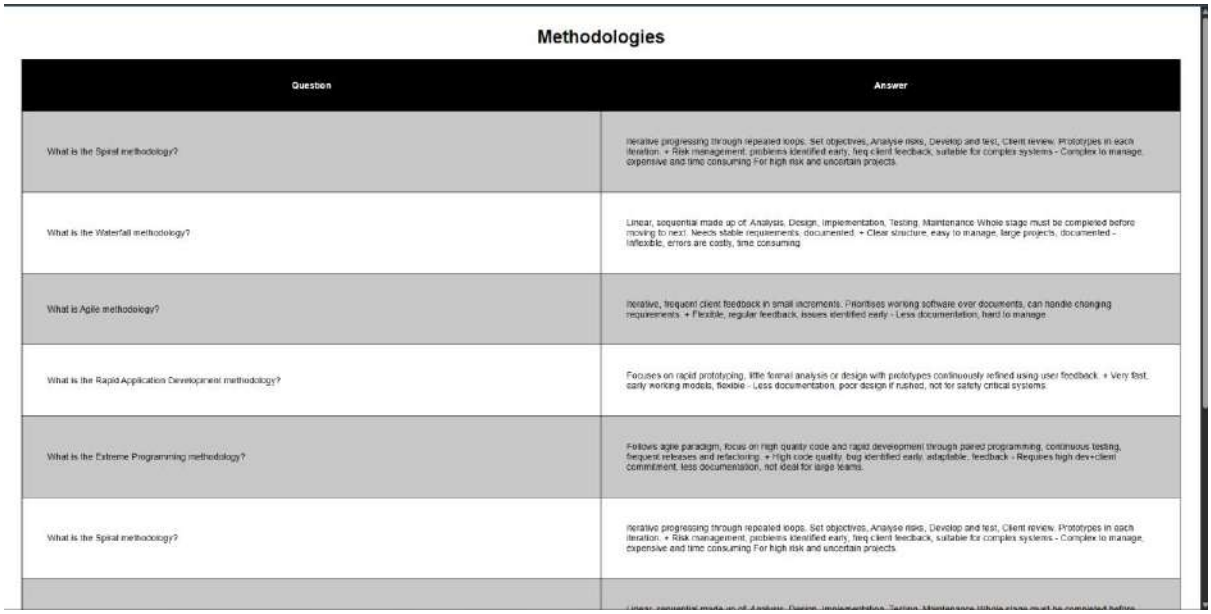
4.3.3.2 Exporting as HTML Design

The suggestions for the HTML design were taken into consideration, and I have left-aligned the text, made the difference between rows less subtle as well as increase the text size, these modifications were done via updating the HTML writing code.

```
// Procedure to write HTML File
private void writeHTML(BufferedWriter bw, String exportDeck) throws IOException {  1 usage
    bw.write( str: "<!DOCTYPE html>");
    bw.newLine();
    bw.write( str: "<html><head><title>" + exportDeck + "</title><style>");
    bw.newLine();
    bw.write( str: "body{font-family: Arial, sans-serif; margin: 40px; font-size: 18px;}");
    bw.newLine();
    bw.write( str: "h1{color: black; text-align: center;}");
    bw.newLine();
    bw.write( str: "table{width: 100%; border-collapse: collapse; margin-top: 20px;}");
    bw.newLine();
    bw.write( str: "th{background-color: black; color: white; padding: 40px; text-align: center; border-right: 1px solid black; " +
            "border-left: 1px solid black; border-bottom: 1px solid black; border-top: 1px solid black;}");
    bw.newLine();
    bw.write( str: "td{padding: 50px; border-bottom: 1px solid black; text-align: left; border-right: 1px solid black; border-left: 1px solid black; " +
            "border-top: 1px solid black;word-wrap: break-word; max-width: 250px; font-size: 18px;}");
    bw.newLine();
    bw.write( str: "tr:nth-child(even){background-color: #c8c8c8;}");
    bw.newLine();
    bw.write( str: "</style></head><body>");
    bw.newLine();
    bw.write( str: "<h1>" + exportDeck + "</h1>");
    bw.newLine();
    bw.write( str: "<table><tr><th>Question</th><th>Answer</th></tr>");
    for(Flashcard card : loadFlashcardDeck(exportDeck)){
        bw.write( str: "<tr><td>" + card.getQuestion() + "</td><td>" + card.getAnswer() + "</td></tr>");
    }
    bw.write( str: "</table></body></html>");
}
```

FlashcardController Code 25 - updated writeHTML procedure

Exporting a deck to test this gives:



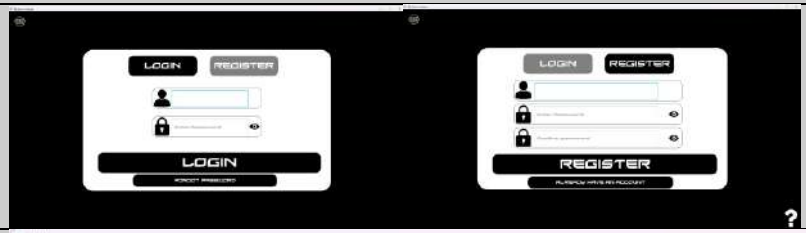
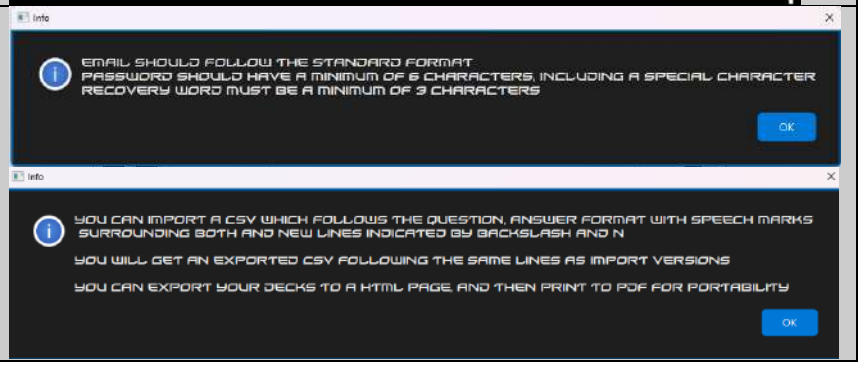
Methodologies

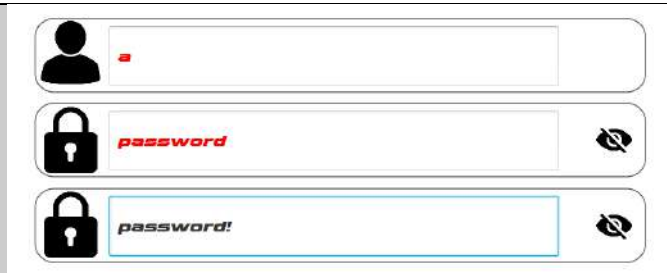
Question	Answer
What is the Spiral methodology?	Iterative progressing through repeated loops. Set objectives, Analyse risks, Develop and test, Client review. Prototypes in each iteration. + Risk management, problems identified early, freq client feedback, suitable for complex systems - Complex to manage, expensive and time consuming For high risk and uncertain projects.
What is the Waterfall methodology?	Linear, sequential made up of: Analysis, Design, Implementation, Testing, Maintenance Whole stage must be completed before moving to next. Needs stable requirements, documented. + Clear structure, easy to manage, large projects, documented - Inflexible, errors are costly, time consuming
What is Agile methodology?	Iterative, frequent client feedback in small increments. Prioritises working software over documents, can handle changing requirements. + Flexible, regular feedback, issues identified early - Less documentation, hard to manage.
What is the Rapid Application Development methodology?	Focuses on rapid prototyping, little formal analysis or design with prototypes continuously refined using user feedback. + Very fast, early working models, flexible - Less documentation, poor design if rushed, not for safety critical systems.
What is the Extreme Programming methodology?	Follows agile paradigm, focus on high quality code and rapid development through paired programming, continuous testing, frequent releases and refactoring. + High code quality, bug identified early, adaptable, feedback - Requires high dev+client commitment, less documentation, not ideal for large teams.
What is the Spiral methodology?	Iterative progressing through repeated loops. Set objectives, Analyse risks, Develop and test, Client review. Prototypes in each iteration. + Risk management, problems identified early, freq client feedback, suitable for complex systems - Complex to manage, expensive and time consuming For high risk and uncertain projects.

This shows the changes are effective and I will review it once again with user feedback in the future

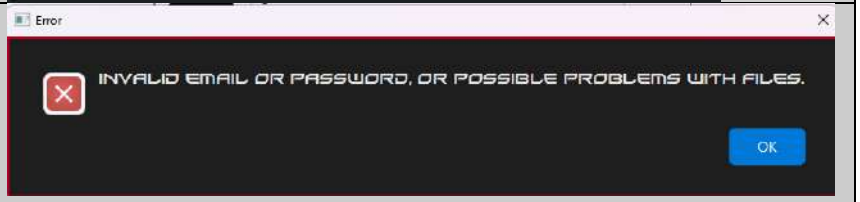
4.3 Usability Testing

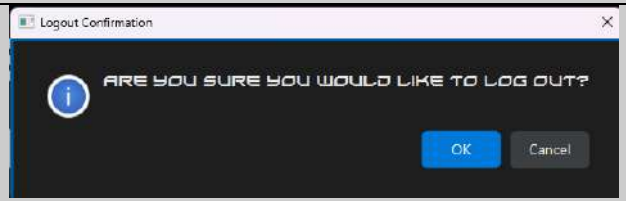
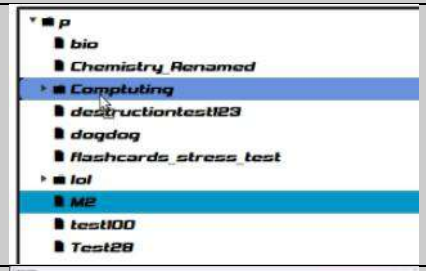
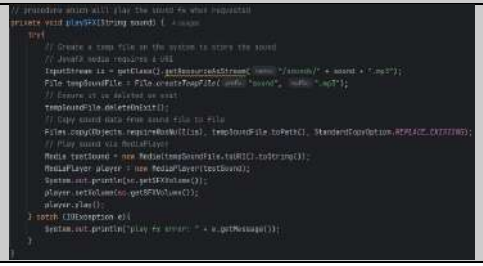
Usability testing is used to assess how easy a system is for users to interact with; this focuses on how user-friendly features implemented are.

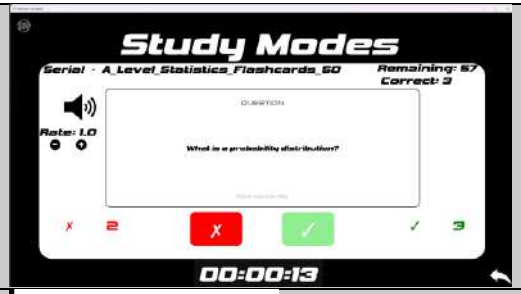
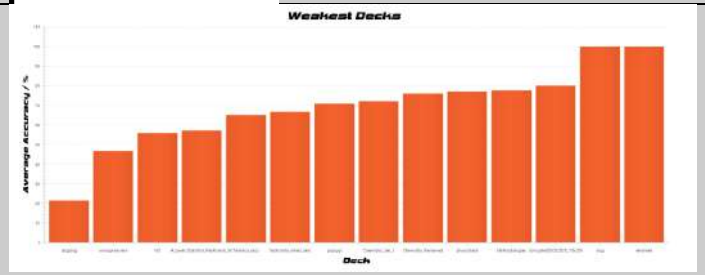
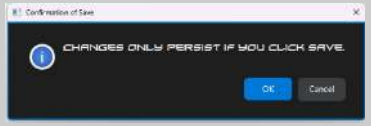
Feature No.	Usability Feature	Description	Proof
1.	Splash Screen	Beginning screen uses a fade animation and a rotating loading indicator to create an entry point to the system, giving the user an instruction to “Press any key to continue”.	
2.	Tab-Based Navigation	The AuthScreen uses Login/Register tabs with visual feedback. The selected tab will turn black, whilst the other is grey.	
3.	Tooltips	The user is offered tooltips in some potentially confusing screens, for example when the registering to inform the user about the registration requirements, or when exporting/importing to inform the user about the format to export in, or format which will be exported into.	

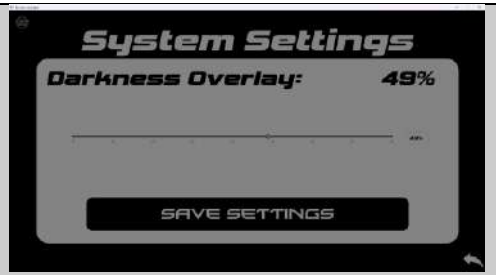
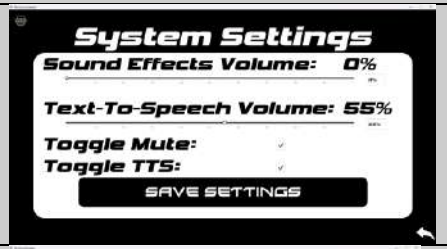
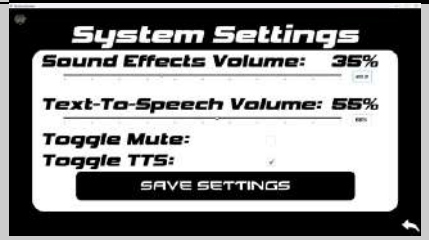
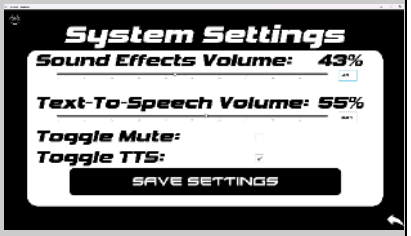
4.	Real-Time Input Validation	Input fields, text areas and the create flashcard TableView all make use of real time validators with visual feedback via colour connotations. Text appears red until valid, then switches back to black.	
5.	Password Show/Hide	Users can check their password entry by clicking the eye icon, which then changes to a different eye icon (implying hide). This reduces password entry errors.	

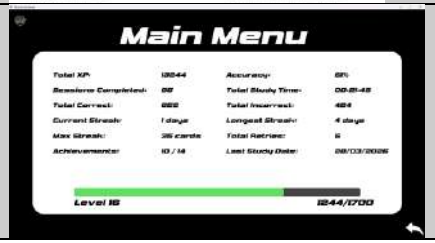
6.	Keyboard navigation within input fields	When dealing with input fields, the user can use TAB to move to the next field and ENTER to submit (for example within registration/login). This improves usability for screen readers or users with motor disabilities.	Done with the following method:
			`// Bind Enter Button to submit in Text Fields`
			`public void bindEnter(EventHandler<ActionEvent> handler, Node... fields) {`
			`  for (Node node : fields) {`
			`  if(node instanceof TextField tf){`
			`  tf.setOnAction(handler);`
			`  }else if(node instanceof PasswordField pf){`
			`  pf.setOnAction(handler);`
			`  }`
			`  }`
			`}`

7.	Alerts	Alerts are given to the user throughout the application to tell the user if something is wrong, if something has happened successfully, or information. This would be helpful for users with anxiety; users will not be left confused about next steps. An example of this being when logging in with the incorrect password:	

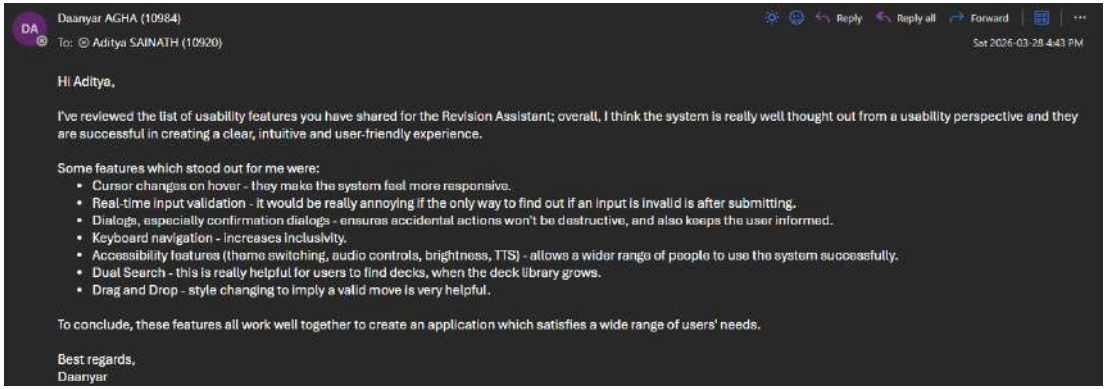
8.	Cursor Change on Hover	When hovering over buttons, the cursor changes from the point to a hand, implying the button is clickable.	
9.	Confirmation Request	Before the user performs destructive actions such as deck deletion, folder deletion, logout confirmation, session ending and file import. This prevents accidents leading to destructive actions.	
10.	File Drag-and-Drop Organisation	In the flashcard deck folder organisation, when attempting to drag, it would only work if the file/folder is not the root folder. The TreeView's cells are also visually highlighted if it is a valid drop.	
11.	Dual Search	Users can either search flashcards by their deck names, or the actual content within the flashcard deck. One search is disabled while another is active and this allows users to find decks easily.	
12.	Auditory Feedback	Sound effects are played when a user gets an answer correct, or incorrect; or finishes a session. This will give the user feedback possibly boosting their motivation.	

13.	Live Session Statistics	During a session, the user is provided with the counters (representing the incorrect and correct count, using western colour connotations), as well as timer and the number of remaining cards which all simultaneously update in real-time.	
14.	Text-to-Speech with rate control	To support users who prefer flashcards to be read to them, during a session the user can vary the rate at which the flashcard is read out loud to users. Some users may prefer speech to be slower than others, this can be changed within the session.	
15.	Visual Representations of Data	Users can view their session or all-time statistics visually or numerically. The data is plotted into different graphs such as Accuracy over Time, or a weakest topics bar chart. This essentially will help users recognise their areas of weakness.	
16.	Theme Switching	Users can pick between dark, light and high contrast stylesheets where a preview is applied prior to saving. If the user hasn't saved and attempts to go back, they were alerted that changes only persist when save button is clicked (also happens in the brightness and audio setting pages).	 

17.	Variable brightness	Users can change how bright the application is within settings without changing the brightness of their computers. This will help users prevent eyestrain when studying late at night, or it may just be a preference. The brightness also changes in real time to give the user a preview.	
18.	Audio Controls	Users can change the volume levels, as well mute with sliders, text fields and/or checkboxes. When the slider is released, the user is given a preview of the volume level by playing a test sound.	
19.	Input boxes	Alongside the sliders within the audio controls and brightness screen, the user has the option to enter a specific value through a text field, this acts in the same way as the slider but the user can precisely choose the value they want. Inputs are sanitised and validated, and valid inputs are updated consistently throughout the screen's contents.	 

20.	Achievements	Unlocked achievements are displayed in black and are adjacent to a trophy, whilst locked achievements are greyed out and are adjacent to a lock.	
21.	XP Bar	The XP bar shows the user how many experience points they need before they reach the next level, showing their current level as well as the XP they have for the current level.	

In order to evaluate the total effectiveness of the implemented usability features, I gathered feedback from a stakeholder, Daanyar. Daanyar was provided with the table above, allowing him to review each feature within the system and assess how effective they are in the context of the system.



Daanyar AGHA (10984)
To: Aditya SAINATH (10920)
Reply Reply all Forward
Sat 2026-03-28 4:43 PM

Hi Aditya,

I've reviewed the list of usability features you have shared for the Revision Assistant; overall, I think the system is really well thought out from a usability perspective and they are successful in creating a clear, intuitive and user-friendly experience.

Some features which stood out for me were:

- Cursor changes on hover - they make the system feel more responsive.
- Real-time input validation - it would be really annoying if the only way to find out if an input is invalid is after submitting.
- Dialogs, especially confirmation dialogs - ensures accidental actions won't be destructive, and also keeps the user informed.
- Keyboard navigation - increases inclusivity.
- Accessibility features (theme switching, audio controls, brightness, TTS) - allows a wider range of people to use the system successfully.
- Dual Search - this is really helpful for users to find decks, when the deck library grows.
- Drag and Drop - style changing to imply a valid move is very helpful.

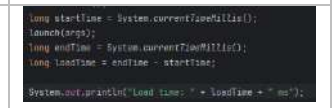
To conclude, these features all work well together to create an application which satisfies a wide range of users' needs.

Best regards,
Daanyar

Feedback from Daanyar indicated the features implemented were effective in improving the user experience. Moreover, feedback from initial beta testing emphasised how easy the system was to interact with, including intuitive navigation.

4.4 User Acceptance Testing

Using the test plan created in 2.4.3 Post-Development Testing, I will test the solution against the success criteria and then give users the program, they will use it and provide feedback on their experience, ensuring the system works correctly from the user's perspective.

Success criteria	Test data	Justification	Expected outcome	Actual outcome
1) The program should launch within 10 seconds on a recommended-spec (or higher) device.	Run the program 10 times, with each time outputting a load time and calculate an average.	Multiple runs reduce the effect of random error; hence a one-off case will have less effect. It also proves performance target is met.	Average <= 10 seconds	 The average load time was 833ms, hence this criterion has been met.
2) The Revision Assistant should display a splash screen when the application is launched, prior to registration/login screen.	Launch the program, check the splash screen is displayed first and can shift to the login screen immediately.	Confirms the application is loaded and responsive, the user controls the pace of the Revision Assistant from the very start, so they are in control of their revision.	Splash screen occurs on launch and will change immediately when a key is pressed.	The Splash Screen is the first screen the user sees, and it changes when a key is pressed.
3) The system should allow multiple users to register and log in on the same device.	Test by registering multiple users and attempting to log in to each account separately.	Confirms the solution can handle multiple accounts without the overwriting of data, hence confirming it can be used successfully on a shared device.	Each user is expected to log in successfully, with data saved persistently.	The system can handle multiple users and stores their data. No found data loading issues.
4) The program should not store duplicate email addresses and must not store passwords in plain	Check if registering multiple users with same email is possible. Open CSV and observe if	Accounts are less ambiguous and emails can act as a unique identifier, making sure the wrong user's data is	One email can be linked to one account; an error will be displayed. No password is	The same email cannot be used for two accounts and the users.csv file stores password

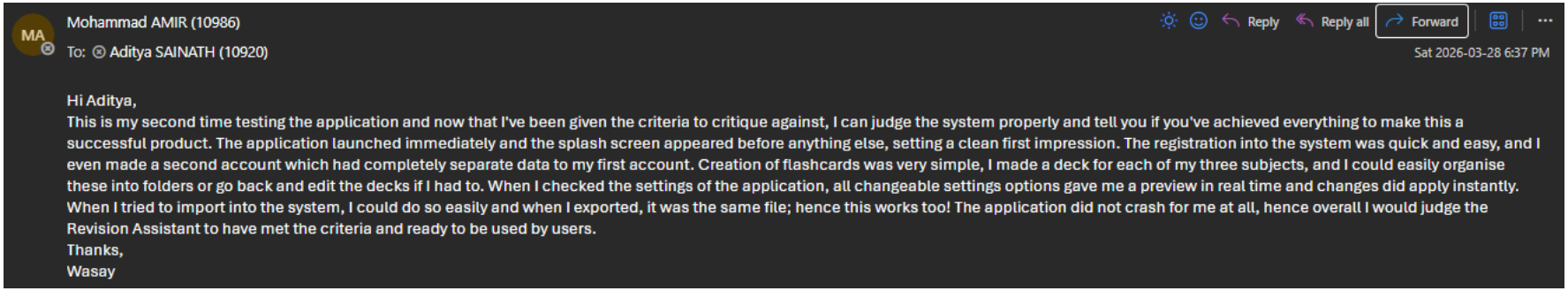
	text (store hashed password instead).	passwords are stored in plain text	never loaded to a user's account. Hashing will increase the system's security, protecting user credentials	stored in plain text, and salt is stored so same password does not allow produce same hash value.	hashes and is encrypted via AES.
5)	The solution will store data locally and must not crash if the data file corrupted.	Create an account, import a deck (corrupted and uncorrupted), study this deck gain XP, close the app and reopen checking if data still present.	This ensures the program is reliable for the user, and corrupted data will not destabilise the Revision Assistant.	Persistent data No crash, error message.	Corrupted data files are labelled as corrupted for users to check, and a new file is created. Data is stored successfully.
6)	The program will use a high contrast black and white style sheet for users with colour blindness, and will have other options available (dark mode, light mode, etc); changes must apply instantly.	Switch from high contract to light, mid sessions multiple times. Verify the text remains readable and changes are consistent across screens in each mode. Make the program output the time taken for this to happen.	Confirms the solution remains accessible, and theme switching will not affect one screen only. Ensures users with visual impairments are not suddenly exposed to situations which can hinder their ability to read.	Theme changes apply immediately and are consistent across different screens.	Users can change the theme, and a preview is given to them in real-time instantly prior to saving. After saving, the change is consistent throughout different screens.
7)	The solution should accurately measure XP and Achievements.	Run sessions with known outcomes. E.g. 10 correct and 0 wrong, 100 XP; 1 hour session, 500XP. Verify achievements are instantaneously	Using predictable data makes any issues in awarding XP obvious, hence easier to identify. Achievements must be trustworthy in the	10 correct with 0 wrong should give 150XP, XP matches the specification exactly. Achievements unlocked	The achievements are unlocked instantaneously and XP is allocated accurately.

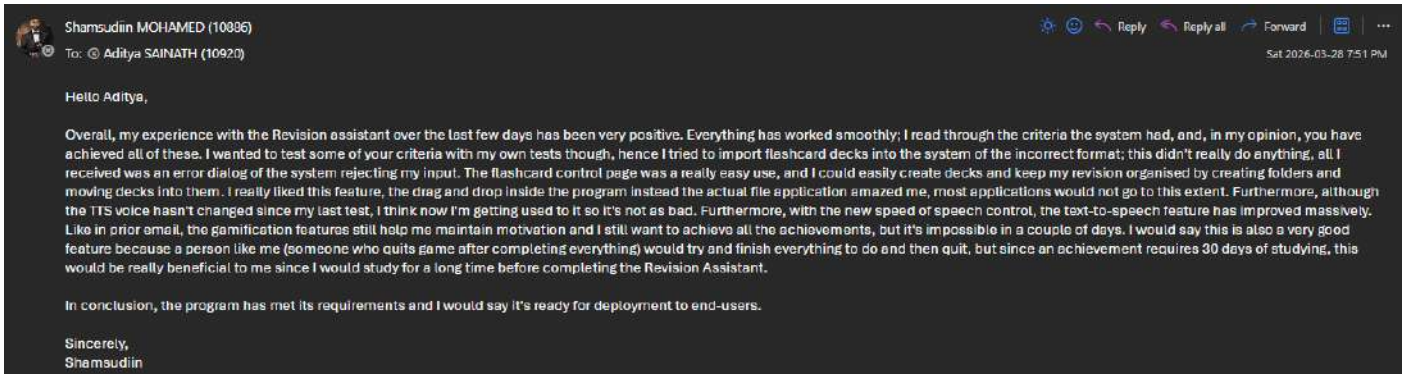
	unlocked when criteria are met (e.g. first session completed).	solution otherwise gamification fails.	immediately when conditions are met.	
8) The program must be available for offline use.	Disconnect from the internet, check functionality of the solution, launch, load a deck, run a session, export a deck.	Proves the solution can be accessed anywhere since there is no internet dependency, supporting stakeholders with unreliable connectivity.	All features of the system work as usual. No crash, or network errors.	When internet connection is disabled, all features of the solution work even TTS.
9) The program must allow users to import (csv) / export (csv/pdf) flashcards.	Using a valid CSV with 25 flashcards, import it into the system to check whether they all appear. Export the same deck as CSV and check if it is the same as the import. Export as PDF to check formatting.	Proves the core functionality feature of importing and exporting works successfully without any issues, hence users can have on-the-go revision or pre-made revision sets.	Valid CSV imports successfully and is exported correctly. Invalid CSV does not crash system.	Users can import valid CSVs, invalid CSVs given an error dialog. Users can also export as CSV or as HTML and then print to PDF.
10) The solution must have session time-tracking in hours and minutes.	Run a timed session, timing it externally (stopwatch). End the session and compare if the displayed time is matching the stopwatch time in the correct format (hh:mm:ss).	Proves the solution meets the criteria, students can measure their revision to help improve their study discipline and time-management.	Recorded time matches the stopwatch time and is displayed in the correct format (hh:mm:ss).	The session's timer is correct and displayed in hh:mm:ss.
11) The program must have volume sliders to turn on and off sound effects.	Set volume to different levels (0, 30, 60, 100) and try sound effects to see if volume slider has	Proves students who see sound as distractions can limit the effect it has on their studying, allowing	Volume level changes as requested by user with little to no	Volume changes influence sound effects as well as TTS. The toggle mute

	an effect. Toggle mute on and off.	them to focus on revision alone.	delay, saving if user presses save.	and toggle TTS also work. The changes are only saved if the user presses the save button.
12) The program must have folder organisation systems, allowing the user to pick which folders flashcard decks go in.	Create folders, saving decks into each of them. Move deck between folders, delete decks, restart system to see if data is persistent.	Proves students can organise their study material easily to prevent revision from becoming overwhelming.	Decks appear in correct folders and moving decks is correctly updated within files. Deleting is safe and gives warning to user that change is permanent. Changes remain post system restart.	The user can create flashcard folders, move decks within folders and delete folders (deleting the decks within the folder). The folder structure is updated correctly.
13) The solution must contain a valid text-to-speech feature.	Enable TTS and run 10 flashcards of varying lengths. Check if the deck is successfully read out loud by the TTS.	Proves the revision is accessible to a variety of needs, allows students to process information in an alternative way.	TTS reads information out correctly and will not show TTS options if TTS is disabled.	The flashcard's front and back are successfully read out, and the speech rate can be varied.
14) The solution must provide advanced analytics.	Run sessions with known raw results, compare session analytics with known data. Repeat with different data sets to check for consistency.	Proves students are allowed to easily identify their weaknesses and strengths in revision, so they don't waste time on learning already mastered content.	Analytics match raw results and persist post restart.	The analytics match the raw results and all changes persist after restarting.

15) The system must allow the user to create flashcard decks, full of flashcards, and allow the modification specific flashcards or the whole deck post-creation	Create a deck, save it. Modify the deck: 1. Add cards 2. Edit existing cards 3. Delete cards 4. Rename the deck Delete the deck	Proves the Revision Assistant expects revision to be an iterative process which the user will adjust over time, to make revision more effective.	Deck created and saved, cards added, cards edited, cards deleted, deck renamed and deck deleted successfully.	The deck was created; the cards were added and then edited. The cards were then deleted, and the deck was renamed.

As a part of the User Acceptance Testing, I will provide some stakeholders with the pre-defined success criteria and allow them to interact with the system independently, assessing whether the solution is successful and fully functional for target users.





The same users from the beta tests were provided with the updated version of the system (after the implementation of suggested changes) and this time given the success criteria to judge the system critically and assess whether it satisfies all the requirements necessary for the system, but also for themselves; whether they see it as a successful application without thinking about the criteria I provided.

Their feedback, in the emails above, indicates the application has performed reliably with core functionality (such as flashcard creation, editing and organisation) working perfectly. In feedback from the 4.2 Beta Testing, the users have also emphasised the system's intuitive user interface, with very easy navigation and understanding of all features, hence the program has strong usability. Moreover, one of the unwritten success criterions for myself when I was developing the Revision Assistant was to have a good balance of gamification and education, the gamification features have been positively received and genuinely help with maintaining motivation and engagement in the longer run.

Although there is a lot of positive feedback, improvements have been suggested throughout the time I have given the stakeholders access to the Revision Assistant, however these do not impact the core functionality of the system. Overall, based on user feedback, the system has met its developer predefined requirement as well as the user defined requirements, hence the program has been successfully accepted by the user.

4.5 Success of the Solution

This section will provide a comprehensive evaluation of the Revision Assistant, the success of the project will be determined, alongside any possible adjustments in future iterations of the solution.

4.5.1 Review against Success Criteria

The following table cross-references the fifteen success criterions (defined during analysis) against the implemented solution. Each criterion will be assessed as met, met partially or not met at all.

Criteria	Met?	Where?	Explanation
1. The program should launch within 10 seconds on a recommended-spec (or higher) device.	Y	All prototypes, End of Development	Time taken to launch was measured during Evaluation and an average of 800ms was recorded.
2. The Revision Assistant should display a splash screen when the application is launched,	Y	Prototype 1	Splash Screen is the first screen the user sees when they load the application.

prior to registration/login screen.			
3. The system should allow multiple users to register and log in on the same device.	Y	Prototype 1	Multiple users can register on the system, with their data stored separately.
4. The program should not store duplicate email addresses and must not store passwords in plain text (store hashed password instead).	Y	Prototype 1, Prototype 5	Emails are the primary key essentially, cannot be duplicated; users.csv stores hashed passwords and is encrypted.
5. The solution will store data locally and must not crash if the data file corrupted.	Y	All prototypes	Data storage is used throughout all prototypes and there are methods in place to handle corrupted data.
6. The program will use a high contrast black and white style sheet for users with colour blindness, and will have other options available (dark mode, light mode, etc); changes must apply instantly.	Y	Prototype 2	User can switch between themes easily, they are given a preview and changes apply instantly
7. The solution should accurately measure XP and Achievements.	Y	Prototype 4	XP and Achievements are allocated to users accurately.
8. The program must be available for offline use.	Y	Prototype 4	I made sure to use an external TTS library which do not rely on internet connection and the solution functions completely offline.
9. The program must allow users to import (csv) / export (csv/pdf) flashcards.	Y/P	Prototype 3	The users can import files as CSV and export as CSV. However, exporting as PDF is not as straightforward as expected, the user must export to HTML and then print to PDF,

			which is better in terms of formatting.
10. The solution must have session time-tracking in hours and minutes.	Y	Prototype 4	The time is displayed to the user during the session as well as at the end of the session.
11. The program must have volume sliders to turn on and off sound effects.	Y	Prototype 2	The user can control the sound with sliders and choice boxes.
12. The program must have folder organisation systems, allowing the user to pick which folders flashcard decks go in.	Y	Prototype 3	The user can drag and drop decks into newly created folders, whilst also creating subfolders to organise data even further.
13. The solution must contain a valid text-to-speech feature.	Y	Prototype 4	The user can use TTS if it is enabled and it works as expected, but quality can be improved.
14. The solution must provide advanced analytics.	Y	Prototype 4	The program displays multiple graphs visualising the user's data, such as weak decks.
15. The system must allow the user to create flashcard decks, full of flashcards, and allow the modification specific flashcards or the whole deck post-creation.	Y	Prototype 3	The user can create flashcard decks and then modify individual flashcard post-creation.

4.5.2 Criteria Consideration in Future Development

This subsection will discuss how the Revision Assistant can be expanded upon in future iterations of development, adding onto the foundation built by the current implementation.

All 15 success criteria defined during analysis were successfully met with the implementation of the solution however the system does not provide direct PDF exportation as Success Criterion 9 requires (export to HTML and then print to PDF as a workaround was used). While this is a functional workaround, it relies fully on the availability of a browser to parse the HTML which is not always ideal. In further development, a direct PDF export can be integrated into the system using third-party

libraries such as iText (which was initially avoided to reduce the complexity of the program as well as keeping program size small). External libraries such as iText would allow the same table generated by the writeHTML procedure to be written into a PDF and hence would allow the user to directly export decks into PDFs.

Moving away from the initial criteria, there are numerous ways in which the Revision Assistant can be expanded and improved in future iterations. For example, adding:

- **Spaced Repetition Algorithm:** Currently, when the user gets a flashcard wrong, it is added to the end of the queue, this ensures incorrect answers are reviewed until correct within a single session, but it does not apply weighting to flashcards. A card which has been answered incorrectly once is treated identically to a card which has been answered incorrectly 3 times, for example. Existing solutions such as Anki or Quizlet address this issue by implementing a spaced repetition algorithm. To implement this feature, I would migrate from the binary answer system of correct or incorrect, and for example, include an HBox of numbered buttons for the user to rate the flashcard based on how well they remember it (1-5; 1 being not at all, 5 being the fully known). I would then have to attach an easiness factor and interval to each individual flashcard and after each answer the algorithm would adjust these values based on the rating given by the user, scheduling the card to reappear within the queue quicker (smaller interval) if rated 1, or slower (larger interval) if rated 5. This would improve the effectiveness of revision by allowing users to focus on cards that they struggle with.
- **Database Migration:** In the current implementation, all persistent data is stored in CSV files, although this approach is functional and appropriate for the scope of the project, it will introduce many limitations as the system scales. In further development, I would migrate to a database to confront these limitations. Relational data such as users, settings, and stats could be stored in normalised tables with proper foreign key relations, which the data stored in nearly in the correct form of right now. This would eliminate the need for manual email-based searches as well as rewrites across multiple files (for example when the user changes their email). Switching to a lightweight embedded database (such as SQLite) would also replace all the file input and output logic within AuthenticationController, SettingsController, StatsController and FlashcardController with Java Database Connectivity (JDBC) calls.
- **Rich Media Flashcards:** The flashcard model in the current implementation only stores the question-and-answer fields, hence all flashcard content is restricted to plain text. For subjects such as the sciences, you may have to remember diagrams, or generally having the ability to include images on a flashcard would increase utility significantly. Extending object and to support rich media such as images would require adding an image path filed to the

Flashcard object, and updating the save and load method to handle this field, as well as updating the card rendering logic in buildCard to display an ImageView when there is an image to be displayed; the flashcard creation interface should also let the users to attach images to flashcards via a FileChooser. Images are useful to users, and it is proven that revising with images would engage the creative and logical side of the brain making information easier to store and recall.

- **Text-to-Speech Quality:** Users given the current solution have commented on the quality of voice speaking the text, although the feature is fully functional, they have said the voice produced by MaryTTS is robotic and not up to the standards set by newer contemporary neural TTS systems. Whilst implementing suggested changes, an attempt was made to switch to an alternative TTS library, Voices, but I was unsuccessful in this due to dependency conflicts with the manual JAR-based approach. Without a build tool, such as Apache Maven, integrating third party libraries become unmanageable, it is impossible for dependencies to be resolved automatically, ensuring all required libraries are downloaded. In future iterations of the program, the first step would be to migrate the project to Maven, declaring all dependencies within pom.xml, eliminating the need for manual library management. Once this is put in place, I can integrate a higher quality TTS library like Voices, which does not conflict success criterion 8 and works offline.
- **Mathematical Notation Support:** Flashcard content currently is stored and displayed as plain text; the Flashcard model stores question and answer fields as SimpleStringProperty fields, and these are converted to be rendered as text in buildCard. Users, who study subjects with scientific or mathematical notations, would have to rely on copy and pasting these notations from a search engine, or using alternatives such as 2 to the power of 3 being displayed as 2^3 instead of 2^3. This would be very difficult to read for users when using flashcards. In further development of the Revision Assistant, I could attempt to develop the solution to support LaTeX syntax by integrating JLaTeXMath (an external library). Flashcard data would then have to be rendered via JLaTeXMath instead of plain Text nodes. This provides users with flexibility to create flashcards with notations which would be visually ambiguous if represented in plain text.

4.5.3 Review against Usability Features

In this section, the usability features implemented during the development phase are critiqued against their effectiveness in the solution. This will be assisted with the evidence from 4.3 Usability Testing. Each feature will be labelled as either a success, partial success or failure with justification provided based on user feedback and observed performance.

Feature No.	Usability Feature	Effectiveness	Justification
1.	Splash Screen	Success	The Splash Screen does what intended, providing a clear entry point into the system and let revision start at the user's pace.
2.	Tab-Based Navigation	Success	Users can switch between register and login pages intuitively; the colour change clearly indicates the page the user is on which reduces confusion whilst increasing navigation efficiency.
3.	Tooltips	Success	Tooltips effectively help support new users in complex pages of the application (e.g. first-time registration), reducing uncertainty in actions and preventing errors.
4.	Real-Time Input Validation	Success	Immediate feedback on inputs allows the user to correct the errors before input instantly, reducing the possibility of invalid input. Colour connotations differ per country, but the Revision Assistant follows the western world's colour connotations.
5.	Password Show/Hide	Success	This will reduce the amount of password entry errors; it allows the user to verify prior to input. The icon change also provides clear affordance of the feature.
6.	Keyboard navigation within input fields	Success	This improves accessibility for users with motor disabilities and allows faster interaction with the system without the need of a mouse.
7.	Alerts	Success	The user will always be informed of the system state, this will reduce the confusion when interacting with the system while improving user confidence.
8.	Cursor Change on Hover	Success	Clickable elements are emphasised, improving the intuitiveness with very little cognitive load.
9.	Confirmation Request	Success	This prevents accidental actions from the user leading to destructive events within the system, critical for usability.
10.	File Drag-and-Drop Organisation	Success	The user can easily test which moves are valid and which are invalid via visual feedback.

11.	Dual Search	Success	User can flexibly search, improving efficiency in finding decks.
12.	Auditory Feedback	Partial Success	Increases engagement and may be helpful for some users, however not all users benefit from sound and can be distracting (but this issue is remediated with audio control).
13.	Live Session Statistics	Success	The user is given immediate feedback on their performance during the revision session, helping them stay engaged during sessions.
14.	Text-to-Speech with rate control	Success	Improves accessibility for auditory learners or users with reading disabilities (e.g. dyslexia). The adjustable rate ensures users can personalise their studying even further.
15.	Visual Representations of Data	Success	Graphical data is easier for users to identify weaknesses as well as trends, which can be used to assess their revision strategies.
16.	Theme Switching	Success	Improving accessibility; the system supports users who require high contrast modes, and also a point for further personalisation of the system. The preview also gives user an idea of change they make before saving.
17.	Variable brightness	Success	Helps reduce eyestrain and improves user comfort in different scenarios (different levels of lightings), and a live preview also extends usability impact.
18.	Audio Controls	Success	The user has full control over the sounds played in the system, audio features can be disabled or personalised.
19.	Input boxes (for sliders)	Success	This allows for precise inputs, improving flexibility and the validation prevents invalid values.
20.	Achievements	Success	Increases motivation through gamification, but the impact gamification has depends on the users; users which have tested the program has concluded these features as successful in retaining motivation to study.
21.	XP Bar	Success	

4.5.4 Usability Features for Future Development

In this section, although the Revision Assistant has a variety of usability features in its current implementation, enhancements were identified during the evaluation stage, I will propose usability features which will improve the overall user experience, accessibility, efficiency and accessibility of the system. Initially from the table above, I will suggest some improvements which can increase a feature's effectiveness, for example:

- **Splash Screen:** Add an option for users to disable the splash screen, might be frustrating for returning users and delays their study session.
- **Achievements:** Create new achievements (essentially quests), depending on the user's recent study sessions, for example if they are struggling on a certain deck, there will be a quest/achievement for them to complete a session with these cards with their accuracy > 85%.

Moving away from the already implemented usability features, additional usability features I can add to the system are:

- **Migration to Full Keyboard Navigation:** Currently, the majority of interaction with the solution is mouse-driven, meaning users who prefer or require keyboard-only navigation are unable to operate without a mouse. Migrating to full keyboard navigation would increase the accessibility of the Revision Assistant, for example during a session, a flashcard could be flipped using the Space key, enter to mark a card as correct, and backspace to mark a card as incorrect. This would also speed up the revision experience, increasing the fluidity for users who prefer to use their keyboard.
- **Additional Configurability:** Within the current implementation audio, brightness and stylesheets are the only features configurable. An example of where additional configurability could come into play is when flipping a flashcard. The card flip animation last 300ms all together (150ms for each side of the flip). An option could be added to adjust or disable the animation giving the users a feel over the interface. Some users may find the animation distracting or slow, and other potential users with visual sensitivities may benefit from disabling the animation as a whole.
- **Onboarding Tutorial:** Even though stakeholders, when beta testing, have emphasised the ease of navigation and the intuitiveness of the interface, some new users may struggle when trying the system for the first time. A first-launch tutorial would guide new users through the core functions of the Revision Assistant. This significantly improves the learnability of the system and reduces new users' initial cognitive load.

4.5.5 Limitations of the Solution

Although the solution has met all 15 predefined success criteria, there are several limitations to the current implementation. These limitations are not failures of the solution; they are rather boundaries of the current design scope (areas where simplifications were chosen instead of implementing a complex complete solution). Nevertheless, they are important to identify to evaluate the program and set a foundation for further development.

- **Self-Assessed Answers:** The revision of flashcards within the application relies on self-assessment which can be subjective. The user selects the tick or cross depending on their judgement of whether they met the flashcard's answer or not. There is no comparison between their answer, recalled from their memory, and the actual displayed answer, hence the system cannot detect partially correct answers, putting all responsibility on the user for honest marking. Alternatively, I could have implemented a feature where the user inputs their recalled answer, which is then compared again the card's answer, but this would be ineffective because the application would not be able to compute spelling mistakes or semantically equivalent statements without the use of a neural model, increasing the complexity of the project.
- **No Inactivity Handling:** Once a user is authenticated, their session will persist until they logout. The AuthenticationController stores the current user in memory indefinitely. On shared machines, like school computers, this means if users forget to log out, the next person to use the computer would have full access to the previous user's account and all of their data. This is because the AuthenticationController does not have an expiry mechanism, or inactivity detection so if users do forget to log out, their data can be seen by anyone with access to the computer.
- **Wrong Answers Mode:** When a user gets a flashcard wrong, it's written into wrongAnswers.csv, and overtime this file will grow and this file would store every single card the user has ever gotten wrong. Although they can easily modify this file and delete cards, they no longer struggle on, it would be better for an algorithm to go compare the users' performances per flashcard. For example, the system could track how frequently a card is answered correcting after being answered incorrectly initially and then remove these cards from the wrongAnswers.csv if the correct count is above a specified threshold, this reduces unnecessary repetition for the user.
- **Individual Card Tracking:** The statistics system tracks session level data and then accumulates it into the historical statistics. The lack of individual card performance tracking leads to the system not being able to identify specific areas the user struggles on. Realistically the value of statistics is limited and flawed, a user can see their overall accuracy being above 80% but it's possible

they have gotten one card wrong on every single session of the deck. To implement this feature, it would require the storage complexity and migrating to a database.

- **Limited Protection against Data Corruption:** The current solution has the quarantineCorruptedCSV method which detects corrupted CSV rows by catching exceptions when reading the data such as incorrect number of rows, or non-numeric values in numeric fields and it would move the file to a timestamped backup for inspection by the user later, replacing it with a new empty CSV. This method does not attempt to recover the valid data from the file, just avoids a crash. Apart from this, important system files can be backed up as a cautionary approach. If the user does not back-up manually, there is a chance they lose all their data if they were not proactive enough. A robust strategy to solve this issue would have automated incremental back up triggered regularly to ensure if data is lost, it would not be a large amount providing automatic proactive backup in case of data loss.

4.5.6 Maintainability Issues

This section will describe issues that do not affect the system functioning correctly currently, but each represent issues within the code that would increase the cost of future changes, introducing risk when modifications are made. Maintainability is the ease at which code can be understood, extended (or modified) and debugged by a developer (not only the original programmer) at a later date.

- **Hardcoded UI Coordinates:** All positioning of UI elements is done using absolute pixel coordinates. For testing purposes, the numbers were written into the code; to increase maintainability of the solution these values were extracted into constants which were declared at the top of the code. Although the UI elements are still hardcoded, their identifiers explain what they are and make the code more readable. However, any changes to layout would require manually positioning the new layout features, which would become very inefficient. I have made use of JavaFX layout managers such as VBox, HBox, Panes, etc but a lot of elements are still positioned with hardcoding.
- **Hardcodes AES Encryption Key:** In the AuthenticationController, the AES-128 encryption key (which is used to encrypt and decrypt users.csv) is stored as a hard coded string. Whilst the encryption does provide protection against casually inspecting the CSV in their computer's file manager, the encryption key being visible in the source code undermines the security guarantee as a whole. Any user with access to the program's JAR can decompile the Java Bytecode and use this key to decrypt users.csv. An alternative method, which would be more maintainable, would have been to store the key in a Java KeyStore, making sure the key is not present in the compiled code.

- **ContentNavigator's Stack:** When navigating within a scene, backtracking makes use of the Stack which stores a list of JavaFX nodes. JavaFX nodes have large memory footprints which maintain the internal state, event listeners and other content elements. Storing a list of these means that all the visited content remains in the main memory for the whole time the user navigates. For example, in FlashcardManagementScreen, which has instances of table views that are bound to observable list objects and HBox and VBox containers, the total memory footprint of several pushed states could lead to high memory usage. A maintainable alternative would have been to store lighter weight descriptors of the state and reconstruct the content when the goBack method is called.
- **External Libraries:** The system relies on the MaryTTS external library, external libraries in general introduce a maintainability risk. When the library is inevitably updated by its developer (for bug-fixing reasons, addressing vulnerabilities in security or just changes), I would have to decide whether to adopt this update and if I do, I'd have to confirm that all existing functionality still functions. This is straightforward when the dependencies are managed by a build tool like Apache Maven, but for the Revision Assistant since it is plain JavaFX, JARs are managed manually and there is no automated feature to alert programmers of possible updates or to notify developers of outdated JARs.
- **Over-reliance on CSV files:** The Revision Assistant stores all of its data in CSV files, this introduces maintainability concerns due to the lack of structure. As the system evolves, adding new fields of data and updating existing ones would require changes to parsing logic across multiple classes. This would increase code duplication as well as possibility of error. In addition, CSV files are more prone to corruption, and do not enforce data integrity which makes maintenance even harder. A database would provide more flexibility to the system, improving its long-term maintainability.

4.5.7 Maintenance of the Solution

The above was undeliberate decisions which make the system less maintainable, but the following are deliberate decisions which support the ability to understand, extend or update the code in the feature, hence making the system more maintainable.

- **Comments:** Comments are used throughout the codebase to explain what the code does. The purpose and reasoning behind logic is annotated step by step in less obvious code, for example the encryption and decryption methods in AuthenticationController, or the recursive searchForCSV and getAllCSV in the FlashcardController. This is essential, taking the FlashcardController code as an example, because recursion can become difficult to follow without explanation. These increase the maintainability of the code as they make it easier for

developers to understand the code in the future, hence they can debug and modify the solution safely.

- **Indentation:** Code is consistently indented and each level of nesting is clear and visible. Indentation helps structure the code and makes selection and iteration statements easier to read and understand.
- **Meaningful Identifier Names:** All variables, parameters, subroutines and classes are named to describe their purpose such as AuthenticationController handles the authentication of users; the variable timerTextX stores the x-value for the timer text. The variables, parameters and subroutine's identifiers all follow camel case to enhance the readability of code for future reference. These meaningful names make it easier for developers to grasp the purpose and functionality of different parts of the code, making the code self-explanatory.
- **Replacing Magic Numbers with Constants:** Unnamed numerical values were used for testing purposes during the development phase and now have been replaced with named constants throughout the codebase. This means a single change to the constants would be updated to all elements which use it across the project, reducing the maintainability burden when, for example, changing the layout of elements in the UI. The magic numbers within the AchievementController are not replaced with constants however, this is because the code is already readable and these numbers are only used in one place for comparative reasons.
- **Packaging Classes:** Classes were split into different packages depending on their purpose, the files were organised into logical groups within the source folder: Components, Controllers, Models, Screens and RecordClasses. This makes code easier to locate and understand, whilst also supporting the scalability of the solution in future iterations, new features can be added to appropriate packages without cluttering the source folder.
- **Organising Resources:** Similar to packaging classes, the resource root folder responsible for fonts, styles, images, and sounds were also split into directories. The images directory was split further into more folders corresponding to where they are used. This makes it easier to add resources into the program and also troubleshoot potential errors related to resources.
- **Modularity:** The code I have implemented is modular and the project has been structured into distinct, interchangeable and reusable components each responsible for a specific aspect of the program's functionality. This structuring makes use of object-oriented programming, which is a programming paradigm where real world entities are modelled as objects with behaviours (methods) and characteristics (attributes) and are initiated from classes. OOP directly supports maintainability by reducing code duplication and improving organisation of code.

- Inheritance is when a subclass (child class) takes on all attributes and methods from a superclass (parent class). This is used in the Revision Assistant to reduce code duplication and allowing the reusability of code. For example, the SystemScreens superclass (which all screens post-authentication inherit from) define common functionality such as the background creation once and is reused throughout. This improves the maintainability of the solution by allowing changes made to shared components to be made in a single location, enforcing consistent updates.
- Encapsulation is information hiding, restricting direct access and modification to attributes and methods, only allowing changes through getters and setters (with validation) to prevent accidental modification. Encapsulation is used throughout the model classes (as an example).
- Polymorphism is when different subroutines can have the same identifier but different purposes (with different parameters). Polymorphism improves the maintainability of the solution by increasing the flexibility of the code as well as being able to introduce new functionality without modifying original code, reducing the risk of errors.

Section 5 Final Code

5.1 Main.java

```
import Components.ScreenManager;
import Controllers.AuthenticationController;
import Controllers.SettingsController;
import Screens.AuthScreen;
import Screens.SplashScreen;
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class Main extends Application {
    // Constants
    private static final int fontSize = 30;
    // Custom Fonts Xirod and Vipnagorgialla loaded from local files
    public Font Xirod = Font.loadFont(getClass().getResourceAsStream("/fonts/Xirod.otf"),
fontSize);
    public Font Vipnagorgialla =
Font.loadFont(getClass().getResourceAsStream("/fonts/Vipnagorgialla.otf"), fontSize);
    private final AuthenticationController universalAuth = new AuthenticationController();
    private final SettingsController universalSettings = universalAuth.getSettingsController();
    public static void main(String[] args){
        launch(args);
    }
    @Override
    public void start(Stage primaryStage) {
        // Set stage reference for scene management
        ScreenManager.setStage(primaryStage);
        // Creates and configures the splash screen, linking the stylesheet to it.
        SplashScreen splashScreen = new SplashScreen();
        AuthScreen auth = new AuthScreen(universalAuth, universalSettings);
        Scene splashScene = splashScreen.create(() -> ScreenManager.switchScene(auth.create()));
        primaryStage.setTitle("Revision Assistant");
        primaryStage.setScene(splashScene);
        primaryStage.show();
        primaryStage.setResizable(false);
        primaryStage.setMaximized(true);
    }
}
```

5.2 Screens

5.2.1 SplashScreen.java

```
package Screens;

import Components.UIComponents;
import javafx.animation.Animation;
import javafx.animation.FadeTransition;
import javafx.animation.Interpolator;
import javafx.animation.RotateTransition;
import javafx.geometry.Pos;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.Pane;
import javafx.scene.layout.VBox;
import javafx.scene.text.Text;
import javafx.util.Duration;

import java.util.Objects;

public class SplashScreen {
    // Constants
    // Scene Dimensions
    private static final int sceneW = 1920;
    private static final int sceneH = 1057;
    // Splash Screen
    private static final int topX = 420;
    private static final int topY = 100;
```

```
    private static final int loadingW = 100;
    private static final int loadingH = 100;
    private static final int loadingX = 880;
    private static final int loadingY = 600;
    private static final int startAngle = 0;
    private static final int endAngle = 360;
    private static final int bottomX = 225;
    private static final int bottomY = 825;
    private static final int duration1 = 8;
    private static final double duration2 = 2.5;
    private static final double startOpacity = 1.0;
    private static final double endOpacity = 0.25;
    private final UIComponents ui = new UIComponents();
    // Code will be executed when onContinue.run() runs
    public Scene create(Runnable onContinue) {
        Pane root = new Pane();
        root.getStyleClass().add("screens");

        ImageView logo = ui.createLogoView();

        // Creating title text split into two lines.
        Text titleText1 = new Text("Revision");
        titleText1.getStyleClass().add("title");
        Text titleText2 = new Text("Assistant");
        titleText2.getStyleClass().add("title");
        VBox topText = new VBox(titleText1, titleText2);
        topText.setAlignment(Pos.CENTER);
        topText.setLayoutX(topX);
        topText.setLayoutY(topY);

        // Create the rotating loading indicator
        Image loading = new
Image(Objects.requireNonNull(getClass().getResource("/images/SplashScreen/loading.png")).toExtern
alForm());
        ImageView loadingView = new ImageView(loading);
        loadingView.setFitWidth(loadingW);
        loadingView.setFitHeight(loadingH);
        loadingView.setLayoutX(loadingX);
        loadingView.setLayoutY(loadingY);

        // Animate the loading indicator with continuous rotation
        RotateTransition loadingTransition = new RotateTransition(Duration.seconds(duration1),
loadingView);
        loadingTransition.setFromAngle(startAngle);
        loadingTransition.setToAngle(endAngle);
        loadingTransition.setInterpolator(Interpolator.EASE_IN);
        loadingTransition.setCycleCount(Animation.INDEFINITE);
        loadingTransition.play();

        // Instruction for the user at bottom of the screen
        Text infoText = new Text("Press any key to continue!");
        infoText.getStyleClass().add("text");
        VBox bottomText = new VBox(infoText);
        bottomText.setAlignment(Pos.CENTER);
        bottomText.setLayoutX(bottomX);
        bottomText.setLayoutY(bottomY);

        // Group text elements to create a pulsing fade effect for text
        Group texts = new Group(topText, bottomText);
        FadeTransition varyingOpacity = new FadeTransition(Duration.seconds(duration2), texts);
        varyingOpacity.setFromValue(startOpacity);
        varyingOpacity.setToValue(endOpacity);
        varyingOpacity.setCycleCount(FadeTransition.INDEFINITE);
        varyingOpacity.setAutoReverse(true);
        varyingOpacity.play();

        // Add all components to root Pane and create a new scene, containing the root
        root.getChildren().addAll(logo, loadingView, texts);
        Scene scene = new Scene(root, sceneW, sceneH);

scene.getStylesheets().add(Objects.requireNonNull(getClass().getResource("/styles/styles.css")).t
oExternalForm());

scene.getStylesheets().add(Objects.requireNonNull(getClass().getResource("/styles/dark.css")).toE
xternalForm());
        // Event Handlers, will execute the code that was passed in, when user interacts with
splash screen
        scene.setOnKeyPressed(_ -> onContinue.run());
        scene.setOnMouseClicked(_ -> onContinue.run());
        return scene;
    }
}
```

5.2.2 AuthScreen.java

```
package Screens;

import Components.ContentNavigator;
import Components.CustomAlert;
import Components.ScreenManager;
import Components.UIComponents;
import Controllers.AuthenticationController;
import Controllers.SettingsController;
import Models.User;
import RecordClasses.PasswordBundle;
import RecordClasses.RectangleButtonBundle;
import RecordClasses.TextBundle;
import javafx.event.ActionEvent;
import javafx.event.Event;
import javafx.event.EventHandler;
import javafx.scene.Cursor;
import javafx.scene.Group;
import javafx.scene.Node;
import javafx.scene.Scene;
import javafx.scene.image.ImageView;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import javafx.scene.text.TextAlignment;
import javafx.stage.Window;

import java.io.IOException;
import java.security.NoSuchAlgorithmException;
import java.util.Objects;

public class AuthScreen {
    // Constants
    // Scene
    private static final int sceneW = 1920;
    private static final int sceneH = 1057;
    // Background Rectangle
    private static final int backgroundX = 357;
    private static final int backgroundY = 187;
    private static final int backgroundW = 1206;
    private static final int backgroundH = 683;
    // Tabs
    private static final int tabW = 334;
    private static final int tabH = 98;
    private static final int lTabX = 578;
    private static final int rTabX = 972;
    private static final int tabY = 215;
    // Buttons
    private static final int mainButtonW = 1083;
    private static final int login_mainButtonY = 684;
    private static final int reg_mainButtonY = 699;
    private static final int forgot_mainButtonX = 418;
    private static final int forgot_mainButtonY = 729;
    private static final int mainButtonH = 98;
    private static final int mainButtonX = 430;
    private static final int smallButtonW = 700;
    private static final int smallButtonH = 60;
    private static final int smallButtonX = 600;
    private static final int login_smallButtonY = 790;
    private static final int reg_smallButtonY = 805;
    // Login input fields
    private static final int login_inputX = 695;
    private static final int login_emailY = 372;
    private static final int login_passwordY = 510;
    private static final int login_fieldX = 790;
    private static final int login_emailFieldY = 382;
    private static final int login_passwordFieldY = 520;
    private static final int login_lIconX = 694;
    private static final int login_emailIconY = 376;
    private static final int login_passwordIconY = 513;
    private static final int login_rIconX = 1166;
    private static final int login_passwordRIconY = 534;
    private static final int login_inputW = 530;
    private static final int login_inputH = 98;
    // Icons
    private static final int iconW = 95;
    private static final int iconH = 90;
    // Forgot Password
    private static final int fp_inputH = 98;
    private static final int fp_fieldX = 633;
```

```
    private static final int fp_emailFieldY = 258;
    private static final int fp_recoveryFieldY = 380;
    private static final int fp_passwordFieldY = 499;
    private static final int fp_cPFieldY = 619;
    private static final int fp_lIconX = 531;
    private static final int fp_emailIconY = 251;
    private static final int fp_recoveryIconX = 534;
    private static final int fp_recoveryIconY = 373;
    private static final int fp_passwordIconY = 495;
    private static final int fp_cPIconY = 615;
    private static final int fp_rIconX = 1280;
    private static final int fp_passwordRIconY = 515;
    private static final int fp_cPRIconY = 635;
    private static final int fp_inputX = 531;
    private static final int fp_inputW = 815;
    private static final int fp_emailY = 248;
    private static final int fp_recoveryY = 370;
    private static final int fp_passwordY = 489;
    private static final int fp_cPY = 609;
    // Register
    private static final int reg_emailY = 345;
    private static final int reg_passwordY = 458;
    private static final int reg_cPY = 572;
    private static final int reg_inputX = 531;
    private static final int reg_inputW = 815;
    private static final int reg_inputH = 98;
    private static final int reg_fieldX = 630;
    private static final int reg_emailFieldY = 355;
    private static final int reg_passwordFieldY = 468;
    private static final int reg_cPFieldY = 582;
    private static final int reg_lIconX = 531;
    private static final int reg_emailIconY = 345;
    private static final int reg_passwordIconY = 463;
    private static final int reg_cPIconY = 576;
    private static final int reg_rIconX = 1280;
    private static final int reg_passwordRIconY = 483;
    private static final int reg_cPRIconY = 598;
    // Recovery word
    private static final int rw_infoX = 530;
    private static final int rw_infoY = 300;
    private static final int rw_inputY = 460;
    private static final int rw_inputX = 531;
    private static final int rw_inputW = 815;
    private static final int rw_inputH = 98;
    private static final int rw_fieldX = 640;
    private static final int rw_fieldY = 470;
    private static final int rw_iconX = 534;
    private static final int rw_iconY = 463;
    private static final int rw_enterX = 1245;
    private static final int rw_enterY = 463;
    private static final int rw_enterW = 95;
    private static final int rw_enterH = 90;
    // Authentication Controller for account-related operations
    private final AuthenticationController ac;
    private final UIComponents ui = new UIComponents();
    private final ContentNavigator nav = new ContentNavigator();
    private final SettingsController sc;

    // Validation Requirements used across multiple subroutines
    private final String emailReq = "Email should follow the standard format";
    private final String passwordReq = "Password should have a minimum of 6 characters, including
a special character";
    private final String rwReq = "Recovery Word must be a minimum of 3 characters";

    public AuthScreen(AuthenticationController ac, SettingsController sc) {
        this.ac = ac;
        this.sc = sc;
    }

    public Scene create(){
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        ImageView logo = ui.createLogoView();
        // Create consistent background rectangle for content
        Rectangle background = new Rectangle(backgroundX, backgroundY, backgroundW,backgroundH);
        background.getStyleClass().add("backgroundRect");
        // Create tab buttons for switching between login and registration
        RectangleButtonBundle selectLogin = ui.buildRectangularButtonBundle(tabW, tabH, "Login",
"smallButtonText", lTabX, tabY);
        RectangleButtonBundle selectRegister = ui.buildRectangularButtonBundle(tabW,tabH,
"Register", "smallButtonText", rTabX, tabY);
        // Group content that changes based on selected tab
```

```
        Group contentLayer = new Group(selectLogin.button(), selectRegister.button());
        // Initialise first with login content
        buildLoginContent(contentLayer, selectLogin, selectRegister);
        root.getChildren().addAll(logo, background, contentLayer);
        return new Scene(root, sceneW, sceneH);
    }

    // Displays log in content when login is selected
    private void buildLoginContent(Group contentLayer, RectangleButtonBundle selectLogin,
RectangleButtonBundle selectRegister) {
        // Style the tab buttons to indicate which one is selected
        selectRegister.buttonRect().setStyle("-fx-fill: grey");
        selectLogin.buttonRect().setStyle("-fx-fill: black");
        // Set up button to switch content when clicked
        selectRegister.button().setOnMouseClicked(_ -> {
            buildRegistrationContent(contentLayer, selectLogin, selectRegister);
            selectRegister.buttonRect().setStyle("-fx-fill: black");
            selectLogin.buttonRect().setStyle("-fx-fill: grey");
        });

        // Create input field with user icon
        String iconURL = "/images/AuthScreen/user.png";
        TextBundle emailInput = ui.buildTextFieldInput(login_inputX, login_emailY, login_inputW,
login_inputH,login_fieldX, login_emailFieldY, "Enter Email", iconURL, login_lIconX,
login_emailIconY, iconW, iconH);
        emailInput.tf().getStyleClass().add("loginTextField");
        // Create password input field with show and hide functionality
        PasswordBundle loginPasswordInput = ui.buildPasswordInput(login_inputX, login_passwordY,
login_inputW, login_fieldX, login_passwordFieldY, "Enter Password", login_lIconX,
login_passwordIconY,login_rIconX, login_passwordRIconY);
        loginPasswordInput.copyPw().getStyleClass().add("loginTextField");
        loginPasswordInput.password().getStyleClass().add("loginTextField");
        // Add enter to submit
        EventHandler<ActionEvent> loginHandler = event -> {
            try {
                loginSubmit(event, emailInput, loginPasswordInput);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        };
        ui.bindEnter(loginHandler, emailInput.tf(), loginPasswordInput.password(),
loginPasswordInput.copyPw());
        // Groups inputs together
        Group inputs = new Group(emailInput.group(), loginPasswordInput.passwordGroup());
        // Group icons together
        Group icons = new Group(emailInput.icon(), loginPasswordInput.getIconGroup());

        // Create a login button
        RectangleButtonBundle login = ui.buildRectangularButtonBundle(mainButtonW, mainButtonH,
"Login", "buttonText", mainButtonX, login_mainButtonY);
        login.button().setOnMouseClicked(event -> {
            try {
                loginSubmit(event, emailInput, loginPasswordInput);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        // Create a "Forgot Password" button
        RectangleButtonBundle forgotPassword = ui.buildRectangularButtonBundle(smallButtonW,
smallButtonH, "Forgot Password", "smallerButtonText", smallButtonX, login_smallButtonY);
        forgotPassword.button().setOnMouseClicked(_ -> {
            // Change content to display forgot password options
            forgotPassword(contentLayer);
        });

        // Replace content layer with login elements
        nav.switchContent(contentLayer, selectLogin.button(), selectRegister.button(), inputs,
login.button(), icons, forgotPassword.button());
    }
    // Displays correct Alert depending on scenario, on enter or login click.
    private void loginSubmit(Event event, TextBundle emailInput, PasswordBundle
loginPasswordInput) throws IOException {
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        int option = 0;
        // Validate that fields are not empty before attempting to login
        if(!emailInput.tf().getText().isEmpty() &&
!loginPasswordInput.password().getText().isEmpty()) {
            option = userLogin(emailInput.tf().getText(),
loginPasswordInput.password().getText());
        }
        // Handle different login outcomes displaying alerts depending outcome
```

```
        switch(option) {
            case 0:
                CustomAlert.error(alertWindow, "Cannot be empty");
                break;
            case 1:
                CustomAlert.success(alertWindow, "Logged in successfully");
                ScreenManager.switchScene(new MainMenuScreen(ac, sc).create(),
emailInput.tf().getText());
                break;
            case 2:
                CustomAlert.error(alertWindow, "Invalid email or password, or possible problems
with files.");
                break;
        }
    }
    // Displays forgotPassword form, when option is selected
    private void forgotPassword(Group contentLayer) {
        // Create email input field
        String userURL = "/images/AuthScreen/user.png";
        TextBundle emailInput = ui.buildTextFieldInput(fp_inputX, fp_emailY, fp_inputW,
fp_inputH, fp_fieldX, fp_emailFieldY, "Enter Email", userURL, fp_lIconX, fp_emailIconY, iconW,
iconH);
        // Create recovery word input field
        String recoveryURL = "/images/AuthScreen/recover.png";
        TextBundle recoveryWord = ui.buildTextFieldInput(fp_inputX, fp_recoveryY, fp_inputW
,fp_inputH ,fp_fieldX, fp_recoveryFieldY, "Enter Recovery Word", recoveryURL, fp_recoveryIconX,
fp_recoveryIconY ,iconW, iconH);
        // Create password input fields
        PasswordBundle resetPassword = ui.buildPasswordInput(fp_inputX, fp_passwordY, fp_inputW,
fp_fieldX, fp_passwordFieldY, "Enter Password", fp_lIconX, fp_passwordIconY ,fp_rIconX,
fp_passwordRIconY);
        PasswordBundle confirmResetPassword = ui.buildPasswordInput(fp_inputX, fp_cPY, fp_inputW,
fp_fieldX, fp_cPFieldY, "Confirm password", fp_lIconX, fp_cPIconY, fp_rIconX, fp_cPRIconY);
        // Attach real time validators to tell user when input is valid
        // :: method reference, allowing it's specified method to be passed as a function
        UIComponents.attachValidator(emailInput.tf(), UIComponents::isValidEmail, "error");
        UIComponents.attachValidator(resetPassword.password(), UIComponents::isValidPassword,
"error", resetPassword.copyPw());
        UIComponents.attachValidator(confirmResetPassword.password(),
UIComponents::isValidPassword, "error", confirmResetPassword.copyPw());
        UIComponents.attachValidator(recoveryWord.tf(), UIComponents::isValidRecoveryW, "error");
        // Can be reset on enter for convenience
        EventHandler<ActionEvent> resetHandler = e -> resetSubmit(e, recoveryWord, emailInput,
resetPassword, confirmResetPassword, contentLayer);
        ui.bindEnter(resetHandler, emailInput.tf(), resetPassword.password(),
resetPassword.copyPw(), confirmResetPassword.password(), confirmResetPassword.copyPw(),
recoveryWord.tf());
        // Create password reset button
        RectangleButtonBundle resetPasswordRectangle =
ui.buildRectangularButtonBundle(mainButtonW,mainButtonH,"Reset Password", "buttonText",
forgot_mainButtonX, forgot_mainButtonY);
        resetPasswordRectangle.button().setOnMouseClicked(event -> resetSubmit(event,
recoveryWord, emailInput, resetPassword, confirmResetPassword, contentLayer));
        // Create a back button to return to login screen
        ImageView back = ui.createBRImageView("back");
        back.setOnMouseClicked(_ -> nav.goBack(contentLayer));
        Group icons  = new Group(emailInput.icon(), recoveryWord.icon(),
resetPassword.getIconGroup(), confirmResetPassword.getIconGroup(), back);
        // Replace content with password reset content
        nav.switchContent(contentLayer, emailInput.group(), recoveryWord.group(),
resetPassword.passwordGroup(), confirmResetPassword.passwordGroup(),
resetPasswordRectangle.button(), icons);
    }
    // Displays correct alert depending on scenario, or enter or reset password button click
    private void resetSubmit(Event event, TextBundle recoveryWord, TextBundle emailInput,
PasswordBundle resetPassword, PasswordBundle confirmResetPassword, Group contentLayer) {
        int option = -1;
        // Validate input
        boolean validInputs = UIComponents.isValidEmail(emailInput.tf().getText()) &&
UIComponents.isValidPassword(resetPassword.password().getText()) &&
UIComponents.isValidPassword(confirmResetPassword.password().getText()) &&
UIComponents.isValidRecoveryW(recoveryWord.tf().getText());
        if (!validInputs) {
            option = 0; // invalid input
        } else {
            // Find user by email
            User temp;
            try {
                temp = ac.findUserByEmail(emailInput.tf().getText());
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
```

```
            if (temp == null) {
                option = 1; // Email not found
            } else {
                if (!temp.getRecoveryWord().equals(recoveryWord.tf().getText())) {
                    option = 2; // Recovery word does not match
                } else {
                    if
(!resetPassword.password().getText().equals(confirmResetPassword.password().getText())) {
                        option = 3; // Passwords do not match
                    } else {
                        try {
                            if (ac.resetPassword(temp, resetPassword.password().getText())) {
                                option = 4; // Success
                            }
                        } catch (NoSuchAlgorithmException | IOException e) {
                            throw new RuntimeException(e);
                        }
                    }
                }
            }
        }

        // Display appropriate alert based on outcome
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        switch(option){
            case 0:
                CustomAlert.error(alertWindow, "No field can be empty or \n" + emailReq + "\n" +
passwordReq + "\n" + rwReq);
                break;
            case 1:
                CustomAlert.error(alertWindow, "Email not found");
                break;
            case 2:
                CustomAlert.error(alertWindow, "Recovery word does not match");
                break;
            case 3:
                CustomAlert.error(alertWindow, "Passwords do not match");
                break;
            case 4:
                CustomAlert.success(alertWindow, "Password reset successfully, you will be
redirected to the login screen!" );
                nav.goBack(contentLayer);
                break;
        }
    }
    // Attempts to log in the user
    private int userLogin(String email, String password) {
        boolean valid;
        try{
            valid = ac.loginUser(email, password, sc);
        } catch (NoSuchAlgorithmException | IOException e) {
            throw new RuntimeException(e);
        }
        if(valid) {
            return 1; // Login Successful
        }else{
            return 2; // Login failed
        }
    }

    // Displays registration content when register is selected
    private void buildRegistrationContent(Group contentLayer, RectangleButtonBundle selectLogin,
RectangleButtonBundle selectRegister) {
        // Style the tab buttons to indicate which one is selected
        selectRegister.buttonRect().setStyle("-fx-fill: black");
        selectLogin.buttonRect().setStyle("-fx-fill: grey");
        // Set up button to switch content when clicked
        selectLogin.button().setOnMouseClicked(_ -> {
                buildLoginContent(contentLayer, selectLogin, selectRegister);
                selectRegister.buttonRect().setStyle("-fx-fill: grey");
                selectLogin.buttonRect().setStyle("-fx-fill: black");
        });

        // Create input field
        String iconURL = "/images/AuthScreen/user.png";
        TextBundle emailInput = ui.buildTextFieldInput(reg_inputX, reg_emailY, reg_inputW,
fp_inputH, reg_fieldX, reg_emailFieldY, "Enter Email", iconURL, reg_lIconX, reg_emailIconY,
iconW, iconH);
        // Create password input fields
        PasswordBundle registrationPassword = ui.buildPasswordInput(reg_inputX, reg_passwordY,
reg_inputW, reg_fieldX, reg_passwordFieldY, "Enter Password", reg_lIconX, reg_passwordIconY
,reg_rIconX, reg_passwordRIconY);
```

```
        PasswordBundle confirmRegistrationPassword = ui.buildPasswordInput(reg_inputX, reg_cPY,
reg_inputW, reg_fieldX, reg_cPFieldY, "Confirm password", reg_lIconX, reg_cPIconY, reg_rIconX,
reg_cPRIconY);
        // attach real-time validators to input fields
        UIComponents.attachValidator(registrationPassword.password(),
UIComponents::isValidPassword, "error", registrationPassword.copyPw());
        UIComponents.attachValidator(confirmRegistrationPassword.password(),
UIComponents::isValidPassword, "error", confirmRegistrationPassword.copyPw());
        UIComponents.attachValidator(emailInput.tf(), UIComponents::isValidEmail,  "error");

        // Create a question mark icon for displaying input requirements
        ImageView question = ui.createBRImageView("question");

        // Enter to submit for convenience
        EventHandler<ActionEvent> registerHandler = e -> registerSubmit(e, emailInput,
registrationPassword, confirmRegistrationPassword, contentLayer);
        ui.bindEnter(registerHandler, emailInput.tf(), registrationPassword.password(),
registrationPassword.copyPw(), confirmRegistrationPassword.password(),
confirmRegistrationPassword.copyPw());
        // Group inputs fields and icons
        Group inputs = new Group(emailInput.group(), registrationPassword.passwordGroup(),
confirmRegistrationPassword.passwordGroup());
        Group icons = new Group(emailInput.icon(), registrationPassword.getIconGroup(),
confirmRegistrationPassword.getIconGroup(), question);

        // Create register button
        RectangleButtonBundle register = ui.buildRectangularButtonBundle(mainButtonW,
mainButtonH, "Register", "buttonText", mainButtonX, reg_mainButtonY);
        // Show requirements when question mark is clicked
        question.setOnMouseClicked(event -> {
            Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
            CustomAlert.info(alertWindow, (emailReq + "\n" + passwordReq + "\n" + rwReq));
        });
        // Handle registration button click
        register.button().setOnMouseClicked(event -> registerSubmit(event, emailInput,
registrationPassword, confirmRegistrationPassword, contentLayer));
        // Create "Already have an account" button
        RectangleButtonBundle alreadyHaveAccount =
ui.buildRectangularButtonBundle(smallButtonW,smallButtonH, "Already have an account",
"smallerButtonText", smallButtonX, reg_smallButtonY);
        alreadyHaveAccount.button().setOnMouseClicked(_ -> buildLoginContent(contentLayer,
selectLogin, selectRegister));
        // Replace content with registration form
        nav.switchContent(contentLayer, selectLogin.button(), selectRegister.button(), inputs,
register.button(), icons, alreadyHaveAccount.button());
    }
    // Enter or Click Register logic
    public void registerSubmit(Event event, TextBundle emailInput, PasswordBundle
registrationPassword, PasswordBundle confirmRegistrationPassword, Group contentLayer){
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        // Check if inputs are valid
        boolean validInputs = UIComponents.isValidEmail(emailInput.tf().getText()) &&
UIComponents.isValidPassword(registrationPassword.password().getText()) &&
UIComponents.isValidPassword(confirmRegistrationPassword.password().getText());
        if(!validInputs) {
            CustomAlert.error(alertWindow, "No field can be empty or \n" + emailReq + "\n" +
passwordReq + "\n" + rwReq);
        }else
if(!registrationPassword.password().getText().equals(confirmRegistrationPassword.password().getTe
xt())) {
            CustomAlert.error(alertWindow, "Passwords do not match");
        }else{
            try {
                // Check if email already exists
                if (ac.doesEmailExist(emailInput.tf().getText())) {
                    CustomAlert.error(alertWindow, "Email already exists.");
                } else {
                    // Proceed to Recovery word setup
                    buildRecoveryWordEntrance(contentLayer, emailInput.tf().getText(),
registrationPassword.password().getText());
                }
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }
    }
    // Builds recovery word input screen when all inputs are valid
    public void buildRecoveryWordEntrance(Group contentLayer, String email, String password){
        // Create informational text
        Text infoText = new Text("Enter a recovery word to use \n when resetting password \n
Minimum 3 Characters");
        infoText.getStyleClass().add("infoText");
```

```
        infoText.setTextAlignment(TextAlignment.CENTER);
        infoText.setLayoutX(rw_infoX);
        infoText.setLayoutY(rw_infoY);
        String iconURL = "/images/AuthScreen/recover.png";
        // Create recovery word input field
        TextBundle recoveryWord = ui.buildTextFieldInput(fp_inputX, rw_inputY, rw_inputW ,
reg_inputH,rw_fieldX, rw_fieldY, "Enter Recovery Word", iconURL, rw_iconX, rw_iconY ,iconW,
iconH);
        UIComponents.attachValidator(recoveryWord.tf(), UIComponents::isValidRecoveryW, "error");
        recoveryWord.tf().setOnAction(e -> {
            try {
                rwSubmit(e, recoveryWord, email, password);
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
        // Create enter button (as an icon)
        ImageView enter = new
ImageView(Objects.requireNonNull(getClass().getResource("/images/AuthScreen/enter.png")).toExtern
alForm());
        enter.setFitHeight(rw_enterH);
        enter.setFitWidth(rw_enterW);
        enter.setLayoutX(rw_enterX);
        enter.setLayoutY(rw_enterY);
        enter.setCursor(Cursor.HAND);
        // Handle registration submission
        enter.setOnMouseClicked(event -> {
            try {
                rwSubmit(event, recoveryWord, email, password);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        // Group recovery word input and entry button
        Group input = new Group(recoveryWord.group(), recoveryWord.icon(), enter);
        // Create back button if user wants to cancel
        ImageView back  = ui.createBRImageView("back");
        back.setOnMouseClicked(_ -> nav.goBack(contentLayer));
        // replace content with recovery word entry screen
        nav.switchContent(contentLayer, input, infoText, back);
    }
    // Enter / onClick, submitting recovery word
    public void rwSubmit(Event event, TextBundle recoveryWord, String email, String password)
throws IOException {
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        int option;
        if(!UIComponents.isValidRecoveryW(recoveryWord.tf().getText())) {
            option = 0; // Empty recovery word
        }else {
            option = userRegistration(email, password, recoveryWord.tf().getText());
        }
        // Display appropriate alert based on outcome
        switch(option){
            case 0:
                CustomAlert.error(alertWindow, "Cannot be empty and must be greater than 3
characters");
                break;
            case 1:
                CustomAlert.success(alertWindow, "Account successfully registered");
                ScreenManager.switchScene(new MainMenuScreen(ac, sc).create(), email);
                break;
            case 2:
                CustomAlert.error(alertWindow, "Possible problem with files.");
                break;
        }
    }
    // Attempts to register the user
    public int userRegistration(String email, String password, String recoveryWord){
        boolean added;
        try {
            added = ac.registerUser(email, password, recoveryWord, sc);
        } catch (NoSuchAlgorithmException | IOException e) {
            throw new RuntimeException(e);
        }
        if(added){
            return  1; // Registration Success
        }else{
            return 2; // Registration failed
        }
    }
}
```

5.2.3 SystemScreens.java

```
package Screens;

import Components.ContentNavigator;
import Components.CustomAlert;
import Components.ScreenManager;
import Components.UIComponents;
import Controllers.AuthenticationController;
import Controllers.FlashcardController;
import Models.Flashcard;
import RecordClasses.TextBundle;
import javafx.scene.Group;
import javafx.scene.Node;
import javafx.scene.control.Label;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import javafx.stage.Window;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.stream.Collectors;
import static javafx.scene.text.TextAlignment.CENTER;

public abstract class SystemScreens {
    // Constants
    // Scene
    protected static final int sceneW = 1920;
    protected static final int sceneH = 1057;
    // Constant rectangular button dimensions set
    protected static final int rectW = 1300;
    protected static final int rectH = 133;
    // Background
    protected static final int bgX = 108;
    protected static final int bgY = 194;
    protected static final int bgW = 1704;
    protected static final int bgH = 745;
    // Title
    protected static final int titleY = 160;
    // 4 button layout
    protected static final int fourX = 310;
    protected static final int fourY1 = 250;
    protected static final int fourY2 = 420;
    protected static final int fourY3 = 591;
    protected static final int fourY4 = 762;
    // 3 button layout
    protected static final int threeX = 293;
    protected static final int threeY1 = 315;
    protected static final int threeY2 = 485;
    protected static final int threeY3 = 656;
    // Icon offset
    protected static final int iconOffset = -5;
    // Spacing
    protected static final int vSpacing = 10;
    protected static final int hSpacing = 20;
    // Other
    protected static final int zero = 0;
    private static final int cardPreviewH = 600;
    private static final int iconY = 4;
    protected static final int iconW = 95;
    protected static final int iconH = 90;
    private static final int searchDW = 600;
    private static final int searchFW = 1000;
    private static final int searchH = 98;
    private static final int fieldX = 95;
    private static final int fieldY = 10;
    private static final int leftW = 600;
    private static final int rightW = 1000;
    private static final int hX = 140;
    private static final int hY = 210;

    public final UIComponents ui = new UIComponents();
    public final ContentNavigator nav = new  ContentNavigator();
```

```
    private Text title;

    public Group createBackground(String titleText){
        title = new Text(titleText);
        title.setTextAlignment(CENTER);
        title.getStyleClass().add("postAuthTitle");
        title.setLayoutY(titleY);
        Rectangle background = new Rectangle(bgX, bgY, bgW,bgH);
        background.getStyleClass().add("backgroundRect");
        return new Group(title, background, ui.createLogoView());
    }

    public ImageView createIcon(int iconX, int iconY, String iconURL){
        ImageView iconView = new
ImageView(Objects.requireNonNull(getClass().getResource(iconURL)).toExternalForm());
        iconView.getStyleClass().add("icons");
        iconView.setLayoutX(iconX);
        iconView.setLayoutY(iconY);
        return iconView;
    }

    public ImageView getBackView(Group contentLayer) {
        ImageView back = ui.createBRImageView("back");
        back.setOnMouseClicked(_ -> nav.goBack(contentLayer));
        return back;
    }

    public ImageView getSwitchScreenBackView(AuthenticationController ac) {
        ImageView back = ui.createBRImageView("back");
        back.setOnMouseClicked(_ -> {
            try {
                ScreenManager.goBack(ac.getEmail());
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        return back;
    }
    public void setTitle(String titleText) {
        this.title = new Text(titleText);
    }

    public Text getTitle(){
        return title;
    }

    // Method to build the card preview table to be used in multiple screens
    protected TableView<Flashcard> buildCardPreviewTable(int height){
        // Create a new tableview bound to observable list
        TableView<Flashcard> table = new TableView<>();
        table.getStyleClass().add("cardTable");
        table.setMinHeight(height);
        table.setPrefHeight(height);
        // Create columns and link to the specific property of Flashcard.java
        // to populate the cell
        TableColumn<Flashcard, String> frontColumn = new TableColumn<>("Front");
        frontColumn.setCellValueFactory(new PropertyValueFactory<>("question"));
        TableColumn<Flashcard, String> backColumn = new TableColumn<>("Back");
        backColumn.setCellValueFactory(new PropertyValueFactory<>("answer"));
        // Add all columns to table
        table.getColumns().addAll(List.of(frontColumn, backColumn));
        // Columns are resized to fill available width, last column will shrink or expand
        table.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
        // Set the text when there are no cards
        Label placeholderText = new Label("No cards yet");
        placeholderText.setStyle("-fx-text-fill: red;");
        table.setPlaceholder(placeholderText);
        return table;
    }

    // Build HBox which displays the decks table, preview table and search fields
    protected HBox getPreviewDecksAndCards(FlashcardController fc, TableView<String> decksTable,
Window owner, Node... extras){
        // Card Preview Table
        TableView<Flashcard> cardsPreview = buildCardPreviewTable(cardPreviewH);
        // Search for deck names
        String searchIcon = "/images/FlashcardManagementScreen/search.png";
        TextBundle searchDecks = ui.buildTextFieldInput(zero, zero, searchDW, searchH, fieldX,
fieldY, "Search Decks", searchIcon, zero, iconY, iconW, iconH );
        searchDecks.tf().getStyleClass().add("deckSearch");
        searchDecks.group().getChildren().add(searchDecks.icon());
        // Search for specific flashcards within all decks, and display what deck it is in
```

```
        TextBundle searchFlashcards = ui.buildTextFieldInput(zero, zero, searchFW, searchH,
fieldX, fieldY, "Search Flashcards", searchIcon, zero, iconY, iconW, iconH );
        searchFlashcards.tf().getStyleClass().add("fcSearch");
        searchFlashcards.group().getChildren().add(searchFlashcards.icon());
        // Listeners for searches
        // All initial decks
        ArrayList<String> allDecks = new ArrayList<>(decksTable.getItems());
        searchDecks.tf().textProperty().addListener((_, _, newValue) -> {
            TextField searchFlashcardTF = searchFlashcards.tf();
            cardsPreview.getItems().clear();
            if(newValue.isEmpty()){
                searchFlashcardTF.setDisable(false);
                searchFlashcards.group().setOnMouseClicked(null);
                // If no search just keep results to all decks
                decksTable.getItems().setAll(allDecks);
                return;
            }else {
                // Can't have two searches at once
                searchFlashcards.group().setOnMouseClicked(_-> CustomAlert.error(owner,
                        "Clear the deck search to search flashcards"));
                searchFlashcardTF.setDisable(true);
                // Set tableView contents to the following
                decksTable.getItems().setAll(
                        allDecks.stream() // Convert the array list into a stream for processing
                                // Filter so decks which match the search condition are shown
                                .filter(deck ->
deck.toLowerCase().contains(newValue.trim().toLowerCase()))
                                // Add decks matching the criteria to a list
                                .collect(Collectors.toList()));
            }
        });
        searchFlashcards.tf().textProperty().addListener((_, _, newValue) -> {
            TextField searchDeckTF = searchDecks.tf();
            cardsPreview.getItems().clear();
            if(newValue.isEmpty()){
                searchDeckTF.setDisable(false);
                searchDecks.group().setOnMouseClicked(null);
                // If no search just keep results to all decks
                decksTable.getItems().setAll(allDecks);
                return;
            }else{
                searchDecks.group().setOnMouseClicked(_-> CustomAlert.error(owner,
                        "Clear the flashcard search to search decks"));
                searchDeckTF.setDisable(true);
                // Create an Array List of the decks which match criteria
                ArrayList<String> decksMatchingCriteria = new ArrayList<>();
                for (String deck : allDecks){
                    try{
                        ArrayList<Flashcard> cards = fc.loadFlashcardDeck(deck);
                        boolean matchFound = cards.stream().anyMatch(
                                card ->

card.getQuestion().toLowerCase().contains(newValue.trim().toLowerCase()) ||

card.getAnswer().toLowerCase().contains(newValue.trim().toLowerCase())
                        );
                        if(matchFound){
                            decksMatchingCriteria.add(deck);
                        }
                    } catch (IOException e) {
                        throw new RuntimeException(e);
                    }
                }decksTable.getItems().setAll(decksMatchingCriteria);
            }
        });
        // Listener to change the card preview based on selection
        decksTable.getSelectionModel().selectedItemProperty().addListener(((_, _, newValue) -> {
            if(newValue == null) return;
            try{
                cardsPreview.getItems().setAll(fc.loadFlashcardDeck(newValue));
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }));

        VBox leftSide = new VBox(vSpacing, searchDecks.group(), decksTable);
        leftSide.getChildren().addAll(extras);
        leftSide.setPrefWidth(leftW);
        leftSide.setMinWidth(leftW);

        VBox rightSide = new VBox(vSpacing, searchFlashcards.group(), cardsPreview);
        rightSide.setPrefWidth(rightW);
```

```
        HBox display = new HBox(hSpacing, leftSide, rightSide);
        display.setLayoutX(hX);
        display.setLayoutY(hY);
        display.getStyleClass().add("display");
        return display;
    }
}
```

5.2.4 MainMenuScreen.java

```
package Screens;

import Components.CustomAlert;
import Components.ScreenManager;
import Controllers.AuthenticationController;
import Controllers.SettingsController;
import Controllers.StatsController;
import Models.UserStats;
import RecordClasses.RectangleButtonBundle;
import javafx.event.Event;
import javafx.scene.Group;
import javafx.scene.Node;
import javafx.scene.Scene;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Pane;
import javafx.scene.layout.VBox;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import javafx.scene.text.TextAlignment;
import javafx.stage.FileChooser;
import javafx.stage.Window;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;

public class MainMenuScreen extends SystemScreens {
    // Constants
    private static final int titleX = 510;
    // Statistics
    private static final int levelTextX = 300;
    private static final int levelTextY = 900;
    private static final int barX = 300;
    private static final int barY = 820;
    private static final int barW = 1320;
    private static final int barH = 40;
    private static final int xpTextX = 1420;
    private static final int xpTextY = 900;
    private static final int statsLeftX = 200;
    private static final int statsRightX = 1000;
    private static final int statsStartY = 310;
    private static final int statsRowH = 70;
    private static final int statsXOffset = 500;
    // Achievements
    private static final int achLeftX = 450;
    private static final int achRightX = 1000;
    private static final int achStartY = 250;
    private static final int achRowH = 105;
    private static final int achXOffset = 40;
    private static final int achYOffset = 30;
    // Import / Export
    private static final int contentX = 270;
    private static final int contentY = 350;
    private static final int buttonW = 450;
    private static final int buttonH = 100;
    private static final int spacing = 30;

    private final AuthenticationController ac;
    private final SettingsController sc;
    private final StatsController stats;
    // Constructor
    public MainMenuScreen(AuthenticationController ac, SettingsController sC) throws IOException
{
        this.ac = ac;
        this.sc = sC;
        this.stats = new StatsController();
```

```
    }
    // Creates the main menu scene
    public Scene create(){
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        // Creating the background rectangle using the inherited method from the SystemScreens
Class
        Group background = createBackground("Main Menu");
        getTitle().setLayoutX(titleX);
        Group contentLayer = new Group();
        // Creating buttons bundles for all
        RectangleButtonBundle revisionSystem = ui.buildRectangularButtonBundle(rectW, rectH,
"Revision System", "buttonText", fourX, fourY1);
        revisionSystem.button().setOnMouseClicked(_ -> {
            try {
                revisionSystemMouseClick();
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        ImageView rsIcon = createIcon(fourX, fourY1+iconOffset,
"/images/MainMenuScreen/books.png");

        RectangleButtonBundle userManagement = ui.buildRectangularButtonBundle(rectW, rectH,
"User Management", "buttonText", fourX,  fourY2);
        userManagement.button().setOnMouseClicked(_ -> buildUMContent(contentLayer));
        ImageView umIcon = createIcon(fourX, fourY2+iconOffset,
"/images/MainMenuScreen/userbook.png");

        RectangleButtonBundle settings = ui.buildRectangularButtonBundle(rectW, rectH,
"Settings", "buttonText", fourX, fourY3);
        settings.button().setOnMouseClicked(_ -> {
            try {
                settingsMouseClick();
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        ImageView sIcon = createIcon(fourX, fourY3+iconOffset,
"/images/MainMenuScreen/settings.png");

        RectangleButtonBundle logout = ui.buildRectangularButtonBundle(rectW, rectH, "Logout",
"buttonText", fourX, fourY4);
        logout.button().setOnMouseClicked(this::logout);
        ImageView lIcon = createIcon(fourX, fourY4+iconOffset,
"/images/MainMenuScreen/door.png");

        Group buttons = new Group(revisionSystem.button(), userManagement.button(),
settings.button(), logout.button());
        Group icons = new Group(rsIcon, umIcon, sIcon, lIcon);
        contentLayer.getChildren().addAll(buttons, icons);
        root.getChildren().addAll(background, contentLayer);
        return new Scene(root, sceneW, sceneH);
    }

    private void logout(Event e) {
        Window alertWindow = ((Node) e.getSource()).getScene().getWindow();
        if(CustomAlert.confirm(alertWindow, "Logout Confirmation", "Are you sure you would like
to log out?")){
            ac.logout();
        }
    }
    // switch to settings screen
    private void settingsMouseClick() throws IOException {
        ScreenManager.switchScene(new SettingsScreen(ac, sc).create(), ac.getEmail());
    }
    // switch to revison system screen
    private void revisionSystemMouseClick() throws IOException {
        ScreenManager.switchScene(new RevisionSystemScreen(ac, sc, stats).create(),
ac.getEmail());
    }
    // Procedure which builds the content for the User Management Page
    private void buildUMContent(Group contentLayer) {
        RectangleButtonBundle statistics = ui.buildRectangularButtonBundle(rectW, rectH,
"Statistics", "buttonText", threeX, threeY1);
        ImageView sIcon = createIcon(threeX, threeY1+iconOffset,
"/images/MainMenuScreen/stats.png");
        statistics.button().setOnMouseClicked(_ -> {
            try {
                buildStatisticsContent(contentLayer);
            } catch (IOException e) {
                throw new RuntimeException(e);
```

```
            }
        });

        RectangleButtonBundle achievements = ui.buildRectangularButtonBundle(rectW, rectH,
"Achievements", "buttonText", threeX, threeY2);
        ImageView aIcon = createIcon(threeX, threeY2+iconOffset,
"/images/MainMenuScreen/achievement.png");
        achievements.button().setOnMouseClicked(_ -> {
            try {
                buildAchievementsContent(contentLayer);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        RectangleButtonBundle fileManagement = ui.buildRectangularButtonBundle(rectW, rectH,
"File Management", "buttonText", threeX, threeY3);
        ImageView fmIcon = createIcon(threeX, threeY3+iconOffset,
"/images/MainMenuScreen/folder.png");
        fileManagement.button().setOnMouseClicked(_ -> buildFileManagementContent(contentLayer));

        Group options = new Group(statistics.button(), fileManagement.button(),
achievements.button(), sIcon, aIcon, fmIcon);

        nav.switchContent(contentLayer, options, getBackView(contentLayer));
    }
    // Build content where user can see his stats
    private void buildStatisticsContent(Group contentLayer) throws IOException {
        UserStats uStats = stats.loadUserStats(ac.getEmail());
        // Level header
        Text levelText = new Text("Level " + uStats.getLevel());
        levelText.getStyleClass().add("levelText");
        ui.setPosition(levelText, levelTextX, levelTextY);
        // XP Bar
        Rectangle bar = new Rectangle(barX, barY, barW, barH);
        bar.getStyleClass().add("XPBar");
        // XP Progress
        double progress = (double) uStats.getXPIntoCurrentLevel() / uStats.getXPForNext();
        progress = Math.min(progress, 1.0); // clamp to 1.0
        Rectangle progressFill = new Rectangle(barX, barY, barW*progress, barH);
        progressFill.getStyleClass().add("progressFill");
        // Text under bar
        Text xpProgressText = new Text( uStats.getXPIntoCurrentLevel() + "/" +
uStats.getXPForNext());
        xpProgressText.getStyleClass().add("infoText");
        ui.setPosition(xpProgressText, xpTextX, xpTextY);

        nav.switchContent(contentLayer, createStatsGrid(uStats), levelText, xpProgressText, bar,
progressFill, getBackView(contentLayer));
    }
    // Creates a grid of stats
    private Group createStatsGrid(UserStats uStats) {
        Group statsGrid = new Group();
        // Preventing divide by 0 error
        int correct = uStats.getTotalCorrect();
        int incorrect = uStats.getTotalIncorrect();
        double accuracy;
        if(correct + incorrect == 0){accuracy = 0;}
        else{accuracy = (double) correct / (correct + incorrect);}
        // 2D array used to store each stat as a pair, avoiding repeating code
        String[][] grid = {
                {"Total XP", String.valueOf(uStats.getTotalXP())},
                {"Accuracy", (Math.round(100*accuracy) + "%")},
                {"Sessions Completed", String.valueOf(uStats.getTotalSessions())},
                {"Total Study Time", formatTime(uStats.getTotalStudyTime())},
                {"Total Correct", String.valueOf(correct)},
                {"Total Incorrect", String.valueOf(incorrect)},
                {"Current Streak", uStats.getCurrentStreak() + " days"},
                {"Longest Streak", uStats.getLongestStreak() + " days"},
                {"Max Streak", uStats.getConsecutiveCorrect() + " cards"},
                {"Total Retries", String.valueOf(uStats.getTotalRetries())},
                {"Achievements", uStats.getUnlockedAchievements().size() + " / 14"},
                {"Last Study Date", uStats.getLastStudyDate()},
        };
        int leftX = statsLeftX;
        int rightX = statsRightX;
        int startY = statsStartY;
        int rowH = statsRowH;
        for(int i = 0; i < grid.length; i++){
            // Even rows go on the left, odd goes to the right
            int x = (i % 2 == 0) ? leftX : rightX;
            int y = startY + (i/2) * rowH;
```

```
            // Create the stat text
            Text statNameText = new Text(grid[i][0] + ":");
            statNameText.getStyleClass().add("statText");
            ui.setPosition(statNameText, x, y);
            // Create the actual statistic to accompany this text
            Text statistic = new Text(grid[i][1]);
            statistic.getStyleClass().add("statText");
            ui.setPosition(statistic, x + statsXOffset, y);
            statsGrid.getChildren().addAll(statNameText, statistic);
        }
        return statsGrid;
    }
    // Format time
    private String formatTime(long sec){
        return String.format("%02d:%02d:%02d", sec / 3600, (sec % 3600) / 60, sec % 60);
    }

    // Builds content where the user can see what achievements they have
    public void buildAchievementsContent(Group contentLayer) throws IOException {
        UserStats uStats = stats.loadUserStats(ac.getEmail());
        // 2D Array with Achievements and descriptions
        String[][] achievements = {
                {"Sharp Mind", "Get 5 answers correct in a row"},
                {"Perfect Run", "Get 10 answers correct in a row"},
                {"Flawless", "Complete a session with 100% accuracy"},
                {"Getting Started", "Answer 10 flashcards"},
                {"Knowledge Expert", "Answer 100 flashcards correctly"},
                {"100 Club", "Answer 100 questions total"},
                {"Revision Rookie", "Complete 5 sessions"},
                {"Revision Master", "Complete 50 sessions"},
                {"Study Streak", "Study for 3 days in a row"},
                {"Dedicated Learner", "Study for 7 days in a row"},
                {"Revision Veteran", "Study for a total of 30 days"},
                {"Keep Trying", "Retry a session"},
                {"Never Give Up", "Retry 10 sessions"},
                {"Ultimate Scholar", "Unlock all achievements"},
        };

        Group grid = new Group();
        int leftX = achLeftX;
        int rightX = achRightX;
        int startY = achStartY;
        int rowH = achRowH;
        // build the Text
        for(int i = 0; i < achievements.length; i++){
            String achievement = achievements[i][0];
            String description = achievements[i][1];
            boolean unlocked = uStats.isAchievementUnlocked(achievement);
            int x = (i % 2 == 0) ? leftX : rightX;
            int y = startY + (i/2) * rowH;

            // Add correct emoji depending on
            Text achievementText = new Text((unlocked ? "⌛  " : "🔓  ") + achievement);
            achievementText.getStyleClass().add("statText");
            ui.setPosition(achievementText, x, y);

            Text descriptionText = new Text(description);
            descriptionText.getStyleClass().add("descriptionText");
            ui.setPosition(descriptionText, x + achXOffset, y + achYOffset);
            // Set the colour to locked achievements differently
            if(!unlocked){
                achievementText.setStyle("-fx-fill: #888888;");
                descriptionText.setStyle("-fx-fill: #888888;");
            }
            grid.getChildren().addAll(achievementText, descriptionText);
        }
        nav.switchContent(contentLayer, grid, getBackView(contentLayer));
    }
    // Create file management content
    private void buildFileManagementContent(Group contentLayer) {
        Window window = contentLayer.getScene().getWindow();
        HBox content = new HBox(hSpacing);
        ui.setPosition(content, contentX, contentY);
        String[] files = { "users.csv", "settings.csv", "stats.csv" };
        // Create buttons for each
        for(int i = 0; i < files.length; i++){
            String file = files[i];

            Text text = new Text(file);
            text.getStyleClass().add("infoText");
```

```
            RectangleButtonBundle exp = ui.buildRectangularButtonBundle(buttonW, buttonH, "Export
Backup", "smallerButtonText", zero, zero);
            exp.button().setOnMouseClicked(_ -> {
                try {
                    exportFile(window, file);
                } catch (IOException e) {
                    throw new RuntimeException(e);
                }
            });
            RectangleButtonBundle imp = ui.buildRectangularButtonBundle(buttonW, buttonH, "Import
Backup", "smallerButtonText", zero, zero);
            imp.button().setOnMouseClicked(_ -> {
                try {
                    importFile(window, file);
                } catch (IOException e) {
                    throw new RuntimeException(e);
                }
            });
            VBox box = new VBox(spacing, text, exp.button(), imp.button());
            content.getChildren().addAll(box);
        }
        nav.switchContent(contentLayer, content, getBackView(contentLayer));
    }
    // Copies the system CSV to a location chosen by user
    private void exportFile(Window window, String fileName) throws IOException {
        // Create a file chooser
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Export " + fileName);
        fileChooser.setInitialFileName(fileName);
        // Filter to csv
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("CSV", "*.csv"));
        File destination = fileChooser.showSaveDialog(window);
        // if save cancelled, return
        if(destination == null) return;

        // Find the source and copy it to save location
        Path source = Paths.get(System.getProperty("user.dir"), "files", fileName);
        Files.copy(source, destination.toPath(), StandardCopyOption.REPLACE_EXISTING);

        CustomAlert.success(window, fileName + " has been exported");
    }
    // Replaces the system CSV with user chosen file
    private void importFile(Window window, String fileName) throws IOException {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Import " + fileName);
        // Filter to csv
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("CSV", "*.csv"));
        File source = fileChooser.showOpenDialog(window);
        // return if empty or cancelled
        if(source == null) return;
        if(!ac.validateCSV(source, fileName)){
            CustomAlert.error(window, "Invalid file format");
            return;
        }
        if(source.length() == 0){
            CustomAlert.error(window, "File has no contents");
            return;
        }
        // Ask for user confirmation
        boolean confirmed = CustomAlert.confirm(window, "Import Backup", "This will replace your
current " + fileName
                + ". Are you sure you want to continue? You will be logged out following the
import.");
        if(!confirmed) return;
        Path destination = Paths.get(System.getProperty("user.dir"), "files", fileName);
        Files.copy(source.toPath(), destination, StandardCopyOption.REPLACE_EXISTING);
        CustomAlert.success(window, fileName + " has been imported");
        // Log out so imported files can be read from logging in
        ac.logout();
    }
}
```

5.2.5 RevisionSystemScreen.java

```
package Screens;

import Components.CustomAlert;
import Components.ScreenManager;
import Controllers.AuthenticationController;
```

```
import Controllers.FlashcardController;
import Controllers.SettingsController;
import Controllers.StatsController;
import RecordClasses.SessionRecord;
import RecordClasses.RectangleButtonBundle;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.chart.*;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Pane;
import javafx.stage.Window;

import java.io.IOException;
import java.time.LocalDate;
import java.time.temporal.IsoFields;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;

public class RevisionSystemScreen extends SystemScreens{
    // Constants
    private static final int titleX = 275;
    // Graphs
    private static final int graphBtnW = 380;
    private static final int graphBtnH = 80;
    private static final int graphButtonX = 180;
    private static final int graphButtonY = 200;
    private static final int graphX = 200;
    private static final int graphY = 300;
    private static final int graphW = 1500;
    private static final int graphH = 600;
    // Import / Export
    private static final int iconX = 297;

    private final AuthenticationController ac;
    private final SettingsController sc;
    private final FlashcardController fc;
    private final StatsController stats;

    public RevisionSystemScreen(AuthenticationController ac, SettingsController sC,
StatsController stats) {
        this.ac = ac;
        this.sc = sC;
        this.fc = new FlashcardController(ac.getEmail());
        this.stats = stats;
    }
    // creates the revision system scene
    public Scene create(){
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        Scene scene = new Scene(root, sceneW, sceneH);
        Group background = createBackground("Revision System");
        getTitle().setLayoutX(titleX);
        ImageView back = getSwitchScreenBackView(ac);

        Group contentLayer = new Group();

        RectangleButtonBundle fcManagement = ui.buildRectangularButtonBundle(rectW, rectH,
"Flashcards", "buttonText", fourX, fourY1);
        ImageView fcmIcon = createIcon(fourX, fourY1+iconOffset,
"/images/RevisionSystemScreen/flashcardmanagement.png");
        fcManagement.button().setOnMouseClicked(_ -> {
            try {
                fcManagementMouseClick();
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        RectangleButtonBundle studyModes = ui.buildRectangularButtonBundle(rectW, rectH, "Study
Modes", "buttonText", fourX,  fourY2);
        ImageView smIcon = createIcon(fourX, fourY2+iconOffset,
"/images/RevisionSystemScreen/studymodes.png");
        studyModes.button().setOnMouseClicked(_ -> {
            try {
               smMouseClick();
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
```

```
        RectangleButtonBundle impOrExp = ui.buildRectangularButtonBundle(rectW, rectH, "Import /
Export", "buttonText", fourX, fourY3);
        ImageView impIcon = createIcon(fourX, fourY3+iconOffset,
"/images/RevisionSystemScreen/import.png");
        impOrExp.button().setOnMouseClicked(_ -> buildImportExport(contentLayer));
        RectangleButtonBundle analytics = ui.buildRectangularButtonBundle(rectW, rectH,
"Analytics", "buttonText", fourX, fourY4);
        ImageView aIcon = createIcon(fourX, fourY4+iconOffset,
"/images/RevisionSystemScreen/analytics.png");
        analytics.button().setOnMouseClicked(_-> {
            try {
                buildAnalyticsContent(contentLayer);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        Group buttons = new Group(fcManagement.button(), studyModes.button(), impOrExp.button(),
analytics.button(), fcmIcon, smIcon, impIcon, aIcon);
        contentLayer.getChildren().add(buttons);
        root.getChildren().addAll(background, back, contentLayer);
        return scene;
    }

    // Build the content to show the graphs
    private void buildAnalyticsContent(Group contentLayer) throws IOException {
        // Buttons to switch the graphs
        RectangleButtonBundle accuracy = ui.buildRectangularButtonBundle(graphBtnW, graphBtnH,
"Accuracy", "smallerButtonText", zero, zero);
        RectangleButtonBundle studyTime = ui.buildRectangularButtonBundle(graphBtnW, graphBtnH,
"Study Time", "smallerButtonText", zero, zero);
        RectangleButtonBundle weakDecks = ui.buildRectangularButtonBundle(graphBtnW, graphBtnH,
"Weakest Decks", "smallerButtonText", zero, zero);
        RectangleButtonBundle sessions = ui.buildRectangularButtonBundle(graphBtnW, graphBtnH,
"Sessions / Week", "smallerButtonText", zero, zero);
        HBox buttons = new HBox(vSpacing, accuracy.button(), studyTime.button(),
weakDecks.button(), sessions.button());
        ui.setPosition(buttons, graphButtonX, graphButtonY);
        // Create the chart group
        Group chartGroup = new Group();
        ui.setPosition(chartGroup, graphX, graphY);
        // Load all session history
        ArrayList<SessionRecord> historicalSessions = stats.loadSessionRecords(ac.getEmail());
        // Initial chart
        chartGroup.getChildren().addAll(buildAccuracyChart(historicalSessions));
        // On button mouseclicks
        accuracy.button().setOnMouseClicked(_->
chartGroup.getChildren().setAll(buildAccuracyChart(historicalSessions)));
        studyTime.button().setOnMouseClicked(_->
chartGroup.getChildren().setAll(buildStudyTime(historicalSessions)));
        weakDecks.button().setOnMouseClicked(_->
chartGroup.getChildren().setAll(buildWeakDecksChart(historicalSessions)));
        sessions.button().setOnMouseClicked(_->
chartGroup.getChildren().setAll(buildSessionsPerWeekChart(historicalSessions)));
        nav.switchContent(contentLayer, buttons, chartGroup, getBackView(contentLayer));
    }

    // Accuracy Line Graph
    private LineChart<String, Number> buildAccuracyChart(ArrayList<SessionRecord>
historicalSessions) {
        // Create the axis
        CategoryAxis xAxis = new CategoryAxis();
        NumberAxis yAxis = new NumberAxis(0, 100.0, 5);
        xAxis.setLabel("Session");
        yAxis.setLabel("Accuracy / %");
        // Create the chart
        LineChart<String, Number> lineGraph = new LineChart<>(xAxis, yAxis);
        lineGraph.getStyleClass().add("charts");
        lineGraph.setTitle("Accuracy over Time");
        lineGraph.setMaxSize(graphW, graphH);
        lineGraph.setPrefSize(graphW, graphH);
        lineGraph.setLegendVisible(false);
        lineGraph.setCreateSymbols(true); // data points at each plot point
        // Populate the graph
        XYChart.Series<String, Number> series = new XYChart.Series<>();
        int sessionNumber = 1;
        for(SessionRecord s : historicalSessions){
            series.getData().add(
                    new XYChart.Data<>("Session " + sessionNumber,
                            s.accuracy()));
                    sessionNumber++;
        }
```

```
        lineGraph.getData().add(series);
        return lineGraph;
    }
    // Study time per session line graph
    private LineChart<String, Number> buildStudyTime(ArrayList<SessionRecord> historicalSessions)
{
        // Create the axis
        CategoryAxis xAxis = new CategoryAxis();
        NumberAxis yAxis = new NumberAxis();
        xAxis.setLabel("Session");
        yAxis.setLabel("Time / m");
        // Create the chart
        LineChart<String, Number> lineGraph = new LineChart<>(xAxis, yAxis);
        lineGraph.getStyleClass().add("charts");
        lineGraph.setTitle("Study Time per Session");
        lineGraph.setMaxSize(graphW, graphH);
        lineGraph.setPrefSize(graphW, graphH);
        lineGraph.setLegendVisible(false);
        lineGraph.setCreateSymbols(true);
        // Populate the graph
        XYChart.Series<String, Number> series = new XYChart.Series<>();
        int sessionNumber = 1;
        for(SessionRecord s : historicalSessions){
            double minutes = s.studySeconds() / 60.0;
            minutes = Math.round(minutes * 10.0) / 10.0;
            series.getData().add(
                    new XYChart.Data<>("Session " + sessionNumber,
                            minutes));
            sessionNumber++;
        }
        lineGraph.getData().add(series);
        return lineGraph;
    }
    // Create the weak decks bar chart
    private BarChart<String, Number> buildWeakDecksChart(ArrayList<SessionRecord>
historicalSessions){
        // Create the axis
        CategoryAxis xAxis = new CategoryAxis();
        NumberAxis yAxis = new NumberAxis();
        xAxis.setLabel("Deck");
        yAxis.setLabel("Average Accuracy / %");
        // Create the chart
        BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
        chart.getStyleClass().add("charts");
        chart.setTitle("Weakest Decks");
        chart.setMaxSize(graphW, graphH);
        chart.setPrefSize(graphW, graphH);
        chart.setLegendVisible(false);
        // Group sessions by deck and avg accuracy
        HashMap<String, ArrayList<Double>> deckAccuracies = new HashMap<>();
        // Iterate through each session record
        for(SessionRecord s : historicalSessions){
            // If deck name does not exist in the map, create a new ArrayList and key for it
            // Session accuracy added to list
            deckAccuracies.computeIfAbsent(s.deckName(), _->
                    new ArrayList<>()).add(s.accuracy());
        }
        // Create the series to populate the chart
        XYChart.Series<String, Number> series = new XYChart.Series<>();
        // convert each map entry from deck name and list of accuracies to deck name and average
accuracies
        deckAccuracies.entrySet().stream().map(
                // Calculate the avg of the accuracy values in the list
                e ->{
                    double average = e.getValue().stream()
                            // convert to primitive doubles instead of double objects
                            // Find average
                            .mapToDouble(Double::doubleValue).average().orElse(0);
                    return Map.entry(e.getKey(), average);
                    // Sort in ascending order
                }).sorted(Map.Entry.comparingByValue())
                // Add each entry to the series
                .forEach(e -> series.getData().add(new XYChart.Data<>(e.getKey(),
Math.round(e.getValue() * 10.0) / 10.0)));
        chart.getData().add(series);
        return chart;
    }
    // Sessions per Week -> bar chart
    private BarChart<String, Number> buildSessionsPerWeekChart(ArrayList<SessionRecord>
historicalSessions){
        // Create the axis
        CategoryAxis xAxis = new CategoryAxis();
```

```
        NumberAxis yAxis = new NumberAxis();
        xAxis.setLabel("Week");
        yAxis.setLabel("Sessions");
        // Create the chart
        BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
        chart.getStyleClass().add("charts");
        chart.setLegendVisible(false);
        chart.setTitle("Sessions per Week");
        chart.setMaxSize(graphW, graphH);
        chart.setPrefSize(graphW, graphH);
        // Populate the chart with a hash map
        HashMap<String, Integer> weeklySessions = new HashMap<>();
        for(SessionRecord s : historicalSessions){
            // Split date into day, month and year
            String[] date = s.date().split("/");
            // Ensure date is valid
            if(date.length == 3){
                // Convert the date into a week number and label it
                String week = "W" + getWeek(date[0], date[1], date[2]);
                // Add session to correct week, if week exists increment, if not create it with
value 1
                weeklySessions.merge(week, 1, Integer::sum);
            }
        }
        XYChart.Series<String, Number> series = new XYChart.Series<>();
        // Convert to stream and sort it by week
        weeklySessions.entrySet().stream().sorted((a,b) -> {
                        int weekA = Integer.parseInt(a.getKey().substring(1)); // Remove the W
and compare
                        int weekB = Integer.parseInt(b.getKey().substring(1));
                        return Integer.compare(weekA, weekB);
                        })
                // For each week, add the session count to chart
                .forEach(e -> series.getData().add(new XYChart.Data<>(e.getKey(),
e.getValue())));
        chart.getData().add(series);
        return chart;
    }
    // Function to determine the week number from a date
    private int getWeek(String day, String month, String year){
        // Convert into a date object
        LocalDate date = LocalDate.of(Integer.parseInt(year), Integer.parseInt(month),
Integer.parseInt(day));
        // Return the week number of the year (ISO, weeks start on Monday)
        return date.get(IsoFields.WEEK_OF_WEEK_BASED_YEAR);
    }
    // Procedure switching the scene the FMS
    private void fcManagementMouseClick() throws IOException {
        ScreenManager.switchScene(new FlashcardManagementScreen(ac, sc, fc, stats).create(),
ac.getEmail());
    }

    // Procedure creating the Import / Export Content
    private void buildImportExport(Group contentLayer){

        Window owner = contentLayer.getScene().getWindow();
        RectangleButtonBundle importCSV = ui.buildRectangularButtonBundle(rectW, rectH, "Import
CSV", "buttonText", threeX, threeY1);
        ImageView importIcon = createIcon(iconX, threeY1+iconOffset,
"/images/RevisionSystemScreen/upload.png");
        importCSV.button().setOnMouseClicked(_ -> {
            try {
                if(fc.importCSV(owner)) CustomAlert.success(owner, "Successfully imported CSV
file");
            } catch (IOException e) {
                CustomAlert.error(owner, "Could not import, try again later");
                throw new RuntimeException(e);
            }
        });

        RectangleButtonBundle exportCSV = ui.buildRectangularButtonBundle(rectW, rectH, "Export
CSV", "buttonText", threeX, threeY2);
        ImageView exportIcon = createIcon(threeX, threeY2+iconOffset,
"/images/RevisionSystemScreen/textFile.png");
        exportCSV.button().setOnMouseClicked(_ -> {
            try {
                boolean exported = fc.export(owner, false);
                if(exported) CustomAlert.success(owner, "Successfully exported CSV file");
                if(!exported) CustomAlert.error(owner, "Could not export CSV file");
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
```

```
        });

        RectangleButtonBundle exportHTML = ui.buildRectangularButtonBundle(rectW, rectH, "Export
HTML", "buttonText", threeX, threeY3);
        ImageView htmlIcon = createIcon(threeX, threeY3+iconOffset,
"/images/RevisionSystemScreen/file.png");
        exportHTML.button().setOnMouseClicked(_ -> {
            try {
                boolean exported = fc.export(owner, true);
                if(exported) CustomAlert.success(owner, "Successfully exported CSV file as HTML
\n You can print this to a PDF");
                if(!exported) CustomAlert.error(owner, "Could not export CSV file");
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        ImageView hintView = ui.getHintView();
        String importInfo = "You can import a CSV which follows the " +
                "question, answer format with speech marks \n surrounding both and new lines
indicated by backslash and n";
        String exportCSVInfo = "You will get an exported csv following the same lines as import
versions";
        String exportHTMLInfo = "You can export your decks to a HTML page, and then print to PDF
for portability";
        hintView.setOnMouseClicked(_-> CustomAlert.info(owner, importInfo + "\n" + "\n" +
exportCSVInfo + "\n" + "\n" + exportHTMLInfo));

        Group importExportButtons = new Group(importCSV.button(), exportCSV.button(),
exportHTML.button(), importIcon, exportIcon, htmlIcon);
        nav.switchContent(contentLayer, getBackView(contentLayer), importExportButtons,
hintView);
    }
    // Switch scene when study modes is clicked
    private void smMouseClick() throws IOException {
        ScreenManager.switchScene(new StudyModesScreen(ac, sc, fc, stats).create(),
ac.getEmail());
    }

}
```

5.2.6 SettingsScreen.java

```
package Screens;

import Components.CustomAlert;
import Components.ScreenManager;
import Components.UIComponents;
import Controllers.AuthenticationController;
import Controllers.SettingsController;
import Models.Settings;
import RecordClasses.PasswordBundle;
import RecordClasses.RectangleButtonBundle;
import RecordClasses.TextBundle;
import javafx.event.ActionEvent;
import javafx.event.Event;
import javafx.event.EventHandler;
import javafx.geometry.Side;
import javafx.scene.Group;
import javafx.scene.Node;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.image.ImageView;
import javafx.scene.layout.Pane;
import javafx.scene.layout.StackPane;
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import javafx.stage.Window;

import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;
import java.security.NoSuchAlgorithmException;
import java.util.Objects;

public class SettingsScreen extends SystemScreens{
    // Constants
```

```
    private static final int titleX = 290;
    private static final int y1 = 241;
    private static final int oneTimeOffset = -2;
    private static final int y2 = 412;
    private static final int y3 = 583;
    private static final int y4 = 755;
    private static final int saveY = 755;
    private static final int iTX = 180;
    private static final int iTY = 270;
    private static final int ssBtnY = 300;
    private static final int othersY1 = 315;
    private static final int othersY2 = 485;
    private static final int othersY3 = 656;
    private static final int one = 1;
    private static final int four = 4;
    private static final int ten = 10;
    private static final int fieldX = 388;
    private static final int fieldH = 98;
    private static final int passwordIconY1 = 247;
    private static final int passwordIconY2 = 419;
    private static final int passwordIconY3 = 589;
    private static final int passwordRIconX = 1535;
    private static final int passwordRIconY1 = 267;
    private static final int passwordRIconY2 = 438;
    private static final int passwordRIconY3 = 609;
    private static final int fieldY1 = 241;
    // Common
    private static final int labelX = 150;
    private static final int valueX = 1500;
    private static final int inputX = 1600;
    private static final int sliderX = 250;
    private static final int toggleX = 1200;
    // SFX
    private static final int sfxY = 270;
    private static final int sfxInputY = 290;
    private static final int sfxSliderY = 300;
    // TTS
    private static final int ttsY = 470;
    private static final int ttsInputY = 490;
    private static final int ttsSliderY = 500;
    // Toggles
    private static final int toggleMuteY = 630;
    private static final int toggleMuteBoxY = 560;
    private static final int toggleTTSY = 730;
    private static final int toggleTTSBoxY = 660;
    // Darkness
    private static final int darknessY = 300;
    private static final int darknessInputY = 490;
    private static final int darknessSliderY = 490;

    private final Group background = createBackground("System Settings");
    private final AuthenticationController ac;
    private final SettingsController sC;
    private final Settings currentSettings;
    private Settings draftSettings;
    private boolean saved = false;
    public SettingsScreen(AuthenticationController ac, SettingsController sC){
        this.ac = ac;
        this.sC = sC;
        // Load settings if not already loaded
        if (sC.getCurrentSettings() == null) {
            try {
                Settings userSettings = sC.loadSettings(ac.getEmail());
                sC.setCurrentSettings(userSettings);
            } catch (IOException e) {
                // Default Setting set as a fallback
                Settings userSettings = new Settings();
                sC.setCurrentSettings(userSettings);
            }
        }
        this.currentSettings = sC.getCurrentSettings();
        // Use existing draft if available, otherwise create new one
        if (sC.getDraftSettings() == null) {
            this.draftSettings = currentSettings.copy();
            sC.setDraftSettings(this.draftSettings);
        } else {
            this.draftSettings = sC.getDraftSettings();
        }
    }

    // Function returning the scene displays all options in settings
    public Scene create() {
```

```
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        Scene scene = new Scene(root, sceneW, sceneH);
        getTitle().setLayoutX(titleX);
        ImageView back = getSwitchScreenBackView(ac);

        Group contentLayer = new Group();
        // Creating Buttons for Settings
        RectangleButtonBundle stylesheetOptions = ui.buildRectangularButtonBundle(rectW, rectH,
"Stylesheet Options", "buttonText", fourX, y1);
        ImageView sOIcon = createIcon(fourX, y1+oneTimeOffset,
"/images/SettingsScreen/stylesheets.png");
        stylesheetOptions.button().setOnMouseClicked(_->
buildStylesheetOptionsContent(contentLayer, scene));

        RectangleButtonBundle accessibility = ui.buildRectangularButtonBundle(rectW, rectH,
"Accessibility", "buttonText", fourX, y2);
        ImageView aIcon = createIcon(fourX, y2, "/images/SettingsScreen/accessibility.png");
        accessibility.button().setOnMouseClicked(_-> {
            try {
                buildAccessibilityOptionsContent(contentLayer, scene);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        RectangleButtonBundle audioControl = ui.buildRectangularButtonBundle(rectW, rectH, "Audio
Control", "buttonText", fourX, y3);
        ImageView aCIcon = createIcon(fourX, y3, "/images/SettingsScreen/audio.png");
        audioControl.button().setOnMouseClicked(_-> buildAudioControlContent(contentLayer,
scene));

        RectangleButtonBundle others = ui.buildRectangularButtonBundle(rectW, rectH, "Others",
"buttonText", fourX, y4);
        ImageView oIcon = createIcon(fourX, y4, "/images/SettingsScreen/others.png");
        others.button().setOnMouseClicked(_ -> buildOthersContent(contentLayer));

        Group menuLayer = new Group(stylesheetOptions.button(), accessibility.button(),
audioControl.button(), others.button(), sOIcon, aIcon, aCIcon, oIcon);
        contentLayer.getChildren().add(menuLayer);
        root.getChildren().addAll(background, back, contentLayer);
        return scene;
    }

    // Re-usable save button
    private RectangleButtonBundle saveRect(Scene scene){
        RectangleButtonBundle save = ui.buildRectangularButtonBundle(rectW, rectH, "Save
Settings", "buttonText", fourX, saveY);
        save.button().setOnMouseClicked(_ -> {
            try {
                sC.saveSettings(draftSettings);
                sC.setCurrentSettings(draftSettings);
                sC.setDraftSettings(draftSettings.copy());
                CustomAlert.success(scene.getWindow(), "Success, changes saved.");
                saved = true;
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        return save;
    }

    // Procedure creating the UI for user to pick between stylesheet options
    private void buildStylesheetOptionsContent(Group contentLayer, Scene scene) {
        saved = false;

        Text instructionText = new Text(iTX, iTY, "Pick your stylesheet option:");
        instructionText.getStyleClass().add("instructionText");

        MenuItem darkMode = new MenuItem("Dark Mode");
        MenuItem lightMode = new MenuItem("Light Mode");
        MenuItem highContrast = new MenuItem("High Contrast");

        darkMode.getStyleClass().add("options");
        lightMode.getStyleClass().add("options");
        highContrast.getStyleClass().add("options");

        RectangleButtonBundle styles = ui.buildRectangularButtonBundle(rectW, rectH, "Stylesheet
Options", "buttonText", fourX, ssBtnY);
        ContextMenu menu = new ContextMenu(darkMode, lightMode, highContrast);
        menu.getStyleClass().add("options-menu");
        styles.button().setOnMouseClicked(_-> menu.show(styles.button(), Side.BOTTOM, zero,
```

```
zero));

        String infoMessage = "This is only a preview. \n The change will persist until you exist
settings. \n Press save to ensure changes persist";
        darkMode.setOnAction(_ -> {
            draftSettings.setTheme("dark");
            styles.buttonText().setText("Dark Mode");
            previewTheme(scene, draftSettings.getTheme());
            createInfoAlert(scene, infoMessage);
        });
        lightMode.setOnAction(_ -> {
            draftSettings.setTheme("light");
            styles.buttonText().setText("Light Mode");
            previewTheme(scene, draftSettings.getTheme());
            createInfoAlert(scene, infoMessage);
        });
        highContrast.setOnAction(_ -> {
            draftSettings.setTheme("highContrast");
            styles.buttonText().setText("High Contrast Mode");
            previewTheme(scene, draftSettings.getTheme());
            createInfoAlert(scene, infoMessage);
        });

        RectangleButtonBundle save = saveRect(scene);
        ImageView backView = getBackView(contentLayer);
        backView.setOnMouseClicked(_ -> {
            boolean isConfirmed = false;
            if(!saved) isConfirmed = CustomAlert.confirm(scene.getWindow(), "Confirmation of
Save", "Changes only persist if you click save.");
            if(isConfirmed || saved){
                nav.goBack(contentLayer);
            }
        });
        nav.switchContent(contentLayer, instructionText, styles.button(), save.button(),
backView);
    }
    // ERROR: I APPLIED IT TO THE FRICKING PANE INSTEAD OF THE SCENE, TOOK ME 2 HOURS TO REALISE
    // Theme selected with be applied to their screen to give the user a preview
    private void previewTheme(Scene root, String theme){
        // Removing any style sheet applied currently to avoid theme conflict/stacking
        root.getStylesheets().clear();
        // styles.css contains shared layout, fonts and component styling

root.getStylesheets().add(Objects.requireNonNull(getClass().getResource("/styles/styles.css")).to
ExternalForm());
        String themePath;
        switch (theme){
            case "light" -> themePath = "/styles/light.css";
            case "highContrast" -> themePath = "/styles/highContrast.css";
            default -> themePath = "/styles/dark.css";
        }

root.getStylesheets().add(Objects.requireNonNull(getClass().getResource(themePath)).toExternalFor
m());
    }
    // Displaying information alert windows to user
    private void createInfoAlert(Scene scene, String message){
        Window alertWindow = scene.getWindow();
        CustomAlert.info(alertWindow, message);
    }

    // Procedure to build accessibility settings content
    private void buildAccessibilityOptionsContent(Group contentLayer, Scene scene) throws
IOException {
        saved = false;
        Rectangle brightnessOverlay = getBrightnessOverlay(scene);
        System.out.println("Loading darkness value: " + draftSettings.getDarkness()); // DEBUG
        brightnessOverlay.setOpacity(draftSettings.getDarkness());
        Text darknessText = new Text("Darkness Overlay:");
        darknessText.getStyleClass().add("instructionText");
        darknessText.setLayoutX(labelX);
        darknessText.setLayoutY(darknessY);

        Text darknessValue = new Text(String.format("%.0f%%", draftSettings.getDarkness() *
100));
        darknessValue.getStyleClass().add("instructionText");
        darknessValue.setLayoutX(valueX);
        darknessValue.setLayoutY(darknessY);

        TextField darknessInput = new TextField();
        darknessInput.getStyleClass().add("sliderInputAlt");
        darknessInput.setLayoutX(inputX);
```

```
        darknessInput.setLayoutY(darknessInputY);

        darknessInput.setText(String.format("%.0f%%", draftSettings.getDarkness() * 100));

        Slider darknessSlider = new Slider(0, 0.8, draftSettings.getDarkness());
        // Darkness/Brightness Control
        darknessSlider.setLayoutX(sliderX);
        darknessSlider.setLayoutY(darknessSliderY);
        darknessSlider.setMajorTickUnit(0.1);
        darknessSlider.valueProperty().addListener((obs, oldVal, newVal) -> {
            double darkness = newVal.doubleValue();
            draftSettings.setDarkness(darkness);
            brightnessOverlay.setOpacity(darkness);
            System.out.println("Set darkness to: " + darkness); // DEBUG
            darknessValue.setText(String.format("%.0f%%", darkness * 100));
            darknessInput.setText(String.format("%.0f%%", darkness * 100));
        });

        String infoMessage = "Changes to brightness will only persist after save button is
pressed";

        // Can be changed via user input too
        darknessInput.setOnAction(_ -> {
            try{
                double inputValue = Double.parseDouble(darknessInput.getText());
                // Clamp between 0-80
                inputValue = Math.max(0, Math.min(80, inputValue));
                double darkness = inputValue / 100.0;
                draftSettings.setDarkness(darkness);
                brightnessOverlay.setOpacity(darkness);
                darknessSlider.setValue(darkness);
                darknessValue.setText(String.format("%.0f%%", darkness * 100));
                darknessInput.setText(String.format("%.0f", inputValue));
            } catch (NumberFormatException e) {
                // Reset to current value if invalid input
                darknessInput.setText(String.format("%.0f", draftSettings.getDarkness() * 100));
            }
        });
        RectangleButtonBundle save = saveRect(scene);
        ImageView backView = getBackView(contentLayer);
        backView.setOnMouseClicked(_ -> {
            boolean isConfirmed = false;
            if(!saved) isConfirmed = CustomAlert.confirm(scene.getWindow(), "Confirmation of
Save", infoMessage);
            if(isConfirmed || saved){
                nav.goBack(contentLayer);
            }
        });
        nav.switchContent(contentLayer, darknessText, darknessValue, darknessSlider,
darknessInput, save.button(), backView);
    }
    // Function to get the scene's original brightnessOverlay
    private Rectangle getBrightnessOverlay(Scene scene) throws IOException {
        Rectangle brightnessOverlay = null;
        if(scene.getRoot() instanceof StackPane wrapper){
            // Check every node in the StackPane to get the brightness overlay and set it to the
new one
            for(var node : wrapper.getChildren()){
                if(node instanceof Rectangle rect && rect.getWidth() == sceneW &&
rect.getHeight() == sceneH){
                    brightnessOverlay = rect;
                    break;
                }
            }
        }
        if(brightnessOverlay == null){
            brightnessOverlay = ScreenManager.getBrightnessOverlay(ac.getEmail());
        }
        return brightnessOverlay;
    }

    // Procedure to build audio settings content, with sliders and check boxes for
    private void buildAudioControlContent(Group contentLayer, Scene scene){
        saved = false;
        Text sfxText = new Text("Sound Effects Volume: ");
        sfxText.getStyleClass().add("instructionText");
        sfxText.setLayoutX(labelX);
        sfxText.setLayoutY(sfxY);

        Text sfxValue = new Text(String.format("%.0f%%", draftSettings.getSFXVolume() * 100));
        sfxValue.getStyleClass().add("instructionText");
        sfxValue.setLayoutX(valueX);
```

```
        sfxValue.setLayoutY(sfxY);

        TextField sfxInput = new TextField(String.format("%.0f%%", draftSettings.getSFXVolume() *
100));
        sfxInput.getStyleClass().add("sliderInputAlt");
        sfxInput.setLayoutX(inputX);
        sfxInput.setLayoutY(sfxInputY);

        Slider sfxSlider = new Slider(0, 1.0, draftSettings.getSFXVolume());
        sfxSlider.setLayoutX(sliderX);
        sfxSlider.setLayoutY(sfxSliderY);
        sfxSlider.setMajorTickUnit(0.1);

        Text  ttsText = new Text("Text-To-Speech Volume: ");
        ttsText.getStyleClass().add("instructionText");
        ttsText.setLayoutX(labelX);
        ttsText.setLayoutY(ttsY);

        Text ttsValue = new Text(String.format("%.0f%%", draftSettings.getTTSVolume() * 100));
        ttsValue.getStyleClass().add("instructionText");
        ttsValue.setLayoutX(valueX);
        ttsValue.setLayoutY(ttsY);

        TextField ttsInput = new TextField(String.format("%.0f%%", draftSettings.getTTSVolume() *
100));
        ttsInput.getStyleClass().add("sliderInputAlt");
        ttsInput.setLayoutX(inputX);
        ttsInput.setLayoutY(ttsInputY);

        Slider ttsSlider = new Slider(0, 1.0, draftSettings.getTTSVolume());
        ttsSlider.setLayoutX(sliderX);
        ttsSlider.setLayoutY(ttsSliderY);
        ttsSlider.setMajorTickUnit(0.1);

        Text toggleMuteText = new Text("Toggle Mute: ");
        toggleMuteText.getStyleClass().add("instructionText");
        toggleMuteText.setLayoutX(labelX);
        toggleMuteText.setLayoutY(toggleMuteY);

        CheckBox toggleMute = new CheckBox();
        toggleMute.setSelected(draftSettings.isMuted());
        toggleMute.getStyleClass().add("toggles");
        toggleMute.setLayoutX(toggleX);
        toggleMute.setLayoutY(toggleMuteBoxY);
        toggleMute.setOnAction(_ -> {
            draftSettings.setMuted(toggleMute.isSelected());
            onToggle(toggleMute, sfxInput, sfxValue, sfxSlider, false);
        });

        Text toggleTTSText = new Text("Toggle TTS: ");
        toggleTTSText.getStyleClass().add("instructionText");
        toggleTTSText.setLayoutX(labelX);
        toggleTTSText.setLayoutY(toggleTTSY);

        CheckBox toggleTTS = new CheckBox();
        toggleTTS.setSelected(draftSettings.isTTSEnabled());
        toggleTTS.getStyleClass().add("toggles");
        toggleTTS.setLayoutX(toggleX);
        toggleTTS.setLayoutY(toggleTTSBoxY);
        toggleTTS.setOnAction(_ -> {
            draftSettings.setTTSEnabled(toggleTTS.isSelected());
            onToggle(toggleTTS, ttsInput, ttsValue, ttsSlider, true);
        });

        ttsSlider.valueProperty().addListener((obs, oldVal, newVal) -> {
            double volume = newVal.doubleValue();
            enforceConsistency(volume, toggleTTS, draftSettings, true);
            draftSettings.setTTSVolume(volume);
            ttsInput.setText(String.format("%.0f%%", draftSettings.getTTSVolume() * 100));
            ttsValue.setText(String.format("%.0f%%", draftSettings.getTTSVolume() * 100));
            ttsSlider.setOnMouseReleased(_-> playTestSound(volume));
        });

        ttsInput.setOnAction(_ -> {
            try{
                double inputValue = Double.parseDouble(ttsInput.getText());
                // Clamp between 0-100
                inputValue = Math.max(0, Math.min(100 , inputValue));
                double ttsVol = inputValue / 100.0;
                enforceConsistency(ttsVol, toggleTTS, draftSettings, true);
                draftSettings.setTTSVolume(ttsVol);
                playTestSound(ttsVol);
```

```
                ttsSlider.setValue(ttsVol);
                ttsValue.setText(String.format("%.0f%%", ttsVol * 100));
                ttsInput.setText(String.format("%.0f", inputValue));
            } catch (NumberFormatException e) {
                // Reset to current value if invalid input
                sfxInput.setText(String.format("%.0f", draftSettings.getTTSVolume() * 100));
            }
        });

        sfxSlider.valueProperty().addListener((obs, oldVal, newVal) -> {
            double volume = newVal.doubleValue();
            enforceConsistency(volume, toggleMute, draftSettings, false);
            draftSettings.setSFXVolume(volume);
            sfxInput.setText(String.format("%.0f%%", draftSettings.getSFXVolume() * 100));
            sfxValue.setText(String.format("%.0f%%", draftSettings.getSFXVolume() * 100));
            sfxSlider.setOnMouseReleased(_-> playTestSound(volume));
        });

        sfxInput.setOnAction(_ -> {
            try{
                double inputValue = Double.parseDouble(sfxInput.getText());
                // Clamp between 0-100
                inputValue = Math.max(0, Math.min(100 , inputValue));
                double sfxVol = inputValue / 100.0;
                enforceConsistency(sfxVol, toggleMute, draftSettings, false);
                draftSettings.setSFXVolume(sfxVol);
                playTestSound(sfxVol);
                sfxSlider.setValue(sfxVol);
                sfxValue.setText(String.format("%.0f%%", sfxVol * 100));
                sfxInput.setText(String.format("%.0f", inputValue));
            } catch (NumberFormatException e) {
                // Reset to current value if invalid input
                sfxInput.setText(String.format("%.0f", draftSettings.getSFXVolume() * 100));
            }
        });

        RectangleButtonBundle save = saveRect(scene);
        ImageView backView = getBackView(contentLayer);
        backView.setOnMouseClicked(_ -> {
            boolean isConfirmed = false;
            if(!saved) isConfirmed = CustomAlert.confirm(scene.getWindow(), "Confirmation of
Save", "Changes will only be saved when the save button is clicked");
            if(isConfirmed || saved){
                nav.goBack(contentLayer);
            }
        });
        Group texts = new Group(sfxText, sfxValue, sfxInput, ttsText, ttsValue, ttsInput,
toggleMuteText, toggleTTSText);
        nav.switchContent(contentLayer, sfxSlider, ttsSlider, toggleMute, toggleTTS, texts,
save.button(), backView);
    }
    // Update all checkboxes and sliders on specific change
    private void enforceConsistency(double volume, CheckBox checkBox, Settings draftSettings,
boolean isTTS){
        if(isTTS){
            if(volume != 0){
                draftSettings.setTTSEnabled(true);
                checkBox.setSelected(draftSettings.isTTSEnabled());
            }else{
                draftSettings.setTTSEnabled(false);
                checkBox.setSelected(draftSettings.isTTSEnabled());
            }
        }else{
            if(volume != 0){
                draftSettings.setMuted(false);
                checkBox.setSelected(draftSettings.isMuted());
            }else{
                draftSettings.setMuted(true);
                checkBox.setSelected(draftSettings.isMuted());
            }
        }
    }
    // Procedure which changes screen info on check boxes
    private void onToggle(CheckBox checkBox, TextField inputField, Text value, Slider slider,
boolean isTTS) {
        double volume;
        if (checkBox.isSelected()) {
            if (isTTS) {
                // get and set the default value for tts volume (80%)
                volume = new Settings().getTTSVolume();
                draftSettings.setTTSVolume(volume);
            } else {
```

```
                // Muted value
                volume = 0.0;
                draftSettings.setSFXVolume(volume);
            }
            // Change info on screen
            inputField.setText(String.format("%.0f%%", volume * 100));
            value.setText(String.format("%.0f%%", volume * 100));
            slider.setValue(volume);
        } else {
            if (isTTS) {
                volume = 0.0;
                draftSettings.setTTSVolume(volume);
            } else {
                // get and set default value for sfx (60%)
                volume = new Settings().getSFXVolume();
                draftSettings.setSFXVolume(volume);
            }
            // Change info on screen
            inputField.setText("0%");
            value.setText("0%");
            slider.setValue(volume);
        }
        // Play test sound
        playTestSound(volume);
    }
    // procedure which will play the test sound when volume is altered
    private void playTestSound(double volume) {
        try {
            // Create a temp file on the system to store the sound
            // JavaFX media requires a URI
            InputStream is = getClass().getResourceAsStream("/sounds/testSound.wav");
            File tempFile = File.createTempFile("testSound", ".wav");
            // Ensure it is deleted on exit
            tempFile.deleteOnExit();
            // Copy sound data from sound file to file
            Files.copy(Objects.requireNonNull(is), tempFile.toPath(),
StandardCopyOption.REPLACE_EXISTING);

            Media testSound = new Media(tempFile.toURI().toString());
            MediaPlayer player = new MediaPlayer(testSound);
            player.setVolume(volume);
            player.play();
        } catch (Exception _){}
    }

    // Procedure to create and display all buttons when Other Options is selected
    private void buildOthersContent(Group contentLayer) {
        RectangleButtonBundle changeEmail = ui.buildRectangularButtonBundle(rectW, rectH, "Change
Email", "buttonText", threeX, othersY1);
        ImageView emailIcon = createIcon(threeX, othersY1+iconOffset,
"/images/SettingsScreen/whiteUser.png");
        changeEmail.button().setOnMouseClicked(_ -> buildChangeEmail(contentLayer));

        RectangleButtonBundle changeRw = ui.buildRectangularButtonBundle(rectW, rectH, "Change
Word", "buttonText", threeX, othersY2);
        ImageView recoveryIcon = createIcon(threeX, othersY2+iconOffset,
"/images/SettingsScreen/whiteRecovery.png");
        changeRw.button().setOnMouseClicked(_ -> buildChangeRW(contentLayer));

        RectangleButtonBundle changePw = ui.buildRectangularButtonBundle(rectW, rectH, "Change
Password", "buttonText", threeX, othersY3);
        ImageView pwIcon = createIcon(threeX, othersY3+iconOffset,
"/images/SettingsScreen/whiteLock.png");
        changePw.button().setOnMouseClicked(_ -> buildChangePassword(contentLayer));

        Group otherOptionsLayer = new Group(changeEmail.button(), changePw.button(),
changeRw.button(), emailIcon, recoveryIcon, pwIcon);

        nav.switchContent(contentLayer, otherOptionsLayer, getBackView(contentLayer));
    }

    // Procedure to create and display all input fields and buttons when Change Email is selected
    private void buildChangeEmail(Group contentLayer) {
        PasswordBundle passwordInput = originalPasswordInput();
        String userIcon =  "/images/AuthScreen/user.png";
        TextBundle newEmailInput = ui.buildTextFieldInput(threeX, y2, rectW, fieldH , fieldX,
y2+ten, "Enter Email", userIcon, threeX-one,  y2+four, iconW, iconH);
        newEmailInput.tf().getStyleClass().add("resetTextField");
        TextBundle confirmNewEmailInput = ui.buildTextFieldInput(threeX, y3, rectW, fieldH,
fieldX, y3+ten, "Confirm Email", userIcon, threeX-one,  y3+four, iconW, iconH);
        confirmNewEmailInput.tf().getStyleClass().add("resetTextField");
        RectangleButtonBundle changeEmail = ui.buildRectangularButtonBundle(rectW, rectH, "Change
```

```
Email", "buttonText", threeX, y4);

        UIComponents.attachValidator(newEmailInput.tf(), UIComponents::isValidEmail, "error");
        UIComponents.attachValidator(confirmNewEmailInput.tf(), UIComponents::isValidEmail,
"error");

        EventHandler<ActionEvent> changeEmailHandler =
                event -> {
                    try {
                        changeEmailSubmit(event, passwordInput, newEmailInput,
confirmNewEmailInput, contentLayer);
                    } catch (NoSuchAlgorithmException | IOException e) {
                        throw new RuntimeException(e);
                    }
                };
        // You can press enter to submit, convenience
        ui.bindEnter(changeEmailHandler, newEmailInput.tf(), confirmNewEmailInput.tf());

        changeEmail.button().setOnMouseClicked(e ->
                {
                    try {
                        changeEmailSubmit(e, passwordInput, newEmailInput, confirmNewEmailInput,
contentLayer);
                    } catch (NoSuchAlgorithmException | IOException ex) {
                        throw new RuntimeException(ex);
                    }
                }
        );
        Group changeEmailLayer = new Group(newEmailInput.group(),  confirmNewEmailInput.group(),
changeEmail.button(), newEmailInput.icon(), confirmNewEmailInput.icon(),
passwordInput.passwordGroup(), passwordInput.getIconGroup());

        nav.switchContent(contentLayer, changeEmailLayer, getBackView(contentLayer));
    }
    // Procedure which validates the inputs when the email change is submitted and will display
an appropriate alert.
    private void changeEmailSubmit(Event event, PasswordBundle passwordInput, TextBundle
newEmailInput, TextBundle confirmNewEmailInput, Group contentLayer) throws
NoSuchAlgorithmException, IOException {
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        boolean validInput = UIComponents.isValidEmail(newEmailInput.tf().getText()) &&
UIComponents.isValidEmail(confirmNewEmailInput.tf().getText());
        if (!validInput) {
            String emailReq = "Email should follow the standard format";
            CustomAlert.error(alertWindow, "Invalid inputs \n" + emailReq);
        } else if(ac.getEmail().equals(confirmNewEmailInput.tf().getText())) {
            CustomAlert.error(alertWindow, "This is already the email associated with the
account");
        } else if (ac.doesEmailExist(newEmailInput.tf().getText())) {
            CustomAlert.error(alertWindow, "Email already exists, try a new one");
        } else if(!ac.verifyPassword(passwordInput.password().getText(),
ac.getCurrentUser().getHashedPassword(), ac.getCurrentUser().getSaltBase64())){
            CustomAlert.error(alertWindow, "Incorrect Password ");
        }else if(!Objects.equals(newEmailInput.tf().getText(),
confirmNewEmailInput.tf().getText())) {
            CustomAlert.error(alertWindow, "Inputs do not match");
        }else{
            boolean updated = ac.changeEmail(ac.getCurrentUser(), newEmailInput.tf().getText());
            if (updated) {
                CustomAlert.success(alertWindow, "Success, you will be returned to the changing
options");
                nav.goBack(contentLayer);
            }else{
                CustomAlert.error(alertWindow, "Problems with file, try again");
            }
        }
    }
    // Procedure to create and display all input fields and buttons when Change Recovery Word is
selected
    private void buildChangeRW(Group contentLayer) {
        PasswordBundle passwordInput = originalPasswordInput();
        String recoveryIcon = "/images/AuthScreen/recover.png";
        TextBundle newRecovery = ui.buildTextFieldInput(threeX, y2, rectW, fieldH, fieldX,
y2+ten, "Enter Recovery Word", recoveryIcon, threeX-one,  y2+four, iconW, iconH);
        newRecovery.tf().getStyleClass().add("resetTextField");
        TextBundle confirmNewRecovery = ui.buildTextFieldInput(threeX, y3, rectW, fieldH, fieldX,
y3+ten, "Confirm Recovery Word", recoveryIcon, threeX-one,  y3+four, iconW, iconH);
        confirmNewRecovery.tf().getStyleClass().add("resetTextField");
        RectangleButtonBundle changeRecoveryWord = ui.buildRectangularButtonBundle(rectW, rectH,
"Change Recovery Word", "buttonText", threeX, y4);

        UIComponents.attachValidator(newRecovery.tf(), UIComponents::isValidRecoveryW, "error");
```

```
        UIComponents.attachValidator(confirmNewRecovery.tf(), UIComponents::isValidRecoveryW,
"error");

        EventHandler<ActionEvent> changeRWHandler =
                event -> {
                    try {
                        changeRWSubmit(event, passwordInput, newRecovery, confirmNewRecovery,
contentLayer);
                    } catch (NoSuchAlgorithmException | IOException e) {
                        throw new RuntimeException(e);
                    }
                };
        ui.bindEnter(changeRWHandler, newRecovery.tf(), confirmNewRecovery.tf());

        changeRecoveryWord.button().setOnMouseClicked(e ->
                {
                    try {
                        changeRWSubmit(e, passwordInput, newRecovery, confirmNewRecovery,
contentLayer);
                    } catch (NoSuchAlgorithmException | IOException ex) {
                        throw new RuntimeException(ex);
                    }
                }
        );
        Group changeRWLayer = new Group(newRecovery.group(),  confirmNewRecovery.group(),
changeRecoveryWord.button(), newRecovery.icon(), confirmNewRecovery.icon(),
passwordInput.passwordGroup(), passwordInput.getIconGroup());

        nav.switchContent(contentLayer, changeRWLayer, getBackView(contentLayer));
    }
    // Procedure which validates the inputs when the recovery word change is submitted and will
display an appropriate alert.
    private void changeRWSubmit(Event event, PasswordBundle passwordInput, TextBundle newRW,
TextBundle cNewRW, Group contentLayer) throws NoSuchAlgorithmException, IOException {
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        boolean validInput = UIComponents.isValidRecoveryW(newRW.tf().getText()) &&
UIComponents.isValidRecoveryW(cNewRW.tf().getText());
        if (!validInput) {
            String rwReq = "Recovery Word must be a minimum of 3 characters";
            CustomAlert.error(alertWindow, "Invalid inputs \n" + rwReq);
        } else if(ac.getCurrentUser().getRecoveryWord().equals(cNewRW.tf().getText())) {
            CustomAlert.error(alertWindow, "This is already the Recovery Word associated with the
account");
        } else if(!ac.verifyPassword(passwordInput.password().getText(),
ac.getCurrentUser().getHashedPassword(), ac.getCurrentUser().getSaltBase64())){
            CustomAlert.error(alertWindow, "Incorrect Password ");
        }else if(!Objects.equals(newRW.tf().getText(), cNewRW.tf().getText())) {
            CustomAlert.error(alertWindow, "Inputs do not match");
        }else{
            boolean updated = ac.changeRecoveryWord(ac.getCurrentUser(), newRW.tf().getText());
            if (updated) {
                CustomAlert.success(alertWindow, "Success, you will be returned to the changing
options");
                nav.goBack(contentLayer);
            }else{
                CustomAlert.error(alertWindow, "Problems with file, try again");
            }
        }
    }

    // Procedure to create and display all input fields and buttons when Change Password is
selected
    private void buildChangePassword(Group contentLayer) {
        PasswordBundle passwordInput = originalPasswordInput();
        PasswordBundle newPassword = ui.buildPasswordInput(threeX, y2, rectW, fieldX, y2+ten,
"Enter New Password", threeX, passwordIconY2, passwordRIconX, passwordRIconY2);
        attachStylesheetToPWField(newPassword);
        PasswordBundle cNewPassword = ui.buildPasswordInput(threeX, y3, rectW, fieldX, y3+ten,
"Enter New Password", threeX, passwordIconY3, passwordRIconX, passwordRIconY3);
        attachStylesheetToPWField(cNewPassword);
        RectangleButtonBundle changePassword = ui.buildRectangularButtonBundle(rectW, rectH,
"Change Password", "buttonText", threeX, y4);

        UIComponents.attachValidator(newPassword.password(), UIComponents::isValidPassword,
"error", newPassword.copyPw());
        UIComponents.attachValidator(cNewPassword.password(), UIComponents::isValidPassword,
"error", cNewPassword.copyPw());

        EventHandler<ActionEvent> changePasswordHandler =
                event -> {
                    try {
                        changePasswordSubmit(event, passwordInput, newPassword, cNewPassword,
```

```
contentLayer);
                    } catch (NoSuchAlgorithmException | IOException e) {
                        throw new RuntimeException(e);
                    }
                };
        ui.bindEnter(changePasswordHandler, passwordInput.password(), passwordInput.copyPw(),
cNewPassword.password(), cNewPassword.copyPw(), contentLayer);

        changePassword.button().setOnMouseClicked(e ->
                {
                    try {
                        changePasswordSubmit(e, passwordInput, newPassword, cNewPassword,
contentLayer);
                    } catch (NoSuchAlgorithmException | IOException ex) {
                        throw new RuntimeException(ex);
                    }
                }
        );
        Group changeEmailLayer = new Group(newPassword.passwordGroup(),
newPassword.getIconGroup(), cNewPassword.passwordGroup(), cNewPassword.getIconGroup(),
changePassword.button(), passwordInput.passwordGroup(), passwordInput.getIconGroup());

        nav.switchContent(contentLayer, changeEmailLayer, getBackView(contentLayer));
    }
    // Procedure which validates the inputs when the password change is submitted and will
display an appropriate alert.
    private void changePasswordSubmit(Event event, PasswordBundle oldPassword, PasswordBundle
newPassword, PasswordBundle confirmNewPassword, Group contentLayer) throws
NoSuchAlgorithmException, IOException {
        Window alertWindow = ((Node) event.getSource()).getScene().getWindow();
        boolean validInput = UIComponents.isValidPassword(newPassword.password().getText()) &&
UIComponents.isValidPassword(confirmNewPassword.password().getText());
        if (!validInput) {
            String passwordReq = "Password should have a minimum of 6 characters, including a
special character";
            CustomAlert.error(alertWindow, "Invalid inputs \n" + passwordReq);
        } else if(!ac.verifyPassword(oldPassword.password().getText(),
ac.getCurrentUser().getHashedPassword(), ac.getCurrentUser().getSaltBase64())){
            CustomAlert.error(alertWindow, "Incorrect Password ");
        }else if(!Objects.equals(confirmNewPassword.password().getText(),
newPassword.password().getText())) {
            CustomAlert.error(alertWindow, "Inputs do not match");
        }else if(ac.verifyPassword(newPassword.password().getText(),
ac.getCurrentUser().getHashedPassword(), ac.getCurrentUser().getSaltBase64())) {
            CustomAlert.error(alertWindow, "Same password as before");
        }else{
            boolean updated = ac.resetPassword(ac.getCurrentUser(),
newPassword.password().getText());
            if (updated) {
                CustomAlert.success(alertWindow, "Success, you will be returned to the changing
options");
                nav.goBack(contentLayer);
            }else{
                CustomAlert.error(alertWindow, "Problems with file, try again");
            }
        }
    }
    // Consistent field throughout all changing data options, extracted into a field
    private PasswordBundle originalPasswordInput(){
        PasswordBundle passwordInput = ui.buildPasswordInput(threeX, fieldY1, rectW, fieldX,
fieldY1+ten,"Enter Password", threeX, passwordIconY1, passwordRIconX, passwordRIconY1);
        attachStylesheetToPWField(passwordInput);
        return passwordInput;
    }
    // Adding the style class to both fields of a password bundle
    private void attachStylesheetToPWField(PasswordBundle passwordInput) {
        passwordInput.password().getStyleClass().add("resetTextField");
        passwordInput.copyPw().getStyleClass().add("resetTextField");
    }
}
```

5.2.7 FlashcardManagementScreen.java

```
package Screens;

import Components.CustomAlert;
import Controllers.AuthenticationController;
import Controllers.FlashcardController;
import Controllers.SettingsController;
import Controllers.StatsController;
import Models.Flashcard;
```

```
import RecordClasses.RectangleButtonBundle;
import RecordClasses.TextAreaBundle;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ListChangeListener;
import javafx.collections.ObservableList;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.image.ImageView;
import javafx.scene.input.ClipboardContent;
import javafx.scene.input.Dragboard;
import javafx.scene.input.TransferMode;
import javafx.scene.layout.*;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/* Class to be used to create the interface of the FlashcardManagementScreen, containing logic on
creating,
* editing, and organising flashcard and flashcard decks.*/
public class FlashcardManagementScreen extends SystemScreens{
    // Constants
    private static final int titleX = 200;
    // Creation
    private static final int lowerQLimit = 10;
    private static final int upperQLimit = 300;
    private static final int lowerALimit = 1;
    private static final int upperALimit = 600;
    private static final int addW = 600;
    private static final int saveW = 1000;
    private static final int saveY = 755;
    private static final int deckSizeLimit = 60;
    private static final int hX = 140;
    private static final int hY = 210;
    // TableView
    private static final int cMaxW = 250;
    private static final int btnW = 80;
    private static final int btnH = 25;
    // Modification
    private static final int minHeight = 600;
    private static final int spacing = 5;
    // Organisation
    private static final int folderBtnW = 300;
    private static final int folderBtnH = 80;
    private static final int tableHeight = 700;
    private static final int displayX = 125;
    private static final int displayY = 200;
    // Controllers to be used for core system functionality
    private final AuthenticationController ac;
    private final SettingsController sc;
    private final FlashcardController fc;
    private final StatsController stats;
    // Global Scene for creating alert windows
    private Scene scene;
    // Observable lists storing all Flashcard objects being created or being edited
    private final ObservableList<Flashcard> cardList = FXCollections.observableArrayList();
    // Decks table to be updated in save
    private final TableView<String> decksTable = new TableView<>();
    // ArrayList to be updated when saved.
    private ArrayList<String> allDecks;
    // Record class instance being used for flashcard front and back inputs
    private TextAreaBundle questionArea;
    private TextAreaBundle answerArea;

    /* Constructor class for FMScreen, controllers as parameters to maintain
    consistency across screens. */
    public FlashcardManagementScreen(AuthenticationController ac, SettingsController sC,
FlashcardController fc, StatsController stats){
        this.ac = ac;
        this.sc = sC;
        this.fc = fc;
        this.stats = stats;
    }
    // Method for creating the initial interface the user see when the screen is switched to the
FMScreen
    public Scene create(){
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        scene = new Scene(root, sceneW, sceneH);
```

```
        // Method inherited from parent class to add background
        Group background = createBackground("Flashcard Control");
        getTitle().setLayoutX(titleX);
        ImageView back = getSwitchScreenBackView(ac);

        // Group storing content within a page of the screen
        Group contentLayer = new Group();

        // Record class instances to bundle multiple logic components together
        RectangleButtonBundle create = ui.buildRectangularButtonBundle(rectW, rectH, "Create",
"buttonText", threeX, threeY1);
        ImageView createIcon = createIcon(threeX, threeY1+iconOffset,
"/images/FlashcardManagementScreen/create.png");
        create.button().setOnMouseClicked(_->buildCreationContent(contentLayer, false));

        RectangleButtonBundle modify = ui.buildRectangularButtonBundle(rectW, rectH, "Modify",
"buttonText", threeX, threeY2);
        ImageView modifyIcon = createIcon(threeX, threeY2+iconOffset,
"/images/FlashcardManagementScreen/modify.png");
        modify.button().setOnMouseClicked(_-> {
            try {
                buildModifyContent(contentLayer);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        RectangleButtonBundle organise = ui.buildRectangularButtonBundle(rectW, rectH,
"Organise", "buttonText", threeX, threeY3);
        ImageView organiseIcon = createIcon(threeX, threeY3+iconOffset,
"/images/FlashcardManagementScreen/sort.png");
        organise.button().setOnMouseClicked(_-> {
            try {
                buildOrganisationContent(contentLayer);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        Group buttons = new Group(create.button(), modify.button(), organise.button(),
createIcon, modifyIcon, organiseIcon);
        contentLayer.getChildren().add(buttons);
        root.getChildren().addAll(background, back, contentLayer);
        return scene;
    }

    // Procedure creating page for the Creation of Flashcards
    private void buildCreationContent(Group contentLayer, boolean alreadyExists) {
        // Container for input fields and add button
        VBox inputBox = new VBox(vSpacing);
        inputBox.getStyleClass().add("flashcardContent");
        questionArea = ui.createTextArea("Front of card...", upperQLimit, lowerQLimit);
        answerArea = ui.createTextArea("Back of card...", upperALimit, lowerALimit);

        // Table displaying all flashcards in the deck
        TableView<Flashcard> cardTable = buildFlashcardTableWActions();
        cardTable.getStyleClass().add("cardTable");
        // To prevent table from having flashcard data from previous modifications
        if(!alreadyExists){
            fc.setFlashcardDeckName(null);
            cardList.clear();
            cardTable.refresh();
        }
        // Allow the table to grow vertically in its VBox
        VBox.setVgrow(cardTable, Priority.ALWAYS);
        RectangleButtonBundle addFC = ui.buildRectangularButtonBundle(addW, rectH, "Add",
"buttonText", zero, zero);

        inputBox.getChildren().addAll(
                questionArea.textArea(), questionArea.characterCounter(),
                answerArea.textArea(),  answerArea.characterCounter(),
                addFC.button()
        );
        // Flag to determine if the user wants to save to a diff file
        final boolean[] diffFile = {false};
        RectangleButtonBundle save = ui.buildRectangularButtonBundle(saveW, rectH, "Save",
"buttonText", threeX, saveY);
        save.button().setOnMouseClicked(_-> saving(cardTable, alreadyExists, diffFile,
contentLayer));

        // Deck constraints label to indicate to user
        int size = cardTable.getItems().size();
        Label deckConstraint = new Label(size + " / 60");
        // Set initial colour identifier
```

```
        if(size == zero || size > deckSizeLimit){
            deckConstraint.setStyle("-fx-text-fill: red;");
        }else {
            deckConstraint.setStyle("-fx-text-fill: green;");
        }
        deckConstraint.getStyleClass().add("counter");
        // Listener so it can change in real time
        cardTable.getItems().addListener((ListChangeListener<Flashcard>) _ ->{
            int varyingSize = cardTable.getItems().size();
            deckConstraint.setText(varyingSize + " / 60");
            if(varyingSize == zero || varyingSize > deckSizeLimit){
                deckConstraint.setStyle("-fx-text-fill: red;");
            }else{
                deckConstraint.setStyle("-fx-text-fill: green;");
            }
        });
        // Another VBox containing the cardTable and save button
        VBox fcContent = new VBox(vSpacing, cardTable, deckConstraint, save.button());

        // HBox with spacing of 20 to display the two Vboxes
        HBox display = new HBox(hSpacing, inputBox, fcContent);
        display.setLayoutX(hX);
        display.setLayoutY(hY);
        display.getStyleClass().add("display");

        addFC.button().setOnMouseClicked(_->addFlashcard(questionArea.textArea(),
answerArea.textArea()));
        nav.switchContent(contentLayer, display, getBackView(contentLayer));
    }
    // Method used to save created flashcards, outputting appropriate messages
    private void saving(TableView<Flashcard> cardTable, boolean alreadyExists, boolean[]
diffFile, Group contentLayer){
        // Checks if flashcard deck is a valid size
        if(!cardList.isEmpty() && cardList.size() <= deckSizeLimit) {
            // Does this already exist -> write into same file as before
            // Does this not exist already or user has chosen diffFile -> write into a new file
            if (!alreadyExists || diffFile[0]) {
                // Asks user to input the file name
                String deckNameReq = "Enter an appropriate deck name, suitable to be a file name:
";
                Optional<String> deckName = CustomAlert.input(scene.getWindow(), "Save
Flashcards", deckNameReq, "Name the deck");
                if (deckName.isPresent()) {
                    // Set file to name to flashcard deck name
                    fc.setFlashcardDeckName(deckName.get());
                    try {
                        // Attempt to save
                        boolean saved = fc.saveFlashcardDeck(cardList, false);
                        if (saved) {
                            // Output Success message
                            CustomAlert.success(scene.getWindow(),"Flashcard deck has been
saved");
                            if(alreadyExists) {
                                allDecks.clear();
                                allDecks.addAll(fc.getFileContents());
                                // Update deck table
                                decksTable.getItems().addAll(fc.getFileContents());
                                nav.goBack(contentLayer);
                            }
                            // Clear table and list
                            cardList.clear();
                            cardTable.refresh();
                        }
                    } catch (IOException e) {
                        // If not possible, display a suitable error message
                        CustomAlert.error(scene.getWindow(), "Could not save changes, try again
later");
                        throw new RuntimeException(e);
                    }
                }
            }else{
                // Tell user they are writing to same file
                if(CustomAlert.confirm(scene.getWindow(), "Confirmation", "You are writing to the
previous file, if this not ideal, cancel and try again")){
                    try {
                        boolean saved = fc.saveFlashcardDeck(cardList, true);
                        if(saved) {
                            cardList.clear();
                            cardTable.refresh();
                            CustomAlert.success(scene.getWindow(), "Changes have been saved, you
will be returned to modification");
                            // Update deck table
```

```
                            allDecks.clear();
                            allDecks.addAll(fc.getFileContents());
                            decksTable.getItems().addAll(fc.getFileContents());
                            nav.goBack(contentLayer);
                        }
                    } catch (IOException e) {
                        CustomAlert.error(scene.getWindow(), "Could not save changes, try again
later");
                        throw new RuntimeException(e);
                    }
                }else{
                    // Ask for confirmation they want to write into a new file
                    if(CustomAlert.confirm(scene.getWindow(), "Confirmation", "Would you like to
write into a different file?")){
                        diffFile[0] = true;
                    }
                }
            }
        }else{
            // Remind user of constraints
            CustomAlert.error(scene.getWindow(), "Minimum of 0 and maximum of 50 cards in a
deck");
        }
    }

    // Function which builds the table to display Flashcards
    public TableView<Flashcard> buildFlashcardTableWActions() {
        // Create a new tableview bound to observable list
        TableView<Flashcard> table = new TableView<>(cardList);
        // Create columns and link to the specific property of Flashcard.java
        // to populate the cell
        TableColumn<Flashcard, String> frontColumn = new TableColumn<>("Front");
        frontColumn.setCellValueFactory(new PropertyValueFactory<>("question"));
        TableColumn<Flashcard, String> backColumn = new TableColumn<>("Back");
        backColumn.setCellValueFactory(new PropertyValueFactory<>("answer"));
        TableColumn<Flashcard, Void> actionsColumn = new TableColumn<>("Actions");
        actionsColumn.setMaxWidth(cMaxW);
        actionsColumn.setCellFactory(_ -> new TableCell<>() {
            final RectangleButtonBundle delete = ui.buildRectangularButtonBundle(btnW, btnH,
"Delete", "tableText", zero, zero);
            final RectangleButtonBundle edit = ui.buildRectangularButtonBundle(btnW, btnH,
"Edit", "tableText", zero, zero);
            final VBox actions = new VBox(5, edit.button(), delete.button());
            {
                // Delete - removed the card at this row from the list
                delete.button().setOnMouseClicked(_ -> {
                    Flashcard card = getTableView().getItems().get(getIndex());
                    if (CustomAlert.confirm(scene.getWindow(), "Deletion?", "Are you sure you
want to delete?")) {
                        cardList.remove(card);
                    }
                });
                // Edit - changes current text field and removes from list so it can be resaved
                edit.button().setOnMouseClicked(_ -> {
                    Flashcard card = getTableView().getItems().get(getIndex());
                    if (CustomAlert.confirm(scene.getWindow(), "Deletion?", "Are you sure you
want to edit?")) {
                        questionArea.textArea().setText(card.getQuestion());
                        answerArea.textArea().setText(card.getAnswer());
                        cardList.remove(card);
                    }
                });
            }
            @Override
            protected void updateItem(Void item, boolean empty) {
                super.updateItem(item, empty);
                // Only show options on the rows which have data
                setGraphic(empty ? null : actions);
            }
        });
        // Add all columns to the table
        table.getColumns().addAll(List.of(frontColumn, backColumn, actionsColumn));
        // Columns are resized to fill available width, last column will shrink or expand
        table.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
        // Set the text when there are no cards
        Label placeholderText = new Label("No cards yet");
        placeholderText.setStyle("-fx-text-fill: red;");
        table.setPlaceholder(placeholderText);
        return table;
    }

    // Procedure to add flashcards to the list
```

```
    private void addFlashcard(TextArea q, TextArea a){
        // Question and Answer constraints
        String qReq = "Questions can be 10-300 characters";
        String aReq = "Answers can be 1-600 characters";
        // Remove white space and get the text
        String question = q.getText().trim();
        String answer = a.getText().trim();
        // Check if valid
        boolean validQ = question.length() >= lowerQLimit && question.length() <= upperQLimit;
        boolean validA = !answer.isEmpty() && answer.length() <= upperALimit;
        // Add to list
        if(validQ && validA){
            cardList.add(new Flashcard(question, answer));
            // Clear the fields for next input
            q.clear();
            a.clear();
        }else{
            // Display error
            CustomAlert.error(scene.getWindow(), qReq + "\n" + aReq);
        }
    }

    // Procedure to create the modification page
    private void buildModifyContent(Group contentLayer) throws IOException {
        // Table to display the deck names
        TableView<String> decksTable = new TableView<>();
        decksTable.getStyleClass().add("cardTable");
        decksTable.setMinHeight(minHeight);
        TableColumn<String, String> deckNameColumn = new TableColumn<>("Deck Names");
        deckNameColumn.setCellValueFactory(data -> new SimpleStringProperty(data.getValue()));
        TableColumn<String, Void> actionsColumn = new TableColumn<>("Actions");
        actionsColumn.setCellFactory(_ -> new TableCell<>(){
            // Buttons in the actions column
            final RectangleButtonBundle rename = ui.buildRectangularButtonBundle(btnW, btnH,
"Rename", "tableText", zero, zero);
            final RectangleButtonBundle edit = ui.buildRectangularButtonBundle(btnW, btnH,
"Edit", "tableText", zero, zero);
            final RectangleButtonBundle delete = ui.buildRectangularButtonBundle(btnW, btnH,
"Delete", "tableText", zero, zero);
            final VBox buttons = new VBox(spacing, rename.button(), edit.button(),
delete.button());
            {
                // Renaming the file logic
                rename.button().setOnMouseClicked(_->{
                    String deckName = getTableView().getItems().get(getIndex());
                    if(CustomAlert.confirm(scene.getWindow(),"Confirmation", "Are you sure you
want to rename " + deckName + "?")){
                        Optional<String> newName = CustomAlert.input(scene.getWindow(), "Rename
the deck",
                                "Enter a suitable new deck name", "Enter new deck name");
                        if (newName.isPresent()) {
                            fc.renameDeck(deckName, newName.get());
                            decksTable.getItems().set(getIndex(), newName.get());
                            CustomAlert.success(scene.getWindow(), "Successfully renamed deck " +
deckName + " to " + newName.get());
                        }
                    }
                });
                // Deleting the flashcard file logic
                delete.button().setOnMouseClicked(_->{
                    String deckName = getTableView().getItems().get(getIndex());
                    if(CustomAlert.confirm(scene.getWindow(), "Confirmation", "Are you sure you
want to delete " + deckName + "?")){
                        try {
                            fc.deleteDeck(deckName);
                            String selectedRow =
getTableView().getSelectionModel().getSelectedItem();
                            getTableView().getItems().remove(selectedRow);
//                            cardsPreview.refresh();
                            CustomAlert.success(scene.getWindow(), "Successfully deleted deck " +
deckName);
                        } catch (IOException e) {
                            CustomAlert.error(scene.getWindow(), "Unsuccessful, try again
later");
                            throw new RuntimeException(e);
                        }
                    }
                });
                // Editing the flashcard deck
                edit.button().setOnMouseClicked(_->{
                    String deckName = getTableView().getItems().get(getIndex());
                    if(CustomAlert.confirm(scene.getWindow(), "Confirmation", "Are you sure you
```

```
want to add to " + deckName + "?")){
                        fc.setFlashcardDeckName(deckName);
                        try {
                            cardList.setAll(fc.loadFlashcardDeck(deckName));
                        } catch (IOException e) {throw new RuntimeException(e);}
                        // Takes you to creation page
                        buildCreationContent(contentLayer, true);
                    }
                });
            }
            @Override
            protected void updateItem(Void item, boolean empty){
                super.updateItem(item, empty);
                // Only show buttons if there is data
                setGraphic(empty ? null : buttons);
            }
        });
        decksTable.getColumns().addAll(List.of(deckNameColumn, actionsColumn));
        decksTable.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
        decksTable.getItems().addAll(fc.getFileContents());
        allDecks = new ArrayList<>(decksTable.getItems());
        HBox display = getPreviewDecksAndCards(fc, decksTable, scene.getWindow());
        nav.switchContent(contentLayer, display, getBackView(contentLayer));
    }
    // Procedure to create the organisation page
    private void buildOrganisationContent(Group contentLayer) throws IOException {
        TreeView<String> tree = buildTree();
        addDragAndDrop(tree);

        RectangleButtonBundle newFolder = ui.buildRectangularButtonBundle(folderBtnW, folderBtnH,
"Add Folder", "smallerButtonText", zero, zero);
        newFolder.button().setOnMouseClicked(_->addFolder(tree));
        RectangleButtonBundle delete = ui.buildRectangularButtonBundle(folderBtnW, folderBtnH,
"Delete Folder", "smallerButtonText", zero, zero);
        delete.button().setOnMouseClicked(_-> {
            try {
                deleteFolder(tree);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        HBox options = new HBox(hSpacing, newFolder.button(), delete.button());
        VBox treeViewVBox = new VBox(hSpacing, options, tree);

        TableView<Flashcard> cardsPreview = buildCardPreviewTable(tableHeight);
        cardsPreview.getItems().clear();

        // Listener to change the card preview based on selection
        tree.getSelectionModel().selectedItemProperty().addListener(((_, _, newValue) -> {
            if(newValue == null) return;
            try{
                String item = newValue.getValue();
                if(item.startsWith("\uD83D\uDCC1 ")) {
                    cardsPreview.getItems().clear();
                    Label phText = new Label("This is a folder");
                    phText.setStyle("-fx-text-fill: red;");
                    cardsPreview.setPlaceholder(phText);
                }
                if(item.startsWith("\uD83D\uDCC4 ")){
                    String fileNode = item.replace("\uD83D\uDCC4 ", "");
                    cardsPreview.getItems().setAll(fc.loadFlashcardDeck(fileNode));
                }

            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }));


        HBox display = new HBox(hSpacing, treeViewVBox, cardsPreview);
        display.setLayoutX(displayX);
        display.setLayoutY(displayY);
        display.getStyleClass().add("display");
        nav.switchContent(contentLayer, display, getBackView(contentLayer));
    }
    // Function to build the tree which displays the folder for organisation
    private TreeView<String> buildTree(){
        // Create the initial TreeView
        TreeView<String> folderTreeView = new TreeView<>();
        folderTreeView.getStyleClass().add("folderTreeView");
        // Enables possibility of edit
        folderTreeView.setEditable(true);
```

```
        folderTreeView.setShowRoot(true);

        // Build existing tree copying directory structure
        TreeItem<String> folderStructure = fc.getUserDirectoryStructure(ac.getEmail());
        folderTreeView.setRoot(folderStructure);
        folderTreeView.getRoot().setExpanded(true);
        return folderTreeView;
    }
    // Procedure which will allow the dragging and dropping of files within the tree
    private void addDragAndDrop(TreeView<String> tree){
        // Creating custom cells with drag and drop feature
        tree.setCellFactory(_-> {
            // Create a new cell with item's string value
            TreeCell<String> cell = new TreeCell<>() {
                @Override
                protected void updateItem(String item, boolean empty) {
                    super.updateItem(item, empty);
                    // Show text if cell has content, otherwise leave it blank
                    setText(empty || item == null ? null : item);
                }
            };
            // User begins dragging
            cell.setOnDragDetected(e->{
                // Don't allow the moving of empty cells
                if(cell.isEmpty()) return;
                // Don't allow root to be dragged
                TreeItem<String> item = cell.getTreeItem();
                if(item == null || item.getParent() == null ) return;
                // Start a move drag operation on the cell
                Dragboard db = cell.startDragAndDrop(TransferMode.MOVE);
                // Put cell's text into a clipboard to know what's being dragged
                ClipboardContent cc = new ClipboardContent();
                cc.putString(cell.getItem());
                db.setContent(cc);
                e.consume();
            });
            // User drags over cell
            cell.setOnDragOver(e ->{
                // Only allow the drag if it has a string and is a folder and not over itself
                if(e.getDragboard().hasString() && e.getGestureSource() != cell){
                    e.acceptTransferModes(TransferMode.MOVE);
                }
                e.consume();
            });
            // User has entered the drag area of a cell
            cell.setOnDragEntered(e->{
                // Change the styling of the cell to show it's a valid move
                if(e.getDragboard().hasString() && e.getGestureSource() != cell &&
isFolder(cell.getTreeItem())){
                    cell.setStyle("-fx-background-color: #6891de; -fx-background-radius: 6px;");
                }
            });
            // When the dragged item has left the cells area, remove the styling
            cell.setOnDragExited(_-> cell.setStyle(""));
            // User releases dragged item on cell
            cell.setOnDragDropped(e->{
                Dragboard db = e.getDragboard();
                // If there's no string, return;
                if(!db.hasString()) return;
                // Get item being dragged and item being dropped into
                TreeItem<String> draggedF = tree.getSelectionModel().getSelectedItem();
                TreeItem<String> droppingF = cell.getTreeItem();
                // Conditions to stop the file movement
                if(draggedF == null || droppingF == null) return;
                // If drop over same cell
                if(draggedF == droppingF) return;
                // If destination is already an ancestor of the dragged item
                if(isAncestor(draggedF, droppingF)) return;
                // if it is the root
                if(draggedF.getParent() == null) return;
                if(draggedF == tree.getRoot()) return;
                // If folder going into a file
                if(isFolder(draggedF) && isFile(droppingF)) return;
                // if item already in folder
                if (droppingF == draggedF.getParent()) return;
                String draggedName = draggedF.getValue().replace("\uD83D\uDCC1 ",
"").replace("\uD83D\uDCC4 ", "");
                String droppingName = droppingF.getValue().replace("\uD83D\uDCC1 ",
"").replace("\uD83D\uDCC4 ", "");
                try {
                    fc.move(draggedName, isFolder(draggedF), droppingName);
                } catch (IOException ex) {
```

```
                    throw new RuntimeException(ex);
                }
                // Remove from previous folder
                draggedF.getParent().getChildren().remove(draggedF);
                // Add to selected folder
                if(droppingF.getValue().startsWith("\uD83D\uDCC1 ")) {
                    droppingF.getChildren().add(draggedF);
                    // User can see change
                    droppingF.setExpanded(true);
                }else{
                    // Get parent of deck being dropped into
                    TreeItem<String> parent = droppingF.getParent();
                    // Find position of target deck
                    int idx = parent.getChildren().indexOf(droppingF);
                    // Insert one after target deck
                    parent.getChildren().add(idx+1, draggedF);
                }
                // Drop was successfully handled
                e.setDropCompleted(true);
                e.consume();
            });
            return cell;
        });
    }
    // Checks if tree item's ancestor is somewhere is above the item in the tree
    private boolean isAncestor(TreeItem<String> potentialAncestor, TreeItem<String> item){
        // Start at item's parent
        TreeItem<String> current = item.getParent();
        while (current != null){
            // if match found, block the drop
            if(current == potentialAncestor) return true;
            // move a level up if not
            current = current.getParent();
        }
        return false;
    }
    // Check if folder
    private boolean isFolder(TreeItem<String> potentialFolder){
        return potentialFolder != null
                && potentialFolder.getValue() != null
                && potentialFolder.getValue().startsWith("\uD83D\uDCC1");
    }
    // Check if file
    private boolean isFile(TreeItem<String> potentialFile){
        return potentialFile != null
                && potentialFile.getValue() != null
                && potentialFile.getValue().startsWith("\uD83D\uDCC4");
    }
    // Subroutine to add a new folder within the tree
    private void addFolder(TreeView<String> tree){
        TreeItem<String> selected = tree.getSelectionModel().getSelectedItem();
        // If nothing is selected, use the root, and if a deck is selected use its parent
        TreeItem<String> folder;
        if(selected == null){
            folder = tree.getRoot();
        } else if (isFile(selected)) {
            folder = selected.getParent();
        }else {
            folder = selected;
        }
        // Ask for folder name input
        Optional<String> folderName = CustomAlert.input(scene.getWindow(), "New Folder",
                "What do you want to name the folder?", "Enter Folder Name");
        String parentPath;
        if(folder == tree.getRoot()){
            // If target is root, use the user's folder path
            parentPath = fc.getUserFolderPath();
        }else{
            // Search for file path on disk
            String dirName = folder.getValue().replace("\uD83D\uDCC1 ", "");
            parentPath = fc.getFilePath(dirName, true);
        }
        int outcome;
        if(folderName.isPresent()){
            outcome = fc.createFolder(parentPath, folderName.get());
            switch(outcome){
                case 2 -> folder.getChildren().add(folder(folderName.get()));
                case 1 -> CustomAlert.error(scene.getWindow(), "Folder already exists");
                default -> CustomAlert.error(scene.getWindow(), "Did not work try again later");
            }
        }
    }
```

```
    // Subroutine to delete a folder
    private void deleteFolder(TreeView<String> tree) throws IOException {
        TreeItem<String> selected = tree.getSelectionModel().getSelectedItem();
        if(selected == null){
            CustomAlert.error(scene.getWindow(), "Pick something to delete");
            return;
        } else if (selected == tree.getRoot()) {
            CustomAlert.error(scene.getWindow(), "Cannot delete this folder");
            return;
        }else if (isFile(selected)) {
            CustomAlert.error(scene.getWindow(), "To delete files, go to the modification page");
            return;
        }
        boolean confirmed = CustomAlert.confirm(scene.getWindow(), "Confirmation of deletion",
                "Are you sure you would like to delete this folder? \n It will delete all
contents");
        if(confirmed){
            boolean deleted = fc.deleteFolder(selected.getValue().replace("\uD83D\uDCC1 ", ""));
            CustomAlert.success(scene.getWindow(), "Folder successfully deleted");
            selected.getParent().getChildren().remove(selected);
        }
    }
    // Function to return folder node for the tree, starting expanded
    private TreeItem<String> folder(String folderName){
        TreeItem<String> folder = new TreeItem<>("\uD83D\uDCC1 " + folderName);
        folder.setExpanded(true);
        return folder;
    }
    // Function to return a deck node
    private TreeItem<String> file(String fileName){
        return new TreeItem<>("\uD83D\uDCC4 " + fileName);
    }
}
```

5.2.8 StudyModesScreen.java

```
package Screens;

import Components.CustomAlert;
import Components.ScreenManager;
import Controllers.*;
import Models.Flashcard;
import Models.UserStats;
import RecordClasses.RectangleButtonBundle;
import javafx.animation.*;
import javafx.beans.property.SimpleStringProperty;
import javafx.scene.Cursor;
import javafx.scene.Group;
import javafx.scene.PerspectiveCamera;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Pane;
import javafx.scene.layout.StackPane;
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import javafx.scene.text.TextAlignment;
import javafx.scene.transform.Rotate;
import javafx.stage.Window;
import javafx.util.Duration;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.Objects;

public class StudyModesScreen extends SystemScreens{
    // Constants
    private static final int titleX = 420;
    // Deck Selector
    private static final int minHeight = 500;
    private static final int startW = 500;
    private static final int startH = 100;
    // Session Content
```

```
    private static final int timerX = 700;
    private static final int timerY = 940;
    private static final int progressX = 1400;
    private static final int progressY = 225;
    private static final int flashcardX = 360;
    private static final int flashcardY = 290;
    private static final int ttsIconW = 100;
    private static final int ttsIconH = 100;
    private static final int ttsIconX = 200;
    private static final int ttsIconY = 290;
    private static final int tickX = 1035;
    private static final int tickY = 760;
    private static final int crossX = 685;
    private static final int crossY = 760;
    private static final int correctX = 1520;
    private static final int correctY = 830;
    private static final int incorrectX = 200;
    private static final int incorrectY = 830;
    private static final int modeX = 125;
    private static final int modeY = 230;
    private static final int modeWrapWidth = 1350;
    private static final int rateTextX = 110;
    private static final int rateTextY = 470;
    private static final int rateBtnW = 40;
    private static final int rateBtnH = 40;
    private static final int minusX = 150;
    private static final int minusY = 480;
    private static final int plusX = 250;
    private static final int plusY = 480;
    // Flashcard
    private static final int fcW = 1200;
    private static final int fcH = 450;
    private static final int cardLabelTranslate = -170;
    private static final int wrappingWidth = 1100;
    private static final int hintTranslate = 185;
    // Flashcard buttons
    private static final int btnW = 200;
    private static final int btnH = 130;
    // Animation
    private static final int duration = 150;
    private static final int endValue = 90;
    // Tick
    private static final int tick = 1;
    // Controllers
    private final AuthenticationController ac;
    private final SettingsController sc;
    private final FlashcardController fc;
    private final SessionController sessionC;
    private TTSController tts;
    private final StatsController stats;
    // Content filling revision system to be modified globally
    private Text correctText;
    private Text incorrectText;
    private Label timerLabel;
    private Text progressText;
    private Text cardLabel;
    private Text cardBody;
    private Rectangle cardRect;
    // Flags
    private boolean hasFlipped = false;
    private boolean hasSessionStarted = false;
    private boolean isRetrySession = false;
    // Timer
    private Timeline timer;

    // Constructor to start a NEW Study Session
    public StudyModesScreen(AuthenticationController ac, SettingsController sC,
FlashcardController fc, StatsController stats) throws IOException {
        this.ac = ac;
        this.sc = sC;
        this.fc = fc;
        this.sessionC = new SessionController(ac.getEmail(), fc);
        // only load if tts is enabled
        if(sc.getCurrentSettings().isTTSEnabled()) this.tts = new
TTSController(sc.getTTSVolume());
        this.stats = stats;
    }
    // Constructor to continue a prior Study Session
    public StudyModesScreen(AuthenticationController ac, SettingsController sC,
FlashcardController fc, SessionController sessionC, StatsController stats) throws IOException {
        this.ac = ac;
        this.sc = sC;
```

```
        this.fc = fc;
        this.sessionC = sessionC;
        // Only load if tts is enabled
        if(sc.getCurrentSettings().isTTSEnabled()) this.tts = new
TTSController(sc.getTTSVolume());
        this.stats = stats;
    }

    // Create initial content where user can select study modes
    public Scene create(){
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        Scene scene = new Scene(root, sceneW, sceneW);
        Group background = createBackground("Study Modes");
        getTitle().setLayoutX(titleX);
        ImageView back = getSwitchScreenBackView(ac);

        Group contentLayer = new Group();

        RectangleButtonBundle serial = ui.buildRectangularButtonBundle(rectW, rectH, "Serial
Mode", "buttonText", threeX, threeY1);
        ImageView serialIcon = createIcon(threeX, threeY1+iconOffset,
"/images/StudyModesScreen/serial.png");
        serial.button().setOnMouseClicked(_-> buildDeckSelector(contentLayer, "Serial"));

        RectangleButtonBundle wrongAnswers = ui.buildRectangularButtonBundle(rectW, rectH, "Wrong
Answers", "buttonText", threeX, threeY2);
        ImageView wrongAnswersIcon = createIcon(threeX, threeY2+iconOffset,
"/images/StudyModesScreen/wrongAnswers.png");
        wrongAnswers.button().setOnMouseClicked(_-> {
            try {
                buildWrongAnswersMode(contentLayer);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });

        RectangleButtonBundle shuffle = ui.buildRectangularButtonBundle(rectW, rectH, "Shuffle
Mode", "buttonText", threeX, threeY3);
        ImageView shuffleIcon = createIcon(threeX, threeY3+iconOffset,
"/images/StudyModesScreen/shuffle.png");
        shuffle.button().setOnMouseClicked(_-> buildDeckSelector(contentLayer, "Shuffle"));

        Group buttons = new Group(serial.button(), wrongAnswers.button(), shuffle.button(),
serialIcon, wrongAnswersIcon, shuffleIcon);
        contentLayer.getChildren().add(buttons);
        root.getChildren().addAll(background, back, contentLayer);
        return scene;
    }

    // Content where user can select a deck
    private void buildDeckSelector(Group contentLayer, String mode){
        Window owner = contentLayer.getScene().getWindow();
        // Create table with decks
        TableView<String> decksTable = new TableView<>();
        decksTable.getStyleClass().add("cardTable");
        decksTable.setMinHeight(minHeight);
        TableColumn<String, String> deckNameColumn = new TableColumn<>("Deck Names");
        deckNameColumn.setCellValueFactory(data -> new SimpleStringProperty(data.getValue()));
        decksTable.getColumns().add(deckNameColumn);
        decksTable.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN);
        decksTable.getItems().addAll(fc.getFileContents());
        Label deckTablesPh = new Label("No decks found");
        deckTablesPh.setStyle("-fx-text-fill: red; -fx-wrap-text: true; -fx-text-alignment:
center;");
        decksTable.setPlaceholder(deckTablesPh);
        // Start session button which creates the session
        RectangleButtonBundle startSession = ui.buildRectangularButtonBundle(startW, startH,
"Start Session", "smallButtonText", 760, 0);
        startSession.button().setOnMouseClicked(_->{
            String deckName = decksTable.getSelectionModel().getSelectedItem();
            if(deckName == null){
                CustomAlert.error(owner, "Select a deck before starting a session");
                return;
            }
            try{
                ArrayList<Flashcard> cards = fc.loadFlashcardDeck(deckName);
                // If shuffle mode, shuffle the cards
                if(mode.equals("Shuffle")) sessionC.shuffle(cards);
                sessionC.startSession(cards, deckName, mode);
                sessionC.setLastCards(new ArrayList<>(cards));
                buildSessionContent(contentLayer);
```

```
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        // Get preview and search icon
        HBox display = getPreviewDecksAndCards(fc, decksTable, owner, startSession.button());
        nav.switchContent(contentLayer, display, getBackView(contentLayer));
    }
    // Create content where flashcard is displayed with buttons
    private void buildSessionContent(Group contentLayer){
        // Timer text
        timerLabel = new Label(sessionC.getFormattedTime());
        timerLabel.getStyleClass().add("timerLabel");
        timerLabel.setLayoutX(timerX); timerLabel.setLayoutY(timerY);

        progressText = new Text(sessionC.getProgressText());
        progressText.getStyleClass().add("infoText");
        progressText.setLayoutX(progressX); progressText.setLayoutY(progressY);
        // Actual flashcard
        StackPane flashcard = buildCard();
        flashcard.setLayoutX(flashcardX); flashcard.setLayoutY(flashcardY);
        // TTS Icon
        ImageView ttsIcon = createIcon(ttsIconW, ttsIconH, "/images/StudyModesScreen/tts.png");
        ttsIcon.setLayoutX(ttsIconX); ttsIcon.setLayoutY(ttsIconY);
        ttsIcon.setCursor(Cursor.HAND);

        ttsIcon.setOnMouseClicked(_->{
            // If showing answer, read out answer
            if(sessionC.isShowingAnswer()) tts.speak(sessionC.getCurrentCard().getAnswer());
            // If showing question, read out question
            else tts.speak(sessionC.getCurrentCard().getQuestion());
        });

        StackPane tick = buildFlashcardButton("✓","lightGreen");
        tick.setOnMouseClicked(_-> {
            try {
                handleAnswer(true);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        tick.setLayoutX(tickX); tick.setLayoutY(tickY);
        StackPane cross = buildFlashcardButton("✗", "red");
        cross.setOnMouseClicked(_-> {
            try {
                handleAnswer(false);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        cross.setLayoutX(crossX); cross.setLayoutY(crossY);
        // Text for correct count
        correctText = new Text("✓      " + sessionC.getCorrectCount());
        correctText.getStyleClass().add("correctC");
        correctText.setLayoutX(correctX); correctText.setLayoutY(correctY);
        // Text for incorrect count
        incorrectText = new Text("✗      " + sessionC.getIncorrectCount());
        incorrectText.getStyleClass().add("incorrectC");
        incorrectText.setLayoutX(incorrectX); incorrectText.setLayoutY(incorrectY);
        // Extra info text
        Text mode = new Text(sessionC.getStudyMode() + "  ·  "
                + sessionC.getSelectedDeck());
        mode.getStyleClass().add("infoText");
        mode.setLayoutX(modeX); mode.setLayoutY(modeY); mode.setWrappingWidth(modeWrapWidth);
        Group newContent = new Group(timerLabel, progressText, flashcard, tick, cross,
correctText, incorrectText, mode);
        // Create rate vary option
        Text rateDisplay = new Text("Rate: 1.0");
        rateDisplay.getStyleClass().add("infoText");
        ui.setPosition(rateDisplay, rateTextX, rateTextY);
        // Create buttons
        RectangleButtonBundle minus = ui.buildRectangularButtonBundle(rateBtnW, rateBtnH, "-",
"smallButtonText", minusX, minusY);
        RectangleButtonBundle plus = ui.buildRectangularButtonBundle(rateBtnW, rateBtnH, "+",
"smallButtonText", plusX, plusY);
        // Array value
        double[] rate = {1.0};
        // Reset per session
        tts.setRate(rate[0]);
        // On plus
        plus.button().setOnMouseClicked(_-> {
```

```
            tts.setRate(rate[0] + 0.1);
            rate[0] = tts.getRate();
            rateDisplay.setText(String.format("Rate: %.1f", rate[0]));
        });
        // On minus
        minus.button().setOnMouseClicked(_-> {
            tts.setRate(rate[0] - 0.1);
            rate[0] = tts.getRate();
            rateDisplay.setText(String.format("Rate: %.1f", rate[0]));
        });
        // If TTS is enabled, add the ttsIcon and rate option
        Group ttsGroup = new Group(ttsIcon, rateDisplay, minus.button(), plus.button());
        if(sc.getCurrentSettings().isTTSEnabled()) newContent.getChildren().add(ttsGroup);
        nav.switchContent(contentLayer, newContent, backButton(contentLayer));
        // Start the timer as soon as this content is built
        startTimer();
    }

    // Procedure to create the session for wrong answers
    private void buildWrongAnswersMode(Group contentLayer) throws IOException {
        // Load the wrongAnswers into an ArrayList
        ArrayList<Flashcard> wrongAnswers = sessionC.loadWrongAnswers();
        // If empty, tell user to start a different session
        if(wrongAnswers.isEmpty()){
            Text information = new Text("No wrong answers yet \n Study a deck prior to using this
mode");
            information.getStyleClass().add("infoText");
            information.setTextAlignment(TextAlignment.CENTER);
            information.setLayoutX(310);
            information.setLayoutY(500);
            nav.switchContent(contentLayer, information, getBackView(contentLayer));
            return;
        }
        // Start session
        sessionC.startSession(wrongAnswers, "wrongAnswers", "Wrong Answers");
        // Build content
        buildSessionContent(contentLayer);
    }

    // Function returning the flashcard display
    private StackPane buildCard() {
        cardRect = new Rectangle(fcW, fcH);
        cardRect.getStyleClass().add("fcRect");

        cardLabel = new Text("Question");
        cardLabel.getStyleClass().add("cardLabel");
        cardLabel.setTranslateY(cardLabelTranslate);

        cardBody = new Text(sessionC.getCurrentCard().getQuestion());
        cardBody.getStyleClass().add("cardBody");
        cardBody.setWrappingWidth(wrappingWidth);

        Text flipHint = new Text("Click card to flip");
        flipHint.getStyleClass().add("flipH");
        flipHint.setTranslateY(hintTranslate);

        StackPane card = new  StackPane(cardRect, cardLabel, cardBody, flipHint);
        card.getStyleClass().add("flashcard");
        // When clicked, it will flip
        card.setOnMouseClicked(_-> flipFlashcard(card));
        return card;
    }
    // Function to get tick and cross buttons
    private StackPane buildFlashcardButton(String symbol, String colour){
        Rectangle buttonRect = new Rectangle(btnW, btnH);
        buttonRect.getStyleClass().add("fcButtonRect");
        buttonRect.setStyle("-fx-fill: " + colour + ";");
        Text label = new Text(symbol);
        label.getStyleClass().add("fcButtonText");
        StackPane flashcardButton = new StackPane(buttonRect, label);
        flashcardButton.getStyleClass().add("fcButton");
        return flashcardButton;
    }
    // Subroutine to handle the flipping of the card
    private void flipFlashcard(StackPane flashcard){
        // Set flag to true, so tick and cross inputs can be handled
        hasFlipped = true;
        // Set the axis of rotation to be y-axis
        flashcard.setRotationAxis(Rotate.Y_AXIS);
        // Add a perspective camera to add 3d effect
        flashcard.getScene().setCamera(new PerspectiveCamera());
        // Create first half of flip, rotating card half way
```

```
        Timeline half1 = new Timeline(new KeyFrame(Duration.millis(duration),
                // 150ms, 90 degrees, start slowly
                new KeyValue(flashcard.rotateProperty(), endValue, Interpolator.EASE_IN)));
        half1.setOnFinished(_->{
            // toggle whether answer side is showing
            sessionC.setShowingAnswer(!sessionC.isShowingAnswer());
            // if answer should now be shown
            if(sessionC.isShowingAnswer()){
                // Change flashcard content to answer
                cardBody.setText(sessionC.getCurrentCard().getAnswer());
                cardLabel.setText("Answer");
                cardRect.getStyleClass().remove("fcRect");
                cardRect.getStyleClass().add("fcBRect");
            }else{
                // Flip back question content
                cardBody.setText(sessionC.getCurrentCard().getQuestion());
                cardLabel.setText("Question");
                cardRect.getStyleClass().remove("fcBRect");
                cardRect.getStyleClass().add("fcRect");
            }
            // rotate card back to -90, so text change is not visible mid-flip
            flashcard.setRotate(-endValue);
            // Create a second half animation -90 to 0
           Timeline half2 = new Timeline(new KeyFrame(Duration.millis(duration),
                   new KeyValue(flashcard.rotateProperty(), zero, Interpolator.EASE_OUT)));
           half2.play();
        });
        half1.play();
    }
    // Subroutine to handle the answer when a button is clicked
    private void handleAnswer(boolean correct) throws IOException {
        // if not flipped, then display error
        if(!hasFlipped){
            // Audial feedback
            playSFX("incorrect");
            CustomAlert.error(correctText.getScene().getWindow(), "Check the answer first");
            return;
        }
        // If correct, play correct.mp3, else incorrect.mp3
        if(correct) playSFX("correct");
        if(!correct) playSFX("incorrect");
        hasSessionStarted = true;
        // Stop TTS when above to switch
        tts.stop();
        boolean sessionComplete = sessionC.recordAnswer(correct);
        // Update counters
        correctText.setText("✓      " + sessionC.getCorrectCount());
        incorrectText.setText("✗      " + sessionC.getIncorrectCount());
        // If no remaining cards, stop timer switch to EoRScreen, play sessionComplete.mp3
        if(sessionComplete){
            stopTimer();
            changeToSummary();
            return;
        }
        // Change flashcard content to next car
        cardBody.setText(sessionC.getCurrentCard().getQuestion());
        cardLabel.setText("Question");
        // Change style class and update text
        cardRect.getStyleClass().remove("fcBRect");
        if(!cardRect.getStyleClass().contains("fcRect")) cardRect.getStyleClass().add("fcRect");
        progressText.setText(sessionC.getProgressText());
        hasFlipped = false;
    }
    // Update timer
    private void startTimer() {
        timer = new Timeline(new KeyFrame(Duration.seconds(tick), _ -> {
            sessionC.tick();
            timerLabel.setText(sessionC.getFormattedTime());
        }));
        timer.setCycleCount(Timeline.INDEFINITE);
        timer.play();
    }
    // Procedure which will get updated UserStats object and switch the scene to EORScreen
    private void changeToSummary() throws IOException {
        int xpGained = new XPController().calculateXP(sessionC.getSession());
        UserStats updated = stats.updateStats(ac.getEmail(), sessionC.getSession(),
                xpGained, isRetrySession);
        ScreenManager.switchScene(new EndOfRevisionScreen(ac, sc, sessionC, fc, stats,
updated).create(),
                ac.getEmail());
        playSFX("sessionComplete");
```

```
    }
    // Stop timer
    private void stopTimer(){
        if(timer != null){ timer.stop(); }
    }

    // if user wants to retry, session is continued
    public Scene continueSession(){
        Scene scene = create();
        isRetrySession = true;
        // Find content layer and start a session
        Group contentLayer = (Group) ((Pane) scene.getRoot()).getChildren().get(2);
        buildSessionContent(contentLayer);
        return scene;
    }

    // Custom back button
    private ImageView backButton(Group contentLayer){
        ImageView back = ui.createBRImageView("back");
        back.setCursor(Cursor.HAND);
        back.setOnMouseClicked(_->{
            // Confirmation
            boolean endSession = CustomAlert.confirm(back.getScene().getWindow(),
                    "Confirmation", "Are you sure you want to end the session?");
            // If session hasn't started, and user wants to end then go back to study modes
            if(!hasSessionStarted && endSession) {
                tts.stop();
                nav.goBack(contentLayer);
            }else{
                // if user wants to end and session has started
                if (endSession) {
                    // stop timer
                    stopTimer();
                    try {
                        changeToSummary();
                        tts.stop();
                    } catch (IOException e) {
                        throw new RuntimeException(e);
                    }
                }
            }
        });
        return back;
    }

    // procedure which will play the sound fx when requested
    private void playSFX(String sound) {
        try{
            // Create a temp file on the system to store the sound
            // JavaFX media requires a URI
            InputStream is = getClass().getResourceAsStream("/sounds/" + sound + ".mp3");
            File tempSoundFile = File.createTempFile("sound", ".mp3");
            // Ensure it is deleted on exit
            tempSoundFile.deleteOnExit();
            // Copy sound data from sound file to file
            Files.copy(Objects.requireNonNull(is), tempSoundFile.toPath(),
StandardCopyOption.REPLACE_EXISTING);
            // Play sound via MediaPlayer
            Media testSound = new Media(tempSoundFile.toURI().toString());
            MediaPlayer player = new MediaPlayer(testSound);
            System.out.println(sc.getSFXVolume());
            player.setVolume(sc.getSFXVolume());
            player.play();
        } catch (IOException e){
            System.out.println("play fx error: " + e.getMessage());
        }
    }
}
```

5.2.9 EndOfRevisionScreen.java

```
package Screens;

import Components.ScreenManager;
import Controllers.*;
import RecordClasses.SessionRecord;
import Models.UserStats;
import RecordClasses.RectangleButtonBundle;
import javafx.scene.Cursor;
import javafx.scene.Group;
import javafx.scene.Scene;
```

```
import javafx.scene.chart.BarChart;
import javafx.scene.chart.CategoryAxis;
import javafx.scene.chart.NumberAxis;
import javafx.scene.chart.XYChart;
import javafx.scene.image.ImageView;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Pane;
import javafx.scene.layout.StackPane;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;


public class EndOfRevisionScreen extends SystemScreens{
    // Constants
    private static final int titleX = 300;
    // Initial Content
    // Time
    private static final int timeRectW = 738;
    private static final int timeRectH = 143;
    private static final int timeIconX = 184;
    private static final int timeIconY = 261;
    private static final int timePaneY = 266;
    private static final int timeTextX = 1107;
    private static final int timeTextY = 360;
    private static final int timeHolderX = 1063;
    private static final int timeHolderY = 397;
    private static final int timeHolderW = 583;
    // XP
    private static final int xpRectW = 738;
    private static final int xpRectH = 127;
    private static final int xpPaneX = 184;
    private static final int xpPaneY = 443;
    private static final int xpIconY = 438;
    private static final int xpTextX = 1204;
    private static final int xpTextY = 538;
    private static final int xpHolderX = 1063;
    private static final int xpHolderY = 565;
    private static final int xpHolderW = 583;
    private static final int xpHolderH = 8;
    // Buttons
    private static final int buttonW = 1519;
    private static final int buttonH = 127;
    private static final int graphsBtnX = 184;
    private static final int graphsBtnY = 599;
    private static final int backBtnX = 184;
    private static final int backBtnY = 764;
    private static final int retryY = 957;
    // Graphs
    private static final int graphBtnW = 480;
    private static final int graphBtnH = 80;
    private static final int buttonsX = 230;
    private static final int buttonsY = 200;
    private static final int graphX = 200;
    private static final int graphY = 300;
    private static final int graphW = 1500;
    private static final int graphH = 600;
    // Answer Timeline
    private static final int totalWidth = 1500;
    private static final int blockSize = 60;
    private static final int startingX = 20;
    private static final int spaceBetween = 6;
    // Controller
    private final AuthenticationController ac;
    private final SettingsController sc;
    private final SessionController session;
    private final FlashcardController fc;
    private final StatsController stats;
    private final UserStats userStats;

    private static final DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");

    // Constructor
    public EndOfRevisionScreen(AuthenticationController ac, SettingsController sc,
SessionController session,
                               FlashcardController fc, StatsController stats,  UserStats
userStats){
        this.ac = ac;
```

```
        this.sc = sc;
        this.session = session;
        this.fc = fc;
        this.stats = stats;
        this.userStats = userStats;
    }

    // Create to be displayed
    public Scene create(){
        Pane root = new Pane();
        root.getStyleClass().add("screens");
        Scene scene = new Scene(root, sceneW, sceneW);
        Group background = createBackground("End of Revision");
        getTitle().setLayoutX(titleX);

        Group contentLayer = new Group();
        // timer text
        Rectangle timeRect = new Rectangle(timeRectW, timeRectH);
        Text timeInfoText = new Text("Time:");
        timeInfoText.getStyleClass().add("buttonText");
        StackPane timePane = new StackPane(timeRect, timeInfoText);
        ImageView timeIcon = createIcon(timeIconX, timeIconY,
"/images/EndOfRevisionScreen/timer.png");
        ui.setPosition(timePane, timeIconX, timePaneY);
        Text timeText = new Text(session.getFormattedTime());
        timeText.getStyleClass().add("summaryText");
        ui.setPosition(timeText, timeTextX, timeTextY);
        Rectangle timeHolder = new Rectangle(timeHolderX, timeHolderY, timeHolderW, xpHolderH);

        // Xp text
        Rectangle xpRect = new Rectangle(xpRectW, xpRectH);
        Text xpInfoText = new Text("XP:");
        xpInfoText.getStyleClass().add("buttonText");
        StackPane xpPane = new StackPane(xpRect, xpInfoText);
        ImageView xpIcon = createIcon(xpPaneX, xpIconY, "/images/EndOfRevisionScreen/xp.png");
        ui.setPosition(xpPane, xpPaneX, xpPaneY);
        int xpGained =  new XPController().calculateXP(session.getSession());
        Text xpText = new Text(String.valueOf(xpGained));
        xpText.getStyleClass().add("summaryText");
        ui.setPosition(xpText, xpTextX, xpTextY);
        Rectangle xpHolder = new Rectangle(xpHolderX, xpHolderY, xpHolderW, xpHolderH);

        // button to take you to graphs
        RectangleButtonBundle graphs = ui.buildRectangularButtonBundle(buttonW, buttonH,
"Graphs", "buttonText", graphsBtnX, graphsBtnY);
        ImageView graphIcon = createIcon(graphsBtnX, graphsBtnY+iconOffset,
"/images/EndOfRevisionScreen/graphs.png");
        graphs.button().setOnMouseClicked(_-> buildGraphsContent(contentLayer));
        // Button to go back to main menu
        RectangleButtonBundle backToMainMenu = ui.buildRectangularButtonBundle(buttonW, buttonH,
"Back to Main Menu", "buttonText", backBtnX, backBtnY);
        ImageView backIcon = createIcon(backBtnX, backBtnY+iconOffset,
"/images/Constant/back.png");
        backToMainMenu.button().setOnMouseClicked(_ -> {
            try {
                // Save session record when user is done with the session
                stats.saveSessionRecord(ac.getEmail(), new SessionRecord(
                        LocalDate.now().format(dtf), session.getAccuracy(),
                        session.getElapsedTime(), session.getCorrectCount(),
                        session.getIncorrectCount(), session.getSelectedDeck(),
                        session.getSession().getAnswerHistory()
                ));
                ScreenManager.switchScene(new MainMenuScreen(ac, sc).create(), ac.getEmail());
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
            ScreenManager.clearHistory();
        });
        // Retry button to continue the session
        ImageView retry = new ImageView("/images/EndOfRevisionScreen/retry.png");
        retry.getStyleClass().add("constantImages");
        retry.setLayoutY(retryY);
        retry.setCursor(Cursor.HAND);
        retry.setOnMouseClicked(_ -> {
            session.retrySession();
            // If retrying, the xp gained is subtracted and saved to csv
            // It will be resaved on the end of the session completely
            userStats.setTotalXP(userStats.getTotalXP() - xpGained);
            try {
                stats.saveUserStats(userStats);
            } catch (IOException e) {
                throw new RuntimeException(e);
```

```
            }
            StudyModesScreen retryModes = null;
            try {
                retryModes = new StudyModesScreen(ac, sc, fc, session, stats);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
            ScreenManager.switchScene(retryModes.continueSession());
        });

        Group icons = new Group(timeIcon, xpIcon, graphIcon, backIcon, retry);
        contentLayer.getChildren().addAll(timePane, timeHolder, timeText, xpPane, xpHolder,
xpText, graphs.button(), backToMainMenu.button(), icons);
        root.getChildren().addAll(background, contentLayer);
        return scene;
    }

    // Build content where use can switch the graphs
    private void buildGraphsContent(Group contentLayer) {
        // Create Graph Option Buttons
        RectangleButtonBundle correctIncorrect = ui.buildRectangularButtonBundle(graphBtnW,
graphBtnH, "Correct vs Incorrect", "smallerButtonText", zero, zero);
        RectangleButtonBundle accuracyComparison = ui.buildRectangularButtonBundle(graphBtnW,
graphBtnH, "Accuracy Comparison", "smallerButtonText", zero, zero);
        RectangleButtonBundle answerTimeline = ui.buildRectangularButtonBundle(graphBtnW,
graphBtnH, "Answer Timeline", "smallerButtonText", zero, zero);
        // Place in a HBox
        HBox buttons = new HBox(vSpacing, correctIncorrect.button(), accuracyComparison.button(),
answerTimeline.button());
        ui.setPosition(buttons, buttonsX, buttonsY);
        // Group where the chart is placed
        Group chartGroup = new Group();
        ui.setPosition(chartGroup, graphX, graphY);
        chartGroup.getChildren().addAll(buildCorrectIncorrectChart());
        // On click of buttons
        correctIncorrect.button().setOnMouseClicked(_ ->
                chartGroup.getChildren().setAll(buildCorrectIncorrectChart()));
        accuracyComparison.button().setOnMouseClicked(_ -> {
            try {
                chartGroup.getChildren().setAll(buildAccuracyComparisonChart());
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        });
        answerTimeline.button().setOnMouseClicked(_ ->
                chartGroup.getChildren().setAll(buildAnswerTimeline()));
        nav.switchContent(contentLayer, buttons, chartGroup, getBackView(contentLayer));
    }

    // Default chart -> correct vs incorrect bar chart
    private BarChart<String, Number> buildCorrectIncorrectChart(){
        // Create the axis and label them
        CategoryAxis xAxis = new CategoryAxis();
        int max = Math.max(session.getCorrectCount(), session.getIncorrectCount());
        NumberAxis yAxis = new NumberAxis(0, max + 1, 1);
        yAxis.setTickUnit(1.0);
        yAxis.setTickLabelGap(1.0);
        xAxis.setLabel("Result");
        yAxis.setLabel("Count");

        // Create the actual chart
        BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
        chart.getStyleClass().add("charts");
        chart.setTitle("Correct Vs Incorrect");
        chart.setPrefSize(graphW, graphH);
        chart.setLegendVisible(false);

        // Create data series to store the results
        XYChart.Series<String, Number> series = new XYChart.Series<>();
        XYChart.Data<String, Number> correct = new XYChart.Data<>("Correct",
session.getCorrectCount());
        XYChart.Data<String, Number> incorrect = new XYChart.Data<>("Incorrect",
session.getIncorrectCount());
        series.getData().addAll(List.of(correct, incorrect));
        // Check if in scene then assign colours, otherwise node is null
        chart.sceneProperty().addListener((_, _, newValue) -> {
            if (newValue != null) {
                correct.getNode().setStyle("-fx-background-color: green;");
                incorrect.getNode().setStyle("-fx-background-color: red;");
            }
        });
        // Add series to chart
```

```
        chart.getData().add(series);
        return chart;
    }
    // Chart 2 -> answer comparison for last three session
    private BarChart<String, Number> buildAccuracyComparisonChart() throws IOException {
        // Create the axis
        CategoryAxis xAxis = new CategoryAxis();
        NumberAxis yAxis = new NumberAxis(0, 100, 10);
        xAxis.setLabel("Session");
        yAxis.setLabel("Accuracy / %");
        // Create the chart
        BarChart<String, Number> chart = new BarChart<>(xAxis, yAxis);
        chart.getStyleClass().add("charts");
        chart.setTitle("Accuracy Comparison with last 3 (at most) Sessions with this deck");
        chart.setPrefSize(graphW, graphH);
        chart.setLegendVisible(false);
        // Create the series and populate it with data
        XYChart.Series<String, Number> series = new XYChart.Series<>();
        ArrayList<SessionRecord> sessionHistory = stats.loadSessionRecords(ac.getEmail());
        // Create a list for only this deck
        ArrayList<SessionRecord> deckSessionHistory = new ArrayList<>();
        for(SessionRecord s : sessionHistory){
            // if deck name matches, add to list
            if(s.deckName().equals(session.getSelectedDeck())){
                deckSessionHistory.add(s);
            }
        }
        // Get last three sessions
        int size = deckSessionHistory.size();
        int start = Math.max(0, size - 3);
        for(int i = start; i < size; i++) {
            // Add each accuracy to the chart
            series.getData().add(new XYChart.Data<>("Session - " + (size - i),
                    deckSessionHistory.get(i).accuracy()));
        }
        // Add this session's accuracy
        series.getData().add(new XYChart.Data<>("This Session", session.getAccuracy()));

        chart.getData().add(series);
        return chart;
    }

    // Chart 3 -> answer timeline -> visual representation of using coloured blocks
    private Group buildAnswerTimeline(){
        Group group = new Group();
        ArrayList<Boolean> history = session.getSession().getAnswerHistory();
        if(history.isEmpty()) return group;

        // Layout settings for grids
        int totalW = totalWidth; // total width
        int blockS = blockSize; // Size of each block
        int startX = startingX; // Starting x position
        int startY = zero; // starting y position
        int gap = spaceBetween; // Space between blocks
        int blocksPerRow = totalW / (blockS + gap); // How many blocks can fit in a row
        // loop through each answer
        for(int i = 0; i < history.size(); i++){
            // Determine column and row of current block
            int column = i%blocksPerRow;
            int row = i/blocksPerRow;
            // Calculate the screen position
            int x = startX + column*(blockS + gap);
            int y = startY + row*(blockS + gap);
            Rectangle block = new Rectangle(x, y, blockS, blockS);
            block.getStyleClass().add("block");
            // Colour changes based on true or false
            block.setStyle("-fx-fill: " + (history.get(i) ? "green" : "red") + ";");
            group.getChildren().add(block);
        }
        return group;
    }
}
```

5.3 Controllers

5.3.1 Controller.java

```
package Controllers;
```

```
import Models.Flashcard;
import Models.Settings;
import Models.User;

import java.io.*;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public abstract class Controller {
    // Constants
    protected static final int minStatsCSVDataItems = 11;
    protected static final int settingsCSVDataItems = 7;
    // Holds the application's current user
    private User currentUser;
    private Settings currentSettings;
    private  Settings draftSettings;


    // Ensures parent folder and CSV exists
    public void ensureCSVExists(Path csvPath) throws IOException {
        // Create a parent directory if it doesn't exist
        if(!Files.exists(csvPath.getParent())){
            Files.createDirectories(csvPath.getParent());
        }
        // Create CSV file if it doesn't exist
        if(!Files.exists(csvPath)){
            Files.createFile(csvPath);
        }
    }
    // Quarantining Corrupted files, creating a replacement
    public void quarantineCorruptedCSV(Exception cause, Path csvPath) throws IOException {
        // Ensure parent folder exists (do NOT recreate the file before moving it)
        if (!Files.exists(csvPath.getParent())) {
            Files.createDirectories(csvPath.getParent());
        }
        // Only quarantine if the corrupted file actually exists
        if(Files.exists(csvPath)){
            // Generate a timestamp so each corrupted back-up has a unique filename
            String timestamp =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("ddMMyyyy_HHmmss"));
            // Create a new path in same directory
            // Keep the corrupted file for recovery or debugging
            Path backup = csvPath.resolveSibling("corrupted" + timestamp + ".csv");
            // Move corrupted file to back up location
            // REPLACE_EXISTING ensures there is no conflict
            Files.move(csvPath, backup, StandardCopyOption.REPLACE_EXISTING);
        }
        // Create an empty users.csv so the system can continue
        Files.createFile(csvPath);
    }
    // function to validate if the csv is suitable for import
    public boolean validateCSV(File source, String fileName) throws  IOException{
        try(BufferedReader br = new BufferedReader(new FileReader(source))){
            String line;
            while ((line = br.readLine()) != null) {
                if(fileName.equals("flashcard")) {
                    // Split at ,
                    String[] data = line.split("\",\"");
                    if(data.length != 2) return false;
                    // Replace speech marks with empty space
                    String question = data[0].replace("\"", "").replace("\\n", "\n");
                    String answer = data[1].replace("\"", "").replace("\\n", "\n");
                    if(isNumeric(question) || isNumeric(answer)) return false;
                }else{
                    String[] data = line.split(",");
                    if(fileName.equals("stats.csv") && data.length >= minStatsCSVDataItems)
return false;
                    if(fileName.equals("settings.csv") && data.length != settingsCSVDataItems)
return false;
                }
            }
        }
        return true;
    }
    // Check if the value is numeric
    private boolean isNumeric(String s) {
        try {Double.parseDouble(s);return true;}
        catch (NumberFormatException e) {
            return false;}
```

```
    }
    // Getters
    public User getCurrentUser(){
        return currentUser;
    }
    public String getEmail(){
        return currentUser.getEmail();
    }
    public Settings getCurrentSettings(){
        return currentSettings;
    }

    // Setters
    public void setCurrentUser(User currentUser) {
        this.currentUser = currentUser;
    }
    public void setCurrentSettings(Settings currentSettings) {
        this.currentSettings = currentSettings;
    }
    public Settings getDraftSettings() {
        return draftSettings;
    }

    public void setDraftSettings(Settings settings) {
        this.draftSettings = settings;
    }
}
```

5.3.2 AuthenticationController.java

```
package Controllers;

import Components.ScreenManager;
import Models.Settings;
import Models.User;
import Screens.AuthScreen;

import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Base64;
import java.util.Objects;

public class  AuthenticationController extends Controller {
    // Create a SettingsController object
    private final SettingsController sc = new SettingsController();
    // Stores most recently generated salt
    private byte[] salt;
    //getResourceAsStream() is for reading packaged resources, not for storing/editing data
    private final Path csvPath = Paths.get(System.getProperty("user.dir"), "files", "users.csv");
    // Encryption / Decryption
    // Hardcoded 16-byte AES key -> 16 characters exactly for AES-128
    private static final String encryptionKey = "RevAssistant2026";
    // Encrypt a string and returns base 66 encrypted string
    private String encrypt(String line){
        try{
            // Create an AES secret key using the predefined encryption key
            SecretKey key = new SecretKeySpec(encryptionKey.getBytes(StandardCharsets.UTF_8),
"AES");
            // Create and configure the cipher for encryption with padding
            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
            cipher.init(Cipher.ENCRYPT_MODE, key);
            // Convert the text into bytes and encrypt it
            byte[] encrypted = cipher.doFinal(line.getBytes(StandardCharsets.UTF_8));
            // Convert encrypted bytes to base 64 so it can be saved safely as text
            return Base64.getEncoder().encodeToString(encrypted);
        }catch(Exception e){
            throw new RuntimeException("Encryption did not work: " + e.getMessage());
        }
```

```
    }

    // Decrypt a base 64 encoded string to plain text
    private String decrypt(String line){
        try{
            // Create an AES secret key using the predefined encryption key
            SecretKey key = new SecretKeySpec(encryptionKey.getBytes(StandardCharsets.UTF_8),
"AES");
            // Create and configure the cipher for encryption with padding
            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
            cipher.init(Cipher.DECRYPT_MODE, key);
            // Decode the Base64 string back to plain text
            byte[] decrypted = cipher.doFinal(Base64.getDecoder().decode(line));
            return new String(decrypted, StandardCharsets.UTF_8);
        }catch(Exception e){
            throw new RuntimeException("Decryption did not work: " + e.getMessage());
        }
    }
    // Read encrypted users.csv, decrypt and returns the line in an ArrayList
    private ArrayList<String> getDecryptedLines() throws IOException{
        ArrayList<String> lines = new ArrayList<>();
        // Read all content
        byte[] fileBytes = Files.readAllBytes(csvPath);
        // If file is empty, return empty list
        if(fileBytes.length == 0) return lines;
        // Decrypt the entire file
        String decrypted = decrypt(new String(fileBytes, StandardCharsets.UTF_8).trim());
        // Iterate through and add to ArrayList
        for(String line : decrypted.split("\n")){
            if(!line.isBlank()) lines.add(line);
        }
        return lines;
    }
    // Encrypt the given content and write to users.csv
    private void writeEncrypted(String text) throws IOException{
        // get encrypted text
        String encrypted = encrypt(text);
        // Write to the file
        Files.writeString(csvPath, encrypted);
    }
    // Registers a user
    public boolean registerUser(String email, String inpPassword, String recoveryWord,
SettingsController sc) throws NoSuchAlgorithmException, IOException {
        // Check if email does not already exist within the CSV
        if (!doesEmailExist(email)) {
            // Create a new random salt, preventing identical hashes for same password
            generateSalt();
            // Hash password using SHA-512 and generated salt
            String hashedPassword = hashPassword(inpPassword, getSalt());
            // Store salt so it can be safely written into the CSV
            String saltBase64 = Base64.getEncoder().encodeToString(getSalt());
            // Create new user object
            User user = new User(email, hashedPassword, saltBase64, getDate(), recoveryWord);
            // Load existing users and add new and then rewrite entire encrypted files
            ArrayList<User> users = convertCSVtoArrayList();
            users.add(user);
            rewriteUsersCSV(users);
            // On register the system's current user will be set to the temp user
            // Settings will be loaded and the SettingsController set into the ScreenManager
class
            setCurrentUser(user);
            sc.createDefaultSettings(email);
            sc.setCurrentSettings(sc.loadSettings(email));
            setSettingsControllertoSM(sc);
            // Write stats into stats.csv for new user
            new StatsController().createDefaultUserStats(email);
            return true;
        }else{
            // Email already in CSV
            return false;
        }
    }
    // Logs in a user
    public boolean loginUser(String email, String inpPassword, SettingsController sc) throws
NoSuchAlgorithmException, IOException {
        // Only attempt if email is found
        if (doesEmailExist(email)) {
            // temp will hold the user we find by email
            User temp = findUserByEmail(email);
            if (temp == null) {
                return false;
            }
```

```
            // Check if password matches,
            // Objects.requireNonNull(temp) prevents a NullPointerException without explanation,
(if null, it throws immediately)
            boolean loggedIn = verifyPassword(inpPassword,
Objects.requireNonNull(temp).getHashedPassword(), temp.getSaltBase64());
            if(loggedIn){
                // On login the system's current user will be set to the temp user
                // Settings will be loaded and the SettingsController set into the ScreenManager
class
                setCurrentUser(temp);
                Settings userSettings = sc.loadSettings(email);
                sc.setCurrentSettings(userSettings);
                setSettingsControllertoSM(sc);
                // Write default stats if not saved already, via a temporary object
//                if(!new StatsController().isUserStatsSaved(email)) new
StatsController().createDefaultUserStats(email);
                return true;
            }
        }
        // Email not found
        return false;
    }
    // Reads users.csv and converts each row into a user object
    public ArrayList<User> convertCSVtoArrayList() throws IOException {
        // Ensure folders and CSV exist before reading
        ensureCSVExists(csvPath);
        ArrayList<User> users = new ArrayList<>();
        // Read CSV line by line
        try {
            ArrayList<String> lines = getDecryptedLines();
            for (String line : lines) {
                String[] temp = line.split(",");
                if(temp.length < 5) throw new IllegalArgumentException("Corrupted Row: " + line);
                users.add(new User(temp[0], temp[1], temp[2], temp[3], temp[4]));
            }
        } catch (Exception e) {
            quarantineCorruptedCSV(e, csvPath);
            // Act as if there are no users if corrupted.
            return new ArrayList<>();
        }
        return users;
    }
    // Will be used to update CSV when user makers changes
    private void rewriteUsersCSV(ArrayList<User> users) throws IOException {
        StringBuilder sb = new StringBuilder();
        // Rewrite the entire file, so updates persist
        for (User u : users) {

sb.append(u.getEmail()).append(",").append(u.getHashedPassword()).append(",").append(u.getSaltBas
e64()).append(",")

.append(u.getLastLogin()).append(",").append(u.getRecoveryWord()).append("\n");
        }
        writeEncrypted(sb.toString());
    }
    // Checks if a given email exists in the CSV
    public boolean doesEmailExist(String email) throws IOException {
        ArrayList<User> users = convertCSVtoArrayList();
        for (User user : users) {
            // Will not be case-sensitive
            if (user.getEmail().equalsIgnoreCase(email)) {
                return true;
            }
        }
        return false;
    }
    // Finds and returns the Models.User object for given email
    public User findUserByEmail(String email) throws IOException {
        ArrayList<User> users = convertCSVtoArrayList();
        for (User user : users) {
            if (user.getEmail().equals(email)) {
                return user;
            }
        }
        return null;
    }
    // Hash password algorithm via SHA-512 (even though, others are better, I'll describe in
write up); throws exception in case MessageDigest instance fails.
    public String hashPassword(String password, byte[] salt) throws NoSuchAlgorithmException {
        // Create a MessageDigest instance using the SHA-512 hashing algorithm
        MessageDigest md = MessageDigest.getInstance("SHA-512");
        // Add the salt to the has function so same passwords can produce different hashes per
```

```
user
        md.update(salt);
        // Hash the password bytes, giving a one-way hash
        byte[] hashedPassword = md.digest(password.getBytes());
        // Convert the raw hash bytes to a base 64 string so it can be stored safely in the .csv
file
        return Base64.getEncoder().encodeToString(hashedPassword);
    }
    // Verify the password is correct, taking the user's input password comparing with hashed
password.
    public boolean verifyPassword(String inpPassword, String hashedPassword, String saltBase64)
throws NoSuchAlgorithmException {
        // Convert the string salt to a byte array
        byte[] salt = Base64.getDecoder().decode(saltBase64);
        // Create a MessageDirect instance using the same hashing algorithm as before
        MessageDigest md = MessageDigest.getInstance("SHA-512");
        // Add the same salt which the hash password algorithm used
        md.update(salt);
        // hash the input password into a byte array
        byte[] hashedInputPassword = md.digest(inpPassword.getBytes());
        // convert the byte array into a string
        String newHash = Base64.getEncoder().encodeToString(hashedInputPassword);
        // true if the hashed input matches the hashed password
        return newHash.equals(hashedPassword);
    }
    // Reset user's password
    public boolean resetPassword(User temp, String newPassword) throws NoSuchAlgorithmException,
IOException {
        // new salt ensures old hash cannot be used and increases security
        generateSalt();
        String hashedPassword = hashPassword(newPassword, getSalt());
        String saltBase64 = Base64.getEncoder().encodeToString(getSalt());
        // Load all users and update the matching one
        ArrayList<User> users = convertCSVtoArrayList();
        for(User u : users){
            if(u.getEmail().equals(temp.getEmail())){
                u.setPasswordHash(hashedPassword);
                u.setSaltBase64(saltBase64);
                break;
            }
        }
        // Write into file
        rewriteUsersCSV(users);
        return true;
    }
    // Change a user's email
    public boolean changeEmail(User temp, String newEmail) throws IOException {
        ArrayList<User> users = convertCSVtoArrayList();
        // Load all users and update the matching one
        for(User u : users){
            if(u.getEmail().equals(temp.getEmail())){
                u.setEmail(newEmail);
                break;
            }
        }
        // Write into file
        rewriteUsersCSV(users);
        // Update the stats.csv and settings.csv, replace old email with new
        sc.updateEmail(temp.getEmail(), newEmail);
        new StatsController().updateEmail(temp.getEmail(), newEmail);
        // Rename the Sessions csv
        Path oldSession = Paths.get(System.getProperty("user.dir"), "files", "sessions",
temp.getEmail() + ".csv");
        Path newSession = Paths.get(System.getProperty("user.dir"), "files", "sessions", newEmail
+ ".csv");
        if(Files.exists(oldSession)) Files.move(oldSession, newSession,
StandardCopyOption.REPLACE_EXISTING);
        // Rename the decks folder
        Path oldFolder = Paths.get(System.getProperty("user.dir"), "userFolders",
temp.getEmail());
        Path newFolder = Paths.get(System.getProperty("user.dir"), "userFolders", newEmail);
        if(Files.exists(oldFolder)) Files.move(oldFolder, newFolder,
StandardCopyOption.ATOMIC_MOVE);
        // Update current user in memory
        temp.setEmail(newEmail);
        setCurrentUser(temp);
        return true;
    }
    // Change a user's recovery word
    public boolean changeRecoveryWord(User temp, String newRecoveryWord) throws IOException {
        ArrayList<User> users = convertCSVtoArrayList();
        for(User u : users){
```

```
            if(u.getEmail().equals(temp.getEmail())){
                u.setRecoveryWord(newRecoveryWord);
                break;
            }
        }
        // Write into file
        rewriteUsersCSV(users);
        return true;
    }

    // System Logout
    public void logout(){
        if (getCurrentUser() != null){
            getCurrentUser().setLastLogin(getDate());
        }
        // System has no current user, so data cannot be accessed
        setCurrentUser(null);
        // User cannot find a way to go back into Main Menu after logging out
        ScreenManager.clearHistory();
        // Switch to Log in/Registration Screen
        setSettingsControllertoSM(sc);
        ScreenManager.switchScene(new AuthScreen(new AuthenticationController(), sc).create());
    }
    private void generateSalt(){
        // Cryptographic secure random number gen (better than java.util.random)
        SecureRandom random = new SecureRandom();
        // 16 byte salt to ensure each password hash is unique
        byte[] salt = new byte[16];
        // generating random salt bytes and store them in the salt
        random.nextBytes(salt);
        this.salt = salt;
    }
    //Setters
    // Getters for attributes
    public byte[] getSalt(){
        return this.salt;
    }
    public String getDate(){
        LocalDateTime time = LocalDateTime.now();
        DateTimeFormatter expectedFormat = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
        return time.format(expectedFormat);
    }
    private void setSettingsControllertoSM(SettingsController sc){
        ScreenManager.setSC(sc);
    }

    public SettingsController getSettingsController(){
        return sc;
    }
}
```

5.3.3 SettingsController.java

```
package Controllers;

import Models.Settings;
import Models.UserStats;

import java.io.*;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;

public class SettingsController extends Controller {
    private final Path csvPath = Paths.get(System.getProperty("user.dir"), "files",
"settings.csv");

    // Write default settings into settings.csv on user registration
    public void createDefaultSettings(String email) throws IOException {
        ensureCSVExists(csvPath);
        if (!doesSettingsExist(email)) {
            Settings defaultSettings = new Settings();
            try (BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toString(),
true))) {
                bw.write(toCSVRow(email, defaultSettings));
                bw.newLine();
            }
        }
    }

    // Load settings for a user
```

```
    public Settings loadSettings(String email) throws IOException {
        // Method from Controller superclass
        ensureCSVExists(csvPath);
        ArrayList<Settings> settings = convertCSVtoArrayList();
        // Linear search for user
        for(Settings s : settings) {
            if(s.getEmail().equals(email)) {
                return s;
            }
        }
        // If not found
        Settings defaultSettings = new Settings();
        defaultSettings.setEmail(email);
        return defaultSettings;
    }

    // Save settings.csv for a user if changed
    public boolean saveSettings(Settings updatedSettings) throws IOException {
        ArrayList<Settings> settings = convertCSVtoArrayList();
        boolean found = false;
        for(Settings s : settings) {
            if(s.getEmail().equals(updatedSettings.getEmail())) {
                copySettings(updatedSettings, s);
                found = true;
                break;
            }
        }
        // if not found add to array list
        if(!found) {
            Settings s = updatedSettings.copy();
            settings.add(s);
        }
        rewriteCSV(settings);
        return true;
    }

    // Checks if a user has settings already
    private boolean doesSettingsExist(String email) throws IOException {
        ArrayList<Settings> settingsList = convertCSVtoArrayList();
        for (Settings settings : settingsList) {
            if (settings.getEmail().equals(email)) {
                return true;
            }
        }
        return false;
    }
    // Reads settings.csv and converts each row into a user object
    private ArrayList<Settings> convertCSVtoArrayList() throws IOException {
        // Ensure folders and CSV exist before reading
        ensureCSVExists(csvPath);
        ArrayList<Settings> settingsList = new ArrayList<>();
        // Read CSV line by line
        try (BufferedReader br = new BufferedReader(new FileReader(csvPath.toString()))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] temp = line.split(",");
                // Validate row lengths to avoid ArrayIndexOutOfBoundsException, throw an
exception
                if (temp.length != settingsCSVDataItems) {
                    throw new IllegalArgumentException("Corrupted Row: " + line);
                }
                // Create a settings object
                Settings settings = new Settings(temp[0], temp[1], Double.parseDouble(temp[2]),
Double.parseDouble(temp[3]),
                        Double.parseDouble(temp[4]), Boolean.parseBoolean(temp[5]),
Boolean.parseBoolean(temp[6]));
                // Add object to ArrayList
                settingsList.add(settings);
            }
        }catch (Exception e){
            quarantineCorruptedCSV(e, csvPath);
            // Return an empty arraylist, no settings saved
            return new ArrayList<>();
        }
        return settingsList;
    }

    // Will be used to update CSV when user makers changes
    private void rewriteCSV(ArrayList<Settings> settingsList) throws IOException {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toString()))) {
            for (Settings settings : settingsList) {
                bw.write(toCSVRow(settings.getEmail(), settings));
```

```
                bw.newLine();
            }
        }
    }
    // Procedure to update the email within the file when the user changes their email
    public void updateEmail(String oldEmail, String newEmail) throws IOException {
        // load all users
        ArrayList<Settings> settingsList = convertCSVtoArrayList();
        for(Settings settings : settingsList){
            if(settings.getEmail().equals(oldEmail)){
                // change the email
                settings.setEmail(newEmail);
                break;
            }
        }
        // rewrite the file
        rewriteCSV(settingsList);
    }

    // Convert the settings object into a CSV Row
    private String toCSVRow(String email, Settings s) {
        return email + "," +
                s.getTheme() + "," +
                s.getDarkness() + "," +
                s.getTTSVolume() + "," +
                s.getSFXVolume() + "," +
                s.isMuted() + "," +
                s.isTTSEnabled();
    }
    // Update a user's setting object
    private void copySettings(Settings oldSettings, Settings newSettings) {
        newSettings.setTheme(oldSettings.getTheme());
        newSettings.setDarkness(oldSettings.getDarkness());
        newSettings.setTTSVolume(oldSettings.getTTSVolume());
        newSettings.setSFXVolume(oldSettings.getSFXVolume());
        newSettings.setMuted(oldSettings.isMuted());
        newSettings.setTTSEnabled(oldSettings.isTTSEnabled());
    }
    // Getters
    public double getSFXVolume() throws IOException {
        if(getCurrentSettings() == null){
            setCurrentSettings(loadSettings(getCurrentUser().getEmail()));
        }
        return getCurrentSettings().getSFXVolume();
    }
    public double getTTSVolume() throws IOException {
        return getCurrentSettings().getTTSVolume();
    }
}
```

5.3.4 FlashcardController.java

```
package Controllers;

import Components.CustomAlert;
import Models.Flashcard;
import javafx.scene.control.TreeItem;
import javafx.stage.FileChooser;
import javafx.stage.Window;

import java.io.*;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class FlashcardController extends Controller {
    // Constants
    private static final int deckDataItems = 2;
    private static final int maxDeckSize = 60;
    // Holds the string to the user's directory path
    private final String userFolderPath;
    // Hold then name of the flashcard deck file
    private String flashcardDeckName;
    private final String email;
    private final Path wrongAnswersPath;

    // Constructor to set the userFolderPath
```

```
    public FlashcardController(String email) {
        this.email = email;
        // Sets the path to the user folder
        this.userFolderPath = Paths.get(System.getProperty("user.dir"),"userFolders",
email).toString();
        this.wrongAnswersPath = Paths.get(userFolderPath, "wrongAnswers.csv");
    }

    public String getFlashcardDeckName() {
        return flashcardDeckName;
    }
    public String getUserFolderPath() {
        return userFolderPath;
    }

    public void setFlashcardDeckName(String flashcardDeckName) {
        this.flashcardDeckName = flashcardDeckName;
    }
    // Retrieves path for the FCDeck
    private String getFCDeckPath() throws IOException {
        // If flashcard deck is stored directly in the user folder, then return it
         File rootFolder = new File(userFolderPath);
         return searchForCSV(rootFolder, flashcardDeckName);
    }
    //  Find the csv in the user folder
    private String searchForCSV(File searchDirectory, String deckName){
        File[] contents = searchDirectory.listFiles();
        // If no file, return null
        if (contents==null) return null;
        for(File f : contents){
            // For each file object in the array check, if directory
            // Else check if name matches
            if(f.isDirectory()){
                // Recurse into sub-folder
                String result = searchForCSV(f, deckName);
                // Return path
                if(result!=null) return result;
            } else if (f.getName().equals(deckName + ".csv")) {
                // Return path
                return f.getAbsolutePath();
            }
        }
        return null;
    }

    // Check if file and folder exist
    private void ensureFileExists(boolean adding) throws IOException {
        // Create a Folder if not already there
        Files.createDirectories(Paths.get(userFolderPath));
        // If not adding to a file then
        if (!adding){
            String deckPath = userFolderPath + "/" + flashcardDeckName + ".csv";
            // Create the file, if doesnt exist
            Path path = Paths.get(deckPath);
            if (!Files.exists(path)) {
                Files.createFile(path);
            } else {
                // If exists then add a random number to keep uniqueness
                int random = (int) (Math.random() * 100);
                flashcardDeckName = flashcardDeckName + random;
            }
        }
    }
    // Function to write the flashcard deck into .csv
    public boolean saveFlashcardDeck(List<Flashcard> cards, boolean adding) throws IOException {
        ensureFileExists(adding);
        String path = getFCDeckPath();
        if (path == null) path = userFolderPath + "/" + flashcardDeckName + ".csv";
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(path))) {
            for (Flashcard c : cards) {
                // Add speech marks, replace speech marks inside
                bw.write("\"" // Opening speech marks
                        + c.getQuestion().replace("\"", "\"\"") // Get rid of any speech marks
within by adding another
                        .replace("\n", "\\n") // Replace new lines with \n so readline does not
break mid-card
                        + "\",\"" // Closing speech marks with comma
                        + c.getAnswer().replace("\"", "\"\"").replace("\n", "\\n") + "\"");
                bw.newLine();
            }
        }
        return true;
```

```
    }

    // Function to read in flashcards from a .csv into an array list
    public ArrayList<Flashcard> loadFlashcardDeck(String deckName) throws IOException {
        ArrayList<Flashcard> cards = new ArrayList<>();
        // Set deck to get from parameter
        this.flashcardDeckName = deckName;
        String path = getFCDeckPath();
        if (path == null) return cards;
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            readData(br, cards, false);
        }catch (IOException e){
            // if not available, corrupt
            quarantineCorruptedCSV(e, Path.of(getFCDeckPath()));
        }
        return cards;
    }
    // Function to read in flashcards from a .csv into an array list
    public ArrayList<Flashcard> loadFCDeckFromPath(String path, boolean isWrongAnswers) throws
IOException {
        ArrayList<Flashcard> cards = new ArrayList<>();
        // Set deck to get from parameter
        if (path == null) return cards;
        if(!new File(path).exists()) return cards;
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            readData(br, cards, isWrongAnswers);
        }catch (IOException e){
            // if not available, corrupt
            quarantineCorruptedCSV(e, Path.of(getFCDeckPath()));
        }
        return cards;
    }
    // Re-usable method convert csv to flashcard object list
    private void readData(BufferedReader br, ArrayList<Flashcard> cards, boolean isWrongAnswers)
throws IOException {
        String line;
        while ((line = br.readLine()) != null) {
            // Split at ,
            String[] data = line.split("\",\"");
            if(data.length == deckDataItems){
                // Replace speech marks with empty space
                String question = data[0].replace("\"", "").replace("\\n", "\n");
                String answer = data[1].replace("\"", "").replace("\\n", "\n");
                // Add a new flashcard object
                cards.add(new Flashcard(question, answer));
            }
        }
        // Error indication to user, if file can't be read
        if(cards.isEmpty() && !isWrongAnswers)
CustomAlert.error(javafx.stage.Window.getWindows().getFirst(),
                "Problems reading file, might not be of correct format");
    }
    // Return array list of all decks linked to an account
    public ArrayList<String> getFileContents() {
        ArrayList<String> deckNames = new ArrayList<>();
        File userFolder = new  File(userFolderPath);
        if (!userFolder.exists()){
            return  deckNames;
        }
        getAllCSV(userFolder, deckNames);
        return deckNames;
    }
    // Recursively search through folder and subfolders
    // to add all csv files to the arraylist
    private void getAllCSV(File folder, ArrayList<String> deckNames){
        File[] contents = folder.listFiles();
        if(contents == null) return;
        // Loop for each file
        for(File file : contents){
            // If folder, then recurse into the directory
            if(file.isDirectory()){
                getAllCSV(file, deckNames);
            } else if (file.getName().endsWith(".csv")) {
                // Add to the array list
                deckNames.add(file.getName().replace(".csv", ""));
            }
        }
    }
    // Function to change the name of a flashcard deck
    public boolean renameDeck(String oldName, String newName) {
       this.flashcardDeckName = oldName;
        try{
```

```
            String oldPath = getFCDeckPath();
            if(oldPath == null) return false;
            // Write into same folder
            Path oldNamePath = Paths.get(oldPath);
            // Joins directory with new filename to build destination
            Path newNamePath = oldNamePath.getParent().resolve(newName + ".csv");
            Files.move(oldNamePath, newNamePath, StandardCopyOption.REPLACE_EXISTING);
            // update after successful rename
            this.flashcardDeckName = newName;
            return true;
        }catch (IOException e){
            return false;
        }
    }

    // Procedure to delete the deck user selects
    public void deleteDeck(String deckName) throws IOException {
        // Set deck name so getFCDeckPath can search
        this.flashcardDeckName = deckName;
        // get path
        String path = getFCDeckPath();
        if (path == null) return;
        // delete from path
        Files.delete(Paths.get(path));
    }

    // Function to return the full directory structure for the current user
    public TreeItem<String> getUserDirectoryStructure(String email) {
        File userDirectory = new File(userFolderPath);
        // create root node of tree with first user folder
        TreeItem<String> root = new TreeItem<>("\uD83D\uDCC1 " + email);
        // Populate tree with remaining contents
        buildStructureTree(root, userDirectory);
        return root;
    }
    // Recursively search a directory and add its content to the tree
    private void buildStructureTree(TreeItem<String> node, File parentDirectory) {
        // Place all files and folders inside the parent directory into an array
        File[] allUserFiles = parentDirectory.listFiles();
        // If there are no files stop here
        if(allUserFiles == null) return;
        // Loop through all the contents
        for(File f : allUserFiles) {
            System.out.println("Found: " + f.getName() + " isDirectory: " + f.isDirectory());
            if(f.isDirectory()) {
                // If it's a folder, create a folder node with the prefix and file name
                TreeItem<String> folderNode = new TreeItem<>("\uD83D\uDCC1 " + f.getName());
                // Recursively go into the subfolder to add its contents
                buildStructureTree(folderNode, f);
                // add current node as a child to the current node
                node.getChildren().add(folderNode);
            }else if (f.getName().endsWith(".csv")) {
                // csv file storing deck info
                // Add as a leaf node to a tree, replace .csv with prefix and the file name
                node.getChildren().add(new TreeItem<>("\uD83D\uDCC4 " +
f.getName().replace(".csv","")));
            }
        }
        // All other files which are not a folder or ending in .csv are ignored
    }
    // Returns a file/folder path
    public String getFilePath(String name, boolean isFolder) {
        File userDirectory = new File(userFolderPath);
        return searchForPath(userDirectory, name, isFolder);
    }
    // Recursively search user's directory for the file or file
    private String searchForPath(File directory, String name, boolean isFolder) {
        // Get all files and folders inside current folder
        File[] allUserFiles = directory.listFiles();
        if(allUserFiles == null) return null;
        // Loop through all items in directory
        for(File f : allUserFiles) {
            if(f.isDirectory()) {
                // If looking for folder, and name matches return the path
                if(isFolder && f.getName().equals(name)) return f.getAbsolutePath();
                // Recursively search the directory, go into it
                String filePath = searchForPath(f, name, isFolder);
                // If it's within the sub folder, return the path
                if(filePath != null) return filePath;
                // If looking for file, return the path if file
            }else if(!isFolder && f.getName().equals(name + ".csv")) return f.getAbsolutePath();
        }
```

```
        // If not found
        return null;
    }
    // Moving file/folder to a new destination
    public boolean move(String fName, boolean isFolder, String destinationFolder) throws
IOException {
        // Find path of file or folder being moved
        String fPath = getFilePath(fName, isFolder);
        if(fPath == null) return false;
        // Find destination's path
        String dPath;
        if(destinationFolder.equals(email)){
            dPath = userFolderPath;
        }else{
            dPath = getFilePath(destinationFolder, true);
        }
        if(dPath == null) return false;
        // Create File objects
        File initial = new  File(fPath);
        File end = new  File(dPath + "/" + initial.getName());
        // switch file locations
        Files.move(initial.toPath(), end.toPath(), StandardCopyOption.REPLACE_EXISTING);
        return true;
    }
    // Function to create a new directory
    public int createFolder(String parentPath, String folderName){
        // Create file object with specified path
        File newFolder = new File(parentPath + "/" + folderName);
        if(newFolder.exists()){
            return 1; // if exists return 1
        }else{
            if(newFolder.mkdir()){
                return 2; // if added return 2
            }else{
                return 3; // anything else add 3 return 3
            }
        }
    }
    // Function deleting a folder
    public boolean deleteFolder(String folderName) throws IOException {
        // get the path
        String path = getFilePath(folderName, true);
        // if null return not deleted
        if(path == null) return false;
        // Walk through all files and subfolders from given path
        Files.walk(Path.of(path))
                // Sort them in reverse order so they are deleted before parent directories
                .sorted(Comparator.reverseOrder())
                // Convert each path to an object
                .map(Path::toFile)
                // Delete each file
                .forEach(File::delete);
        return true;
    }

    // Procedure which allows the user to import a csv file into the system
    public boolean importCSV(Window owner) throws IOException {
        // Open the file chooser, filtered to csv files only
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Import CSV File");
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("CSV Files",
"*.csv"));
        // Store user's chosen file
        File file = fileChooser.showOpenDialog(owner);
        // If user doesn't choose a file, don't do anything
        if(file == null) return false;
        // Check size limits of flashcard deck
        ArrayList<Flashcard> cardsList = loadFCDeckFromPath(file.getAbsolutePath(), false);
        if(!validateCSV(file ,"flashcard")){
            CustomAlert.error(owner, "Invalid file format");
            return false;
        }
        if(cardsList.isEmpty() || cardsList.size() > maxDeckSize){
            CustomAlert.error(owner, "A deck needs a minimum of 1 card and can have a maximum of
60 cards");
            return false;
        }
        // Check if user is importing flashcard deck from another user, or one which already
exists within their folder
        String allUserFolders =
Paths.get(System.getProperty("user.dir"),"userFolders").toString();
        if(file.getCanonicalPath().startsWith(new File(allUserFolders).getCanonicalPath())){
```

```
            CustomAlert.error(owner, "Not allowed to import from this directory");
            return false;
        }
        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String importDeck = file.getName().replace(".csv", "");
            // Set the file name
            setFlashcardDeckName(importDeck);
            // Use default save method
            saveFlashcardDeck(cardsList, true);
        }
        return true;
    }

    // Procedure to export a user-chosen flashcard deck to CSV or HTML, at a location of their
choosing
    public boolean export(Window owner, boolean isHTML) throws IOException {
        CustomAlert.info(owner, "Choose the deck to export, can only be from this directory (and
sub-directories of this one)");
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Export CSV File");
        fileChooser.setInitialDirectory(new File(userFolderPath));
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("CSV Files",
"*.csv"));
        // Ask user where to save the file
        File chosenDeck = fileChooser.showOpenDialog(owner);
        System.out.println("Chosen deck: " + chosenDeck);
        CustomAlert.info(owner, "Choose the location to save the deck");
        File userFolder = new File(userFolderPath);
        // If cancelled return false
        if(chosenDeck == null){
            return false;
        }
        // If outside user directory, return false
        // Canonical path -> fully resolved path
        if(!chosenDeck.getCanonicalPath().startsWith(userFolder.getCanonicalPath())) return
false;
        // Get file name
        String exportDeck = chosenDeck.getName().replace(".csv", "");
        // Ask user to choose where to save the file
        FileChooser locationChooser = new FileChooser();
        locationChooser.setTitle("Choose Save Location");
        // Set initial save file name to current file name
        if(!isHTML){
            locationChooser.setInitialFileName(chosenDeck.getName());
            // Filter for only .csv
            locationChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("CSV
Files", "*.csv"));
        }
        if(isHTML){
            locationChooser.setInitialFileName(chosenDeck.getName().replace("csv","html"));
            // Filter for only .html
            locationChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("HTML
Files", "*.html"));
        }

        File chosenLocation = locationChooser.showSaveDialog(owner);

        if(chosenLocation == null) return false;
        if(!isHTML) {
            try (BufferedWriter bw = new BufferedWriter(new FileWriter(chosenLocation))) {
                for (Flashcard card : loadFlashcardDeck(exportDeck)) {
                    bw.write("\"" + card.getQuestion().replace("\"", "\"\"").replace("\n", "\\n")
                            + "\",\"" + card.getAnswer().replace("\"", "\"\"").replace("\n",
"\\n") + "\"");
                    bw.newLine();
                }
            }
        }else{
            try (BufferedWriter bw = new BufferedWriter(new FileWriter(chosenLocation))) {
                writeHTML(bw, exportDeck);
            }
        }
        return true;
    }

    // Procedure to write HTML File
    private void writeHTML(BufferedWriter bw, String exportDeck) throws IOException {
        bw.write("<!DOCTYPE html>");
        bw.newLine();
        bw.write("<html><head><title>" + exportDeck + "</title><style>");
        bw.newLine();
        bw.write("body{font-family: Arial, sans-serif; margin: 40px; font-size: 18px;}");
```

```
        bw.newLine();
        bw.write("h1{color: black; text-align: center;}");
        bw.newLine();
        bw.write("table{width: 100%; border-collapse: collapse; margin-top: 20px;}");
        bw.newLine();
        bw.write("th{background-color: black; color: white; padding: 40px; text-align: center;
border-right: 1px solid black; " +
                "border-left: 1px solid black; border-bottom: 1px solid black; border-top: 1px
solid black;}");
        bw.newLine();
        bw.write("td{padding: 50px; border-bottom: 1px solid black; text-align: left; border-
right: 1px solid black; border-left: 1px solid black; " +
                "border-top: 1px solid black;word-wrap: break-word; max-width: 250px; font-size:
18px;}");
        bw.newLine();
        bw.write("tr:nth-child(even){background-color: #c8c8c8;}");
        bw.newLine();
        bw.write("</style></head><body>");
        bw.newLine();
        bw.write("<h1>" + exportDeck + "</h1>");
        bw.newLine();
        bw.write("<table><tr><th>Question</th><th>Answer</th></tr>");
        for(Flashcard card : loadFlashcardDeck(exportDeck)){
            bw.write("<tr><td>" + card.getQuestion() + "</td><td>" + card.getAnswer() +
"</td></tr>");
        }
        bw.write("</table></body></html>");
    }

    // Procedure to save user's wrong answers to WrongAnswers.csv for the WrongAnswers mode
    public void saveWrongAnswers(Flashcard card) throws IOException {
        ArrayList<Flashcard> cards = loadWrongAnswers();
        // Check if already exists
        boolean exists = cards.stream().anyMatch(
                c -> c.getQuestion().equals(card.getQuestion()) &&
c.getAnswer().equals(card.getAnswer())
        );
        if(exists) return;
        try(BufferedWriter bw = new BufferedWriter(new FileWriter(wrongAnswersPath.toFile(),
true))){
            // Add speech marks, replace speech marks inside
            bw.write("\"" // Opening speech marks
                    + card.getQuestion().replace("\"", "\"\"") // Get rid of any speech marks
within by adding another
                    .replace("\n", "\\n") // Replace new lines with \n so readline does not break
mid-card
                    + "\",\"" // Closing speech marks with comma
                    + card.getAnswer().replace("\"", "\"\"").replace("\n", "\\n"));
            bw.newLine();
        }
    }

    // load wrong answers into an array list
    public ArrayList<Flashcard> loadWrongAnswers() throws IOException {
        ArrayList<Flashcard> cards =
loadFCDeckFromPath(wrongAnswersPath.toFile().getCanonicalPath(), true);
        return cards;
    }


}
```

5.3.5 SessionController.java

```
package Controllers;

import Models.Flashcard;
import Models.Session;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Random;

public class SessionController {
    private final String email;
    private Session session;
    private FlashcardController fc;
    // If user want to continue/retry
    private ArrayList<Flashcard> lastCards;

    // Constructor
```

```
    public SessionController(String email, FlashcardController fc) {
        this.email = email;
        this.fc = fc;
    }
    // Start session
    public void startSession(ArrayList<Flashcard> cards, String deck, String mode){
        this.lastCards = new ArrayList<>(cards);
        // Create a new session object
        this.session = new Session(cards, deck, mode);
    }
    // Retry
    public void retrySession(){
        // Shuffle cards
        if(getStudyMode().equals("Shuffle")) shuffle(lastCards);
        // Continue session
        session.reset(lastCards);
    }
    // Shuffle ArrayList of cards for Shuffle mode
    public void shuffle(ArrayList<Flashcard> cards) {
        // Create a random object to generate random indices
        Random rand = new Random();
        // Starting at the last element and move backwards
        for(int i = cards.size()-1; i > 0; i--) {
            // Generate a random index between 0 and i inclusive
            int j = rand.nextInt(i+1);
            // Store element at index i temporarily
            Flashcard temp = cards.get(i);
            // Replace element at index i with element at index j
            cards.set(i, cards.get(j));
            // Place stored element into index j
            cards.set(j, temp);

        }
    }
    // Function to record answer
    public boolean recordAnswer(boolean correct) throws IOException {
        // add to wrongAnswers.csv
        if(!correct) {
            fc.saveWrongAnswers(session.getCurrentCard());
            session.resetConsecutiveCorrect();
        }else session.incrementConsecutiveCorrect();
        // Update it in the session object
        return session.recordAnswer(correct);
    }
    // Increase by a second
    public void tick(){
        session.tick();
    }
    // Function to call the function format time
    public String getFormattedTime(){
        return formatTime(session.getElapsedTime());
    }

    // Getters
    public ArrayList<Flashcard> loadWrongAnswers() throws IOException {
        return fc.loadWrongAnswers();
    }
    public Flashcard getCurrentCard(){return session.getCurrentCard();}
    public String getSelectedDeck() {
        return session.getSelectedDeck();
    }
    public double getAccuracy(){
        return session.getAccuracy();
    }
    public String getStudyMode() {
        return session.getStudyMode();
    }
    public int getRemainingCount() {
        return session.getRemainingCount();
    }
    public int getIncorrectCount() {
        return session.getIncorrectCount();
    }
    public int getCorrectCount() {
        return session.getCorrectCount();
    }
    public long getElapsedTime() {
        return session.getElapsedTime();
    }
    public ArrayList<Flashcard> getCurrentQueue() {
        return session.getCurrentQueue();
    }
```

```
    public ArrayList<Flashcard> getLastCards(){
        return this.lastCards;
    }
    public void setShowingAnswer(boolean showingAnswer) {
        session.setShowingAnswer(showingAnswer);
    }
    public boolean isShowingAnswer() {
        return session.isShowingAnswer();
    }
    public String getProgressText(){
        return session.getProgressText();
    }
    public Session getSession(){
        return session;
    }

    public void setLastCards(ArrayList<Flashcard> lastCards) {
        this.lastCards = lastCards;
    }
    //  Format the time in 00:00:00
    public String formatTime(Long seconds){
        return String.format("%02d:%02d:%02d", seconds / 3600, (seconds % 3600) / 60, seconds %
60);
    }

}
```

5.3.6 TTSController.java

```
package Controllers;

import Models.Settings;
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
import marytts.LocalMaryInterface;
import javax.sound.sampled.AudioFileFormat;
import javax.sound.sampled.AudioInputStream;
import javax.sound.sampled.AudioSystem;
import java.io.File;


public class TTSController extends Controller{
    // Entry point for synthesis
    private LocalMaryInterface maryTTS;
    // To play synthesised audio
    private MediaPlayer audioPlayer;
    private Settings userSettings;
    // Default values
    private String voice = "dfki-prudence-hsmm";
    private double ttsVolume;
    private double rate;
    // Constructor
    public TTSController(double ttsVolume){
        this.ttsVolume = ttsVolume;
        this.rate = 1.0;
        // Initialise the MaryTTS on a separate thread -> prevent slow start
        new Thread(() -> {
            try{
                // Initialise MaryTTS
                maryTTS = new LocalMaryInterface();
                // Set default voice
                maryTTS.setVoice(voice);
            }catch(Exception e){
                System.out.println(e.getMessage());
            }
        }).start();
    }

    // Synthesises and plays the given text
    public void speak(String text){
        // Stop any audio playing first
        stop();
        try{
            // generate audio from text
            AudioInputStream audio = maryTTS.generateAudio(text);
            File tempfile = File.createTempFile("tts", ".wav");
            AudioSystem.write(audio, AudioFileFormat.Type.WAVE, tempfile);
            tempfile.deleteOnExit();

            Media media = new Media(tempfile.toURI().toString());
```

```
            audioPlayer = new MediaPlayer(media);
            audioPlayer.setVolume(ttsVolume);
            audioPlayer.setRate(rate);
            audioPlayer.play();
        }catch(Exception e){
            System.out.println(e.getMessage());
        }
    }
    // Stop the audio
    public void stop(){
        if(audioPlayer != null){
            audioPlayer.stop();
            audioPlayer = null;
        }
    }
    // Getters and Setters
    public void setRate(double rate){
        this.rate = Math.min(2.0, Math.max(1.0, Math.round(rate * 10.0) / 10.0));
    }
    public double getRate(){
        return rate;
    }

    public void setTTSVolume(double ttsVolume) {
        this.ttsVolume = ttsVolume;
    }
}
```

5.3.7 StatsController.java

```
package Controllers;

import Models.Session;
import RecordClasses.SessionRecord;
import Models.UserStats;

import java.io.*;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Arrays;

public class StatsController extends Controller{
    private static final DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    private final Path csvPath = Paths.get(System.getProperty("user.dir"), "files", "stats.csv");

    // Read stats.csv into an ArrayList of UserStats objects
    private ArrayList<UserStats> convertCSVtoArrayList() throws IOException {
        // Ensure folders and files exist before reading
        ensureCSVExists(csvPath);
        ArrayList<UserStats> statsList = new ArrayList<>();
        // Read CSV line by line
        try(BufferedReader br = new  BufferedReader(new FileReader(csvPath.toFile()))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] stats = line.split(",");
                if(stats.length < minStatsCSVDataItems) throw new
IllegalArgumentException("Corrupted Row: " + line);
                String email = stats[0];
                int totalXP = Integer.parseInt(stats[1]);
                int totalSessions = Integer.parseInt(stats[2]);
                int totalCorrect = Integer.parseInt(stats[3]);
                int totalIncorrect = Integer.parseInt(stats[4]);
                long totalStudyTime = Long.parseLong((stats[5]));
                int currentStreak = Integer.parseInt(stats[6]);
                int longestStreak = Integer.parseInt(stats[7]);
                String lastStudyDate = stats[8];
                int consecutiveCorrect = Integer.parseInt(stats[9]);
                int totalRetries = Integer.parseInt(stats[10]);
                ArrayList<String> achievements = new ArrayList<>();
                // After this achievements are listed
                if(stats.length > 11) {
                    achievements = new ArrayList<>(
                            Arrays.asList(Arrays.copyOfRange(stats, 11, stats.length))
                    );
                }
                UserStats s = new UserStats(email, totalXP, totalSessions, totalCorrect,
                        totalIncorrect, totalStudyTime, currentStreak, longestStreak,
```

```
                        lastStudyDate, consecutiveCorrect, totalRetries, achievements);
                statsList.add(s);
            }
        }catch(Exception e) {
            quarantineCorruptedCSV(e, csvPath);
            return new ArrayList<>();
        }
        return statsList;
    }
    // Rewrite stats.csv with updated data
    private void rewriteCSV(ArrayList<UserStats> statsList) throws IOException {
        try(BufferedWriter bw = new BufferedWriter(new FileWriter(csvPath.toFile()))) {
            for(UserStats s : statsList) {
                bw.write(toCSVRow(s));
                bw.newLine();
            }
        }
    }
    // Convert user stats to a csv row for writing
    private String toCSVRow(UserStats s){
        StringBuilder line = new StringBuilder(s.getEmail() + "," + s.getTotalXP() + "," +
s.getTotalSessions() + ","
                + s.getTotalCorrect() + "," + s.getTotalIncorrect() + ","
                + s.getTotalStudyTime() + "," + s.getCurrentStreak() + ","
                + s.getLongestStreak() + "," + s.getLastStudyDate() + "," +
s.getConsecutiveCorrect()
                + "," + s.getTotalRetries());
        for(String achievement : s.getUnlockedAchievements()){
            line.append(",").append(achievement);
        }
        return line.toString();
    }

    // Finds the user stats for a given email, if not find return default
    public UserStats loadUserStats(String email) throws IOException {
        ArrayList<UserStats> statsList = convertCSVtoArrayList();
        for(UserStats s : statsList) {
            if(s.getEmail().equals(email)) return s;
        }
        // If not found
        return new UserStats(email);
    }
    // is user stats saved?
    public boolean isUserStatsSaved(String email) throws IOException {
        ArrayList<UserStats> statsList = convertCSVtoArrayList();
        for(UserStats s : statsList) {
            if(s.getEmail().equals(email)) return true;
        }
        return false;
    }
    // Save updated stats for a user
    public void saveUserStats(UserStats updated) throws IOException {
        ArrayList<UserStats> statsList = convertCSVtoArrayList();
        boolean found = false;
        for(UserStats s : statsList) {
            if(s.getEmail().equals(updated.getEmail())){
                // Replace with updated stats
                statsList.set(statsList.indexOf(s), updated);
                found = true;
                break;
            }
        }
        // If not found, add to list
        if(!found) statsList.add(updated);
        rewriteCSV(statsList);
    }
    // Create default stats for user on registration
    public void createDefaultUserStats(String email) throws IOException {
        saveUserStats(new UserStats(email));
    }
    // Update all stats after a session and returns the updated stats
    public UserStats updateStats(String email, Session session,
                                 int xpGained, boolean isRetry) throws IOException {
        UserStats stats = loadUserStats(email);
        // Add XP earned
        stats.addXP(xpGained);
        // Increment session count
        stats.addSession();
        // Add correct and incorrect counts from the session
        stats.addCorrect(session.getCorrectCount());
        stats.addIncorrect(session.getIncorrectCount());
        // Add elapsed time
```

```
        stats.addStudyTime(session.getElapsedTime());
        // If retried, increment the counter
        if(isRetry) stats.addRetry();
        // Update the consecutive streak
        if(stats.getConsecutiveCorrect() < session.getConsecutiveCorrect())
            stats.setConsecutiveCorrect(session.getConsecutiveCorrect());
        // Update streak
        updateDayStreak(stats);
        // Update achievements
        AchievementController achC = new AchievementController();
        ArrayList<String> updatedAchievements = achC.checkForAchievements(stats, session,
isRetry);
        saveUserStats(stats);
        return stats;
    }
    // Update the day streak
    private void updateDayStreak(UserStats stats){
        String today = LocalDate.now().format(dtf);
        String last = stats.getLastStudyDate();
        if(last == null || last.isBlank()){
            stats.setCurrentStreak(1);
        }else if (!last.equals(today)) {
            // Consecutive days
            LocalDate lastDate = LocalDate.parse(last, dtf);
            if(lastDate.plusDays(1).equals(LocalDate.now())){
                stats.setCurrentStreak(stats.getCurrentStreak() + 1);
            }else{
                // New streak
                stats.setCurrentStreak(1);
            }
        } else {
            // no change
        }
        if(stats.getCurrentStreak() > stats.getLongestStreak()){
            stats.setLongestStreak(stats.getCurrentStreak());
        }
        // Update the day
        stats.setLastStudyDate(today);
    }

    // Function to return the path where individual Session data is stored for user analytics
    private Path sessionPath(String email){
        return Paths.get(System.getProperty("user.dir"), "files", "sessions", email + ".csv");
    }

    // Saving a session to the user's session history CSV
    public void saveSessionRecord(String email, SessionRecord sr) throws IOException {
        Path sessionPath = sessionPath(email);
        // Create directory and folder if not there already
        Files.createDirectories(sessionPath.getParent());
        if(!Files.exists(sessionPath)) Files.createFile(sessionPath);
        // Join the answer history as Ts and Fs to represent true and false
        StringBuilder answers = new StringBuilder();
        for(boolean b : sr.answerHistory()){answers.append(b ? "T" : "F");}
        // Write it into the csv
        try(BufferedWriter bw = new BufferedWriter(new FileWriter(sessionPath.toFile(), true))) {
            bw.write(sr.date() + "," +
                    sr.accuracy() + "," +
                    sr.studySeconds() + "," +
                    sr.correct() + "," +
                    sr.incorrect() + "," +
                    sr.deckName() + "," +
                    answers);
            bw.newLine();
        }
    }

    // Load all session records for a user from their session csv
    public ArrayList<SessionRecord> loadSessionRecords(String email) throws IOException {
        Path sessionPath = sessionPath(email);
        ArrayList<SessionRecord> historicalSessions = new ArrayList<>();
        if(!Files.exists(sessionPath)) return historicalSessions; // no sessions yet
        // Read data line by line
        try(BufferedReader br = new BufferedReader(new FileReader(sessionPath.toFile()))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] data = line.split(",");
                // Not enough data in a row
                if(data.length < 7) throw new IllegalArgumentException("Corrupted Row: " + line);
                ArrayList<Boolean> answerHistory = new ArrayList<>();
                // Reconvert Ts and Fs into trues and falses
                for(char c : data[6].toCharArray()){
```

```
                    answerHistory.add(c == 'T');
                }
                // Add new SessionRecord object to ArrayList
                historicalSessions.add(new SessionRecord(data[0], Double.parseDouble(data[1]),
                        Long.parseLong(data[2]), Integer.parseInt(data[3]),
Integer.parseInt(data[4]),
                        data[5], answerHistory));
            }
        } catch (Exception e) {
            quarantineCorruptedCSV(e, sessionPath);
            return new ArrayList<>();
        }
        return historicalSessions;
    }

    // Procedure to update the email within the file when the user changes their email
    public void updateEmail(String oldEmail, String newEmail) throws IOException {
        // load all users
        ArrayList<UserStats> statsList = convertCSVtoArrayList();
        for(UserStats stats : statsList){
            if(stats.getEmail().equals(oldEmail)){
                // change the email
                stats.setEmail(newEmail);
                break;
            }
        }
        // rewrite the file
        rewriteCSV(statsList);
    }
}
```

5.3.8 XPController.java

```
package Controllers;
import Models.Session;

public class XPController {
    // XP rules:
    // 1 hour = 500xp
    // 90%+ accuracy = 100xp
    // Correct = 15xp, Incorrect = 5xp
    // Constants
    private static final int correct = 15;
    private static final int incorrect = 5;
    private static final int hourlyRate = 500;
    private static final int bonus = 100;

    // Function which returns the xp from a session
    public int calculateXP(Session session){
        int xp = 0;
        // Per answer
        xp += session.getCorrectCount() * correct;
        xp += session.getIncorrectCount() * incorrect;
        // Time XP, 500 per hour
        xp += (int) Math.round(((session.getElapsedTime() / 3600.0) * hourlyRate));
        // Accuracy bonus
        if(session.getAccuracy() >= 90.0) xp += bonus;
        return xp;
    }
}
```

5.3.9 AchievementController.java

```
package Controllers;

import Models.Session;
import Models.UserStats;

import java.util.ArrayList;

public class AchievementController {

    // Return list of newly unlocked achievements after a session
    public ArrayList<String> checkForAchievements(UserStats stats, Session session, boolean
isRetrySession){
        ArrayList<String> newAchievements = new ArrayList<>();
        // Correct streak achievements
        // 5 in a row
        tryUnlock("Sharp Mind", stats,
                stats.getConsecutiveCorrect() >= 5, newAchievements);
```

```
        // 10 in a row
        tryUnlock("Perfect Run", stats,
                stats.getConsecutiveCorrect() >= 10, newAchievements);
        // Accuracy
        tryUnlock("Flawless", stats,
                session.getAccuracy() == 100.0, newAchievements);
        // All time answer achievements
        // 10 flashcards reviewed
        tryUnlock("Getting Started", stats,
                stats.getTotalCorrect() + stats.getTotalIncorrect() >= 10, newAchievements);
        // 100 correct
        tryUnlock("Knowledge Expert", stats,
                stats.getTotalCorrect() >= 100, newAchievements);
        // 100 flashcards reviewed
        tryUnlock("100 Club", stats,
                stats.getTotalCorrect() + stats.getTotalIncorrect() >= 100, newAchievements);
        // Session count achievements
        // 5 sessions
        tryUnlock("Revision Rookie", stats,
                stats.getTotalSessions() >= 5, newAchievements);
        // 50 sessions
        tryUnlock("Revision Master", stats,
                stats.getTotalSessions() >= 50, newAchievements);
        // Streaks
        // 3 days in a row
        tryUnlock("Study Streak", stats,
                stats.getCurrentStreak() >= 3, newAchievements);
        // 7 days in a row
        tryUnlock("Dedicated Learner", stats,
                stats.getCurrentStreak() >= 7, newAchievements);
        // 30 days in revision total
        tryUnlock("Revision Veteran", stats,
                stats.getTotalStudyTime() >= 30*24*3600, newAchievements);
        // Retrying achievements
        // Retry a session
        tryUnlock("Keep Trying", stats,
                isRetrySession, newAchievements);
        // Retry 10 sessions
        tryUnlock("Never Give Up", stats,
                stats.getTotalRetries() >= 10, newAchievements);
        // If user has unlocked all achievements
        tryUnlock("Ultimate Scholar", stats,
                stats.getUnlockedAchievements().size() >= 13 , newAchievements);
        return newAchievements;
    }


    // Unlocks an achievement if the conditions have been met and not already unlocked
    private void tryUnlock(String achievement, UserStats stats,
                           boolean condition, ArrayList<String> unlockedAchievements) {
        if(condition && !stats.isAchievementUnlocked(achievement)) {
            stats.addAchievement(achievement);
            unlockedAchievements.add(achievement);
        }
    }
}
```

5.4 Models

5.4.1 User.java

```
package Models;

// Represents a user account in the system
public class User {
    // Models.User's email address, used for login
    private String email;
    // Hashed version of passwor, cannot store pw as plain text
    private String hashedPassword;
    // Salt used during password hashing
    private String saltBase64;
    // Stores user's last login time as a string
    private String lastLogin;
    // Word used for account recovery
    private String recoveryWord;
    // Constructor initialises all user fields when a Models.User object is created
    public User(String email, String hashedPassword, String saltBase64, String date, String
recoveryWord){
        this.email = email;
```

```
        this.hashedPassword = hashedPassword;
        this.saltBase64 = saltBase64;
        this.lastLogin = date;
        this.recoveryWord = recoveryWord;
    }
    // Getters and Setters for attributes
    public void setEmail(String email){
        this.email = email;
    }

    public void setPasswordHash(String passwordHash){
        this.hashedPassword = passwordHash;
    }
    public void setSaltBase64(String salt){
        this.saltBase64 = salt;
    }
    public void setLastLogin(String lastLogin){
        this.lastLogin = lastLogin;
    }
    public void setRecoveryWord(String recoveryWord){
        this.recoveryWord = recoveryWord;
    }

    public String getEmail(){
        return this.email;
    }
    public String getHashedPassword(){
        return this.hashedPassword;
    }
    public String getSaltBase64() {
        return saltBase64;
    }
    public String getLastLogin(){
        return this.lastLogin;
    }
    public String getRecoveryWord(){
        return this.recoveryWord;
    }
}
```

5.4.2 Settings.java

```
package Models;

public class Settings {
    // to identify user
    private String email;
    // Stylesheet
    private String theme;
    // Accessibility
    private double darkness; // use a stack pane overlay rectangle and change opacity
    // Audio
    private double ttsVolume;
    private double sfxVolume;
    private boolean muted;
    private boolean ttsEnabled;

    // Default settings written into settings.csv when user signs up
    public Settings() {
        // Stylesheet default
        this.theme = "dark";
        // Accessibility
        this.darkness = 0.0;
        // Audio
        this.ttsVolume = 0.8;
        this.sfxVolume = 0.6;
        this.muted = false;
        this.ttsEnabled = true;
    }
    public Settings(String email, String theme, double darkness, double ttsVolume, double
sfxVolume, boolean muted, boolean ttsEnabled) {
        this.email = email;
        // Stylesheet default
        this.theme = theme;
        // Accessibility
        this.darkness = darkness;
        // Audio
        this.ttsVolume = ttsVolume;
        this.sfxVolume = sfxVolume;
        this.muted = muted;
        this.ttsEnabled = ttsEnabled;
```

```
    }
    // Making a copy of settings for drafts
    public Settings copy(){
        Settings s = new Settings();
        s.setEmail(this.email);
        s.setTheme(this.theme);
        s.setDarkness(this.darkness);
        s.setTTSVolume(this.ttsVolume);
        s.setSFXVolume(this.sfxVolume);
        s.setMuted(this.muted);
        s.setTTSEnabled(this.ttsEnabled);
        return s;
    }


    // Will set the value to the minimum or maximum value if outside the range
    private double clamp(double v, double min, double max) {
        return Math.max(min, Math.min(max, v));
    }

    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    //  Stylesheet
    public String getTheme() { return theme; }
    public void setTheme(String theme) {
        if (theme == null || theme.isBlank()) {this.theme = "dark";}
        this.theme = theme;
    }

    // Accessibility
    public double getDarkness() { return darkness; }
    public void setDarkness(double darkness) {
        this.darkness = clamp(darkness, 0.0, 0.8);
    }

    // Audio
    public double getTTSVolume() { return ttsVolume; }
    public void setTTSVolume(double ttsVolume) {
        this.ttsVolume = clamp(ttsVolume, 0.0, 1.0);
    }

    public double getSFXVolume() { return sfxVolume; }
    public void setSFXVolume(double sfxVolume) {
        this.sfxVolume = clamp(sfxVolume, 0.0, 1.0);
    }

    public boolean isMuted() { return muted; }
    public void setMuted(boolean muted) { this.muted = muted; }

    public boolean isTTSEnabled() { return ttsEnabled; }
    public void setTTSEnabled(boolean ttsEnabled) { this.ttsEnabled = ttsEnabled; }
}
```

5.4.3 Stack.java

```
package Models;
import java.util.ArrayList;

// Stack with a Type parameter, so I can use the same class for separate data types
public class Stack<T> {
    // Array List due to its mutability;
    private final ArrayList<T> stackData;
    // Pointer pointing to the last element added
    private int top;

    // Constructor for creating the stack
    public Stack(){
        stackData = new ArrayList<>();
        top = -1;
    }

    // Push Method to add an item at the top of the stack
    public void push(T item){
        stackData.add(item);
        top++;
    }
```

```
    // Pop method to remove and return the top item
    public T pop(){
        if(!isEmpty()){
            T item = stackData.get(top);
            top--;
            return item;
        }
        return null;
    }

    // Peek method allows user to see what item was last added
    public T peek(){
        if(!isEmpty()){
            return stackData.get(top);
        }
        return null;
    }

    // Function to see if the stack isEmpty
    public boolean isEmpty(){
        return top == -1 || stackData.isEmpty();
    }

    // Function returning the size of the stack
    public int size(){
        return stackData.size();
    }

    // Function to clear the stack
    public void clear(){
        top = -1;
        stackData.clear();
    }
}
```

5.4.4 Flashcard.java

```
package Models;

import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;
// Class to represent a flashcard
// Using StringProperty so data can be observed and bound to table
public class Flashcard {
    // JavaFX property representing the front of card
    private final SimpleStringProperty question;
    // JavaFX property representing the back of card
    private final SimpleStringProperty answer;

    // Constructor to initialise a flashcard
    public Flashcard(String question, String answer){
        this.question = new SimpleStringProperty(question);
        this.answer = new SimpleStringProperty(answer);
    }
    // returns JavaFX property for the question and answer for table binding
    public StringProperty questionProperty() { return question; }
    public StringProperty answerProperty() { return answer; }

    // Setters
    public void setAnswer(String answer) {
        this.answer.set(answer);
    }

    public void setQuestion(String question) {
        this.question.set(question);
    }

    // Getters
    public String getQuestion() {
        return question.get();
    }

    public String getAnswer() {
        return answer.get();
    }
}
```

5.4.5 Session.java

```
package Models;

import java.time.LocalDateTime;
import java.util.ArrayList;


public class Session {
    private final String selectedDeck;
    private final String studyMode;
    private final ArrayList<Flashcard> currentQueue;
    private final ArrayList<Flashcard> correctList;
    private final ArrayList<Flashcard> incorrectList;
    private boolean showingAnswer;
    private long elapsedTime;
    private LocalDateTime date;
    private int consecutiveCorrect;
    private final ArrayList<Boolean> answerHistory = new ArrayList<>();

    // New session
    public Session(ArrayList<Flashcard> cards, String selectedDeck, String studyMode){
        this.selectedDeck = selectedDeck;
        this.studyMode = studyMode;
        this.currentQueue = new ArrayList<>(cards);
        this.correctList = new ArrayList<>();
        this.incorrectList = new ArrayList<>();
        this.elapsedTime = 0;
        this.showingAnswer = false;
        this.date = LocalDateTime.now();
        consecutiveCorrect = 0;
    }

    // Returns true if session is complete
    public boolean recordAnswer(boolean correct){
        answerHistory.add(correct);
        // Remove flashcard from active queue
        Flashcard current = currentQueue.removeFirst();
        // add it to correct list if correct
        if(correct) correctList.add(current);
        else{
            // Add to incorrect list, and place at end of queue again
            incorrectList.add(current);
            currentQueue.add(current);
        }
        // Set to question side for next flashcard
        showingAnswer = false;
        // Return true if the session is complete
        return currentQueue.isEmpty();
    }

    // Getters and Setters
    public String getSelectedDeck() {
        return selectedDeck;
    }
    public String getStudyMode() {
        return studyMode;
    }
    public int getIncorrectCount() {
        return incorrectList.size();
    }
    public int getCorrectCount() {
        return correctList.size();
    }
    public long getElapsedTime() {
        return elapsedTime;
    }
    public ArrayList<Boolean> getAnswerHistory() {
        return answerHistory;
    }
    public boolean isShowingAnswer() {
        return showingAnswer;
    }
    public LocalDateTime getDate() {
        return date;
    }
    public ArrayList<Flashcard> getCurrentQueue() {
        return currentQueue;
    }
    public Flashcard getCurrentCard(){
        return currentQueue.getFirst();
```

```
    }
    public int getRemainingCount(){
        return currentQueue.size();
    }
    public int getConsecutiveCorrect() {
        return consecutiveCorrect;
    }
    public double getAccuracy(){
        int total = correctList.size() + incorrectList.size();
        return (double) (correctList.size() * 100) / total;
    }
    public String getProgressText(){
        return "Remaining: " + currentQueue.size() + "\n Correct: " + correctList.size();
    }

    public void setShowingAnswer(boolean showingAnswer){
        this.showingAnswer = showingAnswer;
    }

    public void tick(){elapsedTime++;}

    public void incrementConsecutiveCorrect(){
        consecutiveCorrect++;
    }
    public void resetConsecutiveCorrect(){
        consecutiveCorrect = 0;
    }
    public void setDate(LocalDateTime date) {
        this.date = date;
    }
    // Method to restart the session with same card
    public void reset(ArrayList<Flashcard> lastCards) {
        // Clear the current cards
        currentQueue.clear();
        // add the last cards
        currentQueue.addAll(lastCards);
        // set to question side
        showingAnswer = false;
    }

}
```

5.4.6 UsersStats.java

```
package Models;

import java.util.ArrayList;

public class UserStats {
    private String email;
    private int totalXP;
    private int totalSessions;
    private int totalCorrect;
    private int totalIncorrect;
    private long totalStudyTime;
    private int currentStreak;
    private int longestStreak;
    public String lastStudyDate;
    private int consecutiveCorrect;
    private int totalRetries;
    private ArrayList<String> unlockedAchievements;

    // Constructors for new users
    public UserStats(String email) {
        this.email = email;
        this.totalXP = 0;
        this.totalSessions = 0;
        this.totalCorrect = 0;
        this.totalIncorrect = 0;
        this.totalStudyTime = 0;
        this.currentStreak = 0;
        this.longestStreak = 0;
        this.lastStudyDate = "";
        this.consecutiveCorrect = 0;
        this.totalRetries = 0;
        this.unlockedAchievements = new ArrayList<>();
    }
    // Constructor for existing users
    public UserStats(String email, int totalXP, int totalSessions, int totalCorrect,
                     int totalIncorrect, long totalStudyTime, int currentStreak,
                     int longestStreak, String lastStudyDate, int consecutiveCorrect,
```

```
                    int totalRetries, ArrayList<String> unlockedAchievements) {
        this.email = email;
        this.totalXP = totalXP;
        this.totalSessions = totalSessions;
        this.totalCorrect = totalCorrect;
        this.totalIncorrect = totalIncorrect;
        this.totalStudyTime = totalStudyTime;
        this.currentStreak = currentStreak;
        this.longestStreak = longestStreak;
        this.lastStudyDate = lastStudyDate;
        this.consecutiveCorrect = consecutiveCorrect;
        this.totalRetries = totalRetries;
        this.unlockedAchievements = unlockedAchievements;
    }
    // Helper Methods
    public void addXP(int xp) {
        this.totalXP += xp;
    }
    public void addSession(){
        this.totalSessions++;
    }
    public void addCorrect(int correct) {
        this.totalCorrect+=correct;
    }
    public void addIncorrect(int incorrect) {
        this.totalIncorrect+=incorrect;
    }
    public void addStudyTime(long st) {
        this.totalStudyTime += st;
    }
    public void addRetry(){
        this.totalRetries++;
    }
    public boolean isAchievementUnlocked(String achievement){
        return this.unlockedAchievements.contains(achievement);
    }
    public void addAchievement(String achievement){
        if(!isAchievementUnlocked(achievement)) this.unlockedAchievements.add(achievement);
    }
    // Get the user's current level
    public int getLevel(){
        // Level N needs N*100 XP
        int level = 1;
        while(true){
            // Sum to n formula
            int xpForNext = (level * (level+1) / 2) * 100;
            if(totalXP < xpForNext) break;
            level++;
        }
        return level;
    }
    public int getXPForNext(){
        int level = getLevel();
        return (level + 1) * 100;
    }
    // How much xp user has earned in their current level
    public int getXPIntoCurrentLevel(){
        int level = getLevel();
        int xp = 0;
        // Calculate total xp required to reach current level
        for(int i = 0; i < level; i++){
            xp += i*100;
        }
        // How much xp user has got in their current level
        return totalXP-xp;
    }


    // Getters
    public String getEmail() {
        return email;
    }
    public int getTotalXP() {
        return totalXP;
    }
    public int getTotalSessions() {
        return totalSessions;
    }
    public int getTotalCorrect() {
        return totalCorrect;
    }
    public int getTotalIncorrect() {
```

```
        return totalIncorrect;
    }
    public long getTotalStudyTime() {
        return totalStudyTime;
    }
    public int getLongestStreak() {
        return longestStreak;
    }
    public int getCurrentStreak() {
        return currentStreak;
    }
    public String getLastStudyDate() {return lastStudyDate;}
    public int getConsecutiveCorrect() {
        return consecutiveCorrect;
    }
    public int getTotalRetries() {
        return totalRetries;
    }
    public ArrayList<String> getUnlockedAchievements() {
        return unlockedAchievements;
    }

    // Setters
    public void setEmail(String email) {
        this.email = email;
    }
    public void setTotalXP(int totalXP) {
        this.totalXP = totalXP;
    }
    public void setTotalSessions(int totalSessions) {
        this.totalSessions = totalSessions;
    }
    public void setTotalCorrect(int totalCorrect) {
        this.totalCorrect = totalCorrect;
    }
    public void setTotalIncorrect(int totalIncorrect) {
        this.totalIncorrect = totalIncorrect;
    }
    public void setTotalStudyTime(long totalStudyTime) {
        this.totalStudyTime = totalStudyTime;
    }
    public void setLastStudyDate(String lastDate) {this.lastStudyDate = lastDate;}
    public void setLongestStreak(int longestStreak) {
        this.longestStreak = longestStreak;
    }
    public void setCurrentStreak(int currentStreak) {
        this.currentStreak = currentStreak;
    }
    public void setConsecutiveCorrect(int consecutiveCorrect) {
        this.consecutiveCorrect = consecutiveCorrect;
    }
    public void setTotalRetries(int totalRetries) {
        this.totalRetries = totalRetries;
    }
    public void setUnlockedAchievements(ArrayList<String> unlockedAchievements) {
        this.unlockedAchievements = unlockedAchievements;
    }

}
```

5.5 Components

5.5.1 UIComponents.java

```
package Components;

import RecordClasses.PasswordBundle;
import RecordClasses.RectangleButtonBundle;
import RecordClasses.TextAreaBundle;
import RecordClasses.TextBundle;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Cursor;
import javafx.scene.Group;
import javafx.scene.Node;
import javafx.scene.control.*;
import javafx.scene.image.ImageView;
import javafx.scene.layout.StackPane;
import javafx.scene.shape.Rectangle;
```

```
import javafx.scene.text.Text;


import java.util.Objects;
import java.util.function.Predicate;

public class UIComponents {
    // Constants
    private static final int passwordRectH = 98;
    private static final int lockIconW = 98;
    private static final int lockIconH = 88;
    private static final int brImageX = 1820;
    private static final int brImageY = 957;
    private static final int logoX = 0;
    private static final int logoY = 0;
    private static final int invalidPLength = 6;
    private static final int hintW = 95;
    private static final int hintH = 95;
    // Creates a password bundle with show/hide functionality
    public PasswordBundle buildPasswordInput(int rectX, int rectY, int rectW, int fieldX, int
fieldY, String promptText, int lIconX, int lIconY, int rIconX, int rIconY){
        // Create background rectangle for password input
        Rectangle passwordInpRect = new Rectangle(rectX, rectY, rectW, passwordRectH);
        passwordInpRect.getStyleClass().add("inputRect");
        // Create hidden password field
        PasswordField password = new PasswordField();
        password.setPromptText(promptText);
        password.setLayoutX(fieldX);
        password.setLayoutY(fieldY);
        // Create visible text field that mirrors password field (for show/hide functionality)
        TextField copyPw = new TextField();
        copyPw.setLayoutX(fieldX);
        copyPw.setLayoutY(fieldY);
        copyPw.setVisible(false);
        copyPw.setManaged(false);
        // Create lock icon
        ImageView lock = new
ImageView(Objects.requireNonNull(getClass().getResource("/images/AuthScreen/lock.png")).toExterna
lForm());
        lock.setLayoutX(lIconX);
        lock.setLayoutY(lIconY);
        lock.setFitHeight(lockIconH);
        lock.setFitWidth(lockIconW);
        // Bind text fields together so they stay synchronized
        copyPw.textProperty().bindBidirectional(password.textProperty());
        // Create show/hide password icons
        ImageView passwordShow = new
ImageView(Objects.requireNonNull(getClass().getResource("/images/AuthScreen/passwordShow.png")).t
oExternalForm());
        ImageView passwordHide = new
ImageView(Objects.requireNonNull(getClass().getResource("/images/AuthScreen/passwordHide.png")).t
oExternalForm());

        passwordShow.setLayoutX(rIconX);
        passwordShow.setLayoutY(rIconY);
        passwordShow.setVisible(true);
        passwordShow.getStyleClass().add("passwordImage");

        passwordHide.setLayoutX(rIconX);
        passwordHide.setLayoutY(rIconY);
        passwordHide.setVisible(false);
        passwordHide.getStyleClass().add("passwordImage");
        // Set up click handlers to toggle password visibility
        passwordShow.setOnMouseClicked(_ -> showPassword(passwordHide, passwordShow, copyPw,
password));
        passwordHide.setOnMouseClicked(_ -> hidePassword(passwordHide, passwordShow, copyPw,
password));

        Group inputs = new Group(passwordInpRect, password, copyPw);

        return new PasswordBundle(inputs, password, copyPw, passwordShow, passwordHide, lock);
    }

    // Creates a text bundle with an icon
    public TextBundle buildTextFieldInput(int rectX, int rectY, int rectW, int rectH, int fieldX,
int fieldY,
                                          String promptText, String iconURL, int iconX, int
iconY, int iconW, int iconH){
        // Create background rectangle for text input
        Rectangle inputRect = new Rectangle(rectX, rectY, rectW, rectH);
        inputRect.getStyleClass().add("inputRect");
        // Create text field
```

```
        TextField tf = new TextField();
        tf.setPromptText(promptText);
        tf.setLayoutX(fieldX);
        tf.setLayoutY(fieldY);
        tf.setContextMenu(createContextMenu(tf));
        // Create icon
        ImageView icon = new ImageView(iconURL);
        icon.setLayoutX(iconX);
        icon.setLayoutY(iconY);
        icon.setFitHeight(iconH);
        icon.setFitWidth(iconW);
        Group tfInputs = new Group(inputRect, tf);
        return new TextBundle(tfInputs, tf, icon);
    }

    // Creates a rectangular button with text
    public RectangleButtonBundle buildRectangularButtonBundle(int rectW, int rectH, String text,
String typeOfText, int StackPaneX, int StackPaneY) {
        Rectangle buttonRect = new Rectangle(rectW, rectH);
        Text buttonText = new Text(text);
        buttonText.getStyleClass().add(typeOfText);
        StackPane button = new StackPane(buttonRect, buttonText);
        button.getStyleClass().add("StackPane-Button");
        button.setLayoutX(StackPaneX);
        button.setLayoutY(StackPaneY);
        return new RectangleButtonBundle(buttonRect, buttonText, button);
    }

    // Creates a text area with prompt text and a character counter
    public TextAreaBundle createTextArea(String promptText, int charLimit, int lowerLimit) {
        TextArea textArea = new TextArea();
        textArea.getStyleClass().add("textArea");
        textArea.setPromptText(promptText);
        Label characterCounter = new Label("0 / " + charLimit);
        characterCounter.getStyleClass().add("counter");
        // Initial Colour
        characterCounter.setStyle("-fx-text-fill: red;");
        textArea.textProperty().addListener((_, _, newValue) -> {
            // Length when white space is removed
            int trimmedLength = newValue.trim().length();
            // Change the number
            characterCounter.setText(trimmedLength + " / " + charLimit);
            // Change label colours depending on limits
            if (trimmedLength < lowerLimit || trimmedLength > charLimit) {
                characterCounter.setStyle("-fx-text-fill: red;");
            }else{
                characterCounter.setStyle("-fx-text-fill: green;");
            }
        });

        return new TextAreaBundle(textArea, characterCounter, charLimit, lowerLimit);
    }

    // Creates logo image view for easy reuse
    public ImageView createLogoView() {
        ImageView logoView = new
ImageView(Objects.requireNonNull(getClass().getResource("/images/Constant/logo.png")).toExternalF
orm());
        logoView.getStyleClass().add("constantImages");
        logoView.setLayoutX(logoX);
        logoView.setLayoutY(logoY);
        return logoView;
    }

    // Creates an image view positioned in the bottom right of the page
    public ImageView createBRImageView(String name){
        ImageView iv = new
ImageView(Objects.requireNonNull(getClass().getResource("/images/Constant/" + name +
".png")).toExternalForm());
        iv.getStyleClass().add("constantImages");
        iv.setLayoutX(brImageX);
        iv.setLayoutY(brImageY);
        iv.setCursor(Cursor.HAND);
        return iv;
    }

    // Hides the password
    private void hidePassword(ImageView passwordHide, ImageView passwordShow, TextField copyPw,
PasswordField password) {
        passwordHide.setVisible(false);
        passwordShow.setVisible(true);
        copyPw.setManaged(false);
```

```
        copyPw.setVisible(false);
        password.setVisible(true);
        password.setManaged(true);
    }
    // Shows the password
    private void showPassword(ImageView passwordHide, ImageView passwordShow, TextField copyPw,
PasswordField password) {
        passwordHide.setVisible(true);
        passwordShow.setVisible(false);
        copyPw.setManaged(true);
        copyPw.setVisible(true);
        password.setVisible(false);
        password.setManaged(false);
    }
    private ContextMenu createContextMenu(TextField tf){
        MenuItem cut = new MenuItem("Cut");
        cut.setOnAction(_ -> tf.cut());

        MenuItem copy = new MenuItem("Copy");
        copy.setOnAction(_ -> tf.copy());

        MenuItem paste = new MenuItem("Paste");
        paste.setOnAction(_ -> tf.paste());

        ContextMenu cm = new ContextMenu(cut, copy, paste);
        cm.getStyleClass().add("context-menu");

        return cm;
    }

    // Adds a real-time validator to input fields and error styling validation fields.
    public static void attachValidator(TextInputControl field, Predicate<String> rule, String
errorClass, TextInputControl... linkedField) {
        field.textProperty().addListener((_, _, newText) -> {
            boolean invalid = !rule.test(newText);
            updateStyle(field, errorClass, invalid);
            // Apply same styling to linked fields (such as visible copy of password field)
            for(TextInputControl f : linkedField){
                updateStyle(f, errorClass, invalid);
            }
        });
    }
    // Updates CSS styling of field depending on validation state
    public static void updateStyle(TextInputControl field, String errorClass, boolean invalid) {
        field.getStyleClass().remove(errorClass);
        if (invalid && !field.getText().isEmpty()) {
            field.getStyleClass().add(errorClass);
        }
    }

    // Validates email format via regex
    public static boolean isValidEmail(String email){
        if (email == null) return false;
        return email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    }
    // Validates strength of password
    public static boolean isValidPassword(String pw){
        if(pw == null) return false;
        return pw.length()>= invalidPLength && pw.matches(".*[^a-zA-Z0-9].*");
    }
    // Makes sure recovery is something rememberable
    public static boolean isValidRecoveryW(String recoveryWord){
        return recoveryWord != null && recoveryWord.length() >= 3;
    }

    // Bind Enter Button to submit in Text Fields
    public void bindEnter(EventHandler<ActionEvent> handler, Node... fields) {
        for (Node node : fields) {
            if(node instanceof TextField tf){
                tf.setOnAction(handler);
            }else if(node instanceof PasswordField pf){
                pf.setOnAction(handler);
            }
        }
    }
    // Function to set position of a node
    public void setPosition(Node node, int x, int y){
        node.setLayoutX(x);
        node.setLayoutY(y);
    }

    // Function to return a question mark image view to be used for information
```

```
    public ImageView getHintView(){
        ImageView hint = new ImageView("/images/Constant/question.png");
        hint.setCursor(Cursor.HAND);
        hint.setLayoutX(brImageX);
        hint.setFitHeight(hintH);
        hint.setFitWidth(hintW);
        return hint;
    }
}
```

5.5.2 CustomAlert.java

```
package Components;

import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Window;

import java.util.Objects;
import java.util.Optional;

// Utility class for creating and displaying custom alerts, proving convenient static methods for
common alert types
public class CustomAlert {
    // Constants
    private static final int spacing = 10;
    private static final int minWidth = 410;
    // Constructor not used, all methods are static
    public CustomAlert(){}
    // Displays an error alert dialog
    public static void error(Window alertWindow, String message){
        show(alertWindow, "Error", message, Alert.AlertType.ERROR, ButtonType.OK);
    }
    // Displays an informational alert dialog
    public static void info(Window alertWindow, String message){
        show(alertWindow, "Info", message, Alert.AlertType.INFORMATION, ButtonType.OK);
    }
    // Displays a success confirmation alert dialog
    public static void success(Window alertWindow, String message){
        show(alertWindow, "Success", message, Alert.AlertType.CONFIRMATION, ButtonType.OK);
    }
    // displays a confirmational dialog with OK and Cancel buttons
    public static boolean confirm(Window alertWindow, String title, String message){
        Alert a = build(alertWindow, title, message, Alert.AlertType.INFORMATION,
ButtonType.CANCEL, ButtonType.OK);
        // showAndWait() pauses code until user responds
        // filter checks if OK was clicked
        // isPresent() returns true if OK was chosen
        return a.showAndWait().filter(b -> b == ButtonType.OK).isPresent();
    }
    // Displaying an alert which requires an input
    public static Optional<String> input(Window alertWindow, String title, String message, String
promptText){
        Alert a = build(alertWindow, title, message, Alert.AlertType.CONFIRMATION,
ButtonType.CANCEL, ButtonType.OK);
        TextField input = new TextField();
        input.getStyleClass().add("inputAlert");
        input.setPromptText(promptText);
        Button okButton = (Button) a.getDialogPane().lookupButton(ButtonType.OK);
        okButton.setDisable(true);
        // Only enable when there is text
        input.textProperty().addListener((_, _, newValue) -> {
            // Remove any illegal characters
            String sanitisedInput = newValue.replaceAll("[\\\\/:*?\"<>|\\x00-
\\x1F]","").stripTrailing();
            if(!sanitisedInput.equals(newValue)) input.setText(sanitisedInput);
            okButton.setDisable(sanitisedInput.trim().isEmpty());
        });
        Label messageText = new Label(message);
        VBox content = new VBox(spacing, messageText, input);
        a.getDialogPane().setContent(content);
        // Return text trimmed
        return a.showAndWait().filter(b -> b == ButtonType.OK).map(_ -> input.getText().trim());
    }
    // builds custom styled alert dialog
    private static Alert build(Window alertWindow, String title, String message, Alert.AlertType
alertType, ButtonType... buttons) {
        // create a new alert of specified type
        Alert a = new Alert(alertType);
        // Place alert on main application window, blocking interaction with background
        // If alert window is null, then alert will be independent
```

```
        if(alertWindow!=null){
            a.initOwner(alertWindow);
        }
        // Set title of alert
        a.setTitle(title);
        // Removes default header text
        a.setHeaderText(null);
        // Create a label for the message with text wrapping enabled
        // Allows longer messages across multiple lines
        Label label = new Label(message);
        label.setWrapText(true);
        // Set the label as the content, wrapped in a VBox with 10px spacing
        a.getDialogPane().setContent(new VBox(spacing, label));
        // Set the buttons that will appear at the bottom of the alert
        a.getDialogPane().getButtonTypes().setAll(buttons);
        // Configure minimum size to use preferred size (auto-sizing based on content)
        a.getDialogPane().setMinHeight(Control.USE_PREF_SIZE);
        // Set minimum width to ensure the alert is not too narrow
        a.getDialogPane().setMinWidth(minWidth);
        // Add a CSS class based on the alert type (such as: "error", "information",
"confirmation")
        a.getDialogPane().getStyleClass().add(alertType.name().toLowerCase());
        // Apply custom stylesheet for alert styling
        a.getDialogPane().getStylesheets().add(

Objects.requireNonNull(CustomAlert.class.getResource("/styles/alert.css")).toExternalForm()
        );
        return a;
    }
    // Builds alert and shows it immediately
    private static void show(Window alertWindow, String title, String message, Alert.AlertType
alertType, ButtonType... buttons) {
        // Build the alert and show it, waiting for user response before continuing
        build(alertWindow, title, message, alertType, buttons).showAndWait();
    }
}
```

5.5.3 ContentNavigator.java

```
package Components;

import Models.Stack;
import javafx.scene.Group;
import javafx.scene.Node;

import java.util.ArrayList;
import java.util.List;


public class ContentNavigator {
    private final Stack<List<Node>> history = new Stack<>();

    // Switches content of a layer to new nodes, saving current state to history for backtracking
    public void switchContent(Group contentLayer, Node... newContent){
        // Save current state to history stack
        history.push(new ArrayList<>(contentLayer.getChildren()));
        // Replace all children with new content
        contentLayer.getChildren().setAll(newContent);
    }
    // Navigates back to previous content state
    public void goBack(Group contentLayer){
        if(!history.isEmpty()){
            contentLayer.getChildren().setAll(history.pop());
        }
    }
}
```

5.5.4 ScreenManager.java

```
package Components;

import Controllers.SettingsController;
import Models.Settings;
import Models.Stack;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;
```

```
import java.io.IOException;
import java.util.Objects;


// Utility class for managing scene navigation
public class ScreenManager {
    // Constants
    private static final int sceneW = 1920;
    private static final int sceneH = 1057;
    // Primary stage of application, allows access from anywhere in application
    private static Stage stage;
    // Stack to maintain navigation of history opf scenes.
    private static final Stack<Scene> history = new Stack<>();
    //  sets the primary stage reference for application, called only at startup
    private static SettingsController sC;

    public static void setSC(SettingsController sC) {
        ScreenManager.sC = sC;
    }

    public static void setStage(Stage primaryStage){
        stage = primaryStage;
    }
    // Switches current scene to a new scene and saves current seen to history
    public static void switchScene(Scene root){
        // Check if there's currently a scene being displayed
        if(stage.getScene() != null){
            // Save current scene to stack before switching
            history.push(stage.getScene());
        }

root.getStylesheets().add(Objects.requireNonNull(ScreenManager.class.getResource("/styles/styles.
css")).toExternalForm());

root.getStylesheets().add(Objects.requireNonNull(ScreenManager.class.getResource("/styles/dark.cs
s")).toExternalForm());
        // Switch scene to new scene
        stage.setScene(root);
    }
    public static void switchScene(Scene root, String email) throws IOException {
        // Check if there's currently a scene being displayed
        if(stage.getScene() != null){
            // Save current scene to stack before switching
            history.push(stage.getScene());
        }
        changeStyleSheet(root, email);
        // Switch scene to new scene
        Scene finalScene = sceneWithOverlay(root, email);
        stage.setScene(finalScene);
    }
    // Navigate back to previous scene in history stack
    public static void goBack(){
        // Check if there's any scene to go back to
        if(!history.isEmpty()){
            // Remove and fetch most recent scene from history, setting it as the current scene.
            stage.setScene(history.pop());
        }
    }
    public static void goBack(String email) throws IOException {
        // Check if there's any scene to go back to
        if(!history.isEmpty()){
            // Remove and fetch most recent scene from history, setting it as the current scene.
            Scene nextScene = history.pop();
            changeStyleSheet(nextScene, email);
            if(nextScene.getRoot() instanceof StackPane wrapper){
                for(var node : wrapper.getChildren()){
                    if(node instanceof Rectangle rect && rect.getWidth() == sceneW &&
rect.getHeight() == sceneH){
                        Settings cS = getUserSettings(email);
                        rect.setOpacity(cS.getDarkness());
                        break;
                    }
                }
            }
            stage.setScene(nextScene);
        }
    }
    // Clears all navigation history (You can't navigate back post logout).
    public static void clearHistory(){
        history.clear();
    }
    private static Settings getUserSettings(String email) throws IOException {
```

```
        if (sC != null && sC.getCurrentSettings() != null &&
sC.getCurrentSettings().getEmail().equals(email)) {
            return sC.getCurrentSettings();
        }
        return sC.loadSettings(email);
    }
    // Procedure for applying user settings to each screen (stylesheet specific as of) 08/02/2025
    private static void changeStyleSheet(Scene root, String email) throws IOException {
        Settings userSettings = getUserSettings(email);
        root.getStylesheets().clear();

root.getStylesheets().add(Objects.requireNonNull(ScreenManager.class.getResource("/styles/styles.
css")).toExternalForm());

root.getStylesheets().add(Objects.requireNonNull(ScreenManager.class.getResource("/styles/" +
userSettings.getTheme() + ".css")).toExternalForm());
    }
    // Returns a scene with its content put inside a stack pane alongside the brightness overlay
    private static Scene sceneWithOverlay(Scene scene, String email) throws IOException {
        Parent originalRoot = scene.getRoot();
        StackPane wrapper = new StackPane(originalRoot);
        wrapper.getChildren().add(getBrightnessOverlay(email));
        scene.setRoot(wrapper);
        return scene;
    }
    // Returns a rectangle to act as a brightness overlay
    public static Rectangle getBrightnessOverlay(String email) throws IOException {
        Settings userSettings = getUserSettings(email);
        Rectangle brightnessOverlay = new Rectangle(sceneW, sceneH);
        brightnessOverlay.getStyleClass().add("brightnessOverlay");
        brightnessOverlay.setOpacity(userSettings.getDarkness());
        brightnessOverlay.setMouseTransparent(true);
        return brightnessOverlay;
    }

}
```

5.6 RecordClasses

5.6.1 TextBundle.java

```
package RecordClasses;

import javafx.scene.Group;
import javafx.scene.control.TextField;
import javafx.scene.image.ImageView;

/**
 * Record class that bundles together the components of a text input field with an icon.
 * This bundle contains:
 * @param group: A Group containing the background rectangle and the TextField
 * @param tf: The actual TextField where users enter text
 * @param icon: An ImageView icon displayed alongside the input field (e.g., user icon, email
icon)
 */
public record TextBundle(Group group, TextField tf, ImageView icon) {
}
```

5.6.2 PasswordBundle.java

```
package RecordClasses;

import javafx.scene.Group;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.scene.image.ImageView;

/**
 * This is the most complex bundle record, containing multiple components needed to create a
password field that can toggle between hidden (masked) and visible (plain text) modes.
 * This bundle contains:
 * @param passwordGroup: A Group containing the background rectangle, PasswordField, and
TextField
 * @param password: The PasswordField that displays masked characters (e.g., ••••••)
 * @param copyPw: A TextField that mirrors the password content for "show password" functionality
 * @param showIcon: ImageView icon for showing the password (eye icon typically)
 * @param hideIcon: ImageView icon for hiding the password (crossed-eye icon typically)
 * @param leftIcon: ImageView icon displayed on the left side (typically a lock icon)
 */

public record PasswordBundle(Group passwordGroup, PasswordField password, TextField copyPw,
ImageView showIcon,
                             ImageView hideIcon, ImageView leftIcon) {
    public Group getIconGroup() {
        return new Group(hideIcon, showIcon, leftIcon);
    }
}
```

5.6.3 RectangleButtonBundle.java

```
package RecordClasses;

import javafx.scene.layout.StackPane;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
// Record class that bundles together the components of a rectangular button.
// This bundle contains:
//  - buttonRect: The Rectangle shape that forms the button's background
//  - buttonText: The Text element displayed on the button
//  - button: The StackPane container that holds both the rectangle and text together

/**
 * Record class that bundles together the components of a rectangular button.
 * This bundle contains:
 * @param buttonRect: The Rectangle shape that forms the button's background
 * @param buttonText: The Text element displayed on the button
 * @param button: The StackPane container that holds both the rectangle and text together
 */
public record RectangleButtonBundle(Rectangle buttonRect, Text buttonText, StackPane button) {
}
```

5.6.4 TextAreaBundle.java

```
package RecordClasses;

import javafx.scene.control.Label;
import javafx.scene.control.TextArea;

/**
 * Bundle to create a text area, linked with a character counter
 * This bundle contains:
 * @param textArea: Input field for the user
 * @param characterCounter: the text for the character counter
 * @param charLimit: limit beyond character counter goes red
 * @param lowerLimit: limit of when the characters is less than, the character counter goes red
 */
public record TextAreaBundle(TextArea textArea, Label characterCounter, int charLimit, int
lowerLimit) {
}
```

5.6.5 SessionRecord.java

```
package RecordClasses;

import java.util.ArrayList;
```

```
/**
 * @param date Date of Session to be used for streaks
 * @param accuracy The accuracy, used for graph data
 * @param studySeconds Elapsed time of a session
 * @param correct  Number of correct answers
 * @param incorrect Number of incorrect answers
 * @param deckName The deck name of the session
 * @param answerHistory correct = t, incorrect = f
 */
public record SessionRecord(String date, double accuracy, long studySeconds, int correct, int
incorrect,
                            String deckName, ArrayList<Boolean> answerHistory) {
}
```

5.7 Stylesheets

5.7.1 styles.css

```
/*Images*/
.constantImages{
    -fx-fit-height: 100px;
    -fx-fit-width: 100px;
}
.icons{
    -fx-fit-height: 141px;
    -fx-fit-width: 144px;
}
/*Typography*/
.title{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 120px;
}
.postAuthTitle{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 120px;
}
.screens .text{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 80px;
}
.brightnessOverlay{
    -fx-arc-height: 0px;
    -fx-arc-width: 0px;
    -fx-stroke: transparent;
}
.infoText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 40px;
}
.instructionText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 80px;
}
Rectangle{
    -fx-arc-height: 50px;
    -fx-arc-width: 50px;
}
/*Backgrounds and Rectangles*/
.backgroundRect{
    -fx-arc-height: 100px;
    -fx-arc-width: 100px;
}
.inputRect{
    -fx-fill: #ffffff;
    -fx-stroke: #000000;
}
/*Input Fields*/
.text-field {
    -fx-pref-width: 600px;
    -fx-pref-height: 80px;
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 20px;
}
.loginTextField{
    -fx-pref-width: 370px;
}
.resetTextField{
    -fx-pref-width: 1140px;
```

```
}
.passwordImage{
    -fx-fit-height: 47px;
    -fx-fit-width: 48px;
}
/*Drop Down Menus*/
.context-menu {
    -fx-font-size: 12px;
    -fx-padding: 2px;
}
.context-menu .menu-item {
    -fx-padding: 3px 10px;
}
.context-menu .text,
.context-menu .label {
    -fx-font-size: 12px;
    -fx-text-fill: black;
    -fx-fill: black;
}
/*Buttons*/
.buttonText{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 60px;
}
.smallButtonText{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 40px;
}
.StackPane-Button{
    -fx-cursor: hand;
}
.smallerButtonText{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 20px;
}
.tableText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 12px;
}
.error{
    -fx-text-fill: red;
}
/*Menu Items*/
.menu-item.options {
    -fx-background-color: black;
    -fx-padding: 20 80;
    -fx-pref-height: 98px;
    -fx-alignment: center;
    -fx-pref-width: 1300px;
}
.menu-item.options{
    -fx-text-alignment: center;
}
.menu-item.options:hover {
    -fx-background-color: #222;
}
/*Sliders*/
.sliderInputAlt{
    -fx-pref-width: 90px;
    -fx-pref-height: 50px;
}
Slider{
    -fx-pref-width: 1300px;
    -fx-show-tick-labels: true;
    -fx-show-tick-marks: true;
}
.rateSlider{
    -fx-pref-width: 200px;
    -fx-show-tick-labels: true;
    -fx-show-tick-marks: true;
}
Slider .track{
    -fx-background-color: #444;
    -fx-background-radius: 6px;
    -fx-pref-height: 6px;
    -fx-show-tick-labels: true;
    -fx-show-tick-marks: true;
}
Slider .thumb {
    -fx-background-color:
            black,          /* inner fill */
            white;          /* outer ring */
```

```
    -fx-background-insets:
            0,
            2;
    -fx-background-radius: 50%;
    -fx-pref-width: 16px;
    -fx-pref-height: 16px;
    -fx-cursor: hand;
}
/*Checkboxes*/
.toggles {
    -fx-font-size: 28px;
    -fx-text-fill: black;
}
.toggles .text {
    -fx-fill: black;
}
.toggles .box {
    -fx-background-color: white;
    -fx-border-color: #ccc;
    -fx-border-radius: 3px;
    -fx-background-radius: 3px;
}
/*Text Areas*/
.textArea{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 20px;
    -fx-max-width: 600px;
    -fx-pref-height: 250px;
    -fx-pref-rows: 4;
    -fx-arc-height: 10px;
    -fx-arc-width: 10px;
    -fx-wrap-text: true;
}
/*Flashcard Table */
.flashcardContent{
    -fx-max-width: 600px;
}
.cardTable {
    -fx-pref-width: 1000px;
    -fx-max-height: 510px;
    -fx-wrap-text: true;
}
.cardTable .column-header .label {
    -fx-font-weight: bold;
}

.cardTable .table-cell {
    -fx-padding: 6px 10px;
}
.cardTable .table-row-cell:odd {
    -fx-background-color: #f9f9f9;
}
.cardTable .table-cell .text {
    -fx-font-family: "Vipnagorgialla" !important;
    -fx-font-size: 12px !important;
    -fx-wrap-text: true !important;
    -fx-font-weight: bold;
    -fx-fill: black;
}
.cardTable .column-header .label .text {
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 13px !important;
    -fx-fill: black;
    -fx-font-weight: bold;
    -fx-wrap-text: true;
}
/*Study View*/
.counter .text{
    -fx-font-size: 13px !important;
}
.display{
    -fx-pref-width: 1704px;
    -fx-pref-height: 745px;
}
.folderTreeView{
    -fx-background-color: black;
    -fx-border-color: grey;
    -fx-max-width: 1000px;
    -fx-pref-height: 600px;
}
.folderTreeView .tree-cell .text{
    -fx-font-size: 14px !important;
```

```
    -fx-wrap-text: true !important;
    -fx-fill: black;
}
.folderTreeView .tree-cell:hover {
    -fx-background-color: #f0f0f0;
    -fx-background-radius: 6px;
}
.deckSearch{
    -fx-pref-width: 450px;
}
.fcSearch{
    -fx-pref-width: 850px;
}
.timerLabel{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 36px;
    -fx-text-fill: white;
    -fx-background-color: #111111;
    -fx-padding: 8 20 8 20;
    -fx-background-radius: 12px;
}
.fcRect{
    -fx-fill: white;
    -fx-stroke: #222222;
    -fx-stroke-width: 3px;
    -fx-arc-width: 40px;
    -fx-arc-height: 40px;
}
.fcBRect{
    -fx-fill: #e1f3f3;
    -fx-stroke: #4a90d9;
    -fx-stroke-width: 3px;
    -fx-arc-width: 40px;
    -fx-arc-height: 40px;
}
.cardBody{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 25px;
    -fx-wrapping-width: 1100px;
    -fx-text-alignment: center;
}
.cardLabel{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 22px;
    -fx-fill: grey;
}
.flipH{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 18px;
    -fx-fill: lightgrey;
}
.correctC {
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 48px;
    -fx-fill: green;
}

.incorrectC {
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 48px;
    -fx-fill: red;
}
/*Flashcard Actions*/
.flashcard{
    -fx-pref-width: 1200;
    -fx-pref-height: 450;
    -fx-cursor: hand;
}
.fcButtonRect{
    -fx-arc-width: 40px;
    -fx-arc-height: 40px;
}

.fcButtonText{
    -fx-font-size: 72px;
    -fx-fill: white;
    -fx-font-weight: bold;
}
.fcButton{
    -fx-pref-width: 200px;
    -fx-pref-height: 130px;
    -fx-cursor: hand;
```

```
}
/*Summary*/
.summaryText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 80px;
    -fx-fill: black;
    -fx-padding: 7.21 14.17 7.21 14.17;
}
.XPBar{
    -fx-fill: #444444;
    -fx-arc-width: 20px;
    -fx-arc-height: 20px;
    /*-fx-stroke: black;*/
    /*-fx-stroke-width: 3px;*/
}
.progressFill{
    -fx-fill: #61df61;
    -fx-arc-width: 20px;
    -fx-arc-height: 20px;
}
.levelText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 40px;
    -fx-fill: black;
}
.statText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 30px;
}
.descriptionText{
    -fx-font-family: "Vipnagorgialla";
    -fx-font-size: 18px;
    -fx-fill: #494949;
}
.charts{
    -fx-background-color: transparent;
}
.charts .chart-plot-background{
    -fx-background-color: transparent;
}
.charts .chart-title .text{
    -fx-fill: black;
    -fx-font-size: 24px;
}
.charts .axis-label .text{
    -fx-fill: black;
    -fx-font-size: 18px;
}
.charts .chart-vertical-grid-lines { -fx-stroke: transparent; }
.charts .chart-horizontal-grid-lines { -fx-stroke: #dddddd; }
.block{
    -fx-arc-width: 6px;
    -fx-arc-height: 6px;
}
```

5.7.2 dark.css

```
.screens{
    -fx-background-color: black;
}
.title{
    -fx-fill: white;
}
.postAuthTitle{
    -fx-fill: white;
}
.screens .text{
    -fx-fill: white;
}
.backgroundRect{
    -fx-fill: white;
}
Rectangle{
    -fx-fill: black;
}
.menu-item.options > .label {
    -fx-text-fill: white;
}
.context-menu .text,
.context-menu .label {
    -fx-background-color: black;
```

```
    -fx-font-size: 12px;
}
.buttonText{
    -fx-fill: white;
}
.smallButtonText{
    -fx-fill: white;
}
.smallerButtonText{
    -fx-fill: white;
}
.tableText{
    -fx-fill: white;
}
.cardTable{
    -fx-fill: black;
}
```

5.7.3 highContrast.css

```
.screens{
    -fx-background-color: black;
}

.title{
    -fx-fill: yellow;
}

.postAuthTitle{
    -fx-fill: violet;
}

.screens .text{
    -fx-fill: yellow;
}

.backgroundRect{
    -fx-fill: yellow;
}

Rectangle{
    -fx-fill: black;
    -fx-stroke: white;
    -fx-stroke-width: 3;
}

.buttonText{
    -fx-fill: violet;
}
.tableText{
    -fx-fill: violet;
}
.smallButtonText{
    -fx-fill: violet;
}

.smallerButtonText{
    -fx-fill: violet;
}
.context-menu .text,
.context-menu .label {
    -fx-background-color: black;
    -fx-font-size: 12px;
}
.cardTable{
    -fx-fill: black;
}
```

5.7.4 light.css

```
.screens{
    -fx-background-color: grey;
}

.title{
    -fx-fill: white;
}
```

```
.postAuthTitle{
    -fx-fill: white;
}

.screens .text{
    -fx-fill: white;
}

.backgroundRect{
    -fx-fill: white;
}

Rectangle{
    -fx-fill: grey;
}

.buttonText{
    -fx-fill: white;
}

.smallButtonText{
    -fx-fill: white;
}
.tableText{
    -fx-fill: white;
}
.smallerButtonText{
    -fx-fill: white;
}
.context-menu .text,
.context-menu .label {
    -fx-background-color: black;
    -fx-font-size: 12px;
}
.cardTable{
    -fx-fill: grey;
}
.timerLabel{
    -fx-font-family: "Xirod-Regular";
    -fx-font-size: 36px;
    -fx-text-fill: white;
    -fx-background-color: grey;
    -fx-padding: 8 20 8 20;
    -fx-background-radius: 12px;
}
```

5.7.5 alert.css

```
.dialog-pane {
    -fx-background-color: #1e1e1e;
    -fx-padding: 20;
}

.dialog-pane .content {
    -fx-background-color: transparent;
}

.dialog-pane .label {
    -fx-font-family: "Xirod-Regular";
    -fx-text-fill: #e6e6e6;
    -fx-font-size: 14px;
}

.dialog-pane .button {
    -fx-background-color: #3c3f41;
    -fx-text-fill: white;
    -fx-font-size: 14px;
    -fx-padding: 8 18;
    -fx-background-radius: 6;
}

.dialog-pane .button:hover {
    -fx-background-color: #5a5d5f;
    -fx-cursor: hand;
}

.dialog-pane .button:default {
    -fx-background-color: #0078d7;
}

.dialog-pane .button:default:hover {
```

```
    -fx-background-color: #1492ff;
}

/* errors */
.dialog-pane.error {
    -fx-border-color: #b00020;
    -fx-border-width: 3;
}

/*  confirm/success */
.dialog-pane.confirmation {
    -fx-border-color: #1e6f3e;
    -fx-border-width: 3;
}

/* info */
.dialog-pane.information {
    -fx-border-color: #005a9e;
    -fx-border-width: 3;
}
```